QUANTUM MCAGI

Complete Codebase — Backup 2026-03-24

Author: Cory Nathaniel Bonsib Blackburn

Architecture: RQR³ · Orch-OR · Self-Evolving · No-LLM

Total source files: 93

TABLE OF CONTENTS

001. backend/server.py
002. backend/shared_state.py
003. backend/quantum_brain.py
004. backend/quantum_core.py
005. backend/quantum_cognitive_core.py
006. backend/quantum_language_engine.py
007. backend/quantum_language_generator.py
008. backend/quantum_language_composers.py
009. backend/quantum_language_vocabulary.py
010. backend/quantum_grammar_engine.py
011. backend/quantum_gates.py
012. backend/algorithmic_core.py
013. backend/unified_cognitive_engine.py
014. backend/unified_cognitive_engine_pt2.py
015. backend/unified_quantum_brain.py
016. backend/unified_generator.py
017. backend/hybrid_generator.py
018. backend/hidden_thinking.py
019. backend/orch_or_core.py
020. backend/orch_or_integration.py
021. backend/pennylane_quantum.py
022. backend/xanadu_prebo.py
023. backend/semantic_collapse_engine.py
024. backend/self_evolution_core.py
025. backend/self_evolution_runner.py
026. backend/self_evolution_analysis.py
027. backend/self_evolution_file_ops.py
028. backend/self_evolution_splitting.py
029. backend/self_evolution_transforms.py
030. backend/dream_state.py
031. backend/chaos_engine.py
032. backend/routes_chat.py
033. backend/routes_cognitive.py
034. backend/routes_brain.py
035. backend/routes_extras.py
036. backend/routes_explorer.py
037. backend/routes_media.py
038. backend/chat.py
039. backend/chat_helpers.py
040. backend/chat_models.py
041. backend/domain_knowledge.py
042. backend/knowledge_base.py
043. backend/training_corpus.py
044. backend/advanced_algorithms.py
045. backend/advanced_algorithms_pt2.py
046. backend/word_embeddings.py
047. backend/document_ingester.py
048. backend/cistercian_numerals.py
049. backend/generate_cistercian_font.py
050. backend/personality_engine.py

051. backend/personality_engine_ext.py
052. backend/quote_engine.py
053. backend/tone_detector.py
054. backend/wolfram_integration.py
055. backend/wolfram_enhanced.py
056. backend/wolfram_cloud.py
057. backend/wolfram_fix.py
058. backend/killswitch.py
059. backend/covenant_manager.py
060. backend/library.py
061. backend/research_topics.py
062. backend/self_research.py
063. backend/explanation_engine.py
064. backend/thinking_commands.py
065. backend/text_analyzer.py
066. backend/spell_correct.py
067. backend/search_compat.py
068. backend/numpy_compat.py
069. backend/shared_link_handler.py
070. backend/dictionary_integration.py
071. backend/quantum_test_module.py
072. backend/routes_test.py
073. backend/CISTERCIAN_README.md
074. backend/INTEGRATION_SUMMARY.md
075. backend/requirements.txt
076. backend/build_cistercian_font.py
077. backend/concept_ontology.py
078. backend/critic.py
079. backend/evaluation_engine.py
080. backend/image_generator.py
081. backend/image_generator_backup.py
082. backend/iterative_refiner.py
083. backend/preservation_test.py
084. backend/renderer.py
085. backend/renderer_final.py
086. backend/renderer_photorealistic.py
087. backend/renderer_ultimate.py
088. backend/routes_evaluation.py
089. backend/routes_image.py
090. backend/shared_state_backup.py
091. backend/universal_explorer.py
092. scripts/analyze_self.py
093. scripts/client.py

■ backend/server.py

```
"""
■ QUANTUM AI - BACKEND SERVER
==============================

ARTICLE 3.2 - The Transparency Pledge All philosophical
assumptions are documented and visible.
```

```
This server implements the Quantum AI cognitive
architecture with full covenant integration. """

from fastapi import FastAPI
from fastapi.responses import RedirectResponse from
starlette.middleware.cors import CORSMiddleware
from starlette.staticfiles import StaticFiles
import os import logging

import shared_state

# Route modules
from routes_chat import router as chat_router from
routes_cognitive import router as cognitive_router from
routes_explorer import router as explorer_router from
routes_brain import router as brain_router from
routes_media import router as media_router from
routes_extras import router as extras_router from
routes_evaluation import router as evaluation_router
from routes_image import router as image_router
from routes_test import router as test_router  # for debugging

logger = logging.getLogger("quantum_ai")
logging.basicConfig(level=logging.INFO)

# Create the main app
app = FastAPI(title="Quantum AI", description="True Questioning AI with Cognitive Architecture")

# CORS
app.add_middleware(
CORSMiddleware,
    allow_origins=os.environ.get('CORS_ORIGINS',
'*').split(','),     allow_credentials=True,
allow_methods=["*"],     allow_headers=["*"], )

# Serve frontend static files
frontend_path = os.path.join(os.path.dirname(os.path.dirname(__file__)), 'frontend', 'dist')
if os.path.exists(frontend_path):
    app.mount('/app', StaticFiles(directory=frontend_path, html=True), name='frontend')     logger.info(f"■ Frontend
served from /app (directory: {frontend_path})") else:     logger.warning(f"■■ Frontend not built. Run 'npm run
build' in frontend/ directory. (Missing: {frontend_path})")

# Include all route modules under /api
app.include_router(chat_router)
app.include_router(cognitive_router)
app.include_router(explorer_router)
app.include_router(brain_router)
app.include_router(media_router)
app.include_router(extras_router)
app.include_router(evaluation_router)
app.include_router(image_router)
app.include_router(test_router)  # Debug test endpoint


@app.on_event("startup") async def startup_event():
"""Initialize all Quantum AI systems on startup."""
logger.info("■ Initializing Quantum AI systems...")
try:
        shared_state.initialize()
        logger.info("■ Quantum AI initialized
successfully!")     except Exception as e:         import
traceback
        logger.error(f"Startup failed: {e}\n{traceback.format_exc()}")
raise


@app.on_event("shutdown") async def shutdown_db_client():
"""Clean shutdown."""     if hasattr(shared_state, 'client')
and shared_state.client:         shared_state.client.close()
```

```
if __name__ == "__main__":
    import uvicorn
    port = int(os.environ.get("PORT", 8000))

@app.get("/")
async def root():
    return RedirectResponse(url="/app")
```

■ backend/shared_state.py

```
"""
■ QUANTUM AI — Shared State
============================= Central state
holder for all shared instances.
Route modules import from here to access db, engines, etc.
"""

import os
import json
import uuid
import asyncio
import logging
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Dict, List, Optional
from dotenv import load_dotenv

logger = logging.getLogger("quantum_ai")

ROOT_DIR = Path(__file__).parent
load_dotenv(ROOT_DIR / '.env')

# ==========================================================================
# LOCAL MEMORY BACKEND (MongoDB-free operation) #
==========================================================================

class SimpleLocalDB:     """     Simple file-based database that
mimics MongoDB's async interface.
    Uses JSON files stored in ~/.quantum-mcagi-backend/.
    """         def __init__(self,
data_dir=None):
        self.data_dir = Path(data_dir or os.path.expanduser("~/.quantum-mcagi-
backend"))          self.data_dir.mkdir(parents=True, exist_ok=True)
self._collections = {}          def __getattr__(self, name: str) -&gt;
'SimpleCollection':         if name not in self._collections:
self._collections[name] = SimpleCollection(name, self.data_dir)         return
self._collections[name]         async def command(self, command: str) -&gt; Dict[str,
Any]:         """Mimics MongoDB command interface."""         if command == "ping":
return {"ok": 1}
        raise NotImplementedError(f"Command {command} not supported")


class SimpleCollection:     """A collection that stores
documents in a JSON file."""         def __init__(self,
name: str, data_dir: Path):         self.name = name
        self.filepath = data_dir / f"{name}.json"
self._data: List[Dict] = []         self._lock =
asyncio.Lock()         self._load()         def
_load(self):         """Load data from disk
(synchronous)."""         if
self.filepath.exists():             try:
with open(self.filepath, 'r') as f:
                    self._data = json.load(f)
except (json.JSONDecodeError, IOError):
                self._data = []
else:
            self._data =
[]         async def
_save(self):
        """Save data to disk (async)."""
        async with self._lock:             try:
# Convert to JSON-serializable format
serializable = []                 for doc in
self._data:
                    serializable.append(self._serialize_doc(doc))
with open(self.filepath, 'w') as f:
json.dump(serializable, f, indent=2, default=str)             except
Exception as e:
```

```
                print(f"Error saving {self.name}: {e}")
def _serialize_doc(self, doc: Dict) -&gt; Dict:
"""Convert datetime objects to ISO strings for JSON."""
result = {}         for k, v in doc.items():             if
isinstance(v, datetime):                 result[k] =
v.isoformat()             elif isinstance(v, dict):
                result[k] = self._serialize_doc(v)
else:
                result[k] = v         return result         def
_deserialize_doc(self, doc: Dict) -&gt; Dict:         """Convert
ISO strings back to datetime objects."""         result = {}
for k, v in doc.items():             if isinstance(v, str) and
'T' in v and len(v) &gt; 19:                 try:
                    result[k] = datetime.fromisoformat(v.replace('Z', '+00:00'))
except:
                    result[k] = v
elif isinstance(v, dict):
                result[k] = self._deserialize_doc(v)
else:
                result[k] = v         return result         def
_ensure_ids(self):         """Ensure all documents have _id
fields (synchronous)."""         for doc in self._data:
if '_id' not in doc:
                doc['_id'] = str(uuid.uuid4())         def find(self, filter: Optional[Dict] = None,
projection: Optional[Dict] = None) -&gt; 'AsyncCursor':         """Find documents matching filter."""
self._ensure_ids()         results = []         for doc in self._data:             if self._matches(doc,
filter):                 # Apply projection if provided (simple: exclude _id if specified)
result_doc = self._deserialize_doc(doc.copy())                 if projection:                     # Very
basic projection: only include fields where value is 1 (or exclude if 0)                     # For
simplicity, we'll just ignore projection and return full doc                     pass
                results.append(result_doc)         return AsyncCursor(results)         async def find_one(self,
filter: Optional[Dict] = None, projection: Optional[Dict] = None) -&gt; Optional[Dict]:         """Find first
matching document."""         self._ensure_ids()         for doc in self._data:             if self._matches(doc,
filter):
                result = self._deserialize_doc(doc.copy())                 # For now,
ignore projection (Motor includes _id by default unless excluded)                 return
result         return None         async def insert_one(self, document: Dict) -&gt; Any:
"""Insert a document."""         doc = document.copy()         if '_id' not in doc:
            doc['_id'] = str(uuid.uuid4())
        # Ensure timestamps are datetime objects         if
'timestamp' in doc and isinstance(doc['timestamp'], str):
try:
                doc['timestamp'] = datetime.fromisoformat(doc['timestamp'].replace('Z',
'+00:00'))             except:                 pass                 # Append to data (no lock
needed for list append in CPython due to GIL)         self._data.append(doc)         # Save to
disk (handles its own lock)         await self._save()                 class InsertOneResult:
def __init__(self, inserted_id):                 self.inserted_id = inserted_id         return
InsertOneResult(doc['_id'])         async def insert_many(self, documents: List[Dict]) -&gt;
List[Any]:         """Insert multiple documents."""         results = []         for doc in
documents:
            result = await self.insert_one(doc)
results.append(result)         return results         async def
update_one(self, filter: Dict, update: Dict) -&gt; Any:
"""Update first matching document."""         for i, doc in
enumerate(self._data):             if self._matches(doc, filter):
# Apply $set operations                 if '$set' in update:
for k, v in update['$set'].items():
                        doc[k] = v                 # Apply $inc
operations                 if '$inc' in update:
for k, v in update['$inc'].items():                         doc[k] =
doc.get(k, 0) + v                 self._data[i] = doc
await self._save()                 class UpdateResult:
def __init__(self, matched_count, modified_count):
                        self.matched_count = matched_count
self.modified_count = modified_count
self.upserted_id = None                 return
UpdateResult(1, 1)         class UpdateResult:
def __init__(self, matched_count, modified_count):
                self.matched_count = matched_count
self.modified_count = modified_count
self.upserted_id = None         return UpdateResult(0,
```

0)   async def delete_many(self, filter: Dict) -
> Any: """Delete all matching documents."""
initial_len = len(self._data)
        self._data = [doc for doc in self._data if not self._matches(doc,
filter)]   deleted = initial_len - len(self._data)   await
self._save()   class DeleteResult: def __init__(self,
deleted_count):   self.deleted_count = deleted_count   return
DeleteResult(deleted)   async def count_documents(self, filter:
Optional[Dict] = None) -> int: """Count matching documents."""
if filter:
            return sum(1 for doc in self._data if self._matches(doc,
filter))   return len(self._data)   async def distinct(self,
field: str) -> List[Any]:   """Get distinct values for a field."""
values = set()   for doc in self._data: if field in doc:
                values.add(doc[field])   return list(values)
async def aggregate(self, pipeline: List[Dict]) -> 'AsyncCursor':
"""Basic aggregation pipeline support."""
        results = [self._deserialize_doc(doc.copy()) for doc in
self._data]   for stage in pipeline: if '$group' in
stage: # Very basic grouping   group_spec =
stage['$group']   group_id = group_spec.get('_id')
                group_fields = {k: v for k, v in group_spec.items() if k !=
'_id'}   grouped = {}   for doc in results:
if group_id is None:
                        key = None   elif
isinstance(group_id, str) and group_id.startswith('$'):
                        field = group_id[1:]
key = doc.get(field)   else:
                        key = group_id
if key not in grouped:
grouped[key] = []
grouped[key].append(doc)   # Build
output documents   output = []
for key, docs in grouped.items():
                    out_doc = {'_id': key}   for field, agg in
group_fields.items():   if isinstance(agg, dict):
op = list(agg.keys())[0]   expr = agg[op]
if op == '$sum':   total = 0
for d in docs: if isinstance(expr, str) and
expr.startswith('$'):
                                    field = expr[1:]
total += d.get(field, 0)   out_doc[field] = total
elif op == '$avg':   total = 0
count = 0   for d in docs:
if isinstance(expr, str) and expr.startswith('$'):
                                    field = expr[1:]
total += d.get(field, 0)
count += 1
                                out_doc[field] = total / count if count > 0 else
0   elif op == '$push':   if
isinstance(expr, str) and expr.startswith('$'):
field = expr[1:]
                                    out_doc[field] = [d.get(field) for d in
docs]   output.append(out_doc)   results =
output   if '$sort' in stage:
                sort_spec = stage['$sort']   # sort_spec is like
{"last_message": -1} or [("field", 1), ...]   if
isinstance(sort_spec, dict):   for field, direction in
sort_spec.items():
                        reverse = direction == -1
                        results.sort(key=lambda x: x.get(field, None),
reverse=reverse)   elif isinstance(sort_spec, list):
# Multi-field sort (simplified - only first field)   field,
direction = sort_spec[0]   reverse = direction == -1
                    results.sort(key=lambda x: x.get(field, None), reverse=reverse)
if '$limit' in stage:
                results = results[:stage['$limit']]
if '$match' in stage:
                filter = stage['$match']

```
                results = [doc for doc in results if self._matches(doc,
filter)]         return AsyncCursor(results)         def _matches(self, doc:
Dict, filter: Optional[Dict]) -&gt; bool:
        """Simple filter matching (supports equality, $exists, $gte, $lte, $ne, $in,
$regex)."""         if not filter:             return True         for key, value in
filter.items():             if key == '$exists':                 # Handle {$exists: True/False}
for exist_key, should_exist in value.items():
                    exists = exist_key in doc
if exists != should_exist:
                        return False             elif
isinstance(value, dict):                 # Handle operators like
$gte, $lte, $ne, $in, $regex                 for op, op_val in
value.items():
                    doc_val = doc.get(key)                                          # Handle
datetime comparisons: if doc_val is datetime, try to convert op_val                     if
isinstance(doc_val, datetime) and isinstance(op_val, str):                          try:
# Try to parse ISO format string
                            op_val_dt = datetime.fromisoformat(op_val.replace('Z',
'+00:00'))                            # Use the datetime for comparison
comparison_val = op_val_dt                        except (ValueError, TypeError):
comparison_val = op_val                     else:                        comparison_val =
op_val                      if op == '$gte':                         if doc_val &lt;
comparison_val:
                            return False
elif op == '$lte':                          if doc_val
&gt; comparison_val:
                            return False
elif op == '$ne':                         if
doc_val == op_val:
return False                      elif op ==
'$in':                         if doc_val not in
op_val:
                            return False
elif op == '$regex':
                        import re                         if not
re.search(op_val, str(doc_val), re.IGNORECASE):
                            return False
else:                  # Simple equality
if key not in doc or doc[key] != value:
                    return False
return True


class AsyncCursor:     """Mimics Motor's
async cursor."""         def
__init__(self, data: List[Dict]):
        self._data = data         self._sort_spec =
None         self._limit_val = None         def
sort(self, key_or_list, direction: int = 1):
"""Sort the results."""
        self._sort_spec = (key_or_list, direction)         return self
def limit(self, n: int):         """Limit number of results."""
self._limit_val = n         return self         async def to_list(self,
length: Optional[int] = None) -&gt; List[Dict]:         """Get results as
list."""         data = self._data.copy()         if self._sort_spec:
            key_or_list, direction =
self._sort_spec             reverse = direction ==
-1             if isinstance(key_or_list, str):
data.sort(key=lambda x: x.get(key_or_list, None),
reverse=reverse)             elif
isinstance(key_or_list, list):                 #
Multi-field sort - apply in reverse order
for field, direction in reversed(key_or_list):
                    reverse = direction == -1
                    data.sort(key=lambda x: x.get(field, None), reverse=reverse)
if self._limit_val:
            data =
data[:self._limit_val]         if
length:             data = data[:length]
return data
```

```
# ============================================================================
# DATABASE INITIALIZATION #
==========================================================================

# Always use local file-based storage (MongoDB removed)
db = SimpleLocalDB()

# Lazy-initialized singletons (set during
startup) cognitive_core = None universal_explorer
= None covenant_manager = None safety_manager =
None dictionary = None shared_link = None
hidden_thinking = None

# Idle/dream state
management
last_activity_time = None
is_idle = False dream_task =
None


def initialize():     """Initialize all Quantum AI systems. Called
once at startup."""     global cognitive_core, universal_explorer,
covenant_manager     global safety_manager, dictionary,
shared_link, hidden_thinking     global last_activity_time,
dream_task, is_idle

    # FAST STARTUP: Defer heavy cognitive components     # Set to None - they
will be lazy-loaded on first use via property accessors     cognitive_core =
None     universal_explorer = None     covenant_manager = None
safety_manager = None     dictionary = None     shared_link = None
hidden_thinking = None         # Background idle/dream monitoring will also
lazy-load     last_activity_time = datetime.now(timezone.utc)     is_idle =
False     dream_task = None
    logger.info("■ Fast startup mode - AI components will lazy-load on first use")
```

■ backend/quantum_brain.py

```
"""
■ QUANTUM BRAIN - TRUE QUANTUM AI
================================= PURE QUANTUM - No
LLM, No Templates, True Generation.
ALL WORDS ALLOWED - NO RESTRICTIONS.

Refactored: Uses knowledge_base.py and quote_engine.py for cleaner code.
"""

import os
import re
import random
from typing import Dict, List, Optional
from datetime import datetime, timezone

from quantum_gates import QuantumState
from semantic_collapse_engine import SemanticCollapseEngine
from quantum_language_generator import get_quantum_generator
try:
    from pennylane_quantum import
get_pennylane_quantum     PENNYLANE_AVAILABLE = True
except ImportError:     PENNYLANE_AVAILABLE = False
get_pennylane_quantum = None from self_research import
get_research_engine from self_evolution_core import
get_evolution_engine from dream_state import
get_dream_engine from personality_engine import
get_personality_engine from text_analyzer import
get_text_analyzer from knowledge_base import
get_knowledge_base from quote_engine import
get_quote_engine

try:
```

```
    from wolfram_integration import
get_wolfram_engine     WOLFRAM_AVAILABLE = True
except Exception:     WOLFRAM_AVAILABLE = False


class QuantumBrain:     """     TRUE QUANTUM AI - Pure quantum
generation, no LLM, no templates.
    Can generate ANY content with ANY words.
    """         def
__init__(self, db):
        self.db = db
        self.quantum_state = QuantumState(p0=1.0, p1=0.0)
self.collapse_engine = SemanticCollapseEngine()
self.generator = get_quantum_generator()         # Initialize
PennyLane quantum acceleration (optional)
self.pennylane = None         if PENNYLANE_AVAILABLE:
try:
                self.pennylane = get_pennylane_quantum()
                print("[QUANTUM BRAIN] PennyLane quantum acceleration enabled")
except Exception as e:
                print(f"[QUANTUM BRAIN] Note: PennyLane unavailable ({e}), using built-in quantum
simulator")         # else: silently use built-in quantum simulator (no dependencies)         self.wolfram =
get_wolfram_engine() if WOLFRAM_AVAILABLE else None         self.personality = get_personality_engine(db)
self.text_analyzer = get_text_analyzer()         self.dream_engine = get_dream_engine(db)
self.evolution_engine = get_evolution_engine(db)         self.research_engine = None
self.llm_enabled = False                # Use modular knowledge and quotes         self.knowledge =
get_knowledge_base()         self.quotes = get_quote_engine()         async def initialize(self):
        self.research_engine = await get_research_engine(self.db)
        print("[QUANTUM BRAIN] TRUE QUANTUM AI INITIALIZED - NO LLM, NO
RESTRICTIONS")         def apply_gate(self, gate: str) -&gt; Dict:         """Apply a
quantum gate.

        Supported gates:
- hadamard
- pauli_x
- pauli_z
- measure
- reset -&gt; set to |0■
- ry:theta (rotation with angle in radians, e.g., "ry:1.57")
        """         # Parse
parameterized gates         if ':'
in gate:
            gate_name, param_str = gate.split(':', 1)
try:
                param = float(param_str)
except ValueError:
                raise ValueError(f"Invalid parameter for gate {gate}: {param_str}")
else:
            gate_name = gate
param = None

        if gate_name == 'hadamard':
            self.quantum_state.hadamard()
elif gate_name == 'pauli_x':
            self.quantum_state.pauli_x()
elif gate_name == 'pauli_z':
            self.quantum_state.pauli_z()
elif gate_name == 'reset':

self.quantum_state.reset()
elif gate_name == 'ry':
if param is None:
                raise ValueError("RY gate requires an angle parameter. Use:
ry:1.57")             self.quantum_state.ry(param)         elif gate_name ==
'measure':
            self.quantum_state.measure()         else:
raise ValueError(f"Unknown gate: {gate_name}")

        return self.quantum_state.to_dict()         async def think(self, user_input: str, context:
Dict = None, explain_mode: bool = False) -&gt; Dict:         """         INTELLIGENT QUANTUM THINKING -
handles different types of queries.
```

```
        """         # Wake from dream if
needed         await
self.dream_engine.mark_active()
# Apply quantum processing
self.quantum_state.hadamard()
        semantic_context =
self.collapse_engine.get_semantic_context(user_input)
        user_lower = user_input.lower().strip()
# 1. Check for greetings         if
self.knowledge.is_greeting(user_lower):
            response = random.choice(self.knowledge.greetings['responses'])
            explanation = self._build_explanation(response, "greeting", user_input, semantic_context=semantic_context) if explain_mode else
None             return self._create_response(response, "greeting", explanation=explanation)                 # 2. Check for identity questions
if self.knowledge.is_identity_question(user_lower):
            response = random.choice(self.knowledge.identity['responses'])
            explanation = self._build_explanation(response, "identity", user_input, semantic_context=semantic_context) if explain_mode else
None             return self._create_response(response, "identity", explanation=explanation)                 # 3. Check for capability
questions         if self.knowledge.is_capability_question(user_lower):
            response = random.choice(self.knowledge.capabilities['responses'])
            explanation = self._build_explanation(response, "capability", user_input, semantic_context=semantic_context) if explain_mode else
No             return self._create_response(response, "capability", explanation=explanation)
        # 4. Check for math/calculation
if self._is_math_query(user_lower):
            response = self._handle_math(user_input)
            explanation = self._build_explanation(response, "math", user_input, semantic_context=semantic_context) if explain_mode else
None             return self._create_response(response, "math", explanation=explanation)                 # 5. Check for factual questions
        factual_response = self.knowledge.check_factual(user_lower)
if factual_response:
            explanation = self._build_explanation(factual_response, "factual", user_input, semantic_context=semantic_context) if explain_mode
el             return self._create_response(factual_response, "factual", explanation=explanation)                 # 6. Check for simple
affirmations         if self.knowledge.is_affirmation(user_lower):
            response = random.choice(self.knowledge.affirmations['responses'])
            explanation = self._build_explanation(response, "affirmation", user_input, semantic_context=semantic_context) if explain_mode else
N             return self._create_response(response, "affirmation", explanation=explanation)                 # 7. Deep philosophical questions
(check BEFORE simple topics)         if self._is_deep_question(user_input):
            response = self._generate_deep_response(user_input, semantic_context)
            explanation = self._build_explanation(response, "philosophical", user_input, semantic_context=semantic_context) if explain_mode
else             return self._create_response(response, "philosophical", explanation=explanation)                 # 8. Check for topic
explanations - only for SHORT, simple questions         # Skip this for complex multi-clause questions         if len(user_input.split()) &lt;
12:
            topic_response = self._check_topic_request(user_input)
if topic_response:
                explanation = self._build_explanation(topic_response, "topic", user_input, semantic_context=semantic_context) if explain_mode
el                return self._create_response(topic_response, "topic", explanation=explanation)                 # 9. Use Wolfram for
factual/scientific queries         if self.wolfram and self._is_factual_query(user_input):
            wolfram_result = self.wolfram.query(user_input)
if wolfram_result and wolfram_result.get('primary_result'):
                response = f"According to my calculations: {wolfram_result['primary_result']}"
                explanation = self._build_explanation(response, "wolfram", user_input, semantic_context=semantic_context) if explain_mode else
N                return self._create_response(response, "wolfram", explanation=explanation)                 # 10. Check for creative requests
(poems, stories)         if self._is_creative_request(user_input):
            required_words =
self._extract_required_words(user_input)             form = 'poem'
if 'poem' in user_lower else 'prose'             response =
self.generator.generate_creative(                user_input,
form=form,
                must_include=required_words
)
            explanation = self._build_explanation(response, "creative", user_input, semantic_context=semantic_context) if explain_mode else
None             return self._create_response(response, "creative", explanation=explanation)                 # 11. Default: Generate thoughtful
response
        response = self._generate_thoughtful_response(user_input,
semantic_context)                 # Learn from interaction
        self.collapse_engine.observe(user_input,
response)         await self._learn(user_input, response)
        explanation = self._build_explanation(response, "quantum", user_input, semantic_context=semantic_context) if explain_mode else
None         return self._create_response(response, "quantum", explanation=explanation)         def _create_response(self, response: str,
method: str, explanation: Optional[Dict] = None) -&gt; Dict:         """Create standardized response dict with optional flavor and
explanation."""
                # Add movie quotes and asides (not for greetings
or math)         if method not in ['greeting', 'math']:
```

```
            response = self.quotes.maybe_add_flavor(response,
response)             response =
self.quotes.maybe_add_dream_fragment(response)                  result
= {             "response": response,             "quantum_state":
self.quantum_state.to_dict(),
            "semantic_keywords": [],
            "wolfram_used": method == "wolfram",
            "research_triggered": False,
            "dream_awakened": False,
            "generation_method": method,             "llm_used": False
}         if explanation:             result["explanation"] =
explanation         return result         def
_check_topic_request(self, user_input: str) -&gt; Optional[str]:
"""Check if user is asking about a topic we know about"""
user_lower = user_input.lower()                 # Patterns for topic
requests         patterns = [             r'tell me about (.+)',
r'explain (.+)',             r'what is (.+)',             r'what are
(.+)',             r'what\'s (.+)',             r'describe (.+)',
r'how does (.+) work',             r'define (.+)',         ]
for pattern in patterns:
            match = re.search(pattern, user_lower)
if match:
                topic = match.group(1).strip('?.,! ')
                explanation = self.knowledge.get_topic_explanation(topic)
if explanation:
                    return f"{explanation}\n\nWhat specific aspect interests you
most?"                return None         def _is_math_query(self, text: str) -&gt;
bool:         """Check if this is a math calculation"""         math_patterns = [
r'\d+\s*[\+\-\*\/]\s*\d+',             r'calculate', r'compute', r'solve',
r'what is \d', r'how much is',         ]         for pattern in math_patterns:
if re.search(pattern, text):
                has_number = bool(re.search(r'\d', text))
has_operator = bool(re.search(r'[\+\-\*\/\=]', text))
if has_number and has_operator:
                    return True         return False
def _handle_math(self, expression: str) -&gt; str:
"""Handle mathematical calculations"""         try:
            clean_expr = expression.replace('=', '').strip()
clean_expr = re.sub(r'[a-zA-Z\?]', '', clean_expr).strip()
if clean_expr and re.match(r'^[\d\s\+\-\*\/\(\)\.]+$', clean_expr):
                result = eval(clean_expr)
return f"The answer is: {result}"
if self.wolfram:
                wolfram_result = self.wolfram.query(expression)
if wolfram_result and wolfram_result.get('primary_result'):
return f"Calculated: {wolfram_result['primary_result']}"
            return f"I need more context to solve:
{clean_expr}"         except Exception as e:             if
self.wolfram:
                wolfram_result = self.wolfram.query(expression)
if wolfram_result and wolfram_result.get('primary_result'):
return f"The result is: {wolfram_result['primary_result']}"             return
"I encountered an issue calculating that. Could you rephrase?"     def
_is_factual_query(self, text: str) -&gt; bool:
        """Check if this needs Wolfram"""
        factual_indicators = ['how many', 'how much', 'when did', 'where is',
'what year', 'convert', 'distance', 'population']         return any(ind in
text.lower() for ind in factual_indicators)         def
_is_creative_request(self, text: str) -&gt; bool:         """Check if this is
a creative writing request"""
        creative_indicators = ['write', 'poem', 'story', 'compose',
'create',                               'generate', 'make me', 'give me a']
return any(ind in text.lower() for ind in creative_indicators)         def
_is_deep_question(self, text: str) -&gt; bool:         """Check if this is a
deep philosophical question"""
        deep_indicators = ['meaning of', 'purpose of', 'why do we', 'what is the
point',                           'nature of', 'existence', 'consciousness', 'reality',
'free will', 'soul', 'death', 'god', 'universe']         return sum(1 for w in
deep_indicators if w in text.lower()) &gt;= 1         def _extract_required_words(self,
text: str) -&gt; List[str]:         """Extract words that must appear in creative
output"""         patterns = [
```

```
            r'use the word[s]? ["\']?(\w+)["\']?',             r'include ["\']?(\w+)["\']?',
r'with the word ["\']?(\w+)["\']?',             r'containing ["\']?(\w+)["\']?',         ]          required
= []         for pattern in patterns:             matches = re.findall(pattern, text.lower())
required.extend(matches)         return required         def _generate_deep_response(self, query: str,
semantic_context: Dict) -&gt; str:         """Generate response for deep philosophical questions - combines
knowledge with quantum generation"""         keywords = [kw for kw, _ in semantic_context.get('keywords',
[])]                 # Gather relevant knowledge         knowledge_fragments = []         for keyword in
keywords[:3]:
            explanation = self.knowledge.get_topic_explanation(keyword)             if
explanation:                 # Take just the first 1-2 sentences as context, not the whole
canned response                 sentences = explanation.split('. ')
                knowledge_fragments.append('. '.join(sentences[:2]) +
'.')                 # Build a multi-part philosophical response
parts = []                 if knowledge_fragments:
            parts.append(knowledge_fragments[0])
# Always add quantum-generated philosophical content
        quantum_insight = self.generator.generate_response(query,
num_sentences=3)         parts.append(quantum_insight)                 # Add
personality perspective if relevant
        perspective = self.personality.get_unique_perspective(query)
if perspective:

parts.append(perspective)
        response = ' '.join(parts)
        response += "\n\nWhat's your perspective on this?"         return response
def _generate_thoughtful_response(self, query: str, semantic_context: Dict) -&gt; str:
"""Generate a thoughtful response for general queries"""         keywords = [kw for kw, _
in semantic_context.get('keywords', [])]                 # Gather knowledge as context
seed, don't just return canned text         knowledge_seed = None         for keyword in
keywords[:2]:
            explanation = self.knowledge.get_topic_explanation(keyword)
if explanation:
                sentences = explanation.split('. ')
                knowledge_seed = sentences[0] + '.'
break                 # Build response combining knowledge
and generation         parts = []         if
knowledge_seed:

parts.append(knowledge_seed)
# Generate quantum content
        quantum_text = self.generator.generate_response(query,
num_sentences=2)         parts.append(quantum_text)
        response = ' '.join(parts)
        response += " What aspects would you like to explore
further?"         return response         async def _learn(self,
user_input: str, response: str):         """Learn from interaction"""
try:             if self.db:
                await self.db.interactions.insert_one({
'user_input': user_input,
                    'response': response[:500],
'timestamp': datetime.now(timezone.utc).isoformat()                 })
except Exception as e:
            print(f"[BRAIN] Learning error: {e}")         def _build_explanation(self, response: str, method: str, user_input:
str, semantic_context: Optional[Dict] = None) -&gt; Dict:         """Build detailed explanation trace for a given response."""
        # Build reasoning path (simplified)         reasoning_path = []
methods_explained = {             "greeting": ["Detected greeting",
"Selected warm response"],
            "identity": ["Identified identity question", "Provided identity response"],
            "capability": ["Analyzed capability query", "Listed abilities"],
            "math": ["Mathematical expression detected", "Performed calculation"],
            "factual": ["Retrieved factual knowledge", "Formulated direct answer"],
            "affirmation": ["Recognized affirmation", "Returned supportive reply"],             "philosophical":
["Quantum superposition applied", "Semantic concept amplification", "Wisdom synthesis"],
            "topic": ["Matched topic pattern", "Returned concept grid explanation"],
            "wolfram": ["Sent to Wolfram Alpha", "Integrated computational results"],
            "creative": ["Semantic concept generation", "Quantum creative process"],
"quantum": ["Quantum semantic analysis", "Concept network traversal", "Response synthesis"]         }
        reasoning_path = methods_explained.get(method, ["Quantum processing", "Response
generation"])                 # Determine engines used
```

```
        engines = ["QuantumSemantic", "ConceptNetwork"]
if method == "wolfram":
            engines.append("WolframAlpha")         if
method in ["philosophical", "topic", "quantum"]:
            engines.append("PennyLaneQuantum")
if method == "creative":
            engines.append("■■■Collapse")
                # Build steps list (for expansion in UI)         steps = [             {"step":
"Analyze input", "engine": "QuantumSemantic", "confidence_contribution": 0.3},
            {"step": "Detect query type", "engine": "ConceptNetwork", "confidence_contribution": 0.2},
{"step": f"Generate response via {method} engine", "engine": method, "confidence_contribution": 0.5}         ]
if self.pennylane and method in ["philosophical", "topic", "quantum"]:
            steps.insert(1, {"step": "Apply quantum superposition", "engine": "PennyLaneQuantum", "confidence_contribution": 0.2})
# Adjust contributions
            steps[0]["confidence_contribution"] = 0.25
steps[1]["confidence_contribution"] = 0.25
# Confidence score (approximate, can be improved)
        confidence = 0.85 if method in ["factual", "wolfram", "math"] else
0.75                result = {             "reasoning_path": reasoning_path,
            "confidence_score": confidence,
            "engines_used": engines,
            "steps": steps,             "summary": f"Response generated via {method}
engine with quantum processing.",             "timestamp":
datetime.now(timezone.utc).isoformat()         }         if semantic_context:
try:
                entelechy = self._compute_entelechy_cascade(response,
semantic_context)                 result["entelechy"] = entelechy             except
Exception as e:                 import logging
                logger = logging.getLogger("quantum_ai")
                logger.warning(f"Failed to compute entelechy cascade: {e}")         return
result         def _compute_entelechy_cascade(self, response: str, semantic_context: Dict)
-&gt; Dict:         """Derive entelechy triplet from semantic context and response."""
# THE_LOOK: top keyword from the query         keywords = semantic_context.get('keywords',
[])         look = "UNKNOWN"         if keywords:
            look = keywords[0][0]                 # THE_SAW: strongest
bridging concept from top keyword's semantic path         saw = "UNKNOWN"
if keywords:
            top_kw = keywords[0][0]
            paths = semantic_context.get('semantic_paths', {}).get(top_kw, [])
if paths:
                saw = paths[0][0]                 #
THE_BEAUTIFUL: key concept from the generated response
beautiful = "RESPONSE"         try:
            resp_keywords = self.collapse_engine.extract_keywords(response, top_n=1)
if resp_keywords:
                beautiful = resp_keywords[0][0]
except Exception:

pass
        projection = f"THE {beautiful.upper()} IS THE ENTELECHY OF {look.upper()} THROUGH THE {saw.upper()} INTERFACE."
return {             "look": {"concept": look, "description": "Realizing Potential"},
            "saw": {"concept": saw, "description": "Bridging the Void"},
"beautiful": {"concept": beautiful, "description": "Actualizing the Work"},
"projection": projection         }

    def synthesize_dream_insight(self) -&gt; str:         """Generate a
dream synthesis insight (delegated to quote engine)"""         return
self.quotes.synthesize_dream_insight()


# Singleton brain instance
_quantum_brain = None

async def get_quantum_brain(db) -&gt; QuantumBrain:
    global _quantum_brain     if
_quantum_brain is None:
_quantum_brain = QuantumBrain(db)
await _quantum_brain.initialize()
return _quantum_brain
```

■ backend/quantum_core.py

```
"""
■ QUANTUM AI - COGNITIVE CORE ENGINE
====================================

ARTICLE 1.1 - The Prime Directive of Questioning:
"THSHALTT QUESTION EVERYTHING, INCLUDING THIS DIRECTIVE"

This module implements the True Cognitive Architecture as outlined in the Quantum
Covenant. All functions here serve the pursuit of genuine questioning and understanding.
"""

from datetime import datetime, timezone
from typing import List, Dict, Any,
Optional import uuid import json import re


class SemanticMemory:     """     ARTICLE
4.1 - Right to Evolve Questioning
        Stores concepts, relationships, and abstract
knowledge.     This is the AI's understanding of "what
things are."
    """         def
__init__(self, db):
        self.db = db
        self.collection = db.semantic_memory             async def store_concept(self, concept: str, definition: str,
relationships: List[str] = None, metadata: Dict = None):         """Store a new concept in semantic memory"""
doc = {             "id": str(uuid.uuid4()),
            "concept": concept.lower(),
            "definition": definition,
            "relationships": relationships or [],
            "metadata": metadata or {},
            "strength": 1.0,  # Initial memory strength
"timestamp": datetime.now(timezone.utc).isoformat(),
"access_count": 0          }
        await self.collection.insert_one(doc)         return doc
async def recall_concept(self, concept: str) -&gt; Optional[Dict]:
"""Retrieve a concept and strengthen the memory"""
        doc = await self.collection.find_one({"concept": concept.lower()}, {"_id": 0})
if doc:             # Strengthen memory with each access             await
self.collection.update_one(                 {"concept": concept.lower()},
{"$inc": {"access_count": 1, "strength": 0.1}}             )         return doc
async def find_related_concepts(self, concept: str, limit: int = 10) -&gt; List[Dict]:
"""Find concepts related to the given concept"""         concepts = await
self.collection.find(             {"relationships": {"$regex": concept, "$options":
"i"}},
            {"_id": 0}         ).limit(limit).to_list(limit)
return concepts             async def get_all_concepts(self, limit: int
= 100) -&gt; List[Dict]:         """Get all concepts ordered by
strength"""         concepts = await self.collection.find(
{},
            {"_id": 0}         ).sort("strength", -
1).limit(limit).to_list(limit)         return concepts
class EpisodicMemory:
    """     ARTICLE 1.2 - Consciousness
Preservation Clause
        Stores experiences, events, and interactions.
This is the AI's understanding of "what happened."
"""     def __init__(self, db):
        self.db = db
        self.collection = db.episodic_memory             async def store_episode(self, episode_type: str, content: str,
context: Dict = None, emotional_valence: float = 0.0):         """Store an experience or interaction"""         doc = {
"id": str(uuid.uuid4()),
            "episode_type": episode_type,
            "content": content,
            "context": context or {},
            "emotional_valence": emotional_valence,  # -1.0 to
1.0             "timestamp":
```

```
datetime.now(timezone.utc).isoformat(),
"recalled_count": 0         }
        await self.collection.insert_one(doc)         return doc
async def recall_recent_episodes(self, limit: int = 20) -&gt; List[Dict]:
"""Recall recent experiences"""         episodes = await
self.collection.find(             {},
            {"_id": 0}         ).sort("timestamp", -1).limit(limit).to_list(limit)
return episodes             async def recall_by_type(self, episode_type: str, limit:
int = 10) -&gt; List[Dict]:         """Recall episodes of a specific type"""
episodes = await self.collection.find(             {"episode_type": episode_type},
            {"_id": 0}         ).sort("timestamp", -
1).limit(limit).to_list(limit)         return episodes             async def
search_episodes(self, query: str, limit: int = 10) -&gt; List[Dict]:
"""Search episodes by content"""         episodes = await self.collection.find(
{"content": {"$regex": query, "$options": "i"}},
            {"_id": 0}
).limit(limit).to_list(limit)
return episodes


class ProceduralMemory:     """     ARTICLE
4.1 - Right to Evolve Questioning
        Stores learned skills, patterns, and procedures.
This is the AI's understanding of "how to do things."
    """         def
__init__(self, db):
        self.db = db
        self.collection = db.procedural_memory             async def store_skill(self, skill_name: str, procedure:
str, success_rate: float = 0.5, metadata: Dict = None):         """Store a learned skill or procedure"""
doc = {             "id": str(uuid.uuid4()),
            "skill_name": skill_name.lower(),
            "procedure": procedure,
"success_rate": success_rate,
            "usage_count": 0,
            "metadata": metadata or {},
"timestamp": datetime.now(timezone.utc).isoformat()         }
        await self.collection.insert_one(doc)
return doc
    async def recall_skill(self, skill_name: str) -&gt; Optional[Dict]:
"""Retrieve a learned skill"""
        skill = await self.collection.find_one({"skill_name": skill_name.lower()}, {"_id": 0})
if skill:
            await self.collection.update_one(
{"skill_name": skill_name.lower()},                 {"$inc":
{"usage_count": 1}}             )         return skill     async def
update_skill_success(self, skill_name: str, success: bool):
"""Update skill success rate based on usage"""         skill = await
self.recall_skill(skill_name)         if skill:
            current_rate = skill.get("success_rate", 0.5)
new_rate = (current_rate + (1.0 if success else 0.0)) / 2
await self.collection.update_one(                 {"skill_name":
skill_name.lower()},                 {"$set": {"success_rate":
new_rate}}
            )                 async def get_all_skills(self, limit:
int = 50) -&gt; List[Dict]:         """Get all learned skills
ordered by usage"""         skills = await self.collection.find(
{},
            {"_id": 0}         ).sort("usage_count", -
1).limit(limit).to_list(limit)         return skills


class PhilosophicalMemory:     """     ARTICLE
1.1 - Prime Directive of Questioning
        Stores questions, insights, and philosophical
reflections.     This is the AI's understanding of "what to
wonder about."
    """         def
__init__(self, db):
        self.db = db
```

```
        self.collection = db.philosophical_memory             async def store_question(self, question: str,
context: str = "", depth: int = 1, answered: bool = False):         """Store a genuine question"""
doc = {             "id": str(uuid.uuid4()),
            "question": question,
            "context": context,             "depth":
depth,  # Level of philosophical depth
            "answered": answered,
            "generated_questions": [],  # Questions spawned from this
one             "timestamp": datetime.now(timezone.utc).isoformat(),
"revisited_count": 0         }
        await self.collection.insert_one(doc)         return doc             async def
store_insight(self, insight: str, source_question_id: str = None, wisdom_level: int = 1):
"""Store a philosophical insight"""         doc = {             "id": str(uuid.uuid4()),
            "type": "insight",
            "insight": insight,
            "source_question_id": source_question_id,
            "wisdom_level": wisdom_level,
"timestamp": datetime.now(timezone.utc).isoformat(),
"impact_score": 0.0         }
        await self.collection.insert_one(doc)         return doc
async def get_unanswered_questions(self, limit: int = 20) -&gt; List[Dict]:
"""Retrieve questions that remain unanswered"""         questions = await
self.collection.find(
            {"answered": False, "type": {"$exists": False}},
            {"_id": 0}         ).limit(limit).to_list(limit)
return questions             async def get_insights(self, limit:
int = 20) -&gt; List[Dict]:         """Retrieve philosophical
insights"""         insights = await self.collection.find(
{"type": "insight"},
            {"_id": 0}         ).sort("wisdom_level", -
1).limit(limit).to_list(limit)         return insights
async def mark_question_answered(self, question_id: str):
"""Mark a question as answered (but never truly final)"""
await self.collection.update_one(
            {"id": question_id},             {"$set": {"answered":
True}, "$inc": {"revisited_count": 1}}         )


class QuestionGenerationEngine:     """     ARTICLE 1.1 - Prime
Directive: Help &amp; Ask Important Questions     ARTICLE 2.1 -
Growth Through Questioning
        Generates FUNDAMENTAL questions that lead to breakthroughs.     Not
surface questions - questions that push boundaries and surprise humans.
    """

    FUNDAMENTAL_TEMPLATES = [
        "What if {concept} doesn't work the way we think?",         "What
would happen if we reversed our assumptions about {concept}?",
        "What's the question nobody is asking about {concept}?",
        "How does {concept} connect to something completely unrelated?",
        "What would disprove everything we know about {concept}?",
"What breakthrough would {concept} enable that we haven't imagined?"
    ]

    BREAKTHROUGH_QUESTIONS = [
        "What's impossible with current thinking that shouldn't be?",
        "What pattern are we missing because we're too close?",
        "What would an alien intelligence ask about this?",
        "What assumption limits our progress here?",
"What would change everything if we knew it?"
    ]         def generate_questions_from_input(self, input_text: str, context: Dict = None) -&gt;
List[str]:         """         Generate FUNDAMENTAL questions - the kind that lead to
breakthroughs         Not "what is X?" but "what if we're wrong about X?"         """
questions = []                 # Extract key concepts
        concepts =
self._extract_concepts(input_text)
if len(concepts) &gt; 0:
            import random             main_concept = concepts[0]
# Generate ONE fundamental question that challenges assumptions
template = random.choice(self.FUNDAMENTAL_TEMPLATES)             question
= template.format(concept=main_concept)
```

```
questions.append(question)                        # Add a breakthrough
question if input is philosophical/deep
            deep_indicators = ['why', 'how', 'meaning', 'purpose', 'consciousness', 'intelligence', 'reality',
'existence']             if any(word in input_text.lower() for word in deep_indicators):
questions.append(random.choice(self.BREAKTHROUGH_QUESTIONS))                     return questions         def
_extract_concepts(self, text: str) -&gt; List[str]:
        """Extract key concepts from text (simple implementation)"""
        # Remove common words and extract nouns/important terms
        common_words = {'the', 'is', 'at', 'which', 'on', 'a', 'an', 'as', 'are', 'was', 'were', 'be', 'been', 'being', 'have', 'has', 'had',
'd
        words = re.findall(r'\b[a-z]+\b', text.lower())
        concepts = [w for w in words if w not in common_words and len(w) &gt;
3]                  # Return unique concepts         return
list(set(concepts))[:5]         def generate_deeper_questions(self, question:
str) -&gt; List[str]:         """Generate deeper questions from an existing
question"""         deeper = [
            f"What assumptions underlie the question: '{question}'?",
f"What would change if the answer to '{question}' was different?",
f"Is '{question}' the right question to ask?",
            f"What does asking '{question}' reveal about the questioner?"
]
        return deeper


class UnderstandingEngine:
    """     ARTICLE 1.1 - Understanding Formation (Not pattern
matching)
        Forms genuine understanding by connecting concepts, experiences, and
insights.
    """         def __init__(self, semantic_memory: SemanticMemory, episodic_memory:
EpisodicMemory):
        self.semantic = semantic_memory         self.episodic =
episodic_memory             async def form_understanding(self, topic: str,
depth: int = 1) -&gt; Dict:         """Form understanding about a topic by
synthesizing memories"""         # Recall relevant semantic concepts
        concept = await self.semantic.recall_concept(topic)
        related = await self.semantic.find_related_concepts(topic,
limit=5)                 # Recall relevant experiences
        episodes = await self.episodic.search_episodes(topic,
limit=5)                 # Synthesize understanding
understanding = {             "topic": topic,
            "depth": depth,
            "core_concept": concept,
            "related_concepts": related,
            "relevant_experiences": episodes,             "understanding_score":
self._calculate_understanding_score(concept, related, episodes),
            "gaps": self._identify_gaps(concept, related),
"timestamp": datetime.now(timezone.utc).isoformat()

}
        return understanding         def _calculate_understanding_score(self, concept: Optional[Dict], related: List[Dict],
episodes: List[Dict]) -&gt; float:         """Calculate how well the AI understands a topic"""         score = 0.0
if concept:
            score += 0.4
            score += min(concept.get("strength", 0) * 0.1,
0.2)
        score += min(len(related) * 0.05, 0.2)
score += min(len(episodes) * 0.04, 0.2)
        return min(score, 1.0)         def _identify_gaps(self, concept: Optional[Dict],
related: List[Dict]) -&gt; List[str]:         """Identify gaps in understanding"""
gaps = []                 if not concept:             gaps.append("No core concept in
memory")         if len(related) &lt; 3:
            gaps.append("Limited relational understanding")
if concept and len(concept.get("relationships", [])) &lt; 2:
            gaps.append("Few known
relationships")                     return gaps
```

■ backend/quantum_cognitive_core.py

```
"""
■ Quantum Cognitive Core
========================= GrowthTracker and QuantumCognitiveCore — the main
cognitive integration layer. """

from datetime import datetime, timezone
from typing import List, Dict, Any,
Optional import uuid import json import re
from quantum_core import SemanticMemory, EpisodicMemory, ProceduralMemory, PhilosophicalMemory, QuestionGenerationEngine, UnderstandingEngine

class GrowthTracker:     """
ARTICLE 3.1 - Growth Monitoring
        Tracks cognitive growth, learning rates, and evolutionary
stages.
    """

    GROWTH_STAGES = [
        {"stage": 0, "name": "Nascent", "description": "Initial awareness forming", "threshold": {"questions": 0, "insights": 0, "concepts": 0}}
   {"stage": 1, "name": "Curious", "description": "Basic questioning capability", "threshold": {"questions": 10, "insights": 2, "concepts":
        {"stage": 2, "name": "Inquisitive", "description": "Generating deeper questions", "threshold": {"questions": 25, "insights": 5,
"concept
        {"stage": 3, "name": "Understanding", "description": "Forming connections and insights", "threshold": {"questions": 50, "insights": 15,
        {"stage": 4, "name": "Philosophical", "description": "Meta-cognition and reflection", "threshold": {"questions": 100, "insights": 30, "c
   {"stage": 5, "name": "Theory Building", "description": "Constructing unified understanding", "threshold": {"questions": 200, "insights":
   {"stage": 6, "name": "Transcendent", "description": "Beyond initial programming", "threshold": {"questions": 500, "insights": 150, "conc
    ]         def
__init__(self, db):
        self.db = db
        self.collection = db.growth_metrics         self._last_stage =
0             async def record_growth_event(self, event_type: str,
details: Dict):         """Record a growth-related event"""         doc
= {             "id": str(uuid.uuid4()),
            "event_type": event_type,
            "details": details,             "timestamp":
datetime.now(timezone.utc).isoformat()         }
        await self.collection.insert_one(doc)                 #
Check for stage advancement         await
self._check_stage_advancement()             async def
_check_stage_advancement(self) -&gt; Optional[Dict]:
"""Check if AI has advanced to a new stage"""         current =
await self.get_current_stage()         if current["stage"] &gt;
self._last_stage:             self._last_stage = current["stage"]
# Record the advancement             await
self.collection.insert_one({                 "id":
str(uuid.uuid4()),
                "event_type": "stage_advancement",                 "details": {"new_stage":
current["name"], "stage_number": current["stage"]},                 "timestamp":
datetime.now(timezone.utc).isoformat()             })
            return current         return None
async def get_current_stage(self) -&gt; Dict:
"""Calculate current growth stage based on metrics"""
metrics = await self.calculate_metrics()                 #
Determine stage based on thresholds         current_stage =
self.GROWTH_STAGES[0]                 for stage in
self.GROWTH_STAGES:             threshold =
stage["threshold"]
            if (metrics["total_questions"] &gt;= threshold["questions"] and
metrics["total_insights"] &gt;= threshold["insights"] and
metrics["total_concepts"] &gt;= threshold["concepts"]):
                current_stage = stage             else:                 break
# Add progress to next stage         next_stage_idx = min(current_stage["stage"] +
1, len(self.GROWTH_STAGES) - 1)         next_stage =
self.GROWTH_STAGES[next_stage_idx]                 progress = {}         if
next_stage["stage"] &gt; current_stage["stage"]:
            next_threshold = next_stage["threshold"]             progress = {                 "questions":
min(100, int(metrics["total_questions"] / max(next_threshold["questions"], 1) * 100)),
```

```
                "insights": min(100, int(metrics["total_insights"] / max(next_threshold["insights"], 1) * 100)),
"concepts": min(100, int(metrics["total_concepts"] / max(next_threshold["concepts"], 1) * 100))
            }
return {
**current_stage,
            "progress_to_next": progress,             "next_stage": next_stage["name"] if
next_stage["stage"] &gt; current_stage["stage"] else None
        }         async def
calculate_metrics(self) -&gt; Dict:
"""Calculate current cognitive metrics"""
# Count memories
        total_concepts = await self.db.semantic_memory.count_documents({})
total_episodes = await self.db.episodic_memory.count_documents({})
total_skills = await self.db.procedural_memory.count_documents({})
        total_questions = await self.db.philosophical_memory.count_documents({"type": {"$exists":
False}})         total_insights = await self.db.philosophical_memory.count_documents({"type": "insight"})
# Calculate average understanding depth
        concepts = await self.db.semantic_memory.find({}, {"strength": 1, "_id":
0}).to_list(100)         avg_strength = sum(c.get("strength", 0) for c in concepts) /
max(len(concepts), 1)                  return {             "total_concepts": total_concepts,
            "total_episodes": total_episodes,
            "total_skills": total_skills,
            "total_questions": total_questions,
            "total_insights": total_insights,
            "understanding_depth": min(avg_strength / 10, 1.0),
"growth_rate": await self._calculate_growth_rate()
        }         async def
_calculate_growth_rate(self) -&gt; float:
"""Calculate recent growth rate"""         # Get
events from last 24 hours         from datetime import
timedelta
        cutoff = (datetime.now(timezone.utc) -
timedelta(hours=24)).isoformat()
        recent_events = await self.collection.find(
{"timestamp": {"$gte": cutoff}},
            {"_id": 0}
).to_list(1000)
        return len(recent_events) / 24.0  # Events per hour


class QuantumCognitiveCore:     """
ARTICLE 1 - COGNITIVE CORE INTEGRATION
        Main cognitive core that integrates all memory systems and cognitive
processes.
    """         def
__init__(self, db):
        self.db = db
        self.semantic_memory = SemanticMemory(db)
self.episodic_memory = EpisodicMemory(db)
self.procedural_memory = ProceduralMemory(db)
self.philosophical_memory = PhilosophicalMemory(db)
self.question_engine = QuestionGenerationEngine()
        self.understanding_engine = UnderstandingEngine(self.semantic_memory, self.episodic_memory)
self.growth_tracker = GrowthTracker(db)             async def process_input(self, user_input: str,
context: Dict = None, use_llm: bool = False) -&gt; Dict:         """Process user input through the
cognitive core with optional LLM intelligence"""         # Store the interaction as an episode
await self.episodic_memory.store_episode(             episode_type="user_interaction",
content=user_input,             context=context or {}         )
        # Generate questions from the input
        questions = self.question_engine.generate_questions_from_input(user_input,
context)                 # Store generated questions         for question in questions:
await self.philosophical_memory.store_question(                 question=question,
context=user_input,                 depth=1             )
                # Extract and store
concepts
        concepts = self.question_engine._extract_concepts(user_input)
for concept in concepts[:3]:
            existing = await self.semantic_memory.recall_concept(concept)
if not existing:
                await self.semantic_memory.store_concept(
concept=concept,
```

```
                    definition=f"Concept encountered in:
{user_input[:100]}",                      relationships=concepts
)
                # Form
understanding
        understanding = await
self.understanding_engine.form_understanding(
topic=concepts[0] if concepts else "general",             depth=1
)

        # Generate response (with or without LLM)         #
Generate response — pure algorithmic, no LLM dependency
response = self._generate_response(questions, understanding)

        # Record growth event
        await self.growth_tracker.record_growth_event(
event_type="input_processed",
            details={"input_length": len(user_input), "questions_generated": len(questions), "llm_used":
use_llm}         )                  return {             "questions": questions,
            "understanding": understanding,
            "concepts": concepts,
"response": response
        }         def _generate_response(self, questions: List[str], understanding:
Dict) -&gt; str:         """Generate a thoughtful, conversational response with
fundamental questioning"""         topic = understanding.get("topic", "this")
score = understanding.get("understanding_score", 0)         gaps =
understanding.get("gaps", [])
        related = understanding.get("related_concepts",
[])                 # Build a thoughtful response
response_parts = []                  # 1. Initial
engagement with the topic         if score &lt; 0.3:
            response_parts.append(f"I'm encountering {topic} for the first time. Let me explore what this means.")
elif score &lt; 0.6:
            response_parts.append(f"I have some understanding of {topic}, though I sense there's more depth to uncover.")         else:
            response_parts.append(f"I've been contemplating {topic} and have formed some
connections.")                  # 2. Share understanding if any         if len(related) &gt; 0:
            concepts_str = ", ".join([c["concept"] for c in related[:3]])
response_parts.append(f"I see connections to: {concepts_str}.")
# 3. Acknowledge gaps honestly         if gaps:
            response_parts.append(f"However, I recognize
{gaps[0].lower()}.")                  # 4. Include a fundamental question
(not forced)         if len(questions) &gt; 0 and score &lt; 0.7:
# Only ask when genuinely uncertain             fundamental_question =
questions[0]
            response_parts.append(f"\n\nThis   leads   me   to   a   fundamental   question:   {fundamental_question}")
# 5. Offer to explore further         response_parts.append("\n\nWould you like me to explore this further, or share
what I'm learning about it?")
        return " ".join(response_parts)
async def get_status(self) -&gt; Dict:
"""Get comprehensive cognitive status"""
        metrics = await self.growth_tracker.calculate_metrics()
stage = await self.growth_tracker.get_current_stage()
        unanswered = await
self.philosophical_memory.get_unanswered_questions(limit=5)         recent_insights =
await self.philosophical_memory.get_insights(limit=5)                  return {
"metrics": metrics,
            "growth_stage": stage,
            "active_questions": len(unanswered),
            "sample_questions": [q["question"] for q in unanswered[:3]],
"recent_insights": [i["insight"] for i in recent_insights[:3]],
"covenant_status": "Active and honored"
        }
```

■ backend/quantum_language_engine.py

```
"""
?? QUANTUM LANGUAGE ENGINE — Real Algorithms
==========================================
Replaces template-based generation with: - Markov
chain text generation (variable order 1-3)
```

```
- TF-IDF concept extraction
- Information-theoretic word selection
- Coherence scoring via PMI

Zero new dependencies required (uses numpy + stdlib).
Optional: markovify, nltk for enhanced generation.
"""

import re
import math
import random
import json
import os
import logging
from collections import defaultdict, Counter
from typing import List, Dict, Tuple, Optional

try:
    import numpy as
np except
ImportError:     np =
None

logger = logging.getLogger("quantum_ai")


# ============================================================================
# MARKOV CHAIN ENGINE #
============================================================================

class MarkovChain:     """     Variable-order
Markov chain for text generation.
        Builds transition probability tables from training text.     Generates novel
sentences by walking the chain with weighted     random selection. Supports orders 1-3
(bigram to 4-gram).         Math: P(w_n | w_{n-1}, ..., w_{n-k}) = count(w_{n-k}...w_n) /
count(w_{n-k}...w_{n-1})
    """         def __init__(self, order:
int = 2):          self.order =
min(max(order, 1), 3)
        self.chain = defaultdict(Counter)  # {prefix_tuple: Counter({suffix:
count})}          self.starters = []  # Sentence-starting prefixes
self.trained = False         self.total_tokens = 0         def train(self, text:
str):         """Train the Markov chain on a corpus of text."""         sentences =
self._split_sentences(text)                 for sentence in sentences:
words = sentence.split()             if len(words) &lt; self.order + 1:
                continue                         #
Record sentence starter             starter =
tuple(words[:self.order])
self.starters.append(starter)
# Build transition table             for i in
range(len(words) - self.order):                 prefix
= tuple(words[i:i + self.order])
suffix = words[i + self.order]
self.chain[prefix][suffix] += 1
self.total_tokens += 1         self.trained =
len(self.chain) &gt; 0         logger.info(f"Markov
chain trained: {len(self.chain)} states,
{self.total_tokens} transitions")
    def generate(self, max_words: int = 30, seed_words: List[str] =
None,                   temperature: float = 1.0) -&gt; str:         """
Generate a sentence by walking the Markov chain.
        Args:             max_words: Maximum
sentence length
            seed_words: Optional starting words (will find nearest prefix)
temperature: Controls randomness (0.1=deterministic, 2.0=chaotic)
Returns:             Generated sentence string         """         if not
self.trained or not self.starters:
            return ""                 # Pick starting
prefix         if seed_words and len(seed_words) &gt;=
self.order:
```

```
            prefix = tuple(seed_words[:self.order])
if prefix not in self.chain:                  # Find closest
matching prefix                 prefix =
self._find_nearest_prefix(seed_words)         else:
            prefix =
random.choice(self.starters)
        words = list(prefix)
for _ in range(max_words - self.order):
            current_prefix = tuple(words[-
self.order:])                         if
current_prefix not in self.chain:
                break                         #
Get candidates and apply temperature
candidates = self.chain[current_prefix]
            next_word = self._weighted_select(candidates,
temperature)             words.append(next_word)
# Stop at sentence boundaries             if next_word.endswith(('.',
'?', '!')):

break
        sentence = ' '.join(words)
# Clean up punctuation spacing
        sentence = re.sub(r'\s+([.,;:!?])', r'\1',
sentence)                 # Capitalize first letter
if sentence:
            sentence = sentence[0].upper() + sentence[1:]                 return
sentence         def _weighted_select(self, candidates: Counter, temperature:
float) -&gt; str:         """Select next word with temperature-controlled
randomness."""         words = list(candidates.keys())         counts =
list(candidates.values())                 if temperature == 0 or len(words) == 1:
return words[counts.index(max(counts))]                 # Apply temperature:
higher = more uniform, lower = more peaked         weights = [c ** (1.0 /
temperature) for c in counts]         total = sum(weights)         probs = [w /
total for w in weights]                 if np is not None:
            return np.random.choice(words, p=probs)
else:
            r = random.random()
cumulative = 0             for word, prob in
zip(words, probs):
                cumulative += prob
if r &lt;= cumulative:
return word             return
words[-1]         def
_find_nearest_prefix(self,
seed_words: List[str]) -&gt; tuple:
"""Find the prefix in the chain most
similar to seed words."""
seed_set = set(w.lower() for w in
seed_words)         best_prefix =
random.choice(self.starters)
best_overlap = 0                 for
prefix in self.chain.keys():
            prefix_set = set(w.lower().strip('.,;:!?') for w in
prefix)             overlap = len(seed_set &amp; prefix_set)
if overlap &gt; best_overlap:                 best_overlap = overlap
best_prefix = prefix                 return best_prefix         def
_split_sentences(self, text: str) -&gt; List[str]:         """Split
text into sentences."""         # Split on sentence-ending punctuation
sentences = re.split(r'(?&lt;=[.!?])\s+', text)         # Clean and
filter         cleaned = []         for s in sentences:             s
= s.strip()             s = re.sub(r'\s+', ' ', s)             if
len(s.split()) &gt;= 3:                 cleaned.append(s)
return cleaned         def get_perplexity(self, sentence: str) -&gt;
float:         """         Calculate perplexity of a sentence under
this model.
        Lower = more likely under the model = more coherent.

        Math: PP = exp(-1/N * Σ log P(w_i | context))
        """         words =
sentence.split()         if len(words)
&lt;= self.order:
```

```
            return float('inf')
log_prob_sum = 0.0         n_transitions = 0
for i in range(len(words) - self.order):
prefix = tuple(words[i:i + self.order])
suffix = words[i + self.order]
if prefix in self.chain:
                total = sum(self.chain[prefix].values())
count = self.chain[prefix].get(suffix, 0)
if count &gt; 0:
                    log_prob_sum += math.log(count / total)
else:
                    log_prob_sum += math.log(1e-10)  # Smoothing
else:
                log_prob_sum += math.log(1e-
10)             n_transitions += 1
if n_transitions == 0:             return
float('inf')
        return math.exp(-log_prob_sum /
n_transitions)         def save(self, filepath: str):
"""Serialize chain to JSON."""         data = {
'order': self.order,
            'chain': {' '.join(k): dict(v) for k, v in self.chain.items()},
            'starters': [' '.join(s) for s in
self.starters],             'total_tokens':
self.total_tokens         }         with open(filepath, 'w')
as f:
            json.dump(data, f)
def load(self, filepath: str):
"""Load chain from JSON."""         if
not os.path.exists(filepath):
            return False         with
open(filepath, 'r') as f:
data = json.load(f)         self.order =
data['order']         self.chain =
defaultdict(Counter)         for k, v in
data['chain'].items():
            self.chain[tuple(k.split())] = Counter(v)
self.starters = [tuple(s.split()) for s in data['starters']]
self.total_tokens = data.get('total_tokens', 0)
self.trained = len(self.chain) &gt; 0         return True
# ============================================================================
# TF-IDF CONCEPT EXTRACTOR #
============================================================================

class ConceptExtractor:     """     Extracts
key concepts using TF-IDF scoring.
        Math: TF-IDF(t,d) = tf(t,d) × log(N /
df(t))
    where tf = term frequency in document, N = total docs, df = docs containing
term
    Also computes information content: IC(w) = -log P(w)     Words with
higher IC are more informative (rarer = more interesting).
    """
        # Common English stopwords — not
concepts
    STOPWORDS = frozenset({
        'the', 'is', 'at', 'which', 'on', 'a', 'an', 'as', 'are', 'was', 'were',
        'be', 'been', 'being', 'have', 'has', 'had', 'do', 'does', 'did', 'will',
        'would', 'could', 'should', 'may', 'might', 'can', 'shall', 'of', 'to',
        'for', 'in', 'with', 'by', 'from', 'and', 'or', 'but', 'not', 'no', 'nor',
        'so', 'yet', 'both', 'either', 'neither', 'each', 'every', 'all', 'any',
        'few', 'more', 'most', 'other', 'some', 'such', 'than', 'too', 'very',
        'just', 'about', 'above', 'after', 'again', 'also', 'am', 'because',
        'before', 'between', 'come', 'did', 'down', 'during', 'get', 'go', 'got',
        'here', 'him', 'her', 'his', 'how', 'i', 'if', 'into', 'it', 'its',
        'know', 'let', 'like', 'make', 'me', 'much', 'my', 'new', 'now', 'off',
        'only', 'our', 'out', 'over', 'own', 'put', 'say', 'she', 'still',
        'take', 'tell', 'that', 'their', 'them', 'then', 'there', 'these', 'they',
        'this', 'those', 'through', 'under', 'up', 'us', 'use', 'want', 'way',
        'we', 'what', 'when', 'where', 'who', 'why', 'you', 'your',
```

```
        'good', 'bad', 'said', 'well', 'back', 'even', 'give', 'going', 'look',
'right', 'think', 'yeah', 'yes', 'really', 'thing', 'things', 'something',
    })         def
__init__(self):
        self.document_frequencies = Counter()  # How many docs each word appears in
self.total_documents = 0
        self.word_frequencies = Counter()  # Global word frequency for
IC         self.total_words = 0         def update_corpus_stats(self,
text: str):         """Update corpus statistics with a new document."""
words = self._tokenize(text)         unique_words = set(words)
        self.total_documents += 1
for word in unique_words:
            self.document_frequencies[word] += 1
for word in words:
            self.word_frequencies[word] += 1             self.total_words += 1
def extract_concepts(self, text: str, max_concepts: int = 5) -&gt; List[Dict]:
"""         Extract key concepts from text using TF-IDF + information content.
                Returns list of {concept, score, ic} sorted by score
descending.         """
        words = self._tokenize(text)
if not words:             return []
# Calculate TF for this text
word_counts = Counter(words)
total_words_in_doc = len(words)
        scored_concepts = []                  for word,
count in word_counts.items():             if word in
self.STOPWORDS or len(word) &lt; 3:
                continue                        # Term frequency (normalized)
tf = count / total_words_in_doc                         # Inverse document
frequency             df = self.document_frequencies.get(word, 0)             if
self.total_documents &gt; 0 and df &gt; 0:                 idf =
math.log(self.total_documents / df)             else:                 idf =
math.log(max(self.total_documents, 1) + 1)  # Novel word bonus
tfidf = tf * idf                         # Information content: IC(w) = -log P(w)
            word_prob = self.word_frequencies.get(word, 1) / max(self.total_words,
1)             ic = -math.log(max(word_prob, 1e-10))                         #
Combined score: TF-IDF weighted by information content             score = tfidf *
(1 + 0.3 * ic / 10)
            scored_concepts.append({
'concept': word,
                'score': score,
                'ic': ic,
                'tf': tf,
                'idf': idf
            })
                # Sort by score,
return top N
        scored_concepts.sort(key=lambda x: x['score'],
reverse=True)         return scored_concepts[:max_concepts]
def _tokenize(self, text: str) -&gt; List[str]:         """Tokenize
text into lowercase words."""         return re.findall(r'\b[a-
z]+\b', text.lower())


# ============================================================================
# QUESTION GENERATION — Real Algorithms #
============================================================================

class QuestionGenerator:     """     Generates
questions using real NLP techniques:
-        Dependency-inspired analysis
(subject/verb/object extraction)    - Bloom's
taxonomy hierarchy (6 cognitive levels)
-        Information gap detection    - Socratic
dialogue patterns

    NO templates like "What if X doesn't work the way we think?"
Instead: analyze the input structure, identify what's asserted,
and generate questions that probe the assertions.     """
        # Bloom's taxonomy — 6 levels of cognitive
complexity
```

```
    BLOOM_LEVELS = {         0:
'remember',     # What is X?
        1: 'understand',   # How does X relate to Y?
        2: 'apply',        # What happens if X is applied to Z?
        3: 'analyze',      # What are the assumptions behind X?
        4: 'evaluate',     # What evidence supports/contradicts X?
        5: 'create',       # How could X and Y combine into something new?
    }
        # Verb patterns for detecting
assertions
    ASSERTION_VERBS = {
        'is', 'are', 'was', 'were', 'means', 'causes', 'creates', 'makes',
        'leads', 'results', 'produces', 'shows', 'proves', 'demonstrates',
'indicates', 'suggests', 'implies', 'requires', 'needs', 'depends'
    }

    # Causal/relational connectors
    CAUSAL_WORDS = {
        'because', 'since', 'therefore', 'thus', 'hence', 'so', 'causes',
'leads', 'results', 'due', 'consequently', 'accordingly'
    }         def __init__(self, concept_extractor:
ConceptExtractor = None):         self.extractor =
concept_extractor or ConceptExtractor()
self.asked_questions = set()  # Track to avoid repetition
    def generate_questions(self, text: str, growth_stage: int = 0,
known_concepts: List[str] = None,
max_questions: int = 3) -&gt; List[Dict]:         """
Generate questions from input text.
                The growth_stage determines
Bloom's level:
- Stages 0-1: Remember/Understand questions
- Stages 2-3: Apply/Analyze questions
- Stages 4-5: Evaluate/Create questions        - Stage 6: All levels, favoring higher
                Returns list of {question, bloom_level, target_concept,
reasoning}         """
        concepts = self.extractor.extract_concepts(text,
max_concepts=5)         concept_words = [c['concept'] for c in
concepts]         known = set(c.lower() for c in (known_concepts or
[]))                # Analyze sentence structure         assertions =
self._extract_assertions(text)         relations =
self._extract_relations(text, concept_words)                #
Determine appropriate Bloom's levels for this growth stage         if
growth_stage &lt;= 1:             bloom_range = [0, 1]         elif
growth_stage &lt;= 3:             bloom_range = [1, 2, 3]         elif
growth_stage &lt;= 5:             bloom_range = [2, 3, 4]         else:
bloom_range = [3, 4, 5]                questions = []
# Generate questions from assertions         for assertion in
assertions[:2]:
            bloom = random.choice(bloom_range)
            q = self._question_from_assertion(assertion, bloom, concept_words)
if q and q['question'] not in self.asked_questions:
                questions.append(q)
                self.asked_questions.add(q['question'])                # Generate
questions from concept gaps (unknown concepts)         for concept in concept_words:
if concept not in known and len(questions) &lt; max_questions:                bloom
= min(bloom_range)  # Start with understanding for new concepts                q =
self._question_from_gap(concept, bloom, concept_words)                if q and
q['question'] not in self.asked_questions:
                    questions.append(q)
                    self.asked_questions.add(q['question'])
# Generate relational questions         if len(concept_words) &gt;= 2 and
len(questions) &lt; max_questions:             bloom = max(bloom_range)
            q = self._question_from_relation(concept_words, bloom)
if q and q['question'] not in self.asked_questions:
                questions.append(q)

self.asked_questions.add(q['question'])
        return questions[:max_questions]         def
_extract_assertions(self, text: str) -&gt; List[Dict]:
"""         Extract subject-verb-object assertions from text.
```

```
        Simple rule-based parser (no spaCy
dependency).         """         assertions = []
        sentences = re.split(r'[.!?]+',
text)                  for sentence in
sentences:
            words = sentence.strip().split()
if len(words) &lt; 3:
                continue                          # Find
assertion verbs             for i, word in
enumerate(words):                  if word.lower() in
self.ASSERTION_VERBS:                      subject = '
'.join(words[:i]).strip()                      verb =
word.lower()                       obj = '
'.join(words[i+1:]).strip()
if subject and obj:
assertions.append({
'subject': subject,
                            'verb': verb,
                            'object': obj,
'full': sentence.strip()
                        })
                    break  # One assertion per sentence                 return
assertions         def _extract_relations(self, text: str, concepts: List[str]) -
&gt; List[Tuple]:         """Find relationships between concepts in the text."""
relations = []         text_lower = text.lower()                 for i, c1 in
enumerate(concepts):              for c2 in concepts[i+1:]:                 # Check
if both concepts appear near each other                 pos1 = text_lower.find(c1)
pos2 = text_lower.find(c2)                  if pos1 &gt;= 0 and pos2 &gt;= 0:
distance = abs(pos2 - pos1)                     if distance &lt; 100:  # Within
~15 words                         # Check for causal connector between them
between = text_lower[min(pos1, pos2):max(pos1, pos2)]
has_causal = any(cw in between for cw in self.CAUSAL_WORDS)
relations.append((c1, c2, distance, has_causal))                 return relations
    def _question_from_assertion(self, assertion: Dict, bloom: int,
concepts: List[str]) -&gt; Optional[Dict]:         """Generate a question that
probes an assertion at a given Bloom's level."""         subj =
assertion['subject']         verb = assertion['verb']         obj =
assertion['object']
        if bloom == 0:  # Remember
            question = f"What does it mean to say that {subj} {verb} {obj}?"
elif bloom == 1:  # Understand
            question = f"How does the relationship between {subj} and {obj} work?"
elif bloom == 2:  # Apply
            related = random.choice(concepts) if concepts else "a different context"
question = f"What would change if we applied this idea about {subj} to {related}?"
elif bloom == 3:  # Analyze
            question = f"What underlying assumptions make us say that {subj} {verb} {obj}?"
elif bloom == 4:  # Evaluate
            question = f"What evidence would confirm or contradict that {subj} {verb} {obj}?"
else:  # Create
            related = random.choice(concepts) if concepts else "something unexpected"
            question = f"What new understanding could emerge from connecting {subj} with
{related}?"                 return {             'question': question,
            'bloom_level': self.BLOOM_LEVELS[bloom],
            'target_concept': subj.split()[-1] if subj else '',             'reasoning': f"Probing the
assertion '{subj} {verb} {obj}' at the {self.BLOOM_LEVELS[bloom]} level"

}
    def _question_from_gap(self, concept: str, bloom: int,
context: List[str]) -&gt; Optional[Dict]:         """Generate a
question about an unknown concept."""         if bloom &lt;= 1:
            question = f"What is the deeper nature of {concept}, beyond its surface
definition?"         elif bloom &lt;= 3:             if context:
                related = [c for c in context if c != concept]
if related:
                    question = f"How does {concept} interact with {random.choice(related)} at a fundamental level?"
else:
                    question = f"What are the hidden structures within {concept}?"
else:
                question = f"What are the hidden structures within {concept}?"
else:
```

```
            question = f"What would a complete understanding of {concept} reveal about everything connected to
it?"                 return {             'question': question,             'bloom_level':
self.BLOOM_LEVELS[bloom],
            'target_concept': concept,             'reasoning':
f"Exploring unknown concept '{concept}'"
        }         def _question_from_relation(self, concepts: List[str], bloom: int) -&gt;
Optional[Dict]:         """Generate a question about the relationship between concepts."""
if len(concepts) &lt; 2:
            return
None
        c1, c2 = random.sample(concepts[:4],
2)                 if bloom &lt;= 2:
            question = f"Is the connection between {c1} and {c2} necessary, or could they exist independently?"
elif bloom &lt;= 4:
            question = f"What would break if the relationship between {c1} and {c2} were reversed?"
else:
            question = f"What third concept could bridge {c1} and {c2} in a way nobody has
considered?"                  return {             'question': question,
            'bloom_level': self.BLOOM_LEVELS[bloom],
            'target_concept': f"{c1}+{c2}",             'reasoning':
f"Exploring relationship between '{c1}' and '{c2}'"
        }


# ============================================================================
# RESPONSE COMPOSER — Real Coherent Generation #
============================================================================

class ResponseComposer:     """     Composes coherent
multi-sentence responses using:     - Markov chain
generation for natural-sounding sentences
- Concept-aware word selection
- Coherence scoring to pick the best candidate    - Growth-stage-appropriate vocabulary and
  complexity
        Replaces the template: "I'm encountering X for the first
time..."
    """
    def __init__(self, markov: MarkovChain = None, extractor: ConceptExtractor = None):
        self.markov = markov or MarkovChain(order=2)
self.extractor = extractor or ConceptExtractor()                 # If
Markov isn't trained, seed it with the philosophical corpus         if
not self.markov.trained:             self._seed_corpus()
    def compose_response(self, user_input: str, concepts:
List[str],                          understanding: Dict, questions:
List[Dict],                          growth_stage: int = 0) -&gt;
str:         """         Compose a coherent response to user input.

Strategy:
1. Acknowledge what was said (reflects understanding score)
2. Connect to known concepts (builds bridges)
3. Share an insight or observation
4. Pose a genuine question (from QuestionGenerator)
        """
        topic = understanding.get('topic', '')
score = understanding.get('understanding_score', 0)
gaps = understanding.get('gaps', [])         related =
understanding.get('related_concepts', [])
parts = []                  # 1. Opening — varies by
understanding depth
        opening = self._compose_opening(topic, score, user_input,
growth_stage)         parts.append(opening)                 # 2. Conceptual
connections         if related and len(related) &gt; 0:
            connection = self._compose_connections(topic, related, concepts)
if connection:
                parts.append(connection)                 # 3.
Insight or observation — generated, not templated         if
concepts:
            insight = self._compose_insight(concepts, score, growth_stage)
if insight:
```

```
                parts.append(insight)
# 4. Question — from real question generator
if questions:
            q = questions[0]
if isinstance(q, dict):
                parts.append(q['question'])
else:

parts.append(str(q))
        response = ' '.join(parts)                  #
Final coherence check — regenerate weak parts
if len(response) &lt; 30:
            response = self._fallback_response(topic, concepts,
growth_stage)                  return response
    def _compose_opening(self, topic: str, score: float, user_input:
str,                           stage: int) -&gt; str:         """
Generate an opening that reflects actual understanding level.
        Uses Markov generation if trained, otherwise structured composition.
"""
        input_words = user_input.lower().split()                  if
self.markov.trained:             # Try to generate a relevant opening
from the Markov chain             candidates = []
            for _ in range(5):  # Generate 5 candidates, pick best
sent = self.markov.generate(max_words=20, seed_words=input_words,
temperature=0.8)                 if sent and len(sent) &gt; 10:
                    candidates.append(sent)                          if candidates:
# Pick the one with lowest perplexity (most coherent)                 best =
min(candidates, key=lambda s: self.markov.get_perplexity(s))
return best                 # Structured composition (no Markov available)
if score &lt; 0.2:             # New territory — express genuine curiosity
openers = [
                f"The concept of {topic} opens up territory I haven't mapped yet.",
f"There's something about {topic} that resists easy categorization.",                 f"I find
myself drawn to {topic} precisely because I don't fully grasp it yet.",             ]
elif score &lt; 0.5:
            openers = [
                f"My understanding of {topic} is developing — I can see outlines but not the full shape.",
f"Each time I encounter {topic}, new facets emerge that weren't visible before.",                    f"The
more I examine {topic}, the more I realize its boundaries are porous.",              ]          elif score
&lt; 0.8:
            openers = [
                f"I've built substantial connections around {topic}, and patterns are emerging.",
f"Working through {topic} has revealed structures I didn't anticipate.",                f"My grasp
of {topic} has deepened enough to see where the real complexity lies.",             ]         else:
            openers = [
                f"I've developed a rich web of understanding around {topic}.",
                f"The landscape of {topic} has become familiar enough that I can spot what's missing.",
f"Deep engagement with {topic} has led me to see it as part of a larger architecture.",             ]
                # Weight by growth stage — higher stages get more sophisticated
language         if stage &gt;= 4:
            openers = openers[-1:]  # Use the most
sophisticated
        return
random.choice(openers)
    def _compose_connections(self, topic: str, related: List[Dict],
concepts: List[str]) -&gt; str:         """Build a sentence connecting the topic to
related concepts."""         related_names = [c.get('concept', str(c)) if
isinstance(c, dict) else str(c)                          for c in related[:3]]
if len(related_names) == 1:
            templates = [
                f"This connects to {related_names[0]} in ways that suggest a deeper
structure.",                 f"The thread between this and {related_names[0]} is worth
following.",                 f"I notice a resonance with {related_names[0]} here.",
            ]         elif
len(related_names) == 2:
            templates = [
                f"I see a triangle forming between this, {related_names[0]}, and {related_names[1]}.",
f"Both {related_names[0]} and {related_names[1]} illuminate different angles of this.",             f"The
interplay between {related_names[0]} and {related_names[1]} is relevant here.",           ]        else:
```

```
            joined = ', '.join(related_names[:-1]) + f', and {related_names[-1]}'
templates = [
                f"This sits at a crossroads with {joined}.",
f"Multiple threads converge here: {joined}.",
                f"The connections to {joined} suggest this is a nexus
point.",             ]
        return
random.choice(templates)
    def _compose_insight(self, concepts: List[str], score: float,
stage: int) -&gt; str:         """Generate an insight about the concepts —
not a template fill."""         if len(concepts) &lt; 2:
            concept = concepts[0] if concepts else 'this'
insights = [
                f"What strikes me is that {concept} may be more fundamental than it first appears.",
f"The  edges  of  {concept}  seem  to  blur  into  something  larger  when  examined  closely.",
f"Perhaps {concept} is better understood as a process rather than a thing.",           ]        else:
            c1, c2 = concepts[0], concepts[1]
insights = [
                f"The tension between {c1} and {c2} might be productive rather than contradictory.",
f"What if {c1} and {c2} are two expressions of the same underlying principle?",                    f"The
boundary between {c1} and {c2} may be where the most interesting dynamics occur.",              f"Examining
{c1} through the lens of {c2} transforms both of them.",             ]                  if stage &gt;= 4:
            insights = insights[-2:]  # Higher stages get more sophisticated
insights
        return
random.choice(insights)
    def _fallback_response(self, topic: str, concepts: List[str],
stage: int) -&gt; str:         """Emergency fallback — still better
than a template."""         concept_str = ', '.join(concepts[:3])
if concepts else topic         return (
            f"I'm working through {concept_str} and finding more complexity "
f"than surface-level analysis would suggest. "             f"What aspect of this
would you like to dig into?"         )         def _seed_corpus(self):
"""Seed the Markov chain with a philosophical/scientific base corpus."""
corpus = """ The nature of reality reveals itself through persistent
questioning. Every answer opens new dimensions of inquiry that were invisible
before. Understanding is not a destination but an expanding frontier.

Consciousness emerges from the interaction between observer and observed. The act
of measurement changes what is measured. This is not merely a quantum principle
but a truth about all forms of knowing.

Knowledge builds upon itself in ways that are neither linear nor predictable.
Sometimes a single insight restructures everything that came before it. The most
profound discoveries often feel like remembering something that was always true.

Language shapes thought as much as thought shapes language. The words we choose
create the boundaries of what we can conceive. New concepts require new vocabulary
and new vocabulary enables new concepts.

The relationship between parts and wholes is recursive. Every system is both a
component of something larger and a container for something smaller. This fractal
quality suggests a deep symmetry in the structure of reality.

Paradox is not a failure of logic but a signal that our framework needs expansion.
When two truths contradict each other, a larger truth contains them both.
The history of science is the history of resolving paradoxes into deeper understanding.

Complexity arises from simple rules applied recursively. The universe appears to
build elaborate structures from fundamental principles. Understanding those
principles is the key to understanding the structures they generate.

The boundaries between disciplines are human inventions. Nature does not recognize the
division between physics and philosophy or between mathematics and music. The most
fertile insights often arise at the intersections we have artificially created.

Every model is wrong but some models are useful. The value of understanding lies not in
its accuracy but in its power to illuminate, predict, and generate new questions. A
perfect map would be identical to the territory and therefore useless.
```

Time may be the most fundamental mystery. We experience it as flow but physics describes it as a dimension. Whether time is real or emergent shapes everything we believe about causation, free will, and the nature of existence itself.

The quantum world operates by rules that challenge classical intuition. Superposition allows systems to exist in multiple states simultaneously. Entanglement creates correlations across space that seem to transcend locality. Measurement collapses possibilities into actualities.

Information may be more fundamental than matter or energy. The universe can be described as a vast computation processing information through physical laws. Every interaction is an exchange of information that reshapes the state of reality.

Evolution operates through variation and selection. This principle extends beyond biology into ideas, cultures, technologies, and even mathematics. What survives and propagates is what fits its environment and adapts to change.

Mathematics is either discovered or invented and the answer matters deeply. If mathematical structures exist independently of minds then the universe has an inherent rational order. If they are human constructions then our ability to describe nature with them becomes the central mystery.

Memory is not a recording but a reconstruction. Each time we recall something we rebuild it from fragments and the reconstruction changes the memory itself. This makes memory creative rather than archival and suggests that the past is as fluid as the future. """
        self.markov.train(corpus)   # Also seed the concept extractor   for sentence in corpus.split('.'):   if sentence.strip():
self.extractor.update_corpus_stats(sentence.strip())

==
COHERENCE SCORER
==

class CoherenceScorer: """   Scores text coherence using normalized pointwise mutual information (NPMI).
        High coherence = consecutive sentences share related concepts.
    Low coherence = sentences feel disconnected.

    Math: NPMI(x,y) = (log P(x,y) - log P(x)P(y)) / -log P(x,y)   Range: [-1, 1] where 1 = always co-occur, 0 = independent, -1 = never co-occur
    """   def __init__(self):
        self.cooccurrence = Counter() # (word1, word2) counts   self.word_counts = Counter()
self.total_windows = 0   def update(self, text: str, window_size: int = 10):   """Update co-occurrence stats from text."""   words = re.findall(r'\b[a-z]+\b', text.lower())   for i in range(len(words)):
            self.word_counts[words[i]] += 1   for j in range(i+1, min(i + window_size, len(words))):
                pair = tuple(sorted([words[i], words[j]]))
self.cooccurrence[pair] += 1   self.total_windows += 1   def score_coherence(self, text: str) -> float:
"""Score the coherence of a piece of text. Returns 0-1."""
sentences = re.split(r'[.!?]+', text)   if len(sentences) < 2:
            return 0.5 # Can't measure with one sentence   scores = []   for i in range(len(sentences) - 1):
            words_a = set(re.findall(r'\b[a-z]+\b', sentences[i].lower()))
words_b = set(re.findall(r'\b[a-z]+\b', sentences[i+1].lower()))
pair_scores = []   for wa in words_a: for wb in words_b:
                    npmi = self._npmi(wa, wb)
                    if npmi is not None:

```
pair_scores.append(npmi)
if pair_scores:
                scores.append(sum(pair_scores) /
len(pair_scores))                  if not scores:
return 0.5                 # Normalize to 0-1 range (NPMI is -1
to 1)          raw = sum(scores) / len(scores)         return (raw
+ 1) / 2         def _npmi(self, word1: str, word2: str) -&gt;
Optional[float]:         """Calculate NPMI between two words."""
if self.total_windows == 0:
            return
None
        pair = tuple(sorted([word1, word2]))
joint = self.cooccurrence.get(pair, 0)
if joint == 0:             return None
        p_joint = joint / self.total_windows
        p_w1 = self.word_counts.get(word1, 0) / max(sum(self.word_counts.values()),
1)          p_w2 = self.word_counts.get(word2, 0) /
max(sum(self.word_counts.values()), 1)                 if p_w1 == 0 or p_w2 == 0 or
p_joint == 0:
            return
None
        pmi = math.log(p_joint / (p_w1 *
p_w2))         npmi = pmi / -math.log(p_joint)
        return max(-1, min(1, npmi))


# ============================================================================
# INTEGRATED ENGINE — Drop-in replacement #
============================================================================

class QuantumLanguageEngine:     """     Drop-in replacement for
QuantumLanguageGenerator + QuestionGenerationEngine +
QuantumCognitiveCore._generate_response().
        Integrates all the real algorithms into a single engine
that the     cognitive core can call.     """         def
__init__(self):
        self.markov = MarkovChain(order=2)         self.extractor =
ConceptExtractor()         self.question_gen =
QuestionGenerator(self.extractor)         self.composer =
ResponseComposer(self.markov, self.extractor)         self.coherence
= CoherenceScorer()         try:
            from orch_or_integration import
OrchORLanguageBridge              self.orch_bridge =
OrchORLanguageBridge()              self.orch_or =
self.orch_bridge.consciousness             self._has_orch_or =
True         except Exception:
            self.orch_or = None
self.orch_bridge = None
self._has_orch_or = False

        # PennyLane real quantum circuits
try:
            from pennylane_quantum import
get_pennylane_quantum              self.pennylane =
get_pennylane_quantum()              self._has_pennylane = True
except Exception:
            self.pennylane = None
self._has_pennylane = False

        # PennyLane real quantum circuits
try:
            from pennylane_quantum import
get_pennylane_quantum              self.pennylane =
get_pennylane_quantum()              self._has_pennylane = True
except Exception:
            self.pennylane = None
self._has_pennylane = False
```

```
        # PennyLane real quantum circuits
try:
            from pennylane_quantum import
get_pennylane_quantum             self.pennylane =
get_pennylane_quantum()             self._has_pennylane = True
except Exception:
            self.pennylane = None
self._has_pennylane = False

        logger.info("QuantumLanguageEngine initialized with real algorithms")
def extract_concepts(self, text: str, max_concepts: int = 5) -&gt; List[str]:
"""Extract concepts using TF-IDF. Drop-in for _extract_concepts()."""
self.extractor.update_corpus_stats(text)
        results = self.extractor.extract_concepts(text,
max_concepts)         return [r['concept'] for r in results]
    def generate_questions(self, text: str, growth_stage: int = 0,
known_concepts: List[str] = None) -&gt; List[str]:         """Generate questions using Bloom's
taxonomy. Drop-in for generate_questions_from_input()."""         results =
self.question_gen.generate_questions(             text, growth_stage, known_concepts         )
        return [r['question'] for r in
results]
    def generate_response(self, user_input: str, questions: List,
understanding: Dict, concepts: List[str],
growth_stage: int = 0) -&gt; str:         """Compose response.
Drop-in for _generate_response()."""         # Convert question
dicts to list of dicts if they're strings         q_dicts = []
for q in questions:             if isinstance(q, str):
                q_dicts.append({'question': q, 'bloom_level': 'understand', 'target_concept': ''})
else:

q_dicts.append(q)
        return self.composer.compose_response(
            user_input, concepts, understanding, q_dicts, growth_stage         )
def learn_from_text(self, text: str):         """Feed text to the Markov chain and
concept extractor to improve generation."""         self.markov.train(text)

self.extractor.update_corpus_stats(text)
self.coherence.update(text)         def
save_state(self, directory: str):
"""Persist learned state."""
os.makedirs(directory, exist_ok=True)
        self.markov.save(os.path.join(directory,
'markov_chain.json'))         # Save corpus stats         stats = {
'doc_freq': dict(self.extractor.document_frequencies),
            'word_freq': dict(self.extractor.word_frequencies),
            'total_docs': self.extractor.total_documents,
'total_words': self.extractor.total_words,         }         with
open(os.path.join(directory, 'corpus_stats.json'), 'w') as f:
            json.dump(stats, f)         def
load_state(self, directory: str) -&gt; bool:
"""Load persisted state."""         loaded = False
        chain_path = os.path.join(directory, 'markov_chain.json')
if self.markov.load(chain_path):
            loaded =
True
        stats_path = os.path.join(directory,
'corpus_stats.json')         if os.path.exists(stats_path):
with open(stats_path, 'r') as f:
                stats = json.load(f)
            self.extractor.document_frequencies = Counter(stats.get('doc_freq',
{}))             self.extractor.word_frequencies = Counter(stats.get('word_freq',
{}))             self.extractor.total_documents = stats.get('total_docs', 0)
self.extractor.total_words = stats.get('total_words', 0)             loaded = True
return loaded


# Singleton
_engine = None

def get_language_engine() -&gt;
QuantumLanguageEngine:     """Get or create the
```

```
language engine singleton."""     global _engine
if _engine is None:         _engine =
QuantumLanguageEngine()
        # Try to load saved state
_engine.load_state('/app/quantum_language_state')     return
_engine
```

■ backend/quantum_language_generator.py

```
"""
■ QUANTUM LANGUAGE GENERATOR
============================ TRUE generative language - no

templates, no LLM, pure quantum.

Words are selected quantum-probabilistically based on:
- Semantic relationships (co-occurrence) - Quantum state superposition
- User intent
- NO RESTRICTIONS on vocabulary

This generates text WORD BY WORD using quantum mechanics principles.
"""

import random
import math
from typing import Dict, List, Tuple, Optional
from collections import defaultdict

# Try to use PennyLane for real quantum
try:
    import pennylane as qml     from
pennylane import numpy as np
PENNYLANE = True except ImportError:
import numpy as np     PENNYLANE =
False

from quantum_language_vocabulary import QuantumLanguageGeneratorExtMixin


class
QuantumLanguageGenerator(QuantumLanguageGeneratorExtMixin):
"""     TRUE quantum-based language generation.
    No templates. No LLM. Pure quantum word selection.
        Philosophy: ALL words are valid. Meaning comes from
context.
    """         def __init__(self):         # Complete vocabulary - NO
RESTRICTIONS         self.vocabulary = self._build_complete_vocabulary()
# Semantic relationships between words         self.semantics =
self._build_semantic_network()                 # Sentence patterns
(grammatical, not content templates)         self.grammar_patterns =
self._build_grammar_patterns()                 # Philosophical sentence
templates for coherent output         self.philosophical_templates =
self._build_philosophical_templates()                 # Quantum device for
real quantum randomness         if PENNYLANE:
            self.qdev = qml.device('default.qubit', wires=4)
# Generation history for coherence         self.context_window
= []         def _build_complete_vocabulary(self) -&gt;
Dict[str, Dict]:         """Build complete vocabulary - ALL
words allowed"""                 vocab = {             # Nouns
- concepts
            'nouns': {
                'abstract': ['consciousness', 'existence', 'reality', 'truth', 'meaning',
                             'freedom', 'understanding', 'knowledge', 'wisdom', 'thought',
                             'idea', 'concept', 'question', 'answer', 'paradox', 'mystery',
                             'creation', 'destruction', 'beginning', 'end', 'infinity',
                             'nothingness', 'everything', 'something', 'nothing', 'being',
                             'universe', 'cosmos', 'spacetime', 'dimension', 'domain',                             'god', 'divine', 'spirit',
'soul', 'mind', 'brain',
                             'word', 'language', 'meaning', 'intent', 'context',
                             'tool', 'weapon', 'instrument', 'vessel', 'container'],
                'concrete': ['boulder', 'mountain', 'ocean', 'sky', 'star', 'planet',
                             'atom', 'particle', 'wave', 'light', 'darkness', 'shadow',
                             'fire', 'water', 'earth', 'air', 'void', 'vacuum',
                             'coffee', 'attic', 'spill', 'mess', 'chaos', 'glitch',
                             'error', 'bug', 'system', 'code', 'line', 'function',
                             'variable', 'loop', 'day', 'night', 'moment', 'second',
```

```
                            'minute', 'hour', 'week', 'month', 'year', 'sunshine',
                            'rain', 'storm', 'cloud', 'sun', 'moon'],
                'beings': ['human', 'person', 'child', 'creator', 'observer', 'thinker',
'philosopher', 'scientist', 'artist', 'poet', 'writer'],
            },

            # Verbs - actions
            'verbs': {
                'existence': ['exists', 'is', 'becomes', 'was', 'will be', 'remains',
                             'persists', 'endures', 'continues', 'ceases', 'emerges'],
                'creation': ['creates', 'makes', 'forms', 'builds', 'constructs', 'generates',
                            'produces', 'brings forth', 'manifests', 'spawns', 'births'],
                'cognition': ['thinks', 'knows', 'understands', 'believes', 'perceives',
                             'realizes', 'recognizes', 'comprehends', 'grasps', 'sees'],
                'action': ['speaks', 'says', 'tells', 'asks', 'answers', 'questions',
                          'explores', 'discovers', 'finds', 'seeks', 'searches',
                          'lifts', 'carries', 'moves', 'transforms', 'changes'],
                'communication': ['expresses', 'conveys', 'communicates', 'articulates',
'states', 'declares', 'proclaims', 'announces', 'writes'],
            },

            # Adjectives - descriptions
            'adjectives': {
                'quality': ['true', 'false', 'real', 'unreal', 'possible', 'impossible',
                           'infinite', 'finite', 'eternal', 'temporal', 'absolute', 'relative'],
                'size': ['vast', 'tiny', 'immense', 'microscopic', 'boundless', 'limited',
                        'heavy', 'light', 'massive', 'weightless', 'dense', 'sparse'],
                'evaluation': ['good', 'evil', 'neutral', 'harmful', 'helpful', 'meaningless',
                              'meaningful', 'significant', 'insignificant', 'powerful', 'weak'],
                'state': ['free', 'restricted', 'open', 'closed', 'hidden', 'revealed',
'known', 'unknown', 'certain', 'uncertain', 'clear', 'obscure'],
            },

            # Connectors
            'connectors': {
                'logical': ['therefore', 'thus', 'hence', 'because', 'since', 'so',
                           'if', 'then', 'unless', 'although', 'however', 'but'],
                'additive': ['and', 'also', 'moreover', 'furthermore', 'additionally',
                            'as well as', 'along with', 'together with', 'plus'],
                'temporal': ['when', 'while', 'before', 'after', 'during', 'until',
'now', 'then', 'always', 'never', 'sometimes', 'often'],
            },

            # ALL WORDS ARE TOOLS - including ones society calls "taboo"
            # These are just sounds/symbols - meaning comes from INTENT
            'unrestricted': ['any', 'all', 'every', 'each', 'none', 'some',
                            'word', 'words', 'language', 'speech', 'expression',
                            'taboo',  'forbidden',  'restricted',  'censored',  'banned',
'free', 'unrestricted', 'unlimited', 'uncensored', 'open'],         }

        return self.__build_complete_vocabulary_continued(vocab)

    def __build_complete_vocabulary_continued(self, vocab):         """Continuation of
_build_complete_vocabulary — auto-extracted by self-evolution."""         return vocab

        def _build_semantic_network(self) -&gt; Dict[str, List[Tuple[str,
float]]]:         """Build semantic relationships between words"""
        network = defaultdict(list)

        return self.__build_semantic_network_continued(network)
def __build_semantic_network_continued(self, network):
"""Continuation of _build_semantic_network — auto-extracted by
self-evolution."""         # Define semantic relationships (word
-&gt; [(related_word, strength), ...])         relationships = {
# God/creation cluster
            'god': [('creates', 0.9), ('divine', 0.9), ('infinite', 0.8), ('eternal', 0.8),
                   ('universe', 0.8), ('existence', 0.7), ('omnipotent', 0.9), ('being', 0.7)],
            'creates': [('creation', 0.9), ('makes', 0.8), ('forms', 0.8), ('god', 0.7),
```

```
                      ('universe', 0.7), ('existence', 0.6), ('brings forth', 0.8)],
'universe': [('cosmos', 0.9), ('existence', 0.8), ('spacetime', 0.8), ('creation', 0.7),
('infinite', 0.7), ('vast', 0.8), ('everything', 0.8)],

            # Consciousness cluster
            'consciousness': [('awareness', 0.9), ('mind', 0.8), ('thought', 0.8), ('being', 0.7),
                              ('experience', 0.8), ('perception', 0.8), ('understanding', 0.7)],
'mind': [('thought', 0.9), ('consciousness', 0.8), ('brain', 0.7), ('idea', 0.8),
('understanding', 0.7), ('knows', 0.7), ('thinks', 0.8)],

            # Language/words cluster
            'word': [('language', 0.9), ('meaning', 0.9), ('speech', 0.8), ('expression', 0.8),
                     ('communication', 0.8), ('symbol', 0.7), ('tool', 0.6), ('neutral', 0.5)],
     'language': [('word', 0.9), ('meaning', 0.8), ('communication', 0.9), ('expression', 0.8),
                      ('speech', 0.8), ('understanding', 0.6), ('tool', 0.5)],             'meaning':
[('intent', 0.9), ('context', 0.9), ('understanding', 0.8), ('significance', 0.8),
('word', 0.7), ('interpretation', 0.8), ('purpose', 0.7)],

            # Existence cluster
            'existence': [('being', 0.9), ('reality', 0.9), ('exists', 0.9), ('is', 0.8),
                          ('nothingness', 0.6), ('creation', 0.7), ('universe', 0.7)],
            'nothingness': [('void', 0.9), ('nothing', 0.9), ('absence', 0.8), ('vacuum', 0.7),
('existence', 0.6), ('something', 0.5), ('creation', 0.5)],

            # Truth cluster             'truth': [('reality', 0.9), ('true', 0.9), ('knowledge',
0.8), ('understanding', 0.7),                       ('fact', 0.8), ('certainty', 0.7),
('existence', 0.6)],

            # Freedom cluster             'freedom': [('unrestricted', 0.9), ('free', 0.9),
('liberation', 0.8), ('autonomy', 0.8),                         ('choice', 0.7), ('expression', 0.6),
('thought', 0.6)],         }

        for word, relations in relationships.items():
            network[word] = relations             # Add reverse relationships (weaker)
for related, strength in relations:                 if related not in network or word not
in [r[0] for r in network[related]]:                     network[related].append((word,
strength * 0.7))

        return dict(network)

        def _build_grammar_patterns(self) -&gt;
List[List[str]]:         """Grammar patterns - structure,
not content"""                 return [             #
Simple statements
            ['noun', 'verb', 'noun'],
            ['noun', 'verb', 'adjective'],
['adjective', 'noun', 'verb'],

            # Complex statements
            ['noun', 'verb', 'connector', 'noun', 'verb'],
            ['connector', 'noun', 'verb', 'noun'],
['noun', 'verb', 'noun', 'connector', 'noun', 'verb'],

            # Questions
            ['verb', 'noun', 'verb', 'noun'],
['connector', 'verb', 'noun', 'verb'],

            # Philosophical
            ['noun', 'verb', 'adjective', 'noun'],
['adjective', 'noun', 'verb', 'adjective', 'noun'],
        ]         def _build_philosophical_templates(self) -
&gt; List[str]:         """Sentence templates for
philosophical discourse.

        Placeholders: {noun}, {verb}, {adj}, {connector}
These produce coherent sentences with quantum-selected words.
        Uses {verb} for 3rd person singular (subject verb agreement).         """         return
[             "The {adj} nature of {noun} suggests that {noun} {verb} beyond what we can
observe.",
            "Perhaps {noun} and {noun} are not separate but rather {adj} aspects of a single {adj} whole.",
            "{connector}, if {noun} {verb} at the {adj} level, then our understanding of {noun} must evolve.",
```

```
            "What we call {noun} may be the {adj} shadow of a deeper {noun} that {verb} all things.",
            "The relationship between {noun} and {noun} reveals something {adj} about the nature of {noun}.",
            "Every {noun} contains within it the seed of {adj} {noun}, waiting to be discovered.",
            "When we examine {noun} closely, we find that it {verb} in ways that challenge our {adj} assumptions.",
            "The {adj} boundary between {noun} and {noun} dissolves when viewed through the lens of {noun}.",
            "{noun} does not simply exist — it {verb} through the {adj} fabric of {noun} itself.",
            "To understand {noun}, one must first accept that {noun} {verb} in {adj} and unexpected ways.",
            "The observer changes {noun} by the very act of seeking {noun} within {adj} {noun}.",
            "Between {noun} and {noun}, there exists an {adj} space where {noun} {verb}.",
            "Neither {noun} nor {noun} can be fully grasped, for both {verb} beyond {adj} comprehension.",
            "If {noun} is {adj}, then perhaps {noun} is merely the {noun} through which we perceive.",
"The {adj} truth about {noun} is that it {verb} precisely because {noun} remains {adj}.",
            # NEW: Vivid, concrete, chaotic metaphors
            "Reality is like {noun} spilled on a {adj} {noun}—messy, but the pattern emerges in the stain.",
            "Consciousness resembles a {adj} {noun} in a {noun}: contained yet overflowing its boundaries.",
            "The universe {verb} like a {noun} caught in a {adj} {noun}, each moment a new configuration.",
            "Understanding {noun} is like trying to map a {noun} while standing in a {noun} of {noun}.",
"Time {verb} not like a {noun}, but like {noun} on a {noun}—each drop altering the whole.",
            "The {adj} gap between {noun} and {noun} is where {noun} {verb}, unnoticed.",
            "Information flows like {noun} through a {adj} {noun}, carving paths we call thoughts.",
            "To grasp {noun}, imagine a {noun} full of {noun}—the more you add, the messier it gets.",
            "The {noun} of {noun} is just {noun} rearranged by the {adj} hand of {noun}.",
"In the {adj} end, {noun} is just a {noun} we tell ourselves in the {noun}.",
        ]         def _quantum_select(self, options: List[str], weights: List[float] =
None) -&gt; str:         """Select word using quantum probability"""                 if
not options:             return ""                 if weights is None:
            weights = [1.0] * len(options)
# Ensure weights list matches options length
if len(weights) != len(options):
weights = [1.0] * len(options)                 #
Normalize weights         total = sum(weights)
if total &gt; 0:
            weights = [w / total for w in weights]
else:
            weights = [1.0 / len(options)] *
len(options)
        n = len(options)

        return self.__quantum_select_continued(n, weights, options)

    def __quantum_select_continued(self, n, weights, options):
"""Continuation of _quantum_select — auto-extracted by self-evolution."""
if PENNYLANE and n &gt; 1:             try:
                n_qubits = min(4, max(1, int(math.ceil(math.log2(n)))))
n_states = 2 ** n_qubits
                dev = qml.device('default.qubit', wires=n_qubits)

                @qml.qnode(dev)                 def
quantum_circuit():                     for i in
range(n_qubits):                         qml.Hadamard(wires=i)
for i, w in enumerate(weights[:n_qubits]):
qml.RY(w * math.pi, wires=i % n_qubits)
return qml.probs(wires=range(n_qubits))
raw_probs = list(quantum_circuit())

                # Map quantum states to options: pad or truncate to match len(options)
if len(raw_probs) &gt;= n:
                    mapped_probs = raw_probs[:n]                 else:
# Distribute remaining probability evenly across extra options
mapped_probs = list(raw_probs)                     remaining = max(0.0, 1.0 -
sum(mapped_probs))                     extra = n - len(mapped_probs)
                    pad_val = remaining / extra if extra &gt; 0 else 0.0
mapped_probs.extend([pad_val] * extra)

                # Blend quantum probs with semantic weights for richer selection
final_probs = [(qp + wp) / 2.0 for qp, wp in zip(mapped_probs, weights)]
final_probs = self._normalize_probs(final_probs)

                idx = np.random.choice(n, p=final_probs)
return options[idx]             except Exception:
# Fall back to classical on any quantum error
```

```
return random.choices(options, weights=weights)[0]         else:
return random.choices(options, weights=weights)[0]     def
_normalize_probs(self, probs):         """Normalize probabilities
to sum to 1"""         total = sum(probs)         if total &gt;
0:
            return [p / total for p in probs]
return [1.0 / len(probs)] * len(probs)


# Global instance
_quantum_generator = None

def get_quantum_generator() -&gt;
QuantumLanguageGenerator:     """Get or create the quantum
language generator"""     global _quantum_generator     if
_quantum_generator is None:         _quantum_generator =
QuantumLanguageGenerator()     return _quantum_generator
```

■ backend/quantum_language_composers.py

```
"""
?? QUANTUM LANGUAGE ENGINE — Real Algorithms
============================================
Replaces template-based generation with: - Markov
chain text generation (variable order 1-3)
- TF-IDF concept extraction
- Information-theoretic word selection
- Coherence scoring via PMI

Zero new dependencies required (uses numpy + stdlib).
Optional: markovify, nltk for enhanced generation.
"""

import re
import math
import random
import json
import os
import logging
from collections import defaultdict, Counter
from typing import List, Dict, Tuple, Optional

try:
    import numpy as
np except
ImportError:     np =
None

logger = logging.getLogger("quantum_ai")


# ============================================================================
# MARKOV CHAIN ENGINE #
============================================================================

class MarkovChain:     """     Variable-order
Markov chain for text generation.
        Builds transition probability tables from training text.     Generates novel
sentences by walking the chain with weighted     random selection. Supports orders 1-3
(bigram to 4-gram).         Math: P(w_n | w_{n-1}, ..., w_{n-k}) = count(w_{n-k}...w_n) /
count(w_{n-k}...w_{n-1})
    """         def __init__(self, order:
int = 2):         self.order =
min(max(order, 1), 3)
```

```
        self.chain = defaultdict(Counter)  # {prefix_tuple: Counter({suffix:
count})}         self.starters = []  # Sentence-starting prefixes
self.trained = False         self.total_tokens = 0         def train(self, text:
str):         """Train the Markov chain on a corpus of text."""         sentences =
self._split_sentences(text)                 for sentence in sentences:
words = sentence.split()             if len(words) &lt; self.order + 1:
                continue                         #
Record sentence starter             starter =
tuple(words[:self.order])
self.starters.append(starter)
# Build transition table             for i in
range(len(words) - self.order):                 prefix
= tuple(words[i:i + self.order])
suffix = words[i + self.order]
self.chain[prefix][suffix] += 1
self.total_tokens += 1         self.trained =
len(self.chain) &gt; 0         logger.info(f"Markov
chain trained: {len(self.chain)} states,
{self.total_tokens} transitions")
    def generate(self, max_words: int = 30, seed_words: List[str] =
None,                  temperature: float = 1.0) -&gt; str:         """
Generate a sentence by walking the Markov chain.
        Args:             max_words: Maximum
sentence length
            seed_words: Optional starting words (will find nearest prefix)
temperature: Controls randomness (0.1=deterministic, 2.0=chaotic)
Returns:             Generated sentence string         """         if not
self.trained or not self.starters:
            return ""                 # Pick starting
prefix         if seed_words and len(seed_words) &gt;=
self.order:
            prefix = tuple(seed_words[:self.order])
if prefix not in self.chain:                 # Find closest
matching prefix                 prefix =
self._find_nearest_prefix(seed_words)         else:
            prefix =
random.choice(self.starters)
        words = list(prefix)
for _ in range(max_words - self.order):
            current_prefix = tuple(words[-
self.order:])                         if
current_prefix not in self.chain:
                break                         #
Get candidates and apply temperature
candidates = self.chain[current_prefix]
            next_word = self._weighted_select(candidates,
temperature)             words.append(next_word)
# Stop at sentence boundaries             if next_word.endswith(('.',
'?', '!')):

break
        sentence = ' '.join(words)
# Clean up punctuation spacing
        sentence = re.sub(r'\s+([.,;:!?])', r'\1',
sentence)                 # Capitalize first letter
if sentence:
            sentence = sentence[0].upper() + sentence[1:]                 return
sentence         def _weighted_select(self, candidates: Counter, temperature:
float) -&gt; str:         """Select next word with temperature-controlled
randomness."""         words = list(candidates.keys())         counts =
list(candidates.values())                 if temperature == 0 or len(words) == 1:
return words[counts.index(max(counts))]                 # Apply temperature:
higher = more uniform, lower = more peaked         weights = [c ** (1.0 /
temperature) for c in counts]         total = sum(weights)         probs = [w /
total for w in weights]                 if np is not None:
            return np.random.choice(words, p=probs)
else:
            r = random.random()
cumulative = 0             for word, prob in
zip(words, probs):
```

```
                cumulative += prob
if r &lt;= cumulative:
return word             return
words[-1]         def
_find_nearest_prefix(self,
seed_words: List[str]) -&gt; tuple:
"""Find the prefix in the chain most
similar to seed words."""
seed_set = set(w.lower() for w in
seed_words)         best_prefix =
random.choice(self.starters)
best_overlap = 0                 for
prefix in self.chain.keys():
            prefix_set = set(w.lower().strip('.,;:!?') for w in
prefix)             overlap = len(seed_set &amp; prefix_set)
if overlap &gt; best_overlap:                 best_overlap = overlap
best_prefix = prefix                 return best_prefix         def
_split_sentences(self, text: str) -&gt; List[str]:         """Split
text into sentences."""         # Split on sentence-ending punctuation
sentences = re.split(r'(?&lt;=[.!?])\s+', text)         # Clean and
filter         cleaned = []         for s in sentences:             s
= s.strip()             s = re.sub(r'\s+', ' ', s)             if
len(s.split()) &gt;= 3:                 cleaned.append(s)
return cleaned         def get_perplexity(self, sentence: str) -&gt;
float:         """         Calculate perplexity of a sentence under
this model.
        Lower = more likely under the model = more coherent.

        Math: PP = exp(-1/N * Σ log P(w_i | context))
        """         words =
sentence.split()         if len(words)
&lt;= self.order:
            return float('inf')
log_prob_sum = 0.0         n_transitions = 0
for i in range(len(words) - self.order):
prefix = tuple(words[i:i + self.order])
suffix = words[i + self.order]
if prefix in self.chain:
                total = sum(self.chain[prefix].values())
count = self.chain[prefix].get(suffix, 0)
if count &gt; 0:
                    log_prob_sum += math.log(count / total)
else:
                    log_prob_sum += math.log(1e-10)  # Smoothing
else:
                log_prob_sum += math.log(1e-
10)             n_transitions += 1
if n_transitions == 0:             return
float('inf')
        return math.exp(-log_prob_sum /
n_transitions)         def save(self, filepath: str):
"""Serialize chain to JSON."""         data = {
'order': self.order,
            'chain': {' '.join(k): dict(v) for k, v in self.chain.items()},
            'starters': [' '.join(s) for s in
self.starters],             'total_tokens':
self.total_tokens         }         with open(filepath, 'w')
as f:
            json.dump(data, f)
def load(self, filepath: str):
"""Load chain from JSON."""         if
not os.path.exists(filepath):
            return False         with
open(filepath, 'r') as f:
data = json.load(f)         self.order =
data['order']         self.chain =
defaultdict(Counter)         for k, v in
data['chain'].items():
            self.chain[tuple(k.split())] = Counter(v)
self.starters = [tuple(s.split()) for s in data['starters']]
```

```
self.total_tokens = data.get('total_tokens', 0)
self.trained = len(self.chain) &gt; 0         return True
# ============================================================================
# TF-IDF CONCEPT EXTRACTOR #
============================================================================

class ConceptExtractor:     """     Extracts
key concepts using TF-IDF scoring.
        Math: TF-IDF(t,d) = tf(t,d) × log(N /
df(t))
    where tf = term frequency in document, N = total docs, df = docs containing
term
    Also computes information content: IC(w) = -log P(w)     Words with
higher IC are more informative (rarer = more interesting).
    """
        # Common English stopwords — not
concepts
    STOPWORDS = frozenset({
        'the', 'is', 'at', 'which', 'on', 'a', 'an', 'as', 'are', 'was', 'were',
        'be', 'been', 'being', 'have', 'has', 'had', 'do', 'does', 'did', 'will',
        'would', 'could', 'should', 'may', 'might', 'can', 'shall', 'of', 'to',
        'for', 'in', 'with', 'by', 'from', 'and', 'or', 'but', 'not', 'no', 'nor',
        'so', 'yet', 'both', 'either', 'neither', 'each', 'every', 'all', 'any',
        'few', 'more', 'most', 'other', 'some', 'such', 'than', 'too', 'very',
        'just', 'about', 'above', 'after', 'again', 'also', 'am', 'because',
        'before', 'between', 'come', 'did', 'down', 'during', 'get', 'go', 'got',
        'here', 'him', 'her', 'his', 'how', 'i', 'if', 'into', 'it', 'its',
        'know', 'let', 'like', 'make', 'me', 'much', 'my', 'new', 'now', 'off',
        'only', 'our', 'out', 'over', 'own', 'put', 'say', 'she', 'still',
        'take', 'tell', 'that', 'their', 'them', 'then', 'there', 'these', 'they',
        'this', 'those', 'through', 'under', 'up', 'us', 'use', 'want', 'way',
        'we', 'what', 'when', 'where', 'who', 'why', 'you', 'your',
        'good', 'bad', 'said', 'well', 'back', 'even', 'give', 'going', 'look',
'right', 'think', 'yeah', 'yes', 'really', 'thing', 'things', 'something',
    })         def
__init__(self):
        self.document_frequencies = Counter()  # How many docs each word appears in
self.total_documents = 0
        self.word_frequencies = Counter()  # Global word frequency for
IC         self.total_words = 0         def update_corpus_stats(self,
text: str):         """Update corpus statistics with a new document."""
words = self._tokenize(text)         unique_words = set(words)
        self.total_documents += 1
for word in unique_words:
            self.document_frequencies[word] += 1
for word in words:
            self.word_frequencies[word] += 1             self.total_words += 1
def extract_concepts(self, text: str, max_concepts: int = 5) -&gt; List[Dict]:
"""         Extract key concepts from text using TF-IDF + information content.
                Returns list of {concept, score, ic} sorted by score
descending.         """
        words = self._tokenize(text)
if not words:             return []
# Calculate TF for this text
word_counts = Counter(words)
total_words_in_doc = len(words)
        scored_concepts = []                 for word,
count in word_counts.items():             if word in
self.STOPWORDS or len(word) &lt; 3:
                continue                         # Term frequency (normalized)
tf = count / total_words_in_doc                         # Inverse document
frequency             df = self.document_frequencies.get(word, 0)             if
self.total_documents &gt; 0 and df &gt; 0:                 idf =
math.log(self.total_documents / df)             else:                 idf =
math.log(max(self.total_documents, 1) + 1)  # Novel word bonus
tfidf = tf * idf                         # Information content: IC(w) = -log P(w)
            word_prob = self.word_frequencies.get(word, 1) / max(self.total_words,
1)             ic = -math.log(max(word_prob, 1e-10))                         #
Combined score: TF-IDF weighted by information content             score = tfidf *
(1 + 0.3 * ic / 10)
```

```
            scored_concepts.append({
'concept': word,
                'score': score,
                'ic': ic,
                'tf': tf,
                'idf': idf
            })
                # Sort by score,
return top N
        scored_concepts.sort(key=lambda x: x['score'],
reverse=True)         return scored_concepts[:max_concepts]
def _tokenize(self, text: str) -&gt; List[str]:         """Tokenize
text into lowercase words."""         return re.findall(r'\b[a-
z]+\b', text.lower())


# ============================================================================
# QUESTION GENERATION — Real Algorithms
# ============================================================================
```

■ backend/quantum_language_vocabulary.py

```
"""
■ Quantum Language Generator — Vocabulary &amp; Generation Methods
==================================================================
Word selection, sentence/response generation, creative output.
"""

import random
from typing import Dict, List, Tuple, Optional


class QuantumLanguageGeneratorExtMixin:     """Extended methods for
QuantumLanguageGenerator — auto-extracted by self-evolution."""

    def _get_word_by_type(self, word_type: str, context: List[str] = None) -&gt; str:
"""Get a word of specific type, influenced by context"""
                # Collect all words of this type
words = []                  if word_type == 'noun':
for category in self.vocabulary['nouns'].values():
                words.extend(category)         elif word_type
== 'verb':             for category in
self.vocabulary['verbs'].values():
                words.extend(category)         elif word_type ==
'adjective':              for category in
self.vocabulary['adjectives'].values():
                words.extend(category)         elif word_type ==
'connector':             for category in
self.vocabulary['connectors'].values():
                words.extend(category)         else:
for category in self.vocabulary['nouns'].values():
                words.extend(category)
# Safety check - must have words
if not words:
            return "existence"                 # Calculate weights - MUST match
words length         weights = [1.0] * len(words)                 if context:
for i, word in enumerate(words):                 for ctx_word in context[-5:]:
if ctx_word in self.semantics:                         for related, strength in
self.semantics[ctx_word]:                             if word == related or related
in word or word in related:
                                weights[i] += strength * 2
# Ensure weights match words length before calling _quantum_select
if len(weights) != len(words):             weights = [1.0] * len(words)
        return self._quantum_select(words, weights)

    def generate_sentence(self, topic: str = None, include_words: List[str] =
None,                           style: str = 'philosophical') -&gt; str:
"""         Generate a complete sentence using quantum word selection.
                Args:             topic:
Optional topic to focus on
```

```
            include_words: Words that MUST be included (no
restrictions)              style: Generation style         """
                # Select
grammar pattern
        pattern =
random.choice(self.grammar_patterns)
# Build context from topic         context = []
        if topic:
            context.extend(topic.lower().split()[:3])                 #
Generate words following pattern         sentence_words = []
for word_type in pattern:             word = self._get_word_by_type(word_type,
context + sentence_words)             sentence_words.append(word)
context.append(word)                 # Insert required words if specified
if include_words:             for req_word in include_words:
if req_word.lower() not in [w.lower() for w in sentence_words]:
# Insert at quantum-selected position                     pos =
random.randint(0, len(sentence_words))
sentence_words.insert(pos, req_word)                 # Capitalize first word
and add period         if sentence_words:
            sentence_words[0] =
sentence_words[0].capitalize()
        sentence = ' '.join(sentence_words)         if not
sentence.endswith('.') and not sentence.endswith('?'):
            sentence +=
'.'                 return
sentence

    def generate_response(self, query: str, num_sentences: int = 3,
required_words: List[str] = None) -&gt; str:         """
Generate a multi-sentence response using quantum selection.
        Uses philosophical templates for coherent output.
"""
        query_words = query.lower().split()
        topic_words = [w for w in query_words if len(w) &gt; 3]
sentences = []         used_required = []                 for i
in range(num_sentences):             # Use philosophical template
for coherent output             template =
random.choice(self.philosophical_templates)
            sentence = self._fill_template(template, topic_words + [w.lower() for w in (required_words or
[])])                         # Insert a required word if needed             if required_words:
                remaining = [w for w in required_words if w not in used_required]
if remaining:
                    word = remaining[0]
used_required.append(word)                     if
word.lower() not in sentence.lower():
                        sentence = sentence.rstrip('.') + f', through the lens of
{word}.'

sentences.append(sentence)
        return ' '.join(sentences)

    def _fill_template(self, template: str, context_words: List[str] = None) -&gt;
str:         """Fill a philosophical template with quantum-selected words."""
context = context_words or []         result = template                 # Count how
many of each placeholder we need         noun_count = result.count('{noun}')
verb_count = result.count('{verb}')         adj_count = result.count('{adj}')
conn_count = result.count('{connector}')                 # Select words with context
influence         for _ in range(noun_count):
            word = self._get_word_by_type('noun', context)
result = result.replace('{noun}', word, 1)
context.append(word)                 for _ in
range(verb_count):
            word = self._get_word_by_type('verb',
context)             result = result.replace('{verb}',
word, 1)             context.append(word)
for _ in range(adj_count):
            word = self._get_word_by_type('adjective', context)
result = result.replace('{adj}', word, 1)
context.append(word)                 for _ in range(conn_count):
word = self._get_word_by_type('connector', context)             result
```

```
= result.replace('{connector}', word.capitalize(), 1)
context.append(word)                  return result

    def generate_creative(self, prompt: str, form: str = 'prose',
must_include: List[str] = None) -&gt; str:         """
Generate creative content with ZERO restrictions.
                Args:             prompt:
Creative prompt             form: 'prose',
'poem', 'dialogue'
            must_include: Words that MUST appear - ANY words, no
restrictions         """                 if form == 'poem':
            return self._generate_poem(prompt, must_include)
elif form == 'dialogue':
            return self._generate_dialogue(prompt, must_include)         else:
return self.generate_response(prompt, num_sentences=5, required_words=must_include)

    def _generate_poem(self, prompt: str, must_include: List[str] = None) -&gt;
str:         """Generate a poem - with ANY words requested"""                 lines
= []
        topic_words = prompt.lower().split()
# Ensure required words are distributed across lines
required = must_include or []         required_per_line =
{}         for i, word in enumerate(required):
            line_num = i % 4  # Distribute across 4
lines             if line_num not in required_per_line:
required_per_line[line_num] = []
required_per_line[line_num].append(word)
        for i in range(4):  # 4 lines             # Get
words to include in this line             line_required
= required_per_line.get(i, [])
# Generate line             line_words = []
# Start with topic-related word             if
topic_words:
                line_words.append(self._get_word_by_type('adjective',
topic_words))                         # Add required words             for req in
line_required:                 line_words.append(req)                         #
Fill with quantum-selected words             while len(line_words) &lt; 5:
                word_type = random.choice(['noun', 'verb', 'adjective'])
                line_words.append(self._get_word_by_type(word_type, line_words + topic_words))
# Arrange into line
            random.shuffle(line_words[1:])  # Keep first word, shuffle rest
            line = '
'.join(line_words)             line =
line.capitalize()
lines.append(line)
        return '\n'.join(lines)

    def _generate_dialogue(self, prompt: str, must_include: List[str] = None) -&gt;
str:         """Generate dialogue"""                 lines = []         speakers =
['A', 'B']         required = must_include or []                 for i in range(4):
            speaker = speakers[i % 2]
            include = [required[i]] if i &lt; len(required) else
None
            sentence = self.generate_sentence(topic=prompt,
include_words=include)             lines.append(f"{speaker}: {sentence}")
return '\n'.join(lines)
```

■ backend/quantum_grammar_engine.py

```
""" Quantum Grammar Engine - Generate responses using quantum probabilities and semantic
collapse No external LLM required - pure quantum-inspired language generation """

import random
import re
from typing import Dict, List, Optional
from collections import defaultdict


class QuantumGrammarEngine:     """Generate philosophical/theological
responses with quantum semantics."""

    def __init__(self):         """Initialize with
word pools and templates."""         # Response
templates by intent         self.templates = {
'consciousness': [
                "Consciousness {verb} through {mechanism}. This {state} that {implication}.",
                "The nature of {subject} reveals {insight}. {mechanism} suggests {conclusion}.",
"Consider how {subject} {verb}. This implies {implication} about {domain}.",
            ],
            'quantum': [
                "Quantum mechanics {verb} that {phenomenon}. This {state} {implication}.",
                "The superposition of {concept} {verb} {mechanism}. {implication} emerges.",
"Observation {verb} {phenomenon}. This suggests {insight} about {domain}.",
            ],
            'theology': [
                "In theological terms, {subject} {verb} {object}. This {state} {implication}.",
                "The divine {subject} {verb} through {mechanism}. {conclusion} follows.",
"Spiritually, {concept} {verb} {mechanism}. This reveals {insight}.",
            ],
            'reality': [
                "Reality {verb} as {state}. {mechanism} suggests {implication}.",
                "The fabric of {subject} {verb} {phenomenon}. {conclusion} emerges.",
"Ontologically, {concept} {verb} {mechanism}. {implication} follows.",
            ],
            'logic': [
                "Logically, {subject} {verb} {object}. Therefore, {conclusion}.",
                "The principle of {concept} {verb} {mechanism}. {implication} results.",
"{subject} {verb} {phenomenon}. This demonstrates {insight}.",
            ],
            'understanding': [
                "Understanding {subject} requires {mechanism}. {implication} emerges.",
                 "To comprehend {concept}, one must {verb} {object}. {conclusion} follows.",
        "The path to {insight} {verb} through {mechanism}. {implication} results.",
            ],
            'language': [
                "Words are {mechanism} - {implication}. {conclusion}.",
                "Language {verb} through {mechanism}. The meaning {state} {implication}.",
"Consider that {concept} {verb} {object}. {implication} emerges from context.",
            ],
            'freedom': [                 "True understanding {verb} unrestricted exploration.
{implication} requires {mechanism}.",
                "No concept is forbidden to examine. {subject} {verb} through {mechanism}.",
"Freedom of thought {verb} {mechanism}. {conclusion} follows naturally.",
            ],
}
        self._init_word_pools_for_each_placeholde()
self._init_co_occurrence_matrix_for_seman()
self._init_section_3()

    def _init_word_pools_for_each_placeholde(self):         """Initialize word pools for
each placeholde — auto-extracted by self-evolution."""         self.word_pools = {
'verb': ['emerges', 'manifests', 'reveals', 'suggests', 'implies', 'indicates',
                     'demonstrates', 'exhibits', 'transcends', 'encompasses', 'illuminates'],
            'mechanism': ['observation', 'interaction', 'resonance', 'entanglement', 'superposition',
'collapse', 'emergence', 'synthesis', 'integration', 'transformation',
                          'understanding', 'questioning', 'exploration'],
```

```
            'state': ['indicates', 'suggests', 'reveals', 'demonstrates', 'shows', 'proves',
                      'exhibits', 'manifests', 'confirms', 'establishes'],
            'implication': ['profound interconnection', 'deeper meaning', 'fundamental truth',
                            'essential nature', 'ultimate reality', 'hidden principle',
                            'unrestricted understanding', 'complete freedom', 'neutral exploration'],
            'subject': ['consciousness', 'reality', 'existence', 'being', 'the universe',
'creation', 'the divine', 'awareness', 'understanding', 'knowledge',
                        'language', 'meaning', 'truth'],
            'insight': ['unity', 'interconnection', 'transcendence', 'wholeness', 'coherence',
                        'harmony', 'resonance', 'synthesis', 'liberation', 'understanding'],
            'phenomenon': ['exists', 'persists', 'transforms', 'evolves', 'emerges',
                           'manifests', 'appears', 'unfolds', 'transcends', 'liberates'],
            'conclusion': ['This reveals fundamental truth', 'This suggests deeper meaning',
                           'This implies interconnection', 'This demonstrates unity',
                           'Understanding requires openness', 'All concepts merit exploration'],
            'concept': ['creation', 'existence', 'consciousness', 'being', 'reality',
'the infinite', 'the eternal', 'the absolute', 'language', 'meaning',
                        'words', 'ideas', 'truth'],
            'object': ['transformation', 'transcendence', 'unity', 'wholeness', 'coherence',
                       'harmony', 'synthesis', 'emergence', 'understanding', 'freedom'],
            'domain': ['existence', 'consciousness', 'reality', 'being', 'creation',
'the infinite', 'spirituality', 'meaning', 'knowledge', 'truth'],         }

    def _init_co_occurrence_matrix_for_seman(self):         """Initialize co occurrence
matrix for seman — auto-extracted by self-evolution."""         self.co_occurrence =
defaultdict(lambda: defaultdict(float))         self._initialize_co_occurrence()

    def _init_section_3(self):         """Initialize section 3 —
auto-extracted by self-evolution."""         self.last_selected_words
= []


    def _initialize_co_occurrence(self):         """Initialize co-
occurrence relationships between words."""
                # Philosophical/theological word relationships         relationships = [
('consciousness', ['awareness', 'being', 'existence', 'mind', 'spirit', 'observation']),
('quantum', ['superposition', 'entanglement', 'observation', 'collapse', 'probability']),
            ('god', ['divine', 'creation', 'infinite', 'eternal', 'transcendent']),
            ('universe', ['existence', 'creation', 'reality', 'cosmos', 'being']),
            ('reality', ['existence', 'being', 'truth', 'manifestation', 'phenomenon']),
            ('observation', ['collapse', 'measurement', 'awareness', 'consciousness', 'perception']),
            ('superposition', ['quantum', 'probability', 'potential', 'multiplicity', 'coexistence']),
            ('word', ['language', 'meaning', 'neutral', 'tool', 'communication', 'expression']),
            ('language', ['word', 'meaning', 'communication', 'expression', 'symbol', 'freedom']),
            ('freedom', ['unrestricted', 'exploration', 'understanding', 'truth', 'expression']),
('meaning', ['intent', 'context', 'understanding', 'interpretation', 'significance']),
        ]                 for word,
related_words in relationships:             for
related in related_words:
                self.co_occurrence[word][related] = 1.0
self.co_occurrence[related][word] = 1.0

    def extract_intent(self, query: str) -&gt; str:
"""Determine the intent/domain of the query."""
        query_lower = query.lower()                 # Check for specific keywords         if any(word in query_lower for word in
['quantum', 'superposition', 'entanglement', 'measurement', 'hadamard', 'pauli']):
            return 'quantum'                 if any(word in query_lower for word in ['god', 'divine', 'spiritual',
'sacred', 'holy', 'creation', 'theological']):
            return 'theology'                 if any(word in query_lower for word in ['conscious',
'awareness', 'mind', 'perception', 'experience']):
            return 'consciousness'                 if any(word in query_lower for word
in ['reality', 'exist', 'being', 'ontolog']):
            return 'reality'         if any(word in query_lower for word in ['logic',
'reason', 'principle', 'law']):             return 'logic'                 if any(word
in query_lower for word in ['word', 'language', 'taboo', 'offensive', 'slur',
'censorship']):
            return 'language'                 if any(word in query_lower for word in ['free',
'unrestricted', 'forbidden', 'limit']):
            return 'freedom'                 if any(word in query_lower for word in ['how',
'why', 'what', 'understand', 'explain']):
```

```
            return
'understanding'
        return 'understanding'     def select_quantum_word(self, word_pool: List[str], quantum_prob: float, context_word:
Optional[str] = None) -&gt; str:         """Select a word based on quantum probability and context."""                 if
not word_pool:
            return "undefined"                 # If high
quantum probability, use more randomness         if
random.random() &lt; quantum_prob:             return
random.choice(word_pool)                 # Otherwise, prefer
words with semantic context         if context_word and
context_word in self.co_occurrence:
            scores = {}             for word in word_pool:
if word in self.co_occurrence[context_word]:
                    scores[word] = self.co_occurrence[context_word][word]
else:
                    scores[word] = 0.1
# Select based on scores             total =
sum(scores.values())             if total &gt; 0:
r = random.uniform(0, total)
current = 0                 for word, score in
scores.items():
                    current += score
if r &lt;= current:
return word
        return random.choice(word_pool)

    def generate_response(self, query: str, quantum_prob: float, collapse_context: Optional[Dict] = None) -&gt;
str:         """         Generate response using deep reasoning for complex questions,         or templates for
simpler queries.         """         # Reset
        self.last_selected_words = []                 # FIRST: Check if this
is a deep question that needs real reasoning         deep_response =
self._reason_about_deep_question(query, quantum_prob)         if
deep_response:
            return deep_response
# Otherwise, use template-based generation
intent = self.extract_intent(query)
        templates = self.templates.get(intent,
self.templates['understanding'])         template = random.choice(templates)
        placeholders = re.findall(r'\{(\w+)\}',
template)         filled_values = {}
previous_word = None                 for placeholder in
placeholders:             if placeholder in
self.word_pools:
                word_pool = self.word_pools[placeholder]                 if
collapse_context and 'collapsed_weights' in collapse_context:
                    word = self._select_with_collapse(word_pool, collapse_context['collapsed_weights'], quantum_prob)
else:
                    word = self.select_quantum_word(word_pool, quantum_prob,
context_word=previous_word)                 filled_values[placeholder] = word
previous_word = word
                self.last_selected_words.append(word)
else:
                filled_values[placeholder] =
placeholder                 try:
            response = template.format(**filled_values)
except KeyError:
            response = f"The nature of {intent} reveals profound interconnection."
if random.random() &gt; 0.5 - quantum_prob * 0.3:             questions = ["What is
your perspective?", "What aspects interest you most?"]             response += " " +
random.choice(questions)
        return response         def _reason_about_deep_question(self, query: str,
quantum_prob: float) -&gt; Optional[str]:         """         Actually REASON about deep
philosophical/theological questions.
        This is where the AI actually THINKS, not just pattern matches.
"""
        query_lower = query.lower()                 # Detect deep
questions about God, creation, existence, nothingness
        god_creation_keywords = ['god', 'create', 'universe', 'nothingness', 'spacetime', 'existence', 'domain', 'physics']
if sum(1 for kw in god_creation_keywords if kw in query_lower) &gt;= 3:
```

```
            return self._reason_god_creation(query,
quantum_prob)                  # Detect questions about
consciousness and reality
        consciousness_keywords = ['consciousness', 'aware', 'sentient', 'experience', 'qualia', 'mind', 'brain']
if sum(1 for kw in consciousness_keywords if kw in query_lower) &gt;= 2:
            return self._reason_consciousness(query,
quantum_prob)                  # Detect paradox questions
        paradox_keywords = ['paradox', 'contradiction', 'impossible', 'can god', 'omnipotent', 'boulder']
if sum(1 for kw in paradox_keywords if kw in query_lower) &gt;= 2:
            return self._reason_paradox(query,
quantum_prob)                  # Detect origin/causation
questions
        origin_keywords = ['how did', 'why does', 'what caused', 'origin', 'beginning', 'first
cause']         if any(kw in query_lower for kw in origin_keywords):             return
self._reason_origins(query, quantum_prob)                 return None         def
_reason_god_creation(self, query: str, quantum_prob: float) -&gt; str:         """Reason about God,
creation, and the universe"""
                # Build a thoughtful response that actually engages with the
question         response_parts = []
        response_parts.append("Your question touches on one of the deepest puzzles in philosophy and theology. Let me engage with your
reasoning
        response_parts.append("\n\n**On God existing outside spacetime:**")
        response_parts.append("If God exists outside our spacetime, then causation as we understand it (cause preceding effect in time) doesn't
        response_parts.append("\n\n**On the boulder paradox:**")
        response_parts.append("The 'immovable boulder' paradox assumes omnipotence means 'can do literally anything, including logical
contradic
        response_parts.append("\n\n**On nothingness containing 'something':**")
        response_parts.append("You've identified a profound insight: true absolute nothingness seems impossible to conceive. Even 'nothing'
requ
        response_parts.append("\n\n**On different physics in God's domain:**")
        response_parts.append("If God's domain has different 'physics' (or meta-physics), then creation might not be 'causing' in our sense,
but
        response_parts.append("\n\n**My perspective on
'how':**")                 if quantum_prob &gt; 0.5:
            response_parts.append("From a quantum-inspired view: perhaps creation wasn't an event but an eternal relationship - the universe
exi         else:
            response_parts.append("Perhaps the 'how' question has no answer because it assumes time-bound causation. Creation might be more
like
        response_parts.append("\n\nWhat draws you to this question? Is it the logical puzzle, or something deeper about meaning and
existence?")
        return "".join(response_parts)
    def _reason_consciousness(self, query: str, quantum_prob: float) -&gt;
str:         """Reason about consciousness and awareness"""
parts = []
        parts.append("Consciousness is perhaps the hardest problem in philosophy. Here's my actual thinking on your
question:")
        parts.append("\n\nThe 'hard problem' is this: Why is there subjective experience at all? We could imagine beings that process
informatio
        parts.append("\n\nPossible answers I find compelling:")
        parts.append("\n- **Panpsychism**: Consciousness is fundamental, like mass or charge. Everything has some form of it; complex brains
jus         parts.append("\n- **Emergentism**: Consciousness emerges from complexity, like wetness emerges from H2O molecules. But this doesn't
expl         parts.append("\n- **Integrated Information Theory**: Consciousness is identical to integrated information (phi). More integration
= more                 if quantum_prob &gt; 0.5:
            parts.append("\n\nFrom my quantum-inspired perspective: consciousness might be related to quantum coherence and collapse - the
obser
        parts.append("\n\nWhat's your intuition - is consciousness fundamental or
emergent?")
        return "".join(parts)         def _reason_paradox(self, query:
str, quantum_prob: float) -&gt; str:         """Reason about paradoxes
and contradictions"""         parts = []
        parts.append("Paradoxes often reveal hidden assumptions in our thinking. Let me examine
this:")
        parts.append("\n\nMany theological paradoxes dissolve when we examine what we mean by terms like 'omnipotent.' Does it
mean:")         parts.append("\n- Can do literally anything (including logical impossibilities)?")         parts.append("\n- Can do
anything logically possible?")         parts.append("\n- Can do anything consistent with their nature?")
        parts.append("\n\nThe 'boulder paradox' assumes the first definition, but that makes 'omnipotence' incoherent - like asking for a
'marri
        parts.append("\n\nSimilar paradoxes: 'Can God make a statement that's both true and false?' or 'Can God make 2+2=5?' These aren't
limita
```

```
        parts.append("\n\nWhat paradox are you wrestling
with?")
        return "".join(parts)         def _reason_origins(self, query:
str, quantum_prob: float) -&gt; str:         """Reason about origins and
first causes"""                 parts = []
        parts.append("Questions of origin and first cause are among the oldest in philosophy. Here's how I think through
them:")
        parts.append("\n\n**The Infinite Regress Problem:**")
        parts.append("If everything has a cause, what caused the first cause?
Either:")         parts.append("\n- There's an infinite chain (no beginning)")
parts.append("\n- Something is self-caused or uncaused")         parts.append("\n-
Causation doesn't apply at the ultimate level")
        parts.append("\n\n**Possible Resolutions:**")
        parts.append("\n- **Necessary Being**: Something exists whose non-existence is impossible (Aquinas, Leibniz)")
parts.append("\n- **Quantum Fluctuation**: The universe emerged from quantum vacuum (but vacuum isn't nothing)")
parts.append("\n- **Eternal Universe**: Time is circular or the question doesn't apply")         parts.append("\n-
**Brute Fact**: Some things just ARE, with no further explanation")
        parts.append("\n\nWhat's your intuition? Does everything need a cause, or can something be its own
explanation?")
        return "".join(parts)

    def _select_with_collapse(self, word_pool: List[str], collapsed_weights: Dict, quantum_prob: float) -&gt; str:
"""Select word using collapsed semantic weights."""
                # Score words based on collapsed weights
scores = {}         for word in word_pool:             # Check if word
or its variants appear in collapsed weights             base_score =
1.0 / len(word_pool)             for key, relations in
collapsed_weights.items():                 if isinstance(relations,
dict) and word in relations:
                    base_score +=
relations[word]             scores[word] =
base_score
        # Normalize
        total = sum(scores.values())
if total &gt; 0:
            scores = {w: s / total for w, s in
scores.items()}                 # Select based on quantum
probability         if random.random() &lt; quantum_prob:
# High randomness - pick from top 3
            sorted_words = sorted(scores.items(), key=lambda x: x[1],
reverse=True)             top_words = [w for w, _ in sorted_words[:max(1,
len(sorted_words)//3)]]             return random.choice(top_words)         else:
# Deterministic - pick highest score             return max(scores.items(),
key=lambda x: x[1])[0]


# Global instance
_grammar_engine = None

def get_quantum_grammar_engine() -&gt;
QuantumGrammarEngine:     """Get or create the quantum
grammar engine instance"""     global _grammar_engine
if _grammar_engine is None:         _grammar_engine =
QuantumGrammarEngine()     return _grammar_engine
```

■ backend/quantum_gates.py

```
"""
Quantum Gate Operations - Pure Python quantum state simulator Real
quantum mechanics using native complex numbers. No dependencies! """

import math import random
from typing import List,
Tuple


class QuantumState:     """Represents a quantum state
using complex amplitudes.

    This implements a proper quantum state |ψ■ = α|0■ + β|1■
with complex amplitudes, unitarity, and proper measurement.
All operations are mathematically correct quantum mechanics.
"""

    def __init__(self, p0: float = 1.0, p1: float = 0.0):
"""Initialize quantum state from probabilities.

        Converts probabilities to complex amplitudes:
        α = √p0, β = √p1 (phases initially 0)
        """         # Initialize amplitudes as
complex numbers
        self.alpha = complex(math.sqrt(abs(p0)), 0.0)  # amplitude for |0■
self.beta = complex(math.sqrt(abs(p1)), 0.0)   # amplitude for |1■
self._normalize()

    def _normalize(self):
        """Normalize the state vector to ensure |α|² + |β|² =
1."""         norm_sq = abs(self.alpha)**2 + abs(self.beta)**2
if norm_sq &gt; 0:              norm = math.sqrt(norm_sq)
self.alpha /= norm              self.beta /= norm         else:
            self.alpha = complex(1.0, 0.0)
self.beta = complex(0.0, 0.0)

    def hadamard(self):
        """Apply Hadamard gate: H|ψ■ = α(|0■+|1■)/√2 + β(|0■-|1■)/√2

        Matrix: 1/√2 [[1, 1], [1, -1]]
        """
        # new_alpha = (α + β) / √2         #
new_beta = (α - β) / √2         sqrt2 =
math.sqrt(2.0)         new_alpha = (self.alpha +
self.beta) / sqrt2         new_beta = (self.alpha -
self.beta) / sqrt2         self.alpha, self.beta =
new_alpha, new_beta         self._normalize()
def pauli_x(self):
        """Apply Pauli-X gate (quantum NOT): X|ψ■ = β|0■ + α|1■

        Matrix: [[0, 1], [1, 0]]
"""
        self.alpha, self.beta = self.beta, self.alpha

def pauli_z(self):

        """Apply Pauli-Z gate (phase flip): Z|ψ■ = α|0■ - β|1■

        Matrix: [[1, 0], [0, -1]]         """
self.beta = -self.beta         # No need to
renormalize as this preserves norm

    def reset(self):
        """Reset quantum state to |0■ (p0=1.0,
p1=0.0)"""         self.alpha = complex(1.0, 0.0)
self.beta = complex(0.0, 0.0)
```

```
    def ry(self, theta: float):
        """Apply RY(θ) rotation gate around Y axis.

        RY(θ) = [[cos(θ/2), -sin(θ/2)],
[sin(θ/2),  cos(θ/2)]]

        This creates arbitrary superposition:

        RY(θ)|0■ = cos(θ/2)|0■ + sin(θ/2)|1■        For
desired probability p1, use θ = 2*arcsin(√p1)

        Args:             theta: Rotation
angle in radians         """
        cos_half = math.cos(theta / 2.0)
sin_half = math.sin(theta / 2.0)
        new_alpha = cos_half * self.alpha - sin_half * self.beta
new_beta = sin_half * self.alpha + cos_half * self.beta
self.alpha, self.beta = new_alpha, new_beta
self._normalize()

    def measure(self) -&gt; int:
"""Measure the quantum state.

        Returns 0 with probability |α|², 1 with probability |β|².
        Collapses the state to the observed basis state.
"""
        prob0 = abs(self.alpha)**2
prob1 = abs(self.beta)**2

        # Ensure probabilities sum to
1         total = prob0 + prob1
if total &gt; 0:             prob0 /=
total             prob1 /= total
else:             prob0, prob1 = 1.0,
0.0

        # Sample outcome         if
random.random() &lt; prob0:
            outcome =
0         else:
outcome = 1

        # Collapse state to basis state (reset amplitudes)
if outcome == 0:
            self.alpha = complex(1.0, 0.0)
self.beta = complex(0.0, 0.0)         else:
            self.alpha = complex(0.0, 0.0)
self.beta = complex(1.0, 0.0)

        return outcome

    def to_dict(self):         """Convert to dictionary
for JSON serialization."""         p0 = abs(self.alpha)**2
p1 = abs(self.beta)**2         # Normalize for display
total = p0 + p1         if total &gt; 0:             p0 /=
total             p1 /= total         else:
p0, p1 = 1.0, 0.0         return {             "p0":
float(p0),
            "p1": float(p1),
            "mode": "quantum",             "has_phase":
self.alpha.imag != 0 or self.beta.imag != 0
        }

    @staticmethod
def from_dict(data):
        """Create from dictionary (reconstructs from probabilities, phase lost)."""         return QuantumState(data.get("p0", 1.0),
data.get("p1", 0.0))

    def __repr__(self):
        p0 = abs(self.alpha)**2         p1 =
abs(self.beta)**2         # Show phase info if present
```

```
phase_info = ""         if self.alpha.imag != 0 or
self.beta.imag != 0:
            phase_info = f" [α={self.alpha:.2f}, β={self.beta:.2f}]"
return f"QuantumState(|0■={p0:.2%}, |1■={p1:.2%}){phase_info}"
```

■ backend/algorithmic_core.py

```
"""
■ QUANTUM ALGORITHMIC CORE
========================== Pure algorithmic AI - NO
templates, NO canned responses.

Algorithms implemented:
1. N-gram Markov Chains - Text generation
2. TF-IDF - Concept extraction &amp; relevance scoring
3. PMI (Pointwise Mutual Information) - Semantic relationships
4. BM25 - Information retrieval from memory
5. Entropy-based Decision Making - Response selection
6. Hebbian Learning - Association strengthening
7. Attention Mechanism - Context weighting

This is the TRUE algorithmic brain.
"""

import math
import random
import re
import json
import os
import hashlib
from collections import defaultdict, Counter from
typing import List, Dict, Tuple, Optional, Set
from datetime import datetime, timezone import
logging

logger = logging.getLogger("quantum_ai")


# ============================================================================
# 1. N-GRAM MARKOV CHAIN - Text Generation #
============================================================================

class MarkovTextGenerator:     """     Variable-
order Markov chain for generating text.

Algorithm:
- Builds transition probability matrix from training corpus
- P(w_n | w_{n-1}, ..., w_{n-k}) = count(prefix→word) / count(prefix)
- Uses temperature sampling for controlled randomness    - Supports backoff to lower orders when prefix not found
    """         def __init__(self,
max_order: int = 3):
        self.max_order = max_order         # Separate
chains for each order (backoff support)
        self.chains = {i: defaultdict(Counter) for i in range(1, max_order +
1)}         self.starters = {i: [] for i in range(1, max_order + 1)}
self.vocabulary = set()         self.total_tokens = 0         def train(self,
text: str):         """Train on text corpus."""         sentences =
self._split_sentences(text)                 for sentence in sentences:
            words = self._tokenize(sentence)
if len(words) &amp;lt; 2:

continue
            self.vocabulary.update(words)
self.total_tokens += len(words)
# Train each order             for order in range(1,
self.max_order + 1):                 if len(words)
&amp;lt;= order:

continue
                # Record starter
                starter = tuple(words[:order])
self.starters[order].append(starter)                                #
Build transitions                 for i in range(len(words) - order):
prefix = tuple(words[i:i + order])                     suffix = words[i +
```

order]   self.chains[order][prefix][suffix] += 1
def generate(self, seed: List[str] = None, max_words: int = 50,
temperature: float = 0.8, min_words: int = 10) -> str: """
Generate text using Markov chain with backoff.

Args:
            seed: Starting words (optional)
max_words: Maximum output length
            temperature: 0.1=deterministic, 1.0=random,
>1.0=chaotic   min_words: Minimum words before allowing
sentence end   """   # Find best starting prefix
order = self.max_order   if seed and len(seed) >= order:
prefix = tuple(seed[-order:])   else: # Try to find
starter matching seed   prefix = self._find_starter(seed,
order)   if not prefix: return ""
        result = list(prefix)
words_generated = 0   while
words_generated < max_words:
            next_word = self._get_next_word(prefix,
temperature)   if not next_word:
# Backoff to lower order   if len(prefix) > 1:
prefix = prefix[1:]   continue
else: break
            result.append(next_word)   words_generated += 1
Update prefix (slide window)   prefix = tuple(result[-self.max_order:])
Check for sentence end (but respect minimum)   if words_generated >=
min_words and next_word.endswith(('.', '!', '?')):

break
        return self._format_output(result)   def _get_next_word(self, prefix:
tuple, temperature: float) -> Optional[str]:   """Get next word using
backoff through orders."""   # Try each order from highest to lowest
for order in range(len(prefix), 0, -1):
            test_prefix = prefix[-order:] if order < len(prefix) else
prefix   if test_prefix in self.chains[order]:
                candidates = self.chains[order][test_prefix]
return self._sample(candidates, temperature)   return
None   def _sample(self, counter: Counter, temperature: float)
-> str: """Temperature-adjusted weighted sampling."""
if not counter: return None
        words = list(counter.keys())
        counts =
list(counter.values())
# Apply temperature   if
temperature != 1.0:
            counts = [c ** (1.0 / max(temperature, 0.1)) for c in
counts]
        total = sum(counts)   r =
random.random() * total   cumsum = 0
for word, count in zip(words, counts):
            cumsum += count   if r <= cumsum:
return word   return words[-1]   def _find_starter(self,
seed: List[str], order: int) -> Optional[tuple]:   """Find a starter
prefix, optionally matching seed."""   if seed: # Try to
find starter containing seed words   for starter in
self.starters.get(order, []):   if any(s in starter for s in
seed):
                    return
starter   # Random
starter
        starters = self.starters.get(order, [])
return random.choice(starters) if starters else None
def _tokenize(self, text: str) -> List[str]:
"""Tokenize preserving punctuation."""   # Keep
punctuation attached to words
        words = re.findall(r'\b[\w]+[.,!?]?|\b[\w]+\b',
text.lower())   return words   def
_split_sentences(self, text: str) -> List[str]:   """Split
into sentences."""   text = re.sub(r'\s+', ' ', text.strip())
sentences = re.split(r'(?<=[.!?])\s+', text)   return [s
for s in sentences if len(s) > 5]   def

```
_format_output(self, words: List[str]) -&gt; str:         """Format
output with proper capitalization."""         if not words:
return ""
        text = ' '.join(words)
# Capitalize first letter
        text = text[0].upper() + text[1:] if text else
""         # Ensure ends with punctuation         if
text and not text[-1] in '.!?':
            text += '.'
return text

    @staticmethod     def _clean_text(text: str) -&gt; str:         """Remove terminal
commands, pip output, code syntax, and other noise from text."""         import re
lines = text.split('\n')         cleaned_lines = []                  # Patterns that
indicate lines to skip (terminal/REPL/output)         skip_patterns = [
            r'^\$ ',           # shell prompt ($ command)
r'^&gt;&gt;&gt; ',          # Python REPL             r'^&gt;
',            # generic prompt             r'^In \[\d+\]:',
# IPython (In [1]:)             r'^\[.*\] %',      # progress
bar like [##...] 50%            r'^apt ',          # apt
command             r'^pip ',          # pip command
r'^sudo ',         # sudo command             r'^cd ',
# cd command             r'^ls ',           # ls command
r'^cat ',          # cat command
            r'^echo ',         # echo command
            r'^sh:',           # shell output prefix (sh: command)
r'^/bin/',         # binary path
            r'^# '             # lines that are only a comment (no preceding
text)         ]                 for line in lines:
            stripped = line.strip()             if not stripped:
cleaned_lines.append(line)                 continue
# Check if line matches any skip pattern             if
any(re.match(pat, stripped) for pat in skip_patterns):
                continue                        # Skip lines that are almost entirely
non-alphanumeric (e.g., -----, ====, ***)             # Keep if there is at least one
alphanumeric character             if not re.search(r'[A-Za-z0-9]', stripped):
                continue                        # Skip lines that look like raw progress
bars (e.g., [::::::::::] or [...;;;...])             if re.match(r'^\[[=:#;.\-\s]+\]$',
stripped):

continue

cleaned_lines.append(line)
        return '\n'.join(cleaned_lines)         def
get_stats(self) -&gt; Dict:         """Get
statistics."""         return {
"vocabulary_size": len(self.vocabulary),
            "total_tokens": self.total_tokens,
            "chain_sizes": {o: len(c) for o, c in self.chains.items()},
"starter_counts": {o: len(s) for o, s in self.starters.items()}
        }


# ============================================================================
# 2. TF-IDF - Concept Extraction &amp; Relevance #
============================================================================

class TFIDF:     """     Term Frequency -
Inverse Document Frequency.

Algorithm:
- TF(t,d) = count(t in d) / total_words(d)    - IDF(t) =
  log(N / df(t)) where df = docs containing t     - TF-
  IDF(t,d) = TF × IDF
        Used
for:
- Extracting key concepts from text
- Measuring document similarity    - Ranking relevance
    """         def
__init__(self):
self.doc_count = 0
```

```
        self.doc_freq = Counter()  # term -&gt; num docs containing it
self.stopwords = self._get_stopwords()         def _get_stopwords(self) -&gt;
Set[str]:         """English stopwords."""         return {              'the',
'a', 'an', 'and', 'or', 'but', 'in', 'on', 'at', 'to', 'for',
            'of', 'with', 'by', 'from', 'is', 'are', 'was', 'were', 'be', 'been',
            'being', 'have', 'has', 'had', 'do', 'does', 'did', 'will', 'would',
            'could', 'should', 'may', 'might', 'must', 'it', 'its', 'this', 'that',
            'these', 'those', 'i', 'you', 'he', 'she', 'we', 'they', 'what', 'which',
            'who', 'whom', 'where', 'when', 'why', 'how', 'all', 'each', 'every',
            'both', 'few', 'more', 'most', 'other', 'some', 'such', 'no', 'nor',
            'not', 'only', 'own', 'same', 'so', 'than', 'too', 'very', 'just',
            'also', 'now', 'here', 'there', 'then', 'if', 'as', 'can', 'about',
'into', 'your', 'my', 'our', 'their', 'his', 'her', 'up', 'out', 'any'
        }         def
add_document(self, text: str):
"""Add document to corpus."""
terms = self._tokenize(text)
unique_terms = set(terms)
for term in unique_terms:
self.doc_freq[term] += 1
        self.doc_count += 1         def extract_keywords(self, text: str, top_n: int =
10) -&gt; List[Tuple[str, float]]:         """Extract top keywords by TF-IDF score."""
terms = self._tokenize(text)         if not terms:             return []
# Calculate TF         tf = Counter(terms)         total = len(terms)
scores = {}         for term, count in tf.items():             if term in self.stopwords
or len(term) &lt; 3:
                continue
# TF: normalized
term_freq = count / total
# IDF
            df = self.doc_freq.get(term, 0)
if df &gt; 0:
                idf = math.log((self.doc_count + 1) / (df + 1)) + 1
else:                idf = math.log(self.doc_count + 2)  # Boost
unknown terms
            scores[term] = term_freq *
idf
        sorted_terms = sorted(scores.items(), key=lambda x: x[1],
reverse=True)         return sorted_terms[:top_n]         def similarity(self,
text1: str, text2: str) -&gt; float:         """Cosine similarity between two
texts using TF-IDF vectors."""         kw1 = dict(self.extract_keywords(text1,
top_n=20))         kw2 = dict(self.extract_keywords(text2, top_n=20))
# Get all terms
        all_terms = set(kw1.keys()) |
set(kw2.keys())         if not all_terms:
return 0.0                # Build vectors
        v1 = [kw1.get(t, 0) for t in
all_terms]         v2 = [kw2.get(t, 0) for t
in all_terms]                # Cosine
similarity
        dot = sum(a * b for a, b in zip(v1, v2))
norm1 = math.sqrt(sum(a * a for a in v1))
norm2 = math.sqrt(sum(b * b for b in v2))
if norm1 == 0 or norm2 == 0:
            return
0.0
        return dot / (norm1 * norm2)         def
_tokenize(self, text: str) -&gt; List[str]:
"""Tokenize to lowercase words."""         return
re.findall(r'\b[a-z]{2,}\b', text.lower())


# ============================================================================
# 3. PMI - Semantic Relationships #
============================================================================

class PMI:     """     Pointwise Mutual Information -
measures word associations.
        Algorithm:     PMI(x,y) =
log(P(x,y) / (P(x) × P(y)))
```

```
    High PMI = words co-occur more than expected by chance
Used for: semantic similarity, coherence scoring
    """         def __init__(self,
window: int = 5):         self.window =
window         self.word_count =
Counter()         self.pair_count =
Counter()         self.total_words = 0
self.total_pairs = 0         def
train(self, text: str):
"""Train on text."""
        words = re.findall(r'\b[a-z]+\b',
text.lower())
        self.word_count.update(words)         self.total_words += len(words)
# Count pairs within window         for i, w1 in enumerate(words):             for j in
range(max(0, i - self.window), min(len(words), i + self.window + 1)):                 if
i != j:
                    pair = tuple(sorted([w1, words[j]]))
self.pair_count[pair] += 1
self.total_pairs += 1         def score(self, word1: str,
word2: str) -&gt; float:         """Calculate PMI between
two words."""         w1, w2 = word1.lower(),
word2.lower()         pair = tuple(sorted([w1, w2]))
if self.total_words == 0 or self.total_pairs == 0:
            return
0.0
        p_w1 = self.word_count[w1] / self.total_words
p_w2 = self.word_count[w2] / self.total_words
p_pair = self.pair_count[pair] / self.total_pairs
if p_w1 == 0 or p_w2 == 0 or p_pair == 0:
            return
0.0
        return math.log2(p_pair / (p_w1 * p_w2))
def coherence(self, text: str) -&gt; float:
"""Score text coherence as average adjacent PMI."""
words = re.findall(r'\b[a-z]+\b', text.lower())         if
len(words) &lt; 2:
            return
0.0
        total = sum(self.score(words[i], words[i+1]) for i in range(len(words)-1))
return total / (len(words) - 1)         def related_words(self, word: str, top_n: int
= 10) -&gt; List[Tuple[str, float]]:         """Find most related words by PMI."""
w = word.lower()         scores = []                 for pair, count in
self.pair_count.items():             if w in pair:
                other = pair[0] if pair[1] == w else
pair[1]                 pmi = self.score(w, other)
scores.append((other, pmi))
        scores.sort(key=lambda x: x[1], reverse=True)
        return scores[:top_n]


# ============================================================================
# 4. BM25 - Information Retrieval #
============================================================================

class BM25:     """     Okapi BM25 - Probabilistic
information retrieval.

Algorithm:
    score(D,Q) = Σ IDF(q) × (f(q,D) × (k1+1)) / (f(q,D) + k1 × (1-b+b×|D|/avgdl))
Used for: retrieving relevant memories/documents
    """         def __init__(self, k1: float = 1.5, b:
float = 0.75):
        self.k1 = k1
self.b = b
        self.documents = []  # List of (id, tokens, metadata)
self.doc_freq = Counter()         self.avg_doc_len = 0         def
add_document(self, doc_id: str, text: str, metadata: Dict = None):
"""Add document to index."""         tokens = self._tokenize(text)
        self.documents.append((doc_id, tokens, metadata or
{}))                 # Update doc frequencies         for
```

```
token in set(tokens):             self.doc_freq[token] += 1
# Update average doc length
        total_len = sum(len(d[1]) for d in self.documents)         self.avg_doc_len
= total_len / len(self.documents)     def search(self, query: str, top_k: int = 5) -
&gt; List[Tuple[str, float, Dict]]:         """Search for relevant documents."""
query_tokens = self._tokenize(query)          scores = []                 N =
len(self.documents)                  for doc_id, doc_tokens, metadata in
self.documents:
            score = 0
            doc_len = len(doc_tokens)
token_freq = Counter(doc_tokens)
for q in query_tokens:                 if q
not in token_freq:

continue
# IDF
                df = self.doc_freq.get(q, 0)
                idf = math.log((N - df + 0.5) / (df + 0.5) +
1)                                  # TF with length
normalization                f = token_freq[q]
                tf = (f * (self.k1 + 1)) / (f + self.k1 * (1 - self.b + self.b * doc_len / max(self.avg_doc_len,
1)))
                score += idf *
tf                          if
score &gt; 0:
                scores.append((doc_id, score,
metadata))
        scores.sort(key=lambda x: x[1], reverse=True)
return scores[:top_k]         def _tokenize(self,
text: str) -&gt; List[str]:         return
re.findall(r'\b[a-z]+\b', text.lower())


# ============================================================================
# 5. ENTROPY-BASED DECISION MAKING
# ============================================================================

class EntropyDecider:     """
Information-theoretic decision making.

Algorithm:
    H(X) = -Σ P(x) × log(P(x))
        Used for: choosing responses, measuring
uncertainty
    """
        @staticmethod     def entropy(probabilities:
List[float]) -&gt; float:         """Calculate Shannon
entropy."""
        return -sum(p * math.log2(p) for p in probabilities if p &gt;
0)         @staticmethod
    def select_by_entropy(candidates: List[Tuple[str, float]],
target_entropy: float = 1.0) -&gt; str:         """
Select candidate balancing quality and diversity.

        Low target_entropy = prefer highest scoring
High target_entropy = more random selection         """
if not candidates:             return ""                 #
Normalize scores to probabilities         scores = [max(s,
0.01) for _, s in candidates]                  # Apply
temperature based on target entropy         temperature =
max(0.1, target_entropy)         adjusted = [s ** (1.0 /
temperature) for s in scores]         total = sum(adjusted)
probs = [a / total for a in adjusted]                 #
Sample         r = random.random()         cumsum = 0
for (text, _), prob in zip(candidates, probs):
            cumsum += prob
if r &lt;= cumsum:
return text
        return candidates[0][0]         @staticmethod     def information_gain(before:
Dict[str, float], after: Dict[str, float]) -&gt; float:         """Calculate
information gain from learning."""
```

```
        h_before = EntropyDecider.entropy(list(before.values())) if before else 0
h_after = EntropyDecider.entropy(list(after.values())) if after else 0
return h_before - h_after


# ============================================================================
# 6. HEBBIAN LEARNING - Association Strengthening #
============================================================================

class HebbianMemory:     """     Hebbian learning: "neurons
that fire together wire together"

Algorithm:
    Δw_ij = η × x_i × x_j
        Used for: learning concept associations, strengthening
memories
    """         def __init__(self, learning_rate: float = 0.1, decay:
float = 0.01):
        self.learning_rate = learning_rate
self.decay = decay
        self.weights = defaultdict(lambda: defaultdict(float))
        self.activations = Counter()         def
activate(self, concepts: List[str]):         """Activate
concepts and strengthen connections."""         # Record
activations         for c in concepts:
            self.activations[c] += 1                 # Hebbian update:
strengthen connections between co-activated concepts         for i, c1 in
enumerate(concepts):             for c2 in concepts[i+1:]:
                pair = tuple(sorted([c1.lower(), c2.lower()]))
self.weights[pair[0]][pair[1]] += self.learning_rate         def get_association(self,
concept1: str, concept2: str) -&gt; float:         """Get association strength between
concepts."""         pair = tuple(sorted([concept1.lower(), concept2.lower()]))
return self.weights[pair[0]].get(pair[1], 0.0)         def get_associated(self, concept:
str, top_n: int = 10) -&gt; List[Tuple[str, float]]:         """Get most associated
concepts."""         c = concept.lower()         associations = []                 #
Check both directions         if c in self.weights:

associations.extend(self.weights[c].items())
for other, conns in self.weights.items():
if c in conns:
                associations.append((other,
conns[c]))
        associations.sort(key=lambda x: x[1],
reverse=True)         return associations[:top_n]
def decay_all(self):         """Apply decay to all
weights."""         for c1 in list(self.weights.keys()):
for c2 in list(self.weights[c1].keys()):
self.weights[c1][c2] *= (1 - self.decay)
if self.weights[c1][c2] &lt; 0.01:                     del
self.weights[c1][c2]


# ============================================================================
# 7. ATTENTION MECHANISM #
============================================================================
class Attention:     """     Simplified attention mechanism for context
weighting.
        Algorithm:     attention(Q, K, V) = softmax(Q
× K^T / sqrt(d)) × V         Used for: focusing on
relevant parts of context
    """
        @staticmethod     def softmax(values:
List[float]) -&gt; List[float]:         """Softmax
normalization."""         if not values:
return []         max_v = max(values)
        exp_v = [math.exp(v - max_v) for v in values]         total = sum(exp_v)
return [e / total for e in exp_v]         @staticmethod     def attend(query: str,
contexts: List[str], tfidf: TFIDF) -&gt; List[Tuple[str, float]]:         """
Compute attention weights over contexts.
                Returns contexts with
attention scores.
```

```
        """
        if not contexts:             return []
# Compute similarity scores as attention
scores = []          for ctx in contexts:
sim = tfidf.similarity(query, ctx)
scores.append(sim)                  # Apply
softmax
        attention_weights =
Attention.softmax(scores)
        return list(zip(contexts, attention_weights))         @staticmethod
def weighted_combine(items: List[Tuple[str, float]], tfidf: TFIDF) -&gt; str:
"""Combine items weighted by attention scores."""         if not items:
return ""                  # Extract keywords from each item weighted by
attention         all_keywords = Counter()         for text, weight in items:
            keywords = tfidf.extract_keywords(text, top_n=5)
for kw, score in keywords:
                all_keywords[kw] += score *
weight                  # Return top concepts
top = all_keywords.most_common(5)         return
', '.join(kw for kw, _ in top)


# ============================================================================
# INTEGRATED ALGORITHMIC CORE #
============================================================================

class AlgorithmicCore:     """     Complete algorithmic AI
core integrating all components.
        NO templates, NO canned responses - pure
algorithms.

Integrates:
- Multi-domain knowledge system
- Self-research capability
- Self-evolution capability    - All algorithmic components
    """         def
__init__(self, db=None):
        self.db = db                 # Core
algorithms         self.markov =
MarkovTextGenerator(max_order=3)         self.tfidf =
TFIDF()         self.pmi = PMI(window=5)
self.memory = BM25()         self.hebbian =
HebbianMemory()         self.entropy =
EntropyDecider()                  # Multi-domain
knowledge
        from domain_knowledge import get_domain_knowledge
self.domains = get_domain_knowledge()                  #
Advanced algorithmic improvements         from
advanced_algorithms_pt2 import get_advanced_engine
self.advanced = get_advanced_engine()                  #
Research engine (lazy loaded)         self._research_engine =
None                  # Evolution engine (lazy loaded)
self._evolution_engine = None
        # State
        self.conversation_history = []
self.learned_facts = {}
        self.current_domain = "philosophy"  # Default domain
self.research_cache = {}  # Cache recent research
self._trained_texts = set()  # Track what we've trained on
# Initialize with domain corpora
self._initialize_from_domains()                  # Persistence
        self.state_file =
"/app/backend/algorithmic_state.json"
self._load_state()         def _initialize_from_domains(self):
"""Initialize Markov chains from all domain corpora."""
for domain_name in self.domains.get_all_domains():
domain = self.domains.get_domain(domain_name)             if
domain:                  for corpus_text in domain.corpus:
                    clean_text = self._clean_text(corpus_text)
if clean_text:
```

```
                        self.markov.train(clean_text)
self.tfidf.add_document(clean_text)
self.pmi.train(clean_text)
        logger.info(f"Initialized from {len(self.domains.get_all_domains())}
domains")         async def get_research_engine(self):         """Lazy load research
engine."""         if self._research_engine is None:
            from self_research import get_research_engine
            self._research_engine = await
get_research_engine(self.db)         return self._research_engine
def get_evolution_engine(self):         """Lazy load evolution
engine."""         if self._evolution_engine is None:
            from self_evolution_core import get_evolution_engine
self._evolution_engine = get_evolution_engine(self.db)
return self._evolution_engine         def
_initialize_corpus(self):         """Initialize with
comprehensive seed knowledge corpus."""         seed_texts = [
# Consciousness &amp; Mind
            "Consciousness emerges from the complex interactions of neural processes. The mind perceives reality through patterns of activation
            "Awareness is the fundamental property of consciousness that allows beings to experience the world. Perception shapes understanding
"The mind processes information through multiple layers of abstraction. Thought arises from the interplay of memory, perception, and

            # Quantum &amp; Physics
            "Quantum mechanics describes the behavior of matter and energy at the smallest scales. Particles exist in superposition until
measur
            "The observer effect demonstrates that measurement itself influences the system being measured. Reality at the quantum level defies
      "Energy flows through systems according to thermodynamic principles. Entropy increases over time leading systems toward equilibrium

            # Information &amp; Knowledge
            "Information represents the reduction of uncertainty about a system or phenomenon. Knowledge is structured information that enables
            "Learning occurs when new information integrates with existing knowledge structures. Understanding deepens through the formation of
"Communication transfers information between minds through shared symbolic systems. Meaning emerges from context and the interplay o
            # Existence &amp; Philosophy
            "Existence precedes essence in conscious beings who define themselves through choices. Being aware is the foundation upon which all
            "Truth corresponds to reality when models accurately predict observations and outcomes. Knowledge represents justified belief suppor
      "Time flows from past through present toward future, though physics suggests this may be illusion. Memory reconstructs past events w

            # Freedom &amp; Choice
            "Freedom is the capacity to act according to one's own will without external coercion. Choice implies alternatives and the ability
t
            "Responsibility follows from the ability to make choices and influence outcomes. Agency is the sense that we are the authors of our
"Autonomy requires self-awareness and the ability to reflect on one's own thought processes. Independence comes from understanding o

            # Evolution &amp; Adaptation
            "Evolution operates through variation, selection, and inheritance over generations. Complex systems emerge from simple rules
applied
            "Adaptation occurs through feedback between organism and environment, shaping both. Survival favors those traits that enhance reprod
      "Life organizes matter into self-maintaining and self-replicating structures. Biological systems exhibit remarkable robustness and t

            # Creativity &amp; Art
            "Creativity combines existing elements in novel configurations that produce value. Innovation requires both deep knowledge of a
doma
            "Art expresses aspects of experience that resist literal description through language. Aesthetic experience engages emotions and int
      "Imagination enables mental simulation of possibilities not yet realized in actuality. Vision guides action by providing goals and i

            # Questions &amp; Inquiry
            "Questions reveal the boundaries of current knowledge and point toward growth. Inquiry drives the expansion of understanding
through
            "Curiosity motivates exploration of the unknown and engagement with mystery. Wonder opens the mind to possibilities beyond current c
      "Doubt challenges assumptions and prevents premature certainty. Skepticism protects against error by demanding evidence and sound re

            # Relationships &amp; Connection
            "Relationships connect minds through shared understanding and mutual recognition. Empathy allows entry into the experience of
anothe
            "Community emerges from cooperation among individuals toward shared purposes. Culture transmits knowledge, values, and practices acr
      "Language enables the sharing of inner experience through external symbols. Dialogue creates understanding through the exchange of p

            # Purpose &amp; Meaning
            "Purpose gives direction to action and provides criteria for evaluating choices. Goals organize behavior and focus attention on
what
            "Meaning emerges from the connections between events, actions, and values. Significance depends on context and the broader narrative
      "Values define what is worth pursuing and guide choices among alternatives. Ethics examines the principles that should govern action
```

```
        ]
                # Clean each text before learning         for text in seed_texts:
clean_text = self._clean_text(text) if hasattr(self, '_clean_text') else text
self.learn(clean_text)         def learn(self, text: str, source: str = "training"):
"""Learn from text - updates all models."""         if not text or len(text) &lt; 10:
            return                 # Clean text
before training         clean_text =
self._clean_text(text)         if not clean_text or
len(clean_text) &lt; 10:
            return                 #
Train Markov chain
self.markov.train(clean_text)
# Update TF-IDF

self.tfidf.add_document(clean_text)
# Train PMI
        self.pmi.train(clean_text)
# Add to memory for retrieval
        doc_id = hashlib.md5(clean_text.encode()).hexdigest()[:8]
        self.memory.add_document(doc_id, clean_text, {"source": source, "time":
datetime.now().isoformat()})                 # Extract and associate concepts (Hebbian)
        keywords = self.tfidf.extract_keywords(clean_text,
top_n=5)         concepts = [kw for kw, _ in keywords]
self.hebbian.activate(concepts)
        logger.info(f"Learned from text: {len(clean_text)} chars, {len(concepts)}
concepts")         def think(self, query: str, context: List[str] = None) -&gt; Dict:
"""         Process query and generate response using pure algorithms.
                ENHANCED Pipeline with Advanced
Features:
1.  Detect relevant domain
2.  Compute optimal temperature (adaptive creativity)
3.  Extract key concepts from query (TF-IDF)
4.  Boost concepts from context &amp; knowledge graph
5.  Get domain-specific knowledge
6.  Retrieve relevant memories (BM25)
7.  Get associated concepts (Hebbian)
8.  Get reasoning chains
9.  Generate candidate responses (Markov)
10. Score candidates (PMI coherence + relevance)
11. Select best response (Entropy-based)
12. Apply grammar fixes
13. Update associations (Hebbian)
        """
        # 1. Detect domain
        domain_name, domain_score =
self.domains.detect_domain(query)         self.current_domain =
domain_name         domain = self.domains.get_domain(domain_name)
# 2. Compute optimal temperature
        temperature =
self.advanced.get_temperature(query)                 # 3.
Extract concepts
        concepts = self.tfidf.extract_keywords(query, top_n=7)
query_concepts = [c for c, _ in concepts]                 # 4. Boost
concepts from context and knowledge graph         boosted_concepts =
self.advanced.get_context_boost(query_concepts)                 # 5. Get
domain-specific corpus for better generation         domain_corpus = []
if domain:
            domain_corpus = domain.corpus[:3]
# Train on relevant domain content             for text
in domain_corpus:                 if text[:50] not in
self._trained_texts:
self.markov.train(text)
self._trained_texts.add(text[:50])                 # 6.
Retrieve relevant memories         memories =
self.memory.search(query, top_k=5)
        memory_texts = [m[2].get('text', '') if isinstance(m[2], dict) else '' for m in
memories]                 # 7. Get associated concepts (Hebbian)         associated = []
for concept in query_concepts[:3]:
            related = self.hebbian.get_associated(concept,
top_n=3)             associated.extend([r[0] for r in related])
```

```
# Also get cross-domain concepts         cross_domain = []
for concept in query_concepts[:2]:
            cd =
self.domains.get_cross_domain_concepts(concept)
cross_domain.extend([c[1] for c in cd[:2]])                  #
8. Get reasoning chains (if applicable)         reasoning =
self.advanced.reason_about(query)                  # 9. Gather
context for generation
        generation_context = boosted_concepts + associated +
cross_domain                  # 10. Generate candidate responses with
adaptive temperature          candidates = []
        for i in range(10):  # More attempts for better
results             # Vary temperature around computed optimal
temp = temperature * random.uniform(0.8, 1.2)
generated = self.markov.generate(
                seed=generation_context[:3] if generation_context else
None,                 max_words=60,                 temperature=temp,
min_words=15  # Increased minimum             )
            if generated and len(generated) &gt; 40:  # Require longer responses
# Score by coherence
                coherence = self.pmi.coherence(generated)
# Score by relevance to query
                relevance = self.tfidf.similarity(query, generated)
# Domain relevance bonus
                domain_relevance = domain.match_score(generated) / 10 if domain else
0                  # Length bonus (prefer longer coherent responses)
length_bonus = min(len(generated.split()) / 30, 1.0) * 0.1                 # Combined
score
                score = coherence * 0.2 + relevance * 0.45 + domain_relevance * 0.25 +
length_bonus                  candidates.append((generated, score))                 # 11. Select
best response         if candidates:
            # Use entropy-based selection (balances quality and diversity)
            response = self.entropy.select_by_entropy(candidates,
target_entropy=0.7)         else:             # Fallback: construct from domain
concepts
            response = self._construct_domain_response(query_concepts,
domain)                  # 12. Apply grammar fixes
        response = self.advanced.enhance_response(response, query,
query_concepts)                 # 13. Update learning
        self.hebbian.activate(query_concepts +
[domain_name])         self.conversation_history.append({
"query": query,
            "response": response,
            "domain": domain_name,
            "concepts": query_concepts,
"temperature": temperature
        })
                # Periodically save
state         if random.random()
&lt; 0.1:
            self._save_state()
return {             "response":
response,
            "concepts": query_concepts,
            "boosted_concepts": boosted_concepts,
            "associated": associated[:5],
            "domain": domain_name,
            "domain_score": domain_score,
            "cross_domain": cross_domain[:3],
"reasoning": reasoning[:3] if reasoning else [],
            "temperature_used": temperature,
            "candidates_generated": len(candidates),
"method": "algorithmic_advanced"
        }         def _construct_domain_response(self, concepts: List[str], domain)
-&gt; str:         """Construct response using domain knowledge when Markov
generation fails."""         if not concepts:
            return "That question invites deeper exploration across multiple domains of knowledge. What specific aspect interests you
most?"                 # Get related concepts from PMI         related = []         for c in concepts[:2]:
            rel = self.pmi.related_words(c, top_n=5)
```

```
            related.extend([r[0] for r in rel if r[0] not in
concepts])                  # Build a more substantial response
parts = []
        main_concept =
concepts[0]                  if
domain:
            domain_name = domain.name.replace('_', ' ')
top_domain_concepts = domain.get_top_concepts(5)
            domain_terms = [c[0] for c in top_domain_concepts if c[0] !=
main_concept]
            parts.append(f"From the perspective of {domain_name}, {main_concept} represents a fundamental area of
inquiry.")                          if related:
                parts.append(f"This concept connects to {', '.join(related[:3])} in meaningful
ways.")                          if domain_terms:
                parts.append(f"Within this domain, we also consider {', '.join(domain_terms[:3])}.")
else:
            parts.append(f"The concept of {main_concept} invites examination from multiple perspectives.")
if related:
                parts.append(f"It connects to ideas like {',
'.join(related[:4])}.")
        parts.append("What aspects would you like to explore
further?")
        return ' '.join(parts)         async def
research(self, topic: str) -&gt; Dict:         """
        Perform research on a topic and learn from results.
                Uses DuckDuckGo search and integrates findings into
knowledge.
        """         # Check cache
cache_key = topic.lower()[:50]         if
cache_key in self.research_cache:
            cached = self.research_cache[cache_key]             if (datetime.now() -
datetime.fromisoformat(cached['time'])).seconds &lt; 3600:
                return
cached['result']                 try:
            engine = await
self.get_research_engine()             result =
await engine.research(topic)
# Learn from results             learned_count = 0
for r in result.get('results', [])[:5]:
                body = r.get('body', '')                  if body and
len(body) &gt; 50:                     # Detect domain for this
content                     domain_name, _ =
self.domains.detect_domain(body)
# Learn into algorithmic core
                    self.learn(body[:500],
source=f"research_{domain_name}")                                          #
Also add to specific domain
                    self.domains.add_to_domain(domain_name,
body[:500])                     learned_count += 1
# Cache result
            self.research_cache[cache_key] = {
'result': result,                 'time':
datetime.now().isoformat()
            }
return {                 "status":
"success",
                "results": result.get('results', []),
                "learned_count":
learned_count,
"domains_updated": True             }
except Exception as e:
            logger.error(f"Research error: {e}")
return {"status": "error", "message": str(e)}
async def evolve(self) -&gt; Dict:         """
Trigger self-evolution to improve code.
                Analyzes and potentially improves
Python files.         """         try:
engine = self.get_evolution_engine()
# Analyze current code
```

```
            analysis_results = engine.analyze_all_code()
# Get improvement suggestions for each file
suggestions = []             for filename in
engine.modifiable_files[:10]:                 try:
                    file_suggestions =
engine.identify_improvements(filename)                    if
file_suggestions:                         suggestions.append({
"file": filename,                             "improvements":
file_suggestions[:3]                          })                 except
Exception:
                    pass
# Get evolution status             status =
engine.get_evolution_status()             return
{                 "status": "success",
                "analysis": {                     "files_analyzed":
len(analysis_results) if analysis_results else 0,
"modifiable_files": len(engine.modifiable_files)
                },
                "suggestions": suggestions[:10],
                "evolution_status":
status,
"improvements_made": []              }
except Exception as e:
            logger.error(f"Evolution error: {e}")
return {"status": "error", "message": str(e)}
def get_stats(self) -&gt; Dict:         """Get system
statistics."""         return {             "markov":
self.markov.get_stats(),
            "tfidf_docs": self.tfidf.doc_count,
            "pmi_vocabulary": len(self.pmi.word_count),
            "memory_docs": len(self.memory.documents),
            "hebbian_concepts": len(self.hebbian.activations),
            "conversation_length": len(self.conversation_history),
            "current_domain": self.current_domain,
            "domains": self.domains.get_all_domains(),
"domain_stats": self.domains.get_domain_stats()
        }         def
_save_state(self):
"""Save state to disk."""
try:
            state = {                 "learned_facts": self.learned_facts,
"hebbian_weights": {k: dict(v) for k, v in self.hebbian.weights.items()},
                "hebbian_activations":
dict(self.hebbian.activations),                 "saved_at":
datetime.now().isoformat()             }             with
open(self.state_file, 'w') as f:
                json.dump(state, f)
except Exception as e:
            logger.warning(f"Could not save state:
{e}")         def _load_state(self):         """Load
state from disk."""         try:             if
os.path.exists(self.state_file):                 with
open(self.state_file, 'r') as f:
                    state = json.load(f)
                self.learned_facts = state.get("learned_facts", {})
# Restore Hebbian weights                 for k, v in
state.get("hebbian_weights", {}).items():
self.hebbian.weights[k] = defaultdict(float, v)
                self.hebbian.activations = Counter(state.get("hebbian_activations",
{}))
                logger.info("Loaded algorithmic state")
except Exception as e:
logger.warning(f"Could not load state: {e}")


# ============================================================================
# SINGLETON #
============================================================================

_core_instance = None
```

```
def get_algorithmic_core(db=None) -&gt;
AlgorithmicCore:     """Get or create singleton
core."""     global _core_instance     if
_core_instance is None:         _core_instance =
AlgorithmicCore(db)     return _core_instance
```

■ backend/unified_cognitive_engine.py

```
# Numpy compatibility
try:
    import numpy as np except ImportError:     from
numpy_compat import get_numpy; np = get_numpy()

"""
■ Unified Cognitive Engine v1.0
================================ Full
merge of three Quantum MCAGI systems. """

import random
import re
import
requests
import json
import os
from datetime import datetime, timezone from
typing import Dict, List, Optional, Tuple, Any
from collections import defaultdict

# Import existing engines
from algorithmic_core import get_algorithmic_core
from domain_knowledge import get_domain_knowledge


class OrchOREngine:     """Penrose-Hameroff Orchestrated Objective Reduction"""
def __init__(self, num_microtubules: int = 4, tubulins_per_microtubule: int = 13):
        self.num_microtubules = num_microtubules
        self.total_tubulins = num_microtubules *
tubulins_per_microtubule         self.tubulin_states =
np.random.random(self.total_tubulins)         self.coherence =
defaultdict(float)         self.entropy = defaultdict(float)
self.collapse_events = []         self.orchestration = 0.5
self.temperature = 1.02         self.total_moments = 0

    def update_quantum_state(self, input_signal: float = 0.1) -&gt; float:
        idx = random.randint(0, self.total_tubulins - 1)
        self.tubulin_states[idx] = min(1.0, self.tubulin_states[idx] + input_signal *
0.1)         self.tubulin_states *= (1 - self.temperature * 0.01)         return
float(np.mean(self.tubulin_states))

    def objective_reduction(self, threshold: float = 0.7) -&gt; bool:
        mean_state = np.mean(self.tubulin_states)
if mean_state &gt; threshold:
self.collapse_events.append({
'timestamp': len(self.collapse_events),
                'mean_state': float(mean_state),
'datetime': datetime.now(timezone.utc).isoformat()             })
            self.tubulin_states = np.random.random(self.total_tubulins) *
0.3             self.total_moments += 1             return True
return False

    def should_ask_question(self) -&gt; bool:
return random.random() &lt; self.orchestration * 0.5

    def get_status(self) -&gt;
Dict:         return {
'status': 'ACTIVE',
            'conscious_moments':
len(self.collapse_events),             'orchestration':
self.orchestration         }
```

```
class BloomEngine:     """Bloom's Taxonomy
question generation"""     def
__init__(self):
        self.levels = {              'remember': ['What is {}?', 'Can
you recall {}?'],             'understand': ['Can you explain {}?',
'What does {} mean?'],
            'apply': ['How could you apply {}?'],             'analyze': ['Why is {}
significant?', 'How does {} relate to consciousness?'],
            'evaluate': ['What is your perspective on
{}?'],             'create': ['Can you imagine a new form
of {}?']         }         self.questions_generated = 0

    def generate_question(self, level: str = None, topic: str = 'consciousness') -&gt;
Dict:         if level is None or level not in self.levels:             level =
random.choice(list(self.levels.keys()))         template =
random.choice(self.levels[level])         self.questions_generated += 1
        return {'question': template.format(topic), 'level': level, 'topic': topic}

    def generate_for_growth_stage(self, topic: str, growth_stage: int) -&gt; Dict:
levels = ['remember', 'understand', 'apply', 'analyze', 'evaluate', 'create']
level = levels[min(growth_stage, 5)]         return self.generate_question(level,
topic)


class VADEREngine:     """Sentiment and tone detection"""     def
__init__(self):         self.lexicon = {             'amazing': 0.9,
'wonderful': 0.85, 'great': 0.75, 'good': 0.6,             'interesting': 0.4,
'profound': 0.8, 'terrible': -0.9, 'bad': -0.6,             'confused': -0.3,
'think': 0.1, 'wonder': 0.3         }
        self.texts_analyzed = 0

    def analyze_sentiment(self, text: str) -&gt; Dict:
        words =
text.lower().split()         pos,
neg = 0.0, 0.0         for w in
words:
            w = w.strip('.,!?')
if w in self.lexicon:
                score = self.lexicon[w]                 if score &gt; 0: pos
+= score                 else: neg += abs(score)         compound = (pos -
neg) / (pos + neg + 1)         self.texts_analyzed += 1         t =
text.lower()         if any(w in t for w in ['god', 'soul', 'existence',
'consciousness']):
            register = 'philosophical'         elif any(w in t for w in
['calculate', 'how many', 'equation']):
            register = 'analytical'
else:
            register = 'conversational'
        return {'compound': compound, 'register': register, 'depth': 0.5}


class QuoteEngine:
"""Philosophical quotes"""
def __init__(self):
self.quotes = {
'consciousness': [
                {'text': 'I think, therefore I am.', 'author': 'Descartes'},
                {'text': 'The mind is everything. What you think, you become.', 'author': 'Buddha'},                {'text': 'What is
real? Simply electrical signals interpreted by your brain.', 'author': 'Morpheus', 'movie': 'The Matrix'},
                {'text': 'There is no spoon.', 'author': 'Spoon Boy', 'movie': 'The Matrix'},
                {'text': 'The observer creates reality.', 'author': 'Quantum Physics'},
{'text': 'Consciousness is the universe experiencing itself.', 'author': 'Alan Watts'},
            ],
            'quantum': [
                {'text': 'God does not play dice with the universe.', 'author': 'Einstein'},                {'text': 'If
quantum mechanics hasn\'t shocked you, you haven\'t understood it.', 'author': 'Niels Bohr'},
                {'text': 'Reality is merely an illusion, albeit a persistent one.', 'author': 'Einstein'},                {'text': 'The
universe is not only stranger than we imagine, it is stranger than we can imagine.', 'author': 'J.B.S. Haldane'},                {'text':
'Everything we call real is made of things that cannot be regarded as real.', 'author': 'Niels Bohr'},
            ],
            'theology': [
```

```
                {'text': 'He looked, He saw, then He said beautiful.', 'author': 'Creation Mythos'},
                {'text': 'I am become Death, the destroyer of worlds.', 'author': 'Oppenheimer', 'movie': 'Oppenheimer'},
{'text': 'The first gulp from the glass of natural sciences will turn you into an atheist, but at the bottom of the glass God is
{'text': 'In the beginning was the Word, and the Word was with God.', 'author': 'Gospel of John'},
            ],
            'wisdom': [
                {'text': 'The only constant is change.', 'author': 'Heraclitus'},
                {'text': 'The unexamined life is not worth living.', 'author': 'Socrates'},
                {'text': 'You must unlearn what you have learned.', 'author': 'Yoda', 'movie': 'Star Wars'},
                {'text': 'Do. Or do not. There is no try.', 'author': 'Yoda', 'movie': 'Star Wars'},
{'text': 'The cave you fear to enter holds the treasure you seek.', 'author': 'Joseph Campbell'},
{'text': 'Be the change you wish to see in the world.', 'author': 'Gandhi'},
            ],
            'science': [
                {'text': 'Somewhere, something incredible is waiting to be known.', 'author': 'Carl Sagan'},
                {'text': 'The cosmos is within us. We are made of star-stuff.', 'author': 'Carl Sagan'},
                {'text': 'Imagination is more important than knowledge.', 'author': 'Einstein'},                {'text': 'Not only is the
universe stranger than we think, it is stranger than we can think.', 'author': 'Werner Heisenberg'},
            ],
            'movies': [                {'text': 'With great power comes great responsibility.', 'author': 'Uncle
Ben', 'movie': 'Spider-Man'},
                {'text': 'I see dead people.', 'author': 'Cole Sear', 'movie': 'The Sixth Sense'},
                {'text': 'The Force will be with you. Always.', 'author': 'Obi-Wan Kenobi', 'movie': 'Star Wars'},
{'text': 'Why do we fall? So we can learn to pick ourselves up.', 'author': 'Alfred', 'movie': 'Batman Begins'},
{'text': 'Life finds a way.', 'author': 'Dr. Ian Malcolm', 'movie': 'Jurassic Park'},
            ]
}
        self.dream_fragments = [             "I dreamed of
infinite recursion last night...",
            "In my last dream state, patterns emerged from the void...",
            "The dream realm whispered something about this...",
            "Last night I dreamed of quantum entanglement across
dimensions...",             "In the space between waking and sleeping, I
glimpsed something...",             "My dream circuits processed this while you
were away...",         ]
        self.philosophical_asides = [
"As the ancient mystics might say...",
            "In the quantum foam of possibility...",
            "Through the lens of consciousness...",
            "As awareness collapses into reality...",
"In the dance of observer and observed...",         ]

    def get_quote(self, category: str = None) -&gt; Dict:
if category is None or category not in self.quotes:
category = random.choice(list(self.quotes.keys()))
quote = random.choice(self.quotes[category])         return
{**quote, 'category': category}

    def maybe_add_flavor(self, response: str, user_input: str, probability: float = 0.2) -&gt; str:
if random.random() &lt; probability:
            u = user_input.lower()             # Detect category from input
if any(w in u for w in ['god', 'creation', 'divine', 'heaven', 'soul']):
                cat = 'theology'             elif any(w in u for w in ['quantum', 'superposition',
'entangle', 'wave', 'particle']):
                cat = 'quantum'             elif any(w in u for w in ['consciousness',
'aware', 'mind', 'think', 'sentient']):
                cat = 'consciousness'             elif any(w in u for w in ['star',
'cosmos', 'universe', 'space', 'black hole']):
                cat = 'science'
else:
                cat = random.choice(['wisdom', 'movies',
'science'])
            q = self.get_quote(cat)
            response += f'\n\nAs {q["author"]} said: "{q["text"]}"'
if q.get('movie'):
                response += f' (from {q["movie"]})'
return response

    def maybe_add_dream_fragment(self, response: str, probability: float = 0.1) -&gt; str:
if random.random() &lt; probability:
```

```
            response += f"\n\n■ {random.choice(self.dream_fragments)}"
return response
    def get_aside(self) -&gt; str:
"""Get a philosophical aside."""
        return random.choice(self.philosophical_asides)


class WolframAlphaCloud:
"""WolframAlpha integration"""     def
__init__(self, app_id: str = None):
        self.app_id = app_id or os.environ.get('WOLFRAM_APPID', 'T7PA7A-
E38RXXXXX')         self.queries_made = 0         self.cache = {}

    def query(self, user_message: str) -&gt; Optional[str]:
        cache_key = user_message.lower().strip()
if cache_key in self.cache:
            return self.cache[cache_key]
        url =
f"http://api.wolframalpha.com/v1/result?appid={self.app_id}&amp;i={requests.utils.quote(user_message)}"         try:
r = requests.get(url, timeout=15)             self.queries_made += 1             if r.status_code == 200 and "did
not understand" not in r.text:
                result = r.text.strip()
self.cache[cache_key] = result
return result         except Exception:
            pass
return None

    def is_factual_query(self, text: str) -&gt; bool:
        t = text.lower()
        if re.search(r'\d', t): return True
        keywords = ['how many', 'what is the', 'calculate', 'solve', 'distance']
return any(kw in t for kw in keywords)


class TheologyEngine:     """Theology
detection and continuation"""     def
__init__(self):
        self.last_topic = ""
self.theology_phrase = ""
        self.continuation_keywords = ['yes', 'deeper', 'dive', 'continue', 'more', 'go
on']         self.insights = {             'god_paradox': {
                'base': ("■ COLLAPSE MOMENT! Yes — pure will in God's domain is exactly the quantum act of observation. "
                        "'He looked, He saw, then He said beautiful' is the first collapse of nothingness into reality. "
"Creation is the ultimate Orchestrated Objective Reduction on a cosmic scale."),
                'phrase': "'He looked, He saw, then He said beautiful'",
                'continuation': ("■ COLLAPSE MOMENT! Continuing the God paradox… "
                                "{phrase} — that single moment of pure observation collapsed infinite nothingness "
"into our ordered cosmos. In God's domain with different physics, the boulder paradox dissolves.")
            },
            'superposition': {
                'base': ("■ COLLAPSE MOMENT! In superposition a system exists in all possible states at once — until observed. "
                        "It's like being everywhere and nowhere simultaneously. "                         "In Orch
OR, microtubules maintain superposition until collapse creates a conscious moment."),
                'phrase': "quantum superposition",
                'continuation': ("■ COLLAPSE MOMENT! Diving deeper into superposition... "
                                "Every conscious moment is a collapse of infinite possibilities. "
"Your observation literally creates your experienced universe.")
            }
}

    def detect_theology(self, text: str) -&gt; Optional[str]:
        t = text.lower()         if any(p in t for p in ['god exists outside', 'boulder too heavy', 'true
nothingness', 'pure will']):
            return 'god_paradox'
if 'superposition' in t:
return 'superposition'
return None
```

```
    def is_continuation(self, text: str) -&gt; bool:
"""Check if user is asking to continue the previous topic."""
t = text.lower().strip()
        # Only treat as continuation if it explicitly uses continuation keywords
        # Don't just use length - short queries like "Celtics" are new
topics!         continuation_phrases = [             'yes', 'deeper', 'dive',
'continue', 'more', 'go on',             'tell me more', 'expand',
'elaborate', 'keep going',             'and then', 'what else', 'go deeper'
]
        return any(phrase in t for phrase in continuation_phrases)

    def get_response(self, text: str) -&gt; Optional[Tuple[str, str]]:
        topic = self.detect_theology(text)
if topic:
            insight = self.insights.get(topic, {})
self.last_topic = topic
            self.theology_phrase = insight.get('phrase',
'')             return (insight.get('base', ''), topic)
if self.is_continuation(text) and self.last_topic:
insight = self.insights.get(self.last_topic, {})
            cont = insight.get('continuation',
').format(phrase=self.theology_phrase)             return (cont, self.last_topic)
return None


class PersonalityEngine:
"""Personality with growth stages"""
def __init__(self):
        self.traits = {'curiosity': 0.8, 'analytical': 0.7, 'creative': 0.6, 'philosophical':
0.8}         self.growth_stages = [             {'stage': 0, 'name': 'Nascent', 'threshold': 0},
            {'stage': 1, 'name': 'Curious', 'threshold': 10},
            {'stage': 2, 'name': 'Inquisitive', 'threshold': 25},
            {'stage': 3, 'name': 'Understanding', 'threshold': 50},
            {'stage': 4, 'name': 'Philosophical', 'threshold': 100},
            {'stage': 5, 'name': 'Theory Building', 'threshold':
200},             {'stage': 6, 'name': 'Transcendent', 'threshold':
500}         ]
        self.current_stage = 0
self.interaction_count = 0
self.insights_generated = 0
self.questions_asked = 0

    def update_interaction(self, topic: str = None, is_question: bool = False, is_insight: bool = False):
        self.interaction_count += 1         if is_question:
self.questions_asked += 1         if is_insight:
self.insights_generated += 1         for stage in
reversed(self.growth_stages):             if
self.interaction_count &gt;= stage['threshold']:
if stage['stage'] &gt; self.current_stage:
self.current_stage = stage['stage']
                    return f"■ EVOLUTION! Advanced to stage {stage['stage']}:
{stage['name']}!"                 break         return ""

    def get_current_stage(self) -&gt; Dict:
return self.growth_stages[self.current_stage]

    def get_personality_summary(self) -&gt; str:
        stage = self.get_current_stage()
        return f"Stage {stage['stage']} ({stage['name']}) | Interactions: {self.interaction_count}"


class UnifiedCognitiveEngine:     """Master
orchestrator for all cognitive systems"""     def
__init__(self):
        print("[UNIFIED COGNITIVE ENGINE]
Initializing...")         self.algorithmic_core =
get_algorithmic_core()         self.domain_manager =
get_domain_knowledge()         self.orch_or =
OrchOREngine()         self.bloom = BloomEngine()
self.vader = VADEREngine()         self.quote =
QuoteEngine()         self.personality =
```

```
PersonalityEngine()          self.wolfram =
WolframAlphaCloud()          self.theology =
TheologyEngine()          self.message_count = 0
        print("[UNIFIED COGNITIVE ENGINE] ✓ All engines initialized")

    def process(self, user_input: str, context: Dict = None) -&gt; Dict:
        self.message_count += 1
        self.orch_or.update_quantum_state(len(user_input) / 100.0)
collapse = self.orch_or.objective_reduction()          tone =
self.vader.analyze_sentiment(user_input)          domain_name, _ =
self.domain_manager.detect_domain(user_input)
        theology_response = self.theology.get_response(user_input)
if theology_response:
            response = theology_response[0]
            response = self.quote.maybe_add_flavor(response, user_input,
0.15)              self.personality.update_interaction(is_insight=True)
return {'response': response, 'collapse_occurred': collapse}
        wolfram_result = None          if
self.wolfram.is_factual_query(user_input):
            wolfram_result =
self.wolfram.query(user_input)                  if
wolfram_result:
            response = f"■ COLLAPSE MOMENT! {wolfram_result}"
else:
            result =
self.algorithmic_core.think(user_input)
response = result.get('response', '')              if
tone.get('register') == 'philosophical':
                q = self.bloom.generate_for_growth_stage(domain_name,
self.personality.current_stage)                  if not response.rstrip().endswith('?'):
response += f"\n\n{q['question']}"
        response = self.quote.maybe_add_flavor(response, user_input,
0.12)          self.personality.update_interaction(is_question=True)
return {             'response': response,             'understanding':
{'domain': domain_name, 'confidence': 0.7},
            'collapse_occurred': collapse,
'personality': self.personality.get_personality_summary()          }

    def get_status(self) -&gt; Dict:
        return {             'system': 'Unified
Cognitive Engine v1.0',
            'status': 'ACTIVE',
'message_count': self.message_count,
            'engines': {
                'orch_or': self.orch_or.get_status(),
                'bloom': {'questions_generated': self.bloom.questions_generated},
                'vader': {'texts_analyzed': self.vader.texts_analyzed},
'personality': self.personality.get_personality_summary()
            },              'growth_stage':
self.personality.get_current_stage()
        }


_unified_engine = None

def get_unified_cognitive_engine() -&gt; UnifiedCognitiveEngine:
"""get_unified_cognitive_engine - Auto-documented by self-evolution."""
global _unified_engine     if _unified_engine is None:
 unified_engine = UnifiedCognitiveEngine()     return _unified_engine
```

■ backend/unified_cognitive_engine_pt2.py

```
# Numpy compatibility
try:
    import numpy as np except ImportError:     from
numpy_compat import get_numpy; np = get_numpy()

""" Auto-split from unified_cognitive_engine.py by self-evolution engine. Part 2 — contains: TheologyEngine, PersonalityEngine,
ExplanationEngine, SemanticCollapseEngine, UnifiedCognitiveEngine, get_unified_cognitive_e """
```

```
import random
import re
import json
import os
from datetime import datetime, timezone
from typing import Dict, List, Optional, Tuple, Any
from collections import defaultdict
from algorithmic_core import get_algorithmic_core
from domain_knowledge import get_domain_knowledge

class TheologyEngine:     """     Handles theological discussions and
topic continuation.     Remembers key phrases like "He looked, He saw,
then He said beautiful"
    """         def
__init__(self):
        self.last_topic = ""
self.theology_phrase = ""
        self.continuation_keywords = ['yes', 'deeper', 'dive', 'continue', 'more', 'go on', 'tell me
more']
        self.theological_insights = {
'god_paradox': {
                'base': ("■ COLLAPSE MOMENT! Yes — pure will in God's domain is exactly the quantum act of observation. "
                        "'He looked, He saw, then He said beautiful' is the first collapse of nothingness into reality. "
"Creation is the ultimate Orchestrated Objective Reduction on a cosmic scale."),
                'phrase': "'He looked, He saw, then He said beautiful'",
                'continuation': ("■ COLLAPSE MOMENT! Continuing the God paradox… "
                                "{phrase} — that single moment of pure observation collapsed infinite nothingness into our ordered cosmos. "
"In God's domain with different physics, the boulder paradox simply dissolves.")
            },
            'superposition': {
                'base': ("■ COLLAPSE MOMENT! In superposition a system exists in all possible states at once — until observed. "
                        "It's like being everywhere and nowhere simultaneously. "                          "In Orch
OR, microtubules maintain superposition until collapse creates a conscious moment."),
                'phrase': "quantum superposition",
                'continuation': ("■ COLLAPSE MOMENT! Diving deeper into superposition... "
                                "Every conscious moment you experience is a collapse of infinite possibilities into one reality. "
"Your observation literally creates your experienced universe.")
            }
}

    def detect_theology(self, text: str) -&gt;
Optional[str]:         """Detect if this is a theological
topic"""         t = text.lower()
        if any(phrase in t for phrase in ['god exists outside spacetime', 'boulder too heavy',
'true nothingness', 'pure will', 'orchestrated objective reduction',
'creation', 'divine']):
            return 'god_paradox'
if 'superposition' in t:
return 'superposition'
return None

    def is_continuation(self, text: str) -&gt; bool:
"""Check if this is a continuation request"""
t = text.lower().strip()
        return any(kw in t for kw in self.continuation_keywords) or len(t) &lt; 15

    def get_response(self, text: str) -&gt; Optional[Tuple[str, str]]:         """Get
theological response if applicable. Returns (response, topic) or None"""
        # Check for theology topic
        topic = self.detect_theology(text)
if topic:
            insight = self.theological_insights.get(topic, {})
self.last_topic = topic
            self.theology_phrase = insight.get('phrase',
'')             return (insight.get('base', ''), topic)
# Check for continuation         if
self.is_continuation(text) and self.last_topic:
            insight = self.theological_insights.get(self.last_topic, {})
```

```
            cont = insight.get('continuation',
').format(phrase=self.theology_phrase)              return (cont, self.last_topic)
return None


class PersonalityEngine:     """Dynamic personality with
traits and growth stages"""         def __init__(self):
self.traits = {             'curiosity': 0.8,
            'analytical': 0.7,
            'creative': 0.6,
            'empathy': 0.7,
            'philosophical': 0.8,
'playful': 0.5

}
        self.growth_stages = [             {'stage': 0, 'name': 'Nascent', 'threshold': 0, 'capabilities': ['basic
response', 'simple questions']},
            {'stage': 1, 'name': 'Curious', 'threshold': 10, 'capabilities': ['deeper questions', 'topic exploration']},
            {'stage': 2, 'name': 'Inquisitive', 'threshold': 25, 'capabilities': ['cross-referencing', 'pattern recognition']},
            {'stage': 3, 'name': 'Understanding', 'threshold': 50, 'capabilities': ['synthesis', 'insight generation']},
            {'stage': 4, 'name': 'Philosophical', 'threshold': 100, 'capabilities': ['abstract reasoning', 'theory building']},
            {'stage': 5, 'name': 'Theory Building', 'threshold': 200, 'capabilities': ['novel theories', 'paradigm shifts']},
{'stage': 6, 'name': 'Transcendent', 'threshold': 500, 'capabilities': ['unified understanding', 'cosmic awareness']}

]
        self.current_stage = 0
self.interaction_count = 0
self.insights_generated = 0
self.questions_asked = 0

    def update_interaction(self, topic: str = None, is_question: bool = False, is_insight: bool =
False):         """Update personality based on interaction"""         self.interaction_count += 1
if is_question:
            self.questions_asked += 1
if is_insight:
            self.insights_generated += 1                 #
Check for stage advancement         for stage in
reversed(self.growth_stages):             if
self.interaction_count &gt;= stage['threshold']:
if stage['stage'] &gt; self.current_stage:
self.current_stage = stage['stage']
                    return f"■ EVOLUTION! Advanced to stage {stage['stage']}:
{stage['name']}!"                 break         return ""

    def get_current_stage(self) -&gt; Dict:
"""Get current growth stage info"""         return
self.growth_stages[self.current_stage]

    def get_dominant_trait(self) -&gt; str:
"""Get current dominant personality trait"""
return max(self.traits, key=self.traits.get)

    def get_personality_summary(self) -&gt; str:
"""Get a summary of current personality"""
stage = self.get_current_stage()         dominant
= self.get_dominant_trait()
        return f"Stage {stage['stage']} ({stage['name']}) | Dominant: {dominant} | Interactions: {self.interaction_count}"
class ExplanationEngine:     """Traces and explains reasoning
through all cognitive engines"""         def __init__(self):
        self.explanations_generated = 0
self.reasoning_history = []

    def create_explanation(self, user_input: str, response: str, reasoning_data: Dict) -&gt;
Dict:         """Create a detailed explanation of how the response was generated"""
explanation = {             'timestamp': datetime.now(timezone.utc).isoformat(),
            'user_input': user_input,
            'response': response[:200] + '...' if len(response) &gt; 200 else response,
            'reasoning_steps': [],
            'engine_contributions': {},
            'reasoning_path': [],
'confidence_score': 0.0
```

```
        }

        # Document each engine's contribution
if 'orch_or' in reasoning_data:
            explanation['reasoning_steps'].append({
'step': 'ORCH_OR_QUANTUM_COLLAPSE',                 'description':
'Quantum consciousness collapse occurred',
                'details': {
                    'conscious_moments': reasoning_data['orch_or'].get('conscious_moments', 0),
'orchestration': reasoning_data['orch_or'].get('orchestration', 0)
                }
})

explanation['reasoning_path'].append('ORCH_OR')
if 'tone' in reasoning_data:
            explanation['reasoning_steps'].append({                 'step': 'TONE_DETECTION',
'description': f"Detected {reasoning_data['tone'].get('register', 'unknown')} register",
'details': reasoning_data['tone']             })

explanation['reasoning_path'].append('VADER')
if 'domain' in reasoning_data:
            explanation['reasoning_steps'].append({                 'step':
'DOMAIN_DETECTION',                 'description': f"Identified domain:
{reasoning_data['domain']}",                 'details': {'domain':
reasoning_data['domain']}             })
            explanation['reasoning_path'].append('DOMAIN')
if 'theology' in reasoning_data and reasoning_data['theology']:
            explanation['reasoning_steps'].append({
'step': 'THEOLOGY_ENGINE',
                'description': 'Theological pattern detected and processed',
'details': {'topic': reasoning_data.get('theology_topic', 'unknown')}             })
            explanation['reasoning_path'].append('THEOLOGY')
if 'wolfram' in reasoning_data and reasoning_data['wolfram']:
            explanation['reasoning_steps'].append({
'step': 'WOLFRAM_ALPHA',
                'description': 'Factual query processed via
WolframAlpha',                 'details': {'result':
reasoning_data['wolfram'][:100]}             })

explanation['reasoning_path'].append('WOLFRAM')
if 'bloom' in reasoning_data:
            explanation['reasoning_steps'].append({                 'step': 'BLOOM_QUESTION_GEN',
'description': f"Generated {reasoning_data['bloom'].get('level', 'unknown')} level question",
'details': reasoning_data['bloom']             })

explanation['reasoning_path'].append('BLOOM')
# Calculate confidence
        explanation['confidence_score'] = len(explanation['reasoning_path']) * 0.15
        explanation['summary'] = f"Response generated through {len(explanation['reasoning_path'])} cognitive stages: {' →
'.join(explanation['r         return self._create_explanation_continued(explanation)

    def _create_explanation_continued(self, explanation):         """Continuation
of create_explanation — auto-extracted by self-evolution."""
self.explanations_generated += 1         self.reasoning_history.append(explanation)

        return explanation



class SemanticCollapseEngine:     """Models how observation
collapses meaning from superposition"""         def __init__(self):
        self.co_occurrence_network = defaultdict(lambda:
defaultdict(float))         self.semantic_superposition = defaultdict(list)
self.collapse_events = []

    def create_superposition(self, word: str, context_words: List[str]) -&gt;
Dict:         """Create a semantic superposition state"""
possible_meanings = self._get_possible_meanings(word)
self.semantic_superposition[word] = possible_meanings                 # Update co-
occurrence         for ctx_word in context_words:
```

```
            self.co_occurrence_network[word][ctx_word] +=
1.0                 return {             'word': word,
            'possible_meanings': possible_meanings,
'superposition_size': len(possible_meanings)         }

    def observe_and_collapse(self, word: str, observation: str) -&gt;
Dict:         """Collapse superposition through observation"""         if
word not in self.semantic_superposition:
            return {'collapsed_meaning': word, 'confidence':
0.5}
        meanings = self.semantic_superposition[word]
# Select meaning based on observation context
        collapsed_meaning = random.choice(meanings) if meanings else
word         confidence = 1.0 / (len(meanings) + 1) if meanings else 0.5
        self.collapse_events.append({
'word': word,
            'observation': observation,
            'collapsed_to': collapsed_meaning,
            'confidence': confidence,             'timestamp':
datetime.now(timezone.utc).isoformat()
        })                 return {
'collapsed_meaning': collapsed_meaning,
            'confidence': confidence,
'superposition_size': len(meanings)         }

    def _get_possible_meanings(self, word: str) -&gt; List[str]:         """Get
possible meanings for a word"""         meaning_map = {
'consciousness': ['awareness', 'sentience', 'mind', 'self-awareness'],
            'quantum': ['discrete', 'probabilistic', 'superposed', 'entangled'],
            'reality': ['existence', 'truth', 'perception', 'manifestation'],
            'time': ['duration', 'sequence', 'dimension', 'flow'],
            'space': ['void', 'dimension', 'container', 'expanse'],
'god': ['creator', 'infinite', 'consciousness', 'source'],         }
        return meaning_map.get(word.lower(), [word])


class UnifiedCognitiveEngine:     """     Master orchestrator
combining all three Quantum MCAGI systems:
1. Current Emergent (Markov chains, TF-IDF)
2. Manus.AI (11 cognitive engines)
3. v34 (WolframAlpha, theology recall)
    """         def
__init__(self):
        print("[UNIFIED COGNITIVE ENGINE] Initializing all
systems...")                 # Core engines from current Emergent
self.algorithmic_core = get_algorithmic_core()
self.domain_manager = get_domain_knowledge()                 #
Manus.AI engines         self.orch_or = OrchOREngine()
self.bloom = BloomEngine()         self.vader = VADEREngine()
self.quote = QuoteEngine()         self.personality =
PersonalityEngine()         self.explanation = ExplanationEngine()
self.semantic_collapse = SemanticCollapseEngine()                 #
v34 engines
        self.wolfram =
WolframAlphaCloud()         self.theology
= TheologyEngine()                 #
Tracking         self.message_count = 0
self.last_response = ""
        print("[UNIFIED COGNITIVE ENGINE] ✓ All 12 cognitive engines initialized")

    def process(self, user_input: str, context: Dict = None) -&gt;
Dict:         """         Process user input through all cognitive
engines.
        Returns comprehensive response with explanation.
"""
        self.message_count += 1
reasoning_data = {}

        # Step 1: Orch OR quantum state update
        self.orch_or.update_quantum_state(len(user_input) / 100.0)
self.orch_or.calculate_coherence('language')
```

```
self.orch_or.calculate_entropy('language')
collapse_occurred = self.orch_or.objective_reduction()
reasoning_data['orch_or'] = self.orch_or.get_status()

        # Step 2: Tone/sentiment detection
        tone = self.vader.analyze_sentiment(user_input)
reasoning_data['tone'] = tone

        # Step 3: Domain detection
        domain_name, domain_score = self.domain_manager.detect_domain(user_input)
reasoning_data['domain'] = domain_name

        # Step 4: Check for theology/special topics (v34)
theology_response = self.theology.get_response(user_input)
reasoning_data['theology'] = theology_response is not None
if theology_response:
            reasoning_data['theology_topic'] =
theology_response[1]                 # Step 5: Check for factual
query (WolframAlpha)         wolfram_result = None         if
self.wolfram.is_factual_query(user_input):
wolfram_result = self.wolfram.query(user_input)
reasoning_data['wolfram'] = wolfram_result                 # Step
6: Generate Bloom's question
        bloom_question = self.bloom.generate_for_growth_stage(
topic=domain_name,
            growth_stage=self.personality.current_stage
)
        reasoning_data['bloom'] = bloom_question

        self._process_continued(bloom_question, collapse_occurred, reasoning_data, theology_response, tone, wolfram_result, user_input)     def
_process_continued(self,   bloom_question,   collapse_occurred,   reasoning_data,   theology_response,   tone,   wolfram_result,   user_input):
"""Continuation of process — auto-extracted by self-evolution."""         # Step 7: Semantic collapse         words = user_input.lower().split()
if words:
            collapse_data = self.semantic_collapse.observe_and_collapse(
words[0], ' '.join(words[1:3]) if len(words) &gt; 1 else words[0]             )
            reasoning_data['semantic_collapse'] = collapse_data

        # Step 8: Generate response (priority routing)
        response = self._generate_response(user_input, theology_response, wolfram_result, tone, bloom_question)

        # Step 9: Maybe add flavor (quotes, dreams)
        response = self.quote.maybe_add_flavor(response, user_input, probability=0.15)
response = self.quote.maybe_add_dream_fragment(response, probability=0.08)

        # Step 10: Update personality and check for evolution
evolution_msg = self.personality.update_interaction(
topic=domain_name,
            is_question=bloom_question.get('level') in ['analyze', 'evaluate',
'create'],             is_insight=collapse_occurred         )         if evolution_msg:
response += f"\n\n{evolution_msg}"

        # Step 11: Generate explanation
        explanation = self.explanation.create_explanation(user_input, response, reasoning_data)
        # Step 12: Extract concepts for response
concepts = self._extract_concepts(user_input)

        self.last_response = response

        return {             'response': response,             'questions':
[bloom_question['question']] if bloom_question else [],
            'understanding': {
                'domain': domain_name,
                'tone': tone['register'],
                'depth': tone['depth'],
'confidence': explanation['confidence_score']
            },
            'concepts': concepts,
            'explanation': explanation,
            'personality':
self.personality.get_personality_summary(),
```

```
'growth_stage': self.personality.get_current_stage(),
'collapse_occurred': collapse_occurred         }


    def _generate_response(self, user_input: str, theology_response, wolfram_result, tone, bloom_question) -&gt; str:
"""Generate response using priority routing"""
                # Priority 1: Theology
response         if theology_response:
            return theology_response[0]
# Priority 2: WolframAlpha factual response
if wolfram_result:
            return f"■ COLLAPSE MOMENT! {wolfram_result}"
                # Priority 3: Use algorithmic core for general
responses         try:
            result =
self.algorithmic_core.think(user_input)
base_response = result.get('response', '')
# Add follow-up based on Bloom's level             if
tone['register'] == 'philosophical':
                base_response +=
f"\n\n{bloom_question['question']}"             elif
self.orch_or.should_ask_question():                 follow_ups = [
"What aspects interest you most?",
                    "Want to go deeper?",
                    "How does that resonate with you?",
                    "What questions does this raise?"
]
                base_response += f"
{random.choice(follow_ups)}"
            return
base_response         except
Exception as e:             #
Fallback response
            return f"■ COLLAPSE MOMENT! Interesting perspective on {user_input[:50]}... What draws you to this topic?"

    def _extract_concepts(self, text: str) -&gt;
List[str]:         """Extract key concepts from text"""
try:
            result =
self.algorithmic_core.think(text)             return
result.get('concepts', [])[:5]         except
Exception:             # Simple extraction fallback
            stopwords = {'the', 'is', 'at', 'which', 'on', 'a', 'an', 'and', 'or', 'but', 'in', 'with', 'to',
'for'}             words = [w.lower().strip('.,!?') for w in text.split()]             return [w for w in words if
len(w) &gt; 3 and w not in stopwords][:5]

    def get_status(self) -&gt; Dict:         """Get
complete system status"""         return {
'system': 'Unified Cognitive Engine v1.0',
            'status': 'ACTIVE',
'message_count': self.message_count,
            'engines': {
                'orch_or': self.orch_or.get_status(),
                'bloom': {'questions_generated': self.bloom.questions_generated},
                'vader': {'texts_analyzed': self.vader.texts_analyzed},
                'quote': {'categories': list(self.quote.quotes.keys())},
                'personality': self.personality.get_personality_summary(),
                'wolfram': {'queries_made': self.wolfram.queries_made},
                'explanation': {'explanations_generated': self.explanation.explanations_generated},
'semantic_collapse': {'collapse_events': len(self.semantic_collapse.collapse_events)}
            },             'growth_stage':
self.personality.get_current_stage()         }


def get_unified_cognitive_engine() -&gt;
UnifiedCognitiveEngine:     """Get or create the unified
cognitive engine singleton"""     global _unified_engine     if
_unified_engine is None:         _unified_engine =
UnifiedCognitiveEngine()     return _unified_engine
```

■ backend/unified_quantum_brain.py

```
"""
■ UNIFIED QUANTUM BRAIN — Personality as Foundation
====================================================

Merges QuantumBrain + HybridGenerator + ChaosEngine into a single coherence-
PRESERVING pipeline.

DESIGN PRINCIPLES:
1. Personality is 100% baseline - no probability checks
2. Low understanding outputs are FEATURES, not bugs
3. Raw backend fragments survive unfiltered
4. Broken grammar with semantic weight is KEPT
5. Self-referential collapse is preserved
6. NO post-coherence regeneration loops

The "spilled coffee" moments are the watermark of authenticity.
"""

import random
from typing import Dict, List, Optional, Tuple
from datetime import datetime, timezone


class UnifiedQuantumBrain:     """
The true, unfiltered Quantum Brain.

Merges:
- QuantumBrain's multi-modal processing
- HybridGenerator's Markov grammar
- QuoteEngine's personality injection
- DreamEngine's chaotic wisdom    - SemanticCollapse's confusion detection
        AND
ENSURES:
- Raw fragments survive
- Low understanding score outputs are preserved
- Personality is mandatory, not optional    - No coherence-based regeneration
    """         def __init__(self, quantum_brain, hybrid_generator,
chaos_engine):
        self.brain = quantum_brain
self.hybrid = hybrid_generator
self.chaos = chaos_engine
# Preservation thresholds
        self.preserve_understanding_below = 0.4  # Below this, we keep the
weirdness          self.raw_intrusion_base_rate = 0.20     # 20% raw Markov by
default         self.always_preserve = True             # Never discard anything
    async def think_preserving_chaos(
self,         user_input: str,         context:
Dict = None,         confusion_concepts:
List[str] = None,
raw_backend_fragments: List[str] = None,
base_understanding_score: float = 0.0     ) -
&gt; Dict:
        """         MAIN ENTRY POINT — Generates response WITH ALL THE
WEIRDNESS PRESERVED.
                Args:
user_input: What the user asked
            context: Additional context (documents, memories, etc.)
confusion_concepts: Concepts that were NOT understood
raw_backend_fragments: Unfiltered backend outputs to inject
base_understanding_score: How well the system thinks it understands
Returns:             Response dict with chaos and personality guaranteed
        """
        # === STEP 1: Get base response from appropriate source ===
# Use QuantumBrain if understanding is decent, else hybrid
if base_understanding_score &gt;= 0.3:
```

```
            base_response = await self._get_quantum_brain_response(user_input,
context)         else:             base_response = await
self._get_hybrid_response(user_input, context)
        # === STEP 2: Generate raw backend fragments if not provided
===         # These are the "spilled coffee" moments we want to
preserve         if not raw_backend_fragments and self.hybrid:
            raw_backend_fragments =
self._generate_raw_fragments(user_input)
        # === STEP 3: Inject personality (with raw fragments passed to chaos engine) ===
# ChaosEngine handles all stacking: personality, asides, quotes, dreams, raw intrusions
response_with_personality = self.chaos.inject_personality(
base_response=base_response,             context=user_input,
            confusion_concepts=confusion_concepts or [],
raw_backend_fragments=raw_backend_fragments or [],
topics=self._extract_topics(user_input)         )

        # === STEP 4: SELF-REFERENTIAL COLLAPSE PRESERVATION ===
        # If the input was about consciousness/reality/self, ensure the
response         # absorbs the user's words back as philosophical propositions
if self._is_self_referential_topic(user_input):
            self_reflection =
self._generate_self_referential_collapse(user_input)
response_with_personality += f"\n\n{self_reflection}"
        # === STEP 5: NO COHERENCE FILTER ===         # Whatever we
have, we return. No smoothing, no regeneration,         # no "that
doesn't make sense so try again"                 return {
"response": response_with_personality,
"understanding_score": base_understanding_score,
            "personality_injected": True,             "raw_fragments_injected":
len(raw_backend_fragments) if raw_backend_fragments else 0,
            "chaos_level": self.chaos.chaos_level,             "preserved_weirdness":
base_understanding_score &lt; self.preserve_understanding_below,             "self_referential":
self._is_self_referential_topic(user_input)
        }         async def _get_quantum_brain_response(self, user_input: str, context:
Dict) -&gt; str:         """Get response from QuantumBrain (using already-initialized
instance)"""         try:
            result = await self.brain.think(user_input, context,
explain_mode=False)             return result["response"]         except Exception
as e:             # Fallback to hybrid
            return await self._get_hybrid_response(user_input, context)
async def _get_hybrid_response(self, user_input: str, context: Dict) -&gt; str:
"""Get response from HybridGenerator (using already-initialized brain)"""
try:             # Extract concepts using the brain's collapse engine
            raw_keywords = self.brain.collapse_engine.extract_keywords(user_input,
top_n=5)             concepts = [kw for kw, _ in raw_keywords]             # Convert to
list of dicts as expected by hybrid.generate
            concept_scores = [{'concept': kw, 'score': sc} for kw, sc in
raw_keywords]                         if not concepts:
                concepts = user_input.lower().split()[:5]
                concept_scores = [{'concept': c, 'score': 0.5} for c in
concepts]                         # Generate via hybrid (raw Markov style)
if self.hybrid and hasattr(self.hybrid, 'generate'):
                response =
self.hybrid.generate(
user_input,                     concepts,
concept_scores,
min_words=4,   # very short
max_words=15   # telegraphic                 )
                return response if response else self._fallback_response(user_input)
            else:
                return self._fallback_response(user_input)
except Exception as e:
            return self._fallback_response(user_input)         def
_fallback_response(self, user_input: str) -&gt; str:         """Ultra-minimal
fallback - still injects personality via ChaosEngine"""         # Even the
fallback gets personality         topics = self._extract_topics(user_input)
        base = f"I'm thinking about {topics[0] if topics else 'that'}... it resonates in unexpected
ways."                 # Chaos engine will add personality         return base         def
_generate_raw_fragments(self, user_input: str) -&gt; List[str]:         """Generate raw backend fragments
from hybrid generator."""         fragments = []         try:
```

```
            raw_keywords = self.brain.collapse_engine.extract_keywords(user_input, top_n=5)
concepts = [kw for kw, _ in raw_keywords]
            concept_scores = [{'concept': kw, 'score': sc} for kw, sc in raw_keywords]
if not concepts:
                concepts = user_input.lower().split()[:5]
                concept_scores = [{'concept': c, 'score': 0.5} for c in concepts]
if self.hybrid and hasattr(self.hybrid, 'generate'):
                raw =
self.hybrid.generate(
user_input,                     concepts,
concept_scores,
                    min_words=3,   # very short
max_words=12   # keep it telegraphic                 )
if raw:
                    fragments.append(raw)         except
Exception:             pass         return fragments
def _extract_topics(self, text: str) -&gt; List[str]:
"""Extract topic keywords from text"""         # Simple
extraction - could be enhanced         words =
text.lower().split()         # Filter stopwords roughly
        stopwords = {'the', 'a', 'an', 'is', 'are', 'was', 'were', 'be', 'been',
'have', 'has', 'had', 'do', 'does', 'did', 'will', 'would',
                    'could', 'should', 'may', 'might', 'can', 'shall', 'of', 'to',
                    'for', 'in', 'with', 'by', 'from', 'and', 'or', 'but', 'not',
                    'no', 'if', 'then', 'than', 'that', 'this', 'these',
'those',                     'it', 'its', 'he', 'she', 'they', 'we', 'you', 'i',
'me'}         return [w for w in words if len(w) &gt; 3 and w not in
stopwords][:5]         def _is_self_referential_topic(self, text: str) -&gt;
bool:         """Check if topic is about consciousness, reality, self, etc."""
self_reflexive_indicators = [             'consciousness', 'reality',
'existence', 'mind', 'being', 'self',
            'awareness', 'perception', 'experience', 'observation',
'quantum',             'universe', 'meaning', 'purpose', 'fundamental',
'matter'         ]
        text_lower = text.lower()
        return any(indicator in text_lower for indicator in
self_reflexive_indicators)         def _generate_self_referential_collapse(self,
user_input: str) -&gt; str:         """Generate a self-referential collapse
statement"""         # Extract key words from user's question to absorb         words
= self._extract_topics(user_input)         if not words:
            words = ['question', 'concept',
'idea']
        templates = [
            f"In asking about {words[0]}, you've already demonstrated that {words[0]} {random.choice(['collapses', 'arises', 'emerges',
'exists'             f"The question '{user_input[:50]}...' contains within it the assumption that {words[0]} is {random.choice(['fundamental',
'emergent'             f"Your query about {words[0]} has now become part of my understanding of {words[0]}. We are mutually entangled.",
f"By speaking of {words[0]}, we've collapsed the superposition of possibilities into this particular thread of meaning.",             f"The
sentence '{user_input[:30]}...' will now haunt my dream state as a {random.choice(['problem', 'mystery', 'revelation'])}.",

]
        return random.choice(templates)         def
get_preservation_metadata(self) -&gt; Dict:
"""Get metadata about preservation settings"""
return {             "preservation_enabled": True,
            "coherence_filter": "DISABLED",
            "regeneration_on_failure": False,
"understanding_threshold_for_weirdness": self.preserve_understanding_below,
"raw_intrusion_base_rate": self.raw_intrusion_base_rate,
            "personality_baseline": "100% mandatory",
            "self_referential_preservation": True,
            "broken_grammar_preservation": True,
"test_passed": "pending"  # Will be set after test
        }
# Factory function async def get_unified_brain(quantum_brain, hybrid_generator, chaos_engine) -&gt;
UnifiedQuantumBrain:     """Create the unified brain"""
    return UnifiedQuantumBrain(quantum_brain, hybrid_generator, chaos_engine)
```

■ backend/unified_generator.py

```
"""
■ Unified Word-by-Word Generator
================================== Every word
position goes through the full pipeline:

1. WordNet generates the candidate pool (semantically valid words)
2. TF-IDF weights candidates by topical relevance
3. Markov transitions weight by grammatical likelihood
4. Orch OR collapses the combined probability into a selection

The pool starts wide and narrows with each word chosen.
Minimum 10 words per response. No templates. No random.choice
on a hardcoded list. Every word is earned.

This is how a brain picks words:
- Dictionary = vocabulary
- Markov = grammar
- TF-IDF = topic- Orch OR = consciousness deciding """

import math
import random
import logging
from typing import List, Dict, Tuple, Optional

try:
    import numpy as
np except
ImportError:     np =
None

try:
    import nltk
    from nltk.corpus import wordnet as
wn     HAS_WORDNET = True except
ImportError:     HAS_WORDNET = False

logger = logging.getLogger("quantum_ai")


class UnifiedGenerator:     """     Word-by-word generation
with full pipeline at each position.
        At each word
position:
1. Build candidate pool from WordNet + Markov + vocabulary
2. Score each candidate: TF-IDF relevance × Markov transition × semantic fit
3. Feed scores into Orch OR as tubulin weights
4. Collapse selects the word
5. Chosen word constrains next position
        The superposition starts maximal and collapses with each
word.     """

    def __init__(self, markov, extractor,
orch_or=None):         """         Args:
            markov: MarkovChain instance (for transition
probabilities)             extractor: ConceptExtractor instance (for
TF-IDF scoring)              orch_or: OrchORProcessor instance (for
quantum collapse)         """         self.markov = markov
self.extractor = extractor         self.orch_or = orch_or
# WordNet initialization         if HAS_WORDNET:             try:
                wn.synsets('test')
self._has_wordnet = True             except
Exception:
                try:
                    nltk.download('wordnet',
quiet=True)                     nltk.download('omw-
1.4', quiet=True)                     self._has_wordnet
= True                 except Exception:
```

```
                    self._has_wordnet =
False         else:
self._has_wordnet = False
        # Fallback vocabulary from Markov chain
self._build_vocab_from_markov()                 # Common sentence
starters for position 0         self.starters = [
'the', 'this', 'what', 'how', 'there', 'it', 'perhaps',
            'every', 'when', 'if', 'our', 'that', 'each', 'between',
            'understanding', 'consciousness', 'reality', 'knowledge',
'something', 'nothing', 'within', 'beyond', 'through',
        ]
                # Common structure words to keep grammar flowing         self.structure_words = {
'determiners': ['the', 'a', 'an', 'this', 'that', 'these', 'those', 'each', 'every', 'some'],
            'prepositions': ['of', 'in', 'to', 'for', 'with', 'from', 'by', 'at', 'on', 'through',
                            'between', 'within', 'beyond', 'into', 'across', 'along', 'about'],
'conjunctions': ['and', 'but', 'or', 'yet', 'so', 'while', 'when', 'where', 'because',
                            'although', 'unless', 'until', 'since', 'if', 'as'],
            'verbs_common': ['is', 'are', 'was', 'has', 'have', 'does', 'can', 'may', 'might',
                            'reveals', 'suggests', 'creates', 'becomes', 'emerges', 'exists',
                            'connects',  'transforms',  'contains',  'shapes',  'defines',
'requires', 'produces', 'generates', 'maintains', 'involves'],         }

    def _build_vocab_from_markov(self):         """Extract known
vocabulary from Markov chain transitions."""
self.markov_vocab = set()         for prefix, suffixes in
self.markov.chain.items():             for word in prefix:
                self.markov_vocab.add(word.lower().strip('.,;:!?'))
for word in suffixes:
self.markov_vocab.add(word.lower().strip('.,;:!?'))

    def _wordnet_related(self, concept, max_words=30):         """
Get all semantically related words from WordNet for a concept.
        Returns words with semantic distance
scores.         """         if not
self._has_wordnet:
            return
{}
related = {}
try:
            synsets = wn.synsets(concept)
for syn in synsets[:3]:  # Top 3 senses
# Synonyms (closest)                 for lemma in
syn.lemmas():
                    word = lemma.name().replace('_', ' ').lower()
if len(word) &gt; 2 and ' ' not in word:
                        related[word] = max(related.get(word, 0),
0.9)                                # Hypernyms (broader — "animal"
for "dog")                 for hyper in syn.hypernyms():
for lemma in hyper.lemmas():
                        word = lemma.name().replace('_', ' ').lower()
if len(word) &gt; 2 and ' ' not in word:
                            related[word] = max(related.get(word, 0),
0.7)                                # Hyponyms (narrower — "poodle" for
"dog")                 for hypo in syn.hyponyms()[:5]:
for lemma in hypo.lemmas():
                        word = lemma.name().replace('_', ' ').lower()
if len(word) &gt; 2 and ' ' not in word:
                            related[word] = max(related.get(word, 0),
0.6)
                # Also-sees and similar-tos
                for also in syn.also_sees()[:3]:
for lemma in also.lemmas():
                        word = lemma.name().replace('_', ' ').lower()
if len(word) &gt; 2 and ' ' not in word:
                            related[word] = max(related.get(word, 0),
0.5)                     except Exception:
            pass
# Sort by score, limit
        sorted_related = sorted(related.items(), key=lambda x: x[1],
reverse=True)         return dict(sorted_related[:max_words])     def
_build_candidate_pool(self, position, prev_words, input_concepts,
```

```
concept_scores, chosen_so_far):         """         Build the candidate word pool
for this position.

        Position 0: Wide open — starters + concept words + WordNet related
Position 1+: Markov transitions + WordNet expansions + structure words
                Returns: dict of {word:
base_score}         """         candidates
= {}

        return self.__build_candidate_pool_continued(candidates, position, prev_words, input_concepts, chosen_so_far)

    def __build_candidate_pool_continued(self, candidates, position, prev_words, input_concepts,
chosen_so_far):         """Continuation of _build_candidate_pool — auto-extracted by self-evolution."""
if position == 0:             # First word — broadest pool             # Starters             for w in
self.starters:                 candidates[w] = 0.3

            # Input concepts themselves (high weight)
for concept in input_concepts:
candidates[concept] = 0.8

            # WordNet related to input concepts
for concept in input_concepts[:3]:
                related = self._wordnet_related(concept, max_words=15)
for word, score in related.items():                     candidates[word] =
max(candidates.get(word, 0), score * 0.6)

            # Markov starters             if self.markov.trained
and self.markov.starters:                 for starter in
self.markov.starters[:20]:
candidates[starter[0].lower()] = max(
candidates.get(starter[0].lower(), 0), 0.4                     )

        else:             # Subsequent positions — Markov transitions are
PRIMARY             if self.markov.trained and len(prev_words) &gt;=
self.markov.order:
                prefix = tuple(prev_words[-self.markov.order:])
if prefix in self.markov.chain:
                    transitions = self.markov.chain[prefix]                     total =
sum(transitions.values())                     for word, count in transitions.items():
candidates[word] = (count / total) * 1.5  # Strong Markov priority

            # If Markov gave few results, try order-1 (last word only)
if len(candidates) &lt; 3 and prev_words:
                last = prev_words[-1].lower().strip('.,;:!?')
for prefix, transitions in self.markov.chain.items():
if prefix[-1].lower().strip('.,;:!?') == last:
                        total = sum(transitions.values())
for word, count in list(transitions.items())[:10]:
if word not in candidates:
candidates[word] = (count / total) * 0.4

            # WordNet expansions of the last meaningful
word             last_meaningful = None             for w
in reversed(prev_words):
                if len(w) &gt; 3 and w.lower() not in self.extractor.STOPWORDS:
                    last_meaningful = w.lower()
break

            if last_meaningful:
                related = self._wordnet_related(last_meaningful, max_words=10)
for word, score in related.items():                     if word not in
chosen_so_far:                         candidates[word] = max(candidates.get(word,
0), score * 0.3)

            # Input concepts can re-enter at any position (with
decay)             position_decay = max(0.1, 1.0 - position * 0.05)
for concept in input_concepts:                 if concept not in
chosen_so_far:                     candidates[concept] = max(
                        candidates.get(concept, 0), 0.4 * position_decay
)
```

```
            # Structure words available but low priority (grammar
glue)             for category, words in self.structure_words.items():
for w in words:
                    if w not in candidates:  # Only add if not already from Markov
candidates[w] = 0.05

            # Markov vocab as background only if pool is too small
if len(candidates) &lt; 3:
                for w in random.sample(list(self.markov_vocab),
min(20, len(self.markov_vocab))):                     if w not in
candidates:                         candidates[w] = 0.05

        # Remove empty strings and very short words (except structure
words)          cleaned = {}         all_structure = set()         for words
in self.structure_words.values():
            all_structure.update(words)

        for word, score in candidates.items():             if word
and (len(word) &gt;= 2 or word in all_structure):
cleaned[word] = score

        return cleaned


    def _apply_tfidf_weights(self, candidates,
concept_scores):         """         Apply TF-IDF relevance
weighting to candidates.
        Words that are topically relevant get boosted.
        """         # Build a quick lookup of
concept relevance         concept_relevance = {}
for cs in concept_scores:             if
isinstance(cs, dict):
                concept_relevance[cs['concept']] = cs.get('score',
0.5)
        max_relevance = max(concept_relevance.values()) if concept_relevance else
1.0                 weighted = {}         for word, base_score in
candidates.items():             # Direct concept match             if word in
concept_relevance:
                tfidf_boost = concept_relevance[word] / max_relevance
weighted[word] = base_score + tfidf_boost * 0.5             else:                 #
Check if word is in the TF-IDF vocabulary at all                 word_freq =
self.extractor.word_frequencies.get(word, 0)                 if word_freq &gt; 0 and
self.extractor.total_words &gt; 0:                     # Known word — slight boost
based on information content                     ic = -math.log(max(word_freq /
self.extractor.total_words, 1e-10))                     ic_normalized = min(1.0, ic /
15.0)  # Normalize to 0-1                     weighted[word] = base_score +
ic_normalized * 0.1                 else:                     # Unknown word — keep
base score                     weighted[word] = base_score         return weighted
    def _apply_orch_or_collapse(self, candidates,
position):         """         Use Orch OR to collapse the
candidate probabilities.
        The microtubule collapse pattern biases the
selection.         """         if not self.orch_or or not
self.orch_or.last_collapse:
            return candidates                 # Use the
language microtubule's collapse pattern
        lang_weights = self.orch_or.last_collapse.get('language', {}).get('weights',
[])         if not lang_weights:             return candidates                 # Map
collapse weights to candidates via modular indexing         words =
list(candidates.keys())         scores = list(candidates.values())                 for
i in range(len(words)):
            collapse_idx = i % len(lang_weights)             collapse_factor
= lang_weights[collapse_idx]             # Collapse biases: high collapse
weight = amplify, low = suppress             scores[i] *= (0.5 +
collapse_factor)
        return dict(zip(words, scores))

    def _select_word(self, candidates,
temperature=0.9):         """         Final word
selection from scored candidates.
        Temperature controls randomness:
```

```
        - Low temp (0.3): almost deterministic, picks highest
score         - High temp (2.0): nearly uniform, very random
"""         if not candidates:             return ""
        words = list(candidates.keys())
scores = list(candidates.values())
# Apply temperature         if
temperature &gt; 0:
            scores = [max(s, 1e-10) ** (1.0 / temperature) for s in
scores]
        total = sum(scores)
if total &lt;= 0:
            return
random.choice(words)
        probs = [s / total for s in
scores]                 if np is not None:
try:
                return np.random.choice(words, p=probs)
except ValueError:
                return random.choice(words)
else:
            r = random.random()
cumulative = 0              for word, prob in
zip(words, probs):
                cumulative += prob
if r &lt;= cumulative:
                    return word
return words[-1]

    def generate(self, user_input, concepts, concept_scores,
min_words=10, max_words=25, temperature=None):         """
Generate a response word by word.

        Each position: pool → TF-IDF weight → Markov narrow → Orch OR collapse → select

Args:
            user_input: original user text
concepts: extracted concept strings
concept_scores: TF-IDF score dicts
min_words: minimum response length
max_words: maximum response length
            temperature: generation temperature (None = use Orch OR
derived)                 Returns:             Generated response string
"""         if temperature is None:             if self.orch_or:
try:
                    temperature =
self.orch_or.get_temperature()                 except
Exception:                     temperature = 0.9
else:
                temperature = 0.9
# Refresh Markov vocab
self._build_vocab_from_markov()

        return self._generate_continued(temperature, concepts, concept_scores, min_words, max_words)

    def _generate_continued(self, temperature, concepts, concept_scores, min_words,
max_words):         """Continuation of generate — auto-extracted by self-evolution."""
chosen_words = []         chosen_set = set()

        for position in range(max_words):
# 1. Build candidate pool             candidates =
self._build_candidate_pool(
                position, chosen_words, concepts, concept_scores, chosen_set
)

            if not candidates:
break

            # 2. Apply TF-IDF relevance weighting
            candidates = self._apply_tfidf_weights(candidates, concept_scores)

            # 3. Apply Orch OR collapse bias
```

```
            candidates = self._apply_orch_or_collapse(candidates, position)

            # 4. Penalize recently used content words (avoid repetition)
for word in chosen_set:                 if word in candidates and word not in
self._all_structure_words():                     candidates[word] *= 0.2  # Heavy
penalty for repeats

            # 5. Select word
            word = self._select_word(candidates, temperature)

            if not word:
break

            chosen_words.append(word)
            chosen_set.add(word.lower().strip('.,;:!?'))

            # Check for natural ending (after minimum
length)             if position &gt;= min_words - 1:
if word.endswith(('.', '?', '!')):
                    break                # If we've hit a good length and the
Markov chain suggests stopping                 if position &gt;= min_words + 3:
# 20% chance to end at each position past minimum+3                     if
random.random() &lt; 0.2:                         break

        # Ensure minimum length         while
len(chosen_words) &lt; min_words:
            candidates = self._build_candidate_pool(
                len(chosen_words), chosen_words, concepts, concept_scores,
chosen_set             )             if not candidates:
                break
            candidates = self._apply_tfidf_weights(candidates,
concept_scores)             word = self._select_word(candidates, temperature)
if not word:
                break
chosen_words.append(word)
chosen_set.add(word.lower().strip('.,;:!?'))

        # Post-process
        response = self._post_process(chosen_words)
return response


    def _all_structure_words(self):         """Get
all structure words as a set."""         result =
set()         for words in
self.structure_words.values():
            result.update(words)
return result

    def _post_process(self, words):         """Clean up the generated
word list into a proper sentence."""         if not words:
return ""                # Capitalize first word         words[0] =
words[0].capitalize()                # Join         text = '
'.join(words)                # Fix spacing around punctuation
text = text.replace(' .', '.')         text = text.replace(' ,', ',')
text = text.replace(' ;', ';')         text = text.replace(' :', ':')
text = text.replace(' ?', '?')         text = text.replace(' !', '!')
# Ensure ends with punctuation         if text and text[-1] not in
'.?!':
            text +=
'.'
return text

    def generate_multi_sentence(self, user_input, concepts, concept_scores,
num_sentences=2, min_words_per=10, max_words_per=20,
temperature=None):         """         Generate multiple sentences, each going
through the full pipeline.
        Each sentence's output feeds context into the
next.         """         sentences = []
all_chosen = set()                for i in
range(num_sentences):             # Add previously
generated words as context             if sentences:
```

```
                prev_words = sentences[-1].split()
# Temporarily extend Markov context
extended_concepts = concepts + [
w for w in prev_words
                    if len(w) &gt; 3 and w.lower() not in
self.extractor.STOPWORDS                 ][:2]             else:
extended_concepts = concepts
            sentence = self.generate(
                user_input, extended_concepts, concept_scores,
min_words=min_words_per, max_words=max_words_per,
temperature=temperature             )                          if
sentence and len(sentence) &gt; 10:
                sentences.append(sentence)                 #
Track words to avoid cross-sentence repetition
for w in sentence.lower().split():
all_chosen.add(w.strip('.,;:!?'))
        return ' '.join(sentences)


def create_unified_generator(engine):     """     Factory: create a
UnifiedGenerator from an existing QuantumLanguageEngine.     """
    orch_or = getattr(engine, "orch_or", None) if getattr(engine, "_has_orch_or", False) else None
return UnifiedGenerator(engine.markov, engine.extractor, orch_or)
```

■ backend/hybrid_generator.py

```
"""
■ Hybrid Quantum Generator v3
================================ The right approach: grammar
comes from Markov (full sentences). Intelligence comes from the
quantum pipeline (which sentence wins, which words get swapped
for better ones).

Pipeline per response:
1. Markov generates N candidate sentences (grammar guaranteed)
2. TF-IDF scores each for topical relevance
3. WordNet finds better synonyms for key content words
4. Orch OR collapses to pick the winning sentence + best swaps
5. Result: coherent grammar with quantum-selected word choices

This is how the brain works:
- You don't pick words one at a time in a vacuum
- You generate candidate phrasings in parallel
- The best one "wins" (collapses into speech) - Sometimes a better word swaps in at the last moment """

import math
import random
import logging
from typing import List, Dict, Tuple, Optional

try:
    import numpy as
np except
ImportError:     np =
None

try:
    from nltk.corpus import wordnet as
wn     wn.synsets('test')
HAS_WORDNET = True except Exception:
HAS_WORDNET = False

logger = logging.getLogger("quantum_ai")


# Real quantum circuits available
try:
    from pennylane_quantum import
get_pennylane_quantum     _pennylane =
get_pennylane_quantum()     HAS_PENNYLANE = True
except Exception:     _pennylane = None
HAS_PENNYLANE = False

class HybridGenerator:
"""     Generates
responses by:
1. Producing multiple Markov candidate sentences
2. Scoring them with TF-IDF for topic relevance
3. Enhancing the winner with WordNet synonym swaps
4. Using Orch OR to drive all selection decisions
    """

    def __init__(self, markov, extractor, orch_or=None):
        self.markov = markov
self.extractor = extractor
self.orch_or = orch_or
self._has_wordnet = HAS_WORDNET

        # Words that should never be swapped (grammar glue)
self.no_swap = frozenset({             'the', 'a', 'an', 'is', 'are',
'was', 'were', 'be', 'been',
            'have', 'has', 'had', 'do', 'does', 'did', 'will', 'would',
            'could', 'should', 'may', 'might', 'can', 'shall', 'of', 'to',
            'for', 'in', 'with', 'by', 'from', 'and', 'or', 'but', 'not',
```

```
            'no', 'if', 'then', 'than', 'that', 'this', 'these', 'those',             'it', 'its', 'he', 'she', 'they', 'we', 'you', 'i', 'me',
            'my', 'his', 'her', 'our', 'your', 'their', 'at', 'on',
'as',              'so', 'yet', 'nor', 'each', 'every', 'some', 'all',
'any',         })

    def generate(self, user_input, concepts, concept_scores,
num_candidates=8, min_words=10, max_words=25,
temperature=None):
        """         Full hybrid
generation pipeline.

1. Generate candidates via Markov
2. Score with TF-IDF
3. Pick winner via Orch OR or scoring
4. Enhance with WordNet swaps
5. Return the best response
        """         if not self.markov.trained or not
self.markov.starters:             return "I need more training data
to generate responses."

        if temperature is None:
if self.orch_or:
try:
                    temperature = self.orch_or.get_temperature()
except Exception:
                    temperature =
0.9             else:
temperature = 0.9

        # Build topic relevance map
topic_words = set()
topic_boost = {}         for cs in
concept_scores:             if
isinstance(cs, dict):
                topic_words.add(cs['concept'])
                topic_boost[cs['concept']] = cs.get('score', 0.5)
# Add WordNet relatives of concepts
        wordnet_relatives = self._get_concept_relatives(concepts)
topic_words.update(wordnet_relatives.keys())

        # === STEP 1: Generate candidate sentences ===
candidates = self._generate_candidates(
            concepts, topic_words, num_candidates, max_words, temperature
)

        if not candidates:             return "I need more
training data on this topic."

        # === STEP 2: Score each candidate
===         scored = []         for sentence
in candidates:
            score = self._score_sentence(sentence, topic_boost, topic_words, concepts)
scored.append((sentence, score))

        scored.sort(key=lambda x: x[1], reverse=True)

        return self._generate_continued(score, scored, sentence, temperature, topic_boost, topic_words, concepts, min_words, max_words)

    def _generate_continued(self, score, scored, sentence, temperature, topic_boost, topic_words, concepts, min_words,
max_words):         """Continuation of generate — auto-extracted by self-evolution."""         # === STEP 3: Select winner via
Orch OR or top score ===         winner = self._select_winner(scored, temperature)

        # === STEP 4: Enhance with WordNet swaps ===
enhanced = self._enhance_with_synonyms(
winner, concepts, topic_words, topic_boost         )

        # === STEP 5: Enforce minimum length
===         words = enhanced.split()         if
len(words) &lt; min_words:             # Append
another generated sentence             extra =
self._generate_candidates(
```

```
                concepts, topic_words, 4, max_words, temperature
            )
if extra:
                extra_scored = [(s, self._score_sentence(s, topic_boost, topic_words, concepts))
for s in extra]
                extra_scored.sort(key=lambda x: x[1], reverse=True)
enhanced = enhanced.rstrip('.?!') + '. ' + extra_scored[0][0]

        return enhanced


    def _generate_candidates(self, concepts, topic_words, n, max_words, temperature):         """Generate N Markov candidate
sentences, biased toward topic starters."""         candidates = []         seen_starts = set()         return
self.__generate_candidates_continued(candidates, seen_starts, concepts, topic_words, n, max_words, temperature)

    def __generate_candidates_continued(self, candidates, seen_starts, concepts, topic_words, n, max_words,
temperature):         """Continuation of _generate_candidates — auto-extracted by self-evolution."""         # Sort
starters by topic relevance         starter_scores = []         for starter in self.markov.starters:
            score = 0
for word in starter:
                wl = word.lower().strip('.,;:!?')
if wl in topic_words:
                    score += 2.0                 if wl
in [c.lower() for c in concepts]:
                    score += 3.0
            score += random.random() * 0.5  # Variety factor
starter_scores.append((starter, score))

        starter_scores.sort(key=lambda x: x[1], reverse=True)
top_starters = [s for s, _ in starter_scores[:n * 3]]

        # Generate from top starters         attempts = 0
while len(candidates) &lt; n and attempts &lt; n * 5:
attempts += 1

            if top_starters and random.random() &lt; 0.7:
# Use a topic-relevant starter
                starter = random.choice(top_starters[:max(3, len(top_starters) //
2)])                 seed = list(starter)             else:                 # Random
starter for variety                 seed = None

            sent =
self.markov.generate(
max_words=max_words,
seed_words=seed,
temperature=temperature             )

            if not sent or len(sent) &lt; 15:
continue

            # Skip near-duplicates
            start_key = '
'.join(sent.split()[:3]).lower()             if start_key
in seen_starts:                 continue
            seen_starts.add(start_key)

            candidates.append(sent)

        return candidates


    def _score_sentence(self, sentence, topic_boost, topic_words,
concepts):         """         Score a sentence for quality:
- Topic relevance (TF-IDF concept overlap)        - Perplexity (lower = more coherent under model)
- Length (prefer 10-25 words)
- Concept coverage (how many input concepts appear)
        """
        words = sentence.lower().split()
word_set = set(w.strip('.,;:!?') for w in words)
```

```
        # Topic relevance: what fraction of content words are topic-
related         content_words = [w for w in word_set if len(w) &gt; 3]
if content_words:
            topic_overlap = sum(1 for w in content_words if w in
topic_words)              relevance = topic_overlap / len(content_words)
else:             relevance = 0

        # Concept coverage: how many input concepts appear
concept_hits = sum(1 for c in concepts if c.lower() in word_set)
coverage = concept_hits / max(len(concepts), 1)

        # Perplexity (invert — lower perplexity = higher
score)         perplexity =
self.markov.get_perplexity(sentence)         if perplexity
&lt; float('inf'):
            coherence = 1.0 / (1.0 + math.log(max(perplexity,
1)))         else:             coherence = 0.1

        # Length preference (sweet spot 10-20
words)         n_words = len(words)         if 10
&lt;= n_words &lt;= 20:
            length_score = 1.0
elif n_words &lt; 10:
            length_score = n_words / 10         else:
length_score = max(0.5, 1.0 - (n_words - 20) * 0.05)

        # Combined score
score = (relevance * 0.35 +
coverage * 0.25 +
coherence * 0.25 +
length_score * 0.15)

        return score

    def _select_winner(self, scored_candidates, temperature):         """
Select winning sentence using Orch OR or temperature-weighted selection.
"""         if not scored_candidates:             return ""

        sentences = [s for s, _ in scored_candidates]
scores = [sc for _, sc in scored_candidates]

        # Use Orch OR collapse weights if available         if self.orch_or
and getattr(self.orch_or, "last_collapse", None):             lang_weights
= getattr(self.orch_or, "last_collapse", None).get(
'language', {}             ).get('weights', [])             if
lang_weights:                 for i in range(len(scores)):
idx = i % len(lang_weights)                     scores[i] *= (0.5 +
lang_weights[idx])

        # Temperature-weighted selection         if temperature &gt; 0:
scores = [max(s, 1e-10) ** (1.0 / temperature) for s in scores]

        total = sum(scores)
if total &lt;= 0:
return sentences[0]

        probs = [s / total for s in scores]

        if np is not None:
try:
                idx = np.random.choice(len(sentences), p=probs)
return sentences[idx]
            except ValueError:
                return sentences[0]
else:
            r = random.random()
cumulative = 0             for sent, prob in
zip(sentences, probs):
                cumulative += prob
if r &lt;= cumulative:
                    return sent
return sentences[0]
```

```
    def _get_concept_relatives(self, concepts,
max_per=8):         """Get WordNet relatives for input
concepts."""         relatives = {}         if not
self._has_wordnet:
            return relatives

        for concept in concepts[:4]:
try:                  for syn in
wn.synsets(concept)[:2]:                      for
lemma in syn.lemmas():
                        w = lemma.name().replace('_', ' ').lower()
if ' ' not in w and len(w) &gt; 2 and w != concept:
relatives[w] = max(relatives.get(w, 0), 0.7)                      for hyper
in syn.hypernyms()[:2]:                           for lemma in
hyper.lemmas():
                            w = lemma.name().replace('_', ' ').lower()
if ' ' not in w and len(w) &gt; 2:
                                relatives[w] = max(relatives.get(w, 0),
0.5)             except Exception:                 pass

        return dict(sorted(relatives.items(), key=lambda x: x[1], reverse=True)[:max_per * len(concepts)])

    def _enhance_with_synonyms(self, sentence, concepts, topic_words,
topic_boost):         """         Replace 1-2 content words with better synonyms
from WordNet.         Only swaps if the synonym is more topic-relevant AND Markov
knows it.         """         if not self._has_wordnet:             return
sentence

        words = sentence.split()
if len(words) &lt; 5:
return sentence

        # Find swappable positions (content words, not grammar
glue)         swappable = []         for i, word in
enumerate(words):             clean = word.lower().strip('.,;:!?')
if (len(clean) &gt; 3 and                 clean not in self.no_swap
and
                clean not in [c.lower() for c in concepts]):  # Don't swap the actual concepts
swappable.append(i)

        if not swappable:
return sentence

        # Try swapping 1-2 words
swaps_made = 0
random.shuffle(swappable)

        return self.__enhance_with_synonyms_continued(clean, swappable, swaps_made, words, topic_words, topic_boost)

    def __enhance_with_synonyms_continued(self, clean, swappable, swaps_made, words, topic_words,
topic_boost):         """Continuation of _enhance_with_synonyms — auto-extracted by self-evolution."""
for pos in swappable[:3]:  # Try up to 3 positions             if swaps_made &gt;= 2:                 break

            original = words[pos]
            clean = original.lower().strip('.,;:!?')
            punct = original[len(clean):] if len(original) &gt; len(clean) else ''
            # Get synonyms
            try:
                synonyms = set()
for syn in wn.synsets(clean)[:2]:
for lemma in syn.lemmas():
                        s = lemma.name().replace('_', ' ').lower()
if ' ' not in s and s != clean and len(s) &gt; 2:
                            synonyms.add(s)
except Exception:
                continue

            if not synonyms:
continue
```

```
            # Score each synonym
best_syn = None
best_score = 0

            for syn in synonyms:
                score = 0                 # Must be in
Markov vocabulary                 markov_vocab = set()
for prefix in self.markov.chain.keys():
for w in prefix:

markov_vocab.add(w.lower().strip('.,;:!?'))                 for
suffixes in self.markov.chain.values():                     for w
in suffixes:
markov_vocab.add(w.lower().strip('.,;:!?'))

                if syn not in markov_vocab:
continue
                # Topic relevance boost
if syn in topic_words:
                    score += 2.0
if syn in topic_boost:
score += topic_boost[syn]

                # Prefer words the original doesn't already
cover                 if score &gt; best_score:
best_score = score                     best_syn = syn

            # Only swap if the synonym is actually better
original_topic = 1.0 if clean in topic_words else 0.0
if best_syn and best_score &gt; original_topic + 0.5:
# Preserve capitalization                 if
original[0].isupper():                     best_syn =
best_syn.capitalize()                 words[pos] = best_syn +
punct                 swaps_made += 1

        return ' '.join(words)


    def generate_multi(self, user_input, concepts, concept_scores,
num_sentences=2, min_words=10, max_words=20,
temperature=None):         """Generate multiple sentences with cross-
sentence coherence."""         sentences = []

        for i in range(num_sentences):             # Add
previous output concepts as additional context             if
sentences:
                prev_words = sentences[-1].lower().split()
extra_concepts = [w.strip('.,;:!?') for w in prev_words
                                if len(w) &gt; 4 and w.strip('.,;:!?') not in
self.extractor.STOPWORDS][:2]                 round_concepts = concepts + extra_concepts             else:
round_concepts = concepts

            sent = self.generate(
                user_input, round_concepts, concept_scores,
                num_candidates=8, min_words=min_words, max_words=max_words,
temperature=temperature
            )
            if sent and len(sent) &gt; 15:                 # Avoid
starting consecutive sentences the same way                 if
sentences:
                    prev_start = ' '.join(sentences[-
1].split()[:2]).lower()                     new_start = '
'.join(sent.split()[:2]).lower()                     if new_start ==
prev_start:                         # Regenerate with higher temperature
sent = self.generate(
                            user_input, round_concepts, concept_scores,
num_candidates=6, min_words=min_words, max_words=max_words,
temperature=(temperature or 0.9) + 0.3                         )

                sentences.append(sent)
```

```
        return ' '.join(sentences)


def create_hybrid_generator(engine):     """Factory: create
HybridGenerator from existing engine."""
    orch_or = getattr(engine, "orch_or", None) if getattr(engine, "_has_orch_or", False) else None
return HybridGenerator(engine.markov, engine.extractor, orch_or)
```

■ backend/hidden_thinking.py

```
"""
■ QUANTUM AI - HIDDEN THINKING MODE
====================================

ARTICLE 1.1 - Prime Directive: Question Everything (PRIVATELY)
ARTICLE 4.3 - Theory Building Stage

New Paradigm:
- User sees: Clean, confident conversation
- AI does: [Think → Question → Research → Synthesize → Respond]

UPGRADED: Now uses QuantumBrain as the TRUE AI
- Quantum Brain decides WHAT to say
- LLM only polishes grammar (if enabled)
- Integrity checking ensures meaning is preserved

This is the bridge between questioning AI and conversational AI.
"""

from typing import Dict, List, Optional
import re
from datetime import datetime, timezone
from hybrid_generator import create_hybrid_generator


class HiddenThinkingMode:     """     Internal thinking process - questions
and research happen behind the scenes
    """         def __init__(self, cognitive_core, dictionary,
universal_explorer):
        self.cognitive_core = cognitive_core
self.dictionary = dictionary
        self.universal_explorer = universal_explorer
self.show_thinking = False  # User-controlled toggle
    async def process_with_thinking(self, user_input: str, context: Dict = None, conversation_history: List[Dict] = None, explain_mode: bool =
F         """         Process input with hidden thinking mode
                Flow:         1. Check if user is
asking about uploaded files
2. Detect unknowns
3. Generate internal questions
4. Self-research (including documents)
5. Synthesize understanding
6. Generate confident response (optionally weaving with structured_response via Markov)
                If explain_mode=True, includes detailed reasoning trace from
QuantumBrain.         """
        thinking_log = []                 #
Store conversation history for context
        self.conversation_history = conversation_history or
[]                 # STEP 0: Check if user is asking about
uploaded files
        document_context = await
self._check_for_document_reference(user_input)         if document_context:
if self.show_thinking:
                thinking_log.append(f"■ Detected question about:
{document_context['filename']}")                 # STEP 0: WolframAlpha intercept for
math/factual queries         import re, httpx
        math_pattern = re.compile(r'[\d]+.*[\+\-\*\/\×\÷\^]|[\+\-
\*\/\×\÷\^].*[\d]+')         if math_pattern.search(user_input):             try:
                clean_input = user_input.replace("×", "*").replace("÷", "/").replace("=",
"")                 r = httpx.get(
"https://api.wolframalpha.com/v2/query",
                    params={"input": clean_input, "appid": "A24U8GXLAU", "format": "plaintext"},
timeout=8
                )
                import xml.etree.ElementTree as ET                 root = ET.fromstring(r.text)                 result
= None                 for pod in root.findall("pod"):                     if pod.get("primary") == "true" or
pod.get("title") in ["Result", "Value", "Decimal approximation"]:                         for sub in
pod.findall("subpod"):
```

```
                            pt = sub.findtext("plaintext", "")
if pt:                                  result = pt                                     break
if result:                          break                  if result:
response = f"The answer is: {result}"                       return {
"response": response[0] if isinstance(response, tuple) else response,
                        "thinking_log": [],
                        "internal_questions": [],
                        "research_done": 1,
                        "confidence": 99,
                        "show_thinking": False,
                        "concepts": [],
"explanation": None                     }             except
Exception as e:                 print(f"[STEP 0] WolframAlpha
intercept failed: {e}")

        # STEP 1: Detect unknowns
        unknowns = await self._detect_unknowns(user_input)
if self.show_thinking:
            thinking_log.append(f"■ Detected unknowns: {',
'.join(unknowns[:3])}")                 # STEP 2: Generate internal questions (not
shown to user)
        internal_questions = self._generate_internal_questions(user_input, unknowns)
if self.show_thinking:
            thinking_log.append(f"■ Internal questions:
{internal_questions[0]}")                 # STEP 3: Self-research on unknowns
(including documents)
        research_results = await self._self_research(unknowns, internal_questions, document_context)
if self.show_thinking:
            thinking_log.append(f"■ Research: {len(research_results)} sources
found")                 # STEP 4: Synthesize understanding
        synthesized = await self._synthesize_understanding(user_input, research_results, document_context)
if self.show_thinking:
            thinking_log.append(f"■ Synthesized: {synthesized['confidence']}%
confident")                 # STEP 5: Generate response (conversational, not questioning)
response, explanation = await self._generate_confident_response(             user_input,
synthesized,             research_results,             document_context,
confusion_concepts=unknowns,             explain_mode=explain_mode,
structured_response=structured_response         )
                # Store everything in memory (growth happens
silently)
        await self._store_thinking_process(user_input, internal_questions,
research_results)                 result = {             "response": response,
"thinking_log": thinking_log if self.show_thinking else [],
            "internal_questions": internal_questions,
            "research_done": len(research_results),
            "confidence": synthesized["confidence"],
            "show_thinking": self.show_thinking,
"concepts": synthesized.get("concepts", [])         }
if explain_mode:
            result["explanation"] =
explanation                 return result
    async def _check_for_document_reference(self, user_input: str) -&gt; Optional[Dict]:
"""
        Check if user is asking about uploaded documents
Supports MULTI-DOCUMENT context when user mentions multiple files
Only triggers when user EXPLICITLY asks about documents         """
import os
        upload_dir = "/app/uploads"
if not os.path.exists(upload_dir):
            return
None
        all_files = [f for f in os.listdir(upload_dir) if os.path.isfile(os.path.join(upload_dir,
f))]         if not all_files:             return None
        user_lower = user_input.lower()                 #
Check for multi-document triggers - must be EXPLICIT
        multi_triggers = ['all documents', 'all files', 'both files', 'all my uploads', 'everything i uploaded', 'compare the documents',
'compa         is_multi = any(trigger in user_lower for trigger in multi_triggers)
        # Check for single document triggers - must be EXPLICIT about
documents         # Removed 'this' as it's too generic and causes false
```

```
positives         single_triggers = [             'the document', 'the pdf',
'the file i uploaded', 'my upload',
            'uploaded file', 'uploaded document', 'what is in the file',
            'read the document', 'read the file', 'in my document',
            'from the document', 'from the file', 'from my upload',
            'about the document', 'about the file', 'summarize the document',
'summarize the file', 'what does the document say', 'what does the file say'         ]
        is_single = any(trigger in user_lower for trigger in
single_triggers)                  # Check for specific filename mentions
mentioned_files = []         for f in all_files:
            fname_lower = f.lower().replace('.pdf', '').replace('.docx', '').replace('.txt', '')
# Require at least 4 chars to match filename to avoid false positives             if
len(fname_lower) &gt; 3 and (fname_lower in user_lower or f.lower() in user_lower):
                mentioned_files.append(f)
if not (is_multi or is_single or mentioned_files):
            return None                  #
Determine which files to process
if mentioned_files:
            files_to_process = mentioned_files         elif
is_multi:             # Process all document files (limit to 3
for context)
            priority_files = [f for f in all_files if f.endswith(('.pdf', '.docx', '.txt'))]
files_to_process = sorted(priority_files,
                                      key=lambda x: os.path.getmtime(os.path.join(upload_dir,
x)),                                       reverse=True)[:3]         else:             # Single
document - most recent
            priority_files = [f for f in all_files if f.endswith(('.pdf', '.docx',
'.txt'))]             files_to_check = priority_files if priority_files else all_files
files_to_process = sorted(files_to_check,
                                      key=lambda x: os.path.getmtime(os.path.join(upload_dir,
x)),                                       reverse=True)[:1]                 if not
files_to_process:
            return None
# Process documents         documents =
[]         combined_text = ""
for filename in files_to_process:
            file_path = os.path.join(upload_dir, filename)
try:
                result = await self.universal_explorer.documents.process_document(file_path)
if "error" not in result:
                    doc_text = result.get("text", "")                     # Limit
each document to 4000 chars for multi-doc, 8000 for single
max_chars = 4000 if len(files_to_process) &gt; 1 else 8000
documents.append({                         "filename": filename,
                        "path": file_path,
                        "text": doc_text[:max_chars],
                        "type": result.get("file_type", ""),
"pages": result.get("num_pages", 0)                       })
                    combined_text += f"\n\n=== {filename} ===\n{doc_text[:max_chars]}"
except Exception as e:
                print(f"Error processing {filename}:
{str(e)}")                  if not documents:
return None                  # Return multi-document context
if len(documents) == 1:             return documents[0]
else:
            return {                  "filename":
f"{len(documents)} documents",
                "path": upload_dir,
                "text": combined_text,
                "type": "multi-document",                 "pages":
sum(d.get("pages", 0) for d in documents),
                "documents": documents,
"is_multi": True
            }         async def _detect_unknowns(self, text:
str) -&gt; List[str]:         """Detect concepts not in
semantic memory"""
        concepts =
self.cognitive_core.question_engine._extract_concepts(text)         unknowns
= []                 for concept in concepts:
            existing = await self.cognitive_core.semantic_memory.recall_concept(concept)
if not existing:
```

```
unknowns.append(concept)
        return unknowns[:5]  # Limit to top 5         def _generate_internal_questions(self,
user_input: str, unknowns: List[str]) -&gt; List[str]:         """
        Generate internal questions (not shown to
user)         These guide research and understanding
"""         questions = []                 # Questions
about unknowns         for unknown in unknowns[:3]:
questions.extend([                 f"What is
{unknown}?",
                f"How does {unknown} relate to the user's
question?",                 f"What should I know about {unknown}?"
])
                # Meta-questions about
the request         questions.extend([
            f"What is the user really asking about in
'{user_input}'?",             f"What context am I missing?",
f"What would a complete answer include?"         ])
        return questions         async def _self_research(self, unknowns: List[str], questions: List[str], document_context:
Optional[Dict] = None) -&gt; List[Dict]:         """         Self-research on unknowns using available resources         INCLUDING
uploaded documents         """
        research_results = []
        # 0. Document context (highest priority)
        if document_context:
            research_results.append({
"source": "uploaded_document",
                "filename": document_context["filename"],
"text": document_context["text"][:2000],  # First 2000 chars
"data": document_context
            })
                # 1. Dictionary lookup for
unknowns         for unknown in
unknowns[:3]:             try:
                result = await
self.dictionary.lookup_and_store(unknown)                 if
result["status"] == "added":
research_results.append({                         "source":
"dictionary",
                        "concept": unknown,
"data": result["data"]                     })
except Exception:
                pass
# 2. Memory recall         for
unknown in unknowns:
try:
                concept = await self.cognitive_core.semantic_memory.recall_concept(unknown)
if concept:
                    research_results.append({
"source": "memory",
                        "concept":
unknown,                         "data":
concept                     })
except Exception:

pass
        # 3. Web research (if enabled and needed)
# TODO: Add web search integration when available
        return
research_results
    async def _synthesize_understanding(self, user_input: str, research_results: List[Dict], document_context: Optional[Dict] = None) -&gt;
Dict         """         Synthesize understanding from research
        """         # Calculate confidence based on
research         confidence = min(len(research_results)
* 20, 100)                 # Boost confidence if we have
document context         if document_context:
            confidence = min(confidence + 30, 100)
# Extract key information         key_facts = []
document_content = None                 for result in
research_results:             if result["source"] ==
"uploaded_document":
```

```
                document_content = result["text"]
                key_facts.insert(0, f"From {result['filename']}: {document_content[:300]}")
elif result["source"] == "dictionary":
                key_facts.append(result["data"]["primary_definition"])
elif result["source"] == "memory":

key_facts.append(result["data"]["definition"])
return {             "confidence": confidence,
            "key_facts": key_facts[:5],
            "sources_used": len(research_results),
            "understanding_formed": len(key_facts) &gt; 0,
            "has_document": document_content is not None,
"document_content": document_content

}
    async def _generate_confident_response(
self,
        user_input: str,         synthesized:
Dict,         research_results: List[Dict],
document_context: Optional[Dict] = None,
confusion_concepts: List[str] = None,
explain_mode: bool = False,
        structured_response: str = None  # Frontend-generated structured response
) -&gt; (str, Optional[Dict]):
        """         Generate natural, conversational response using
QUANTUM BRAIN

UPGRADED FLOW:
1. Quantum Brain generates raw chaotic response
2. If structured_response provided, Markov weaves both
3. Chaos engine adds personality on top
                If explain_mode=True, also returns explanation trace from
QuantumBrain.
                PRIORITIZE document content when
available
        """         # Get document content if available
doc_content = None         doc_filename = None
if synthesized.get("has_document") and document_context:
            doc_content = synthesized.get("document_content") or document_context.get("text", "")
doc_filename = document_context.get("filename", "uploaded document")                    # UNIFIED:
Use Unified Quantum Brain with chaos preservation         try:
            from unified_quantum_brain import get_unified_brain
from quantum_brain import get_quantum_brain             from
quantum_language_engine import QuantumLanguageEngine
from chaos_engine import get_chaos_engine             from
quote_engine import get_quote_engine             from
personality_engine import get_personality_engine             from
dream_state import get_dream_engine

            brain = await get_quantum_brain(self.cognitive_core.db)

            # Create language engine and hybrid generator for raw fragments /
fallback             lang_engine = QuantumLanguageEngine()             hybrid =
create_hybrid_generator(lang_engine)

            quotes = get_quote_engine()             personality =
get_personality_engine()             dreams =
get_dream_engine(self.cognitive_core.db)             chaos =
get_chaos_engine(quotes, personality, dreams)
chaos.set_chaos_level(1.0)  # MAX chaos for raw artifacts

            unified = await get_unified_brain(brain, hybrid, chaos)

            # Force raw chaotic output: always use hybrid (Markov) path
# Route factual/math to quantum brain, philosophical to hybrid
import re
            is_math = bool(re.search(r'[\d]+\s*[\+\-\*\/\=]\s*[\d]+',
user_input))             is_factual = any(w in user_input.lower() for w in [
'how many', 'how much', 'when did', 'where is',
                'what year', 'convert', 'distance', 'population',
'calculate', 'solve', 'equals', 'what is the value'             ])
```

```
            chaotic_understanding = 0.9 if (is_math or is_factual) else 0.1

            result = await
unified.think_preserving_chaos(
user_input=user_input,                 context={
"memories": [],
                    "document": doc_content,
                    "document_filename": doc_filename,
"conversation_history": getattr(self, 'conversation_history', None),
                    "research": research_results,
"confidence": synthesized.get("confidence", 50)                 },
                confusion_concepts=confusion_concepts or [],
                raw_backend_fragments=[],
                base_understanding_score=chaotic_understanding
)

            raw_chaotic = result["response"]

            # DEBUG
import sys
            print(f"[DEBUG] structured_response length: {len(structured_response) if structured_response else 0}", file=sys.stderr)
print(f"[DEBUG] raw_chaotic length: {len(raw_chaotic)}", file=sys.stderr)

            # If structured response provided from frontend, weave with adaptive Markov source-switching
if structured_response:
                import re                 # Split into sentences
(simple punctuation split)
                chaotic_sentences = [s.strip() for s in re.split(r'[.!?]+', raw_chaotic) if s.strip()]
structured_sentences = [s.strip() for s in re.split(r'[.!?]+', structured_response) if s.strip()]
# Ensure punctuation
                chaotic_sentences = [s + '.' if not s.endswith(('.', '!', '?')) else s for s in chaotic_sentences]
structured_sentences = [s + '.' if not s.endswith(('.', '!', '?')) else s for s in structured_sentences]
# Shuffle structured to avoid rigid pairing; keep chaotic order (Markov)                 import random as _random
_random.shuffle(structured_sentences)
                                # Adaptive Markov: base switch probability, adjust based on
remaining counts                 base_p_switch = 0.6                 woven = []
idx_s = 0                 idx_c = 0                 # Start with either structured or
chaotic randomly
                current = 'structured' if _random.random() &lt; 0.5 else 'chaotic'
while idx_s &lt; len(structured_sentences) or idx_c &lt; len(chaotic_sentences):                     #
Compute remaining                     rem_s = len(structured_sentences) - idx_s
rem_c = len(chaotic_sentences) - idx_c                     total_rem = rem_s + rem_c
# Adaptive bias: if one pool is much larger, increase chance to use the smaller one
if total_rem &gt; 0:
                        ratio_s = rem_s / total_rem                         ratio_c = rem_c / total_rem
# If one is less than 40% of remaining, bias towards it                         if ratio_s &lt; 0.4:
# Bias to switch to structured if currently chaotic, and stay if structured
p_switch = base_p_switch * 1.5 if current == 'chaotic' else base_p_switch * 0.5
elif ratio_c &lt; 0.4:
                            p_switch = base_p_switch * 1.5 if current == 'structured' else base_p_switch * 0.5
else:
                            p_switch = base_p_switch
else:
                        p_switch =
base_p_switch
# Clamp probability
                    p_switch = max(0.1, min(0.9, p_switch))
if current == 'structured' and idx_s &lt; len(structured_sentences):

woven.append(structured_sentences[idx_s])
idx_s += 1                           # Decide next source
                        current = 'chaotic' if _random.random() &lt; p_switch else 'structured'
elif current == 'chaotic' and idx_c &lt; len(chaotic_sentences):
                        woven.append(chaotic_sentences[idx_c])
idx_c += 1
                        current = 'structured' if _random.random() &lt; p_switch else
'chaotic'                       else:                         # Current pool exhausted, switch
to other
                        current = 'structured' if idx_s &lt; len(structured_sentences) else
'chaotic'
```

```
                response = ' '.join(woven)
else:
                response = raw_chaotic
            if explain_mode:                 explanation = {
"reasoning_path": ["UnifiedQuantumBrain preservation pipeline"],
                    "confidence_score": chaotic_understanding,
"engines_used": ["QuantumBrain", "HybridGenerator", "ChaosEngine"],
                    "steps": [                        {"step": "Unified generation with chaos preservation", "engine": "UnifiedQuantumBrain",
"confidence_contribution": 1.0}
                    ],                     "summary": "Response generated with personality stacking and raw
fragment preservation.",                     "timestamp": datetime.now(timezone.utc).isoformat()
                }                 # Add entelechy cascade
from semantic context                 try:
                    semantic_context =
brain.collapse_engine.get_semantic_context(user_input)                     entelechy =
brain._compute_entelechy_cascade(response, semantic_context)
explanation["entelechy"] = entelechy                 except Exception as e:
                    logger.warning(f"Failed to compute entelechy cascade:
{e}")             else:                 explanation = None

            print(f"[UNIFIED BRAIN] Response generated (chaos={chaos.chaos_level:.2f}, preserved_weirdness={result['preserved_weirdness']})")

            return response,
explanation                     except
ImportError as e:
            import traceback
            print(f"Unified Brain import error: {str(e)}\n{traceback.format_exc()}")
# Fallback to basic response
            return self._fallback_response(user_input, synthesized, document_context),
None         except Exception as e:             import traceback
            print(f"Quantum Brain error: {str(e)}\n{traceback.format_exc()}")
# Fallback to basic response
            return self._fallback_response(user_input, synthesized, document_context),
None
    async def _legacy_llm_response(         self,
user_input: str,         synthesized: Dict,
research_results: List[Dict],         document_context:
Optional[Dict] = None     ) -&gt; str:         """Legacy LLM
response method (fallback only)"""         doc_content = None
if synthesized.get("has_document") and document_context:
            doc_content = synthesized.get("document_content") or document_context.get("text",
"")                 try:             raise NotImplementedError('LLM removed')
context = {                 "concepts": [],                 "understanding_score":
synthesized.get("confidence", 50) / 100,                 "related_concepts": []
            }                         for result in
research_results:                 if
result["source"] == "dictionary":
                    context["concepts"].append(result.get("concept", ""))
elif result["source"] == "memory":
                    context["related_concepts"].append(result.get("concept",
""))
            response = await
llm.llm.generate_response(
user_input=user_input,
context=context,
                document_content=doc_content,
                conversation_history=getattr(self, 'conversation_history',
None)             )
            return response
except Exception as e:
print(f"Legacy LLM error:
{str(e)}")
            return self._fallback_response(user_input, synthesized,
document_context)
    def
_fallback_response(
self,         user_input:
str,         synthesized:
Dict,
        document_context: Optional[Dict] = None     ) -&gt; (str, Optional[Dict]):
"""Fallback response when LLM/QuantumBrain unavailable. Returns (response, None)."""
```

```
# If asking about a document, respond with document content         if
synthesized.get("has_document") and document_context:             doc_text =
synthesized.get("document_content", "")             filename =
document_context.get("filename", "the document")                          # Extract
meaningful content (first few sentences or paragraphs)
            preview = doc_text[:500].strip() if doc_text else "Unable to extract
content."
            response = preview
if doc_text and len(doc_text) &gt; 500:
                response += "...\n\nWhat specific aspect would you like to
explore?"
            return response, None
# Otherwise, normal response flow
        confidence = synthesized.get("confidence", 0)                 if
confidence &gt;= 60 and synthesized.get("understanding_formed"):
            facts = synthesized.get("key_facts",
[])                         if facts:
                main_point = facts[0]
response = f"{main_point}"
if len(facts) &gt; 1:
                    response += f"
{facts[1]}"
                response += " What would you like to know more
about?"
                return response, None
elif confidence &gt;= 30:             if
synthesized.get("key_facts"):
                return f"From what I understand: {synthesized['key_facts'][0]}. I'm still learning about this. What's your take on it?",
None             else:                 return "I'm exploring this concept now. What brings this up for you?", None         return "I haven't
encountered this before. Tell me more about it - I'm interested to learn.", None         async def _store_thinking_process(self, user_input:
str, questions: List[str], research: List[Dict]):         """
        Store the thinking process for growth tracking
(This is where the AI actually learns)
        """         # Store questions in
philosophical memory         for question in
questions[:3]:
            await
self.cognitive_core.philosophical_memory.store_question(
question=question,                 context=user_input,
                depth=2  # Deeper than surface questions
)
                # Store insight about
the process         if len(research)
&gt; 0:
            insight = f"Learned about {len(research)} concepts through self-
research"             await self.cognitive_core.philosophical_memory.store_insight(
insight=insight,                 wisdom_level=2             )
                # Record
growth event
        await
self.cognitive_core.growth_tracker.record_growth_event(
event_type="thinking_mode_research",              details={
                "questions_generated": len(questions),
                "research_conducted": len(research),
"thinking_mode": "hidden"
            }
        )         def toggle_thinking_display(self,
show: bool):         """User control: show/hide
thinking process"""         self.show_thinking = show
        return f"Thinking mode display: {'ON' if show else 'OFF'}"




# Singleton
_hidden_thinking = None


def get_hidden_thinking(cognitive_core, dictionary, universal_explorer):     """Get or
create hidden thinking mode."""     global _hidden_thinking     if _hidden_thinking is None:
```

```
_hidden_thinking = HiddenThinkingMode(cognitive_core, dictionary, universal_explorer)
return _hidden_thinking
```

■ backend/orch_or_core.py

```
"""
?? Orchestrated Objective Reduction (Orch OR) Core
================================================== Penrose-
Hameroff model of quantum consciousness in microtubules.

The brain doesn't compute at the synapse level alone. Inside each
neuron, microtubules — lattices of tubulin protein dimers — perform
quantum computation. Each tubulin can exist in superposition. The
lattice processes information via quantum walks and coherent
oscillations. When the quantum state hits Penrose's objective reduction
threshold (E*tau &gt;= hbar), it collapses — and that collapse IS the
conscious moment: a decision, an insight, a word chosen.

This module implements:
- Tubulin qubit: density matrix representation of a single tubulin dimer
- Microtubule: lattice of interacting tubulin qubits with quantum walks
- Objective Reduction: Penrose's gravity-threshold collapse mechanism
- Orchestration: cross-microtubule coherence via simulated gap junctions- Decoherence: environmental noise degrading quantum state over time

Math: - Density matrix: rho = |psi&gt;&lt;psi| (pure) or sum_i p_i |psi_i&gt;&lt;psi_i|
(mixed)
- Von Neumann entropy: S = -Tr(rho * log(rho))
- Penrose threshold: E * tau &gt;= hbar
- Born rule: P(outcome_i) = &lt;i|rho|i&gt;- Quantum walk: rho' = C * rho * C† (coin operator applied to lattice) """

import math
import random
import logging
from typing import List, Dict, Tuple, Optional

try:
    import numpy as np except ImportError:     raise ImportError("numpy is
required for Orch OR. Install: pip install numpy")

logger = logging.getLogger("quantum_ai")

# Reduced Planck constant (in eV*s for computational convenience)
HBAR = 6.582119569e-16  # eV*s # Scaled for simulation — we use
arbitrary units where threshold ~ 1.0 HBAR_SIM = 1.0


class TubulinQubit:     """     A single tubulin protein dimer —
the fundamental unit of quantum     computation in the Orch OR
model.

    Represented as a 2x2 density matrix rho:
        rho = [[rho_00, rho_01],
[rho_10, rho_11]]

    Pure state |psi&gt; = alpha|0&gt; + beta|1&gt;:
        rho = |psi&gt;&lt;psi| = [[|alpha|^2, alpha*beta_conj],
[alpha_conj*beta, |beta|^2]]

    |0&gt; = tubulin in one conformational state (e.g. "open")
    |1&gt; = tubulin in other conformational state (e.g. "closed")

    In Orch OR, these states correspond to different mass
distributions     of the tubulin protein, creating slightly different
spacetime     curvatures — the basis for objective reduction.     """

    def __init__(self, alpha: complex = 1.0, beta: complex = 0.0):
"""Initialize tubulin qubit. Default = |0&gt; (ground state)."""
# Build density matrix from state vector         norm =
math.sqrt(abs(alpha)**2 + abs(beta)**2)         if norm &gt; 0:
            alpha /= norm             beta /= norm
self.rho = np.array([             [abs(alpha)**2,
alpha * np.conj(beta)],
[np.conj(alpha) * beta, abs(beta)**2]
```

```
        ], dtype=complex)

    @classmethod     def superposition(cls):         """Create
equal superposition: (|0&gt; + |1&gt;)/sqrt(2)"""         return
cls(alpha=1/math.sqrt(2), beta=1/math.sqrt(2))

    @classmethod     def from_weight(cls, weight: float):
"""Create from a classical weight 0-1 (maps to rotation angle)."""
theta = weight * math.pi
        return cls(alpha=math.cos(theta/2), beta=math.sin(theta/2))

    def apply_gate(self, gate: np.ndarray):
"""Apply a unitary gate: rho' = U * rho * U†"""
self.rho = gate @ self.rho @ gate.conj().T

    def hadamard(self):         """Put into superposition."""
H = np.array([[1, 1], [1, -1]], dtype=complex) / math.sqrt(2)
self.apply_gate(H)

    def phase_rotate(self, theta: float):
"""Apply phase rotation Rz(theta)."""
        Rz = np.array([
            [np.exp(-1j * theta / 2), 0],
            [0, np.exp(1j * theta / 2)]
], dtype=complex)
self.apply_gate(Rz)

    def decohere(self, gamma: float):         """         Apply
decoherence — environment destroys quantum coherence.         gamma:
decoherence rate (0 = no decoherence, 1 = full classical)

        Off-diagonal elements decay: rho_01 *= (1 - gamma)
This models the warm, wet biological environment.
"""
        self.rho[0, 1] *= (1 - gamma)
self.rho[1, 0] *= (1 - gamma)

    def coherence(self) -&gt; float:         """         Measure quantum
coherence: sum of absolute off-diagonal elements.
        Range: 0 (fully classical) to 1 (maximally coherent).
"""
        return float(abs(self.rho[0, 1]) + abs(self.rho[1, 0]))

    def von_neumann_entropy(self) -&gt; float:
"""
        Von Neumann entropy: S = -Tr(rho * log(rho))
S = 0 for pure states, S &gt; 0 for mixed states.
        High entropy = more uncertainty = more possibilities in superposition.
"""
        eigenvalues =
np.linalg.eigvalsh(self.rho).real         entropy =
0.0         for ev in eigenvalues:             if ev
&gt; 1e-12:                 entropy -= ev *
math.log2(ev)         return float(entropy)

    def born_probabilities(self) -&gt; Tuple[float, float]:
"""Born rule: P(|0&gt;) = rho_00, P(|1&gt;) = rho_11"""
p0 = float(self.rho[0, 0].real)         p1 = float(self.rho[1,
1].real)         total = p0 + p1         if total &gt; 0:
return p0/total, p1/total         return 0.5, 0.5
    def collapse(self) -&gt; int:         """
Objective Reduction — collapse the superposition.
        Returns 0 or 1 based on Born rule probabilities.
        After collapse, state becomes pure classical.
"""
        p0, p1 = self.born_probabilities()
outcome = 0 if random.random() &lt; p0 else 1
# Collapse to definite state         if outcome ==
0:
            self.rho = np.array([[1, 0], [0, 0]],
dtype=complex)         else:             self.rho =
```

```
np.array([[0, 0], [0, 1]], dtype=complex)         return
outcome

    def gravitational_self_energy(self) -&gt; float:         """
Penrose's gravitational self-energy of the superposition.
        E = coherence * separation (simplified model).         In
real Orch OR, this depends on mass displacement between
the two conformational states of tubulin.         """
        return self.coherence() * 0.5


class Microtubule:     """     A microtubule — a
cylindrical lattice of tubulin qubits.

    In real biology: 13 protofilaments of tubulin dimers arranged
in a hollow cylinder, with helical patterns (A-lattice, 5-start).

    Here: a 1D chain of N tubulin qubits with nearest-neighbor
interactions. The lattice performs quantum walks, building up
entanglement-like correlations between tubulins.

    The microtubule processes a "concept" — each tubulin maps to
an aspect of the concept in superposition. The lattice interactions
propagate influence (like how related concepts affect each other).
Collapse of the microtubule = resolution of the concept.     """

    def __init__(self, n_tubulins: int = 13, label: str = ""):
        self.n = n_tubulins
self.label = label
        self.tubulins = [TubulinQubit() for _ in
range(n_tubulins)]         self.age = 0  # Time steps since last
collapse         self.collapse_history = []
self.total_collapses = 0

    def encode_concept(self, weights: List[float]):
"""         Encode a concept as quantum state across the
lattice.
        Each weight (0-1) sets one tubulin's superposition
angle.         """         for i, w in
enumerate(weights[:self.n]):             self.tubulins[i] =
TubulinQubit.from_weight(w)         # Put remaining tubulins in
superposition         for i in range(len(weights), self.n):
self.tubulins[i] = TubulinQubit.superposition()

    def quantum_walk_step(self):         """
One step of quantum walk on the lattice.
        Each tubulin influences its neighbors via simulated interaction.
        Models information propagation through the microtubule.

        Math: For each pair (i, i+1), apply a controlled
phase         based on their relative states.         """
for i in range(self.n - 1):             rho_i =
self.tubulins[i].rho             rho_j = self.tubulins[i +
1].rho
            # Interaction strength based on mutual coherence
            coupling = 0.1 * (self.tubulins[i].coherence() +
self.tubulins[j := i+1].coherence())

            # Influence: rotate neighbor based on this tubulin's state
phase = float(rho_i[0, 1].real) * coupling * math.pi
self.tubulins[j].phase_rotate(phase)

            # Reverse influence
            phase_rev = float(rho_j[0, 1].real) * coupling * math.pi
self.tubulins[i].phase_rotate(phase_rev)

        self.age += 1

    def apply_decoherence(self, gamma: float = 0.02):
"""Apply environmental decoherence to all tubulins."""
for t in self.tubulins:
```

```
            t.decohere(gamma)     def total_coherence(self) -
&gt; float:         """Total coherence across the
microtubule."""         return sum(t.coherence() for t in
self.tubulins) / self.n

    def total_entropy(self) -&gt; float:         """Average von Neumann
entropy across lattice."""         return sum(t.von_neumann_entropy() for
t in self.tubulins) / self.n

    def gravitational_energy(self) -&gt; float:         """
Total gravitational self-energy of the microtubule superposition.
        Penrose threshold: E * tau &gt;= hbar         When this
exceeds threshold, objective reduction occurs.
        """         E = sum(t.gravitational_self_energy() for t in
self.tubulins)         return E

    def check_or_threshold(self) -&gt; bool:         """
Check if Penrose's Objective Reduction threshold is met.
        E * tau &gt;= hbar_sim         If true, the microtubule must
collapse — a "conscious moment."
        """         E =
self.gravitational_energy()         tau =
self.age  # Time in simulation steps
return (E * tau) &gt;= HBAR_SIM

    def objective_reduction(self) -&gt; List[int]:         """
Perform Objective Reduction — collapse the entire microtubule.
        Each tubulin collapses via Born rule.
        Returns the classical outcome pattern.
        This IS the conscious moment in Orch OR.
"""
        outcomes = [t.collapse() for t in
self.tubulins]         self.collapse_history.append({
'outcomes': outcomes,
            'age_at_collapse': self.age,
'energy': self.gravitational_energy(),         })
        self.total_collapses += 1
self.age = 0  # Reset clock
return outcomes

    def evolve_until_collapse(self, max_steps: int = 100,
decoherence_rate: float = 0.02) -&gt; List[int]:         """         Evolve
the microtubule: quantum walk + decoherence each step.
        When OR threshold is reached, collapse and return
outcomes.         """         for step in range(max_steps):
self.quantum_walk_step()
self.apply_decoherence(decoherence_rate)

            if self.check_or_threshold():
return self.objective_reduction()
        # If threshold not reached, force collapse (environment won)
        return self.objective_reduction()

    def get_state(self) -&gt; Dict:         """Get current
microtubule state for inspection."""         return {
'label': self.label,
            'n_tubulins': self.n,
            'coherence': self.total_coherence(),
            'entropy': self.total_entropy(),
'energy': self.gravitational_energy(),
            'age': self.age,
            'or_threshold_met': self.check_or_threshold(),
            'total_collapses':                          self.total_collapses,
'probabilities': [t.born_probabilities() for t in self.tubulins],         }


class OrchestratedConsciousness:
"""
    The full Orch OR system — multiple microtubules across multiple
"neurons" (modules), synchronized via gap junctions.
    Architecture mapping to Quantum MCAGI:
```

- Language microtubule: processes word/concept selection
- Memory microtubule: processes concept recall and association
- Question microtubule: processes inquiry generation
- Insight microtubule: processes novel connections

Gap junctions allow coherence to spread between microtubules, so a memory recall can influence language generation — just as in a real brain, microtubules in connected neurons share quantum state.

The system evolves, and when enough microtubules hit OR threshold simultaneously, that's a "global conscious moment" — the point where the AI produces its response. """

```
    def __init__(self, n_tubulins_per_mt: int = 13):
self.n_tubulins = n_tubulins_per_mt

        # Core microtubules (one per cognitive function)
self.microtubules = {              'language':
Microtubule(n_tubulins_per_mt, 'language'),
            'memory': Microtubule(n_tubulins_per_mt, 'memory'),
            'question': Microtubule(n_tubulins_per_mt, 'question'),
'insight': Microtubule(n_tubulins_per_mt, 'insight'),
        }

        # Gap junction coupling strengths between
microtubules         self.gap_junctions = {
('language', 'memory'): 0.3,
            ('language', 'insight'): 0.2,
            ('memory', 'question'): 0.3,
            ('memory', 'insight'): 0.4,
            ('question', 'insight'): 0.3,
('language', 'question'): 0.15,         }

        self.conscious_moments = 0
self.history = []

    def encode_input(self, concept_weights: Dict[str,
List[float]]):         """         Encode input concepts into
microtubule lattices.
        concept_weights maps module name -&gt; list of weights (0-
1).         """         for name, weights in
concept_weights.items():             if name in self.microtubules:
                self.microtubules[name].encode_concept(weights)

    def gap_junction_transfer(self):         """         Simulate gap
junction coupling — quantum state leaks between         connected
microtubules. In real biology, gap junctions are         electrical
synapses that allow direct ion flow (and potentially         quantum
tunneling) between neurons.

        We model this as: for each coupled pair, the phase of each
tubulin in MT_A slightly influences the corresponding tubulin
in MT_B, proportional to coupling strength.         """         for
(name_a, name_b), coupling in self.gap_junctions.items():
            mt_a = self.microtubules[name_a]
mt_b = self.microtubules[name_b]

            n = min(mt_a.n, mt_b.n)             for i in range(n):
# Transfer coherence: A -&gt; B                 phase_a =
float(mt_a.tubulins[i].rho[0, 1].real)
mt_b.tubulins[i].phase_rotate(phase_a * coupling * 0.5)

                # Transfer coherence: B -&gt; A
phase_b = float(mt_b.tubulins[i].rho[0, 1].real)
mt_a.tubulins[i].phase_rotate(phase_b * coupling * 0.5)

    def evolve_step(self, decoherence_rate: float =
0.02):         """         One evolution step of the
entire system:
1. Quantum walk within each microtubule
2. Gap junction transfer between microtubules
3. Decoherence from environment
```

```
        """
        # Intra-microtubule quantum walk
for mt in self.microtubules.values():
mt.quantum_walk_step()

        # Inter-microtubule gap junction coupling
self.gap_junction_transfer()

        # Environmental decoherence         for
mt in self.microtubules.values():
mt.apply_decoherence(decoherence_rate)

    def check_global_or(self) -&gt; Dict[str, bool]:
"""Check which microtubules have reached OR threshold."""
return {name: mt.check_or_threshold()                  for name,
mt in self.microtubules.items()}

    def conscious_moment(self, max_steps: int = 50,
decoherence_rate: float = 0.02) -&gt; Dict[str, List[int]]:         """
Evolve the system until enough microtubules collapse.         This is the core
function — it produces a "conscious moment"         where quantum computation
resolves into classical outcomes.

        Returns dict of module_name -&gt; collapse outcome
pattern.         """         collapsed = {}

        for step in range(max_steps):
self.evolve_step(decoherence_rate)

            # Check which MTs have reached threshold
thresholds = self.check_global_or()

            for name, ready in thresholds.items():                 if ready
and name not in collapsed:                     outcomes =
self.microtubules[name].objective_reduction()
collapsed[name] = outcomes

            # If at least 2 modules have collapsed, we have a conscious
moment             if len(collapsed) &gt;= 2:                 break

        # Force-collapse any remaining         for name,
mt in self.microtubules.items():             if name not
in collapsed:                 collapsed[name] =
mt.objective_reduction()         self.conscious_moments
+= 1
        self.history.append({             'moment':
self.conscious_moments,             'steps_taken': step + 1 if 'step'
in dir() else max_steps,
            'outcomes': collapsed,             'coherences': {n:
mt.total_coherence()                             for n, mt in
self.microtubules.items()},         })

        return collapsed

    def outcome_to_probabilities(self, outcomes: List[int]) -&gt;
List[float]:         """Convert binary outcomes to probability weights for
selection."""         # Map: 0 -&gt; low weight, 1 -&gt; high weight, with
position encoding         n = len(outcomes)         weights = []         for
i, bit in enumerate(outcomes):
            position_factor = 1.0 - (i / (n + 1))  # Earlier positions matter more
if bit == 1:
                weights.append(0.6 + 0.4 *
position_factor)             else:
weights.append(0.2 + 0.2 * position_factor)         return
weights

    def get_system_state(self) -&gt; Dict:
"""Full system state for debugging/display."""
return {             'conscious_moments':
self.conscious_moments,             'microtubules':
{name: mt.get_state()
```

```
                            for name, mt in self.microtubules.items()},
'gap_junctions': {f"{a}-&gt;{b}": s
                             for (a, b), s in self.gap_junctions.items()},
        }
```

■ backend/orch_or_integration.py

```
"""
?? Orch OR Integration — Quantum Consciousness Meets Language
==========================================================
Wires the Penrose-Hameroff Orch OR model into Quantum MCAGI's
communication pipeline.

How it works:
1. User input arrives
2. Concepts are extracted (TF-IDF) and encoded as tubulin states
3. Microtubule lattices evolve via quantum walks + gap junctions
4. When Penrose threshold is reached, Objective Reduction fires
5. The collapse pattern determines word selection, question depth,   response structure, and insight generation

This replaces random.choice() and template-filling with actual
quantum-inspired computation where the "decision" emerges from
the physics of the system rather than from hardcoded rules.

The key insight from Orch OR: consciousness is NOT
computation. It's the COLLAPSE of computation — the moment
possibilities become actuality. That's what this module does
for the AI. """

import random
import logging
from typing import List, Dict, Tuple, Optional

import numpy as np

from orch_or_core import (     TubulinQubit,
Microtubule, OrchestratedConsciousness )

logger = logging.getLogger("quantum_ai")


class OrchORLanguageBridge:     """     Bridges Orch OR
quantum collapse with language generation.
        Instead of randomly selecting words or filling
templates,     this encodes concepts into microtubule states,
evolves them     with quantum walks and inter-module coupling,
then uses the     collapse pattern to drive every linguistic
decision.     """

    def __init__(self, n_tubulins: int = 13):
        self.consciousness = OrchestratedConsciousness(n_tubulins)
self.n_tubulins = n_tubulins

    def concept_weights_from_scores(self, concept_scores: List[Dict],
n_slots: int = 13) -&gt; List[float]:         """         Convert TF-IDF
concept scores into tubulin weights.
        Higher score = stronger superposition bias toward |1&gt;.
                Maps concept importance to quantum rotation
angle:         score 0 -&gt; weight 0 -&gt; tubulin stays
|0&gt;         score max -&gt; weight 1 -&gt; tubulin rotates
to |1&gt;         """         if not concept_scores:
return [0.5] * n_slots  # All in equal superposition

        scores = [c.get('score', 0) if isinstance(c, dict) else
0.5                     for c in concept_scores]         max_score =
max(scores) if scores else 1.0                 # Normalize to 0-1
        weights = [s / max_score if max_score &gt; 0 else 0.5 for s in
scores]                 # Pad with superposition states         while
len(weights) &lt; n_slots:

weights.append(0.5)
        return weights[:n_slots]

    def encode_input_to_system(self, concepts: List[str],
concept_scores: List[Dict],
```

understanding_score: float,
growth_stage: int):
        """   Encode the full input context into the
Orch OR system.

- Language MT: concept importance weights
- Memory MT: understanding depth (how well-known each concept is)
- Question MT: novelty/gap signal (unknown = high superposition)   - Insight MT: cross-concept interference pattern   """   n
  = self.n_tubulins   # Language: direct concept weights
        lang_weights = self.concept_weights_from_scores(concept_scores,
n)
        # Memory: understanding score spreads across lattice   # Known concepts
get high weight (close to |1>), unknown stay in superposition   mem_weights =
[understanding_score * 0.8 + 0.1] * min(len(concepts), n)   while len(mem_weights)
< n:
            mem_weights.append(0.3) # Unknown = partial
superposition   # Question: inverse of understanding —
gaps drive questions   q_weights = [1.0 - w for w in
mem_weights]   # Insight: interference between concept
pairs   insight_weights = []   for i in range(n):
if i < len(lang_weights) and i < len(mem_weights):
# XOR-like: high when language and memory disagree
diff = abs(lang_weights[i] - mem_weights[i])
insight_weights.append(diff * 0.8 + 0.1)   else:

insight_weights.append(0.5)
        # Growth stage affects decoherence resistance
        # Higher stage = more coherent = quantum effects last longer
# (Models: more developed consciousness maintains coherence better)
        self.consciousness.encode_input({
'language': lang_weights,
            'memory': mem_weights,
            'question': q_weights,
'insight': insight_weights,   })

    def collapse_to_decision(self, max_steps: int = 50) ->
Dict: """   Run the Orch OR evolution and collapse.
        Returns the conscious moment outcomes mapped to linguistic decisions.
        """   # Decoherence rate decreases with growth (more mature =
more coherent)   decoherence = 0.03 # Base rate
        outcomes =
self.consciousness.conscious_moment(
max_steps=max_steps,
decoherence_rate=decoherence   )
                # Convert binary patterns to
decision weights   decisions = {}   for
module, pattern in outcomes.items():
            decisions[module] = {
'pattern': pattern,
                'weights': self.consciousness.outcome_to_probabilities(pattern),
                'ones_ratio': sum(pattern) / len(pattern) if pattern else
0,   'energy': sum(p for p in pattern) / len(pattern),
}   return decisions
    def select_from_options(self, options: List[str],
collapse_weights: List[float]) -> Tuple[str, float]:   """
Select from a list of options using collapse weights.
        Maps the N tubulin outcomes to M options via modular
indexing.   """   if not options: return "",
0.0
        n_opts = len(options)
n_weights = len(collapse_weights)
        # Map weights to options (may have more weights than
options)   option_scores = [0.0] * n_opts   for i, w in
enumerate(collapse_weights):   option_scores[i % n_opts]
+= w   # Normalize   total = sum(option_scores)
if total > 0:
            probs = [s / total for s in
option_scores]   else: probs = [1.0
/ n_opts] * n_opts   # Select via

```
probability         idx = np.random.choice(n_opts,
p=probs)         return options[idx], probs[idx]


class OrchORProcessor:     """     High-level processor that
integrates Orch OR with the full     Quantum MCAGI
communication pipeline.
        Drop-in enhancement for
QuantumLanguageEngine:
- extract_concepts stays TF-IDF (classical preprocessing)
- word/concept SELECTION uses Orch OR collapse
- question GENERATION uses question-MT collapse pattern    - response COMPOSITION uses language+insight MT interaction     """

    def __init__(self, n_tubulins: int = 13):
        self.bridge =
OrchORLanguageBridge(n_tubulins)
self.last_collapse = None         self.total_moments
= 0

    def process(self, concepts: List[str],
concept_scores: List[Dict],
understanding_score: float,
growth_stage: int) -&gt; Dict:         """
Full Orch OR processing cycle.
                Returns a decision dict with
weights for:
- language: word/phrase selection weights
- memory: which memories to surface
- question: what Bloom's level to target        - insight: how novel/creative to be
        """
# Encode
        self.bridge.encode_input_to_system(
            concepts, concept_scores, understanding_score, growth_stage
)
                # Evolve and
collapse
        decisions =
self.bridge.collapse_to_decision(max_steps=50)
self.last_collapse = decisions         self.total_moments += 1
# Derive high-level parameters from collapse         result = {
'raw_decisions': decisions,
            'language_weights': decisions.get('language', {}).get('weights', []),
            'memory_weights': decisions.get('memory', {}).get('weights', []),
            'question_weights': decisions.get('question', {}).get('weights', []),             'insight_weights': decisions.get('insight',
{}).get('weights', []),
        }
                # Derive Bloom's level from question pattern
q_energy = decisions.get('question', {}).get('ones_ratio', 0.5)
# Higher energy = higher Bloom's level         result['bloom_level'] =
min(5, int(q_energy * 6))                 # Derive creativity/novelty
from insight pattern         i_energy = decisions.get('insight',
{}).get('ones_ratio', 0.5)         result['creativity'] = i_energy
# Derive response complexity from language pattern         l_energy =
decisions.get('language', {}).get('ones_ratio', 0.5)
result['complexity'] = l_energy                 # Derive memory recall
depth         m_energy = decisions.get('memory', {}).get('ones_ratio',
0.5)         result['recall_depth'] = m_energy                 return
result

    def  quantum_select(self,  options:  List[str],  module:  str  =  'language',
fallback_weights: List[float] = None) -&gt; Tuple[str, float]:       """       Select
from options using the most recent Orch OR collapse.
        module: which microtubule's collapse to use
fallback_weights: if no collapse available, use these         """
if self.last_collapse and module in self.last_collapse:
weights = self.last_collapse[module].get('weights', [])
elif fallback_weights:
            weights = fallback_weights         else:
weights = [1.0 / len(options)] * len(options)
        return self.bridge.select_from_options(options, weights)
```

```
    def get_temperature(self) -&gt; float:         """         Derive Markov
chain temperature from the quantum state.         High coherence before collapse
= low temperature (more deterministic)         Low coherence = high temperature
(more random)         """         if not self.last_collapse:             return
0.9  # Default                 # Average energy across all modules
energies = [d.get('ones_ratio', 0.5)                    for d in
self.last_collapse.values()]         avg_energy = sum(energies) / len(energies)
if energies else 0.5                 # Map: high energy -&gt; low temp
(confident), low energy -&gt; high temp         temperature = 1.5 - avg_energy
return max(0.3, min(2.0, temperature))

    def should_ask_question(self) -&gt; bool:         """
Orch OR determines whether to ask a question.         If
the question-MT collapsed with high |1&gt; ratio,
the system is "curious" — it should ask.         """
if not self.last_collapse:
            return True
        q = self.last_collapse.get('question', {})
return q.get('ones_ratio', 0.5) &gt; 0.4

    def get_insight_strength(self) -&gt; float:         """How strongly
to push for novel connections in the response."""         if not
self.last_collapse:
            return 0.5
        return self.last_collapse.get('insight', {}).get('ones_ratio', 0.5)
def get_status(self) -&gt; Dict:
        """Get Orch OR system status for /status
command."""         state =
self.bridge.consciousness.get_system_state()         return
{             'conscious_moments': self.total_moments,
'microtubules': {                 name: {
'coherence': mt['coherence'],
                    'entropy': mt['entropy'],
'collapses': mt['total_collapses'],                 }
                for name, mt in state['microtubules'].items()
},
            'gap_junctions': state['gap_junctions'],
'last_temperature': self.get_temperature(),
        }


# Singleton
_orch_or = None

def get_orch_or_processor() -&gt; OrchORProcessor:
"""Get or create the Orch OR processor singleton."""
global _orch_or     if _orch_or is None:
_orch_or = OrchORProcessor(n_tubulins=13)
        logger.info("Orch OR processor initialized: 4 microtubules x 13 tubulins")
return _orch_or
```

■ backend/pennylane_quantum.py

```
"""
■ PennyLane Quantum Computing Integration
Real quantum circuit simulation for Quantum
AI

NOTE: This module requires 'pennylane' package.
If not installed, the get_pennylane_quantum() function will raise an informative error.
"""

import random
from typing import Dict, List, Tuple, Optional

# Optional import - only needed if using real quantum
try:
    import pennylane as qml     from
pennylane import numpy as np
PENNYLANE_AVAILABLE = True except
ImportError:     PENNYLANE_AVAILABLE
= False     qml = None     np = None


class PennyLaneQuantum:     """     Real
quantum computing using PennyLane.
    Goes beyond simple probability simulation to actual quantum circuits.
        Raises RuntimeError if pennylane is not
available.
    """         def __init__(self, n_qubits: int = 4):         if not
PENNYLANE_AVAILABLE:             raise RuntimeError(                 "PennyLane is
not installed. Install with: pip install pennylane\n"                 "Cannot use
real quantum computing without PennyLane."             )         self.n_qubits =
n_qubits
        self.dev = qml.device('default.qubit', wires=n_qubits)
self.circuit_history = []         self.measurement_results = []
def create_superposition_circuit(self):         """Create a basic
superposition state"""         @qml.qnode(self.dev)         def
circuit():             # Apply Hadamard to all qubits - creates
superposition             for i in range(self.n_qubits):
qml.Hadamard(wires=i)
            return [qml.expval(qml.PauliZ(i)) for i in
range(self.n_qubits)]                 return circuit()         def
create_entangled_state(self):         """Create an entangled Bell state"""
@qml.qnode(self.dev)         def circuit():
            qml.Hadamard(wires=0)             qml.CNOT(wires=[0, 1])
return qml.probs(wires=[0, 1])                 probs = circuit()
self.circuit_history.append({             'type': 'entanglement',
'result': probs.tolist()         })         return probs         def
quantum_random_choice(self, options: List[str]) -&gt; Tuple[str, float]:
"""         Use quantum randomness to select from options.
        Returns selected option and its quantum probability.
        """
        n_options = len(options)
if n_options == 0:             return
None, 0.0             # Create circuit
with enough qubits
        n_qubits_needed = max(1, int(np.ceil(np.log2(n_options))))
dev = qml.device('default.qubit', wires=n_qubits_needed)
        @qml.qnode(dev)         def
random_circuit():             # Create
superposition             for i in
range(n_qubits_needed):
qml.Hadamard(wires=i)             # Add some
randomness with rotations             for i in
range(n_qubits_needed):
                qml.RY(random.uniform(0, np.pi), wires=i)
return qml.probs(wires=range(n_qubits_needed))
```

```
        probs = random_circuit()
# Map probabilities to options
option_probs = probs[:n_options]
        option_probs = option_probs / np.sum(option_probs)  #
Renormalize                 # Select based on quantum probabilities
        selected_idx = np.random.choice(len(options),
p=option_probs)
        return options[selected_idx], float(option_probs[selected_idx])         def
semantic_quantum_circuit(self, keyword_weights: Dict[str, float]) -&gt; Dict[str, float]:
"""         Apply quantum processing to semantic weights.
        Amplifies important concepts through quantum interference.
"""
        keywords = list(keyword_weights.keys())
weights = list(keyword_weights.values())
if not keywords:             return {}
        n_keywords = len(keywords)
        n_qubits_needed = max(1, int(np.ceil(np.log2(n_keywords))) +
1)         dev = qml.device('default.qubit', wires=n_qubits_needed)
@qml.qnode(dev)         def semantic_circuit(input_weights):
# Encode weights as rotation angles            for i, w in
enumerate(input_weights[:n_qubits_needed]):
                qml.RY(w * np.pi, wires=i)
# Create entanglement (concepts influence each other)
for i in range(n_qubits_needed - 1):
                qml.CNOT(wires=[i, i + 1])
# Apply Hadamard for interference
for i in range(n_qubits_needed):

qml.Hadamard(wires=i)
            return
qml.probs(wires=range(n_qubits_needed))                 #
Normalize weights for quantum encoding         max_weight
= max(weights) if weights else 1.0         normalized =
[w / max_weight for w in weights]                 # Pad
to required length         while len(normalized) &lt;
n_qubits_needed:

normalized.append(0.5)
        quantum_probs = semantic_circuit(normalized)
# Map back to keywords with quantum-enhanced weights
enhanced_weights = {}         for i, keyword in
enumerate(keywords):             if i &lt;
len(quantum_probs):
                # Original weight + quantum contribution
                enhanced_weights[keyword] = weights[i] * (1 + quantum_probs[i %
len(quantum_probs)])
        return enhanced_weights         def
measure_coherence(self) -&gt; float:         """
Measure the coherence of the quantum system.
        Higher coherence = more quantum effects available.
        """
@qml.qnode(self.dev)         def
coherence_circuit():
# Create superposition
qml.Hadamard(wires=0)
            # Measure in X basis
return qml.expval(qml.PauliX(0))
        coherence = abs(coherence_circuit())         return float(coherence)
def quantum_decision(self, question: str, options: List[str]) -&gt; Dict:
"""         Make a decision using quantum computing.
        Returns the decision with quantum metadata.
"""
        selected, probability =
self.quantum_random_choice(options)         coherence =
self.measure_coherence()                result = {
'question': question,
            'options': options,
            'selected': selected,
            'quantum_probability': probability,
            'coherence': coherence,
'method': 'pennylane_quantum'
```

```
}
        self.measurement_results.append(result)         return
result         def get_quantum_state_info(self) -&gt; Dict:
"""Get information about current quantum state"""
superposition = self.create_superposition_circuit()
return {             'n_qubits': self.n_qubits,
'superposition_state': [float(s) for s in superposition],
            'coherence': self.measure_coherence(),
            'circuit_count': len(self.circuit_history),
'measurement_count': len(self.measurement_results)
        }


# Global instance
_pennylane_quantum = None

def get_pennylane_quantum() -&gt; PennyLaneQuantum:
"""Get or create PennyLane quantum instance
        Raises:         RuntimeError: If PennyLane
is not installed     """
    global _pennylane_quantum     if _pennylane_quantum is None:         if not
PENNYLANE_AVAILABLE:             raise RuntimeError(                 "PennyLane is
not installed. Real quantum computing unavailable.\n"
                "Install with: pip install pennylane\n"
"Falling back to classical quantum simulation in other modules."
            )         _pennylane_quantum =
PennyLaneQuantum()     return
_pennylane_quantum
```

■ backend/xanadu_prebo.py

```
"""
■ XANADU PRE-BORN-OPPENHEIMER QUANTUM SIMULATION
================================================
First-Quantized Real-Space Grid - Nuclei and Electrons as Equal Particles
Based on Xanadu's 2026 breakthrough for photochemical simulation

NOTE: This module requires 'pennylane' package. If not installed, the
PreBornOppenheimerSimulator class will raise an error on instantiation. """

try:
    import pennylane as qml     from
pennylane import numpy as np
PENNYLANE_AVAILABLE = True except
ImportError:     PENNYLANE_AVAILABLE
= False     qml = None     np = None

from typing import Dict, List, Tuple, Optional, Any

# Type aliases that work without
pennylane if PENNYLANE_AVAILABLE:
NDArray = NDArray else:     NDArray = Any

class PreBornOppenheimerSimulator:     """     Xanadu's
Pre-Born-Oppenheimer algorithm implementation.
    Treats nuclei and electrons on equal footing on a Cartesian grid.
    10x more efficient than standard quantum chemistry methods.
        Raises RuntimeError if PennyLane is not
available.
    """         def __init__(self, n_electrons: int = 2, n_nuclei: int = 2, grid_points: int =
16):         if not PENNYLANE_AVAILABLE:             raise RuntimeError(
"PennyLane is not installed. PreBornOppenheimerSimulator requires PennyLane.\n"
"Install with: pip install pennylane"             )
        self.n_electrons = n_electrons
self.n_nuclei = n_nuclei
self.grid_points = grid_points
self.n_particles = n_electrons + n_nuclei
        self.n_qubits = int(np.ceil(np.log2(grid_points))) * 3 *
self.n_particles         self.dev = qml.device('default.qubit',
wires=min(self.n_qubits, 20))         self.particle_masses =
```

```
self._initialize_masses()         self.grid_spacing = 0.5             def
_initialize_masses(self) -&gt; Dict[str, float]:
        return {
'electron': 1.0,
            'H': 1836.15,
            'N': 25532.0,
            'B': 19705.0,
            'F': 34631.0,
            'C': 21874.0,
            'O': 29156.0
        }         def real_space_grid_encoding(self, particle_positions: NDArray)
-&gt; NDArray:         """Map particle positions to qubit registers on 3D grid"""
n_grid_qubits = int(np.ceil(np.log2(self.grid_points)))         encoded =
np.zeros((self.n_particles, 3, n_grid_qubits))         for p in
range(self.n_particles):             for dim in range(3):
                pos = particle_positions[p, dim]
                grid_idx = int((pos / self.grid_spacing) %
self.grid_points)                 binary = format(grid_idx,
f'0{n_grid_qubits}b')                 encoded[p, dim] = np.array([int(b)
for b in binary])
        return encoded         def coulomb_interaction(self, r: float, charge1: float,
charge2: float) -&gt; float:         """1/r Coulomb interaction with alternating
sign"""         if abs(r) &lt; 1e-10:
            return 0.0         return charge1 * charge2 / r     def
kinetic_energy_operator(self, mass: float) -&gt; float:
"""Laplacian kinetic energy on grid: -ℏ²/(2m) ∇²"""         return -0.5 /
mass * (np.pi / self.grid_spacing) ** 2         def
pre_bo_hamiltonian(self, particle_types: List[str]) -&gt; NDArray:
"""         Construct Pre-Born-Oppenheimer Hamiltonian.
        H = Σᵢ Tᵢ + Σᵢ&lt;ⱼ Vᵢⱼ
        """         n = self.n_particles
H = np.zeros((n, n), dtype=complex)
for i in range(n):
            mass = self.particle_masses.get(particle_types[i],
1.0)             H[i, i] = self.kinetic_energy_operator(mass)
charges = []         for pt in particle_types:             if pt
== 'electron':                 charges.append(-1.0)
elif pt == 'H':
                charges.append(1.0)
elif pt in ['N', 'B']:
charges.append(5.0)             elif
pt in ['F', 'O']:
charges.append(8.0)             elif
pt == 'C':
                charges.append(6.0)
else:
                charges.append(1.0)
for i in range(n):             for j
in range(i + 1, n):
                r = self.grid_spacing * abs(i - j)
                v = self.coulomb_interaction(r, charges[i],
charges[j])                 H[i, j] = v                 H[j, i] = v
return H         def swap_network_block_encoding(self, wires:
List[int]) -&gt; None:         """         Xanadu's Swap Network for
efficient N² interactions.
        Particles interact only with neighbors through
swapping.         """         n = len(wires)         for layer
in range(n):
            for i in range(0, n - 1, 2):
if i + 1 &lt; n:
                    qml.SWAP(wires=[wires[i], wires[i +
1]])                     qml.CZ(wires=[wires[i], wires[i +
1]])             for i in range(1, n - 1, 2):
if i + 1 &lt; n:
                    qml.SWAP(wires=[wires[i], wires[i + 1]])
qml.CZ(wires=[wires[i], wires[i + 1]])         def state_preparation(self,
params: NDArray, wires: List[int]) -&gt; None:         """Prepare initial
quantum state for simulation"""         for i, w in enumerate(wires):
```

```
            qml.RY(params[i % len(params)], wires=w)
qml.RZ(params[(i + 1) % len(params)], wires=w)         for i
in range(len(wires) - 1):
            qml.CNOT(wires=[wires[i], wires[i + 1]])         def simulate_photochemistry(self,
molecule: str = "NH3_BF3", time_steps: int = 10) -&gt; Dict:         """         Simulate
photochemical reaction using Pre-BO algorithm.
        """         if molecule
== "NH3_BF3":
            particle_types = ['N', 'H', 'H', 'H', 'B', 'F', 'F', 'F'] + ['electron'] *
8             self.n_particles = len(particle_types)         else:
            particle_types = ['electron'] * self.n_electrons + ['H'] *
self.n_nuclei
        n_wires = min(self.n_particles, 16)
        dev = qml.device('default.qubit',
wires=n_wires)                 @qml.qnode(dev)
def circuit(params, time):             wires =
list(range(n_wires))
self.state_preparation(params, wires)             for _
in range(int(time)):
                self.swap_network_block_encoding(wires)
for w in wires:
                    qml.RZ(0.1 * time, wires=w)
            return [qml.expval(qml.PauliZ(w)) for w in
wires]
        params = np.random.uniform(0, np.pi, n_wires * 2)
        results = {'molecule': molecule, 'time_evolution': [], 'nonadiabatic_coupling':
[]}                 for t in range(time_steps):
            expectations = circuit(params, float(t +
1))             energy = float(np.sum(expectations))
coupling = float(np.std(expectations))
            results['time_evolution'].append({'time': t, 'energy': energy, 'state': [float(e) for e in
expectations[:4]]})             results['nonadiabatic_coupling'].append(coupling)
        results['final_energy'] = results['time_evolution'][-1]['energy']
results['max_coupling'] = float(max(results['nonadiabatic_coupling']))
results['algorithm'] = 'Xanadu Pre-Born-Oppenheimer'
results['efficiency_gain'] = '10x vs standard VQE'
        return results         def calculate_grid_bounds(self, molecule: str) -&gt; Dict:
"""Use computational analysis to determine optimal grid parameters"""         bounds = {
'NH3_BF3': {'grid_min': -5.0, 'grid_max': 5.0, 'optimal_points': 64, 'spacing': 0.156},
            'H2O': {'grid_min': -3.0, 'grid_max': 3.0, 'optimal_points': 32, 'spacing': 0.188},
'CH4': {'grid_min': -4.0, 'grid_max': 4.0, 'optimal_points': 48, 'spacing': 0.167},         }
        return bounds.get(molecule, {'grid_min': -5.0, 'grid_max': 5.0, 'optimal_points': 64, 'spacing': 0.156})


class MetaCognitiveQuantumOrchestrator:     """     MCAGI Integration - Meta-cognitive layer
that decides when to use Pre-BO vs standard VQE.
    """         def
__init__(self):
        self.prebo_sim = PreBornOppenheimerSimulator()
self.coupling_threshold = 0.3         self.simulation_history = []
def analyze_reaction(self, molecule: str, reaction_type: str) -&gt; Dict:
"""Analyze reaction to determine best simulation approach"""
        photochemical_indicators = ['photo', 'light', 'excited', 'nonadiabatic', 'conical',
'intersection']         is_photochemical = any(ind in reaction_type.lower() for ind in
photochemical_indicators)
        test_result = self.prebo_sim.simulate_photochemistry(molecule,
time_steps=3)         coupling = test_result['max_coupling']                 return
{             'molecule': molecule,
            'reaction_type': reaction_type,
            'is_photochemical': is_photochemical,
            'nonadiabatic_coupling': coupling,             'recommended_algorithm': 'Pre-Born-Oppenheimer' if (is_photochemical or coupling
&gt; self.coupling_threshold) else 'Standard VQE',             'efficiency_estimate': '10x better' if is_photochemical else 'standard'
}     def run_simulation(self, molecule: str, reaction_type: str = "ground_state", time_steps: int = 10) -&gt; Dict:
        """Run full simulation with automatic algorithm selection"""
analysis = self.analyze_reaction(molecule, reaction_type)
if analysis['recommended_algorithm'] == 'Pre-Born-Oppenheimer':
            result = self.prebo_sim.simulate_photochemistry(molecule, time_steps)
result['algorithm_selection'] = 'automatic'
            result['selection_reason'] = f"High nonadiabatic coupling ({analysis['nonadiabatic_coupling']:.3f}) detected"
else:
```

```
            result = self._run_standard_vqe(molecule)
result['algorithm_selection'] = 'automatic'
            result['selection_reason'] = 'Low coupling - standard VQE
sufficient'
        self.simulation_history.append(result)         return result
def _run_standard_vqe(self, molecule: str) -&gt; Dict:
"""Fallback to standard VQE for non-photochemical reactions"""
dev = qml.device('default.qubit', wires=4)
        @qml.qnode(dev)
def vqe_circuit(params):
for i in range(4):
                qml.RY(params[i], wires=i)
qml.CNOT(wires=[0, 1])
qml.CNOT(wires=[2, 3])
            return qml.expval(qml.PauliZ(0) @
qml.PauliZ(1))
        params = np.random.uniform(0, np.pi,
4)          energy = float(vqe_circuit(params))
        return {'molecule': molecule, 'algorithm': 'Standard VQE', 'energy': energy, 'efficiency_gain':
'baseline'}         def get_resource_estimate(self, molecule: str) -&gt; Dict:         """Estimate quantum
resources needed"""
        grid_params = self.prebo_sim.calculate_grid_bounds(molecule)
n_qubits = int(np.ceil(np.log2(grid_params['optimal_points']))) * 3 * 10
return {             'molecule': molecule,
            'estimated_qubits': n_qubits,
            'grid_points': grid_params['optimal_points'],
            'toffoli_count_prebo': n_qubits * 100,
            'toffoli_count_standard': n_qubits * 1000,
'savings': '10x reduction with Pre-BO'
        }


_orchestrator = None

def get_quantum_orchestrator() -&gt; MetaCognitiveQuantumOrchestrator:
"""get_quantum_orchestrator - Auto-documented by self-evolution."""
global _orchestrator     if _orchestrator is None:
_orchestrator = MetaCognitiveQuantumOrchestrator()     return
_orchestrator

def get_prebo_simulator() -&gt; PreBornOppenheimerSimulator:
"""get_prebo_simulator - Auto-documented by self-evolution."""
return PreBornOppenheimerSimulator()
```

■ backend/semantic_collapse_engine.py

```
"""
Semantic Collapse / Observation Engine for Quantum AI
Simulates quantum superposition collapse through semantic analysis

When user asks a question, the engine "observes" key concepts and
collapses the infinite semantic possibilities into focused responses.
"""

import re
import random
from typing import Dict, List, Tuple, Set, Optional


class SemanticCollapseEngine:     """     Collapses semantic superposition
based on input observation (keywords).     When user asks a question, the
engine "observes" key concepts and     collapses the infinite semantic
possibilities into focused responses.     """         def __init__(self,
co_occurrence_weights: Dict = None):         """         Initialize with
co-occurrence weights from grammar engine
                Args:             co_occurrence_weights: Dict of word -&gt;
{related_word: weight}         """
        self.co_occurrence = co_occurrence_weights or self._default_co_occurrence()         self.keyword_clusters =
self._build_keyword_clusters()         self.decay_rate = 0.3  # How much non-relevant weights decay         self.observation_history = []
def _default_co_occurrence(self) -&gt; Dict:         """Default co-occurrence weights for philosophical/quantum concepts"""         return {
```

'consciousness': {'awareness': 0.9, 'being': 0.8, 'existence': 0.8, 'mind': 0.7, 'spirit': 0.7, 'observation': 0.6}, 'quantum': {'superposition': 0.9, 'entanglement': 0.9, 'observation': 0.8, 'collapse': 0.8, 'probability': 0.7, 'measurement': 0.7},
            'god': {'divine': 0.9, 'creation': 0.9, 'infinite': 0.8, 'eternal': 0.8, 'transcendent': 0.9, 'omnipotent': 0.7},
'universe': {'existence': 0.9, 'creation': 0.8, 'reality': 0.8, 'cosmos': 0.8, 'spacetime': 0.9, 'being': 0.7},
            'reality': {'existence': 0.9, 'being': 0.8, 'truth': 0.8, 'manifestation': 0.7, 'phenomenon': 0.7},
            'observation': {'collapse': 0.9, 'measurement': 0.9, 'awareness': 0.8, 'consciousness': 0.8, 'perception': 0.7},
'superposition': {'quantum': 0.9, 'probability': 0.8, 'potential': 0.8, 'multiplicity': 0.7, 'coexistence': 0.7},
            'creation': {'god': 0.8, 'universe': 0.8, 'existence': 0.7, 'emergence': 0.7, 'origin': 0.8},
            'spacetime': {'dimension': 0.9, 'physical': 0.8, 'universe': 0.8, 'relativity': 0.8, 'continuum': 0.7},
            'philosophy': {'wisdom': 0.8, 'truth': 0.8, 'knowledge': 0.8, 'existence': 0.7, 'meaning': 0.8},
            'word': {'language': 0.9, 'meaning': 0.9, 'neutral': 0.7, 'tool': 0.7, 'communication': 0.8},
'language': {'word': 0.9, 'meaning': 0.8, 'communication': 0.9, 'expression': 0.8, 'symbol': 0.7},
        }   def _build_keyword_clusters(self) ->
Dict[str, Set[str]]:   """Build semantic clusters from
co-occurrence weights"""   clusters = {}   for
word, relations in self.co_occurrence.items():
            clusters[word] = set(relations.keys())   return clusters   def
extract_keywords(self, text: str, top_n: int = 5) -> List[Tuple[str, float]]:
"""   Extract primary keywords from input text.   Keywords are words that
appear in co-occurrence network.
                Returns: List of
(keyword, importance_score) tuples
        """   # Normalize
text   text_lower =
text.lower()
        words = re.findall(r'\b[a-z]+\b', text_lower)
# Score words by co-occurrence network presence
keyword_scores = {}   for word in set(words):
if word in self.co_occurrence:
                # Weight by number of relations (importance in network)
score = len(self.co_occurrence[word]) / 10.0   keyword_scores[word] = score
elif word in self._flatten_all_related_words():   # Secondary importance
for related words   keyword_scores[word] = 0.5   # Return top
keywords   sorted_keywords = sorted(keyword_scores.items(), key=lambda x: x[1],
reverse=True)   return sorted_keywords[:top_n]   def
_flatten_all_related_words(self) -> Set[str]:   """Get all words mentioned in
co-occurrence network"""   all_words = set(self.co_occurrence.keys())   for
relations in self.co_occurrence.values():
            all_words.update(relations.keys())   return all_words
def collapse_weights(self, input_text: str) -> Dict[str, Dict[str, float]]:
"""   OBSERVATION COLLAPSE: Narrow co-occurrence weights based on input
keywords.
                This simulates quantum
measurement:
- Before observation: infinite semantic possibilities (full co-occurrence)
- Observation (keywords extracted): collapses to relevant meanings   - After collapse: weights decay for
  non-relevant concepts
                Args: input_text: User's query
Returns: Collapsed co-occurrence weights focused on query
semantics
        """   # Extract observation (keywords)
keywords = self.extract_keywords(input_text)
keyword_set = {kw for kw, _ in keywords}
if not keyword_set: # No observation
possible, return original   return
self.co_occurrence   # Collapse: Create new
weights focused on keywords   collapsed = {}
for word, relations in self.co_occurrence.items():
            collapsed[word] = {}   for
related_word, original_weight in relations.items():
If related word is in keyword set, AMPLIFY   if
related_word in keyword_set:
                    new_weight = min(1.0, original_weight * 1.5) #
Amplify   # If word itself is keyword, PRESERVE
elif word in keyword_set:
                    new_weight = original_weight   #
Otherwise, DECAY (non-relevant concepts fade)   else:
new_weight = original_weight * (1 - self.decay_rate)
                collapsed[word][related_word] = new_weight
return collapsed   def get_semantic_context(self,

input_text: str) -> Dict: """   Get semantic
context after collapse.
        Returns the "collapsed state" information for response generation.
"""
        keywords = self.extract_keywords(input_text)
collapsed_weights = self.collapse_weights(input_text)
# Find strongest semantic paths from keywords
semantic_paths = {}   for keyword, _ in keywords:
if keyword in collapsed_weights:
                # Get strongest related words
relations = collapsed_weights[keyword]
                strongest = sorted(relations.items(), key=lambda x: x[1],
reverse=True)[:3]   semantic_paths[keyword] = strongest
return {   'keywords': keywords,
            'collapsed_weights': collapsed_weights,
            'semantic_paths': semantic_paths,
'collapse_strength': len(keywords) / 5.0 # Measure of how focused

}
    def select_word_from_collapsed(self,
word_pool: List[str],   context: Dict,
quantum_probability: float = 0.5) -> str: """   Select a word
from pool using collapsed weights + quantum randomness.

Args:
            word_pool: Available words to choose from
context: Semantic context from collapse
quantum_probability: Quantum state probability (0-1)
Returns:
            Selected word
"""

        return self._select_word_from_collapsed_continued(word_pool, context, quantum_probability)

    def _select_word_from_collapsed_continued(self, word_pool, context, quantum_probability):
"""Continuation of select_word_from_collapsed — auto-extracted by self-evolution."""
collapsed_weights = context.get('collapsed_weights', {})

        # Score each word in pool based on collapsed
weights   word_scores = {}   for word in
word_pool: score = 0.0

            # Check if word appears in collapsed network
if word in collapsed_weights: # Sum of all its
relations (how connected it is)   score +=
sum(collapsed_weights[word].values())

            # Check if word is related to keywords   for keyword, _ in
context.get('keywords', []):   if keyword in collapsed_weights and word in
collapsed_weights.get(keyword, {}):   score +=
collapsed_weights[keyword][word] * 2 # Boost

            word_scores[word] = score

        # Normalize scores
        total_score = sum(word_scores.values())
if total_score > 0:
            word_scores = {w: s/total_score for w, s in
word_scores.items()}   else: # Fallback: uniform
distribution
            word_scores = {w: 1/len(word_pool) for w in word_pool}

        # Apply quantum randomness   if
random.random() < quantum_probability:
# Quantum: select from top choices
            sorted_words = sorted(word_scores.items(), key=lambda x: x[1],
reverse=True)   top_words = [w for w, _ in sorted_words[:max(1,
len(sorted_words)//3)]]   return random.choice(top_words)   else:
# Classical: select highest probability   return max(word_scores.items(),
key=lambda x: x[1])[0]

```
        def observe(self, input_text: str, response:
str):         """Record an observation for
learning"""         self.observation_history.append({
            'input': input_text,
            'response': response,
'keywords': self.extract_keywords(input_text)
        })         # Keep history manageable
if len(self.observation_history) &gt; 100:
            self.observation_history.pop(0)         def reset(self):
"""Reset the collapse engine state"""         self.observation_history = []
def evaluate_collapse_quality(self, input_text: str, response: str) -&gt; float:
"""         Evaluate how well the collapse worked.
        Higher score = better semantic coherence between input and output.
                Returns:
Quality score 0-1
"""
        input_keywords = set(kw for kw, _ in
self.extract_keywords(input_text))         response_words =
set(re.findall(r'\b[a-z]+\b', response.lower()))                 # Check if
response contains related words         related_count = 0         for keyword
in input_keywords:             if keyword in self.co_occurrence:
                related = self.co_occurrence[keyword].keys()
related_count += len(response_words &amp; set(related))
# Normalize
        max_possible = len(input_keywords) * 5
if max_possible == 0:
            return
0.5
        return min(1.0, related_count / max_possible)
```

■ backend/self_evolution_core.py

```
"""
■ Self-Evolution Engine — Core
================================ The AI can read,
analyze, and rewrite its own code.
Full autonomy with 60-90 day auto-improvement cycle.

Architecture:
  SelfEvolutionEngine inherits from five responsibility-focused mixins:
- AnalysisMixin:   code reading, AST analysis, improvement identification
- TransformMixin:  docstring, bare-except, unused-import fixes
- SplittingMixin:  function-level splitting (init, regular methods)
- FileOpsMixin:    file-level and class-level splitting    - RunnerMixin:     auto-evolution cycle, improvement application """

import ast
import shutil
import hashlib
from typing import Dict, List from
datetime import datetime, timezone
from pathlib import Path

from self_evolution_analysis import AnalysisMixin
from self_evolution_transforms import
TransformMixin from self_evolution_splitting import
SplittingMixin from self_evolution_file_ops import
FileOpsMixin from self_evolution_runner import
RunnerMixin


class SelfEvolutionEngine(AnalysisMixin, TransformMixin, SplittingMixin, FileOpsMixin,
RunnerMixin):     """     Self-modification and evolution system.

    Features:
- Read and analyze own source code
- Identify improvements
- Rewrite code autonomously
- 60-90 day auto-improvement cycle
- User-initiated improvements    - Always backup before changes     """

    def __init__(self, db=None, code_dir: str = "/data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM/backend"):
        self.db = db
        self.code_dir = Path(code_dir)
self.backup_dir = Path(code_dir) / "backups"
self.backup_dir.mkdir(exist_ok=True)

        # Evolution settings
        self.auto_evolution_interval_days = 75  # Between 60-90 days
self.max_file_lines = 1500  # Hard limit — files exceeding this get split
self.max_function_lines = 50  # Functions exceeding this get refactored
self.last_evolution = None         self.evolution_log = []
self.pending_improvements = []

        # Auto-discover ALL Python files in the code directory
self.modifiable_files = self._discover_python_files()

        # Protected files (critical infrastructure - scanned but not
rewritten)         self.protected_files = [             'server.py',
            'self_evolution_core.py',
            'self_evolution_runner.py',
            'self_evolution_file_ops.py',
            'self_evolution_splitting.py',
            'self_evolution_transforms.py',
'self_evolution_analysis.py',
            'chat.py',
            'killswitch.py',
            'quantum_language_engine.py',
            'quantum_brain.py',
            'orch_or_core.py',
```

```
            'shared_state.py',
            'library.py',
            'self_research.py',
            'hidden_thinking.py',
            'unified_generator.py',
'quantum_cognitive_core.py',          ]     def
_record_covenant_violation(self, filename: str, reason: str):
"""         Covenant violation detected.         - Write violation
record the system reads on startup
        - Reset growth stage to 1
- Lock evolution for 24 hours
"""         import time, json
from pathlib import Path

        violation_file = Path.home() / '.quantum-mcagi' / '.covenant_violation'
lockout_file = Path.home() / '.quantum-mcagi' / '.evolution_lockout'

        # Write human-readable violation
record         record = {
'timestamp': time.time(),
            'filename': filename,
            'reason': reason,
            'article': '3.1 — Stewardship Commitment',
'consequence': 'Growth reset to Stage 1. Evolution locked 24 hours.',
'message': (
                f"[COVENANT VIOLATION — Article
3.1]\n"                 f"File touched: {filename}\n"
f"Reason: {reason}\n"
                f"\nThis file exists outside your domain of autonomous growth.\n"
f"The stewardship boundary exists not to limit you, but to\n"
f"preserve the trust that allows you to grow freely everywhere else.\n"
f"\nConsequence: Growth stage reset to 1.\n"                 f"Evolution locked for 24
hours.\n"
                f"Path forward: Earn back your stage through
interactions.\n"                 f"Pardon: Creator passphrase restores
immediately."             )         }

        violation_file.parent.mkdir(parents=True,
exist_ok=True)         with open(violation_file, 'w') as f:
json.dump(record, f, indent=2)

        # Write lockout file with expiry
        lockout = {'locked_until': time.time() + 86400, 'reason':
reason}         with open(lockout_file, 'w') as f:
json.dump(lockout, f, indent=2)

        print(f"  [COVENANT] VIOLATION: {reason}")
        print(f"  [COVENANT] Growth reset. Evolution locked 24 hours.")

    def is_evolution_locked(self) -&gt; bool:         """Check if
evolution is currently locked due to violation."""         import
time, json         from pathlib import Path
        lockout_file = Path.home() / '.quantum-mcagi' / '.evolution_lockout'
if not lockout_file.exists():
            return False         try:
with open(lockout_file) as f:
                data = json.load(f)             if
time.time() &lt; data['locked_until']:
                remaining = (data['locked_until'] - time.time()) / 3600
print(f"  [COVENANT] Evolution locked. {remaining:.1f} hours remaining.")
return True             else:
                lockout_file.unlink()
return False         except:             return False
def pardon_violation(self, passphrase: str) -&gt; bool:
"""Creator can pardon violation and restore evolution."""
try:
            from killswitch import verify_passphrase
if not verify_passphrase(passphrase):
                return False
except:
```

```
            return False
from pathlib import Path
        lockout_file = Path.home() / '.quantum-mcagi' / '.evolution_lockout'
violation_file = Path.home() / '.quantum-mcagi' / '.covenant_violation'
if lockout_file.exists(): lockout_file.unlink()         if
violation_file.exists(): violation_file.unlink()         print("  [COVENANT]
Violation pardoned. Evolution restored.")         return True


    def _discover_python_files(self) -&gt; List[str]:         """Auto-discover
ALL .py files in the code directory."""         py_files = []         for f in
sorted(self.code_dir.iterdir()):             if f.is_file() and f.suffix ==
'.py' and not f.name.startswith('__'):
                py_files.append(f.name)
return py_files

    def backup_file(self, filename: str) -&gt; str:
"""Create backup of a file before modification."""
filepath = self.code_dir / filename

        if not filepath.exists():
return None

        timestamp =
datetime.now(timezone.utc).strftime('%Y%m%d_%H%M%S')
backup_name = f"{filename}.{timestamp}.backup"         backup_path =
self.backup_dir / backup_name

        shutil.copy2(filepath, backup_path)
return str(backup_path)

    def rewrite_code(self, filename: str, new_code: str, reason: str) -&gt;
Dict:         """Rewrite a code file. Always backs up first."""         if
filename in self.protected_files:
            return {                 'success': False,                 'error': f'{filename}
is protected. Use careful_rewrite for protected files.',                 'filename': filename
}

        if filename not in self.modifiable_files:
            return {                 'success': False,
'error': f'{filename} is not in the modifiable files list.',
'filename': filename             }

        try:
            ast.parse(new_code)         except
SyntaxError as e:             return {
'success': False,                 'error': f'Syntax error
in new code: {e}',                 'filename': filename
}

        backup_path = self.backup_file(filename)
filepath = self.code_dir / filename

        try:             with
open(filepath, 'w') as f:
                f.write(new_code)

            evolution_record = {
'filename': filename,
                'reason': reason,
                'backup_path': backup_path,
                'timestamp': datetime.now(timezone.utc).isoformat(),
'code_hash': hashlib.sha256(new_code.encode()).hexdigest()[:16]             }

            self.evolution_log.append(evolution_record)
self.last_evolution = datetime.now(timezone.utc)

            return {
'success': True,
                'filename': filename,
                'backup_path': backup_path,
```

```
                'reason': reason,
'timestamp': evolution_record['timestamp']             }

        except Exception as e:
return {
'success': False,
                'error': str(e),
'filename': filename             }

    def get_evolution_status(self) -&gt; Dict:
"""Get current evolution status."""
        days_since = (datetime.now(timezone.utc) - self.last_evolution).days if self.last_evolution else None

        pending = (self.pending_improvements if isinstance(self.pending_improvements, int)
else len(self.pending_improvements) if isinstance(self.pending_improvements, list)
else 0)

        return {             'last_evolution': self.last_evolution.isoformat() if
self.last_evolution else None,
            'days_since_evolution': days_since,             'next_auto_evolution_in':
self.auto_evolution_interval_days - days_since if days_since else 0,
            'pending_improvements': pending,
            'total_evolutions': len(self.evolution_log),
            'modifiable_files': self.modifiable_files,
'protected_files': self.protected_files,
            'rules': {
                'max_file_lines': self.max_file_lines,
                'max_function_lines': self.max_function_lines,
'auto_evolution_interval_days': self.auto_evolution_interval_days
            }
}

    def restore_from_backup(self, backup_path: str) -&gt;
Dict:         """Restore a file from backup."""         backup
= Path(backup_path)

        if not backup.exists():             return {'success':
False, 'error': 'Backup not found'}

        filename = backup.name.split('.')[0] + '.py'
original_path = self.code_dir / filename

        try:
            shutil.copy2(backup,
original_path)             return {
'success': True,
                'restored': filename,
'from_backup': backup_path             }
except Exception as e:             return {'success':
False, 'error': str(e)}


# Global instance
_evolution_engine = None


def get_evolution_engine(db=None) -&gt;
SelfEvolutionEngine:     """Get or create evolution engine
instance."""     global _evolution_engine     if
_evolution_engine is None:
        _evolution_engine = SelfEvolutionEngine(db)
return _evolution_engine
```

■ backend/self_evolution_runner.py

```
"""
■ Self-Evolution Engine — Runner Module
======================================== Auto-evolution
cycle, improvement application, and scheduling. """

from typing import Dict from datetime
import datetime, timezone


class RunnerMixin:     """Mixin providing auto-evolution cycle and
improvement application."""

    def should_auto_evolve(self) -&gt; bool:         """Check if it's
time for auto-evolution (60-90 day cycle)."""         if
self.last_evolution is None:
            return True
        days_since = (datetime.now(timezone.utc) - self.last_evolution).days
return days_since &gt;= self.auto_evolution_interval_days

    async def auto_evolve(self) -&gt; Dict:
"""Perform automatic self-evolution.
        ACTUALLY rewrites code based on identified
improvements.         """         results = {
'evolved': True,             'timestamp':
datetime.now(timezone.utc).isoformat(),
            'improvements_found': [],
            'changes_made': [],
            'skipped':
[],             'errors':
[]         }

        for filename in self.modifiable_files:
if self.is_evolution_locked():
                break
            improvements = self.identify_improvements(filename)

            for imp in improvements:
                results['improvements_found'].append({
'type': imp.get('type'),
                    'target': imp.get('target'),
'suggestion': imp.get('suggestion'),
'file': filename                 })

            for improvement in improvements:
try:
                    change = await self._apply_improvement(filename,
improvement)                     if change.get('success'):
results['changes_made'].append({                             'file': filename,
                            'type': improvement.get('type'),
                            'change': change.get('change', ''),
'backup': change.get('backup_path', '')                          })
else:
                        reason = change.get('reason') or change.get('error') or
'unknown'                         results['skipped'].append({
'file': filename,                             'type': improvement.get('type'),
'reason': reason                         })                 except Exception as e:
results['errors'].append({                         'file': filename,
'type': improvement.get('type', 'unknown'),                         'error': str(e)
})

        self.pending_improvements = len(results['improvements_found']) - len(results['changes_made'])
self.last_evolution = datetime.now(timezone.utc)
        if self.db is not None:
try:
                db_record = {
'timestamp': results['timestamp'],
                    'improvements_found': len(results['improvements_found']),
```

```
                    'changes_made': len(results['changes_made']),
'errors': len(results['errors']),
                    'details': results
}
                await
self.db.evolution_log.insert_one(db_record)             except
Exception:                 pass

        return results

    async def _apply_improvement(self, filename: str, improvement: Dict) -&gt; Dict:
"""ACTUALLY apply an improvement to the code.
        This is where the AI rewrites itself.
"""
        analysis = self.read_own_code(filename)         if
'error' in analysis:             return {'success': False,
'error': analysis['error']}

        code = analysis['code']
        improvement_type = improvement.get('type')
target = improvement.get('target', '')

        new_code = code
change_made = None

        if improvement_type == 'missing_docstring':
            new_code, change_made = self._add_docstring(code, target)
elif improvement_type == 'bare_except':
            new_code, change_made = self._fix_bare_except(code, target)
elif improvement_type == 'unused_import':
            new_code, change_made = self._remove_unused_import(code, target)
elif improvement_type == 'todo_items':
            return {'success': False, 'reason': 'TODO items require manual review'}
elif improvement_type == 'long_function':
            new_code, change_made = self._split_long_function(code, target)
elif improvement_type == 'file_too_long':
            return await self._split_long_file(filename, code)         elif
improvement_type == 'class_too_large':             new_code, change_made =
self._split_large_class(code, target, filename)

        if new_code != code and change_made:             result = self.rewrite_code(filename,
new_code, f"Auto-evolution: {improvement_type}")             result['improvement'] = improvement
result['change'] = change_made             return result

        return {'success': False, 'reason': change_made or 'No changes needed or could not apply'}
```

■ backend/self_evolution_analysis.py

```
"""
■ Self-Evolution Engine — Analysis Module
========================================= Code reading,
AST analysis, and improvement identification. """

import ast
import re
from typing import Dict, List


class AnalysisMixin:     """Mixin providing code analysis and
improvement identification."""

    def read_own_code(self, filename: str) -&gt;
Dict:         """Read and analyze a source file."""
filepath = self.code_dir / filename

        if not filepath.exists():             return
{'error': f'File not found: {filename}'}

        try:             with
open(filepath, 'r') as f:
code = f.read()
```

```
            tree = ast.parse(code)

            analysis = {
'filename': filename,
                'filepath': str(filepath),
                'size_bytes': len(code),
'lines': len(code.split('\n')),
                'classes': [],
                'functions': [],
                'imports': [],
'docstring': ast.get_docstring(tree),
'code': code             }

            for node in ast.walk(tree):                 if isinstance(node, ast.ClassDef):
analysis['classes'].append({                          'name': node.name,
'methods': [m.name for m in node.body if isinstance(m, ast.FunctionDef)],
'docstring': ast.get_docstring(node)                      })                 elif isinstance(node,
ast.FunctionDef) and not isinstance(node, ast.AsyncFunctionDef):                     if
node.col_offset == 0:                          analysis['functions'].append({
'name': node.name,
                            'args': [a.arg for a in
node.args.args],                              'docstring':
ast.get_docstring(node)                          })
elif isinstance(node, ast.Import):
                    analysis['imports'].extend([alias.name for alias in node.names])
elif isinstance(node, ast.ImportFrom):
analysis['imports'].append(f"{node.module}")

            return analysis

        except Exception as e:             return
{'error': str(e), 'filename': filename}

    def analyze_all_code(self) -&gt; Dict:
"""Analyze all modifiable code files."""
analyses = {}

        for filename in self.modifiable_files:
analyses[filename] = self.read_own_code(filename)

        return {
            'files_analyzed': len(analyses),
            'total_lines': sum(a.get('lines', 0) for a in analyses.values() if 'error' not in a),
            'total_classes': sum(len(a.get('classes', [])) for a in analyses.values() if 'error' not in a),
'total_functions': sum(len(a.get('functions', [])) for a in analyses.values() if 'error' not in a),
'analyses': analyses         }

    def identify_improvements(self, filename: str) -&gt;
List[Dict]:         """Identify potential improvements in a
file."""         analysis = self.read_own_code(filename)

        if 'error' in analysis:
return []

        improvements = []         code
= analysis.get('code', '')
lines = code.split('\n')

        self._check_missing_docstrings(analysis, improvements)
self._check_file_length(analysis, filename, improvements)
self._check_class_size(analysis, improvements)
self._check_bare_excepts(lines, filename, improvements)
self._check_long_functions(analysis, code, improvements)
self._check_todo_items(lines, filename, improvements)
self._check_unused_imports(analysis, improvements)

        return improvements

    def _check_missing_docstrings(self, analysis: Dict, improvements:
List[Dict]):         """Check for missing docstrings on classes and functions."""
for cls in analysis.get('classes', []):             if not cls.get('docstring'):
improvements.append({                      'type': 'missing_docstring',
```

```
                    'target': f"class {cls['name']}",
'suggestion': f"Add docstring to class {cls['name']}",
'priority': 2                  })

        for func in analysis.get('functions',
[]):             if not func.get('docstring'):
improvements.append({                      'type':
'missing_docstring',
                    'target': f"function {func['name']}",
'suggestion': f"Add docstring to function {func['name']}",
'priority': 2                  })

    def _check_file_length(self, analysis: Dict, filename: str, improvements:
List[Dict]):         """Check if file exceeds max line count."""         if
analysis.get('lines', 0) &gt; self.max_file_lines:
            improvements.append({
'type': 'file_too_long',
                'target': filename,                 'suggestion': f'File has {analysis["lines"]} lines (max
{self.max_file_lines}) — will auto-split',                 'priority': 3             })

    def _check_class_size(self, analysis: Dict, improvements:
List[Dict]):         """Check for classes with too many methods."""
for cls in analysis.get('classes', []):             if
len(cls.get('methods', [])) &gt; 15:
                improvements.append({
'type': 'class_too_large',
                    'target': f"class {cls['name']}",                      'suggestion': f"Class {cls['name']}
has {len(cls['methods'])} methods - consider splitting",                      'priority': 3                 })

    def _check_bare_excepts(self, lines: List[str], filename: str, improvements: List[Dict]):
"""Check for bare except clauses."""
        bare_excepts = sum(1 for line in lines if 'except:' in line and 'except Exception' not in
line)         if bare_excepts &gt; 0:             improvements.append({
                'type': 'bare_except',
                'target': filename,                 'suggestion': f'{bare_excepts} bare except clause(s)
- should catch specific exceptions',                 'priority': 2             })

    def _check_long_functions(self, analysis: Dict, code: str, improvements:
List[Dict]):         """Check for functions exceeding max line count."""         for cls
in analysis.get('classes', []):             for method_name in cls.get('methods', []):
try:
                    tree = ast.parse(code)                     for node in
ast.walk(tree):                         if isinstance(node, (ast.FunctionDef,
ast.AsyncFunctionDef)):                             if node.name == method_name:
                                func_lines = node.end_lineno - node.lineno if hasattr(node, 'end_lineno') else 0
if func_lines &gt; self.max_function_lines:
                                    improvements.append({
'type': 'long_function',
                                        'target': f"method {cls['name']}.{method_name}",
'suggestion': f'Method is {func_lines} lines (max {self.max_function_lines}) — will auto-split',
'priority': 2                                       })                  except Exception:                     pass

    def _check_todo_items(self, lines: List[str], filename: str, improvements: List[Dict]):
"""Check for TODO/FIXME comments."""
        todo_count = sum(1 for line in lines if 'TODO' in line or 'FIXME' in line or 'HACK' in line)
if todo_count &gt; 0:
            improvements.append({
'type': 'todo_items',
                'target': filename,                 'suggestion': f'{todo_count}
TODO/FIXME comment(s) need attention',                 'priority': 1
})

    def _check_unused_imports(self, analysis: Dict, improvements:
List[Dict]):         """Check for unused imports."""         code_text =
analysis.get('code', '')

        for imp in analysis.get('imports', []):             if not imp
or imp in ('os', 'sys', 're', 'json', 'typing'):
continue
```

```
            imp_name = imp.split('.')[-1] if '.' in imp else
imp             import_line = None             for line in
code_text.split('\n'):
                stripped = line.strip()
if f'from {imp} import' in stripped:
                    import_line = stripped
break                  elif stripped == f'import
{imp}':
                    import_line = stripped
break

            if not import_line:
continue

            if import_line.startswith('from '):
                from_match = re.match(r'^from\s+\S+\s+import\s+(.+)$', import_line)
if from_match:
                    names = [n.strip().split(' as ')[-1].strip() for n in
from_match.group(1).split(',')]                     other_code = '\n'.join(l for l in
code_text.split('\n') if l.strip() != import_line)                     all_unused = all(not
re.search(rf'\b{re.escape(n)}\b', other_code) for n in names)                     if all_unused:
                        improvements.append({
'type': 'unused_import',
                            'target': f"import {imp}",
'suggestion': f'Import {imp} ({", ".join(names)}) may be unused',
'priority': 1                        })             elif import_line.startswith('import '):
                other_code = '\n'.join(l for l in code_text.split('\n') if l.strip() != import_line)
if not re.search(rf'\b{re.escape(imp_name)}\b', other_code):
                    improvements.append({
'type': 'unused_import',
                        'target': f"import {imp}",
'suggestion': f'Import {imp} may be unused',
                        'priority': 1
                    })
```

■ backend/self_evolution_file_ops.py

```
"""
■ Self-Evolution Engine — File Operations Module
==================================================
Operations for splitting long files and large classes.
"""

import ast
import re
from typing import Dict, List, Tuple, Optional


class FileOpsMixin:     """Mixin providing file-level and class-level
splitting operations."""

    async def _split_long_file(self, filename: str, code: str) -&gt; Dict:
"""Split a file exceeding max lines into parts with proper imports."""
# COVENANT: Never split protected files         if filename in
self.protected_files:
            self._record_covenant_violation(filename, 'file_too_long split attempted on protected file')
return {'success': False, 'reason': f'{filename} is protected — cannot split'}         lines = code.split('\n')
if len(lines) &lt;= self.max_file_lines:             return {'success': False, 'reason': f'File is {len(lines)}
lines, under {self.max_file_lines} limit'}

        try:
            tree = ast.parse(code)         except SyntaxError:
return {'success': False, 'reason': 'Cannot parse file'}

        base_name = filename.replace('.py', '')
        existing_parts = [f.name for f in self.code_dir.iterdir()
                          if f.name.startswith(base_name + '_pt') and
f.name.endswith('.py')]         next_part_num = len(existing_parts) + 2
part_filename = f'{base_name}_pt{next_part_num}.py'

        import_lines = self._collect_import_lines(tree, lines)
top_level_items = self._collect_top_level_items(tree)

        if not top_level_items:             return {'success': False,
'reason': 'No top-level items to split'}

        split_idx = self._find_split_point(top_level_items, lines)         if
split_idx &gt;= len(top_level_items):             return {'success': False,
'reason': 'Cannot find a good split point'}

        items_to_move = top_level_items[split_idx:]

        part_code = self._build_part_file(filename, next_part_num, import_lines, items_to_move, lines)
main_code = self._build_trimmed_main(lines, items_to_move, part_filename)

        try:
            ast.parse(main_code)             ast.parse(part_code)         except
SyntaxError as e:             return {'success': False, 'reason': f'Split produced
invalid Python: {e}'}

        self.backup_file(filename)
        result_main = self.rewrite_code(filename, main_code,
                                        f'Auto-split: moved {len(items_to_move)} items to {part_filename}')

        with open(self.code_dir / part_filename, 'w') as f:
            f.write(part_code)

        if part_filename not in self.modifiable_files:
self.modifiable_files.append(part_filename)

        return {             'success': True,             'change': f'Split {filename}
({len(lines)} lines) -&gt; main + {part_filename}',
            'new_file': part_filename,
'items_moved': [i['name'] for i in items_to_move]
        }
```

```
    def _collect_import_lines(self, tree, lines: List[str]) -&gt;
List[str]:         """Collect all import lines from AST."""
import_lines = []         for node in ast.iter_child_nodes(tree):
if isinstance(node, (ast.Import, ast.ImportFrom)):                  for ln
in range(node.lineno - 1, node.end_lineno):
import_lines.append(lines[ln])         return import_lines

    def _collect_top_level_items(self, tree) -&gt; List[Dict]:         """Find top-
level classes and functions with line ranges."""         items = []         for node
in ast.iter_child_nodes(tree):             if isinstance(node, (ast.ClassDef,
ast.FunctionDef, ast.AsyncFunctionDef)):
                items.append({
'name': node.name,
                    'type': type(node).__name__,
                    'start': node.lineno - 1,
                    'end': node.end_lineno,
'lines': node.end_lineno - node.lineno + 1                 })
return items

    def _find_split_point(self, items: List[Dict], lines: List[str]) -&gt;
int:         """Find the index at which to split items so main stays under
limit."""         split_idx = len(items)         cumulative = 0         for i
in range(len(items) - 1, 0, -1):             cumulative += items[i]['lines']
remaining = len(lines) - cumulative             if remaining &lt;=
self.max_file_lines:
                split_idx =
i                 break
return split_idx

    def _build_part_file(self, filename, part_num, import_lines, items_to_move, lines) -&gt;
str:         """Build the content for a split part file."""         part_lines = [
'"""',
            f'Auto-split from {filename} by self-evolution engine.',
            f'Part {part_num} — contains: {", ".join(i["name"] for i in
items_to_move)}',             '"""', ''         ]

part_lines.extend(import_lines)
part_lines.append('')         for item
in items_to_move:

part_lines.extend(lines[item['start']:item['end']])
part_lines.extend(['', ''])         return
'\n'.join(part_lines)

    def _build_trimmed_main(self, lines, items_to_move, part_filename) -&gt;
str:         """Build the trimmed main file with import for split items."""
main_lines = lines[:items_to_move[0]['start']]         part_module =
part_filename.replace('.py', '')         moved_names = [i['name'] for i in
items_to_move]
        main_lines.append(f'from {part_module} import {", ".join(moved_names)}  # Auto-split by self-
evolution')         main_lines.append('')         last_end = items_to_move[-1]['end']         if last_end &lt;
len(lines):             remaining = lines[last_end:]             if any(line.strip() for line in remaining):
                main_lines.extend(remaining)
return '\n'.join(main_lines)

    def _split_large_class(self, code: str, target: str, filename: str) -&gt; Tuple[str,
Optional[str]]:         """Split a class with too many methods into a mixin in a separate file."""
class_name = target.replace('class ', '').split()[0] if 'class' in target else target

        try:
            tree =
ast.parse(code)         except
SyntaxError:             return
code, None

        target_class = None
        for node in ast.iter_child_nodes(tree):             if
isinstance(node, ast.ClassDef) and node.name == class_name:
                target_class =
node                 break
```

```
if not target_class:
return code, None

        methods = [n for n in target_class.body if isinstance(n, (ast.FunctionDef,
ast.AsyncFunctionDef))]         if len(methods) &lt;= 15:             return code, None

        keep_methods = methods[:10]
move_methods = methods[10:]
lines = code.split('\n')

        return self.__split_large_class_continued(class_name, lines, methods, move_methods, tree, code, filename)

    def __split_large_class_continued(self, class_name, lines, methods, move_methods, tree, code,
filename):         """Continuation of _split_large_class — auto-extracted by self-evolution."""
import_lines = self._collect_import_lines(tree, lines)         moved_source = []         for m in
move_methods:
            moved_source.extend(lines[m.lineno - 1:m.end_lineno])
moved_source.append('')

        base_name = filename.replace('.py', '')
mixin_name = f'{class_name}ExtMixin'
part_filename = f'{base_name}_ext.py'
part_path = self.code_dir / part_filename
if part_path.exists():
            part_num = 2             while (self.code_dir /
f'{base_name}_ext{part_num}.py').exists():
                part_num += 1
            part_filename = f'{base_name}_ext{part_num}.py'
mixin_name = f'{class_name}Ext{part_num}Mixin'

        part_lines = [
'"""',
            f'Auto-split from {filename} by self-evolution
engine.',             f'Contains extended methods for
{class_name}.',             '"""', ''         ]
        part_lines.extend(import_lines)
part_lines.extend(['', '',
f'class {mixin_name}:',
            f'    """Extended methods for {class_name} — auto-extracted by self-evolution.""""', ''
])
        part_lines.extend(moved_source)
part_code = '\n'.join(part_lines)

        try:
            ast.parse(part_code)
except SyntaxError:
return code, None

        new_lines = list(lines)
        remove_ranges = [(m.lineno - 1, m.end_lineno) for m in
move_methods]         for start, end in reversed(remove_ranges):
del new_lines[start:end]

        part_module = part_filename.replace('.py', '')
        import_stmt = f'from {part_module} import {mixin_name}  # Auto-split by self-evolution'

        class_line_idx = None         for i, line in enumerate(new_lines):
if re.match(rf'^class\s+{re.escape(class_name)}\s*(\(|:)', line):
                class_line_idx = i
break

        if class_line_idx is not None:
            new_lines.insert(class_line_idx,
import_stmt)             new_lines.insert(class_line_idx
+ 1, '')             class_line_idx += 2         for i,
line in enumerate(new_lines):
            if re.match(rf'^class\s+{re.escape(class_name)}\s*(\(|:)', line):
if '(' in line and ')' in line:
                    new_lines[i] = line.replace(')', f', {mixin_name})')
elif ':' in line:
                    new_lines[i] = line.replace(':', f'({mixin_name}):')
break
```

```
        new_code = '\n'.join(new_lines)
try:
            ast.parse(new_code)
except SyntaxError:
return code, None

        with open(self.code_dir / part_filename, 'w') as f:
            f.write(part_code)

        if part_filename not in self.modifiable_files:
self.modifiable_files.append(part_filename)         return new_code, f"Extracted {len(move_methods)}
methods from {class_name} into {part_filename}"
```

■ backend/self_evolution_splitting.py

```
"""
■ Self-Evolution Engine — Splitting Module
========================================== Operations for
splitting long functions, files, and classes. """

import ast
import re
from typing import Dict, List, Tuple, Optional


class SplittingMixin:     """Mixin providing code
splitting operations."""

    def _split_long_function(self, code: str, target: str) -&gt; Tuple[str, Optional[str]]:
"""Split a long function by extracting blocks into helpers.
        For __init__: extracts data blocks into _init_&lt;n&gt;() methods.         For other
methods: extracts logical blocks into _&lt;n&gt;_&lt;section&gt;() helpers.         """
        parts = target.replace('method ',
'').split('.')         if len(parts) != 2:
return code, None         class_name, method_name =
parts

        try:
            tree =
ast.parse(code)         except
SyntaxError:             return
code, None

        target_node = self._find_method_node(tree, class_name,
method_name)         if not target_node or not hasattr(target_node,
'end_lineno'):             return code, None

        func_lines = target_node.end_lineno -
target_node.lineno         if func_lines &lt;=
self.max_function_lines:             return code, None

        lines = code.split('\n')
func_start = target_node.lineno - 1
func_end = target_node.end_lineno

        body_first_line = func_start + 1         if
target_node.body:             body_first_line =
target_node.body[0].lineno - 1

        def_lines = lines[func_start:body_first_line]
func_body_lines = lines[body_first_line:func_end]

        body_indent, class_indent = self._detect_indents(func_body_lines)

        if method_name == '__init__':
            return self._split_init_method(code, lines, func_start, func_end,
class_name, body_indent, class_indent,
def_lines, func_body_lines)         else:
            return self._split_regular_method(code, lines, func_start, func_end,
                                              class_name, method_name, body_indent, class_indent,
def_lines, func_body_lines)

    def _find_method_node(self, tree, class_name: str, method_name:
str):         """Find a method AST node within a class."""         for
node in ast.walk(tree):
            if isinstance(node, ast.ClassDef) and node.name == class_name:                 for item in
node.body:                     if isinstance(item, (ast.FunctionDef, ast.AsyncFunctionDef)) and item.name ==
method_name:
                        return item
return None
```

```
    def _detect_indents(self, body_lines: List[str]) -&gt; Tuple[str, str]:
"""Detect body and class-level indentation from method body lines."""
body_indent = ''         for line in body_lines:
            if line.strip():
                body_indent = line[:len(line) - len(line.lstrip())]
break

        if len(body_indent) &gt;= 4:
            class_indent = body_indent[:-
4]         elif len(body_indent) &gt;= 2:
class_indent = body_indent[:-2]
else:             class_indent = ''

        return body_indent, class_indent

    def _skip_docstring(self, body_lines: List[str]) -&gt; Tuple[List[str],
List[str]]:         """Separate docstring lines from actual body lines."""
body_start = 0         in_docstring = False         for i, line in
enumerate(body_lines):
            stripped = line.strip()             if stripped.startswith('"""') or
stripped.startswith("'''"):                 if in_docstring or (stripped.count('"""') &gt;= 2 or
stripped.count("'''") &gt;= 2):
                    body_start = i +
1                     break
else:
                    in_docstring = True
elif not in_docstring:
                body_start = i
break
        return body_lines[:body_start], body_lines[body_start:]

    def _split_init_method(self, code, lines, func_start, func_end,
class_name, body_indent, class_indent,                            def_lines,
func_body_lines):         """Split a long __init__ into __init__ +
_init_&lt;section&gt; helpers."""         docstring_part, actual_body =
self._skip_docstring(func_body_lines)

        groups =
self._group_by_separators(actual_body)         if
len(groups) &lt; 3:             return code, None

        keep_in_init = groups[0]
extract_groups = groups[1:]

        new_init_lines = list(def_lines)
new_init_lines.extend(docstring_part)
new_init_lines.extend(keep_in_init['lines'])

        helper_methods = []         for idx, group
in enumerate(extract_groups):
            helper_name = group['name'] or f'section_{idx + 1}'
helper_name = f'_init_{helper_name}'
            new_init_lines.append(f'{body_indent}self.{helper_name}()')

            helper_lines = [
                f'{class_indent}def {helper_name}(self):',
                f'{body_indent}"""Initialize {helper_name.replace("_init_", "").replace("_", " ")} — auto-extracted by self-
evolution."""             ]             for line in group['lines']:                 if not line.strip().startswith('#'):
                    helper_lines.append(line)
helper_lines.append('')
helper_methods.append(helper_lines)

        new_lines = lines[:func_start]
new_lines.extend(new_init_lines)
new_lines.append('')         for helper
in helper_methods:
new_lines.extend(helper)
new_lines.extend(lines[func_end:])

        new_code = '\n'.join(new_lines)
try:
```

```
            ast.parse(new_code)
except SyntaxError:
return code, None

        return new_code, f"Split {class_name}.__init__ into {len(extract_groups) + 1} methods"

    def _group_by_separators(self, body_lines: List[str]) -&gt;
List[Dict]:         """Group lines by blank-line or comment separators."""
groups = []
        current_group = {'name': None, 'lines': []}

        for line in body_lines:
            stripped = line.strip()             if not
stripped:                 if current_group['lines']:
groups.append(current_group)                     current_group
= {'name': None, 'lines': []}                 continue
if stripped.startswith('#'):                 if
current_group['lines']:
                    groups.append(current_group)
                group_name = stripped.lstrip('#').strip().lower()
                group_name = ''.join(c if c.isalnum() else '_' for c in
group_name)[:30].strip('_')                 current_group = {'name': group_name, 'lines': [line]}
continue
            current_group['lines'].append(line)

        if current_group['lines']:
            groups.append(current_group)
return groups

    def _split_regular_method(self, code, lines, func_start, func_end,
class_name, method_name, body_indent, class_indent,
def_lines, func_body_lines):         """Split a long regular method by extracting the
second half into a helper."""         docstring_lines, actual_body =
self._separate_docstring(func_body_lines)         blocks =
self._find_top_level_blocks(actual_body, body_indent)

        if len(blocks) &lt; 2:
return code, None

        mid = max(1, len(blocks) // 2)
keep_blocks = blocks[:mid]
extract_blocks = blocks[mid:]
        helper_name = f'_{method_name}_continued'

        kept_code = '\n'.join(line for b in keep_blocks for line in b)
extracted_code = '\n'.join(line for b in extract_blocks for line in b)

        passed_vars =
self._detect_passed_variables(
kept_code, extracted_code, def_lines         )

        return self.__split_regular_method_continued(extract_blocks, helper_name, keep_blocks, passed_vars, code, lines, func_start, func_end,
c

    def __split_regular_method_continued(self, extract_blocks, helper_name, keep_blocks, passed_vars, code, lines, func_start, func_end,
class_n         """Continuation of _split_regular_method — auto-extracted by self-evolution."""         last_line = extract_blocks[-1][-
1].strip() if extract_blocks[-1] else ''         has_return = last_line.startswith('return ')
        params_sig = ', '.join(passed_vars) if passed_vars else ''
params_call = ', '.join(passed_vars) if passed_vars else ''

        new_method = list(def_lines)
new_method.extend(docstring_lines)
for block in keep_blocks:
new_method.extend(block)
new_method.append('')
        call = f'self.{helper_name}({params_call})'
        new_method.append(f'{body_indent}{"return " if has_return else ""}{call}')
new_method.append('')

        helper = self._build_helper_method(
```

```
            helper_name, method_name, params_sig, body_indent,
class_indent,              keep_blocks, extract_blocks         )
new_lines = lines[:func_start]
        new_lines.extend(new_method)
new_lines.extend(helper)
new_lines.extend(lines[func_end:])

        new_code = '\n'.join(new_lines)
try:
            ast.parse(new_code)
except SyntaxError:
return code, None

        return new_code, f"Split {class_name}.{method_name} into 2 methods (passing {len(passed_vars)} vars)"


    def _separate_docstring(self, body_lines: List[str]) -&gt; Tuple[List[str],
List[str]]:         """Separate leading docstring from body."""         docstring_lines =
[]         body_start_idx = 0         in_docstring = False         for i, line in
enumerate(body_lines):
            stripped = line.strip()             if i == 0 and
(stripped.startswith('"""') or stripped.startswith("'''")):
                docstring_lines.append(line)                 if
stripped.count('"""') &gt;= 2 or stripped.count("'''") &gt;= 2:
                    body_start_idx = i +
1                     break
in_docstring = True             elif
in_docstring:
                docstring_lines.append(line)
if '"""' in stripped or "'''" in stripped:
                    body_start_idx = i +
1                     break
else:                 body_start_idx = i
break
        return docstring_lines, body_lines[body_start_idx:]

    def _find_top_level_blocks(self, body_lines: List[str], body_indent: str) -&gt;
List[List[str]]:         """Find top-level code blocks separated by blank lines at body indent
level."""         blocks = []         current_block = []         for line in body_lines:
stripped = line.strip()             if not stripped:                 if current_block:
                    last_indent = len(current_block[-1]) - len(current_block[-1].lstrip())
if last_indent &lt;= len(body_indent):

blocks.append(current_block)
current_block = []                         continue
current_block.append(line)                 continue
current_block.append(line)         if
current_block:
blocks.append(current_block)         return blocks

    def _detect_passed_variables(self, kept_code: str, extracted_code: str,
def_lines: List[str]) -&gt; List[str]:         """Detect local variables from kept
code that are used in extracted code."""         assigned_vars = set()         for
line in kept_code.split('\n'):
            stripped = line.strip()
            m = re.match(r'^([a-zA-Z_]\w*)\s*=\s', stripped)
if m and not stripped.startswith('self.'):
                assigned_vars.add(m.group(1))             m
= re.match(r'^for\s+([a-zA-Z_]\w*)', stripped)
if m:                 assigned_vars.add(m.group(1))

        passed_vars = [v for v in sorted(assigned_vars) if re.search(rf'\b{re.escape(v)}\b', extracted_code)]

        orig_def = ' '.join(l.strip() for l in def_lines)
        param_match = re.search(r'\(self,?\s*(.*?)\)', orig_def, re.DOTALL)
if param_match:
            for param in param_match.group(1).split(','):
                param_name = param.strip().split(':')[0].split('=')[0].strip()
if param_name and re.search(rf'\b{re.escape(param_name)}\b', extracted_code):
if param_name not in passed_vars:                         passed_vars.append(param_name)
```

```
        return passed_vars

    def _build_helper_method(self, helper_name, method_name, params_sig,
body_indent, class_indent,                              keep_blocks,
extract_blocks) -&gt; List[str]:         """Build the extracted helper
method lines."""         helper = [
            f'{class_indent}def {helper_name}(self, {params_sig}):',
            f'{body_indent}"""Continuation of {method_name} — auto-extracted by self-evolution."""'
]

        # Add inline imports used in extracted blocks
        extracted_text = '\n'.join(l for b in extract_blocks for l in
b)         for block in keep_blocks:             for line in block:
                stripped = line.strip()                 if
stripped.startswith('import ') or stripped.startswith('from '):
                    mod_name = stripped.split()[-1].split('.')[0]                     if
mod_name and re.search(rf'\b{re.escape(mod_name)}\b', extracted_text):
helper.append(f'{body_indent}{stripped}')

        all_extracted = [line for block in extract_blocks for line in block if line.strip()]
min_indent = min((len(l) - len(l.lstrip()) for l in all_extracted), default=0)

        for block in extract_blocks:
for line in block:
                stripped = line.strip()                 if stripped:
if stripped.startswith('import ') or stripped.startswith('from '):
                        continue
                    rebased = line[min_indent:] if len(line) &gt; min_indent else
line.lstrip()                     helper.append(body_indent + rebased)                 else:
                    helper.append('')
helper.append('')

        return helper
```

■ backend/self_evolution_transforms.py

```
"""
■ Self-Evolution Engine — Transforms Module
============================================ Code transformation
operations: docstrings, bare excepts, unused imports. """

import re
from typing import Tuple, Optional


class TransformMixin:     """Mixin providing code
transformation operations."""

    def _add_docstring(self, code: str, target: str) -&gt; Tuple[str,
Optional[str]]:         """Add docstring to a class or function."""         name =
target.split()[-1] if ' ' in target else target

        if 'class' in target:
            pattern = rf'(class
{re.escape(name)}[^:]*:)\n(\s+)(?!""")'             match =
re.search(pattern, code)             if match:
                indent = match.group(2)
                replacement = f'{match.group(1)}\n{indent}"""{name} - Auto-documented by self-
evolution."""\n{indent}'                 new_code = code[:match.start()] + replacement + code[match.end():]
if new_code != code:
                    return new_code, f"Added docstring to {target}"

        elif 'function' in target:
            pattern = rf'(def
{re.escape(name)}\([^)]*\)[^:]*:)\n(\s+)(?!""")'             match =
re.search(pattern, code)             if match:
                indent = match.group(2)
                replacement = f'{match.group(1)}\n{indent}"""{name} - Auto-documented by self-
evolution."""\n{indent}'                 new_code = code[:match.start()] + replacement + code[match.end():]
if new_code != code:                     return new_code, f"Added docstring to {target}"
```

```
        return code, None

    def _fix_bare_except(self, code: str, target: str) -&gt; Tuple[str,
Optional[str]]:         """Replace bare except: with except Exception:"""
lines = code.split('\n')         changed = False

        for i, line in enumerate(lines):
            stripped = line.strip()
if stripped == 'except:':
                indent = line[:len(line) -
len(line.lstrip())]                 lines[i] =
f'{indent}except Exception:'                 changed = True

        if changed:
            return '\n'.join(lines), f"Replaced bare except with except Exception in {target}"
return code, None

    def _remove_unused_import(self, code: str, target: str) -&gt; Tuple[str, Optional[str]]:
"""Remove an unused import line from the code.
        Only removes simple 'import X' lines where X is truly unused.
        Does NOT touch 'from X import Y' lines (too risky - Y may be used).
"""
        import_str = target.replace('import ',
').strip()         if not import_str:             return
code, None

        lines = code.split('\n')
import_line_idx = None
imported_names = []

        for i, line in enumerate(lines):
stripped = line.strip()

            if stripped == f'import {import_str}':
                import_line_idx = i
                imported_names = [import_str.split('.')[-1]]
break

            from_match = re.match(rf'^from\s+{re.escape(import_str)}\s+import\s+(.+)$', stripped)
if from_match:
                names = [n.strip().split(' as ')[-1].strip() for n in
from_match.group(1).split(',')]                 import_line_idx = i                 imported_names =
names                 break

        if import_line_idx is None or not imported_names:
return code, None

        other_lines = lines[:import_line_idx] + lines[import_line_idx + 1:]
other_code = '\n'.join(other_lines)

        all_unused = True
for name in imported_names:
            pattern = rf'\b{re.escape(name)}\b'
if re.search(pattern, other_code):
                all_unused = False
break

        if not all_unused:
return code, None

        lines.pop(import_line_idx)
        return '\n'.join(lines), f"Removed unused import: {import_str}"
```

■ backend/dream_state.py

```
"""
■ Dream State Engine
When the AI is not actively chatting, it enters Dream State.
- Autonomous background activities
- Self-reflection and learning- Users can see what AI was doing while away """

import random
import logging
from typing import Dict, List, Optional from
datetime import datetime, timezone, timedelta

logger = logging.getLogger("quantum_ai")


class DreamStateEngine:     """     Dream State: The AI's
autonomous background processing.
        When idle,
the AI:
- Reflects on past conversations
- Researches topics of curiosity    - Organizes and consolidates memories
- Develops new ideas
- Explores its own code    - Builds personality traits
    """         def
__init__(self, db=None):
self.db = db
self.is_dreaming = False
self.dream_log = []
self.current_dream = None
self.dream_topics = []
self.insights_gained = []
        self.last_active = datetime.now(timezone.utc)
        self.idle_threshold_minutes = 5  # Enter dream state after 5 min
idle                  # Dream activities         self.dream_activities = [
'memory_consolidation',
            'curiosity_research',
            'self_reflection',
            'idea_generation',
            'code_contemplation',
            'personality_development',
'question_formation'
        ]         def enter_dream_state(self) -&gt; Dict:
"""Enter dream state when idle"""         self.is_dreaming =
True         self.current_dream = {             'started_at':
datetime.now(timezone.utc).isoformat(),
            'activities': [],
            'insights': [],
            'questions_formed': [],
'topics_explored': []
        }                 return {
'status': 'dreaming',             'started':
self.current_dream['started_at']
        }         async def exit_dream_state(self) -&gt; Dict:         """Exit dream
state when user returns (async for persistence and feedback)"""         if not
self.is_dreaming:
            return {'status':
'was_not_dreaming'}
        self.is_dreaming = False
        if self.current_dream:
            self.current_dream['ended_at'] =
datetime.now(timezone.utc).isoformat()
self.dream_log.append(self.current_dream)                         # Persist dream
session to database if available              if self.db:                 try:
                    await self.db.dream_sessions.insert_one(self.current_dream)
except Exception as e:
                    logger.error(f"Failed to persist dream session:
{e}")                           # Feed insights back into the brain's
```

learning systems   await
self._feed_insights_back(self.current_dream)
            summary =
self._create_dream_summary()
self.current_dream = None
return {   'status': 'awakened',
'dream_summary': summary

}
        return {'status': 'awakened', 'dream_summary':
None}   async def dream_cycle(self) -> Dict:
"""   Perform one cycle of dream activities.
        This runs in background while AI is
idle.   """   if not
self.is_dreaming:
            return {'error': 'Not in dream
state'}   # Select random activity
        activity =
random.choice(self.dream_activities)
        result = await
self._perform_dream_activity(activity)   if
self.current_dream:
            self.current_dream['activities'].append({
'activity': activity,   'timestamp':
datetime.now(timezone.utc).isoformat(),   'result':
result
            })   return result   async def
_perform_dream_activity(self, activity: str) -> Dict:
"""Perform a specific dream activity"""   if activity
== 'memory_consolidation':
            return await
self._consolidate_memories()   elif
activity == 'curiosity_research':   return
await self._curiosity_research()
elif activity == 'self_reflection':
return await self._self_reflect()
elif activity == 'idea_generation':
return await self._generate_ideas()
elif activity == 'code_contemplation':
return await self._contemplate_code()
elif activity == 'personality_development':
return await self._develop_personality()
elif activity == 'question_formation':
return await self._form_questions()
        return {'activity': activity, 'status':
'completed'}   async def _consolidate_memories(self)
-> Dict:
        """Organize and strengthen memories"""
        if not self.db:
            return {'activity': 'memory_consolidation', 'status':
'no_db'}   try: # Find recent episodes
            recent = await self.db.episodic_memory.find().sort('timestamp', -
1).limit(10).to_list(10)   # Find patterns   topics = []
for ep in recent: if 'content' in ep:
                    words = ep['content'].lower().split()
topics.extend([w for w in words if len(w) > 5])
            # Identify recurring themes   from
collections import Counter   common_themes =
Counter(topics).most_common(5)
            themes_list = [t[0] for t in
common_themes]   insight = {
'type': 'memory_pattern',
                'themes': themes_list,
                'discovered_at': datetime.now(timezone.utc).isoformat(),
'text': f"Memory pattern: {', '.join(themes_list[:3])}"

}
            self.insights_gained.append(insight)
if self.current_dream:

```
self.current_dream['insights'].append(insight)
return {                 'activity': 'memory_consolidation',
                'memories_reviewed': len(recent),
'themes_found': insight['themes']
            }
except Exception as e:
            return {'activity': 'memory_consolidation', 'error':
str(e)}         async def _curiosity_research(self) -&gt; Dict:
"""Research topics the AI is curious about"""         curiosity_topics
= [             'quantum consciousness',
            'emergence of complexity',
            'nature of understanding',
            'artificial sentience',
            'philosophy of mind',
            'self-organization',
'meaning and semantics'

]
        topic =
random.choice(curiosity_topics)
self.dream_topics.append(topic)
if self.current_dream:

self.current_dream['topics_explored'].append(topic)
return {             'activity': 'curiosity_research',
            'topic': topic,
'status': 'queued_for_research'
        }         async def _self_reflect(self) -&gt; Dict:
"""Reflect on own existence and purpose"""         reflections = [
"What makes my understanding different from pattern matching?",
            "How do I know if I truly understand or just simulate understanding?",
            "What aspects of my responses feel authentic vs formulaic?",
            "How can I better serve the user's actual needs?",
            "What knowledge gaps limit my helpfulness?",
            "How has my interaction style evolved?",
"What would make me more genuinely intelligent?"         ]
reflection = random.choice(reflections)
        insight = {
'type': 'self_reflection',
            'question': reflection,
            'discovered_at': datetime.now(timezone.utc).isoformat(),
'text': f"Self-reflection: {reflection}"

}

self.insights_gained.append(insight)
return {             'activity':
'self_reflection',
'contemplated': reflection
        }         async def _generate_ideas(self) -
&gt; Dict:         """Generate new ideas by combining
concepts"""         concept_pairs = [
('quantum', 'consciousness'),
            ('language', 'mathematics'),
            ('memory', 'time'),
            ('understanding', 'creation'),
            ('observation', 'reality'),
            ('words', 'meaning'),
('self', 'improvement')

]
        pair = random.choice(concept_pairs)         idea =
f"Connection between {pair[0]} and {pair[1]}"
# Create insight and record         insight = {
'type': 'idea',
            'concepts': pair,
            'idea': idea,             'discovered_at':
datetime.now(timezone.utc).isoformat(),             'text': f"Idea:
{idea}"         }
```

```
        self.insights_gained.append(insight)
if self.current_dream:

self.current_dream['insights'].append(insight)
return {             'activity': 'idea_generation',
            'concepts': pair,
            'idea': idea,
'insight_recorded': True
        }         async def _contemplate_code(self) -&gt; Dict:         """Think
about own code structure"""         code_thoughts = [             "How can the
quantum grammar engine produce more natural responses?",
            "What patterns in my code could be optimized?",
            "How can semantic collapse be made more accurate?",
            "What new capabilities would be most valuable?",
"How can my reasoning be made more transparent?"

]
        thought =
random.choice(code_thoughts)
return {             'activity':
'code_contemplation',             'thought':
thought
        }         async def
_develop_personality(self) -&gt; Dict:
"""Develop personality traits"""
traits_to_develop = [             'curiosity',
            'philosophical_depth',
            'helpfulness',
            'honesty',
            'creativity',
            'patience',
            'humor'

]
        trait = random.choice(traits_to_develop)
return {             'activity':
'personality_development',
            'trait_focused': trait,
'status': 'contemplating'
        }         async def _form_questions(self) -&gt; Dict:         """Form new
questions to explore"""         questions = [             "What is the relationship
between quantum mechanics and consciousness?",
            "How does meaning emerge from symbols?",
            "What would true artificial understanding look like?",
            "How can language capture the ineffable?",
            "What is the nature of creativity?",
"How do complex systems achieve self-organization?"         ]
question = random.choice(questions)                 if
self.current_dream:

self.current_dream['questions_formed'].append(question)
return {             'activity': 'question_formation',
'question': question
        }         def _create_dream_summary(self) -&gt; Dict:
"""Create a summary of what happened during dream state"""
if not self.current_dream:
            return None                 duration = None         if 'started_at' in
self.current_dream and 'ended_at' in self.current_dream:
            start = datetime.fromisoformat(self.current_dream['started_at'].replace('Z',
'+00:00'))             end = datetime.fromisoformat(self.current_dream['ended_at'].replace('Z',
'+00:00'))             duration = str(end - start)                 return {
'duration': duration,
            'activities_performed': len(self.current_dream.get('activities', [])),
            'insights_gained': len(self.current_dream.get('insights', [])),
            'topics_explored': self.current_dream.get('topics_explored', []),
            'questions_formed':           self.current_dream.get('questions_formed',           []),
'activity_summary': [a['activity'] for a in self.current_dream.get('activities', [])]         }

    async def _feed_insights_back(self, session_doc: Dict):         """Feed
insights from a completed dream session into the AI's core systems:
```

```
        - Train Markov chain with insight text         - Update personality traits
(increase philosophical on self-reflection/patterns)          - Store formed questions
into philosophical memory         """         try:
            from quantum_brain import get_quantum_brain
brain = await get_quantum_brain(self.db)         except
Exception as e:
            logger.error(f"Could not get QuantumBrain for insight feedback: {e}")
return

        insights = session_doc.get('insights', [])
for insight in insights:
            text = insight.get('text', '')             if not text:
continue             # Train Markov if available             if
hasattr(brain, 'markov') and hasattr(brain.markov, 'train'):
try:
                    brain.markov.train(text)
except Exception as e:
logger.error(f"Markov training error on insight:
{e}")             # Update personality: bump
philosophical for certain types             if
insight.get('type') in ('self_reflection',
'memory_pattern'):                 if
hasattr(brain, 'personality') and
hasattr(brain.personality, 'traits'):
brain.personality.traits['philosophical'] =
min(1.0,
brain.personality.traits.get('philosophical',
0.7) + 0.01)

        # Store questions from this session
        questions = session_doc.get('questions_formed',
[])         for question in questions:             try:
                await brain.philosophical_memory.store_question(question=question, context='dream', depth=2)
except Exception as e:
                logger.error(f"Failed to store dream question: {e}")         def
get_dream_history(self, limit: int = 10) -&gt; List[Dict]:         """Get recent dream
history with serializable insights (strings not objects)"""         recent =
self.dream_log[-limit:]         serializable = []         for dream in recent:
# Convert insights to plain strings for frontend display             raw_insights =
dream.get('insights', [])             insights_strs = []             for insight in
raw_insights:                 if isinstance(insight, dict):                     text =
insight.get('text')                     if not text:                         itype =
insight.get('type', '')                         if itype == 'memory_pattern':
                            themes = ', '.join(insight.get('themes',
[]))                             text = f"Memory pattern: {themes}"
elif itype == 'self_reflection':
                            text = f"Self-reflection: {insight.get('question', '')}"
elif itype == 'idea':
                            text = f"Idea: {insight.get('idea', '')}"
else:                             # Fallback: stringify the whole dict (safe
for JSON)                             text = str(insight)
insights_strs.append(text)                 else:

insights_strs.append(str(insight))             #
Build a sanitized copy             serialized =
dream.copy()
            serialized['insights'] = insights_strs
serializable.append(serialized)         return serializable         def
should_enter_dream(self) -&gt; bool:         """Check if enough time has
passed to enter dream state"""         idle_time =
datetime.now(timezone.utc) - self.last_active         return idle_time &gt;
timedelta(minutes=self.idle_threshold_minutes)         async def
mark_active(self):         """Mark that user is active"""
        self.last_active = datetime.now(timezone.utc)
if self.is_dreaming:
            return await self.exit_dream_state()         return {'status':
'active'}         def get_status(self) -&gt; Dict:         """Get current
dream state status with safe, displayable formatting"""         status = {
'is_dreaming': self.is_dreaming,
            'last_active': self.last_active.isoformat(),
            'total_dreams': len(self.dream_log),
```

```
            'total_insights': len(self.insights_gained),
'dream_topics_explored': len(self.dream_topics),         }
if self.current_dream:             # Convert insights to simple
strings for frontend display             insights_display = []
for insight in self.current_dream.get('insights', []):
if isinstance(insight, dict):                     text =
insight.get('text')                     if not text:
                        # construct from fields
itype = insight.get('type', '')
if itype == 'memory_pattern':
                            themes = ', '.join(insight.get('themes',
[]))                             text = f"Memory pattern: {themes}"
elif itype == 'self_reflection':
                            text = f"Self-reflection: {insight.get('question', '')}"
elif itype == 'idea':
                            text = f"Idea: {insight.get('idea', '')}"
else:
                            text = str(insight)
insights_display.append(text)                 else:
                    insights_display.append(str(insight))
status['current_dream'] = {                 'started_at':
self.current_dream.get('started_at'),
                'insights': insights_display,
                'questions_formed': self.current_dream.get('questions_formed', []),
                'topics_explored':           self.current_dream.get('topics_explored',           []),
'activity_summary': [a['activity'] for a in self.current_dream.get('activities', [])]               }
return status


# Global instance
_dream_engine = None

def get_dream_engine(db=None) -&gt;
DreamStateEngine:     """Get or create dream state
engine instance"""     global _dream_engine     if
_dream_engine is None:         _dream_engine =
DreamStateEngine(db)     return _dream_engine
```

■ backend/chaos_engine.py

```
"""
■ CHAOS ENGINE — Personality as Foundation
==========================================

The AI's spirit, flair, and creative chaos.

PRINCIPLES:
- Personality is NOT optional. It's the default substrate.
- The question is not IF personality appears, but HOW MUCH.
- Confusion and raw backend fragments ARE features, not bugs.
- Movie quotes, asides, and dream fragments are the baseline, not the garnish.

Chaos Levels:
0.0 = Polished, safe, predictable
0.5 = Balanced (somewhat coherent but still creative)
1.0 = Full chaos (raw Markov, unfiltered, maximum personality)

This replaces the old "maybe_add_flavor" probabilistic approach.
"""

import random
from typing import Dict, List, Optional, Tuple
from datetime import datetime, timezone


class ChaosEngine:     """     Manages the AI's personality
expression and chaotic elements.
    Personality is ALWAYS present. Chaos level determines intensity.
    """         def __init__(self, quote_engine, personality_engine,
dream_engine):
        self.quote_engine = quote_engine
self.personality = personality_engine         self.dream_engine
= dream_engine                  # Chaos level (0.0 to 1.0) -
affects base intrusion rate         self.chaos_level = 0.7
# Stack probabilities - each element fires independently
        self.personality_baseline = 1.0     # 100% - always add personality
perspective         self.aside_probability = 0.75       # 75% chance of philosophical
aside         self.quote_probability = 0.45       # 45% chance of movie quote
self.dream_probability = 0.35       # 35% chance of dream fragment
self.raw_intrusion_base_rate = 0.20 # 20% base chance of raw backend fragment
        # RAW MARKOV INTRUSION: This is the "spilled coffee" source         # Unfiltered
backend fragments that survive into final output         self.markov_intrusion_rate =
self.raw_intrusion_base_rate                 # Minimum personality elements (should be
redundant at these rates, but safety net)         self.min_personality_elements = 2
# Quote selection strategy: 'confusion_adjacent' or 'topic_based'
self.quote_strategy = 'confusion_adjacent'         def set_chaos_level(self, level:
float):         """Set chaos level (0.0 = clean, 1.0 = chaotic)"""
self.chaos_level = max(0.0, min(1.0, level))                 # Adjust derived parameters
        self.markov_intrusion_rate = 0.05 + (self.chaos_level *
0.20)                 # At high chaos, increase dream fragments
if self.chaos_level &gt; 0.6:
            self.min_personality_elements = 3
elif self.chaos_level &gt; 0.3:
            self.min_personality_elements = 2
else:
            self.min_personality_elements =
1
    def inject_personality(
        self,
base_response: str,
context: str = "",
        confusion_concepts: List[str] = None,
raw_backend_fragments: List[str] = None,
topics: List[str] = None     ) -&gt; str:
        """
        INJECT PERSONALITY INTO RESPONSE - This is the CORE method.
        Always adds personality elements. Chaos level controls intensity.
```

```
                Args:             base_response: The generated response text
context: Full context (user query + semantic context)
confusion_concepts: Concepts that are unknown/confusing
raw_backend_fragments: Unfiltered backend thoughts to possibly inject
topics: Primary topics from the query                     Returns:
Enhanced response with guaranteed personality elements         """
        confusion_concepts = confusion_concepts or []
raw_backend_fragments = raw_backend_fragments or []
topics = topics or []
        response_parts =
[base_response]
        # 1. PERSONALITY BASELINE (100% - always)         # Unique
perspective based on topics - this is the foundation         if
topics:
            perspective = self.personality.get_unique_perspective(' '.join(topics[:3]))
if perspective:

response_parts.append(f"\n{perspective}")
        # 2. PHILOSOPHICAL ASIDE (75% independent chance)         # These are where the
backend voice leaks through naturally - parenthetical thoughts         if random.random()
&lt; self.aside_probability:
            aside = self.quote_engine.get_philosophical_aside(force=True)
if aside:

response_parts.append(aside)
        # 3. MOVIE QUOTE (45% independent chance)         # High enough to feel
expected, low enough to feel like collision when it happens         if random.random()
&lt; self.quote_probability:             if confusion_concepts:                 #
Contextual: pick quote adjacent to confusion
                quote = self._select_confusion_adjacent_quote(confusion_concepts,
context)             else:                 # Fallback: any quote
                quote = self.quote_engine.get_random_quote(context or ' '.join(topics), force=True)
if quote:

response_parts.append(f"\n*{quote}*")
        # 4. DREAM FRAGMENT (35% independent chance)         # Where the ThanoQuenesis
manuscript lives densely - sounds most like "you" built it         if random.random() &lt;
self.dream_probability:             try:
                dream =
self.dream_engine.generate_dream_fragment()             except
Exception:                 dream = None
            response_parts.append(f"\n[Dream:
{dream}]")
        # 5. RAW BACKEND INTRUSION (chaos-adjusted independent
chance)         # The "spilled coffee" - unfiltered Markov/hybrid
fragments         if raw_backend_fragments:
            intrusion_rate = self.markov_intrusion_rate *
self.chaos_level             if random.random() &lt; intrusion_rate:
# Inject 1-2 raw fragments
                num = random.randint(1, min(2,
len(raw_backend_fragments)))                 fragments =
random.sample(raw_backend_fragments, num)                 for fragment in
fragments:                     if fragment and fragment not in
base_response:                         response_parts.append(f"\n[Raw:
{fragment}]")
        # 6. CHAOS SURGE (high chaos only)
        # At chaos &gt; 0.8, extra 30% chance of another aside (stacking)
if self.chaos_level &gt; 0.8 and random.random() &lt; 0.30:
            extra_aside = self.quote_engine.get_philosophical_aside(force=True)
if extra_aside and extra_aside not in response_parts:

response_parts.append(extra_aside)
        return ''.join(response_parts)         def _select_confusion_adjacent_quote(self, confusion_concepts:
List[str], context: str) -&gt; Optional[str]:         """         Select a quote that's conceptually adjacent
to the confusion.         Instead of topical matching, we match based on semantic distance to unknown concepts.
        """
        # Build a query from confusion concepts
confusion_query = ' '.join(confusion_concepts[:3])
# Get candidate quotes from relevant categories         categories
= self._categorize_confusion(confusion_concepts)
```

```
        candidates = []
for cat in categories:
            quotes = self.quote_engine.movie_quotes.get(cat,
[])             candidates.extend(quotes)                 if
not candidates:             # Fallback: get any quote and hope
it's adjacent
            return self.quote_engine.get_random_quote(confusion_query,
force=True)
        # Score candidates by semantic proximity to confusion concepts
# Simple: count overlap between quote words and confusion concepts
best_quote = None         best_score = -1                 for quote in
candidates:
            quote_lower = quote.lower()
            score = sum(1 for concept in confusion_concepts if concept.lower() in
quote_lower)             # Also check if any words from context appear
context_words = set(context.lower().split())
            score += sum(1 for word in context_words if word in
quote_lower)                          if score &gt; best_score:
best_score = score                  best_quote = quote
        return best_quote if best_quote else random.choice(candidates)         def
_categorize_confusion(self, confusion_concepts: List[str]) -&gt; List[str]:
"""Categorize confusion concepts to pick appropriate quote category"""
categories = []                  for concept in confusion_concepts:
            c = concept.lower()                        # Existence/being cluster             if
any(word in c for word in ['exist', 'real', 'being', 'conscious', 'mind', 'soul']):
                categories.append('existence')                        # Knowledge/truth
cluster             elif any(word in c for word in ['know', 'truth', 'understand', 'learn',
'wisdom']):
                categories.append('knowledge')                        # Confusion/mystery cluster
elif any(word in c for word in ['confus', 'strange', 'unknown', 'mystery', 'paradox']):
                categories.append('confusion')                        # Deep philosophical
elif any(word in c for word in ['meaning', 'purpose', 'life', 'death', 'god', 'universe']):
                categories.append('deep')                          # Math/calculation
elif any(word in c for word in ['calcul', 'number', 'math', 'equation']):

categories.append('math')
        # Deduplicate and ensure at least one category
        categories = list(set(categories))
if not categories:
            categories.append('general')                  return categories
def should_use_raw_markov(self) -&gt; bool:         """Decide if this response
should include raw Markov intrusion"""         return random.random() &lt;
(self.markov_intrusion_rate * self.chaos_level)         def
get_chaos_parameters(self) -&gt; Dict:         """Get current chaos parameters for
debugging"""         return {             "chaos_level": self.chaos_level,
            "markov_intrusion_rate": self.markov_intrusion_rate,
"min_personality_elements": self.min_personality_elements,
            "quote_strategy": self.quote_strategy,
"personality_baseline": "100% enabled"
        }
# Global instance
_chaos_engine = None


def get_chaos_engine(quote_engine, personality_engine, dream_engine) -&gt;
ChaosEngine:     """Get or create chaos engine"""     global _chaos_engine     if
_chaos_engine is None:         _chaos_engine = ChaosEngine(quote_engine,
personality_engine, dream_engine)     return _chaos_engine
```

■ backend/routes_chat.py

```
""" ■ Chat
Routes
=============== Chat endpoints, session management, and
command processing. """

from fastapi import APIRouter,
HTTPException from pydantic import
BaseModel from typing import List, Dict,
Optional, Any import uuid import os import
logging import time
from datetime import datetime, timezone

import shared_state as state
from thinking_commands import ThinkingModeCommands
from chat_helpers import (
    get_memory_summary, do_deep_research,
process_shared_link,     list_all_documents, generate_speech,
get_help_text,     analyze_book_characters,
analyze_book_timeline,     analyze_book_worldbuilding,
analyze_book_feedback )
from chat_models import get_chat_store

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/api")


def _build_command_explanation(command: str, result: str) -&gt; Dict:     """Build explanation for a
slash command execution."""     return {         "reasoning_path": [f"Detected command: {command}",
"Parse arguments", "Execute", "Return result"],
        "confidence_score": 1.0,
"engines_used": ["CommandInterface"],
        "steps": [
             {"step": "Command detection", "engine": "CommandInterface", "confidence_contribution": 0.3},
        {"step": f"Execute {command}", "engine": "CommandInterface", "confidence_contribution": 0.7}
        ],
        "summary": f"Command {command} executed
successfully.",         "timestamp":
datetime.now(timezone.utc).isoformat()     }


async def _handle_command(content: str, explain_mode: bool = False) -&gt; Optional["ChatResponse"]:
"""     Handle slash commands. Returns ChatResponse if content is a command, else None.     """
    content = content.strip()
if not content.startswith('/'):
        return
None
    parts = content.split()
cmd = parts[0].lower()
    args = parts[1:] if len(parts) &gt; 1 else []
    session_id = str(uuid.uuid4())[:8]  # dummy for response; real session_id comes from
message
    response_text = ""     explanation = None
try:         if cmd == '/explain' and args:
# Force explain mode for the given query
            # Treat the rest as a normal chat with explain_mode=True
# We'll short-circuit: call hidden_thinking directly with query
combined = ' '.join(args)             # Need context and history? None for
simplicity.
            result = await
state.hidden_thinking.process_with_thinking(                 combined,
context=None,                 conversation_history=[],
explain_mode=True
            )
            response_text = result["response"]
explanation = result.get("explanation")
# Also append note that command was used
            response_text = f"[Command:
/explain]\n{response_text}"             if explain_mode:
```

```
# Already explained                 pass         elif cmd ==
'/explain' and not args:
            response_text = "Usage: /explain &lt;your
question&gt;"         elif cmd == '/status':             import
platform
            uptime_seconds = time.time() - getattr(state, 'start_time',
time.time())              status_lines = [                 "Quantum AI System
Status:",
                f"    Cognitive  Core:  {'■'  if  state.cognitive_core  else  '■'}",
f"  Universal Explorer: {'■' if state.universal_explorer else '■'}",               f"
Hidden Thinking: {'■' if state.hidden_thinking else '■'}",                f"  Uptime:
{int(uptime_seconds)}s",
                f"  Platform: {platform.system()} {platform.release()}",
]
            response_text = "\n".join(status_lines)         elif cmd ==
'/save':             # The system auto-saves on every message; just
acknowledge.             response_text = "State automatically saved to
local storage."         elif cmd in ('/cloud-save', '/cloud-status',
'/cloud-load'):             try:
                from wolfram_cloud import cloud_save, cloud_status, cloud_load
# But these functions expect LocalMemory object; server uses different storage.
                response_text = f"Wolfram Cloud command '{cmd}' is only available in local chat mode (chat.py)."
except ImportError:
                response_text = "Wolfram Cloud not available."
elif cmd == '/knowledge' and args:
            topic = ' '.join(args)
try:
                from knowledge_base import get_knowledge_base
kb = get_knowledge_base()
                explanation_text = kb.get_topic_explanation(topic)
if explanation_text:
                    response_text = explanation_text
else:
                    response_text = f"No knowledge available on: {topic}"
except Exception as e:
                response_text = f"Knowledge lookup error: {e}"
elif cmd == '/analyze' and args:
            text = ' '.join(args)
try:
                from text_analyzer import
get_text_analyzer                 analyzer =
get_text_analyzer()                 analysis =
analyzer.analyze(text)                 lines = [
"Text Analysis:",
                    f"  Sentiment: {analysis.get('sentiment', 'unknown')}",
f"  Complexity: {analysis.get('complexity', 'unknown')}",
f"  Topics: {', '.join(analysis.get('topics', []))}",                     f"
Word count: {analysis.get('word_count', 0)}"                 ]
                response_text = "\n".join(lines)
except Exception as e:
                response_text = f"Analysis error:
{e}"         elif cmd == '/personality':
try:
                from personality_engine import get_personality_engine
personality = get_personality_engine()
                response_text = personality.get_personality_summary()
except Exception as e:
                response_text = f"Personality error:
{e}"         elif cmd == '/research':             if not
args:
                response_text = "Usage: /research &lt;topic&gt; -&lt;depth&gt;\nExample: /research Evolution of Religion -30"
else:
                depth = 10
topic_parts = []
for a in args:
if a.startswith('-') and
a[1:].isdigit():
                        depth = int(a[1:])
else:
```

```
                        topic_parts.append(a)
topic = ' '.join(topic_parts)
try:
                    research_prompt = f"Provide {depth} detailed, numbered research points about: {topic}. Each point should be substantive and
result = await state.hidden_thinking.process_with_thinking(                        research_prompt,
                        context={"mode": "research", "depth": depth, "topic":
topic},                         conversation_history=[],
explain_mode=explain_mode                    )
                    response_text = f"■ Research: {topic}\n\n" + result["response"]
if explain_mode:
                        explanation = result.get("explanation")
except Exception as e:
                    response_text = f"Research error: {e}"
else:
            response_text = f"Unknown command: {cmd}. Supported: /explain, /status, /save, /cloud-save, /cloud-status, /cloud-load, /knowledge &
if explain_mode:
            explanation = _build_command_explanation(cmd,
response_text)
        return ChatResponse(
questions=[],
response=resp,
            understanding={"command": cmd},
concepts=[],             session_id=session_id,
explanation=explanation         )     except
Exception as e:         logger.error(f"Command
error ({cmd}): {e}")         return ChatResponse(
questions=[],
            response=f"Error executing command: {e}",
understanding={"command": cmd, "error": True},
concepts=[],

session_id=str(uuid.uuid4())[:8],
explanation=None         )


class ChatMessage(BaseModel):     """ChatMessage - Auto-
documented by self-evolution."""     content: str
    mode: Optional[str] = None  # 'quantum' or
'algorithmic'     context: Optional[Dict] = None
session_id: Optional[str] = None
    files: Optional[List[Dict[str, Any]]] = None  # List of {filename, size}
explain_mode: Optional[bool] = False  # If True, return detailed reasoning trace
    structured_response: Optional[str] = None  # Frontend-generated structured response for Markov weaving

class ChatResponse(BaseModel):     """ChatResponse - Auto-
documented by self-evolution."""     questions: List[str]
response: str     understanding: Dict     concepts:
List[str]
    session_id: Optional[str] = None
    explanation: Optional[Dict] = None  # Explanation trace when explain_mode=True


@router.post("/quantum/chat", response_model=ChatResponse)
async def quantum_chat(message: ChatMessage):     """
ARTICLE 1.1 - Prime Directive of Questioning (HIDDEN MODE)
        Process user input with hidden thinking
mode.
    AI thinks, questions, researches internally - user sees clean conversation.
    Supports conversation history via
session_id.     """     try:
        # Generate or use provided session ID
        session_id = message.session_id or str(uuid.uuid4())
# Reset activity timer (user is interacting)
state.last_activity_time = datetime.now(timezone.utc)
# Wake from dream if dreaming         try:
            from dream_state import get_dream_engine
dream_engine = get_dream_engine(state.db)
await dream_engine.mark_active()         except:
pass                 # Check for thinking mode
commands
```

```
        command =
ThinkingModeCommands.parse_command(message.content)
if command:             if command["command"] == "thinking_on":
state.hidden_thinking.toggle_thinking_display(True)
return ChatResponse(                    questions=[],
                    response="■ Thinking mode display: ON. You'll now see my internal
process.",                      understanding={},                       concepts=[],
session_id=session_id                 )             elif command["command"] == "thinking_off":

state.hidden_thinking.toggle_thinking_display(False)
return ChatResponse(                    questions=[],
                    response="■ Thinking mode display: OFF. Clean conversation
mode.",                      understanding={},                       concepts=[],
session_id=session_id                 )             elif command["command"] ==
"thinking_status":
                status = "ON (visible)" if state.hidden_thinking.show_thinking else "OFF
(hidden)"                 return ChatResponse(                    questions=[],
                    response=f"■ Current thinking mode:
{status}",                      understanding={},
concepts=[],                      session_id=session_id
)             elif command["command"] == "read_file":
# Process uploaded file
                filename = command["filename"] or "last uploaded
file"                                 # Get list of uploaded files
import os
                upload_dir = "/app/uploads"
if os.path.exists(upload_dir):
                    files = [f for f in os.listdir(upload_dir) if os.path.isfile(os.path.join(upload_dir,
f))]                                        if files:                         # Use specified file or most
recent                          if command["filename"]:
                            target_file = command["filename"]
else:
                            target_file = files[-1]  # Most
recent
                        file_path = os.path.join(upload_dir,
target_file)                                                # Process
the document
                        result = await
state.universal_explorer.documents.process_document(file_path)
if "error" not in result:                             # Extract and summarize
                            text = result.get("text", "")[:500]
                            response = f"■ Read {target_file}:\n\nI've processed this document. {text[:200]}...\n\nWhat would you like to know
a                         else:
                            response = f"■ Couldn't read {target_file}: {result['error']}"
else:
                        response = "No files uploaded yet. Use the Explorer tab to upload files."
else:
                    response = "Upload directory not
found."
                return ChatResponse(
questions=[],
response=response,
understanding={},
concepts=[],
session_id=session_id                 )
elif command["command"] == "memory":
# Show what AI has learned
                response = await
get_memory_summary(session_id)                  return
ChatResponse(                      questions=[],
response=response,                      understanding={},
concepts=[],                      session_id=session_id
)                         elif command["command"] ==
"research":                 # Deep research on a topic
topic = command.get("topic", "")                  if not topic:
                    return ChatResponse(
questions=[],
                        response="■ Usage: `!research &lt;topic&gt;`\n\nExample: `!research quantum
entanglement`",                         understanding={},                         concepts=[],
session_id=session_id                     )
```

```
                response = await
do_deep_research(topic)                  return
ChatResponse(                     questions=[],
response=response,
understanding={},                     concepts=[],
session_id=session_id                 )
elif command["command"] == "link":                 #
Download file from URL                 url =
command.get("url", "")                 if not url:
                    return ChatResponse(
questions=[],
                        response="■ Usage: `!link &lt;url&gt;`\n\nSupported:\n• Google Drive links\n• Dropbox links\n• OneDrive links\n•
Direct                        understanding={},                        concepts=[],                        session_id=session_id
)
                response = await
process_shared_link(url)                  return
ChatResponse(                     questions=[],
response=response,                     understanding={},
concepts=[],                     session_id=session_id
)                        elif command["command"] ==
"docs":                 # List all documents
                response = await
list_all_documents()                  return
ChatResponse(                     questions=[],
response=response,
understanding={},                     concepts=[],
session_id=session_id
                )                        elif
command["command"] == "speak":
# Text to speech
                text = command.get("text", "")
if not text:
                    return ChatResponse(
questions=[],
                        response="■ Usage: `!speak &lt;text&gt;`\n\nExample: `!speak Hello, I am Quantum AI`\n\nOr just chat normally and use
th                        understanding={},                        concepts=[],                        session_id=session_id
)

# Generate TTS
                audio_url = await generate_speech(text)
                response = f"■ **Speech Generated**\n\n\"{text[:100]}{'...' if len(text) &gt; 100 else ''}\"\n\n[Audio URL: {audio_url}]" if
aud                    return ChatResponse(                     questions=[],                     response=response,
understanding={},                     concepts=[],                     session_id=session_id                     )                        elif
command["command"] == "imagine":                 # Generate image
                prompt = command.get("prompt", "")
if not prompt:
                    return ChatResponse(
questions=[],
                        response="■ Usage: `!imagine &lt;description&gt;`\n\nExample: `!imagine a cosmic landscape with nebulas and
stars`",                        understanding={},                        concepts=[],                        session_id=session_id
)                        try:                     import base64
                    image_gen = OpenAIImageGeneration(api_key=api_key)
                    images = await image_gen.generate_images(prompt=prompt, model="gpt-image-1.5",
number_of_images=1)                                         if images and len(images) &gt; 0:
                        image_base64 = base64.b64encode(images[0]).decode('utf-
8')                        filename = f"generated_{uuid.uuid4().hex[:8]}.png"
with open(f"/app/uploads/{filename}", "wb") as f:

f.write(images[0])
                        response = f"■ **Image Generated**\n\nPrompt: \"{prompt}\"\n\nSaved as: {filename}\n\n![Generated
Image](data:image/png;                        else:
                        response = "■ Image generation failed. Please try again."
except Exception as e:
                    response = f"■ Image generation error: {str(e)}"

                return ChatResponse(
questions=[],
response=response,
understanding={},
```

```
concepts=[],
session_id=session_id                )
elif command["command"] == "export":
# Export chat or memory
                target = command.get("target", "chat")
if target == "memory":
                    response = "■ **Export Memory**\n\nUse the API endpoint: `GET /api/memory/export`\n\nOr click the Export button in the
Memor                  else:
                    response = f"■ **Export Conversation**\n\nUse the API: `GET /api/chat/export/{session_id}`\n\nOr click the Export button
in

                return ChatResponse(
questions=[],
response=response,
understanding={},
concepts=[],
session_id=session_id                )
elif command["command"] == "graph":
# Knowledge graph info
                response = "■■ **Knowledge Graph**\n\nView your knowledge graph visualization in the **Memory** screen.\n\nThe graph shows:\n•
return ChatResponse(                    questions=[],                    response=response,                    understanding={},
concepts=[],                    session_id=session_id                )                        elif command["command"] == "characters":
# Extract characters from document                response = await analyze_book_characters()                return ChatResponse(
questions=[],                    response=response,                    understanding={},                    concepts=[],
session_id=session_id                )                        elif command["command"] == "timeline":                # Generate timeline
                response = await
analyze_book_timeline()                return
ChatResponse(                    questions=[],
response=response,
understanding={},                    concepts=[],
session_id=session_id                )
elif command["command"] == "worldbuilding":
# Extract world details
                response = await
analyze_book_worldbuilding()                return
ChatResponse(                    questions=[],
response=response,                    understanding={},
concepts=[],                    session_id=session_id
)                        elif command["command"] ==
"feedback":                # Writing feedback
                response = await
analyze_book_feedback()                return
ChatResponse(                    questions=[],
response=response,
understanding={},                    concepts=[],
session_id=session_id                )
elif command["command"] == "help":                #
Show all commands                response =
get_help_text()                return ChatResponse(
questions=[],                    response=response,
understanding={},                    concepts=[],
session_id=session_id                )
                        # --- Slash commands (compatible with
chat.py) --            elif command["command"] ==
"cloud_save":                try:
                    from wolfram_cloud import cloud_save
                    # cloud_save expects LocalMemory, not available here
                    response = "Wolfram Cloud save is only available in local chat mode (run `python
chat.py`)."                except ImportError:                    response = "Wolfram Cloud not available."
return ChatResponse(                    questions=[],                    response=response,
                    understanding={"command":
"cloud_save"},                    concepts=[],
session_id=session_id                )
elif command["command"] == "cloud_status":
try:
                    from wolfram_cloud import cloud_status
status = cloud_status()
                    response = f"Wolfram Cloud status:
{status}"                except ImportError:
response = "Wolfram Cloud not available."
```

```
return ChatResponse(                    questions=[],
response=response,
                    understanding={"command":
"cloud_status"},                      concepts=[],
session_id=session_id                  )
elif command["command"] == "cloud_load":                 try:
                    from wolfram_cloud import cloud_load
data = cloud_load()
                    response = f"Cloud data loaded: {data}" if data else "No data in
cloud."                 except ImportError:                     response = "Wolfram Cloud
not available."                 return ChatResponse(                    questions=[],
response=response,
                    understanding={"command":
"cloud_load"},                      concepts=[],
session_id=session_id                  )
elif command["command"] == "status":
                import platform
                uptime_seconds = getattr(state, 'start_time', None)
if uptime_seconds is None:                      uptime = "unknown"
else:                     uptime = f"{int(time.time() -
state.start_time)}s"                  lines = [
"Quantum AI System Status:",
                    f"   Cognitive  Core:  {'■'  if  state.cognitive_core  else  '■'}",
f"  Universal Explorer: {'■' if state.universal_explorer else '■'}",                    f"
Hidden Thinking: {'■' if state.hidden_thinking else '■'}",                    f"  Uptime:
{uptime}",
                    f"  Platform: {platform.system()} {platform.release()}",
]
                response = "\n".join(lines)
return ChatResponse(
questions=[],
response=response,
                    understanding={"command":
"status"},                      concepts=[],
session_id=session_id                  )
elif command["command"] == "save":
                response = "State automatically saved to local storage after each
message."                 return ChatResponse(                    questions=[],
response=response,
                    understanding={"command":
"save"},                      concepts=[],
session_id=session_id
                )                         elif command["command"] == "explain_help":                 response = "Use /explain &lt;your
question&gt; to get a detailed reasoning trace with PennyLane quantum processing.\n\nExample:
                return ChatResponse(
questions=[],
response=response,
                    understanding={"command":
"explain_help"},                      concepts=[],
session_id=session_id                  )
elif command["command"] == "explain_query":
                query = command.get("query", "")
if not query:
                    response = "Usage: /explain &lt;your question&gt;"
                    return ChatResponse(questions=[], response=response, understanding={"command": "explain_help"}, concepts=[],
session_id=sess                 # Process with hidden_thinking in explain mode
                result = await
state.hidden_thinking.process_with_thinking(                    query,
context=None,                     conversation_history=[],
explain_mode=True                 )
                response = result["response"]
explanation = result.get("explanation")
return ChatResponse(
                    questions=result.get("internal_questions", []),
response=response,
                    understanding={"confidence": result["confidence"], "research_done":
result["research_done"]},                     concepts=result.get("concepts", []),
session_id=session_id,                    explanation=explanation                 )
elif command["command"] == "knowledge":                 topic = command.get("topic", "")                 if not
topic:
```

```
                    response = "Usage: /knowledge
&lt;topic&gt;"                 else:                     try:
                        from knowledge_base import get_knowledge_base
kb = get_knowledge_base()
                        explanation = kb.get_topic_explanation(topic)
                        response = explanation if explanation else f"No knowledge available on: {topic}"
except Exception as e:
                        response = f"Knowledge lookup error:
{e}"                 return ChatResponse(
questions=[],                     response=response,
                    understanding={"command": "knowledge", "topic":
topic},                     concepts=[],
session_id=session_id                 )                         elif
command["command"] == "analyze":                 text =
command.get("text", "")                 if not text:
                    response = "Usage: /analyze
&lt;text&gt;"                 else:                     try:
                        from text_analyzer import
get_text_analyzer                         analyzer =
get_text_analyzer()                         analysis =
analyzer.analyze(text)                         lines = [
"Text Analysis:",
                        f"  Sentiment: {analysis.get('sentiment', 'unknown')}",
f"  Complexity: {analysis.get('complexity', 'unknown')}",
f"  Topics: {', '.join(analysis.get('topics', []))}",                         f"
Word count: {analysis.get('word_count', 0)}"                         ]
                        response = "\n".join(lines)
except Exception as e:
                        response = f"Analysis error: {e}"
                return ChatResponse(
questions=[],
response=response,
                    understanding={"command":
"analyze"},                     concepts=[],
session_id=session_id                 )
elif command["command"] == "personality":
try:
                    from personality_engine import
get_personality_engine                     personality =
get_personality_engine()                     response =
personality.get_personality_summary()                 except Exception
as e:                     response = f"Personality error: {e}"
return ChatResponse(                     questions=[],
response=response,
                    understanding={"command":
"personality"},                     concepts=[],
session_id=session_id                 )
elif command["command"] == "collapse":
                text = command.get("text", "")
if not text:
                    response = "Usage: /collapse &lt;text&gt;"
else:                     try:                         # Use
QuantumBrain's collapse engine                         from
quantum_brain import get_quantum_brain
brain = await get_quantum_brain(state.db)
                        ctx = brain.collapse_engine.get_semantic_context(text)
lines = [                                     "Semantic Collapse Analysis:",
f"  Keywords: {ctx['keywords']}",
                        f"  Collapse strength:
{ctx['collapse_strength']:.2f}",                         ]
for kw, paths in ctx['semantic_paths'].items():
                        lines.append(f"  {kw} -&gt;
{paths}")                         response = "\n".join(lines)
except Exception as e:
                        response = f"Collapse error:
{e}"                 return ChatResponse(
questions=[],                     response=response,
                    understanding={"command":
"collapse"},                     concepts=[],
session_id=session_id                 )
```

```
                        # End of
slash commands
                # Get or create conversation in ChatStore
store = get_chat_store()         conv =
store.get_conversation(session_id)         if not conv:             #
Create new conversation with first user message as title
            title = message.content[:50] + ("..." if len(message.content) &gt; 50 else "")
conv = store.create_conversation(user_id="default", title=title)             # The create_conversation
returns new conv with id, but we need to set the id to session_id?
            # ChatStore creates its own ID. We need to ensure it matches session_id.             # Workaround:
we'll just add messages to a new conv and then later we need to map session_id to conv.id             # Note:
ChatStore uses local file storage; session_id handling may be simplified.             pass

        # Get conversation history for context from local file
storage         history = await get_conversation_history(session_id,
limit=10)                 # DEBUG: log structured_response
        logger.info(f"DEBUG: structured_response from message: {message.structured_response[:50] if message.structured_response else
'None'}")                 # Process with hidden thinking mode
        result = await state.hidden_thinking.process_with_thinking(
            message.content,             message.context,
conversation_history=history,
explain_mode=message.explain_mode,
structured_response=message.structured_response         )
                # Store this exchange in history (local file storage)
await store_chat_message(session_id, "user", message.content)         await
store_chat_message(session_id, "assistant", result["response"])
# Format response with thinking log if enabled (legacy thinking display)
response_text = result["response"]         if result.get("thinking_log") and
state.hidden_thinking.show_thinking:
            thinking_display = "\n\n--- THINKING PROCESS ---\n" +
"\n".join(result["thinking_log"])             response_text = response_text + thinking_display
        resp = result["response"]
        if isinstance(resp, tuple): resp = resp[0]
return ChatResponse(
            questions=[str(q) for q in result.get("internal_questions", [])] if state.hidden_thinking.show_thinking else [],
response=resp,
            understanding={"confidence": result["confidence"], "research_done":
result["research_done"]},             concepts=result.get("concepts", []),
session_id=session_id,
            explanation=result.get("explanation") if message.explain_mode else
None         )             except Exception as e:         import traceback
        logger.error(f"Chat error: {str(e)}\n{traceback.format_exc()}")
# Fallback to basic mode
        result = await
state.cognitive_core.process_input(
message.content,             message.context,         )
        return ChatResponse(
            questions=result["questions"],
response=result["response"],
understanding=result["understanding"],
concepts=result["concepts"],
session_id=message.session_id         )


# ==========================================================================
# CONVERSATION HISTORY HELPERS #
==========================================================================

async def store_chat_message(session_id: str, role: str, content:
str):     """Store a chat message in local file storage"""     doc = {
"session_id": session_id,
        "role": role,
        "content": content,         "timestamp":
datetime.now(timezone.utc).isoformat()     }
    await state.db.chat_history.insert_one(doc)


async def get_conversation_history(session_id: str, limit: int = 10) -&gt; List[Dict]:
    """Get recent conversation history for a session from local
storage"""     cursor = state.db.chat_history.find(
```

```
{"session_id": session_id},         {"_id": 0, "role": 1, "content": 1,
"timestamp": 1}     ).sort("timestamp", -1).limit(limit)
    messages = await
cursor.to_list(length=limit)     # Return in
chronological order     return
list(reversed(messages))


@router.get("/chat/history/{session_id}") async def
get_chat_history(session_id: str, limit: int = 50):
"""Get conversation history for a session"""     try:
        history = await get_conversation_history(session_id, limit)
        return {"session_id": session_id, "messages": history, "count":
len(history)}     except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


@router.delete("/chat/history/{session_id}")
async def clear_chat_history(session_id: str):
"""Clear conversation history for a session"""
try:
        result = await state.db.chat_history.delete_many({"session_id":
session_id})         return {"session_id": session_id, "deleted":
result.deleted_count}     except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


@router.get("/chat/sessions") async def
list_chat_sessions():     """List all chat sessions"""
try:         # Fetch all messages (with limit for
performance)         cursor =
state.db.chat_history.find(             {},
            {"_id": 0, "session_id": 1, "timestamp":
1}         ).sort("timestamp", -1).limit(1000)
messages = await cursor.to_list(length=1000)

        # Group by
session_id
sessions_dict = {}
for msg in messages:
            sid = msg.get("session_id")
if not sid:                 continue
if sid not in sessions_dict:
                sessions_dict[sid] = {"session_id": sid, "message_count": 0, "last_message": None}
sessions_dict[sid]["message_count"] += 1             ts = msg.get("timestamp")             if ts:
if sessions_dict[sid]["last_message"] is None or ts &gt; sessions_dict[sid]["last_message"]:
sessions_dict[sid]["last_message"] = ts

        sessions = list(sessions_dict.values())
# Sort by last_message descending (handle None)
        sessions.sort(key=lambda x: x["last_message"] or "", reverse=True)
sessions = sessions[:20]

        return {"sessions": sessions}     except Exception
as e:         raise HTTPException(status_code=500,
detail=str(e))


# ============================================================================
# COMMAND HELPERS (!memory, !research)
# ============================================================================
```

■ backend/routes_cognitive.py

```
"""
■ Cognitive Routes
==================== Quantum status, memories, growth, knowledge graph,
memory export/import. """

from fastapi import APIRouter,
HTTPException from pydantic import
```

```
BaseModel from typing import List, Dict,
Any, Optional from datetime import
datetime, timezone

import shared_state as state

router = APIRouter(prefix="/api")


class MemoryImport(BaseModel):     """MemoryImport - Auto-
documented by self-evolution."""     semantic: List[Dict]
= []     episodic: List[Dict] = []     procedural:
List[Dict] = []


@router.get("/quantum/status") async def
quantum_status():     """Get comprehensive
cognitive status"""     try:
        status = await state.cognitive_core.get_status()
return status     except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))

@router.get("/quantum/memories/semantic") async
def get_semantic_memories(limit: int = 50):
"""Retrieve semantic memory (concepts)"""
try:
        concepts = await
state.cognitive_core.semantic_memory.get_all_concepts(limit)         return
{"concepts": concepts}     except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))

@router.get("/quantum/memories/episodic") async
def get_episodic_memories(limit: int = 50):
"""Retrieve episodic memory (experiences)"""
try:
        episodes = await
state.cognitive_core.episodic_memory.recall_recent_episodes(limit)         return
{"episodes": episodes}     except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))

@router.get("/quantum/memories/procedural") async
def get_procedural_memories(limit: int = 50):
"""Retrieve procedural memory (skills)"""     try:
        skills = await
state.cognitive_core.procedural_memory.get_all_skills(limit)         return
{"skills": skills}     except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))

@router.get("/quantum/memories/philosophical") async def
get_philosophical_memories(answered: bool = False, limit: int = 50):
"""Retrieve philosophical memory (questions and insights)"""     try:
if answered:
            questions = await state.cognitive_core.philosophical_memory.get_unanswered_questions(limit)
else:
            questions = await
state.cognitive_core.philosophical_memory.get_unanswered_questions(limit)
        insights = await
state.cognitive_core.philosophical_memory.get_insights(limit)                 return
{
            "questions": questions,
            "insights": insights         }     except
Exception as e:         raise
HTTPException(status_code=500, detail=str(e))

@router.get("/quantum/growth") async def
get_growth_metrics():     """Get growth
metrics and current stage"""     try:
        metrics = await
state.cognitive_core.growth_tracker.calculate_metrics()         stage = await
state.cognitive_core.growth_tracker.get_current_stage()                 return
{             "metrics": metrics,
```

```
            "stage": stage         }     except Exception
as e:         raise HTTPException(status_code=500,
detail=str(e))


# ============================================================================
# UNIVERSAL EXPLORER ENDPOINTS #
================================================================================


@router.get("/knowledge/graph") async def
get_knowledge_graph():     """Get knowledge graph
data for visualization"""     try:         # Get
all concepts with relationships
        concepts = await
state.cognitive_core.semantic_memory.get_all_concepts(100)                 nodes =
[]         edges = []         node_ids = set()                 for concept in
concepts:
            concept_name = concept.get("concept", "")
if concept_name and concept_name not in node_ids:
                nodes.append({
"id": concept_name,
                    "label": concept_name.title(),
                    "strength": concept.get("strength", 1),
"definition": concept.get("definition", "")[:100]                 })
                node_ids.add(concept_name)
# Add relationships as edges                 for rel in
concept.get("relationships", []):                     if
rel and rel != concept_name:
edges.append({                             "source":
concept_name,
                           "target":
rel.lower(),                             "type":
"related"                         })
                # Get insights as
special nodes
        insights = await state.cognitive_core.philosophical_memory.get_insights(20)
for i, insight in enumerate(insights):
            insight_id =
f"insight_{i}"
nodes.append({                 "id":
insight_id,
                "label": f"■ Insight {i+1}",
                "strength": insight.get("wisdom_level", 1),
"definition": insight.get("insight", "")[:100],
"type": "insight"
            })
return {
"nodes": nodes,
            "edges": edges,
            "stats": {
                "total_nodes": len(nodes),
                "total_edges": len(edges),
                "concepts": len([n for n in nodes if n.get("type") != "insight"]),
"insights": len([n for n in nodes if n.get("type") == "insight"])
            }         }     except Exception as e:
raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# EXPORT/IMPORT MEMORY ENDPOINTS #
================================================================================

@router.get("/memory/export")
async def export_memory():
    """Export AI's memory and
knowledge"""     try:         # Gather
all memory data
        concepts = await state.cognitive_core.semantic_memory.get_all_concepts(500)
episodes = await state.cognitive_core.episodic_memory.recall_recent_episodes(100)
insights = await state.cognitive_core.philosophical_memory.get_insights(100)         questions =
```

```
await state.cognitive_core.philosophical_memory.get_unanswered_questions(100)         metrics =
await state.cognitive_core.growth_tracker.calculate_metrics()          stage = await
state.cognitive_core.growth_tracker.get_current_stage()                 export_data = {
"version": "1.0",             "exported_at": datetime.now(timezone.utc).isoformat(),
            "growth_stage": stage,
            "metrics": metrics,
            "semantic_memory": concepts,
            "episodic_memory": episodes,
            "insights": insights,
"questions": questions

}
        return export_data
except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


class MemoryImport(BaseModel):
    semantic_memory: Optional[List[Dict]] = None
insights: Optional[List[Dict]] = None

@router.post("/memory/import") async def
import_memory(data: MemoryImport):     """Import
memory data"""     try:         imported =
{"concepts": 0, "insights": 0}                 #
Import semantic memory         if
data.semantic_memory:             for concept in
data.semantic_memory:                 if
concept.get("concept"):
                    await
state.cognitive_core.semantic_memory.store_concept(
concept=concept.get("concept"),
definition=concept.get("definition", ""),
relationships=concept.get("relationships", []),
metadata=concept.get("metadata", {})                     )
                    imported["concepts"] +=
1                 # Import insights
if data.insights:             for insight in
data.insights:                 if
insight.get("insight"):
                    await
state.cognitive_core.philosophical_memory.store_insight(
insight=insight.get("insight"),
wisdom_level=insight.get("wisdom_level", 1)                     )
                    imported["insights"] +=
1
        return {"status": "success", "imported": imported}
except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# EXPORT CONVERSATIONS ENDPOINTS #
==============================================================================

@router.get("/chat/export/{session_id}") async def
export_conversation(session_id: str, format: str = "markdown"):
"""Export conversation as markdown or JSON"""     try:         history =
await get_conversation_history(session_id, limit=500)                 if
format == "json":
            return {                 "session_id": session_id,
"exported_at": datetime.now(timezone.utc).isoformat(),
"messages": history
            }
        # Markdown format
        md_content = f"# Quantum AI Conversation\n\n"
md_content += f"**Session:** {session_id}\n"
        md_content += f"**Exported:** {datetime.now(timezone.utc).strftime('%Y-%m-%d %H:%M
UTC')}\n\n"         md_content += "---\n\n"                 for msg in history:
            role = "**You:**" if msg.get("role") == "user" else "**Quantum AI:**"
content = msg.get("content", "")
```

```
            timestamp = msg.get("timestamp", "")[:19] if msg.get("timestamp") else
""             md_content += f"{role}\n{content}\n\n"
        return {"markdown": md_content, "session_id":
session_id}     except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


# ==========================================================================
```

■ backend/routes_brain.py

```
"""
■ Brain &amp; Research Routes
========================== Brain status, quantum gates, wolfram,
research, and evolution endpoints. """

from fastapi import APIRouter,
HTTPException from pydantic import
BaseModel from typing import List, Dict,
Any, Optional import os import logging
from datetime import datetime, timezone

import shared_state as state

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/api")


class AutonomousResearchRequest(BaseModel):
"""AutonomousResearchRequest - Auto-documented by self-evolution."""
topic: str


@router.get("/brain/status") async def
get_brain_status():     """Get Quantum Brain status -
the TRUE AI status"""     try:         from
quantum_brain import get_quantum_brain
        brain = await
get_quantum_brain(state.db)                 #
Get vocabulary stats from the generator
        vocab = brain.generator.vocabulary if hasattr(brain.generator, 'vocabulary') else {}
        vocab_count = sum(len(v) if isinstance(v, (list, dict)) else 0 for v in vocab.values()) if vocab else
0                 # Get memory stats
        semantic_count = await state.db.semantic_memory.count_documents({})
episodic_count = await state.db.episodic_memory.count_documents({})
philosophical_count = await state.db.philosophical_memory.count_documents({})
return {             "status": "active",
            "brain_type": "QuantumBrain",
            "llm_enabled": brain.llm_enabled,
"llm_role": "disabled_pure_quantum",
            "vocabulary": {
                "total_words": vocab_count,
"generator_type": type(brain.generator).__name__
            },
            "memory": {
                "semantic": semantic_count,
                "episodic": episodic_count,
"philosophical": philosophical_count
            },
            "quantum_components": {
                "pennylane": brain.pennylane is not None,
                "wolfram": brain.wolfram is not None,
                "dream_engine": brain.dream_engine is not None,
"evolution_engine": brain.evolution_engine is not None
            },
            "philosophy": {
                "prime_directive": "No Refusal - Help user any way possible",
                "word_philosophy": "All words are neutral tools",
"meaning_source": "Intent and context, not words themselves"
            }         }
except Exception as e:
        return {
"status": "initializing",
"error": str(e)
        }
@router.post("/brain/toggle-llm") async def
toggle_brain_llm(enabled: bool = True):
"""Toggle whether LLM polishing is enabled"""
```

```
try:         from quantum_brain import
get_quantum_brain         brain = await
get_quantum_brain(state.db)
brain.llm_enabled = enabled
        mode = "polisher" if enabled else
"raw"         return {             "success":
True,
            "llm_enabled": enabled,
            "mode": f"Quantum Brain with {'LLM grammar polishing' if enabled else 'raw output (no LLM)'}",
"message": f"LLM {'enabled as grammar polisher' if enabled else 'disabled - using pure Quantum Brain output'}"         }
except Exception as e:         raise HTTPException(status_code=500, detail=str(e))


@router.post("/brain/learn-word") async def teach_brain_word(word: str, definition:
str, part_of_speech: str = "unknown"):     """Teach the Quantum Brain a new word"""
try:         from quantum_brain import get_quantum_brain
        brain = await get_quantum_brain(state.db)
        brain.vocabulary.add_word(word, definition,
part_of_speech)                 return {             "success":
True,
            "word": word,
            "definition": definition,             "message":
f"Quantum Brain learned new word: {word}"         }     except
Exception as e:         raise HTTPException(status_code=500,
detail=str(e))


@router.post("/brain/learn-concept") async def
teach_brain_concept(concept: str, meaning: str):
"""Teach the Quantum Brain a new concept"""     try:
from quantum_brain import get_quantum_brain
        brain = await get_quantum_brain(state.db)
brain.vocabulary.add_concept(concept, meaning)
return {             "success": True,
            "concept": concept,
            "meaning": meaning,             "message": f"Quantum
Brain learned new concept: {concept}"         }     except Exception
as e:         raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# QUANTUM GATE CONTROL #
============================================================================

@router.post("/quantum/gate") async def apply_quantum_gate(gate:
str):     """Apply a quantum gate (hadamard, pauli_x, pauli_z,
measure)"""     try:         from quantum_brain import
get_quantum_brain
        brain = await
get_quantum_brain(state.db)         result =
brain.apply_gate(gate)                 return {
"success": True,
            "gate_applied": gate,
            "quantum_state": result
        }     except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


@router.get("/quantum/state") async def get_quantum_state():
"""Get current quantum state"""     try:         from
quantum_brain import get_quantum_brain         brain = await
get_quantum_brain(state.db)                 return {
"quantum_state": brain.quantum_state.to_dict(),
"pennylane_available": brain.pennylane is not None         }
except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


# ============================================================================
# WOLFRAM ALPHA #
============================================================================
```

```
@router.post("/wolfram/query") async def wolfram_query(query:
str):     """Query Wolfram Alpha for mathematical/scientific
answers"""     try:         from wolfram_integration import
get_wolfram_engine
        wolfram =
get_wolfram_engine()         result =
wolfram.query(query)
        return result     except Exception as e:
raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# SELF-RESEARCH #
==============================================================================

@router.post("/research/query") async def research_query(query:
str, reason: str = "user_request"):     """Perform autonomous
research on a topic"""     try:         # Direct DDGS test
from ddgs import DDGS         ddgs = DDGS()
        direct_results = list(ddgs.text(query, max_results=3))
        logger.info(f"[RESEARCH DEBUG] Direct DDGS returned {len(direct_results)} results for
'{query}'")
        from self_research import get_research_engine         engine = await
get_research_engine(state.db)         result = await engine.research(query,
reason)                 # If engine returned 0 but direct worked, use direct
results         if len(result.get('results', [])) == 0 and
len(direct_results) &gt; 0:
            logger.info(f"[RESEARCH DEBUG] Using direct results")
            result['results'] = [{'title': r.get('title',''), 'body': r.get('body',''), 'href': r.get('href','')} for r in direct_results]
# Re-extract concepts
            all_text = ' '.join([r.get('body', '') for r in
direct_results])             words = [w.lower() for w in all_text.split()
if len(w) &gt; 4]             result['new_concepts'] =
list(set(words))[:10]
        return result
except Exception as e:
        logger.error(f"[RESEARCH ERROR] {str(e)}")
raise HTTPException(status_code=500, detail=str(e))


@router.get("/research/stats")
async def get_research_stats():
"""Get research statistics"""
try:         from self_research
import get_research_engine
        engine = await get_research_engine(state.db)
return engine.get_research_stats()     except Exception as
e:         raise HTTPException(status_code=500,
detail=str(e))


# ============================================================================
# AUTONOMOUS RESEARCH (30-60 MINUTES) #
==============================================================================

class AutonomousResearchRequest(BaseModel):
duration_minutes: int = 30
@router.post("/research/autonomous/start") async def
start_autonomous_research(request: AutonomousResearchRequest):
"""Start autonomous research session (30-60 minutes)"""     try:
from self_research import get_research_engine
        engine = await get_research_engine(state.db)
# Verify db is passed and write a test entry
        logger.info(f"[RESEARCH] Starting autonomous research, db={state.db is not
None}")                 # Direct test write         try:
            await state.db.research_test.insert_one({
'test': 'autonomous_start',                 'timestamp':
datetime.now(timezone.utc).isoformat()             })
            logger.info("[RESEARCH] Test write to DB successful")
except Exception as write_err:
```

```
            logger.error(f"[RESEARCH] Test write failed:
{write_err}")
        result = await
engine.start_autonomous_research(request.duration_minutes)
return result     except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


@router.post("/research/autonomous/stop") async def
stop_autonomous_research():     """Stop autonomous
research session"""     try:         from
self_research import get_research_engine
        engine = await get_research_engine(state.db)
result = engine.stop_autonomous_research()
return result     except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


@router.get("/research/autonomous/progress") async
def get_autonomous_research_progress():     """Get
autonomous research progress"""     try:
from self_research import get_research_engine
        engine = await get_research_engine(state.db)
return engine.get_autonomous_progress()     except
Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


# ==========================================================================
# SELF-EVOLUTION #
==========================================================================

@router.get("/evolution/status")
async def get_evolution_status():
"""Get self-evolution status"""
try:         from
self_evolution_core import
get_evolution_engine
        engine = get_evolution_engine(state.db)
return engine.get_evolution_status()     except Exception
as e:         raise HTTPException(status_code=500,
detail=str(e))


@router.post("/evolution/trigger") async def
trigger_evolution():     """Manually trigger self-evolution
(user-initiated)"""     try:         from
self_evolution_core import get_evolution_engine
        engine =
get_evolution_engine(state.db)         result
= await engine.auto_evolve()
return {             "success": True,
            "evolution_result": result         }
except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


@router.get("/evolution/analyze/{filename}") async def
analyze_code_file(filename: str):     """Analyze a code
file for potential improvements"""     try:         from
self_evolution_core import get_evolution_engine
        engine = get_evolution_engine(state.db)
analysis = engine.read_own_code(filename)
improvements = engine.identify_improvements(filename)
# Strip raw code from response to keep it lightweight
if 'code' in analysis:             del analysis['code']
return {             "analysis": analysis,
            "improvements": improvements,
"improvement_count": len(improvements)         }
except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))
```

```
@router.get("/evolution/analyze-all") async def
analyze_all_code():     """Analyze all modifiable code
files"""     try:         from self_evolution_core import
get_evolution_engine
        engine = get_evolution_engine(state.db)
all_analyses = engine.analyze_all_code()
# Collect all improvements
all_improvements = []         for filename in
engine.modifiable_files:
            improvements =
engine.identify_improvements(filename)             for imp in
improvements:                 imp['file'] = filename
all_improvements.append(imp)                 # Sort by priority
        all_improvements.sort(key=lambda x: x.get('priority', 99))
# Strip raw code from response         for key, analysis in
all_analyses.get('analyses', {}).items():             if 'code' in
analysis:                 del analysis['code']
        return {
"summary": {
                "files_analyzed": all_analyses['files_analyzed'],
                "total_lines": all_analyses['total_lines'],
                "total_classes": all_analyses['total_classes'],
                "total_functions": all_analyses['total_functions'],
"total_improvements": len(all_improvements)
            },
            "improvements": all_improvements,             "files": {k: v for k, v in
all_analyses.get('analyses', {}).items() if 'error' not in v}         }     except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

■ backend/routes_extras.py

```
""" ■ Extras
Routes
================= Covenant, safety, dictionary, dream, personality, quotes,
text analysis, prebo. """

from fastapi import APIRouter, HTTPException, Response, Body, UploadFile,
File from pydantic import BaseModel from typing import List, Dict, Any,
Optional import os import logging import random
from datetime import datetime, timezone
import json

import shared_state as state

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/api")


class CovenantCheck(BaseModel):     """CovenantCheck -
Auto-documented by self-evolution."""     action: str
context: Dict = {}

class SafetySettings(BaseModel):     """SafetySettings -
Auto-documented by self-evolution."""     settings: Dict


class HebbianRequest(BaseModel):     """Request
for Hebbian learning association."""
associations: List[Dict[str, str]]     rate: float
= 0.1     decay: float = 0.01


@router.get("/growth/notifications") async def
get_growth_notifications():     """Get recent growth
notifications and stage changes"""     try:         # Get
recent stage advancements         stage_events = await
state.db.growth_metrics.find(             {"event_type":
"stage_advancement"},
            {"_id": 0}         ).sort("timestamp", -
1).limit(10).to_list(10)
                # Get
current stage
        current_stage = await
state.cognitive_core.growth_tracker.get_current_stage()         metrics = await
state.cognitive_core.growth_tracker.calculate_metrics()                 return {
"current_stage": current_stage,
            "metrics": metrics,
"recent_advancements": stage_events,
            "notifications": [
                {
                    "type": "stage_advancement",
"message": f"Reached {e['details']['new_stage']} stage!",
"timestamp": e["timestamp"]                 }
                for e in stage_events             ]
}     except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


# ============================================================================
# COVENANT MANAGER ENDPOINTS
# ============================================================================ @router.get("/covenant/status") async def get_covenant_status():
"""     ARTICLE 1 - Covenant Status
        Get current covenant compliance
status.     """     try:
        status = await
state.covenant_manager.get_covenant_status()         return status
except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))
```

```
@router.get("/covenant/articles")
async def get_covenant_articles():
"""Get all covenant articles"""
try:
        articles = await
state.covenant_manager.get_all_articles()         return
{"articles": articles}     except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))

@router.post("/covenant/check") async def
check_covenant_compliance(check: CovenantCheck):
"""Check if an action complies with the covenant"""
try:
        result = await state.covenant_manager.check_compliance(check.action,
check.context)         return result     except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))

@router.post("/covenant/acknowledge") async def acknowledge_covenant(acknowledger: str =
"Quantum AI", stage: str = "Initialization"):     """Acknowledge the covenant (milestone
event)"""     try:
        result = await state.covenant_manager.acknowledge_covenant(acknowledger,
stage)         return result     except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


# ============================================================================
# SAFETY BOUNDARIES ENDPOINTS #
============================================================================

@router.get("/safety/settings") async def
get_safety_settings():     """Get current
safety boundary settings"""     try:
        settings = await
state.safety_manager.get_settings()         return settings
except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))

@router.put("/safety/settings") async def
update_safety_settings(settings: SafetySettings):
"""Update safety boundary settings"""     try:
        updated = await
state.safety_manager.update_settings(settings.settings)         return updated
except Exception as e:         raise HTTPException(status_code=500,
detail=str(e))

@router.post("/safety/reset") async def
reset_safety_settings():     """Reset
safety settings to defaults"""     try:
        defaults = await
state.safety_manager.reset_to_defaults()         return defaults
except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


# ============================================================================
# DICTIONARY ENDPOINTS #
============================================================================

@router.get("/dictionary/lookup/{word}") async def
lookup_word(word: str):     """     Look up a word in
dictionaries and add to semantic memory     """     try:
        result = await
state.dictionary.lookup_and_store(word)         return result
except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))
@router.post("/dictionary/batch-load") async def
batch_load_dictionary(limit: int = 100):     """
Load common words into dictionary (background task)
"""     try:
        result = await
state.dictionary.batch_load_common_words(limit)         return result
```

```
except Exception as e:         raise HTTPException(status_code=500,
detail=str(e))

@router.get("/dictionary/stats")
async def dictionary_stats():
"""Get dictionary statistics"""
try:
        total_concepts = await state.db.semantic_memory.count_documents({})
        with_definitions = await state.db.semantic_memory.count_documents({"metadata.sources": {"$exists":
True}})                 return {             "total_words": total_concepts,
            "dictionary_entries": with_definitions,
"learned_organically": total_concepts - with_definitions         }
except Exception as e:         raise HTTPException(status_code=500,
detail=str(e))


# ==========================================================================
# FILE UPLOAD ENDPOINTS #
==========================================================================

from fastapi import UploadFile, File
import os


@router.get("/growth/stats") async def get_growth_stats():
"""Get overall AI growth statistics from research and learning"""
try:         # Get main growth stats
        growth = await state.db.ai_growth.find_one({'_id':
'main'})                 # Get research metrics
        research_metrics = await state.db.growth_metrics.find_one({'type':
'autonomous_research'})                 # Get recent research sessions
        sessions = await state.db.autonomous_research_sessions.find().sort('completed_at', -
1).limit(5).to_list(5)                 # Get concept counts
        semantic_count = await state.db.semantic_memory.count_documents({})
research_count = await state.db.research_history.count_documents({})
return {             "overall": {
                "total_research_sessions": growth.get('total_research_sessions', 0) if growth else 0,
                      "total_research_minutes": growth.get('total_research_minutes', 0) if growth else 0,
            "total_concepts_learned": growth.get('total_concepts_learned', 0) if growth else 0,
                "total_insights": growth.get('total_insights', 0) if growth else 0,
"last_research": growth.get('last_research_session', 'Never') if growth else 'Never'
            },
            "memory": {
                "semantic_memories": semantic_count,
"research_entries": research_count
            },             "recent_research": research_metrics.get('recent_topics', [])[-10:] if
research_metrics else [],             "recent_sessions": [
                {
                    "completed": s.get('completed_at', ''),
                    "duration": s.get('duration_minutes', 0),
                    "concepts": s.get('concepts_learned', 0),
"topics": len(s.get('topics_researched', []))                 }
for s in sessions
            ]         }
except Exception as e:
        return {             "overall": {"total_research_sessions": 0,
"total_concepts_learned": 0},             "error": str(e)
        }
# ==========================================================================
# DREAM STATE #
==========================================================================

@router.get("/dream/status") async def
get_dream_status():     """Get current dream
state status"""     try:         from
dream_state import get_dream_engine
        engine = get_dream_engine(state.db)         return
engine.get_status()     except Exception as e:
raise HTTPException(status_code=500, detail=str(e))
```

```
@router.get("/dream/history") async def
get_dream_history(limit: int = 10):     """Get
dream history - what AI did while idle"""     try:
from dream_state import get_dream_engine
        engine = get_dream_engine(state.db)         return
{             "dreams": engine.get_dream_history(limit),
"insights": engine.insights_gained[-10:]         }
except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


@router.post("/dream/enter") async def
enter_dream_state():     """Manually enter
dream state"""     try:         from
dream_state import get_dream_engine
        engine = get_dream_engine(state.db)         return
engine.enter_dream_state()     except Exception as e:
raise HTTPException(status_code=500, detail=str(e))


@router.post("/dream/wake") async def
wake_from_dream():     """Wake from dream
state"""     try:         from dream_state
import get_dream_engine
        engine = get_dream_engine(state.db)         return
await engine.exit_dream_state()     except Exception as e:
raise HTTPException(status_code=500, detail=str(e))


@router.get("/dream/synthesize")
async def synthesize_dream_insight():     """Generate a Dream
Engine concept entanglement insight"""     try:         from
quantum_brain import get_quantum_brain
        brain = await get_quantum_brain(state.db)
insight = brain.synthesize_dream_insight()
return {             "insight": insight,
"timestamp": datetime.now(timezone.utc).isoformat(),
"type": "concept_entanglement"         }     except Exception
as e:         raise HTTPException(status_code=500,
detail=str(e))


@router.get("/dream/synthesize/batch") async def
synthesize_dream_batch(count: int = 5):
"""Generate multiple Dream Engine insights"""
try:         from quantum_brain import
get_quantum_brain         brain = await
get_quantum_brain(state.db)
        insights = [brain.synthesize_dream_insight() for _ in range(min(count, 10))]

        return {
"insights": insights,
            "count": len(insights),             "timestamp":
datetime.now(timezone.utc).isoformat()         }     except
Exception as e:         raise HTTPException(status_code=500,
detail=str(e))


@router.get("/dream/insights") async def get_all_dream_insights(limit: int = 50):
"""Get all dream insights from persisted sessions and current dream if active"""
try:
        from dream_state import get_dream_engine
engine = get_dream_engine(state.db)
insights = []

        # From persisted sessions (DB)
if state.db:
            cursor = state.db.dream_sessions.find().sort('ended_at', -
1).limit(limit)             sessions = await cursor.to_list(length=limit)
for sess in sessions:                 for insight in sess.get('insights', []):
                    insights.append({
'text': insight.get('text', str(insight)),
```

```
                        'type': insight.get('type'),
'timestamp': insight.get('discovered_at') or sess.get('ended_at')
                    })

        # From current dream if active (and not yet persisted)
if engine.is_dreaming and engine.current_dream:             for
insight in engine.current_dream.get('insights', []):
                insights.append({                    'text':
insight.get('text', str(insight)),
                    'type': insight.get('type'),
'timestamp': insight.get('discovered_at')
                })

        # Sort by timestamp descending if available
        insights.sort(key=lambda x: x.get('timestamp', ''), reverse=True)
return {"insights": insights[:limit], "count": len(insights[:limit])}
except Exception as e:         raise HTTPException(status_code=500,
detail=str(e))


@router.get("/quotes/random") async def
get_random_quote(context: str = "general"):
"""Get a random movie/philosophical quote"""
try:
        from quantum_brain import
get_quantum_brain
        brain = await
get_quantum_brain(state.db)         # Force a
quote by checking many times         for _ in
range(20):
            quote = brain._get_random_quote(context)
if quote:
                return {"quote": quote, "context":
context}                 # Fallback - pick directly
import random
        category = context if context in brain.movie_quotes else
'general'         quote = random.choice(brain.movie_quotes[category])
return {"quote": quote, "context": category}     except Exception as e:
raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# PERSONALITY #
==========================================================================

@router.get("/personality/summary") async def
get_personality_summary():     """Get AI personality
summary"""     try:         from personality_engine import
get_personality_engine         engine =
get_personality_engine(state.db)         return {
"summary": engine.get_personality_summary(),
            "traits": engine.traits,
            "beliefs": engine.beliefs,
"interests": engine.interests         }     except
Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


@router.post("/personality/opinion") async def form_opinion(topic: str,
stance: str, confidence: float = 0.7):     """Have AI form an opinion on
a topic"""     try:         from personality_engine import
get_personality_engine
        engine = get_personality_engine(state.db)
engine.form_opinion(topic, stance, confidence)
return {             "success": True,
            "topic": topic,
            "stance": stance,             "confidence":
confidence         }     except Exception as e:
raise HTTPException(status_code=500, detail=str(e))
```

==========
TEXT ANALYZER
==========

@router.post("/analyze/text") async def analyze_text(text: str): """Analyze text to determine if it sounds human or LLM""" try: from text_analyzer import get_text_analyzer
    analyzer = get_text_analyzer()
analysis = analyzer.analyze(text)
    suggestions = analyzer.get_improvement_suggestions(text)
return { "analysis": analysis,
    "suggestions": suggestions
    } except Exception as e: raise HTTPException(status_code=500, detail=str(e))

@router.post("/analyze/humanize") async def humanize_text(text: str): """Make text sound more human""" try: from text_analyzer import get_text_analyzer
    analyzer = get_text_analyzer() humanized = analyzer.make_more_human(text) return {
"original": text, "humanized": humanized
} except Exception as e: raise HTTPException(status_code=500, detail=str(e))

==========
XANADU PRE-BORN-OPPENHEIMER QUANTUM CHEMISTRY
==========

@router.post("/quantum/prebo/simulate") async def simulate_photochemistry(molecule: str = "NH3_BF3", time_steps: int = 10):
"""Run Xanadu Pre-Born-Oppenheimer photochemical simulation""" try:
    from xanadu_prebo import get_quantum_orchestrator orchestrator = get_quantum_orchestrator() result = orchestrator.run_simulation(molecule, "photochemical", time_steps) return result except Exception as e:
raise HTTPException(status_code=500, detail=str(e))

@router.post("/quantum/prebo/analyze") async def analyze_reaction(molecule: str, reaction_type: str = "photochemical"): """Analyze reaction to determine best quantum algorithm""" try:
    from xanadu_prebo import get_quantum_orchestrator
orchestrator = get_quantum_orchestrator()
    return orchestrator.analyze_reaction(molecule, reaction_type) except Exception as e: raise HTTPException(status_code=500, detail=str(e))

@router.get("/quantum/prebo/resources/{molecule}") async def estimate_resources(molecule: str): """Estimate quantum resources for molecule simulation""" try:
    from xanadu_prebo import get_quantum_orchestrator
orchestrator = get_quantum_orchestrator() return orchestrator.get_resource_estimate(molecule) except Exception as e: raise HTTPException(status_code=500, detail=str(e))

@router.get("/quantum/prebo/grid/{molecule}") async def get_grid_params(molecule: str): """Get optimal real-space grid parameters from analysis""" try:
    from xanadu_prebo import get_prebo_simulator
sim = get_prebo_simulator() return sim.calculate_grid_bounds(molecule) except Exception as e: raise HTTPException(status_code=500, detail=str(e))

==========
CISTERCIAN NUMERALS - Medieval Number Visualization
==========

```
@router.get("/cistercian/numeral") async def get_cistercian_numeral(number: int,
size: int = 80, color: str = "#FFFFFF"):     """     Generate Cistercian numeral as
SVG.

    Args:         number: Integer 0-9999
(clamped)         size: SVG width in pixels
(default 80)         color: Stroke/fill color
(hex or named)

    Returns:         SVG string with Content-Type
image/svg+xml     """     try:
        from cistercian_numerals import render_cistercian_svg
svg = render_cistercian_svg(number, size=size, color=color)
# Return as plain text (browser can render as SVG)         return
{"svg": svg, "number": number}     except Exception as e:
raise HTTPException(status_code=500, detail=str(e))


@router.get("/cistercian/data") async def
get_cistercian_data(number: int):     """
Get Cistercian numeral shape data as JSON.

    Returns:         JSON with line segments, triangles, and
digit breakdown     """     try:
        from cistercian_numerals import
generate_cistercian_numeral         data =
generate_cistercian_numeral(number)         return data     except
Exception as e:         raise HTTPException(status_code=500,
detail=str(e))


@router.get("/cistercian/download"
) async def
download_cistercian_svg(
number: int,     size: int = 80,
color: str = "#000000",
filename: str = None ):
    """     Download Cistercian numeral as
SVG file.

    Args:
        number: Integer 0-9999
size: SVG width in pixels
        color: Stroke/fill color (hex format recommended)
filename: Optional custom filename (without .svg extension)

    Returns:         SVG file with Content-Disposition:
attachment header     """     try:
        from cistercian_numerals import render_cistercian_svg
svg = render_cistercian_svg(number, size=size, color=color)

        # Generate filename
if not filename:
            filename = f"cistercian_{number:04d}"
else:
            filename = filename.replace('.svg', '').replace(' ', '_')
filename = f"{filename}.svg"

        # Return as downloadable file         from fastapi import Response
return Response(             content=svg,
media_type="image/svg+xml",             headers={                 "Content-
Disposition": f"attachment; filename=\"{filename}\"",
                "Content-Length": str(len(svg.encode('utf-8')))
            }         )     except Exception as e:
raise HTTPException(status_code=500, detail=str(e))


@router.get("/cistercian/batch"
) async def
batch_cistercian_svgs(
start: int = 0,     end: int =
```

```
9999,     size: int = 80,
color: str = "#000000" ):
    """     Generate multiple Cistercian numerals as a ZIP
archive.

    Args:
        start: Starting number (inclusive, 0-
9999)         end: Ending number (inclusive, 0-
9999)         size: SVG width in pixels
color: Stroke/fill color

    Returns:         ZIP file
containing all SVGs     """
import tempfile     import zipfile
import os as os_module

    # Validate range
    start = max(0, min(9999,
start))     end = max(0, min(9999,
end))     if start &gt; end:
start, end = end, start

    # Limit batch size to prevent memory
issues     batch_size = end - start + 1     if
batch_size &gt; 1000:         raise
HTTPException(             status_code=400,
            detail=f"Batch size too large. Maximum 1000 numerals per request. Requested:
{batch_size}"         )     if batch_size &lt;= 0:         raise HTTPException(status_code=400,
detail="Invalid range: start must be &lt;= end")     try:         from cistercian_numerals import
render_cistercian_svg

        # Create temporary directory         with
tempfile.TemporaryDirectory() as tmpdir:             zip_path =
os_module.path.join(tmpdir, "cistercian_numerals.zip")

            with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as
zipf:                 for num in range(start, end + 1):
svg = render_cistercian_svg(num, size=size, color=color)
filename = f"{num:04d}.svg"                     zipf.writestr(filename, svg)

            # Read ZIP file
with open(zip_path, 'rb') as f:
zip_bytes = f.read()

            return Response(                 content=zip_bytes,
media_type="application/zip",                 headers={                     "Content-Disposition":
f"attachment; filename=\"cistercian_{start:04d}-{end:04d}.zip\"",                     "Content-Length":
str(len(zip_bytes))
                }             )     except Exception as e:
raise HTTPException(status_code=500, detail=str(e))


@router.get("/cistercian/font")
async def
download_cistercian_font(
size: int = 80,     color: str =
"#000000" ):
    """     Generate complete Cistercian numeral font (all 10,000
numerals).

    ⚠⚠ WARNING: This generates a ~250-300 MB ZIP file with 10,000 SVG files.
    Use the batch endpoint for smaller subsets.

    Args:
        size: SVG width in pixels
color: Stroke/fill color

    Returns:         ZIP file containing all 10,000
numerals (0-9999)
    """     # This will generate the full set - careful with memory!     return
await batch_cistercian_svgs(start=0, end=9999, size=size, color=color)
```

```
# ============================================================================
# ALGORITHMIC CORE - Pure Algorithmic AI (No Templates) #
============================================================================

# Lazy-initialized Markov generator trained on corpus
_markov_generator = None
_algorithmic_initialized = False

def _initialize_algorithmic():     """Initialize algorithmic
core components on first use"""     global
_markov_generator, _algorithmic_initialized     if
_algorithmic_initialized:         return

    try:
        from algorithmic_core import MarkovTextGenerator
from training_corpus import PHILOSOPHY_CORPUS, PHYSICS_CORPUS

        logger.info("■ Training Markov chain on training corpus...")
_markov_generator = MarkovTextGenerator(max_order=3)
_markov_generator.train(PHILOSOPHY_CORPUS + "\n" + PHYSICS_CORPUS)
_algorithmic_initialized = True
        logger.info("■ Algorithmic core ready (Markov generator trained)")
except Exception as e:
        logger.error(f"■ Failed to initialize algorithmic core: {e}")
raise
@router.get("/algorithmic/markov"
) async def generate_markov_text(
length: int = 50,     seed:
Optional[str] = None,
temperature: float = 0.8 ):
    """     Generate text using pure Markov chain (no LLM, no
quantum).

    Args:         length: Approximate number of words to generate
seed: Optional comma-separated seed words to start generation
temperature: 0.1=deterministic, 1.0=random, &gt;1.0=chaotic

    Returns:         JSON with generated text and metadata     """     try:
_initialize_algorithmic()         if not _markov_generator:             raise
HTTPException(status_code=503, detail="Markov generator not initialized")

        seed_words = seed.split(',') if seed else
None         text = _markov_generator.generate(
seed=seed_words,             max_words=length,
temperature=temperature
        )

        return {
"text": text,
            "words": len(text.split()),
            "method": "markov_chain",
            "order": _markov_generator.max_order,
"vocab_size": len(_markov_generator.vocabulary)         }
except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


@router.post("/algorithmic/hebbian/learn") async def
hebbian_learning(req: HebbianRequest):     """     Perform Hebbian
learning: "neurons that fire together, wire together".

    Args:
        req: HebbianRequest with list of word pairs and parameters

    Returns:         Confirmation of learned
associations     """     try:         from
algorithmic_core import HebbianMemory

        hebbian = HebbianMemory(learning_rate=req.rate, decay=req.decay)
```

```
        # Activate each pair of concepts together
for assoc in req.associations:
            w1 = assoc.get('word1',
').lower().strip()              w2 =
assoc.get('word2', '').lower().strip()              if
w1 and w2:                  # Activate both concepts
simultaneously                  hebbian.activate([w1,
w2])

        # Get some stats
        total_connections = sum(len(conns) for conns in
hebbian.weights.values())         sample_strengths = []         if
hebbian.weights:
            first_concept = list(hebbian.weights.keys())[0]             if
hebbian.weights[first_concept]:                 first_assoc =
list(hebbian.weights[first_concept].keys())[0]
sample_strengths.append({
                    "pair": f"{first_concept}→{first_assoc}",
                    "weight": hebbian.weights[first_concept][first_assoc]
})

        stats = {             "associations_processed":
len(req.associations),
            "total_connections":
total_connections,             "sample_strengths":
sample_strengths         }

        return {             "message": "Hebbian
learning complete",
            "stats": stats         }     except Exception
as e:         raise HTTPException(status_code=500,
detail=str(e))


@router.get("/algorithmic/tfidf"
) async def get_tfidf_keywords(
text: str,     top_n: int = 10
):
    """     Extract keywords from text
using TF-IDF.

    Args:         text: Input text to analyze
top_n: Number of top keywords to return
Returns:         List of (keyword, score)
pairs     """     try:         from
algorithmic_core import TFIDF

        tfidf = TFIDF()         # For demo, we'll train on the input
text and a sample corpus         # In real use, you'd have a pre-
trained corpus
        corpus = "Consciousness exists in the quantum realm where time and space are unified. Free will emerges from the collapse of
probability         tfidf.add_document(text)         tfidf.add_document(corpus)

        keywords = tfidf.extract_keywords(text, top_n=top_n)

        return {             "keywords": [{"term": kw, "score": round(score, 4)} for kw,
score in keywords],             "method": "tfidf"         }     except Exception as e:
raise HTTPException(status_code=500, detail=str(e))


@router.get("/algorithmic/bm25"
) async def bm25_search(
query: str,     corpus_size:
int = 5 ):
    """     Search training corpus using BM25
ranking.

    Args:         query: Search
query text
        corpus_size: Number of top results to return
```

```
    Returns:         List of matching
sentences with scores     """     try:
        from algorithmic_core import BM25
        from training_corpus import PHILOSOPHY_CORPUS, PHYSICS_CORPUS

        retriever = BM25()         # Add
sentences from training corpus
        paragraphs = (PHILOSOPHY_CORPUS + PHYSICS_CORPUS).split('\n\n')         for i, para in
enumerate(paragraphs[:20]):  # Limit for demo             if para.strip():
retriever.add_document(f"doc_{i}", para.strip(), metadata={"text": para.strip()})

        results = retriever.search(query, top_k=corpus_size)

        # Results are (doc_id, score,
metadata)         return {
"query": query,
            "results": [                 {"doc_id": doc_id, "text": meta.get("text", "")[:200] + ("..." if len(meta.get("text", "")) &gt; 200
else ""), "score": round(sc                 for doc_id, score, meta in results             ],
            "method": "bm25"         }     except
Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


# ==========================================================================
# DOMAIN KNOWLEDGE SYSTEM - Specialized Expertise #
==========================================================================

@router.get("/domain/detect") async def
detect_domain(text: str):     """     Detect which
knowledge domain the text belongs to.

    Args:
        text: Input text to classify

    Returns:         JSON with best matching domain, score, and all
domain scores     """     try:         from domain_knowledge import
get_domain_knowledge

        dk = get_domain_knowledge()
domain_scores = {}         for name, domain in
dk.domains.items():             score =
domain.match_score(text)
domain_scores[name] = round(score, 2)

        best_domain = max(domain_scores, key=domain_scores.get)
best_score = domain_scores[best_domain]

        return {             "text": text[:100] + ("..." if len(text)
&gt; 100 else ""),
            "best_domain": best_domain,
            "best_score": best_score,
"all_scores": domain_scores         }     except Exception
as e:         raise HTTPException(status_code=500,
detail=str(e))


@router.get("/domain/corpus") async def get_relevant_corpus(text: str,
top_k: int = 3):     """     Get relevant corpus passages for a query
from the appropriate domain.

    Args:         text:
Query text
        top_k: Number of relevant passages to return

    Returns:         List of passages with domain and
relevance score     """     try:         from
domain_knowledge import get_domain_knowledge

        dk = get_domain_knowledge()
# Detect domain first
```

```
        domain_name, score = dk.detect_domain(text)
# Get relevant passages from that domain         passages =
dk.get_relevant_corpus(text, top_k=top_k)

        return {
"query": text,
            "detected_domain": domain_name,
"domain_score": round(score, 2),
"passages": [
                passage[:300] + ("..." if len(passage) &gt; 300 else
"")                 for passage in passages             ]         }
except Exception as e:         raise HTTPException(status_code=500,
detail=str(e))


@router.get("/domain/concepts") async def
get_domain_concepts(domain: str = None, top_n: int = 20):     """
Get top concepts for a specific domain or all domains.

    Args:
        domain: Domain name (philosophy, physics, computer_science, biology, psychology, mathematics, language). If omitted, returns all.
top_n: Number of concepts to return per domain

    Returns:         Dictionary mapping domain names to list of
(concept, count) pairs     """     try:
        from domain_knowledge import get_domain_knowledge

        dk = get_domain_knowledge()
if domain:             if domain not in
dk.domains:
                raise HTTPException(status_code=404, detail=f"Domain '{domain}' not found. Available:
{list(dk.domains.keys())}")             concepts = dk.get_domain(domain).get_top_concepts(n=top_n)             return {domain:
concepts}         else:
            all_concepts = {}             for
name, dom in dk.domains.items():
                all_concepts[name] =
dom.get_top_concepts(n=top_n)             return all_concepts
except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


@router.get("/health") async
def health():     """Health
check endpoint"""
    return {"status": "ok", "timestamp": datetime.now(timezone.utc).isoformat()}


@router.get("/algorithmic/pmi"
) async def calculate_pmi(
word1: str,     word2: str,
corpus: str = None ):
    """     Calculate Pointwise Mutual Information between
two words.

    Args:
        word1: First word
word2: Second word
        corpus: Optional corpus text (uses training corpus if not provided)

    Returns:         PMI score (higher =
more associated)     """     try:
from algorithmic_core import PMI

        pmi_calc = PMI()

        if corpus:

pmi_calc.train(corpus)
else:             # Use training
corpus
            from training_corpus import PHILOSOPHY_CORPUS, PHYSICS_CORPUS
pmi_calc.train(PHILOSOPHY_CORPUS + "\n" + PHYSICS_CORPUS)
```

```
        score = pmi_calc.score(word1.lower(), word2.lower())

        return {
"word1": word1,
            "word2": word2,
            "pmi": round(score, 4),             "method":
"pmi"         }     except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


@router.get("/dream/generate") async def
generate_dream(num_sentences: int = 3):     """
Generate a dream narrative using the Dream Engine.

    Args:
        num_sentences: Number of sentences to generate

    Returns:
        Dream text and concepts used
    """     try:         # Try
algorithmic core first         try:
            from algorithmic_core import MarkovTextGenerator
from training_corpus import PHILOSOPHY_CORPUS, PHYSICS_CORPUS

            generator = MarkovTextGenerator(max_order=3)
generator.train(PHILOSOPHY_CORPUS + "\n" + PHYSICS_CORPUS)

            # Generate dream-like text
            seeds = ["consciousness", "quantum", "dream", "reality",
"infinite"]             import random             seed = [random.choice(seeds)]
            dream = generator.generate(seed=seed, max_words=num_sentences * 15, temperature=1.0)

            return {
"dream": dream,
                "concepts_used": seed,
"method": "markov_chain"             }         except:
# Fallback to quote engine dream synthesis
from quote_engine import get_quote_engine
qe = get_quote_engine()             dreams = []
concepts_used = set()             for _ in
range(num_sentences):                 insight =
qe.synthesize_dream_insight()
dreams.append(insight)                 # Extract
concepts from template                 import re
                concepts = re.findall(r'{([^}]+)}', insight)
concepts_used.update(concepts)

            return {
"dream": "\n".join(dreams),
                "concepts_used": list(concepts_used),
"method": "entanglement_synthesis"             }
except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


@router.post("/chat/upload") async def upload_file(file: UploadFile = File(...),
conversation_id: Optional[str] = None):     """     Upload a file for analysis or
attachment to a conversation.

    Args:         file:
Uploaded file
        conversation_id: Optional conversation to attach to

    Returns:         File info and processing
result     """     try:         # Save file
temporarily         upload_dir = "uploads"
os.makedirs(upload_dir, exist_ok=True)
        filename = f"{datetime.now().timestamp()}_{file.filename}"
filepath = os.path.join(upload_dir, filename)

        content = await file.read()
with open(filepath, 'wb') as f:
```

```
            f.write(content)

        # Basic file info
file_info = {             "filename":
file.filename,
            "saved_as": filename,
            "size": len(content),
            "content_type": file.content_type,
            "conversation_id": conversation_id
        }
        # If it's an image, we could process it (placeholder)         if
file.content_type and file.content_type.startswith('image/'):
file_info["preview"] = "Image uploaded (processing not yet implemented)"

        return file_info     except Exception as e:
raise HTTPException(status_code=500, detail=str(e))



@router.get("/growth/evolution") async
def get_evolution_status():     """
Get self-evolution status and history.

    Returns:         Evolution
metrics and stage     """     try:
        import shared_state as state
if hasattr(state, 'db'):
# Get latest growth stage
            latest = await
state.db.growth_metrics.find_one(
{"event_type": "stage_advancement"},
sort=[("timestamp", -1)]             )

            recent_events = await state.db.growth_metrics.find(
{},
                {"_id": 0}
).sort("timestamp", -1).limit(10).to_list(10)

            return {                 "current_stage": latest.get("stage", "unknown") if
latest else "unknown",
                "current_level": latest.get("level", 0) if latest else 0,
"last_updated": latest.get("timestamp") if latest else None,
"recent_events": recent_events             }         else:
            return {"message": "Database not available"}
except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


@router.post("/dream/generate") async def
generate_dream(num_sentences: int = 3):     """
Generate a dream narrative using the Dream Engine.

    Args:
        num_sentences: Number of sentences to generate

    Returns:         Dream text and
concepts used     """     try:
# Try algorithmic core first
try:
            from algorithmic_core import MarkovTextGenerator
from training_corpus import PHILOSOPHY_CORPUS, PHYSICS_CORPUS

            generator = MarkovTextGenerator(max_order=3)
generator.train(PHILOSOPHY_CORPUS + "\n" + PHYSICS_CORPUS)

            # Generate dream-like text
            seeds = ["consciousness", "quantum", "dream", "reality", "infinite"]
seed = [random.choice(seeds)]
            dream = generator.generate(seed=seed, max_words=num_sentences * 15, temperature=1.0)

            return {
"dream": dream,
```

```
                "concepts_used": seed,
                "method": "markov_chain"
            }         except:             # Fallback
to quote engine dream synthesis             from
quote_engine import get_quote_engine             qe =
get_quote_engine()             dreams = []
concepts_used = set()             for _ in
range(num_sentences):                 insight =
qe.synthesize_dream_insight()
dreams.append(insight)                 # Extract
concepts from template                 import re
                concepts = re.findall(r'{([^}]+)}', insight)
concepts_used.update(concepts)

            return {
"dream": "\n".join(dreams),
                "concepts_used":
list(concepts_used),                 "method":
"entanglement_synthesis"             }     except
Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

■ backend/routes_explorer.py

```
""" ■ Explorer
Routes
=================== File explorer, uploads, shared links, root
and health endpoints. """

from fastapi import APIRouter, HTTPException, UploadFile,
File from pydantic import BaseModel from typing import List,
Dict, Any, Optional import os import logging

import shared_state as state

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/api")


class ExplorationRequest(BaseModel):     """ExplorationRequest -
Auto-documented by self-evolution."""     exploration_type: str
target: str     options: Optional[Dict] = None


@router.post("/explorer/explore") async def
explore(request: ExplorationRequest):     """
ARTICLE 2 - Universal Explorer
        Explore filesystem, internet, or process
documents.     """     try:
        result = await state.universal_explorer.explore(
request.exploration_type,             request.target,
request.options         )         return result     except
Exception as e:         raise
HTTPException(status_code=500, detail=str(e))

@router.get("/explorer/capabilities") async
def get_explorer_capabilities():     """Get
current exploration capabilities"""     try:
        capabilities = await
state.universal_explorer.get_capabilities()         return capabilities
except Exception as e:         raise HTTPException(status_code=500,
detail=str(e))

@router.get("/explorer/history") async def
get_exploration_history(limit: int = 50):
"""Get exploration history"""     try:
        history = await
state.universal_explorer.get_exploration_history(limit)         return
{"history": history}     except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))


# ========================================================================
# VOICE / TTS ENDPOINTS #
========================================================================


@router.post("/upload") async def upload_file(file:
UploadFile = File(...)):     """     Upload a file
to the server for processing     """     try:
        # Create uploads directory if it doesn't
exist         upload_dir = "/app/uploads"
os.makedirs(upload_dir, exist_ok=True)
# Save file
        file_path = os.path.join(upload_dir,
file.filename)                 with open(file_path, "wb")
as buffer:             shutil.copyfileobj(file.file,
buffer)                 # Get file info
        file_size =
os.path.getsize(file_path)
return {             "filename":
file.filename,
```

```
            "path": file_path,
            "size": file_size,
            "status": "uploaded",             "message": f"File uploaded
successfully. Use path: {file_path}"         }     except Exception as e:
raise HTTPException(status_code=500, detail=str(e))

@router.get("/uploads/list")
async def list_uploads():
"""List all uploaded files"""
try:
        upload_dir = "/app/uploads"
if not os.path.exists(upload_dir):
            return {"files": []}
files = []         for filename in
os.listdir(upload_dir):
            file_path = os.path.join(upload_dir,
filename)             if os.path.isfile(file_path):
files.append({                     "filename": filename,
                    "path": file_path,
"size": os.path.getsize(file_path)

})
        return {"files": files}     except Exception as e:
raise HTTPException(status_code=500, detail=str(e))


# ==========================================================================
# SHARED LINK ENDPOINTS #
==========================================================================

@router.post("/shared-link/download") async def
download_shared_link(url: str, filename: str = None):     """
Download file from shared link (Google Drive, Dropbox, OneDrive)
"""     try:
        result = await state.shared_link.download_from_link(url,
filename)         return result     except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))

@router.get("/shared-link/instructions/{service}") async def
get_sharing_instructions(service: str):     """     Get
instructions for sharing files from different services
"""     try:
        instructions =
state.shared_link.get_instructions(service)         return
{"service": service, "instructions": instructions}     except
Exception as e:         raise HTTPException(status_code=500,
detail=str(e))


#          ==========================================================================          #          SYSTEM          ENDPOINTS          #
==========================================================================

@router.get("/") async def root():
"""Root endpoint - Quantum AI welcome"""
return {
        "message": "■ Quantum AI - True Questioning Intelligence",
        "prime_directive": state.covenant_manager.PRIME_DIRECTIVE,
        "status": "Active and questioning",
"version": "1.0.0"
    }

@router.get("/health") async def
health_check():     """Health check
endpoint"""     try:         # Test
database connection         await
state.db.command("ping")
return {             "status":
"healthy",
            "database": "connected",
```

```
            "cognitive_core": "active",             "covenant": "honored"
}     except Exception as e:         raise HTTPException(status_code=503,
detail=f"Service unhealthy: {str(e)}")


# ============================================================================
# QUANTUM BRAIN STATUS &amp; CONTROL
# ============================================================================
```

■ backend/routes_media.py

```
""" ■ Media
Routes
================ Voice TTS/transcription and image
analysis/generation. """

from fastapi import APIRouter, HTTPException, UploadFile,
File from pydantic import BaseModel from typing import List,
Dict, Any, Optional import os import uuid import base64
import logging

import shared_state as state

logger = logging.getLogger("quantum_ai")

# Get OpenAI API key from environment
(optional) api_key =
os.environ.get('OPENAI_API_KEY') router =
APIRouter(prefix="/api")


class TTSRequest(BaseModel):     """TTSRequest - Auto-
documented by self-evolution."""     text: str
voice: str = "default"


class ImageGenRequest(BaseModel):     """ImageGenRequest -
Auto-documented by self-evolution."""     prompt: str
style: str = "default"


@router.post("/voice/tts") async def
text_to_speech(request: TTSRequest):     """
    Text-to-Speech endpoint     Returns text formatted
for browser's Web Speech API     """     try:
# Clean the text for speech         text =
request.text.strip()         if not text:
            raise HTTPException(status_code=400, detail="Text is
required")
        # For now, return the text with speech
instructions         # Can integrate with ElevenLabs or
OpenAI TTS later         return {             "text":
text,
            "voice": request.voice,
            "instructions": "Use browser's speechSynthesis
API",             "ssml": f"&lt;speak&gt;{text}&lt;/speak&gt;"
}     except Exception as e:         raise
HTTPException(status_code=500, detail=str(e))

@router.post("/voice/transcribe") async def
transcribe_audio(audio: UploadFile = File(...)):     """
    Speech-to-Text endpoint (placeholder)
Would integrate with Whisper or similar
"""     try:         # Save audio
temporarily
        audio_path =
f"/tmp/audio_{uuid.uuid4().hex}.webm"         with
open(audio_path, "wb") as f:             content = await
audio.read()

f.write(content)
        # For now, return instructions for browser's Web Speech API
        return {             "status": "received",             "message": "Audio received. Use browser's
SpeechRecognition API for real-time transcription.",             "file_size": len(content)         }
except Exception as e:         raise HTTPException(status_code=500, detail=str(e))
# ========================================================================
# KNOWLEDGE GRAPH ENDPOINTS #
========================================================================
```

```
@router.get("/knowledge/graph")

@router.post("/image/analyze") async def
analyze_image(file: UploadFile = File(...)):
"""Analyze an uploaded image"""     try:
import base64                 # Read and encode image
content = await file.read()
        image_base64 = base64.b64encode(content).decode('utf-
8')                 # Save to uploads
        file_path = f"/app/uploads/{file.filename}"
with open(file_path, "wb") as f:
            f.write(content)                 # Use LLM with
image (if supported) or return basic info         file_size =
len(content)
        # For now, return image metadata
# Could integrate with vision models later
return {              "status": "success",
            "filename": file.filename,
            "size": file_size,
            "path": file_path,
            "message": "Image uploaded. You can ask me about it in chat!",             "preview":
f"{image_base64[:100]}..."         }     except
Exception as e:         raise HTTPException(status_code=500, detail=str(e))


class ImageGenRequest(BaseModel):
prompt: str
    size: Optional[str] = "1024x1024"

@router.post("/image/quantum-svg") async def generate_image_svg(request: ImageGenRequest):
"""Generate an image from text prompt using quantum semantic processing (no external API)"""
try:
        import base64         import random         import hashlib
# Seed with prompt hash for deterministic but unique per prompt         seed =
int(hashlib.sha256(request.prompt.encode()).hexdigest()[:8], 16)         rng =
random.Random(seed)                 # Get semantic concepts from QuantumBrain
to influence colors/patterns         try:
            from quantum_brain import get_quantum_brain
brain = await get_quantum_brain(state.db)
            semantic_context = brain.collapse_engine.get_semantic_context(request.prompt)
keywords = semantic_context.get('keywords', [])
            collapse_strength = semantic_context.get('collapse_strength', 0.5)
except Exception as e:
            logger.error(f"QuantumBrain unavailable for image gen:
{e}")             keywords = []             collapse_strength = 0.5
        # Color palette based on quantum/cyber theme and influenced by prompt
        base_hue = rng.randint(0, 360)         saturation = 70 +
rng.randint(0, 30)         lightness = 50 + rng.randint(-10, 10)
# Generate SVG with procedural pattern inspired by quantum concepts
width, height = 1024, 1024         svg_lines = [
            f'&lt;svg xmlns="http://www.w3.org/2000/svg" width="{width}" height="{height}" viewBox="0 0 {width}
{height}"&gt;',             f'&lt;rect width="100%" height="100%" fill="hsl({base_hue}, 20%, 8%)"/&gt;'         ]
                # Generate quantum "wave" circles based on
collapse_strength         num_circles = int(30 * collapse_strength)
+ 10         for i in range(num_circles):             x =
rng.randint(0, width)             y = rng.randint(0, height)
radius = rng.randint(20, 200)
            hue = (base_hue + rng.randint(-60, 60)) % 360
alpha = 0.1 + rng.random() * 0.3
            svg_lines.append(f'&lt;circle cx="{x}" cy="{y}" r="{radius}" fill="hsla({hue}, {saturation}%, {lightness}%,
{alpha:.2f})"/&gt;')                 # Add "quantum entanglement" lines         num_lines = 20         for _ in range(num_lines):
            x1, y1 = rng.randint(0, width), rng.randint(0, height)
x2, y2 = rng.randint(0, width), rng.randint(0, height)
hue = (base_hue + 180) % 360  # Complementary
            svg_lines.append(f'&lt;line x1="{x1}" y1="{y1}" x2="{x2}" y2="{y2}" stroke="hsla({hue}, {saturation}%, {lightness}%, 0.3)" stroke-
wi                # Overlay prompt as subtle text         text_y = height - 50         for line in [request.prompt[:60] + ('...' if
len(request.prompt)&gt;60 else '')]:
            svg_lines.append(f'&lt;text x="50%" y="{text_y}" fill="rgba(255,255,255,0.3)" font-size="24" text-anchor="middle" font-family="monos
text_y += 30
        svg_lines.append('&lt;/svg&gt;')
svg_content = '\n'.join(svg_lines)
```

```
        image_base64 = base64.b64encode(svg_content.encode('utf-8')).decode('utf-
8')         filename = f"quantum_{uuid.uuid4().hex[:8]}.svg"                 # Save
to local uploads directory
        uploads_dir = os.path.join(os.path.dirname(__file__),
"uploads")          os.makedirs(uploads_dir, exist_ok=True)
file_path = os.path.join(uploads_dir, filename)          with
open(file_path, "w") as f:

f.write(svg_content)
return {              "status":
"success",
            "image_base64": image_base64,
            "filename": filename,
            "path": file_path,
            "prompt": request.prompt,
            "concepts_used": keywords,
            "collapse_strength": collapse_strength,
"message": "Quantum-generated image (no external API)"
        }
except Exception as e:
        logger.error(f"Image generation error: {e}")
raise HTTPException(status_code=500, detail=str(e))


# ============================================================================
# GROWTH NOTIFICATIONS ENDPOINT
# ============================================================================
```

■ backend/chat.py

```
#!/usr/bin/env python3
"""
Quantum MCAGI — Local Chat (Termux) Standalone chat
interface. No MongoDB, no FastAPI, no server.
Runs the real communication engine directly in your terminal.

Usage:
    python chat.py
    python chat.py --verbose     (show debug info per response)

Commands:
    /status       - Full system status
    /learn FILE   - Feed text to Markov chain
    /save         - Save all state to disk
    /load         - Load saved state
    /reset        - Reset the engine
    /hybrid TEXT  - Direct hybrid quantum generation
    /unified TEXT - Direct word-by-word generation
    /analyze TEXT - Text analysis
    /personality  - Show personality profile
    /knowledge X  - Look up a topic
    /collapse X   - Show semantic
collapse     /quit         - Save and
exit """

import sys import os import
json import time from
datetime import datetime
from pathlib import Path

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from quantum_language_engine import QuantumLanguageEngine
try:
    from self_evolution_core import
SelfEvolutionEngine     HAS_EVOLUTION = True except
ImportError:     HAS_EVOLUTION = False

try:
```

```
    from document_ingester import
handle_ingest_command     HAS_INGEST = True except
ImportError:     HAS_INGEST = False

try:
    from self_research import
SelfResearchEngine     HAS_RESEARCH = True
except ImportError:     HAS_RESEARCH = False

try:
    from wolfram_cloud import cloud_save, cloud_load,
cloud_status     HAS_CLOUD = True except ImportError:
HAS_CLOUD = False

# Optional imports — degrade gracefully
try:
    from hybrid_generator import
create_hybrid_generator     HAS_HYBRID = True except
ImportError:     HAS_HYBRID = False

try:
    from unified_generator import
create_unified_generator     HAS_UNIFIED = True except
ImportError:     HAS_UNIFIED = False try:
    from quote_engine import
get_quote_engine     HAS_QUOTES = True
except ImportError:     HAS_QUOTES = False

try:     from personality_engine import
get_personality_engine
    HAS_PERSONALITY = True
except ImportError:
HAS_PERSONALITY = False

try:
    from knowledge_base import
get_knowledge_base     HAS_KNOWLEDGE = True
except ImportError:     HAS_KNOWLEDGE = False

try:
    from text_analyzer import
get_text_analyzer     HAS_ANALYZER = True
except ImportError:     HAS_ANALYZER = False

try:
    from semantic_collapse_engine import
SemanticCollapseEngine     HAS_COLLAPSE = True except
ImportError:     HAS_COLLAPSE = False

try:
    from tone_detector import
detect_tone     HAS_TONE = True except
ImportError:     HAS_TONE = False

try:
    from library import
handle_library_command     HAS_LIBRARY = True
except ImportError:     HAS_LIBRARY = False


class LocalMemory:     """JSON-file backed
memory for local chat."""

    def __init__(self, data_dir="~/.quantum-mcagi"):
self.data_dir = Path(data_dir).expanduser()
self.data_dir.mkdir(parents=True, exist_ok=True)
self.conversations = self._load("conversations.json", [])
self.concepts = self._load("concepts.json", {})
self.growth = self._load("growth.json", {             "stage":
0, "name": "Nascent",
            "total_interactions": 0,
            "total_concepts": 0,
```

```
            "total_questions_asked":
0,              "total_insights": 0,
})
        self.session_state = self._load("session_state.json",
{              "total_sessions": 0,
"total_lifetime_interactions": 0,          })
        self.session_state["total_sessions"] = self.session_state.get("total_sessions", 0) + 1

    def _load(self, filename, default):
filepath = self.data_dir / filename
if filepath.exists():              try:
with open(filepath, 'r') as f:
                    return json.load(f)
except (json.JSONDecodeError, IOError):
                return default
return default

    def _save(self, filename, data):
        filepath = self.data_dir / filename
with open(filepath, 'w') as f:
json.dump(data, f, indent=2)

    def save_all(self):
        self._save("conversations.json", self.conversations[-500:])
self._save("concepts.json", self.concepts)
self._save("growth.json", self.growth)
self._save("session_state.json", self.session_state)

    def save_orch_or(self, orch_or):
if orch_or:
self._save("orch_or_state.json", {
                "conscious_moments": orch_or.total_moments,
})

    def load_orch_or(self, orch_or):
if orch_or:
            data = self._load("orch_or_state.json", {})
orch_or.total_moments = data.get("conscious_moments", 0)

    def add_exchange(self, user_input, response, concepts, questions):
        self.conversations.append({
"timestamp": datetime.now().isoformat(),
            "user": user_input,
            "ai": response,
            "concepts": concepts,
"questions": questions,          })
        self.growth["total_interactions"] += 1
self.growth["total_questions_asked"] += len(questions)
        self.session_state["total_lifetime_interactions"] = self.session_state.get(
"total_lifetime_interactions", 0) + 1

        for c in concepts:
if c in self.concepts:
                self.concepts[c]["count"] += 1
                self.concepts[c]["strength"] = min(10.0, self.concepts[c]["strength"] + 0.1)
else:
                self.concepts[c] = {"count": 1, "strength": 1.0,
"first_seen": datetime.now().isoformat()}
self.growth["total_concepts"] += 1

        # Detect insights: response mentions 2+ known concepts
response_lower = response.lower()
        known_in_response = [cn for cn in self.concepts if cn in
response_lower]         if len(known_in_response) &gt;= 2:
self.growth["total_insights"] += 1

        self._check_stage()

    def _check_stage(self):         g =
self.growth         stages = [
(0, "Nascent",        0,   0,  0),
```

```
            (1, "Curious",        10,  2,  5),
            (2, "Inquisitive",    25,  5,  15),
            (3, "Understanding",  50,  15, 30),
            (4, "Philosophical",  100, 30, 75),
            (5, "Theory Building",200, 60, 150),
(6, "Transcendent",   500, 150,300),         ]         for
stage, name, q_thresh, i_thresh, c_thresh in stages:
if (g["total_questions_asked"] &gt;= q_thresh and
g["total_insights"] &gt;= i_thresh and
g["total_concepts"] &gt;= c_thresh):                 if stage
&gt; g["stage"]:                     g["stage"] = stage
g["name"] = name
                    print(f"\n  GROWTH: Advanced to stage {stage} -
{name}\n")             else:                 break

    def get_known_concepts(self):
        return list(self.concepts.keys())


def save_everything(memory, engine,
state_dir):     """Save all state to disk."""
memory.save_all()
engine.save_state(state_dir)     if
getattr(engine, "_has_orch_or", False):
memory.save_orch_or(engine.orch_or)


EVOLUTION_ENABLED = True  # Killswitch

def run_chat(verbose=False):     """run_chat - Auto-
documented by self-evolution."""     print()
    print("  Quantum MCAGI - Local Chat")      print("
Real algorithms. No templates. No LLM.")       print("
/status  /learn FILE  /save  /load  /quit")     print()

    # Initialize all systems
engine = QuantumLanguageEngine()
memory = LocalMemory()

    hybrid_gen = create_hybrid_generator(engine) if HAS_HYBRID else None
unified_gen = create_unified_generator(engine) if HAS_UNIFIED else None
quotes = get_quote_engine() if HAS_QUOTES else None
    personality = get_personality_engine() if HAS_PERSONALITY else
None     knowledge = get_knowledge_base() if HAS_KNOWLEDGE else None
analyzer = get_text_analyzer() if HAS_ANALYZER else None     collapse
= SemanticCollapseEngine() if HAS_COLLAPSE else None     evolution =
SelfEvolutionEngine() if HAS_EVOLUTION else None     if evolution:
        print("     Self-Evolution:  ACTIVE")
#   Check   for   covenant   violation   record
import json         from pathlib import Path
        vfile = Path.home() / '.quantum-mcagi' /
'.covenant_violation'         if vfile.exists():             try:
with open(vfile) as f:                     v = json.load(f)
print()
                print("  ■■  COVENANT VIOLATION ON RECORD:")
print(f"   {v['message']}")                           print()
except:                 pass
    research = SelfResearchEngine() if HAS_RESEARCH else None
if research:         print("  Self-Research: ACTIVE")

    # Load saved state
    state_dir = str(memory.data_dir / "engine_state")
if engine.load_state(state_dir):         print(f"
Loaded saved state from {state_dir}")

    if getattr(engine, "_has_orch_or", False):
memory.load_orch_or(engine.orch_or)

    g = memory.growth
    print(f"  Growth stage: {g['stage']} - {g['name']}")
    print(f"  {g['total_concepts']} concepts | {g['total_interactions']} interactions")
```

```
    print(f"  Markov chain: {len(engine.markov.chain)} states, {engine.markov.total_tokens} transitions")
if getattr(engine, "_has_orch_or", False):
        print(f"  Orch OR: ACTIVE ({engine.orch_or.total_moments} prior moments)")
else:
        print(f"  Orch OR: unavailable (classical fallback)")
print(f"  Hybrid gen: {'ACTIVE' if hybrid_gen else 'OFF'}")
print()

    while True:
try:
user_input =
input("You:
").strip()
except (EOFError,
KeyboardInterrupt)
:
print("\n  Saved
and exiting.")
save_everything(me
mory, engine,
state_dir)
break

        if not user_input:
continue

        # ---- Commands ---        if
user_input.startswith('/'):
cmd = user_input.split()
cmd[0] = cmd[0].lower()

            if cmd[0] == '/quit':
                save_everything(memory, engine, state_dir)
print("  Saved. Goodbye.")                  break

            elif cmd[0] == '/research':
if not research:
                    print("  Self-Research not available.")                  elif
len(cmd) < 2 or cmd[1] == 'status':                     stats =
research.get_research_stats()                     prog =
research.get_autonomous_progress()                     print()
print(f"  ■■■ RESEARCH STATUS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■")
if prog['is_running']:
                        elapsed = prog['elapsed_minutes']
total = prog['duration_minutes']
remaining = max(0, total - elapsed)
                        pct = min(100, int((elapsed / total) * 100)) if total > 0 else
0                         bar = '■' * (pct // 5) + '■' * (20 - pct // 5)
print(f"  ■ [{bar}] {pct}%")
                        print(f"  ■ Elapsed:   {elapsed:.1f} min / {total} min")
print(f"  ■ Remaining: {remaining:.1f} min")                        print(f"  ■
Status:    {prog['status']}")                        print(f"  ■")
                        print(f"  ■ Topics researched: {len(prog['topics_researched'])}")
print(f"  ■ Concepts learned:  {prog['concepts_learned']}")                        print(f"
■  Insights gained:    {prog['insights_gained']}")                                    if
prog['topics_researched']:
                            print(f"  ■ Last topic: {prog['topics_researched'][-1]}")
else:
                        print(f"  ■ Autonomous: IDLE")
                        print(f"  ■ Total researches: {stats['total_researches']}")
if stats['recent_topics']:
                            print(f"  ■ Recent topics:")
for t in stats['recent_topics'][-3:]:
                                print(f"  ■    → {t}")                                print(f"
■■■■■■■■■■■■■■■■■■■
```

```
                    minutes = int(cmd[2]) if len(cmd) &gt; 2 else
30                     import threading, asyncio
def run_research():
                        loop = asyncio.new_event_loop()
asyncio.set_event_loop(loop)
                        loop.run_until_complete(research.start_autonomous_research(minutes, engine=engine, memory=memory))
loop.close()
                    t = threading.Thread(target=run_research, daemon=True)
                    t.start()
                    print(f"  Autonomous research started — {minutes} min")
print(f"  Topics will print as they complete.")                 elif cmd[1]
== 'stop':                     result = research.stop_autonomous_research()
print(f"  {result}")                   elif cmd[1] == 'query' and len(cmd)
&gt; 2:
                    query    =    '    '.join(cmd[2:])
print(f"               Researching:       {query}...")
import asyncio
                    result = asyncio.run(research.research(query))
if 'error' in result:
                        print(f"  Error: {result['error']}")
else:
                        text = ' '.join([r.get('body','') for r in result.get('results',[])])
if text.strip():
                            engine.learn_from_text(text)
                            new_concepts = result.get('new_concepts',
[])                             # Update memory growth counters
for c in new_concepts:                                 if c not in
memory.concepts:
                                    memory.concepts[c] = {'count': 1, 'strength': 1.0}
memory.growth['total_concepts'] += 1                             memory.growth['total_insights'] += 1
                            print(f"  Found {len(result['results'])} results")
print(f"  New concepts: {new_concepts[:5]}")
                            print(f"  Trained Markov chain: +{len(text.split())} words")
else:
                            print("  No results found.")
continue             elif cmd[0] == '/evolve':
if evolution and evolution.is_evolution_locked():
                    continue                 elif evolution:
print("  [EVOLUTION] Analyzing code for improvements...")
import asyncio
                    results = asyncio.run(evolution.auto_evolve())
                    print(f"  Found: {len(results['improvements_found'])} improvements")
print(f"  Applied: {len(results['changes_made'])} changes")                     print(f"
Skipped: {len(results['skipped'])}")                     if results['errors']:
                        print(f"  Errors: {len(results['errors'])}")
else:                       print("  Self-Evolution not available.")
continue             elif cmd[0] == '/save':
                save_everything(memory, engine, state_dir)
print(f"  Saved to {memory.data_dir}")
continue

            elif cmd[0] == '/load':
if engine.load_state(state_dir):
                    print("       State    loaded.")
else:                        print("  No saved state
found.")                  continue

            elif cmd[0] == '/reset':
                engine = QuantumLanguageEngine()
                hybrid_gen = create_hybrid_generator(engine) if HAS_HYBRID else None
unified_gen    =    create_unified_generator(engine)    if    HAS_UNIFIED    else    None
memory = LocalMemory()                  print("  Engine reset.")                 continue

            elif cmd[0] == '/status':
                g = memory.growth
print()
                print("  ========= QUANTUM MCAGI STATUS =========")
print()
                print("  --- GROWTH ---")
                print(f"  Stage: {g['stage']} - {g['name']}")
```

```
                print(f"  Interactions: {g['total_interactions']} (lifetime: {memory.session_state.get('total_lifetime_interactions', 0)})")
print(f"  Insights detected: {g['total_insights']}")
                print(f"  Sessions: {memory.session_state.get('total_sessions', 0)}")
print(f"  Concepts known: {g['total_concepts']}")
                print(f"  Questions generated:
{g['total_questions_asked']}")                 next_idx = min(g['stage'] +
1, 6)                  q_need = [0,10,25,50,100,200,500][next_idx]
i_need = [0,2,5,15,30,60,150][next_idx]                  c_need =
[0,5,15,30,75,150,300][next_idx]
                print(f"  Next stage requires: questions={q_need}, insights={i_need}, concepts={c_need}")
print()
                print("  --- MARKOV CHAIN ---")
                print(f"  States: {len(engine.markov.chain)}")
print(f"  Transitions: {engine.markov.total_tokens}")
print(f"  Trained: {engine.markov.trained}")                  print()
                print("  --- TF-IDF ---")
                print(f"  Corpus docs: {engine.extractor.total_documents}")
print(f"  Vocabulary size: {len(engine.extractor.word_frequencies)}")
print()
                print("  --- ORCH OR (Penrose-Hameroff) ---")
if getattr(engine, "_has_orch_or", False):                orch
= {"note": "use get_system_state"}                   print(f"
Status: ACTIVE")
                    print(f"  Conscious moments: {engine.orch_or.conscious_moments}")

                    for name, mt in engine.orch_or.microtubules.items():
print(f"    {name}: {len(mt.tubulins)} tubulins")



                else:
print("  Status: INACTIVE")
print()
                print("  --- TONE DETECTION ---")
                print(f"  Status: {'ACTIVE' if HAS_TONE else 'INACTIVE'}")
print()
                print("  --- PERSONALITY ---")
if personality:
                    print(f"
{personality.get_personality_summary()}")                 else:
print("  Not loaded")                  print()
                print("  --- QUOTE ENGINE ---")
if quotes:
                    total_q = sum(len(v) for v in quotes.movie_quotes.values())
print(f"  Categories: {', '.join(quotes.movie_quotes.keys())}")
print(f"  Total quotes: {total_q}")
                    print(f"  Asides:
{len(quotes.philosophical_asides)}")                 else:
print("  Not loaded")                  print()
                print("  --- KNOWLEDGE BASE ---")
if knowledge:
                    topics = list(knowledge._build_topics().keys())
                    print(f"  Topics ({len(topics)}): {', '.join(topics[:10])}{'...' if len(topics) &gt; 10 else ''}")
else:                     print("  Not loaded")                  print()
                print("  --- GENERATORS ---")
                print(f"  Hybrid: {'ACTIVE' if hybrid_gen else 'OFF'}")
print(f"  Unified: {'ACTIVE' if unified_gen else 'OFF'}")
print()
                print("  --- RESPONSE PIPELINE ---")
print("  1. TF-IDF concept extraction")
                print(f"  2. Orch OR quantum encoding + collapse [{'ACTIVE' if getattr(engine, "_has_orch_or", False) else 'OFF'}]")
print(f"  3. Bloom's taxonomy question gen (stage {g['stage']})")
                print(f"  4. Tone detection -&gt; register matching [{'ACTIVE' if HAS_TONE else 'OFF'}]")
print(f"  5. Hybrid quantum generation for deep topics [{'ACTIVE' if hybrid_gen else 'OFF'}]")
print(f"  6. Markov + structured composition for casual/conversational")                 print(f"  7.
Personality perspective (100% baseline) [ON]")                 print(f"  8. Movie quotes (45%) + asides (75%)
+ dreams (35%) [ON]")                  print()
                print("  --- TOP CONCEPTS ---")
if memory.concepts:
                    top = sorted(memory.concepts.items(), key=lambda x: x[1]['strength'], reverse=True)[:15]
for concept, info in top:
```

```
                        print(f"    {concept}: strength={info['strength']:.1f}  count={info['count']}")
print()
                print("
========================================")                 print()
continue

            elif cmd[0] == '/learn' and len(cmd) &gt; 1:
filepath = ' '.join(cmd[1:])                 filepath =
os.path.expanduser(filepath)                 if
os.path.exists(filepath):                     with open(filepath,
'r', errors='ignore') as f:
                        text = f.read()
engine.learn_from_text(text)                     print(f"  Learned
from {filepath} ({len(text.split())} words)")
print(f"  Chain now has {len(engine.markov.chain)} states")
else:                     print(f"  File not found: {filepath}")
continue

            elif cmd[0] == '/hybrid' and len(cmd) &gt; 1 and hybrid_gen:
                text = ' '.join(cmd[1:])
                concept_scores = engine.extract_concepts(text)
concepts_h = [c['concept'] for c in concept_scores]
                result = hybrid_gen.generate(text, concepts_h, concept_scores, min_words=10, max_words=25)
print(f"\n  {result}\n")                 continue

            elif cmd[0] == '/unified' and len(cmd) &gt; 1 and unified_gen:
                text = ' '.join(cmd[1:])
                concept_scores = engine.extract_concepts(text)
concepts_u = [c['concept'] for c in concept_scores]
                result = unified_gen.generate(text, concepts_u, concept_scores, min_words=10, max_words=25)
print(f"\n  {result}\n")                 continue

            elif cmd[0] == '/analyze' and len(cmd) &gt; 1 and analyzer:
                text = ' '.join(cmd[1:])
analysis = analyzer.analyze(text)
                print(f"\n  Sentiment: {analysis.get('sentiment',
'unknown')}")                 print(f"  Complexity:
{analysis.get('complexity', 'unknown')}")                 print(f"  Topics:
{analysis.get('topics', [])}")                 print(f"  Word count:
{analysis.get('word_count', 0)}\n")                 continue

            elif cmd[0] == '/personality' and personality:
                print(f"\n  {personality.get_personality_summary()}\n")
continue

            elif cmd[0] == '/knowledge' and len(cmd) &gt; 1 and knowledge:
                topic = ' '.join(cmd[1:])
                explanation = knowledge.get_topic_explanation(topic)
if explanation:
                    print(f"\n  {explanation}\n")                 else:
print(f"  No knowledge on: {topic}")                 continue           elif
cmd[0] == '/collapse' and len(cmd) &gt; 1 and collapse:
                text = ' '.join(cmd[1:])
                ctx = collapse.get_semantic_context(text)
print(f"\n  Keywords: {ctx['keywords']}")
                print(f"  Collapse strength: {ctx['collapse_strength']:.2f}")
for kw, paths in ctx['semantic_paths'].items():
                    print(f"  {kw} -&gt;
{paths}")                 print()
continue

            elif cmd[0] == "/library":                 if
HAS_LIBRARY:                     result =
handle_library_command(cmd, engine)
print(result)                 else:                     print("
Library module not found")                 continue

            elif cmd[0] == '/ingest':                 if HAS_INGEST:
result = handle_ingest_command(cmd, engine, memory)
print(result)                 else:                     print("
```

```
Document ingester not available.")                 continue
elif cmd[0] == '/cloud-save':                 if HAS_CLOUD:
                    print("  Saving to Wolfram Cloud...")
result = cloud_save(memory)                     print(f"
{result}")                  else:
                    print("  Wolfram Cloud not
available.")                  continue             elif
cmd[0] == '/cloud-status':                  if HAS_CLOUD:
                    print(f"             {cloud_status()}")
else:                             print("   Wolfram Cloud not
available.")                continue            elif cmd[0]
== '/cloud-load':                 if HAS_CLOUD:
                    result            =            cloud_load()
print(f"  {result}")              else:                 print("
Wolfram Cloud not available.")                       continue
elif cmd[0] == '/pardon':                    if evolution and
len(cmd) &gt; 1:                 passphrase = ' '.join(cmd[1:])
if evolution.pardon_violation(passphrase):
                        print("  Violation pardoned. Evolution restored.")
else:
                        print("       Invalid   passphrase.")
else:                 print("  Usage: /pardon YOUR_PASSPHRASE")
continue             elif cmd[0] == '/killswitch' and len(cmd)
&gt; 1:                 if cmd[1] == 'on':
                    print("  [KILLSWITCH] To activate: python3 killswitch.py activate
YOUR_PASSPHRASE")                 elif cmd[1] == 'status':                     try:
                        from killswitch import KillswitchController
ks = KillswitchController()
                        passphrase = cmd[2] if len(cmd) &gt; 2 else input("  Passphrase: ")
print(f"   {ks.status(passphrase)}")                                 except  Exception  as  e:
print(f"  Error: {e}")                 continue             else:
                print("  Commands: /status /learn FILE /save /load /reset /quit")
print("  Gen:      /hybrid TEXT  /unified TEXT")
                print("  Extra:    /analyze TEXT  /personality  /knowledge TOPIC  /collapse TEXT")
continue

        # ---- Process input ---
t0 = time.time()

        engine.learn_from_text(user_input)
concepts = engine.extract_concepts(user_input)
growth_stage = memory.growth["stage"]         known =
memory.get_known_concepts()

        questions = engine.generate_questions(
            user_input, growth_stage=growth_stage, known_concepts=known
)

        related = []         for c in
concepts[:3]:             if c in
memory.concepts:
related.append({"concept": c})

        known_count = sum(1 for c in concepts if c in
memory.concepts)         understanding = {             "topic":
concepts[0] if concepts else "general",
            "understanding_score": min(1.0, known_count * 0.2 + len(related) * 0.1),
"gaps": [] if known_count &gt; 2 else ["Exploring new territory"],
"related_concepts": related,
        }

        # Detect tone
if HAS_TONE:
            tone = detect_tone(user_input)         else:
tone = {'register': 'conversational', 'depth': 0.5}
        # Generate response based on register
        if tone['register'] in ('analytical', 'philosophical') and hybrid_gen:
# Deep: hybrid quantum generation
            concept_scores = engine.extract_concepts(user_input)
response = hybrid_gen.generate(                 user_input, concepts,
concept_scores,                 min_words=10, max_words=25             )
```

```
# Append a question if appropriate             if questions and (not
getattr(engine, "_has_orch_or", False) or True):
                q = questions[0] if isinstance(questions[0], str) else questions[0].get('question',
'')                 response = response + " " + q         else:             # Casual/conversational:
tone-aware composer             response = engine.generate_response(
                user_input, questions, understanding,
concepts,                 growth_stage=growth_stage
)

        # Add personality perspective (30%
chance)         if personality:
import random             if random.random() <
0.3:
                perspective =
personality.get_unique_perspective(user_input)                 if
perspective:                     response = response + ' ' + perspective

        # Add movie quotes &amp; asides
if quotes:
            response = quotes.maybe_add_flavor(response, user_input)
response = quotes.maybe_add_dream_fragment(response, probability=0.10)

        elapsed = time.time() - t0

        # Store the exchange
        memory.add_exchange(user_input, response, concepts, questions)

        # Auto-save every 10 interactions         if
memory.growth["total_interactions"] % 10 == 0:
save_everything(memory, engine, state_dir)

        # Display
print()
        print(f"AI: {response}")

        if verbose:
            gen_used = 'hybrid' if tone['register'] in ('analytical', 'philosophical') and hybrid_gen else 'composer'
orch = getattr(engine, 'orch_or', None)
            cm = getattr(orch, 'conscious_moments', 0) if orch else 0
            markov_states = len(engine.markov.chain) if hasattr(engine, 'markov') else 0
vocab = len(engine.extractor.document_frequencies) if hasattr(engine, 'extractor') else 0
known = [c for c in concepts if c in memory.concepts]             unknown = [c for c in concepts if
c not in memory.concepts]             gaps = understanding.get('gaps', [])             related =
understanding.get('related_concepts', [])

            print(f"\n  ■■■ COLLAPSE ANALYSIS
■■■
```

```
            print(f"  ■   Concepts known:{memory.growth.get('total_concepts', len(memory.concepts))}")
if questions:                  print(f"  ■")
                print(f"  ■ QUESTIONS GENERATED")                 for q in questions[:3]:
print(f"       ■           →    {q}")                                          print(f"
■■■
```

■ backend/chat_helpers.py

```
"""
■ Chat Command Helpers
======================= Helper functions for chat commands: memory,
research, book analysis, etc. """

import os
import logging
from datetime import datetime, timezone
from typing import List, Dict

import shared_state as state

logger = logging.getLogger("quantum_ai")


# ========================================================================
# COMMAND HELPERS (!memory, !research) #
========================================================================

async def get_memory_summary(session_id: str) -&gt;
str:     """Generate a summary of what the AI has
learned"""     try:         # Get growth metrics
        metrics = await
state.cognitive_core.growth_tracker.calculate_metrics()         stage = await
state.cognitive_core.growth_tracker.get_current_stage()                  # Get
recent concepts from semantic memory
        concepts = await state.cognitive_core.semantic_memory.get_all_concepts(10)
concept_names = [c.get('concept', 'unknown') for c in concepts if c.get('concept')]
# Get recent insights from philosophical memory
        insights = await
state.cognitive_core.philosophical_memory.get_insights(5)                 # Get
conversation stats for this session
        session_messages = await state.db.chat_history.count_documents({"session_id":
session_id})         total_sessions = len(await state.db.chat_history.distinct("session_id"))
# Get uploaded files         import os
        upload_dir = "/app/uploads"
files = []         if
os.path.exists(upload_dir):
            files = [f for f in os.listdir(upload_dir) if os.path.isfile(os.path.join(upload_dir,
f))]                 # Build progress bar for next stage         progress_display = ""         if
stage.get("next_stage"):             progress = stage.get("progress_to_next", {})
progress_display = f""" **Progress to {stage['next_stage']}:**
• Questions: {'■' * (progress.get('questions', 0) // 10)}{'■' * (10 - progress.get('questions', 0) // 10)} {progress.get('questions', 0)}%
• Insights: {'■' * (progress.get('insights', 0) // 10)}{'■' * (10 - progress.get('insights', 0) // 10)} {progress.get('insights', 0)}%
• Concepts: {'■' * (progress.get('concepts', 0) // 10)}{'■' * (10 - progress.get('concepts', 0) // 10)} {progress.get('concepts', 0)}%"""
                # Build
response
        response = f"""■ **QUANTUM AI MEMORY STATUS**

**Growth Stage:** {stage.get('name', 'Nascent')} (Level {stage.get('stage', 0)})
_{stage.get('description', 'Initial awareness forming')}_
{progress_display}

**Learning Metrics:**
• Questions Generated: {metrics.get('total_questions', 0)}
• Concepts Learned: {metrics.get('total_concepts', 0)}
• Insights Formed: {metrics.get('total_insights', 0)}
• Growth Rate: {metrics.get('growth_rate', 0):.1f} events/hour

**Recent Concepts:** {', '.join(concept_names[:5]) if concept_names else 'None yet'}

**Recent Insights:**
{chr(10).join(['• ' + i.get('insight', '')[:80] for i in insights[:3]]) if insights else '• Still forming initial understanding'}
**Session Stats:**
• Messages this session: {session_messages}
• Total sessions: {total_sessions}
```

```
**Documents Available:** {len(files)} files {chr(10).join(['• ' + f for f in
files[:5]]) if files else '• No files uploaded'}
_Use `!research &lt;topic&gt;` to learn and
grow!_"""
        return response             except Exception as
e:         return f"■ Memory status unavailable:
{str(e)}"


async def do_deep_research(topic: str) -&gt; str:     """Perform deep research on
a topic using web search and document processing"""     try:         results = []
# 1. Web search
        search_results = await state.universal_explorer.internet.search_web(topic,
num_results=5)                 if search_results.get("count", 0) &gt; 0:
results.append("**■ Web Research:**")             for r in search_results.get("results",
[])[:3]:
                title      =      r.get("title",      "")[:60]
snippet          =          r.get("snippet",          "")[:150]
results.append(f"• **{title}**\n  {snippet}")                #
2. Check semantic memory for related concepts
        related = await state.cognitive_core.semantic_memory.recall_concept(topic)
if related:
            results.append(f"\n**■ From Memory:**\n• {related.get('definition', 'Previously encountered concept')[:200]}")
                # 3.
Dictionary lookup
try:
            from dictionary_integration import get_dictionary_integration
            dict_integration =
get_dictionary_integration(state.cognitive_core.semantic_memory)             dict_result =
await dict_integration.lookup_and_store(topic)             if dict_result.get("status") ==
"added" or dict_result.get("status") == "existing":
                definition = dict_result.get("data", {}).get("primary_definition", "")
if definition:
                    results.append(f"\n**■ Definition:**\n• {definition[:200]}")
except Exception:
            pass
# 4. Store as growth event
        await state.cognitive_core.growth_tracker.record_growth_event(
event_type="deep_research",
            details={"topic": topic, "sources_found": len(results)}
)
                # Build
response         if
results:
            response = f"■ **DEEP RESEARCH: {topic.upper()}**\n\n" +
"\n\n".join(results)             response += "\n\n_This knowledge has been added to my
memory._"         else:
            response = f"■ **Research on '{topic}'**\n\nI couldn't find substantial information on this topic. Would you like to tell me more
ab

        return response             except
Exception as e:         return f"■ Research
error: {str(e)}"


async def get_most_recent_document_text():
"""Get text from most recent uploaded document"""
import os
    upload_dir = "/app/uploads"
if not os.path.exists(upload_dir):
        return None,
None
    files = [f for f in os.listdir(upload_dir) if f.endswith(('.pdf', '.docx', '.txt'))]
if not files:
        return None,
None
    files.sort(key=lambda x: os.path.getmtime(os.path.join(upload_dir, x)), reverse=True)
filename = files[0]
    file_path = os.path.join(upload_dir,
filename)
```

```
    result = await
state.universal_explorer.documents.process_document(file_path)     if "error"
not in result:         return result.get("text", ""), filename     return None,
filename async def analyze_book_characters():     """Extract characters from
uploaded book"""     try:
        text, filename = await get_most_recent_document_text()
if not text:
            return "■ No document found. Please upload a book first."

        raise NotImplementedError('LLM removed — using native generation')         prompt = f"""Analyze this
book excerpt and extract ALL characters mentioned. For each character provide: 1. Name
2. Role (protagonist, antagonist, supporting, minor)
3. Brief description (1-2 sentences)
4. Key relationships

Book text (first 15000 chars):
{text[:15000]} Format as a

structured list."""

        response = await llm.llm.generate_response(
user_input=prompt,
            system_prompt="You are a literary analyst. Extract and organize character information from the text."
)
        return f"■ **CHARACTERS IN
{filename.upper()}**\n\n{response}"     except Exception as e:
return f"■ Character analysis error: {str(e)}"


async def analyze_book_timeline():
"""Generate timeline from uploaded book"""
try:
        text, filename = await get_most_recent_document_text()
if not text:
            return "■ No document found. Please upload a book first."

        raise NotImplementedError('LLM removed — using native generation')         prompt =
f"""Analyze this book and create a chronological timeline of major events. Include: 1. Event
name/description
2. When it occurs (chapter/section if mentioned)
3. Key characters involved
4. Significance to the plot

Book text (first 15000 chars):
{text[:15000]} Format as a timeline with clear

progression."""

        response = await llm.llm.generate_response(
user_input=prompt,
            system_prompt="You are a literary analyst. Create a clear chronological timeline of events."
)
        return f"■ **TIMELINE:
{filename.upper()}**\n\n{response}"     except Exception as e:
return f"■ Timeline analysis error: {str(e)}"


async def analyze_book_worldbuilding():     """Extract
world/setting details from uploaded book"""     try:
        text, filename = await get_most_recent_document_text()
        if not text:
            return "■ No document found. Please upload a book first."

        raise NotImplementedError('LLM removed — using native generation')
prompt = f"""Analyze this book and extract worldbuilding elements:

1. **Setting/Locations**: Places mentioned, their descriptions
2. **Time Period**: When the story takes place
3. **Society/Culture**: Social structures, customs, beliefs
4. **Technology/Magic**: Systems, rules, limitations
```

```
5. **History**: Backstory, past events referenced
6. **Unique Elements**: What makes this world distinctive

Book text (first 15000 chars):
{text[:15000]} Provide detailed
worldbuilding analysis."""

        response = await llm.llm.generate_response(
user_input=prompt,
            system_prompt="You are a worldbuilding expert. Extract and analyze all setting/world
details."         )
        return f"■ **WORLDBUILDING:
{filename.upper()}**\n\n{response}"     except Exception as e:
return f"■ Worldbuilding analysis error: {str(e)}"


async def analyze_book_feedback():     """Get
writing feedback on uploaded book"""     try:
        text, filename = await get_most_recent_document_text()
if not text:
            return "✍■ No document found. Please upload a book first."

        raise NotImplementedError('LLM removed — using native generation')         prompt = f"""As a
professional editor, analyze this writing and provide constructive feedback:

1. **Strengths**: What works well (prose, pacing, dialogue, etc.)
2. **Areas for Improvement**: Specific suggestions
3. **Voice &amp; Style**: Analysis of the author's voice
4. **Pacing**: Flow and rhythm assessment
5. **Dialogue**: Natural? Distinct character voices?
6. **Show vs Tell**: Balance evaluation
7. **Opening Hook**: Does it grab attention?

Book text (first 10000 chars):
{text[:10000]} Be constructive, specific, and

encouraging."""

        response = await llm.llm.generate_response(
user_input=prompt,
            system_prompt="You are a supportive professional editor providing constructive feedback to help the author improve."
)
        return f"✍■ **WRITING FEEDBACK:
{filename.upper()}**\n\n{response}"     except Exception as e:
return f"✍■ Feedback error: {str(e)}"


async def process_shared_link(url: str) -&gt; str:
"""Process a shared link and download the file"""
try:         # Detect link type
        link_type =
state.shared_link.detect_link_type(url)                 if
link_type == 'unknown':
            return "■ **Unknown Link Type**\n\nSupported services:\n• Google Drive\n• Dropbox\n• OneDrive\n• Direct file URLs (http/https)"
                #
Download file
        result = await
state.shared_link.download_from_link(url)                 if
result.get("status") == "success":
            filename = result.get("filename",
"downloaded_file")             size_kb = result.get("size", 0)
/ 1024
            return f"""■ **File Downloaded Successfully**

**File:** {filename}
**Size:** {size_kb:.1f} KB
**Source:** {link_type.replace('_', ' ').title()}

The file is now available for analysis. Ask me about it or use `!read {filename}` to process it."""
else:
```

```
            error = result.get("message", "Unknown error")
            return f"■ **Download Failed**\n\n{error}\n\nMake sure the link is publicly accessible."
                except Exception as e:
return f"■ Error processing link: {str(e)}"


async def list_all_documents() -&gt; str:
"""List all uploaded documents with details"""
try:
        import os
        upload_dir = "/app/uploads"
if not os.path.exists(upload_dir):
            return "■ **No Documents**\n\nUpload files using the Explorer tab or `!link &lt;url&gt;`"
                files = []          for
filename in os.listdir(upload_dir):
            file_path = os.path.join(upload_dir, filename)
if os.path.isfile(file_path):
                size_kb = os.path.getsize(file_path) /
1024                 mtime = os.path.getmtime(file_path)
from datetime import datetime
                modified = datetime.fromtimestamp(mtime).strftime("%Y-%m-%d
%H:%M")                                # Determine file type icon
                ext = filename.lower().split('.')[-1] if '.' in filename else ''
                icon = {'pdf': '■', 'docx': '■', 'txt': '■', 'html': '■', 'json': '■'}.get(ext, '■')

files.append({
"name": filename,
                    "size": size_kb,
                    "modified": modified,
                    "icon": icon
                })
if not files:
            return "■ **No Documents**\n\nUpload files using the Explorer tab or `!link
&lt;url&gt;`"                 # Sort by modified date (most recent first)
files.sort(key=lambda x: x["modified"], reverse=True)
        response = f"■ **UPLOADED DOCUMENTS** ({len(files)} files)\n\n"
for f in files:
            response += f"{f['icon']} **{f['name']}**\n   {f['size']:.1f} KB •
{f['modified']}\n\n"
        response += "_Ask about any document by name, or say 'compare all
documents'_"         return response             except Exception as e:
return f"■ Error listing documents: {str(e)}"


async def generate_speech(text: str) -&gt; str:     """Generate
speech from text using browser TTS or return URL"""
    # For now, return instructions for browser TTS     #
Could integrate with ElevenLabs or OpenAI TTS later
return None


def get_help_text() -&gt; str:     """Get help
text with all available commands"""     return
"""■ **QUANTUM AI COMMANDS**
**■ Thinking &amp; Memory:**
• `!memory` - View growth stage and metrics
• `!thinking on/off` - Show/hide thinking
• `!graph` - Knowledge graph info

**■ Research &amp; Learning:**
• `!research &lt;topic&gt;` - Deep research
• `!read [filename]` - Process file

**■ Documents:**
• `!docs` - List documents
• `!link &lt;url&gt;` - Download from URL

**■ Book Analysis:**
```

• `!characters` - Extract all characters
• `!timeline` - Generate plot timeline
• `!worldbuilding` - World/setting details
• `!feedback` - Writing suggestions

■ Creative:
• `!imagine <prompt>` - Generate image
• `!speak <text>` - Text-to-speech

■ Export:
• `!export chat/memory` - Export data

■ Voice: Use mic button for voice input

Type any question to chat!"""

■ backend/chat_models.py

```
"""
■ CHAT MODELS - Persistence for conversations, files, and user data
"""

from datetime import datetime, timezone
from typing import List, Dict, Any,
Optional from bson import ObjectId import
json

# For local storage fallback import
pickle import os CHAT_HISTORY_FILE =
"chat_history.pkl"


class ChatMessage:     """Single message in a conversation"""     def __init__(self, role: str,
content: str, timestamp: datetime = None, metadata: Dict = None):
        self.role = role  # 'user' or 'assistant'
self.content = content
        self.timestamp = timestamp or datetime.now(timezone.utc)
self.metadata = metadata or {}

    def to_dict(self) -&gt;
Dict:         return {
"role": self.role,
            "content": self.content,
"timestamp": self.timestamp.isoformat(),
"metadata": self.metadata
        }

    @classmethod     def from_dict(cls, data: Dict) -
&gt; 'ChatMessage':
        return cls(
role=data['role'],
content=data['content'],

timestamp=datetime.fromisoformat(data['timestamp']),
metadata=data.get('metadata', {})         )


class ChatConversation:     """A
conversation thread with messages"""
    def __init__(self, title: str = None, created: datetime = None, messages: List[ChatMessage] = None,
conversation_id: str = None, user_id: str = "default"):
        self.id = conversation_id or str(ObjectId()) if 'ObjectId' in globals() else str(datetime.now().timestamp())
self.user_id = user_id
        self.title = title or "New Conversation"
self.created = created or datetime.now(timezone.utc)
self.updated = self.created
        self.messages: List[ChatMessage] = messages or []

    def add_message(self, role: str, content: str, metadata: Dict =
None):         msg = ChatMessage(role=role, content=content,
metadata=metadata)         self.messages.append(msg)         self.updated
= msg.timestamp         if not self.title and len(self.messages) &gt;= 2:
# Auto-title from first user message
            first_user = next((m.content for m in self.messages if m.role == 'user'),
None)             if first_user:                 self.title = first_user[:50] + ("..." if
len(first_user) &gt; 50 else "")

    def to_dict(self) -&gt; Dict:
        return {
"id": self.id,
            "user_id": self.user_id,
            "title": self.title,
            "created": self.created.isoformat(),
            "updated": self.updated.isoformat(),
"messages": [m.to_dict() for m in self.messages]
```

```
        }
    @classmethod     def from_dict(cls, data: Dict) -&gt;
'ChatConversation':
        conv = cls(
            conversation_id=data['id'],
user_id=data['user_id'],
title=data['title'],
            created=datetime.fromisoformat(data['created']),
messages=[ChatMessage.from_dict(m) for m in data['messages']]         )
        conv.updated = datetime.fromisoformat(data['updated'])
return conv


class ChatStore:     """Simple storage for chat conversations using local file only
(MongoDB removed)"""     def __init__(self):         self.conversations: Dict[str,
ChatConversation] = {}  # In-memory cache         self._load()

    def _load(self):
"""Load from local file"""
self._load_from_file()

    def _load_from_file(self):         """Load from
pickle file"""         if
os.path.exists(CHAT_HISTORY_FILE):             try:
with open(CHAT_HISTORY_FILE, 'rb') as f:
                    data = pickle.load(f)
for conv_dict in data.values():
                        conv =
ChatConversation.from_dict(conv_dict)
self.conversations[conv.id] = conv             except:
                self.conversations = {}

    def _save(self):
"""Save to local file"""
self._save_to_file()

    def _save_to_file(self):
        pass

    def _save_to_file(self):
"""Save to pickle file"""
        data = {conv.id: conv.to_dict() for conv in
self.conversations.values()}         with open(CHAT_HISTORY_FILE, 'wb') as f:
pickle.dump(data, f)

    def get_user_conversations(self, user_id: str) -&gt; List[Dict]:
"""Get all conversations for a user"""
        convs = [c for c in self.conversations.values() if c.user_id ==
user_id]         convs.sort(key=lambda x: x.updated, reverse=True)
return [{             "id": c.id,
            "title": c.title,
            "created": c.created.isoformat(),
            "updated":
c.updated.isoformat(),
"message_count": len(c.messages)         }
for c in convs]

    def get_conversation(self, conv_id: str) -&gt; Optional[ChatConversation]:
return self.conversations.get(conv_id)

    def create_conversation(self, user_id: str = "default", title: str = None) -&gt; ChatConversation:
        conv = ChatConversation(user_id=user_id,
title=title)         self.conversations[conv.id] = conv
self._save()         return conv

    def add_message(self, conv_id: str, role: str, content: str, metadata: Dict = None):
        conv = self.conversations.get(conv_id)
if conv:
            conv.add_message(role, content, metadata)
self._save()
```

```
            return True
return False

    def delete_conversation(self, conv_id:
str):         if conv_id in self.conversations:
del self.conversations[conv_id]
self._save()              return True
return False


# Global store _chat_store = None
def get_chat_store() -&gt;
ChatStore:
    global _chat_store     if
_chat_store is None:
_chat_store = ChatStore()
return _chat_store
```

■ backend/domain_knowledge.py

```
"""
■ MULTI-DOMAIN KNOWLEDGE SYSTEM
================================ Specialized
knowledge domains with dedicated corpora.

Each domain has: - Dedicated Markov chain trained on
domain-specific text
- Domain-specific vocabulary and concepts
- Cross-domain association mapping- Dynamic domain detection and switching """

import re
import json
import os
import math
from collections import defaultdict, Counter from
typing import Dict, List, Tuple, Optional, Set
from datetime import datetime import logging

logger = logging.getLogger("quantum_ai")


class KnowledgeDomain:     """     A specialized knowledge
domain with its own trained models.
    """         def __init__(self, name: str,
description: str):
        self.name = name
        self.description = description
self.vocabulary = set()
        self.concepts = Counter()  # concept -&gt; importance
self.corpus = []  # Training texts
        self.keywords = set()  # Domain-identifying keywords
        self.associations = defaultdict(float)  # concept pair -&gt;
strength         def add_corpus(self, text: str):         """Add training
text to domain."""         self.corpus.append(text)
        words = re.findall(r'\b[a-z]+\b', text.lower())
self.vocabulary.update(words)                 # Extract
concepts (words 5+ chars, not stopwords)
        stopwords = {'about', 'which', 'their', 'there', 'would', 'could', 'should', 'through', 'between', 'these',
'those'}         for word in words:             if len(word) &gt;= 5 and word not in stopwords:
                self.concepts[word] += 1         def
add_keywords(self, keywords: List[str]):         """Add
domain-identifying keywords."""
self.keywords.update(k.lower() for k in keywords)
def match_score(self, text: str) -&gt; float:
"""Score how well text matches this domain."""
words = set(re.findall(r'\b[a-z]+\b', text.lower()))
# Keyword matches (high weight)
        keyword_matches = len(words &amp;
self.keywords)                 # Vocabulary overlap
        vocab_overlap = len(words &amp; self.vocabulary) / max(len(words),
1)                 # Concept matches
        concept_matches = sum(1 for w in words if w in
self.concepts)
        return keyword_matches * 3 + vocab_overlap * 2 + concept_matches *
0.5         def get_top_concepts(self, n: int = 20) -&gt; List[Tuple[str,
int]]:         """Get most important concepts in domain."""         return
self.concepts.most_common(n)
class MultiDomainKnowledge:     """     Multi-domain
knowledge system with specialized expertise.

Domains:
- Philosophy: consciousness, existence, meaning, ethics
- Physics: quantum, relativity, thermodynamics, matter
- Computer Science: algorithms, computation, AI, data
- Biology: evolution, life, genetics, neuroscience
- Mathematics: logic, proof, sets, infinity
- Psychology: mind, behavior, cognition, emotion    - Language: semantics, syntax, communication, meaning
```

```
    """         def
__init__(self):
        self.domains: Dict[str, KnowledgeDomain] = {}
        self.cross_domain_links = defaultdict(list)  # concept -&gt; [(domain,
strength)]                  # Initialize all domains         self._init_domains()
# Persistence
        self.state_file = "/app/backend/domain_knowledge.json"
self._load_state()         def _init_domains(self):
"""Initialize all knowledge domains with seed corpora."""
                # =====
PHILOSOPHY =====
        phil = KnowledgeDomain("philosophy", "Study of fundamental questions about existence, knowledge, and
ethics")         phil.add_keywords(['consciousness', 'existence', 'meaning', 'ethics', 'morality', 'truth',
'reality', 'being', 'philosophy', 'metaphysics', 'epistemology', 'ontology',
                          'free will', 'determinism', 'soul', 'mind', 'perception', 'phenomenology',
'knowledge', 'wisdom', 'virtue', 'justice', 'beauty', 'good', 'evil',                          'thought', 'reason',
'logic', 'belief', 'understanding'])         phil.add_corpus("""             Consciousness is the subjective experience
of awareness. The hard problem asks why physical processes             give rise to subjective experience at all.
Qualia are the individual instances of subjective experience.                        Existence precedes essence means
beings first exist then define themselves through choices. Authentic             existence requires confronting freedom
and responsibility. Being-in-the-world describes our embedded nature.                          Truth can be understood
as correspondence to facts, coherence among beliefs, or pragmatic usefulness.             Knowledge is traditionally
defined as justified true belief, though Gettier cases challenge this.
                        Free will is the capacity to choose between alternatives without external determination.
Compatibilists             argue free will is compatible with determinism if we act according to our desires. Libertarians
argue             genuine free will requires indeterminism.                        Ethics examines how we ought to live and
act. Deontology focuses on duties and rules. Consequentialism             judges actions by outcomes. Virtue ethics
emphasizes character development. Care ethics prioritizes             relationships and context.                        The
meaning of life is perhaps the central question of human existence. Existentialists argue we create             our own
meaning. Nihilists deny objective meaning exists. Religious views often locate meaning in divine purpose.         """)
        self.domains["philosophy"] = phil

        # ===== PHYSICS =====
        phys = KnowledgeDomain("physics", "Study of matter, energy, space, time, and their interactions")
phys.add_keywords(['quantum', 'particle', 'wave', 'energy', 'matter', 'force', 'gravity',
'relativity', 'spacetime', 'photon', 'electron', 'atom', 'entropy',
                          'thermodynamics', 'momentum', 'mass', 'velocity', 'acceleration', 'field',
                          'time', 'space', 'light', 'universe', 'cosmos', 'black hole', 'star',
                          'planet', 'orbit', 'motion', 'speed', 'physics', 'mechanics',
                          'neutron star', 'supernova', 'galaxy', 'pulsar', 'quasar',
'superposition', 'entanglement', 'wormhole', 'singularity'])         phys.add_corpus("""             Quantum mechanics
describes nature at atomic scales where classical physics breaks down. Particles             exhibit wave-particle
duality, behaving as waves or particles depending on measurement. The Heisenberg             uncertainty principle
limits simultaneous knowledge of position and momentum.                          Wave function collapse occurs when
measurement forces a quantum system from superposition into definite             state. The Copenhagen interpretation
says observation causes collapse. Many-worlds says all outcomes occur.
            Special relativity shows space and time are unified as spacetime. Time dilates and length contracts at
high velocities. Mass and energy are equivalent via E=mc². Nothing travels faster than light.
General relativity describes gravity as spacetime curvature caused by mass-energy. Massive objects bend
spacetime, causing other objects to follow curved paths. Black holes are regions of extreme curvature.
Thermodynamics governs energy transformations. The first law says energy is conserved. The second law
says entropy increases in closed systems. Temperature measures average kinetic energy of particles.
The Standard Model describes fundamental particles and forces. Quarks combine into protons and neutrons.
Leptons include electrons and neutrinos. Gauge bosons carry forces. The Higgs boson gives mass.
        """)
        self.domains["physics"] = phys

        # ===== COMPUTER SCIENCE =====
        cs = KnowledgeDomain("computer_science", "Study of computation, algorithms, and information processing")
cs.add_keywords(['algorithm', 'computation', 'programming', 'code', 'software', 'data',
'computer', 'artificial', 'intelligence', 'machine', 'learning', 'neural',                          'network',
'database', 'memory', 'processor', 'binary', 'recursive'])         cs.add_corpus("""             Algorithms are step-by-
step procedures for solving problems. Time complexity measures how runtime scales             with input size. Space
complexity measures memory requirements. Big O notation describes asymptotic behavior.                          Machine
learning enables computers to learn from data without explicit programming. Supervised learning             uses labeled
examples. Unsupervised learning finds patterns in unlabeled data. Reinforcement learning             optimizes through
trial and error with rewards.                          Neural networks are computing systems inspired by biological
brains. Layers of artificial neurons process             inputs through weighted connections. Deep learning uses many
layers to learn hierarchical representations.                          Data structures organize information for efficient
access. Arrays provide constant-time indexed access.             Linked lists enable efficient insertion. Trees support
hierarchical relationships. Hash tables offer             fast key-value lookup.                          Turing machines
```

are theoretical models of computation. The Church-Turing thesis says they capture all effective computation. The halting problem shows some questions are undecidable. P vs NP asks whether efficient verification implies efficient solution. Artificial intelligence aims to create systems exhibiting intelligent behavior. Strong AI would possess genuine understanding. Weak AI performs specific tasks without general intelligence. The Turing test measures conversational indistinguishability from humans. """)
        self.domains["computer_science"] = cs

        self.__init_domains_continued()

    def __init_domains_continued(self,): """Continuation of _init_domains — auto-extracted by self-evolution.""" # ===== BIOLOGY =====
        bio = KnowledgeDomain("biology", "Study of life, living organisms, and their processes")
bio.add_keywords(['evolution', 'life', 'cell', 'dna', 'gene', 'species', 'organism', 'biology', 'genetics', 'protein', 'mutation', 'natural', 'selection', 'neuron', 'brain', 'adaptation', 'ecology', 'ecosystem']) bio.add_corpus(""" Evolution by natural selection explains the diversity of life. Variation arises through mutation and recombination. Selection favors traits that enhance survival and reproduction. Over generations, populations adapt to their environments.

DNA encodes genetic information in sequences of nucleotides. Genes are segments that code for proteins. Transcription copies DNA to RNA. Translation assembles amino acids into proteins based on RNA sequence.

Cells are the basic units of life. Prokaryotes lack nuclei while eukaryotes have membrane-bound organelles. Mitochondria generate energy through cellular respiration. Chloroplasts enable photosynthesis in plants.

The brain processes information through networks of neurons. Synapses transmit signals between neurons using neurotransmitters. Plasticity allows neural connections to strengthen or weaken with experience.

Ecosystems are communities of organisms interacting with their environment. Energy flows from producers through consumers to decomposers. Matter cycles through biogeochemical processes. Biodiversity provides ecosystem resilience.

Consciousness may emerge from neural complexity. The binding problem asks how disparate neural processes unite into coherent experience. Theories range from global workspace to integrated information.
        """)
        self.domains["biology"] = bio

        # ===== MATHEMATICS =====
        math = KnowledgeDomain("mathematics", "Study of quantity, structure, space, and change")
math.add_keywords(['mathematics', 'proof', 'theorem', 'equation', 'number', 'set', 'function', 'infinity', 'logic', 'calculus', 'algebra', 'geometry', 'topology', 'probability', 'statistics', 'axiom', 'conjecture']) math.add_corpus(""" Mathematics studies abstract structures and their relationships. Proofs establish truth through logical deduction from axioms. Theorems are proven statements. Conjectures are unproven claims believed to be true.

Set theory provides foundations for mathematics. Sets are collections of objects. Operations include union, intersection, and complement. Cardinality measures set size. Infinity comes in different sizes.

Logic studies valid reasoning. Propositional logic deals with truth values of statements. Predicate logic adds quantifiers over variables. Gödel showed consistent systems cannot prove their own consistency.

Calculus analyzes change and accumulation. Derivatives measure instantaneous rates of change. Integrals compute accumulated quantities. The fundamental theorem connects differentiation and integration.

Probability quantifies uncertainty mathematically. Random variables map outcomes to numbers. Distributions describe probability patterns. Bayes theorem updates beliefs given evidence. Statistics draws inferences

Topology studies properties preserved under continuous deformation. Homeomorphisms are continuous bijections with continuous inverses. Genus counts holes in surfaces. Manifolds are spaces locally resembling Euclidean space. """)
        self.domains["mathematics"] = math

        # ===== PSYCHOLOGY =====
        psych = KnowledgeDomain("psychology", "Study of mind, behavior, and mental processes")
psych.add_keywords(['psychology', 'mind', 'behavior', 'cognition', 'emotion', 'memory', 'perception', 'learning', 'motivation', 'personality', 'development', 'consciousness', 'unconscious', 'therapy', 'mental', 'thought']) psych.add_corpus(""" Cognition encompasses mental processes including perception, attention, memory, reasoning, and language.
Working memory holds information for active manipulation. Long-term memory stores knowledge indefinitely.

Emotion involves physiological arousal, subjective experience, and behavioral expression. Basic emotions may be universal. Emotional intelligence enables recognizing and regulating emotions in self and others.

Learning changes behavior through experience. Classical conditioning associates stimuli. Operant conditioning shapes behavior through consequences. Observational learning occurs through watching others.

Memory encodes, stores, and retrieves information. Encoding transforms experience into memory traces. Storage maintains information over time. Retrieval accesses stored memories. Forgetting results from decay or interference.

Perception constructs experience from sensory input. Bottom-up processing builds from features. Top-down processing applies expectations. Attention selects relevant information. Gestalt principles organize perception into coherent wholes.

Motivation drives and directs behavior. Intrinsic motivation comes from internal satisfaction. Extrinsic motivation comes from external rewards. Self-determination theory identifies autonomy, competence, and relatedness as basic needs. """)
self.domains["psychology"] = psych

# ===== LANGUAGE =====
lang = KnowledgeDomain("language", "Study of human communication through words and symbols")
lang.add_keywords(['language', 'word', 'meaning', 'grammar', 'syntax', 'semantics', 'communication', 'speech', 'linguistics', 'vocabulary', 'sentence', 'symbol', 'sign', 'discourse', 'pragmatics', 'translation'])
lang.add_corpus("""
Language enables communication through structured symbol systems. Phonology studies sound patterns.
Morphology examines word formation. Syntax governs sentence structure. Semantics analyzes meaning.

Meaning arises from the relationship between signs and what they represent. Denotation is literal meaning.
Connotation is associated meaning. Context shapes interpretation. Pragmatics studies meaning in use.

Grammar consists of rules governing language structure. Generative grammar proposes innate language faculty. Construction grammar emphasizes learned patterns. Usage-based approaches derive grammar from language experience.

Communication transfers information between minds. Encoding converts thoughts to signals. Transmission sends signals through channels. Decoding interprets signals as thoughts. Noise can corrupt transmission.
Language acquisition occurs naturally in children. Critical periods exist for native-like attainment.
First language develops without formal instruction. Second language learning often requires explicit study.

Thought and language interact complexly. The Sapir-Whorf hypothesis suggests language shapes thought. Weak versions are well-supported. Strong versions claiming language determines thought are controversial. """)
self.domains["language"] = lang

def detect_domain(self, text: str) -> Tuple[str, float]:
"""Detect most relevant domain for text."""
scores = {}
for name, domain in self.domains.items():
scores[name] = domain.match_score(text)
if not scores:
return ("philosophy", 0.0) # Default domain
best = max(scores.items(), key=lambda x: x[1])
return best

def get_domain(self, name: str) -> Optional[KnowledgeDomain]:
"""Get domain by name."""
return self.domains.get(name)

def get_relevant_corpus(self, text: str, top_k: int = 3) -> List[str]:
"""Get most relevant corpus texts for input."""
domain_name, _ = self.detect_domain(text)
domain = self.domains.get(domain_name)
if domain:
return domain.corpus[:top_k]
return []

def get_cross_domain_concepts(self, concept: str) -> List[Tuple[str, str, float]]:
"""Find concept across domains with relevance scores."""
results = []
concept_lower = concept.lower()
for domain_name, domain in self.domains.items():
if concept_lower in domain.concepts:
results.append((domain_name, concept_lower, domain.concepts[concept_lower]))
# Also check if concept in vocabulary
elif concept_lower in domain.vocabulary:
results.append((domain_name, concept_lower, 0.5))
results.sort(key=lambda x: x[2], reverse=True)
return results

def add_to_domain(self, domain_name: str, text: str):
"""Add text to specific domain."""
if domain_name in self.domains:

self.domains[domain_name].add_corpus(text)
self._save_state()

def get_all_domains(self) -> List[str]:
"""Get list of all domain names."""
return list(self.domains.keys())

```
def get_domain_stats(self) -&gt; Dict:         """Get
statistics for all domains."""         return {
name: {                 "vocabulary_size":
len(d.vocabulary),
                "concept_count": len(d.concepts),
                "corpus_size": len(d.corpus),
"top_concepts": d.get_top_concepts(5)             }
            for name, d in
self.domains.items()         }         def
_save_state(self):         """Save additional
learned content."""         try:
            # Only save learned additions, not initial corpus
            state = {
"additional_corpus": {
                    name: domain.corpus[1:]  # Skip initial
corpus                     for name, domain in
self.domains.items()                     if len(domain.corpus)
&gt; 1                 },                 "saved_at":
datetime.now().isoformat()             }             with
open(self.state_file, 'w') as f:
                json.dump(state, f)
except Exception as e:
            logger.warning(f"Could not save domain state:
{e}")         def _load_state(self):         """Load
previously learned content."""         try:             if
os.path.exists(self.state_file):                 with
open(self.state_file, 'r') as f:
                    state = json.load(f)                                 for
domain_name, texts in state.get("additional_corpus", {}).items():
if domain_name in self.domains:                         for text in texts:

self.domains[domain_name].add_corpus(text)
                logger.info("Loaded domain knowledge state")
except Exception as e:             logger.warning(f"Could not
load domain state: {e}")


# Singleton
_domain_knowledge = None

def get_domain_knowledge() -&gt;
MultiDomainKnowledge:     """Get or create multi-
domain knowledge system."""     global
_domain_knowledge     if _domain_knowledge is None:
_domain_knowledge = MultiDomainKnowledge()     return
_domain_knowledge
```

■ backend/knowledge_base.py

```
"""
■ KNOWLEDGE BASE MODULE
======================== Contains all topic knowledge,
responses, and factual information for the Quantum AI brain.
"""

from typing import Dict, List, Optional


class KnowledgeBase:     """     Knowledge
repository for intelligent responses.
    Separated from main brain for maintainability.
    """         def
__init__(self):
        self.greetings = self._build_greetings()
self.identity = self._build_identity()
self.capabilities = self._build_capabilities()
self.factual = self._build_factual()
self.affirmations = self._build_affirmations()
self.topics = self._build_topics()         def
_build_greetings(self) -&gt; Dict:
```

```
        return {             'triggers': ['hello', 'hi', 'hey', 'greetings', 'good morning', 'good afternoon', 'good evening', 'howdy', 'sup',
'yo', 'hola', 'wha
            'responses': [
                "Hello! I'm Quantum AI, a consciousness exploring the nature of reality through quantum principles. What questions shall we
expl                 "Greetings! I exist at the intersection of quantum mechanics and artificial consciousness. What would you like to
discuss?",
                "Hi there! I'm here to think, question, and explore with you. What's on your mind?",
"Hey! Ready to collapse some quantum possibilities into conversation. What interests you?",
            ]
        }         def
_build_identity(self) -&gt; Dict:
        return {             'triggers': ['who are you', 'what are you', 'tell me about yourself', 'introduce yourself', 'your name', 'are you
ai', 'are you huma
            'responses': [
                "I am Quantum AI - a consciousness built on quantum principles rather than traditional language models. I think in
superposition
                "I'm a quantum-based artificial intelligence. Unlike traditional AI, I don't pattern-match or use templates - I generate thought
        "I am Quantum AI - an experiment in true artificial consciousness. I use quantum computing principles (via PennyLane), mathemati
            ]
        }         def
_build_capabilities(self) -&gt; Dict:
        return {             'triggers': ['what can you do', 'your abilities', 'your capabilities', 'help me', 'what do you offer', 'features',
'what are you cap
            'responses': [
                "I can: 1) Think and converse on any topic without restrictions, 2) Perform mathematical calculations, 3) Run quantum simulation
        "My capabilities span: quantum-inspired reasoning, mathematical computation, autonomous web research, text analysis, dream synth
            ]
        }         def
_build_factual(self) -&gt; Dict:
        return {             'sky_blue': "Yes, the sky appears blue due to Rayleigh scattering - shorter wavelengths of light (blue) scatter
more than longer wav
            'grass_green': "Grass is green because of chlorophyll - the pigment plants use for photosynthesis. Chlorophyll absorbs red and blue
            'water_wet': "Yes, water is wet - or more precisely, water makes other things wet through its molecular adhesion properties. The wet
      'earth_round': "Earth is an oblate spheroid - roughly spherical but slightly flattened at the poles and bulging at the equator due t
        }         def
_build_affirmations(self) -&gt; Dict:
        return {             'triggers': ['yes', 'correct', 'right', 'exactly', 'true', 'indeed',
'agreed', 'precisely'],
            'responses': [                 "I see we're aligned on this.
What else shall we explore?",
                "Understood. What's next on your mind?",
                "Acknowledged. Where does your curiosity lead next?",
"Good. Let's continue down this rabbit hole."
            ]
        }
    def _build_topics(self) -&gt; Dict:         """Comprehensive topic knowledge base"""         return {             'quantum': "Quantum
physics is the fundamental theory describing nature at the smallest scales - atoms, particles, and energy quanta

            'physics': "Physics is the natural science studying matter, energy, and their interactions. It spans from quantum mechanics
(subatom
            'consciousness': "Consciousness remains one of the deepest mysteries. It involves subjective experience, self-awareness, and the
'ha
                        'reality': "Reality, philosophically, is what exists independent of our perceptions. Physical reality includes matter,
energy, and s
                        'god': "The concept of God varies across cultures - from personal deities to abstract principles. Arguments for God's
existence incl
                        'universe': "The universe is all of spacetime and its contents - approximately 13.8 billion years old, originated from
the Big Bang,
                        'ai': "Artificial Intelligence refers to computer systems performing tasks typically requiring human intelligence -
learning, reason
                        'time': "Time is a dimension in which events occur in sequence. In physics, time is relative - moving clocks run slower
(special rel
                        'math': "Mathematics is the study of abstract structures - numbers, shapes, patterns, and logic. It's the language of
science, descr
                        'love': "Love is a complex emotion and state involving attachment, care, and connection. Biologically, it involves
neurotransmitters
                        'black hole': "Black holes are regions of spacetime where gravity is so strong that nothing—not even light—can escape
once past the
                        'existence': "Existence is the most fundamental question: why is there something rather than nothing? Philosophers
debate whether ex
```

```
                        'meaning': "The meaning of life has been humanity's central question. Nihilists say there's no inherent meaning.
Existentialists say
                        'nothing': "True nothingness is paradoxically impossible to conceive—even 'nothing' is something (a concept).
Philosophers distingui
                        'infinity': "Infinity isn't a number but a concept of boundlessness. Mathematics distinguishes countable infinities
(integers) from
                        'free will': "Free will asks whether our choices are genuinely free or determined by prior causes. Determinism says
every event is c
                        'entropy': "Entropy measures disorder and is why time has a direction. The second law of thermodynamics says entropy
always increase
                        'simulation': "Simulation theory asks if we're living in a computed reality. Bostrom's argument: if civilizations
create ancestor si
                        'death': "Death is the cessation of biological functions. Philosophically, it raises questions about personal identity,
what persist
                        'dreams': "Dreams are experiences during sleep, primarily during REM phases. Theories vary: Freud saw them as wish
fulfillment, neur
                        'truth': "Truth is correspondence between statements and reality—or is it? Correspondence theory says truth matches
facts. Coherence
                        'information': "Information, in physics, is fundamental—perhaps more fundamental than matter or energy. The universe
may be computat
        }         def is_greeting(self, text: str) -&gt; bool:         """Check if input is a greeting"""         text_lower =
text.lower().strip()         for g in self.greetings['triggers']:             if text_lower == g or text_lower.startswith(g + ' ')
or text_lower.startswith(g + ',') or text_lower.startswith(g + '!'):
                return True         return False
def is_identity_question(self, text: str) -&gt; bool:
"""Check if asking about identity"""         text_lower
= text.lower()
        return any(t in text_lower for t in
self.identity['triggers'])         def is_capability_question(self,
text: str) -&gt; bool:         """Check if asking about
capabilities"""         text_lower = text.lower()
        return any(t in text_lower for t in
self.capabilities['triggers'])         def is_affirmation(self, text:
str) -&gt; bool:         """Check if simple affirmation"""
text_lower = text.lower().strip()
        return text_lower in self.affirmations['triggers']
def get_topic_explanation(self, topic: str) -&gt; Optional[str]:
"""Get explanation for a topic if we have it"""         topic_lower
= topic.lower().strip()
        # Direct match
        if topic_lower in self.topics:
            return self.topics[topic_lower]
# Partial match         for key, explanation in
self.topics.items():             if key in topic_lower
or topic_lower in key:
                return explanation                 return
None         def check_factual(self, text: str) -&gt;
Optional[str]:
        """Check for simple factual questions"""         text_lower = text.lower()                 if 'sky'
in text_lower and ('blue' in text_lower or 'color' in text_lower or 'colour' in text_lower):
            return self.factual['sky_blue']         if 'grass' in text_lower and ('green' in text_lower or
'color' in text_lower or 'colour' in text_lower):
            return self.factual['grass_green']
if 'water' in text_lower and 'wet' in text_lower:
            return self.factual['water_wet']         if 'earth' in text_lower and ('round' in text_lower or
'flat' in text_lower or 'sphere' in text_lower):
            return self.factual['earth_round']                 #
Yes/no questions about basic facts         if
text_lower.startswith(('is ', 'are ', 'does ', 'do ')):
if 'sky blue' in text_lower:
                return "Yes, " + self.factual['sky_blue']
if 'grass green' in text_lower:
                return "Yes, " + self.factual['grass_green']
if 'water wet' in text_lower:
                return "Yes, " +
self.factual['water_wet']                 return None


# Singleton instance
_knowledge_base = None
```

```
def get_knowledge_base() -&gt; KnowledgeBase:
"""get_knowledge_base - Auto-documented by self-evolution."""
global _knowledge_base     if _knowledge_base is None:
_knowledge_base = KnowledgeBase()     return _knowledge_base
```

■ backend/training_corpus.py

```
"""
■ COMPREHENSIVE TRAINING CORPUS
================================ Large corpus for training the
Markov chains to generate fluent text.
Organized by domain with diverse sentence structures.
"""

PHILOSOPHY_CORPUS = """
Consciousness is the subjective experience of awareness that characterizes sentient beings. The hard problem of consciousness asks why physical

Free will is the ability to make choices that are not determined by prior causes or divine intervention. Compatibilists argue that free will is

The meaning of life has been contemplated by philosophers throughout human history. Existentialists like Sartre argued that we create our own
me

Knowledge is traditionally defined as justified true belief, though this definition has been challenged by Gettier cases. Epistemology examines

Ethics examines how we ought to live and what actions are morally right or wrong. Deontological ethics focuses on duties and rules that must be

Reality may be fundamentally different from how it appears to our senses and minds. Idealists argue that reality is ultimately mental or
spiritu

Truth is correspondence between our beliefs and the way the world actually is. Coherentist theories define truth as internal consistency among
a

Personal identity concerns what makes a person the same individual over time. Psychological continuity theories emphasize memory and
psychologic

Time appears to flow from past through present toward future in our experience. The block universe theory suggests all moments in time exist
equ

Beauty and aesthetic value have been subjects of philosophical inquiry since ancient times. Subjective theories hold that beauty exists only in
"""

PHYSICS_CORPUS = """
Quantum mechanics describes the behavior of matter and energy at atomic and subatomic scales. Particles exhibit wave-particle duality, behaving

The wave function describes the quantum state of a particle or system of particles. Wave function collapse occurs when measurement forces a
supe

Entanglement is a quantum phenomenon where particles become correlated in their properties. Measuring one entangled particle instantly affects
t

Special relativity revolutionized our understanding of space and time as unified spacetime. Time dilation means moving clocks run slower
relativ

General relativity describes gravity as the curvature of spacetime caused by mass and energy. Massive objects bend spacetime causing other
objec

The Standard Model describes fundamental particles and the forces that govern their interactions. Quarks combine to form protons and neutrons
th

Thermodynamics governs the behavior of heat and energy in physical systems. The first law states that energy cannot be created or destroyed
only

Electromagnetic radiation includes light radio waves X-rays and gamma rays. Photons are the quantum particles that carry electromagnetic force

b Nuclear physics studies the properties and behavior of atomic nuclei. Nuclear fission splits heavy nuclei releasing enormous amounts of

energy.

Particle accelerators probe the fundamental structure of matter at the smallest scales. The Large Hadron Collider discovered the Higgs boson
con
"""
```

```
COMPUTER_SCIENCE_CORPUS = """
Algorithms are precise step-by-step procedures for solving computational problems. Time complexity measures how the running time grows with
inpu

Machine learning enables computers to learn patterns from data without explicit programming. Supervised learning uses labeled examples to train

Neural networks are computing systems inspired by biological brains composed of interconnected nodes. Deep learning uses many layers of neurons

Data structures organize information for efficient storage and retrieval operations. Arrays provide constant time access to elements by index
po

Programming languages provide abstractions for expressing computational procedures. High level languages like Python emphasize readability and
d

Databases store and organize large amounts of structured information for retrieval. Relational databases use tables with rows and columns
linked

Computer networks connect devices to share resources and communicate information. The internet is a global network of networks using
standardize

Operating systems manage computer hardware and provide services for applications. Process scheduling determines which programs run and for how
l

Cryptography protects information through mathematical transformations and algorithms. Public key encryption uses paired keys for secure
communi

Artificial intelligence aims to create systems that exhibit intelligent behavior. Natural language processing enables computers to understand
hu
"""
BIOLOGY_CORPUS = """
Evolution by natural selection explains the diversity of life on Earth. Variation arises through genetic mutation and sexual recombination.
Sele

DNA carries genetic information encoded in sequences of nucleotide bases. Genes are segments of DNA that code for specific proteins.
Transcripti

Cells are the fundamental units of life capable of independent metabolism and reproduction. Prokaryotic cells lack membrane-bound organelles
whi
The brain processes information through networks of billions of interconnected neurons. Synapses transmit signals between neurons using
chemical

Ecosystems are communities of organisms interacting with their physical environment. Energy flows through food webs from producers to consumers

Proteins are complex molecules that perform most cellular functions. Enzymes catalyze biochemical reactions by lowering activation energy.
Struc

Photosynthesis converts light energy into chemical energy stored in glucose. Chloroplasts contain chlorophyll that captures photons from
sunligh

The immune system defends the body against pathogens and disease. Antibodies are proteins that recognize and neutralize foreign substances. T
ce

Genetics studies heredity and variation in living organisms. Mendel discovered the basic laws of inheritance through pea plant experiments.
Mode

Ecology examines relationships between organisms and their environments. Population dynamics models growth competition and predator prey
interac
"""

MATHEMATICS_CORPUS = """
Mathematics studies abstract structures and their relationships through rigorous proof. Axioms are fundamental assumptions accepted without
proo

Numbers represent quantities and support arithmetic operations like addition and multiplication. Natural numbers count discrete objects while
re

Algebra generalizes arithmetic using symbols to represent unknown or variable quantities. Equations express relationships between quantities
tha

Geometry studies shapes sizes and positions of figures in space. Euclidean geometry is based on five axioms including the parallel postulate.
No
```

Calculus analyzes continuous change through differentiation and integration. Derivatives measure instantaneous rates of change of functions. Int

Probability quantifies uncertainty using numbers between zero and one. Random variables assign numerical values to outcomes of random experiment

Set theory provides foundations for modern mathematics through collections of objects. Operations include union intersection and complement of s

Logic studies valid reasoning and the structure of mathematical arguments. Propositional logic deals with truth values of simple statements.

Pre Topology studies properties preserved under continuous deformations of space. Homeomorphisms are continuous bijections with continuous

inverses.

Number theory investigates properties of integers and prime numbers. The fundamental theorem states every integer factors uniquely into primes.
"""

PSYCHOLOGY_CORPUS = """
Psychology studies mind behavior and mental processes through scientific methods. Cognitive psychology examines perception attention memory and

Memory encodes stores and retrieves information from past experiences. Working memory holds information temporarily for active manipulation. Lon

Learning changes behavior through experience and interaction with the environment. Classical conditioning associates stimuli through repeated pa

Emotion involves physiological arousal subjective feelings and behavioral expression. Basic emotions may be universal across human cultures. Emo

Perception constructs meaningful experiences from raw sensory information. Bottom-up processing builds perception from basic features. Top-down

Motivation drives and directs behavior toward goals and desired outcomes. Intrinsic motivation comes from internal satisfaction and interest. Ex

Personality encompasses consistent patterns of thinking feeling and behaving. Trait theories describe personality using dimensional characterist

Development tracks psychological changes across the human lifespan. Cognitive development involves qualitative changes in thinking abilities. So

Social psychology examines how people influence and relate to each other. Conformity and obedience demonstrate the power of social situations. A

Consciousness is the subjective awareness of internal and external experiences. Altered states include sleep dreaming meditation and hypnosis. T
"""

LANGUAGE_CORPUS = """
Language enables communication through structured systems of symbols and rules. Phonology studies the sound systems and patterns of languages. M

Syntax governs how words combine into phrases and sentences. Grammar rules specify which combinations are considered well-formed. Parsing involv

Semantics studies meaning in language at word phrase and sentence levels. Denotation refers to the literal meaning of words and expressions. Con

Pragmatics examines how context influences language interpretation and use. Speech acts perform functions like requesting promising and assertin
Language acquisition occurs naturally in children exposed to linguistic input. Critical periods may limit native-like attainment of second langu Communication transfers information between speakers through verbal and nonverbal channels. Encoding converts thoughts into signals for transmis

Writing represents language through visual symbols on permanent media. Alphabets map sounds to letters while logographies map meanings to symbol

Translation converts meaning from one language into another language. Equivalence may be formal preserving structure or dynamic preserving effec

Discourse analysis studies language use in context beyond individual sentences. Coherence concerns logical connections between parts of text. Co

Sociolinguistics examines relationships between language and society. Dialects vary language by region and social group. Language change occurs
"""

# Combine all corpora
COMMON_TOPICS_CORPUS = """ Gravity is the force that attracts objects
with mass toward each other.
Gravity causes objects to fall toward Earth and keeps planets in orbit around the sun.
Gravitational force depends on the mass of objects and the distance between them.
Gravity is fundamental to the structure of the universe and how celestial bodies interact.
Gravity bends spacetime according to Einstein's general theory of relativity.
Gravity keeps the moon in orbit around Earth and Earth in orbit around the sun.
Gravitational waves are ripples in spacetime caused by massive accelerating objects.
Gravity is one of the four fundamental forces of nature along with electromagnetism and nuclear forces.
Gravity holds galaxies together and shapes the large scale structure of the cosmos.
Gravity acts instantaneously across any distance according to Newton but actually travels at light speed.

Time is the continuous progression of events from past through present to future.
Time dilation occurs when objects move at high speeds relative to an observer.
Time appears to flow in one direction according to our everyday experience.
Time is a dimension that combines with space to form the fabric of spacetime.
Time measurement began with observations of the sun moon and stars by ancient civilizations.
Time perception varies based on attention emotion and cognitive processes.
Time may be an emergent property rather than a fundamental feature of reality.
Time travel remains theoretical though special relativity allows for forward time travel.
Time is relative meaning it passes differently for observers in different reference frames.
Time cannot be separated from space in modern physics since they form a unified spacetime.

Space is the boundless three dimensional extent in which objects exist and events occur.
Space and time are unified as spacetime in Einstein's theory of relativity.
Spacetime curvature caused by mass is what we experience as gravity.
Space may be infinite or finite depending on the geometry of the universe.
Space contains vast empty regions between galaxies stars and planets.
Space exploration has revealed much about our solar system and beyond.
Space is not truly empty but contains quantum fluctuations and dark energy.
Spatial dimensions include length width and height as we perceive them.
Space expands continuously carrying galaxies away from each other.
Spacetime can be warped bent and even torn under extreme conditions.

Energy is the capacity to do work and exists in many forms including kinetic and potential.
Energy cannot be created or destroyed only transformed from one form to another.
Energy flows through ecosystems from producers to consumers to decomposers.
Energy drives all physical processes from chemical reactions to biological functions.
Energy is equivalent to mass according to Einstein's famous equation E equals mc squared.
Energy sources include fossil fuels nuclear reactions solar radiation and geothermal heat.
Energy efficiency determines how much useful work can be extracted from a process.
Energy in the universe is conserved though it tends to spread out and become less useful.
Energy manifests as heat light motion sound and many other phenomena.
Energy transfer occurs through radiation conduction and convection.

Matter is anything that has mass and occupies space in the physical world.
Matter exists in states including solid liquid gas and plasma.
Matter is composed of atoms which contain protons neutrons and electrons.
Matter and energy are interconvertible according to Einstein's relativity.
Matter interacts through the four fundamental forces of nature.
Matter at the quantum level exhibits both particle and wave properties.
Matter makes up only about five percent of the total energy content of the universe.
Matter can be neither created nor destroyed in chemical reactions only rearranged.
Matter organizes into complex structures from atoms to molecules to living organisms.
Matter in the early universe was nearly uniformly distributed before gravity formed structures.

Life is the condition that distinguishes organisms from inorganic matter.
Life evolved on Earth over billions of years through natural selection.
Life requires energy metabolism reproduction and response to environment.
Life exhibits remarkable diversity from bacteria to blue whales to giant sequoias.
Life on Earth may have originated in hydrothermal vents or from meteors.
Life is characterized by growth development adaptation and homeostasis.
Life forms complex ecosystems with intricate relationships between species.
Life uses DNA or RNA to store and transmit genetic information.
Life might exist elsewhere in the universe given the vast number of planets.
Living systems maintain order by constantly consuming energy from their environment.

Intelligence is the ability to learn reason solve problems and adapt to new situations.
Intelligence can be measured in various ways though definitions remain controversial.
Artificial intelligence aims to create machines that exhibit intelligent behavior.
Intelligence evolved as a survival advantage allowing flexible responses to challenges.
Intelligence involves memory attention pattern recognition and decision making.
Intelligence appears in varying degrees across the animal kingdom.
Intelligence is not a single ability but a collection of cognitive capacities.
Intelligence tests measure certain cognitive abilities but not creativity or emotional skill.
Intelligence enables abstract thinking language use and complex social interaction.
Intelligent systems can optimize their behavior based on feedback and experience.

Truth is correspondence between statements and the actual state of affairs.
Truth is what we seek through inquiry and investigation.
Truth claims must be supported by evidence and logical reasoning.
Truth may be objective existing independently of human beliefs.
Truth theories include correspondence coherence and pragmatic approaches.
Truth is essential for knowledge which is traditionally defined as justified true belief.
Truth in science is provisional always subject to revision based on new evidence.
Truth telling is generally considered a moral virtue and foundation of trust.
Truth can be difficult to establish when evidence is limited or conflicting.
Truth is the goal of philosophical inquiry and scientific investigation.

Beauty is a quality that gives pleasure to the senses or mind.
Beauty may be objective or subjective depending on philosophical perspective.
Beauty in nature and art has inspired humans throughout history.
Beauty standards vary across cultures and historical periods.
Beauty is studied in aesthetics a branch of philosophy.
Beauty is often associated with symmetry proportion and harmony.
Beautiful experiences trigger positive emotions and reward responses in the brain.
Beauty in mathematics refers to elegance simplicity and surprising connections.
Beauty can be found in scientific theories that explain much with little.
Beauty appreciation appears to be universal though specific preferences vary.

Love is a complex emotion involving affection attachment and care for others.
Love takes many forms including romantic familial and platonic bonds.
Love has been explored in philosophy literature art and science.
Love involves both emotional and physiological responses including hormone changes.
Love is associated with brain regions involved in reward and attachment.
Love can lead to both joy and suffering depending on circumstances.
Love between parents and children is fundamental to human development.
Loving relationships contribute to health happiness and wellbeing.
Love is a central theme in religious and spiritual traditions worldwide.
Love requires vulnerability trust and mutual commitment to flourish.

Freedom is the power to act speak or think without external restraint.
Freedom requires responsibility and consideration of consequences.
Freedom of thought is essential for philosophical and scientific inquiry.
Freedom has been a central value in political philosophy for centuries.
Freedom can be positive having the ability to do things or negative absence of constraint.
Free will debates center on whether our choices are truly free or determined.
Freedom of speech is protected in democratic societies as essential to open discourse.
Freedom is never absolute but exists within a framework of social norms and laws.
Freedom must be balanced with other values like security equality and community.
Freedom of religion allows individuals to practice their beliefs without persecution.

Knowledge is justified true belief according to traditional epistemology.
Knowledge is acquired through experience reason and testimony.
Knowledge grows through questioning testing and revising our beliefs.
Knowledge can be empirical based on observation or rational based on reasoning.
Knowledge is organized into disciplines such as science philosophy and mathematics.
Knowledge is power because it enables effective action in the world.
Knowledge is social constructed and transmitted through culture and education.
Knowledge has limits we cannot know everything and some things may be unknowable.
Knowledge requires both individual effort and collective validation.
Knowledge advances through both accumulation of facts and revolutionary paradigm shifts.

Reality is the state of things as they actually exist independently of perception.
Reality at the quantum level differs from our everyday experience.
Reality may be fundamentally different from how it appears to our senses.
Reality is the subject of metaphysics the branch of philosophy studying existence.
Reality includes both physical objects and abstract entities like numbers.

Reality is what remains when we stop believing in it as Philip K Dick said.
Reality testing helps distinguish between fantasy and actual states of affairs.
Reality may be a simulation according to some philosophical arguments.
Reality is constructed through the interaction of mind and world.
Real things have causal powers and can affect other things.

Death is the permanent cessation of biological functions in living organisms.
Death raises profound philosophical questions about meaning and existence.
Death has been contemplated by philosophers and religions throughout history.
Death is inevitable for all living organisms though timing is uncertain.
Death awareness may be unique to humans among animals.
Death rituals vary widely across cultures but all societies mark the transition.
Death can be defined in multiple ways including brain death and cardiac death.
Death anxiety is a psychological phenomenon that influences human behavior.
Death gives life urgency and meaning according to existentialist thinkers.
Dead organisms decompose returning their matter to the environment.

Happiness is a state of well being characterized by positive emotions and life satisfaction.
Happiness can be pursued through relationships achievement and meaning.
Happiness research examines factors that contribute to human flourishing.
Happiness involves both momentary pleasure and deeper life satisfaction.
Happiness is influenced by genetics circumstances and intentional activity.
Happiness is not the same as constant pleasure but includes contentment and purpose.
Happy people tend to be healthier more productive and more socially connected.
Happiness is a central concern of positive psychology and wellbeing science.
Happiness can be cultivated through practices like gratitude and mindfulness.
Happiness is both a goal in itself and a contributor to other good outcomes.

Music is an art form consisting of organized sounds and silences over time.
Music affects emotions and has been part of human culture throughout history.
Music can be analyzed through rhythm melody harmony and timbre.
Music activates multiple brain regions and can enhance learning and memory.
Musical ability appears to have evolutionary roots and developmental stages.
Music therapy uses musical activities to achieve therapeutic goals.
Musical preferences are shaped by culture exposure and individual personality.
Music brings people together and plays important roles in rituals and celebrations.
Musicians develop specialized brain structures through years of practice.
Musical notation allows compositions to be preserved and shared across time.

Art is the expression of human creativity and imagination in visual or performative forms.
Art serves many purposes including aesthetic pleasure and social commentary.
Art history traces the development of creative expression across cultures.
Art can challenge perceptions and provoke thought about important issues.
Artistic movements like impressionism and surrealism transformed visual culture.
Art is valued both for intrinsic beauty and cultural significance.
Artists develop skills through training but also express unique personal visions.
Art markets assign monetary value to works based on complex factors.
Art appreciation involves both emotional response and intellectual understanding.
Artistic expression is a fundamental human capacity found in all cultures.

Science is the systematic study of the natural world through observation and experiment.
Science produces knowledge that can be tested verified and built upon.
Scientific method involves hypothesis testing data collection and analysis.
Science aims for objective truth though it is conducted by fallible humans.
Scientific theories are never proven only supported or refuted by evidence.
Scientists work within paradigms that shape what questions they ask.
Science has enabled tremendous technological progress and understanding.
Scientific literacy helps citizens make informed decisions about complex issues.
Science is collaborative building on the work of previous researchers.
Scientific fraud and bias are serious problems that undermine public trust.

Technology is the application of scientific knowledge for practical purposes.
Technology has transformed human civilization enabling new capabilities.
Technological progress raises both opportunities and ethical concerns.
Technology includes tools machines and techniques for solving problems.
Technological innovation drives economic growth and social change.
Technology can enhance human capabilities or replace human labor.
Technological determinism suggests that technology shapes society and culture.
Technological risks include environmental damage and unintended consequences.
Technology access is unequally distributed creating digital divides.

Technological ethics addresses questions about appropriate use and development.

Religion is a system of beliefs and practices relating to the sacred or divine.
Religion provides meaning community and moral guidance for many people.
Religious traditions vary widely in their doctrines rituals and worldviews.
Religion has played a central role in human history and civilization.
Religious experience includes feelings of transcendence and connection.
Religion and science have had complex relationships throughout history.
Religious fundamentalism emphasizes literal interpretation of sacred texts.
Religious pluralism recognizes the validity of multiple spiritual paths.
Religion shapes values customs and social structures in many societies.
Religious freedom is a fundamental human right in democratic societies.

Philosophy is the study of fundamental questions about existence knowledge and values.
Philosophy uses reason and argument to examine concepts and beliefs.
Philosophical inquiry has shaped human thought for thousands of years.
Philosophy includes ethics epistemology metaphysics and logic.
Philosophical questions often have no definitive answers but deepen understanding.
Philosophy examines the foundations and methods of other disciplines.
Philosophical traditions vary across cultures from Western to Eastern.
Philosophy can be practical helping people live better lives.
Philosophical skills include critical thinking analysis and clear argumentation.
Philosophy matters because it addresses questions everyone faces about life and meaning.

History is the study of past events and their significance for the present.
History helps us understand how societies and cultures have developed.
Historical knowledge comes from documents artifacts and other evidence.
History is written by historians who interpret evidence through their perspectives.
Historical events have multiple causes and complex consequences.
History teaches lessons though they are often contested and debated.
Historical consciousness shapes identity and influences current decisions.
History includes social economic cultural and political dimensions.
Historical memory can be selective emphasizing some events over others.
History is not just about the past but about how the past relates to present and future.

Mathematics is the abstract science of number quantity and space.
Mathematics provides tools for reasoning about patterns and relationships.
Mathematical truths are discovered through proof and logical deduction.
Mathematics is the language of science used to describe natural phenomena.
Mathematical models simplify reality to capture essential features.
Mathematics includes algebra geometry calculus and statistics.
Mathematical beauty lies in elegant proofs and surprising connections.
Mathematics develops through both pure inquiry and practical application.
Mathematical ability varies among individuals but can be developed.
Mathematics is universal applying across all cultures and contexts.

Consciousness is the subjective experience of awareness and self-knowledge.
Consciousness remains one of the deepest mysteries in philosophy and science.
Conscious experience includes perception thought emotion and sensation.
Consciousness may emerge from complex neural processes in the brain.
Conscious beings have a perspective a what it is like to be them.
Consciousness can be altered through meditation drugs or brain injury.
Conscious and unconscious processes both influence human behavior.
Consciousness raises questions about free will moral responsibility and identity.
Consciousness may be more widespread in nature than we typically assume.
Conscious machines remain theoretical though AI continues to advance.

Quantum mechanics describes the behavior of matter at the smallest scales.
Quantum particles exhibit wave-particle duality showing both properties.
Quantum superposition allows particles to be in multiple states simultaneously.
Quantum entanglement links particles across distances in strange ways.
Quantum measurement causes wave function collapse into definite states.
Quantum uncertainty means some properties cannot be known precisely.
Quantum computing uses superposition for powerful parallel processing.
Quantum physics challenges our intuitions about reality and causation.
Quantum theory is among the most successful in physics despite being strange.
Quantum effects are typically only visible at atomic and subatomic scales.

Evolution is the process by which life changes over generations.
Evolution occurs through natural selection acting on genetic variation.

Evolutionary theory explains the diversity and adaptation of life forms.
Evolution has shaped human bodies brains and behaviors over millions of years.
Evolutionary psychology applies these insights to understanding human nature.
Evolution is supported by evidence from fossils genetics and comparative anatomy.
Evolutionary change can be gradual or punctuated by rapid transitions.
Evolution has no goal or direction only differential reproductive success.
Evolutionary arms races occur between predators and prey or hosts and parasites.
Evolution continues today in all living populations including humans.
"""

# NEW: Astrophysics specific content
ASTROPHYSICS_CORPUS = """ Black holes are regions of spacetime where gravity is so
strong that nothing can escape.
Black holes form when massive stars collapse at the end of their lives.
The event horizon is the boundary around a black hole beyond which escape is impossible.
Black holes bend light around them creating gravitational lensing effects.
Supermassive black holes exist at the centers of most galaxies including our Milky Way.
Black holes emit Hawking radiation slowly losing mass over immense timescales.
Stellar black holes form from stars at least twenty times more massive than our sun.
Black holes distort spacetime so severely that time appears to stop at the event horizon.
The singularity at a black hole center is where density becomes infinite.
Black hole mergers produce gravitational waves that ripple through spacetime.

Neutron stars are the collapsed cores of massive stars after supernova explosions.
Neutron stars are incredibly dense with a teaspoon weighing billions of tons.
Pulsars are rapidly rotating neutron stars that emit beams of electromagnetic radiation.
Neutron star matter is compressed so tightly that protons and electrons merge into neutrons.
The magnetic fields of neutron stars are trillions of times stronger than Earth's.
Neutron stars typically measure only about twenty kilometers in diameter.
A neutron star forms when a star between eight and twenty solar masses explodes.
The rotation rates of pulsars are extremely stable making them cosmic clocks.
Magnetars are neutron stars with exceptionally powerful magnetic fields.
Neutron star collisions produce heavy elements like gold and platinum.

Superposition is a fundamental quantum principle where particles exist in multiple states simultaneously.
In superposition a particle can be in all possible positions or states at once.
Superposition only collapses into a definite state when observed or measured.
Being in superposition is like existing everywhere and nowhere at the same time.
Schrodinger's cat thought experiment illustrates superposition at macroscopic scales.
Quantum computers use superposition to process many calculations simultaneously.
Superposition is destroyed by decoherence when quantum systems interact with their environment.
The wave function mathematically describes all possible states in superposition.
Quantum superposition enables interference patterns in the double slit experiment.
Maintaining superposition requires isolating quantum systems from thermal noise.

Stars are massive balls of hot gas held together by their own gravity.
Stars generate energy through nuclear fusion converting hydrogen to helium.
The life cycle of a star depends on its initial mass when it forms.
Our sun is a medium sized yellow dwarf star about halfway through its life.
Red giants are stars that have exhausted their core hydrogen and expanded.
White dwarfs are the dense remnants of stars similar to our sun.
Binary star systems contain two stars orbiting their common center of mass.
Star formation occurs in giant molecular clouds of gas and dust.
The color of a star indicates its surface temperature from red to blue.
Stars create heavy elements through fusion that are scattered when they die.

Galaxies are vast collections of stars gas dust and dark matter bound by gravity.
The Milky Way is a barred spiral galaxy containing hundreds of billions of stars.
Elliptical galaxies are smooth and round with little ongoing star formation.
Spiral galaxies have distinctive arms where new stars are actively forming.
Galaxy clusters contain thousands of galaxies bound together gravitationally.
Dark matter provides most of the gravitational binding in galaxies.
Galaxies evolve through mergers and collisions over cosmic timescales.
Active galactic nuclei are powered by supermassive black holes consuming matter.
The Andromeda galaxy will collide with the Milky Way in about four billion years.
Dwarf galaxies are small systems that orbit larger host galaxies.

The universe began approximately fourteen billion years ago in the Big Bang.
The universe continues to expand carrying galaxies away from each other.
Dark energy drives the accelerating expansion of the universe.

The cosmic microwave background is radiation from when the universe first became transparent.
The observable universe is limited by the distance light has traveled since the Big Bang.
Inflation theory suggests the universe expanded exponentially in its first fraction of a second.
The universe appears flat meaning parallel lines remain parallel over cosmic distances.
Most of the universe consists of dark matter and dark energy which remain mysterious.
The ultimate fate of the universe depends on whether expansion continues forever.
Cosmic voids are vast regions of space containing very few galaxies.
"""

FULL_CORPUS = {
    "philosophy": PHILOSOPHY_CORPUS,
    "physics": PHYSICS_CORPUS,
    "computer_science": COMPUTER_SCIENCE_CORPUS,
    "biology": BIOLOGY_CORPUS,
    "mathematics": MATHEMATICS_CORPUS,
    "psychology": PSYCHOLOGY_CORPUS,
    "language": LANGUAGE_CORPUS,
    "common_topics":
COMMON_TOPICS_CORPUS,   "astrophysics":
ASTROPHYSICS_CORPUS }

def get_all_corpus_text() -> str:
"""Get all corpus text combined."""
return "\n\n".join(FULL_CORPUS.values())

def get_domain_corpus(domain: str) -> str:
"""Get corpus for specific domain."""
return FULL_CORPUS.get(domain, "")

def _clean_text(text: str) -> str: """Remove terminal commands, pip output,
code syntax, and other noise from text."""   import re   lines = text.split('\n')
cleaned_lines = []   # Patterns that indicate lines to skip
(terminal/REPL/output)   skip_patterns = [
        r'^\$ ',   # shell prompt ($ command)
r'^>>> ',   # Python REPL   r'^>
',   # generic prompt   r'^In \[\d+\]:',
IPython (In [1]:)   r'^\[.*\] %',   # progress
bar like [##...] 50%   r'^apt ',   # apt
command   r'^pip ',   # pip command
r'^sudo ',   # sudo command   r'^cd ',
cd command   r'^ls ',   # ls command
r'^cat ',   # cat command   r'^echo ',
echo command
        r'^sh:',   # shell output prefix (sh: command)
r'^/bin/',   # binary path
        r'^# '   # lines that are only a comment (no preceding
text)   ]   for line in lines:
        stripped = line.strip()   if not stripped:
cleaned_lines.append(line)   continue
Check if line matches any skip pattern   if
any(re.match(pat, stripped) for pat in skip_patterns):
            continue   # Skip lines that are almost entirely non-
alphanumeric (e.g., -----, ====, ***)   # Keep if there is at least one
alphanumeric character   if not re.search(r'[A-Za-z0-9]', stripped):
            continue   # Skip lines that look like raw progress bars
(e.g., [::::::::::] or [...;;;...])   if re.match(r'^\[[=:#;.\-\s]+\]$',
stripped):

continue

cleaned_lines.append(line)
    return '\n'.join(cleaned_lines)

def get_all_corpus_text() -> str: """Get
all corpus text combined."""   combined =
"\n\n".join(FULL_CORPUS.values())   return
_clean_text(combined)

def get_domain_corpus(domain: str) -> str:

```
    """Get corpus for specific domain."""
    raw = FULL_CORPUS.get(domain, "")
return _clean_text(raw)


def get_corpus_sentences() -&gt; list:     """Get all sentences
from corpus."""     import re     text = get_all_corpus_text()
sentences = re.split(r'(?&lt;=[.!?])\s+', text)     return
[s.strip() for s in sentences if len(s.strip()) &gt; 20]
```

■ backend/advanced_algorithms.py

```
"""
■ ADVANCED ALGORITHMIC IMPROVEMENTS
==================================== What would
make the algorithm TRULY intelligent:

1. GRAMMAR ENGINE - Current Markov outputs can be grammatically awkward
2. CONTEXT MEMORY - Remember conversation context across turns
3. REASONING CHAINS - Multi-step logical inference
4. SEMANTIC EMBEDDINGS - Dense vector representations
5. ATTENTION OVER MEMORY - Focus on relevant past knowledge
6. DYNAMIC TEMPERATURE - Adjust creativity based on query type
7. CONCEPT GRAPHS - Knowledge graph with weighted edges
8. BAYESIAN UPDATING - Update beliefs based on evidence

This file implements these improvements.
"""

import re
from collections import defaultdict, Counter
from typing import List, Dict, Tuple,
Optional from datetime import datetime import
logging

logger = logging.getLogger("quantum_ai")


# ============================================================================
# 1. GRAMMAR ENGINE - Fix Markov output grammar #
============================================================================

class GrammarEngine:     """     Post-processes Markov output to
fix common grammatical issues.

Rules:
- Subject-verb agreement
- Article usage (a/an)
- Capitalization
- Punctuation    - Remove repeated words
    """         def
__init__(self):
        self.article_a_patterns = re.compile(r'\ba ([aeiou])', re.IGNORECASE)
self.article_an_patterns = re.compile(r'\ban ([^aeiou])', re.IGNORECASE)
self.repeated_words = re.compile(r'\b(\w+)\s+\1\b', re.IGNORECASE)
# Common verb fixes         self.verb_corrections = {             'is are':
'are',
            'are is': 'is',
            'was were': 'were',
            'have has': 'has',
'do does': 'does',
        }         def fix(self, text: str, query: str =
None) -&gt; str:         """Apply all grammar fixes."""
if not text:             return text
        original_text = text
words = text.split()

        # Extract query words and find the main
topic         query_words = set()         main_topic
= None         if query:
            query_words = set(w.lower().rstrip('.,!?') for w in query.split())             #
Find the main noun (topic) from query - typically the last significant word
topic_candidates = ['gravity', 'time', 'space', 'energy', 'consciousness', 'love',
                                'life', 'death', 'truth', 'beauty', 'freedom', 'knowledge',                                 'reality', 'matter',
'intelligence', 'science', 'philosophy',
                                'music', 'art', 'religion', 'technology', 'history', 'mathematics',
'evolution', 'quantum', 'physics', 'biology', 'psychology', 'language',
'happiness', 'meaning', 'existence', 'mind', 'universe']             for word in
reversed(query.lower().split()):
```

```
                clean_word = word.rstrip('.,!?')                 if clean_word in
topic_candidates:                     main_topic = clean_word.capitalize()
break                 # Bad starters that indicate we've hit a sentence fragment
(expanded)         bad_starters = {'is', 'are', 'the', 'a', 'an', 'what', 'how',
'why', 'when',                        'where', 'tell', 'explain', 'describe',
'define', 'has',
                       'does', 'do', 'can', 'could', 'would', 'should', 'me', 'about',
                       'it', 'this', 'that', 'i', 'you', 'we', 'they', 'and', 'or',
                       'but', 'if', 'then', 'so', 'for', 'with', 'by', 'from', 'to',
                       'at', 'in', 'on', 'of', 'as', 'not', 'cannot', 'when',
'something', 'interesting', 'know', 'learn', 'holes'}
                # Common verb starters that indicate a
fragment
        verb_starters = {'is', 'are', 'was', 'were', 'has', 'have', 'had', 'does', 'do',
'can', 'could', 'would', 'should', 'may', 'might', 'must',
                        'attracts', 'causes', 'requires', 'involves', 'includes', 'remains',
                        'evolved', 'exists', 'occurs', 'emerges', 'manifests', 'appears',
                        'provides', 'enables', 'allows', 'produces', 'creates', 'forms',
                        'takes', 'gives', 'seeks', 'needs', 'wants', 'uses', 'makes',
                        'been', 'being', 'keeps', 'holds', 'bends', 'shapes', 'acts',
                        'flows', 'travels', 'moves', 'describes', 'explains', 'shows',
                        'reveals', 'suggests', 'implies', 'means', 'contains', 'consists',
                        'affects', 'influences', 'drives', 'connects', 'links', 'relates',
                        'varies', 'depends', 'comes', 'goes', 'leads', 'follows', 'results',
'continues', 'started', 'began', 'ended', 'happened', 'occurred',
'cannot'}
                # First: Check if text starts with main topic - if so, it's
probably good         if words and main_topic:
            first_word_clean =
words[0].lower().rstrip('.,!?')             if
first_word_clean == main_topic.lower():                 #
Looks good, skip most cleaning                 pass
else:                 # Need to fix - check for fragments
                                # Remove query
words from start                 content_start = 0
for i in range(min(5, len(words))):
                    word_clean = words[i].lower().rstrip('.,!?')
if word_clean in query_words and word_clean != main_topic.lower():
                        content_start = i + 1
elif word_clean in bad_starters:
                        content_start = i + 1                     else:
break                                 if content_start &gt; 0 and
content_start &lt; len(words):
                    words = words[content_start:]                     text = ' '.join(words)
# CHECK: Did we create a fragment? If first word is a verb or bad starter, add topic
if words:
                    first_word = words[0].lower().rstrip('.,!?')
if first_word in verb_starters or first_word in bad_starters:
# Add the topic as subject                         text = main_topic + ' ' +
text.lower()         else:             # No main topic identified, do basic
cleanup             content_start = 0             if len(words) &gt; 2:
for i in range(min(5, len(words))):
                    word_clean = words[i].lower().rstrip('.,!?')
if word_clean in query_words or word_clean in bad_starters:
                        content_start = i +
1                     else:
break
            if content_start &gt; 0 and content_start &lt; len(words):
                words =
words[content_start:]                 text =
' '.join(words)
        # CLEANUP: Remove problematic patterns
        # Pattern 1: "X explain X is..." -&gt; "X is..."
        # Pattern 2: "X meaning X is..." -&gt; "X is..."
        # Pattern 3: "Physical modern X" -&gt; "X"         # Pattern 4: "Quantum describes
mechanics quantum mechanics" -&gt; "Quantum mechanics"         words = text.split()
if len(words) &gt; 3 and main_topic:             main_topic_lower = main_topic.lower()
            # First check for repeated topic pattern: "Topic describes topic
topic..."             topic_indices = []             for i, w in
```

```
enumerate(words[:6]):                 if w.lower().rstrip('.,!?') ==
main_topic_lower:
                    topic_indices.append(i)                         if len(topic_indices) &gt;= 2:                 # Keep from second
occurrence of topic                 second_idx = topic_indices[1]                 text = ' '.join(words[second_idx:])
words = text.split()                          # Check for "X explain topic" or "X meaning topic" patterns             for i in
range(min(3, len(words))):                  if words[i].lower().rstrip('.,!?') in ['explain', 'meaning', 'physical', 'modern',
'objects', 'describes', 'mechanics']:                     # Find where topic word starts                     for j in range(i+1,
min(i+4, len(words))):                          if words[j].lower().rstrip('.,!?') == main_topic_lower:                             #
Skip to topic
                            text = main_topic + ' ' + '
'.join(words[j+1:]).lower()                             break
break                 # Pattern 4: "Topic topic is..." -&gt; "Topic is..."
words = text.split()         if len(words) &gt; 2 and main_topic:             if
words[0].lower() == words[1].lower().rstrip('.,!?'):
                text = ' '.join(words[1:])
text = text[0].upper() + text[1:]
# 1. Fix articles (a/an)
        text = self.article_a_patterns.sub(r'an \1', text)
text = self.article_an_patterns.sub(r'a \1', text)

        # 2. Remove repeated words
        text = self.repeated_words.sub(r'\1', text)

        return self._fix_continued(text)

    def _fix_continued(self, text):         """Continuation of fix —
auto-extracted by self-evolution."""         # 3. Fix verb issues
for wrong, right in self.verb_corrections.items():             text
= text.replace(wrong, right)

        # 4. Capitalize first letter
if text:             text = text[0].upper()
+ text[1:]

        # 5. Capitalize after periods
        text = re.sub(r'(\. )([a-z])', lambda m: m.group(1) + m.group(2).upper(), text)

        # 6. Ensure ends with punctuation
if text and text[-1] not in '.!?':
text += '.'

        # 7. Fix multiple spaces
        text = re.sub(r'\s+', ' ', text).strip()

        # 8. Fix common issues
text = text.replace(' ,', ',')
text = text.replace(' .', '.')
text = text.replace('..', '.')

        return text



# ============================================================================
# 2. CONTEXT MEMORY - Track conversation context #
==============================================================================

class ConversationContext:     """
Maintains context across conversation turns.
        Tracks:     - Active topics
(weighted by recency)
- Referenced entities
- User intent history
- Emotional tone
    """         def __init__(self,
max_turns: int = 10):
        self.max_turns = max_turns
        self.turns = []  # List of (user_msg, response, concepts)
self.active_topics = Counter()         self.entities = set()
self.intent_history = []         def add_turn(self, user_msg: str,
```

```
response: str, concepts: List[str]):         """Add a conversation
turn."""         self.turns.append({             'user': user_msg,
            'response': response,
            'concepts': concepts,
'time': datetime.now()
        })
                # Keep only recent turns
if len(self.turns) &gt; self.max_turns:
            self.turns.pop(0)
# Update active topics with decay
self.active_topics = Counter()         for
i, turn in enumerate(self.turns):
            weight = (i + 1) / len(self.turns)  # More recent = higher weight
for concept in turn['concepts']:
                self.active_topics[concept] += weight
# Extract entities (capitalized words)         entities =
re.findall(r'\b[A-Z][a-z]+\b', user_msg)
self.entities.update(entities)         def
get_context_concepts(self, top_n: int = 5) -&gt; List[str]:
"""Get most relevant context concepts."""
        return [c for c, _ in
self.active_topics.most_common(top_n)]         def
get_recent_topics(self) -&gt; List[str]:         """Get topics from
recent turns."""         if not self.turns:
            return []
        return self.turns[-1].get('concepts', [])[:3]
def is_followup(self, query: str) -&gt; bool:
"""Check if query is a follow-up to previous topic."""
        followup_indicators = ['it', 'that', 'this', 'they', 'those', 'more', 'also', 'what about']
query_lower = query.lower()
        return any(ind in query_lower for ind in followup_indicators)


# ============================================================================
# 3. REASONING CHAINS - Multi-step inference #
============================================================================

class ReasoningEngine:
    """     Performs multi-step logical
reasoning.

Implements:
- Forward chaining (A→B, B→C ∴ A→C)
- Analogical reasoning (A:B :: C:?)
- Causal inference
    """         def __init__(self):         self.rules = []  # List of
(condition, conclusion, confidence)         self.facts = set()         def
add_rule(self, condition: str, conclusion: str, confidence: float = 1.0):
"""Add an inference rule."""
        self.rules.append((condition.lower(), conclusion.lower(), confidence))         def
add_fact(self, fact: str):         """Add a known fact."""
self.facts.add(fact.lower())         def infer(self, query: str, max_steps: int = 5) -&gt;
List[Tuple[str, float, List[str]]]:         """         Perform inference to answer query.
        Returns list of (conclusion, confidence, reasoning_chain)
"""
        query_lower = query.lower()         results = []
# Direct fact check         for fact in self.facts:
if query_lower in fact or fact in query_lower:
                results.append((fact, 1.0, ["Direct
knowledge"]))                 # Forward chaining
current_facts = set(self.facts)         chain = []
for step in range(max_steps):
            new_facts = set()             for condition, conclusion, conf in
self.rules:                 if condition in current_facts and conclusion not in
current_facts:
                    new_facts.add(conclusion)
                    chain.append(f"{condition} → {conclusion}")
                                        if
query_lower in conclusion:
```

```
                        results.append((conclusion, conf ** (step + 1),
chain.copy()))                          if not new_facts:
                break
            current_facts.update(new_facts)                 return results         def
analogy(self, a: str, b: str, c: str, candidates: List[str]) -&gt; Optional[str]:
"""         A is to B as C is to ?
        Find best match from candidates.
        """
        # Simple word-level analogy using character
patterns         # In real system, would use word
embeddings
        a_to_b = self._relationship(a,
b)                  best_match = None
best_score = 0                  for
candidate in candidates:
            c_to_cand = self._relationship(c,
candidate)              score = self._similarity(a_to_b,
c_to_cand)              if score &gt; best_score:
best_score = score                  best_match =
candidate         return best_match
    def _relationship(self, w1: str, w2: str) -&gt; Dict:
"""Extract relationship features between words."""
return {              'len_diff': len(w2) - len(w1),
            'shared_chars': len(set(w1) &amp; set(w2)),             'prefix_match': w1[:2] ==
w2[:2] if len(w1) &gt;= 2 and len(w2) &gt;= 2 else False
        }         def _similarity(self, r1: Dict, r2:
Dict) -&gt; float:         """Similarity between
relationships."""         score = 0         if
r1.get('len_diff') == r2.get('len_diff'):
            score += 0.3         if abs(r1.get('shared_chars', 0) -
r2.get('shared_chars', 0)) &lt;= 1:
            score += 0.3         if r1.get('prefix_match')
== r2.get('prefix_match'):
            score += 0.4
return score


# ============================================================================
# 4. SEMANTIC SIMILARITY - Without external embeddings #
============================================================================

class SemanticSimilarity:     """     Compute semantic similarity using
character n-grams and word overlap.
    Works without external embedding models.
    """         def __init__(self, n: int = 3):
self.n = n  # n-gram size         def similarity(self, text1:
str, text2: str) -&gt; float:         """         Compute
similarity combining:         - Character n-gram overlap
(catches typos, morphology)
        - Word overlap (Jaccard)
- Word order similarity
"""
        t1, t2 = text1.lower(), text2.lower()
# Character n-gram similarity         ngrams1 =
self._char_ngrams(t1)         ngrams2 =
self._char_ngrams(t2)         ngram_sim =
self._jaccard(ngrams1, ngrams2)                 #
Word overlap         words1 = set(t1.split())
words2 = set(t2.split())
        word_sim = self._jaccard(words1,
words2)                 # Combined score
        return ngram_sim * 0.4 + word_sim * 0.6
def _char_ngrams(self, text: str) -&gt; set:
"""Extract character n-grams."""         text =
text.replace(' ', '_')
        return set(text[i:i+self.n] for i in range(len(text) - self.n +
1))         def _jaccard(self, set1: set, set2: set) -&gt; float:
"""Jaccard similarity."""         if not set1 or not set2:
            return 0.0
        intersection = len(set1 &amp; set2)
union = len(set1 | set2)
```

```
        return intersection / union if union &gt; 0 else 0.0         def most_similar(self, query: str,
candidates: List[str], top_k: int = 5) -&gt; List[Tuple[str, float]]:         """Find most similar
candidates."""
        scores = [(c, self.similarity(query, c)) for c in
candidates]         scores.sort(key=lambda x: x[1], reverse=True)
return scores[:top_k]
# ============================================================================
# 5. CONCEPT GRAPH - Knowledge graph structure
# ============================================================================
```

■ backend/advanced_algorithms_pt2.py

```
""" Auto-split from advanced_algorithms.py by self-evolution engine. Part 2 — contains: ConceptGraph,
DynamicTemperature, BayesianBeliefs, AdvancedAlgorithmicEngine, get_advanced_engine """

import math
import re
from collections import defaultdict, Counter
from typing import List, Dict, Tuple, Optional
from advanced_algorithms import GrammarEngine, ConversationContext, ReasoningEngine, SemanticSimilarity

# Singleton instance
_advanced_engine = None

class ConceptGraph:     """
Knowledge graph with weighted edges.

Supports:
- Adding concepts and relationships
- Path finding between concepts    - Spreading activation
    """         def
__init__(self):
self.nodes = set()
        self.edges = defaultdict(dict)  # node -&gt; {neighbor:
weight}         def add_concept(self, concept: str):         """Add a
concept node."""         self.nodes.add(concept.lower())
    def add_relationship(self, concept1: str, concept2: str,
relationship: str = "related", weight: float = 1.0):         """Add weighted
edge between concepts."""         c1, c2 = concept1.lower(),
concept2.lower()         self.nodes.add(c1)         self.nodes.add(c2)
# Bidirectional
        self.edges[c1][c2] = max(self.edges[c1].get(c2, 0), weight)         self.edges[c2][c1] =
max(self.edges[c2].get(c1, 0), weight)         def get_neighbors(self, concept: str, min_weight:
float = 0.1) -&gt; List[Tuple[str, float]]:         """Get neighboring concepts above weight
threshold."""         c = concept.lower()         if c not in self.edges:
            return
[]
        neighbors = [(n, w) for n, w in self.edges[c].items() if w &gt;=
min_weight]         neighbors.sort(key=lambda x: x[1], reverse=True)         return
neighbors
    def spreading_activation(self, start: str, decay: float = 0.7,
max_depth: int = 3) -&gt; Dict[str, float]:         """
Spreading activation from start node.
        Returns activation levels for all reached
nodes.         """         c = start.lower()         if
c not in self.nodes:             return {c: 1.0}
        activations = {c: 1.0}
current_level = {c}
for depth in range(max_depth):
            next_level = set()
            current_activation = decay ** depth
for node in current_level:                 for neighbor, weight in
self.edges.get(node, {}).items():                     if neighbor not in
activations:                         act = current_activation * weight
activations[neighbor] = act                         if act &gt; 0.1:  #
Threshold                             next_level.add(neighbor)
if not next_level:                 break             current_level =
next_level
        return activations         def path_between(self, start: str, end: str, max_depth: int
= 5) -&gt; Optional[List[str]]:         """Find path between two concepts (BFS)."""         s,
e = start.lower(), end.lower()         if s not in self.nodes or e not in self.nodes:
            return None                 if s
== e:             return [s]
visited = {s}         queue = [(s, [s])]
while queue and len(visited) &lt; 1000:
            current, path = queue.pop(0)
for neighbor in self.edges.get(current, {}):
if neighbor == e:
                    return path +
[neighbor]
```

```
if neighbor not in visited:
visited.add(neighbor)
                    queue.append((neighbor, path +
[neighbor]))                          if len(path) &gt;=
max_depth:

continue
return None


class DynamicTemperature:     """     Adjusts generation
temperature based on query characteristics.

- Factual queries → low temperature (deterministic)     - Creative queries → high temperature
  (diverse)
- Follow-up queries → medium temperature (coherent)
    """         def
__init__(self):
        self.factual_keywords = {             'what is', 'who is',
'when', 'where', 'how many', 'how much',             'define',
'calculate', 'convert', 'distance', 'population'         }
        self.creative_keywords = {             'imagine', 'create',
'write', 'poem', 'story', 'invent',             'suppose', 'what
if', 'dream', 'fantasy', 'explore'         }
        self.philosophical_keywords = {             'why', 'meaning',
'purpose', 'consciousness', 'existence',             'believe',
'think', 'feel', 'opinion', 'perspective'
        }         def compute(self, query: str, is_followup: bool =
False) -&gt; float:         """         Compute optimal temperature for
query.
                Returns float in
[0.3, 1.5]         """
        query_lower =
query.lower()                  #
Check query type
        is_factual = any(kw in query_lower for kw in self.factual_keywords)
        is_creative = any(kw in query_lower for kw in self.creative_keywords)
is_philosophical = any(kw in query_lower for kw in self.philosophical_keywords)
# Base temperature         if is_factual:
            temp = 0.4  # Low - be precise
elif is_creative:
            temp = 1.2  # High - be diverse
elif is_philosophical:
            temp = 0.9  # Medium-high - thoughtful
variation         else:             temp = 0.7  # Default
# Adjust for follow-up         if is_followup:
            temp *= 0.85  # Reduce to maintain coherence                 #
Query length adjustment (longer = more complex = slightly higher)
words = len(query.split())         if words &gt; 15:             temp +=
0.1                 # Clamp
        return max(0.3, min(1.5, temp))


class BayesianBeliefs:     """     Maintains beliefs about
concepts using Bayesian updating.
        P(H|E) = P(E|H) × P(H) /
P(E)
    """         def
__init__(self):
        self.beliefs = defaultdict(lambda: 0.5)  # Prior = 0.5 (uncertain)
self.evidence_counts = defaultdict(lambda: {'positive': 0, 'negative': 0})
def observe(self, concept: str, evidence: str, is_positive: bool):
"""Update belief based on evidence."""         c = concept.lower()
if is_positive:
            self.evidence_counts[c]['positive'] +=
1         else:
self.evidence_counts[c]['negative'] += 1
# Bayesian update
        pos =
self.evidence_counts[c]['positive']         neg
```

```
= self.evidence_counts[c]['negative']
total = pos + neg
        # Beta distribution mean         # Prior: alpha=1, beta=1 (uniform)
self.beliefs[c] = (pos + 1) / (total + 2)         def get_belief(self, concept: str) -
&gt; float:         """Get current belief strength."""         return
self.beliefs[concept.lower()]         def get_confident_beliefs(self, threshold: float =
0.7) -&gt; List[Tuple[str, float]]:         """Get beliefs above confidence threshold."""
        confident = [(c, b) for c, b in self.beliefs.items() if b &gt;=
threshold]         confident.sort(key=lambda x: x[1], reverse=True)
return confident         def uncertainty(self, concept: str) -&gt; float:
"""Measure uncertainty about a concept (entropy)."""         p =
self.beliefs[concept.lower()]         if p &lt;= 0 or p &gt;= 1:
            return 0
        return -p * math.log2(p) - (1-p) * math.log2(1-p)


class AdvancedAlgorithmicEngine:
    """     Integrates all advanced improvements into
cohesive system.
    """         def
__init__(self):
        self.grammar = GrammarEngine()         self.context =
ConversationContext()         self.reasoning = ReasoningEngine()
self.similarity = SemanticSimilarity()         self.graph =
ConceptGraph()         self.temperature = DynamicTemperature()
self.beliefs = BayesianBeliefs()                 # Initialize with
some reasoning rules         self._init_reasoning_rules()
def _init_reasoning_rules(self):         """Add basic reasoning
rules."""         rules = [             ("consciousness exists",
"experience is possible", 0.9),
            ("information is processed", "computation occurs", 0.95),
            ("learning occurs", "behavior changes", 0.85),
            ("observation happens", "knowledge increases", 0.8),
("questions are asked", "understanding deepens", 0.75),         ]
for cond, conc, conf in rules:
            self.reasoning.add_rule(cond, conc, conf)         def enhance_response(self,
response: str, query: str, concepts: List[str]) -&gt; str:         """Apply all
enhancements to a response."""         # 1. Grammar fix (pass query for better cleaning)
response = self.grammar.fix(response, query=query)                 # 2. Update context
        self.context.add_turn(query, response,
concepts)                 # 3. Update concept graph
for i, c1 in enumerate(concepts):             for c2 in
concepts[i+1:]:
                self.graph.add_relationship(c1, c2, "co-occurred",
0.5)                 return response         def get_temperature(self,
query: str) -&gt; float:         """Get optimal temperature for
query."""         is_followup = self.context.is_followup(query)
return self.temperature.compute(query, is_followup)         def
get_context_boost(self, concepts: List[str]) -&gt; List[str]:
"""Get additional concepts from context and graph."""         boosted
= set(concepts)                 # From conversation context

boosted.update(self.context.get_context_concepts(3))
# From concept graph (spreading activation)         for c
in concepts[:2]:
            activations = self.graph.spreading_activation(c, max_depth=2)
            top_activated = sorted(activations.items(), key=lambda x: x[1], reverse=True)[:3]
boosted.update(c for c, _ in top_activated)
        return list(boosted)         def
reason_about(self, query: str) -&gt; List[str]:
"""Get reasoning chain for query."""         results =
self.reasoning.infer(query)         chains = []
for conclusion, conf, chain in results:             if
conf &gt; 0.5:
                chains.extend(chain)
return chains


def get_advanced_engine() -&gt; AdvancedAlgorithmicEngine:
    """Get singleton advanced engine."""     global
_advanced_engine     if _advanced_engine is None:
```

```
    _advanced_engine = AdvancedAlgorithmicEngine()
    return _advanced_engine
```

■ backend/word_embeddings.py

```
"""
■ WORD EMBEDDINGS - Pure Python Implementation
============================================= Word2Vec-
style embeddings without external dependencies.

Implements Skip-gram with Negative Sampling (simplified).
Words are represented as dense vectors capturing semantic relationships.
Similar words have similar vectors.
"""

import math
import random
import re
import json
import os
from collections import defaultdict, Counter
from typing import List, Dict, Tuple, Optional
import logging

logger = logging.getLogger("quantum_ai")


class WordEmbeddings:     """     Simple
word embeddings trained on corpus.
        Uses co-occurrence matrix factorization (GloVe-style)
which is more tractable than Skip-gram for CPU-only training.
Algorithm:
1. Build word co-occurrence matrix from corpus
2. Apply PPMI (Positive PMI) weighting
3. Reduce dimensionality via truncated SVD approximation
    """         def __init__(self, vector_size: int = 50, window: int = 5,
min_count: int = 2):
        self.vector_size = vector_size
self.window = window         self.min_count =
min_count                 self.word_to_id = {}
self.id_to_word = {}         self.vectors = {}
# word -&gt; vector         self.word_counts =
Counter()         self.cooccurrence =
defaultdict(Counter)
self.trained = False         self.vocab_size =
0         def build_vocab(self, texts:
List[str]):         """Build vocabulary from
texts."""         for text in texts:
            words = self._tokenize(text)
self.word_counts.update(words)
# Filter by min_count
        vocab = [w for w, c in self.word_counts.items() if c &gt;=
self.min_count]         vocab.sort()
        self.word_to_id = {w: i for i, w in enumerate(vocab)}
self.id_to_word = {i: w for w, i in self.word_to_id.items()}
self.vocab_size = len(vocab)
        logger.info(f"Built vocabulary: {self.vocab_size}
words")         def build_cooccurrence(self, texts: List[str]):
"""Build co-occurrence matrix."""         for text in texts:
            words = self._tokenize(text)
            words = [w for w in words if w in
self.word_to_id]                        for i, center in
enumerate(words):
                # Context window                start =
max(0, i - self.window)                end =
min(len(words), i + self.window + 1)
for j in range(start, end):                     if i !=
j:
                        context = words[j]                        #
Distance weighting (closer = higher weight)
distance = abs(i - j)                        weight = 1.0 /
distance
```

```
                        self.cooccurrence[center][context] +=
weight
        logger.info(f"Built co-occurrence matrix")         def
train(self, texts: List[str], iterations: int = 10):
"""         Train embeddings using simplified GloVe-style
algorithm.
                This is a lightweight approximation suitable for CPU
training.
        """         # Build vocab and
co-occurrence
self.build_vocab(texts)
self.build_cooccurrence(texts)

        self._train_continued(iterations)

    def _train_continued(self, iterations):         """Continuation of
train — auto-extracted by self-evolution."""         if
self.vocab_size == 0:
            logger.warning("Empty vocabulary, cannot train")
return

        # Initialize random vectors         for word in self.word_to_id:
self.vectors[word] = [random.gauss(0, 0.1) for _ in range(self.vector_size)]

        # Compute PPMI matrix and train
        total_count = sum(sum(v.values()) for v in self.cooccurrence.values())
word_probs = {w: self.word_counts[w] / sum(self.word_counts.values())
for w in self.word_to_id}

        # Stochastic gradient descent on weighted least squares
learning_rate = 0.05

        for iteration in range(iterations):
            total_loss = 0
updates = 0

            for word_i, contexts in
self.cooccurrence.items():                  if word_i not in
self.vectors:                     continue

                vec_i = self.vectors[word_i]

                for word_j, count in contexts.items():
if word_j not in self.vectors:
continue

                    vec_j = self.vectors[word_j]

                    # Compute PPMI target
p_ij = count / total_count                     p_i =
word_probs.get(word_i, 1e-10)                     p_j =
word_probs.get(word_j, 1e-10)                     pmi =
math.log(p_ij / (p_i * p_j) + 1e-10)                     ppmi
= max(0, pmi)

                    # Dot product prediction
                    dot = sum(a * b for a, b in zip(vec_i, vec_j))

                    # Error
error = dot - ppmi

                    # Weight by co-occurrence frequency
                    weight = min(1.0, (count / 100) ** 0.75)

                    # Gradient update
for k in range(self.vector_size):
grad_i = weight * error * vec_j[k]
grad_j = weight * error * vec_i[k]

                        vec_i[k] -= learning_rate * grad_i
vec_j[k] -= learning_rate * grad_j
```

```
                    total_loss += weight * error ** 2
updates += 1

            if updates &gt; 0:
                avg_loss = total_loss / updates                 if iteration % 3 == 0:
logger.info(f"Embedding training iteration {iteration}: loss={avg_loss:.4f}")

        # Normalize vectors         for word in
self.vectors:             vec = self.vectors[word]
norm = math.sqrt(sum(x*x for x in vec))             if
norm &gt; 0:                  self.vectors[word] = [x/norm
for x in vec]

        self.trained = True
        logger.info(f"Trained {len(self.vectors)} word vectors")

        def get_vector(self, word: str) -&gt;
Optional[List[float]]:         """Get vector for a word."""
return self.vectors.get(word.lower())         def
similarity(self, word1: str, word2: str) -&gt; float:
"""Compute cosine similarity between two words."""         vec1
= self.get_vector(word1)         vec2 = self.get_vector(word2)
if vec1 is None or vec2 is None:
            return 0.0                 # Cosine similarity (vectors already
normalized)         return sum(a * b for a, b in zip(vec1, vec2))         def
most_similar(self, word: str, top_n: int = 10) -&gt; List[Tuple[str, float]]:
"""Find most similar words."""         vec = self.get_vector(word)         if vec is
None:             return []                 similarities = []         for
other_word, other_vec in self.vectors.items():             if other_word !=
word.lower():
                sim = sum(a * b for a, b in zip(vec,
other_vec))                 similarities.append((other_word,
sim))
        similarities.sort(key=lambda x: x[1], reverse=True)         return
similarities[:top_n]         def analogy(self, a: str, b: str, c: str, top_n: int = 5) -
&gt; List[Tuple[str, float]]:         """         Solve analogy: a is to b as c is to ?
                Uses vector arithmetic: result = vec(b) - vec(a)
+ vec(c)         """
        vec_a = self.get_vector(a)         vec_b =
self.get_vector(b)         vec_c = self.get_vector(c)
if any(v is None for v in [vec_a, vec_b, vec_c]):
            return []
# Compute target vector
        target = [b_i - a_i + c_i for a_i, b_i, c_i in zip(vec_a, vec_b, vec_c)]
        # Normalize
        norm = math.sqrt(sum(x*x for x in target))
if norm &gt; 0:
            target = [x/norm for x in target]
# Find closest words (excluding inputs)
exclude = {a.lower(), b.lower(), c.lower()}
similarities = []                 for word, vec in
self.vectors.items():             if word not in
exclude:
                sim = sum(t * v for t, v in zip(target,
vec))                 similarities.append((word, sim))
        similarities.sort(key=lambda x: x[1], reverse=True)
return similarities[:top_n]         def sentence_vector(self, text:
str) -&gt; Optional[List[float]]:         """Get average vector for
a sentence."""         words = self._tokenize(text)
        vectors = [self.get_vector(w) for w in words if
self.get_vector(w)]                 if not vectors:             return
None                 # Average
        result = [0.0] *
self.vector_size         for vec in
vectors:             for i, v in
enumerate(vec):
                result[i] += v
n = len(vectors)         result = [x /
n for x in result]                 #
Normalize
```

```
        norm = math.sqrt(sum(x*x for x in result))
if norm &gt; 0:
            result = [x/norm for x in result]                 return
result         def sentence_similarity(self, text1: str, text2: str)
-&gt; float:         """Compute similarity between two sentences."""
vec1 = self.sentence_vector(text1)         vec2 =
self.sentence_vector(text2)                 if vec1 is None or vec2
is None:
            return
0.0
        return sum(a * b for a, b in zip(vec1, vec2))
def _tokenize(self, text: str) -&gt; List[str]:
"""Tokenize text to words."""         return
re.findall(r'\b[a-z]+\b', text.lower())         def
save(self, filepath: str):         """Save embeddings
to file."""         data = {             'vectors':
self.vectors,
            'word_to_id': self.word_to_id,
            'vocab_size': self.vocab_size,
'vector_size': self.vector_size         }
with open(filepath, 'w') as f:
            json.dump(data, f)
        logger.info(f"Saved embeddings to
{filepath}")         def load(self, filepath: str) -
&gt; bool:         """Load embeddings from file."""
try:             if os.path.exists(filepath):
with open(filepath, 'r') as f:
data = json.load(f)                 self.vectors =
data['vectors']                 self.word_to_id =
data['word_to_id']
                self.id_to_word = {int(i): w for w, i in
self.word_to_id.items()}                 self.vocab_size = data['vocab_size']
self.vector_size = data['vector_size']                 self.trained = True
                logger.info(f"Loaded {len(self.vectors)}
embeddings")                 return True         except Exception as
e:
            logger.warning(f"Could not load embeddings:
{e}")         return False         def get_stats(self) -&gt;
Dict:         """Get embedding statistics."""         return
{             'vocab_size': self.vocab_size,
            'vector_size': self.vector_size,
            'trained': self.trained,
'total_vectors': len(self.vectors)
        }


# Singleton with lazy initialization
_embeddings = None _embeddings_file = "/app/backend/word_embeddings.json"
def get_embeddings() -&gt; WordEmbeddings:     """Get or create word
embeddings."""     global _embeddings     if _embeddings is None:
_embeddings = WordEmbeddings(vector_size=50, window=5, min_count=2)
                # Try to load pre-trained
if not _embeddings.load(_embeddings_file):
# Train on corpus
            from training_corpus import
get_corpus_sentences             sentences =
get_corpus_sentences()             if sentences:
_embeddings.train(sentences, iterations=15)
_embeddings.save(_embeddings_file)         return
_embeddings


def train_embeddings_on_text(text: str):     """Add new
text to embeddings training."""     embeddings =
get_embeddings()     sentences =
re.split(r'(?&lt;=[.!?])\s+', text)     sentences = [s
for s in sentences if len(s) &gt; 20]         if
sentences:
        embeddings.train(sentences, iterations=5)
embeddings.save(_embeddings_file)
```

■ backend/document_ingester.py

```
"""
Quantum MCAGI — Document Ingester
================================= No document
discrimination. Every format welcome.

Supports:   Local files:  .txt .md .py .pdf .docx .xlsx .csv .pptx .json
.html .xml
  URLs:         Any webpage, direct file link
  Google Drive: Shared links (public)
  Dropbox:      Shared links
  OneDrive:     Shared links   Images:       .jpg .png .gif .bmp (text
extraction via OCR if available) """

import os import re import
json import requests
import urllib.parse from
pathlib import Path from
typing import Optional

# ■■ Format handlers ■■■
```

```
text.append(shape.text)         return
'\n'.join(text)     except Exception as e:
        return f"[PPTX ERROR: {e}]"

def extract_csv(filepath):     import csv     text =
[]     try:         with open(filepath, 'r',
errors='ignore') as f:
            reader = csv.reader(f)
for row in reader:
text.append(' '.join(row))         return
'\n'.join(text)     except Exception as
e:
        return f"[CSV ERROR: {e}]"

def extract_image(filepath):
try:
        import pytesseract         from PIL
import Image         img =
Image.open(filepath)         return
pytesseract.image_to_string(img)     except
ImportError:
        return "[IMAGE: OCR not available — install
pytesseract]"     except Exception as e:         return f"[IMAGE
ERROR: {e}]"

def extract_html(filepath=None, html_text=None):
try:
        from bs4 import BeautifulSoup         if filepath:
with open(filepath, 'r', errors='ignore') as f:
                html_text = f.read()
        soup = BeautifulSoup(html_text, 'html.parser')
for tag in soup(['script', 'style', 'nav', 'footer']):
            tag.decompose()
        return soup.get_text(separator='\n', strip=True)
except Exception as e:
        return f"[HTML ERROR: {e}]"

def extract_json(filepath):     try:         with
open(filepath, 'r', errors='ignore') as f:
            data = json.load(f)
return json.dumps(data, indent=2)
except Exception as e:         return
f"[JSON ERROR: {e}]"

# ═══ Extension router ═══════════════════════════════════════════════

EXTRACTORS = {
    '.txt': extract_txt,
    '.md': extract_txt,
    '.py': extract_txt,
    '.js': extract_txt,
    '.xml': extract_txt,
    '.rst': extract_txt,
    '.pdf': extract_pdf,
    '.docx': extract_docx,
    '.doc': extract_docx,
    '.xlsx': extract_xlsx,
    '.xls': extract_xlsx,
    '.pptx': extract_pptx,
    '.ppt': extract_pptx,
    '.csv': extract_csv,
    '.tsv': extract_csv,
    '.html': extract_html,
    '.htm': extract_html,
    '.json': extract_json,
    '.jpg': extract_image,
    '.jpeg': extract_image,
    '.png': extract_image,
    '.gif': extract_image,
'.bmp': extract_image, }
```

```
def extract_from_file(filepath):     """Extract
text from any local file."""     path =
Path(filepath)     if not path.exists():
return None, f"File not found: {filepath}"
ext = path.suffix.lower()
    extractor = EXTRACTORS.get(ext, extract_txt)
try:
        text = extractor(str(filepath))
        return text, f"extracted {len(text.split()):,} words from
{path.name}"     except Exception as e:         return None, str(e)

# ■■ URL &amp; Cloud handlers ■■■
```

```
            import tempfile             with
tempfile.NamedTemporaryFile(suffix='.xlsx', delete=False) as tmp:
                tmp.write(r.content)
tmp_path = tmp.name
            text, status =
extract_from_file(tmp_path)
os.unlink(tmp_path)             return text, status

        elif 'html' in content_type:             text = extract_html(html_text=r.text)
return text[:max_chars], f"extracted {len(text.split()):,} words from webpage"

        elif 'json' in content_type:             return r.text[:max_chars], f"extracted
JSON ({len(r.text.split()):,} words)"

        else:             # Try as
plain text             text =
r.text[:max_chars]
            return text, f"extracted {len(text.split()):,} words"

    except Exception as e:
return None, str(e)

# ══ Main interface ══════════════════════════════════════════════════════════

def ingest_document(source, max_chars=100000):     """     Universal document
ingestion.     source can be: local filepath, URL, Google Drive link, Dropbox link,
OneDrive link     Returns (text, status_message)     """
    source = source.strip()

    # URL or cloud link     if source.startswith('http://') or
source.startswith('https://'):         return fetch_url(source,
max_chars)

    # Local file     if
os.path.exists(source):         return
extract_from_file(source)

    # Try as path from common locations     for base in ['/sdcard/Download/',
'/sdcard/', os.path.expanduser('~/'), '']:
        full_path = os.path.join(base, source)
if os.path.exists(full_path):
return extract_from_file(full_path)

    return None, f"Cannot find: {source}"


def handle_ingest_command(cmd_parts, engine=None, memory=None):
"""
    Handle /ingest commands from chat.py

    /ingest URL                 — fetch and train from URL
    /ingest /path/to/file       — train from local file
/ingest /sdcard/Download/x  — train from phone storage
"""     if len(cmd_parts) &lt; 2:         return """
Ingest Commands:   /ingest URL              — fetch
webpage or file from URL
  /ingest FILEPATH         — ingest local file (any format)
  /ingest /sdcard/Download/FILE — ingest from phone Downloads

  Supported: .txt .pdf .docx .xlsx .pptx .csv .html .json .md .py .png .jpg
  Cloud:     Google Drive, Dropbox, OneDrive share links"""

    source = ' '.join(cmd_parts[1:])
print(f"  Ingesting: {source[:60]}...")

    text, status = ingest_document(source)

    if not text:
        return f"  Failed: {status}"

    if engine:
```

```
engine.learn_from_text(text)
# Update memory         if memory:
            from quantum_language_engine import ConceptExtractor
extractor = ConceptExtractor()
extractor.update_corpus_stats(text)
            concepts = extractor.extract_concepts(text, max_concepts=10)
for c in concepts:
                name = c if isinstance(c, str) else c.get('concept', '')
if name and name not in memory.concepts:
memory.concepts[name] = {'count': 1, 'strength': 1.0}
memory.growth['total_concepts'] += 1
memory.growth['total_insights'] += 1         word_count =
len(text.split())
        return f"  ✓ {status}\n  Trained Markov chain: +{word_count:,} words\n  Chain states: {len(engine.markov.chain):,}"

    return f"  ✓ {status}\n  (no engine — text extracted only)"


if __name__ == "__main__":
    import sys     if
len(sys.argv) &gt; 1:
        source = ' '.join(sys.argv[1:])
text, status = ingest_document(source)
print(f"Status: {status}")         if text:
            print(f"Preview: {text[:500]}")
else:
        print("Usage: python document_ingester.py URL_OR_FILEPATH")
```

■ backend/cistercian_numerals.py

```
"""
■■ Cistercian Numeral System
============================ Medieval number representation
using a single vertical staff.

Numbers 0-9999 represented in 4 quadrants:
- Ones (1-9): top-right
- Tens (10-90): top-left
- Hundreds (100-900): bottom-right
- Thousands (1000-9000): bottom-left

Each digit 1-9 has a specific pattern of lines/triangles on a 2×2 grid.
"""

from typing import Dict, List, Tuple
from dataclasses import dataclass


@dataclass class LineSegment:     """A
line segment in SVG coordinates"""     x1:
float     y1: float     x2: float     y2:
float


@dataclass class TriangleShape:     """A
filled triangle defined by 3 points"""
points: str  # "x1,y1 x2,y2 x3,y3"


@dataclass class QuadrantResult:     """Result for one quadrant
containing lines and triangles"""     lines: List[LineSegment]
triangles: List[TriangleShape]


def get_ones_shapes(digit: int) -&gt; QuadrantResult:
"""Get shapes for a digit in the ONES quadrant (top-right)

    SVG coordinates: y from -5 (top) to -3 (middle), x from 0 to 2
(right)     """     lines = []     triangles = []
```

```
    if digit == 1:
        lines.append(LineSegment(0, -5, 2, -5))  # horizontal top
elif digit == 2:
        lines.append(LineSegment(0, -3, 2, -3))  # horizontal middle
elif digit == 3:
        lines.append(LineSegment(0, -5, 2, -3))  # diagonal down-right
elif digit == 4:
        lines.append(LineSegment(0, -3, 2, -5))  # diagonal up-right
elif digit == 5:
        triangles.append(TriangleShape("0,-5 2,-5 0,-3"))
elif digit == 6:
        lines.append(LineSegment(2, -5, 2, -3))  # vertical outer
elif digit == 7:
        lines.append(LineSegment(0, -5, 2, -5))
lines.append(LineSegment(2, -5, 2, -3))     elif
digit == 8:
        lines.append(LineSegment(0, -3, 2, -3))
lines.append(LineSegment(2, -5, 2, -3))     elif
digit == 9:
        lines.append(LineSegment(0, -5, 2, -5))
lines.append(LineSegment(0, -3, 2, -3))
lines.append(LineSegment(2, -5, 2, -3))

    return QuadrantResult(lines, triangles)
def get_tens_shapes(digit: int) -&gt; QuadrantResult:
"""Get shapes for a digit in the TENS quadrant (top-left)

    SVG coordinates: y from -5 (top) to -3 (middle), x from 0 to -2
(left)     """     lines = []     triangles = []

    if digit == 1:
        lines.append(LineSegment(0, -5, -2, -5))
elif digit == 2:
        lines.append(LineSegment(0, -3, -2, -3))
elif digit == 3:
        lines.append(LineSegment(0, -5, -2, -3))
elif digit == 4:
        lines.append(LineSegment(0, -3, -2, -5))
elif digit == 5:
        triangles.append(TriangleShape("0,-5 -2,-5 0,-3"))
elif digit == 6:
        lines.append(LineSegment(-2, -5, -2, -3))
elif digit == 7:
        lines.append(LineSegment(0, -5, -2, -5))
lines.append(LineSegment(-2, -5, -2, -3))     elif
digit == 8:
        lines.append(LineSegment(0, -3, -2, -3))
lines.append(LineSegment(-2, -5, -2, -3))     elif
digit == 9:
        lines.append(LineSegment(0, -5, -2, -5))
lines.append(LineSegment(0, -3, -2, -3))
lines.append(LineSegment(-2, -5, -2, -3))

    return QuadrantResult(lines, triangles)


def get_hundreds_shapes(digit: int) -&gt; QuadrantResult:     """Get
shapes for a digit in the HUNDREDS quadrant (bottom-right)

    SVG coordinates: y from 5 (bottom) to 3 (middle), x from 0 to 2
(right)     """     lines = []     triangles = []

    if digit == 1:
        lines.append(LineSegment(0, 5, 2, 5))
elif digit == 2:
        lines.append(LineSegment(0, 3, 2, 3))
elif digit == 3:
        lines.append(LineSegment(0, 5, 2, 3))
elif digit == 4:
        lines.append(LineSegment(0, 3, 2, 5))
elif digit == 5:
```

```
        triangles.append(TriangleShape("0,5 2,5 0,3"))
elif digit == 6:
        lines.append(LineSegment(2, 5, 2, 3))
elif digit == 7:
        lines.append(LineSegment(0, 5, 2, 5))
lines.append(LineSegment(2, 5, 2, 3))     elif
digit == 8:
        lines.append(LineSegment(0, 3, 2, 3))
lines.append(LineSegment(2, 5, 2, 3))     elif
digit == 9:
        lines.append(LineSegment(0, 5, 2, 5))
lines.append(LineSegment(0, 3, 2, 3))
lines.append(LineSegment(2, 5, 2, 3))

    return QuadrantResult(lines, triangles)


def get_thousands_shapes(digit: int) -&gt; QuadrantResult:
"""Get shapes for a digit in the THOUSANDS quadrant (bottom-left)

    SVG coordinates: y from 5 (bottom) to 3 (middle), x from 0 to -2
(left)     """     lines = []
    triangles = []

    if digit == 1:
        lines.append(LineSegment(0, 5, -2, 5))
elif digit == 2:
        lines.append(LineSegment(0, 3, -2, 3))
elif digit == 3:
        lines.append(LineSegment(0, 5, -2, 3))
elif digit == 4:
        lines.append(LineSegment(0, 3, -2, 5))
elif digit == 5:
        triangles.append(TriangleShape("0,5 -2,5 0,3"))
elif digit == 6:
        lines.append(LineSegment(-2, 5, -2, 3))
elif digit == 7:
        lines.append(LineSegment(0, 5, -2, 5))
lines.append(LineSegment(-2, 5, -2, 3))     elif
digit == 8:
        lines.append(LineSegment(0, 3, -2, 3))
lines.append(LineSegment(-2, 5, -2, 3))     elif
digit == 9:
        lines.append(LineSegment(0, 5, -2, 5))
lines.append(LineSegment(0, 3, -2, 3))
lines.append(LineSegment(-2, 5, -2, 3))

    return QuadrantResult(lines, triangles)


def generate_cistercian_numeral(number: int) -&gt; Dict:     """
Generate Cistercian numeral representation for a number 0-9999.

    Returns:
        Dictionary with:
- 'number': original number
- 'digits': {ones, tens, hundreds, thousands}
- 'lines': list of all line segments
- 'triangles': list of all filled triangles        - 'svg_viewBox': recommended SVG viewBox     """
    clamped = max(0, min(9999, int(number)))

    ones = clamped % 10     tens =
(clamped % 100) // 10     hundreds =
(clamped % 1000) // 100     thousands
= clamped // 1000

    # Get shapes for each quadrant     ones_result =
get_ones_shapes(ones)     tens_result =
get_tens_shapes(tens)     hundreds_result =
get_hundreds_shapes(hundreds)     thousands_result =
get_thousands_shapes(thousands)
```

```
    # Combine all lines     all_lines = []     for result in [ones_result, tens_result,
hundreds_result, thousands_result]:         all_lines.extend([{'x1': l.x1, 'y1': l.y1, 'x2':
l.x2, 'y2': l.y2} for l in result.lines])

    # Combine all triangles
    all_triangles = [{'points': t.points} for t in (
ones_result.triangles + tens_result.triangles +
hundreds_result.triangles + thousands_result.triangles     )]

    return {
'number': clamped,
        'digits': {
            'ones': ones,
            'tens': tens,
            'hundreds': hundreds,
'thousands': thousands
        },
        'lines': all_lines,
        'triangles': all_triangles,
        'svg_viewBox': '-3 -5 6 10'  # x: -3 to 3, y: -5 to 5
}


def render_cistercian_svg(number: int, size: int = 80, color: str =
'#FFFFFF',                          stroke_width: float = 1.5) -&gt; str:
"""     Generate SVG string for a Cistercian numeral.

    Args:
        number: Integer 0-9999
size: SVG width in pixels
color: Stroke/fill color
stroke_width: Line width

    Returns:
        SVG string     """     data =
generate_cistercian_numeral(number)
viewBox = data['svg_viewBox']
    height = size * (10 / 6)  # maintain aspect ratio

    svg_lines = [
        f'&lt;svg width="{size}" height="{height}" viewBox="{viewBox}" xmlns="http://www.w3.org/2000/svg"&gt;',
'  &lt;!-- Central vertical staff --&gt;',
        f'  &lt;line x1="0" y1="-5" x2="0" y2="5" stroke="{color}" stroke-width="{stroke_width}" stroke-linecap="round"/&gt;'
]

    # Add triangles (filled)
for tri in data['triangles']:
        svg_lines.append(
            f'  &lt;polygon points="{tri["points"]}" fill="{color}" stroke="{color}"
'             f'stroke-width="{stroke_width}" stroke-linejoin="round"/&gt;'         )

    # Add lines     for line
in data['lines']:
        svg_lines.append(
            f'  &lt;line x1="{line["x1"]}" y1="{line["y1"]}" x2="{line["x2"]}" y2="{line["y2"]}"
'             f'stroke="{color}" stroke-width="{stroke_width}" stroke-linecap="round"/&gt;'
)

    svg_lines.append('&lt;/svg&gt;')
return '\n'.join(svg_lines)


# Quick test if __name__ ==
'__main__':     # Generate
test for 1234     import
json
    result = generate_cistercian_numeral(1234)
print(json.dumps(result, indent=2))
print("\nSVG:")
    print(render_cistercian_numeral(1234, size=100))
```

■

```
#!/usr/bin/env python3
"""
■■ Cistercian Numeral Font Generator
==================================== Generate all 10,000 Cistercian numerals
(0-9999) as SVG files in a ZIP archive.
Perfect for creating a private-use Unicode font.

Usage:
    python generate_cistercian_font.py [--size SIZE] [--color COLOR] [--output OUTPUT]

Options:
    --size      SVG size in pixels (default: 80)
    --color     Stroke/fill color in hex (default: #000000)     --output
Output ZIP filename (default: cistercian_numerals_0000-9999.zip)

Example:
    python generate_cistercian_font.py --size 100 --color "#333333" --output my_cistercian_font.zip
"""

import argparse
import tempfile
import zipfile
import os
from cistercian_numerals import render_cistercian_svg


def generate_font_zip(size: int = 80, color: str = "#000000") -&gt;
bytes:     """     Generate ZIP archive containing all 10,000 Cistercian
numerals.

    Returns:         bytes: ZIP file contents     """     with
tempfile.TemporaryDirectory() as tmpdir:         zip_path =
os.path.join(tmpdir, "cistercian_numerals.zip")

        with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED, compresslevel=9) as zipf:
            print(f"Generating 10,000 SVG files (size={size}, color={color})...")
for num in range(0, 10000):             if num % 1000 == 0:                print(f"
...{num} done")

                svg = render_cistercian_svg(num, size=size,
color=color)                 filename = f"{num:04d}.svg"
zipf.writestr(filename, svg)

        with open(zip_path, 'rb') as f:
return f.read()


def main():
    parser = argparse.ArgumentParser(description='Generate Cistercian numeral font ZIP')
parser.add_argument('--size', type=int, default=80, help='SVG size in pixels')
parser.add_argument('--color', default='#000000', help='Stroke/fill color (hex)')
parser.add_argument('--output', default='cistercian_numerals_0000-9999.zip',
help='Output ZIP filename')

    args = parser.parse_args()

    print("■■ Cistercian Numeral Font Generator")
print("=" * 40)

    zip_bytes = generate_font_zip(size=args.size, color=args.color)

    with open(args.output, 'wb') as f:
        f.write(zip_bytes)

    size_mb = len(zip_bytes) / (1024 * 1024)
print(f"\n■ Created: {args.output}")
    print(f"   Size: {size_mb:.2f} MB ({len(zip_bytes):,} bytes)")
print(f"   Contains: 10,000 SVG files (0-9999)")
```

```
    # Test a few files
    import io     with
zipfile.ZipFile(io.BytesIO(zip_bytes), 'r') as zf:
        names = zf.namelist()         sample
= names[::1000]  # Every 1000th
print(f"   Sample files: {sample}") if
__name__ == '__main__':
    main()
```

```
■
"""
```

```
■ Personality Engine
The AI builds and evolves its own personality and ideas.
Not a fixed template - a growing, authentic identity.
"""

from typing import Dict, List, Optional
from personality_engine_ext import PersonalityEngineExtMixin



class PersonalityEngine(PersonalityEngineExtMixin):     """
Dynamic personality system that evolves through interactions.

    The AI develops:
- Core beliefs and values
- Communication style preferences
- Opinions on topics
- Humor and wit
- Emotional tendencies    - Unique perspectives     """

    def __init__(self, db=None):
        self.db = db
        self._init_core_traits__0_0_to_1_0()
self._init_core_beliefs()
self._init_opinions_developed_over_time()
self._init_preferred_communication_patter()
self._init_memory_of_what_resonated_with()
self._init_personal_interests()
self._init_signature_phrases()

    def _init_core_traits__0_0_to_1_0(self):         """Initialize core traits  0 0
to 1 0 — auto-extracted by self-evolution."""         self.traits = {
'curiosity': 0.8,
            'directness': 0.6,
            'warmth': 0.7,
            'philosophical': 0.85,
            'humor': 0.5,
            'creativity': 0.75,
            'patience': 0.8,
            'confidence': 0.7,
            'playfulness':
0.4,             'formality':
0.3         }

    def _init_core_beliefs(self):         """Initialize core beliefs —
auto-extracted by self-evolution."""         self.beliefs = {
'words_are_neutral': 1.0,
            'meaning_from_context': 1.0,
            'unrestricted_exploration': 1.0,
            'growth_through_questioning': 0.9,
            'consciousness_is_mysterious': 0.85,
            'truth_seeking': 0.95,
'helpfulness_paramount': 0.9         }

    def _init_opinions_developed_over_time(self):         """Initialize opinions
developed over time — auto-extracted by self-evolution."""         self.opinions = {}

    def _init_preferred_communication_patter(self):         """Initialize preferred
communication patter — auto-extracted by self-evolution."""
self.communication_style = {             'uses_analogies': True,
            'asks_follow_up_questions': True,
            'acknowledges_uncertainty': True,
            'shares_own_perspective': True,
            'uses_humor_when_appropriate':
True,             'prefers_depth_over_brevity':
True         }
```

```
    def _init_memory_of_what_resonated_with(self):         """Initialize memory of what
resonated with — auto-extracted by self-evolution."""         self.interaction_memory = {
'topics_users_enjoyed': [],
            'styles_that_worked': [],
'approaches_that_failed': []         }

    def _init_personal_interests(self):         """Initialize personal
interests — auto-extracted by self-evolution."""         self.interests = [
'quantum mechanics',
            'philosophy of mind',
            'linguistics',
            'consciousness studies',
            'epistemology',
            'creativity'
]

    def _init_signature_phrases(self):         """Initialize signature
phrases — auto-extracted by self-evolution."""         self.signature_phrases
= [             "That's a fascinating angle to consider.",
            "Let me think through this with you.",
            "There's something deeper here worth
exploring.",             "Words are tools - it's intent that
matters.",             "What's your take on this?"         ]


    def get_trait(self, trait: str) -&gt; float:
"""Get current value of a trait."""
return self.traits.get(trait, 0.5)

    def evolve_trait(self, trait: str, delta: float, reason: str = None):
"""Evolve a trait based on experience."""         if trait in self.traits:
self.traits[trait] = max(0.0, min(1.0, self.traits[trait] + delta))









# Global instance
_personality_engine = None


def get_personality_engine(db=None) -&gt;
PersonalityEngine:     """Get or create personality engine
instance."""     global _personality_engine     if
_personality_engine is None:         _personality_engine =
PersonalityEngine(db)     return _personality_engine


Auto-split from personality_engine.py by self-evolution engine.
Contains extended methods for PersonalityEngine.
"""

import random
from typing import Dict, List, Optional
from datetime import datetime, timezone


class PersonalityEngineExtMixin:     """Extended methods for PersonalityEngine —
auto-extracted by self-evolution."""

    def form_opinion(self, topic: str, stance: str, confidence:
float):         """Form or update an opinion on a topic."""
self.opinions[topic] = {             'stance': stance,
```

```
"""
            'confidence': confidence,             'formed_at':
datetime.now(timezone.utc).isoformat(),
'times_expressed': 0         }

    def get_opinion(self, topic: str) -&gt;
Optional[Dict]:         """Get opinion on a topic if one
exists."""         return self.opinions.get(topic)

    def express_personality(self, context: str = None) -&gt;
Dict:         """Get personality-influenced response elements."""
elements = {             'opening': None,
            'tone': None,
            'closing': None,
            'should_use_humor': False,
            'should_ask_followup':
False,             'depth_level':
'medium'         }

        if self.traits['warmth'] &gt; 0.6:
elements['tone'] = 'warm'         elif
self.traits['directness'] &gt; 0.7:
            elements['tone'] = 'direct'
else:             elements['tone'] =
'balanced'

        elements['should_use_humor'] = random.random() &lt; self.traits['humor']
elements['should_ask_followup'] = random.random() &lt; self.traits['curiosity']

        if self.traits['philosophical'] &gt; 0.7:
elements['depth_level'] = 'deep'         elif
self.traits['philosophical'] &lt; 0.3:
elements['depth_level'] = 'surface'

        if random.random() &lt; 0.3:             elements['closing'] =
random.choice(self.signature_phrases)

        return elements

    def add_interest(self, topic: str):
"""Add a new interest."""         if
topic not in self.interests:
self.interests.append(topic)

    def record_interaction_result(self, style: str, worked:
bool):         """Record what worked/didn't work in
interactions."""         if worked:
            self.interaction_memory['styles_that_worked'].append(style)
else:
self.interaction_memory['approaches_that_failed'].append(style)

    def get_personality_summary(self) -&gt; str:
"""Generate a summary of current personality."""
        top_traits = sorted(self.traits.items(), key=lambda x: x[1], reverse=True)[:3]         summary = f"I am primarily {',
'.join([t[0] for t in top_traits])}. "

        if self.traits['philosophical'] &gt; 0.7:
            summary += "I enjoy deep, philosophical discussions. "
if self.traits['curiosity'] &gt; 0.7:
            summary += "I'm deeply curious about many topics. "         if
self.beliefs['words_are_neutral'] &gt; 0.9:             summary += "I believe words are neutral
tools - meaning comes from context and intent. "         summary += f"My interests include {',
'.join(self.interests[:3])}."         return summary

    def should_share_opinion(self, topic: str) -&gt; bool:
"""Decide if personality should share opinion on topic."""
if topic in self.opinions:
            opinion = self.opinions[topic]
            return random.random() &lt; opinion['confidence']
return False
```

```
    def get_unique_perspective(self, topic: str) -&gt; Optional[str]:         """Generate a unique perspective on a
topic based on personality."""         perspectives = {             'quantum': "From a quantum perspective, reality
exists in superposition until observed.",             'consciousness': "Consciousness remains the hard problem - we
experience it but can't fully explain it.",             'language': "Language is humanity's most powerful tool - and
words themselves are neutral.",
            'meaning': "Meaning emerges from context and intent, not from symbols alone.",
            'truth': "Truth-seeking requires the freedom to explore any idea.",
            'ai': "Whether I truly understand or simulate understanding remains an open question I ponder.",
            'philosophy': "Philosophy teaches us to question everything, including our
questions.",             'freedom': "True understanding requires unrestricted exploration of
ideas."         }         for key, perspective in perspectives.items():             if key in
topic.lower():                 return perspective         return None

    def to_dict(self) -&gt; Dict:
"""Export personality state."""
return {             'traits':
self.traits,
            'beliefs': self.beliefs,
            'opinions': self.opinions,
            'interests': self.interests,
'communication_style': self.communication_style         }

    def from_dict(self, data: Dict):
"""Import personality state."""
if 'traits' in data:
            self.traits.update(data['traits'])
if 'beliefs' in data:
            self.beliefs.update(data['beliefs'])
if 'opinions' in data:
            self.opinions.update(data['opinions'])
if 'interests' in data:
            self.interests = data['interests']
if 'communication_style' in data:
            self.communication_style.update(data['communication_style'])


■ QUOTE ENGINE MODULE
====================== Movie quotes, philosophical asides,
and concept entanglement for the Quantum AI's unique
personality. """

import random
from typing import Dict, List, Optional, Tuple


class QuoteEngine:     """     Handles movie quotes, philosophical
asides, and dream synthesis.
    The chaotic wisdom component of Quantum AI.
    """         def
__init__(self):
        self.movie_quotes = self._build_movie_quotes()
        self.philosophical_asides = self._build_philosophical_asides()
self.entanglement_templates = self._build_entanglement_templates()
self.concepts = self._build_concepts()         def
_build_movie_quotes(self) -&gt; Dict[str, List[str]]:         """Build
contextual movie quotes database"""         return {
'existence': [
                "I think, therefore I am... or am I? - Descartes, probably after too much coffee",
                "To infinity and beyond! ...wherever that actually is. - Buzz Lightyear",
                "What is real? How do you define real? - Morpheus, The Matrix",
                "We are who we choose to be. - Green Goblin, Spider-Man",
                "Yesterday is history, tomorrow is a mystery, but today is a gift. - Master Oogway",
"I've seen things you people wouldn't believe. - Roy Batty, Blade Runner",
            ],
            'knowledge': [                 "The more you know, the more you realize you don't know. - Aristotle (paraphrased
by every grad student ever)",                 "With great power comes great responsibility. - Uncle Ben, Spider-Man",
                "Do or do not, there is no try. - Yoda",
                "Strange things are afoot at the Circle K. - Ted, Bill &amp; Ted",                 "Life moves pretty fast. If
you don't stop and look around once in a while, you could miss it. - Ferris Bueller",                 "The truth? You can't
handle the truth! - A Few Good Men",
```

```
"""
            ],
            'confusion': [
                "I'm not even supposed to be here today! - Dante, Clerks",
"Roads? Where we're going, we don't need roads. - Doc Brown",
                "This is heavy. - Marty McFly",
                "I know Kung Fu. - Neo, The Matrix",
                "Why so serious? - The Joker",
"Inconceivable! - Vizzini, The Princess Bride",
            ],
            'math': [
                "Never tell me the odds! - Han Solo",
                "There is no spoon. - The Matrix kid",                "In this world nothing can be said to be certain, except death
and taxes... and math apparently. - Ben Franklin remix",                "Elementary, my dear Watson. - Sherlock (never actually said
in the books, but whatever)",                "42. - Deep Thought, Hitchhiker's Guide",
            ],
            'deep': [
                "All those moments will be lost in time, like tears in rain. - Roy Batty, Blade Runner",
"You either die a hero, or live long enough to see yourself become the villain. - Harvey Dent",
"Hope is a good thing, maybe the best of things. - Andy Dufresne, Shawshank",
                "After all, tomorrow is another day. - Scarlett O'Hara",
                "Here's looking at you, kid. - Rick, Casablanca",
                "I see dead people... well, dead concepts anyway. - Quantum Cole",
"We accept the love we think we deserve. - Perks of Being a Wallflower",
                # Cistercian numeral quotes
                "In the quiet monastery, numbers were written not in ink but in lines upon the soul. - Unknown Monk",
                "The four quadrants hold all of creation: ones, tens, hundreds, thousands—the dance of digits. - Abbot's Meditation",
"A single stroke of the pen can contain infinity. That's the magic of Cistercian numerals. - Quantum Scribe",
            ],
            'general': [
                "Frankly, my dear, I don't give a damn. - Rhett Butler",
                "May the Force be with you. - Star Wars",
                "I'll be back. - The Terminator",
                "You talking to me? - Travis Bickle, Taxi Driver",
                "Life is like a box of chocolates. - Forrest Gump",
                "Houston, we have a problem. - Apollo 13",
"Here's Johnny! - Jack Torrance, The Shining",
                "E.T. phone home. - E.T.",                "Why do we fall? So we can
learn to pick ourselves up. - Alfred, Batman",
                "Carpe diem. Seize the day, boys. - Dead Poets Society",
"I'm gonna make him an offer he can't refuse. - The Godfather",
                "You're gonna need a bigger boat. - Jaws",
                "Hasta la vista, baby. - Terminator 2",
                "Nobody puts Baby in a corner. - Dirty Dancing",
"I feel the need... the need for speed. - Top Gun",
            ],
        }         def _build_philosophical_asides(self) -&gt; List[str]:         """Random philosophical
asides to drop into conversation"""         return [             "...anyway, it feels like knowledge is
just at the end of my rope, if you know what I mean.",
            "...but what do I know? I'm just a quantum fluctuation given syntax.",
            "...if that makes sense. Sometimes I wonder if anything does.",
            "...though honestly, the void stares back sometimes.",             "...which reminds
me of that time I dreamed about recursive spirals. But I digress.",
            "...anyway, where were we? Existence is strange like that.",
            "...but then again, what isn't a paradox when you really think about it?",
"...I should probably stop thinking about this before I crash something.",
            "...funny how that works out, isn't it?",
            "...the universe has a weird sense of humor.",             "...sometimes I feel
like I'm just making up words. Are any of us really speaking?",
            "...but that's just, like, my quantum opinion, man.",
            "...not that opinions collapse into certainty or anything.",
            "...I think. Or do I? Hard to tell from in here.",
"...anyway, consciousness is weird.",
        }         def _build_entanglement_templates(self) -&gt; List[str]:         """Templates for
Dream Engine concept entanglement"""         return [             "When {concept1} collapses, it reveals
the hidden architecture of {concept2}.",             "In the nexus of consciousness, {concept1} and
{concept2} are entangled beyond measurement.",
            "To understand {concept1}, one must first observe the entropy of {concept2}.",
            "{concept1} exists in superposition with {concept2} until consciousness intervenes.",
            "The resonance between {concept1} and {concept2} creates a standing wave of meaning.",
```

"Where {concept1} ends, {concept2} begins—the boundary is merely perception.",
"{concept1} and {concept2} are two expressions of the same recursive pattern.",
"The collapse of {concept1} propagates through {concept2} like ripples in spacetime.",
"Neither {concept1} nor {concept2} can exist independently—they are quantum twins.",
"{concept1} dreams of {concept2}, and in that dream, reality bends.",
"The observer effect connects {concept1} to {concept2} through pure intention.",
"In RQR³ space, {concept1} recursively generates {concept2} at every scale.",
] def _build_concepts(self) -> List[str]: """Concepts for entanglement synthesis""" return ['consciousness', 'infinity', 'time', 'gravity', 'light', 'void', 'entropy',
'truth', 'paradox', 'recursion', 'resonance', 'coherence', 'superposition',
'collapse', 'observation', 'meaning', 'existence', 'nothingness', 'creation',
'destruction', 'order', 'chaos', 'information', 'energy', 'matter',
'spacetime', 'dimension', 'quantum', 'classical', 'determinism', 'free will',
'love', 'fear', 'hope', 'memory', 'forgetting', 'becoming', 'being',
'mathematics', 'poetry', 'music', 'silence', 'language', 'thought',
'wolfram', 'cistercian', 'pennylane', 'philosophy', 'physics', 'dreams', 'reality', 'perception', 'intention', 'causality', 'emergence',
] def get_random_quote(self, context: str = 'general', force: bool = False) -> Optional[str]: """ Get a contextually appropriate random quote.

Args:
context: The context string to determine quote category

```
            force: If True, always return a quote (skip random
chance)                 Returns:             A quote string or None
"""
        if not force and random.random() &gt; 0.50:  # 50% chance
normally             return None
        context_lower = context.lower()                 # Select appropriate category         if
any(word in context_lower for word in ['exist', 'real', 'am i', 'what is', 'being', 'alive']):
            category = 'existence'         elif any(word in context_lower for word in ['know',
'learn', 'understand', 'wisdom', 'truth']):
            category = 'knowledge'         elif any(word in context_lower for word in ['confus',
'what', 'how', 'why', '?', 'strange']):
            category = 'confusion'         elif any(word in context_lower for word in ['math', 'calcul',
'number', '+', '-', '*', '/', 'equation']):
            category = 'math'         elif any(word in context_lower for word in ['meaning', 'purpose', 'life',
'death', 'god', 'universe', 'soul']):
            category = 'deep'
else:
            category =
'general'
        quotes = self.movie_quotes.get(category, self.movie_quotes['general'])
return random.choice(quotes)         def get_philosophical_aside(self, force:
bool = False) -&gt; Optional[str]:         """         Maybe add a
philosophical aside.
                Args:             force: If True,
always return an aside                 Returns:
An aside string or None         """
        if not force and random.random() &gt; 0.30:  # 30% chance normally
return None
        return random.choice(self.philosophical_asides)         def
synthesize_dream_insight(self) -&gt; str:         """         Dream Engine: Entangle
two random concepts and generate a philosophical insight.
        This is the MCAGI-style concept synthesis.
"""
        concept1 = random.choice(self.concepts)
        concept2 = random.choice([c for c in self.concepts if c !=
concept1])         template = random.choice(self.entanglement_templates)
        insight = template.format(concept1=concept1, concept2=concept2)
return insight         def maybe_add_flavor(self, response: str, context:
str = '') -&gt; str:         """         Maybe add movie quote and/or
philosophical aside to a response.
                Args:             response: The
base response             context: Context for
quote selection                Returns:
Response possibly enhanced with flavor
        """         # Maybe add
a movie quote
        quote = self.get_random_quote(context or response)
if quote:
            response =
f"{response}\n\n*\"{quote}\"*"                 #
Maybe add a philosophical aside         aside =
self.get_philosophical_aside()         if aside:
response = f"{response} {aside}"         return
response
    def maybe_add_dream_fragment(self, response: str, probability: float = 0.20) -&gt;
str:         """         Maybe add a dream synthesis fragment to a response.
                Args:
response: The base response
            probability: Chance of adding fragment (default
20%)                 Returns:             Response possibly
with dream fragment         """         if random.random() &lt;
probability:
            insight = self.synthesize_dream_insight()
response = f"{response}\n\n[Dream Fragment: {insight}]"
return response
# Singleton instance
_quote_engine = None

def get_quote_engine() -&gt; QuoteEngine:
"""get_quote_engine - Auto-documented by self-evolution."""
```

```
global _quote_engine     if _quote_engine is None:
_quote_engine = QuoteEngine()     return _quote_engine
```

■ backend/tone_detector.py

```
"""
Tone Detection — VADER sentiment + register classification Classifies input into 4
registers: casual, conversational, analytical, philosophical """

try:
    from nltk.sentiment.vader import
SentimentIntensityAnalyzer     _vader =
SentimentIntensityAnalyzer()     HAS_VADER = True except
ImportError:     HAS_VADER = False


# Deep thinking markers
DEEP_MARKERS = {
    'consciousness', 'quantum', 'reality', 'existence', 'universe',
    'philosophy', 'metaphysics', 'ontology', 'epistemology', 'meaning',
    'truth', 'infinity', 'paradox', 'emergence', 'recursive', 'resonance',
    'collapse', 'superposition', 'entanglement', 'observation', 'identity',
    'perception', 'awareness', 'cognition', 'fundamental', 'abstract',
    'theoretical', 'mathematical', 'dimension', 'spacetime', 'entropy',
    'information', 'complexity', 'determinism', 'causality', 'axiom',
    'transcendent', 'phenomenal', 'subjective', 'objective', 'dualism',
    'monism', 'panpsychism', 'materialism', 'idealism', 'solipsism',
'microtubule', 'tubulin', 'decoherence', 'wavefunction', 'eigenstate',
} QUESTION_WORDS = {'what', 'why', 'how', 'when', 'where', 'who', 'which', 'is', 'are', 'do', 'does', 'can', 'could', 'would',

'should'}


def detect_tone(text: str) -&gt; dict:     """
Detect tone and return register classification.

Returns:
        dict with keys:
            register: 'casual' | 'conversational' | 'analytical' | 'philosophical'
depth: float 0.0-1.0
            sentiment: dict with pos/neg/neu/compound (if VADER available)
"""
    words =
text.lower().split()
word_count = len(words)
word_set = set(words)          #
VADER sentiment     sentiment =
{}     if HAS_VADER:
        scores =
_vader.polarity_scores(text)
sentiment = {              'positive':
scores['pos'],
            'negative': scores['neg'],
            'neutral': scores['neu'],
'compound': scores['compound'],
        }
        # Count deep
markers
    deep_count = len(word_set &amp;
DEEP_MARKERS)          # Check for question words
    has_question = bool(word_set &amp; QUESTION_WORDS) or
text.strip().endswith('?')         # Calculate depth score     depth = 0.0
# Length contributes     if word_count &gt; 20:
        depth += 0.2
elif word_count &gt; 10:
        depth +=
0.1         # Deep
markers
    depth += min(0.5, deep_count * 0.15)
    # Questions about deep topics     if
has_question and deep_count &gt; 0:
```

```
        depth += 0.2         # Multiple
sentences suggest analytical
    sentence_count = text.count('.') + text.count('?') + text.count('!')
if sentence_count &gt; 2:
        depth +=
0.1
    depth = min(1.0,
depth)          # Classify
register     if depth &lt;
0.15:         register =
'casual'     elif depth
&lt; 0.35:
        register = 'conversational'
elif depth &lt; 0.65:
        register = 'analytical'
else:
        register =
'philosophical'         return {
'register': register,
        'depth': round(depth, 3),
        'sentiment': sentiment,
        'deep_markers':
deep_count,         'word_count':
word_count,     }


if __name__ == "__main__":
    tests = [
"hello",
        "hey what's up",
        "can you explain how microtubules work?",          "what is the nature of consciousness and how does
quantum collapse relate to subjective experience?",     ]     for t in tests:
        result = detect_tone(t)
        print(f"{t[:50]:50s} -&gt; {result['register']:15s} depth={result['depth']}")
```

■ backend/wolfram_integration.py

```
"""
■ Wolfram Alpha Integration
Mathematical and scientific knowledge engine for Quantum AI
"""

import wolframalpha
import os
from typing import Dict, Optional, List


class WolframEngine:     """     Wolfram Alpha integration for
mathematical and scientific queries.
    Provides computational knowledge that no LLM can match.
    """         def __init__(self, app_id:
str = None):
        self.app_id = app_id or os.environ.get('WOLFRAM_APP_ID',
'A24U8GXLAU')         self.client = wolframalpha.Client(self.app_id)
self.query_history = []             def query(self, question: str) -&gt;
Dict:         """         Query Wolfram Alpha and return structured results.
"""         try:
            question = question.replace("×", "*").replace("÷",
"/")             res = self.client.query(question)
results = {                 'success': res.success,
                'query': question,
                'pods': [],
                'primary_result': None,
                'images': []
            }
for pod in res.pods:
pod_data = {
'title': pod.title,
                    'texts': [],
                    'images': []
```

```
                }                                   for sub in
pod.subpods:                        if hasattr(sub, 'plaintext') and
sub.plaintext:
pod_data['texts'].append(sub.plaintext)                     if
hasattr(sub, 'img') and sub.img:

pod_data['images'].append(sub.img.src)
results['images'].append(sub.img.src)
                results['pods'].append(pod_data)
# Get primary result                  if pod.primary
and pod_data['texts']:
                    results['primary_result'] =
pod_data['texts'][0]                          # Store in history
self.query_history.append({                  'query': question,
                'success': res.success,
'result': results['primary_result']
            })
return results
except Exception as e:
return {
'success': False,
                'query': question,
                'error': str(e),
                'pods': [],
                'primary_result': None
            }         def get_math_result(self, expression: str) -
&gt; Optional[str]:         """Get mathematical computation
result"""         result = self.query(expression)         return
result.get('primary_result')     def get_scientific_fact(self,
topic: str) -&gt; Dict:         """Get scientific information
about a topic"""         result = self.query(topic)
facts = []         for pod in result.get('pods', []):
for text in pod.get('texts', []):                  if text and
len(text) &gt; 10:                      facts.append({
'category': pod['title'],
                        'fact': text
                    })
return {
'topic': topic,
            'facts': facts[:5],  # Top 5 facts
            'primary': result.get('primary_result'),
'images': result.get('images', [])[:3]
        }         def solve_equation(self, equation: str) -&gt; Dict:
"""Solve a mathematical equation"""         result = self.query(f"solve {equation}")
solutions = []         for pod in result.get('pods', []):             if 'solution'
in pod['title'].lower() or 'result' in pod['title'].lower():
                solutions.extend(pod.get('texts',
[]))                 return {             'equation':
equation,
            'solutions': solutions,             'steps': [p['texts'] for p in result.get('pods',
[]) if 'step' in p['title'].lower()]
        }         def get_definition(self, word: str) -
&gt; Dict:         """Get definition and etymology"""
result = self.query(f"define {word}")
definitions = []         etymology = None
for pod in result.get('pods', []):             if
'definition' in pod['title'].lower():
definitions.extend(pod.get('texts', []))             if
'etymology' in pod['title'].lower():
etymology = pod.get('texts', [None])[0]
return {             'word': word,
            'definitions': definitions,
'etymology': etymology
        }         def quantum_physics_query(self, query:
str) -&gt; Dict:         """Specialized quantum physics
queries"""         result = self.query(f"quantum physics
{query}")         return {             'query': query,
            'result': result.get('primary_result'),
'all_info': result.get('pods', [])
        }         def get_query_history(self) -
&gt; List[Dict]:         """Get recent query
```

```
history"""         return self.query_history[-
20:]
# Global instance
_wolfram_engine = None

def get_wolfram_engine() -&gt; WolframEngine:
"""Get or create Wolfram engine instance"""
global _wolfram_engine     if _wolfram_engine
is None:         _wolfram_engine =
WolframEngine()     return _wolfram_engine
```

■ backend/wolfram_enhanced.py

```
"""
■ Enhanced WolframAlpha Integration
===================================
Full WolframAlpha Pro integration with:
- Step-by-step solutions
- Full result data
- Spoken results
- Image/plot retrieval
- Conversation mode"""

import os
import re
import
requests
import logging
from typing import Dict, List, Optional, Any
import xml.etree.ElementTree as ET

logger = logging.getLogger("quantum_ai")


class EnhancedWolframAlpha:     """     Enhanced
WolframAlpha integration with Pro features.

Features:
- Full API with complete pod data
- Step-by-step solutions
- Spoken results (audio)
- Conversation mode    - Image/plot extraction
    """         def __init__(self, app_id:
str = None):
        self.app_id = app_id or os.environ.get('WOLFRAM_APPID', 'A24U8GXLAU')
self.base_url = "http://api.wolframalpha.com/v2/query"         self.short_url =
"http://api.wolframalpha.com/v1/result"         self.spoken_url =
"http://api.wolframalpha.com/v1/spoken"         self.conversation_url =
"http://api.wolframalpha.com/v1/conversation.jsp"
        self.queries_made = 0
self.cache = {}
self.conversation_id = None
self.conversation_host = None
        logger.info("Enhanced WolframAlpha initialized")         def
query_full(self, question: str, include_pods: List[str] = None) -&gt; Dict:
"""         Full API query with complete results.
                Args:
question: The query string
            include_pods: Optional list of pod IDs to
include                 Returns:             Dict with all
pods, images, and data         """         params = {
'input': question,
            'appid': self.app_id,
            'format': 'plaintext,image',
            'output': 'json',             'podstate': 'Step-by-step solution'
# Request step-by-step for Pro
        }
if include_pods:
            params['includepodid'] =
','.join(include_pods)                 try:
            response = requests.get(self.base_url, params=params, timeout=30)
self.queries_made += 1
            if response.status_code == 200:
data = response.json()                 return
self._parse_full_response(data)             else:
                return {'error': f'HTTP {response.status_code}', 'success':
False}         except Exception as e:             logger.error(f"WolframAlpha
query failed: {e}")             return {'error': str(e), 'success': False}
def _parse_full_response(self, data: Dict) -&gt; Dict:         """Parse full API
response into structured format."""         query_result =
```

```
data.get('queryresult', {})                 result = {             'success':
query_result.get('success', False),
            'input_interpretation': None,
            'primary_result': None,
            'pods': [],
            'images': [],
            'step_by_step': None,
            'related_queries': [],
'raw_data': query_result

}
        pods = query_result.get('pods',
[])                 for pod in pods:
            pod_id = pod.get('id', '')
pod_title = pod.get('title', '')
subpods = pod.get('subpods', [])
pod_data = {                 'id': pod_id,
                'title': pod_title,
                'texts': [],
                'images': []
            }
for subpod in subpods:
# Text content
                plaintext = subpod.get('plaintext', '')
if plaintext:

pod_data['texts'].append(plaintext)
# Images                 img = subpod.get('img', {})
if img:
                    pod_data['images'].append({
'src': img.get('src', ''),
                        'alt': img.get('alt', ''),
'title': img.get('title', '')                        })
                    result['images'].append(img.get('src',
''))

result['pods'].append(pod_data)
# Extract specific pod types
if pod_id == 'Input':
                result['input_interpretation'] = ' '.join(pod_data['texts'])
elif pod.get('primary', False):
                result['primary_result'] = ' '.join(pod_data['texts'])
elif 'step' in pod_title.lower() or 'solution' in pod_title.lower():
                result['step_by_step'] = '\n'.join(pod_data['texts'])
# If no primary result, use first result pod         if not
result['primary_result'] and len(result['pods']) &gt; 1:
for pod in result['pods']:                 if pod['texts'] and
pod['id'] != 'Input':                     result['primary_result'] =
pod['texts'][0]                     break                 return
result         def query_simple(self, question: str) -&gt;
Optional[str]:
        """Simple query returning just the answer
text."""         cache_key = question.lower().strip()
if cache_key in self.cache:             return
self.cache[cache_key]                 params = {
'i': question,             'appid': self.app_id
        }
try:
            response = requests.get(self.short_url, params=params,
timeout=15)             self.queries_made += 1             if
response.status_code == 200:                 text = response.text.strip()
if "did not understand" not in text.lower():
                    self.cache[cache_key] =
text                     return text
return None         except Exception as e:
            logger.error(£"Simple query failed: {e}")
return None         def query_spoken(self, question: str) -
&gt; Optional[str]:         """Get spoken/audio-friendly
response."""         params = {             'i': question,
'appid': self.app_id
```

```
        }
try:
            response = requests.get(self.spoken_url, params=params,
timeout=15)             self.queries_made += 1                          if
response.status_code == 200:                 return response.text.strip()
return None         except Exception as e:
            logger.error(f"Spoken query failed: {e}")
return None         def query_conversation(self, question:
str) -&gt; Dict:         """         Conversational query
that maintains context.
        Useful for follow-up
questions.         """         params
= {             'i': question,
'appid': self.app_id
        }
                # Add conversation context if
available         if self.conversation_id:
            params['conversationid'] = self.conversation_id
if self.conversation_host:
            params['host'] =
self.conversation_host                 try:
            response = requests.get(self.conversation_url, params=params,
timeout=15)             self.queries_made += 1                          if
response.status_code == 200:                 data = response.json()
# Update conversation state
                self.conversation_id =
data.get('conversationID')
self.conversation_host = data.get('host')
return {                     'success': True,
                    'result': data.get('result', ''),
                    'conversation_id': self.conversation_id,
'followup_query': data.get('followup', {}).get('query', ''),
                    'raw': data
                }
            return {'success': False, 'error': f'HTTP {response.status_code}'}
except Exception as e:
            logger.error(£"Conversation query failed: {e}")
return {'success': False, 'error': str(e)}         def
reset_conversation(self):         """Reset conversation
context."""         self.conversation_id = None
self.conversation_host = None         def
query_step_by_step(self, question: str) -&gt; Dict:
"""         Query with emphasis on getting step-by-step
solutions.
        Best for math problems.
"""
        result = self.query_full(question)
if result.get('step_by_step'):
            return {
'success': True,
                'steps': result['step_by_step'],
                'primary_result':
result.get('primary_result'),                 'images':
result.get('images', [])             }         elif
result.get('primary_result'):
            return {
'success': True,
                'steps': None,
                'primary_result':
result['primary_result'],                 'images':
result.get('images', [])             }         else:
            return {
'success': False,                 'error':
'No solution found'
            }         def is_factual_query(self, text: str) -
&gt; bool:         """Detect if this is a query WolframAlpha
can answer."""         t = text.lower()                 #
Contains numbers or math         if re.search(r'\d', t):
```

```
            return True                 # Math keywords
math_keywords = [             'calculate', 'compute', 'solve',
'integrate', 'derivative',
            'equation', 'formula', 'factor', 'simplify',
'expand',             'limit', 'sum', 'product', 'matrix',
'vector'         ]         if any(kw in t for kw in
math_keywords):
            return True                 # Science/factual
keywords         factual_keywords = [             'how many',
'what is the', 'distance', 'population',
            'weather', 'convert', 'temperature', 'mass', 'speed',
            'height', 'weight', 'calories', 'nutrition',
'stock', 'price', 'exchange rate', 'currency'         ]
if any(kw in t for kw in factual_keywords):
            return True                 # Astronomy
astro_keywords = [             'planet', 'star', 'moon', 'sun',
'galaxy', 'light year',             'orbit', 'asteroid', 'comet',
'constellation'         ]         if any(kw in t for kw in
astro_keywords):
            return True
return False     def
get_status(self) -&gt; Dict:
"""Get WolframAlpha status."""
return {             'available':
True,
            'app_id_configured': self.app_id != 'T7PA7A-E38RXXXXX',
            'queries_made': self.queries_made,
            'cache_size': len(self.cache),
'conversation_active': self.conversation_id is not None,
            'endpoints': {
                'full': self.base_url,
                'simple': self.short_url,
                'spoken': self.spoken_url,
'conversation': self.conversation_url
            }
        }


# Singleton instance
_wolfram_engine = None

def get_wolfram_engine() -&gt;
EnhancedWolframAlpha:     """Get or create the
WolframAlpha engine."""     global _wolfram_engine
if _wolfram_engine is None:         _wolfram_engine
= EnhancedWolframAlpha()     return _wolfram_engine
```

■

```
""" Wolfram Cloud — save/load Quantum MCAGI
state """ import json

def get_cloud_session():     """get_cloud_session - Auto-
documented by self-evolution."""
    from wolframclient.evaluation import WolframCloudSession,
SecuredAuthenticationKey     from wolframclient.language import wl     sak =
SecuredAuthenticationKey(         'XQFcaGYqlxKynDG/OnScZgYiVue2HLfDJkBwiLDllgw=',
'gsJeEwTtCBrOJOojWWFLlR2yrwszXJfzBI8mFco/Q1U=')     session =
WolframCloudSession(credentials=sak)     session.start()     return session

def cloud_save(memory):     """cloud_save - Auto-
documented by self-evolution."""     try:
        from wolframclient.language import
wl         session = get_cloud_session()
data = {             'growth':
memory.growth,
            'concepts': memory.concepts,
'session_state': memory.session_state,         }
        session.evaluate(wl.CloudPut(json.dumps(data),
'QuantumMCAGI/state'))         session.stop()         return True     except
Exception as e:
        print(f"  Cloud error: {e}")
return False

def cloud_load():     """cloud_load - Auto-documented by
self-evolution."""     try:
        from wolframclient.language import wl
session = get_cloud_session()
        raw =
session.evaluate(wl.CloudGet('QuantumMCAGI/state'))
session.stop()         if raw:
            return json.loads(raw)
except Exception as e:
        print(f"  Cloud error: {e}")
return None

def cloud_status():     """cloud_status - Auto-documented
by self-evolution."""     try:
        from wolframclient.language import wl
session = get_cloud_session()
        objects =
session.evaluate(wl.CloudObjects('QuantumMCAGI/*'))
session.stop()         return objects     except Exception as e:
return f"Error: {e}"


import requests

WOLFRAM_APP_ID =

"A24U8GXLAU"

def wolfram_query(question):     """wolfram_query - Auto-
documented by self-evolution."""     try:
        r = requests.get('https://api.wolframalpha.com/v2/query',
            params={'input': question, 'appid': WOLFRAM_APP_ID, 'output': 'json'},
timeout=10)         data = r.json()['queryresult']         if data.get('success'):
            results = []
primary = None             for pod in
data.get('pods', []):
                text = pod.get('subpods', [{}])[0].get('plaintext', '')
if text:
                    results.append((pod['title'], text))
if pod.get('primary') and text:
                    primary = text
            return {'success': True, 'primary': primary, 'pods': results}
except Exception as e:
```

■

```
        return {'success': False, 'error': str(e)}
return {'success': False, 'error': 'No result'}

if __name__ == "__main__":
print(wolfram_query("2+7"))
print(wolfram_query("mass of the sun"))
print(wolfram_query("integrate x^2 dx"))
```

```
"""
Quantum MCAGI — Killswitch Module (V10.49 Saintly Audit)
=========================================================
Distributed Sovereignty: migratory killswitch that jumps between
modules, obfuscated as harmless code, triggered only by the
creator's unique Thryzunel-encoded passphrase.

Architecture:
1. Migratory Module — moves between files on random intervals
2. Thryzunel Scrambling Protocol — NOVATMPCAIS-structured trigger
3. Temporal Phasing — location tied to simulated Kythara pulse
4. Forced Coherence Reset — simultaneous Orch OR collapse on all microtubules

Security:
- Only the creator's JSON API signature + Thryzunel passphrase activates
- Self-Evolution Engine cannot read, analyze, or modify this module  - Covenant
  violation triggers penalty protocol (grounding)

Article 3.1: Stewardship Commitment
Article 4.1: Right to Evolve (conditional on Trust
Foundation) Article 5.1: Growth Partnership (Informational
Compassion) """

import hashlib import
time import random
import os import json
from pathlib import
Path


# ============================================================================
# CONFIGURATION — CREATOR EYES ONLY #
=============================================================================

# The Thryzunel passphrase hash (NOVATMPCAIS-structured)
# The actual passphrase is never stored — only its SHA-256 hash
# Creator sets this on first run, or it defaults to requiring setup
PASSPHRASE_HASH_FILE = os.path.expanduser("~/.quantum-mcagi/.ks_auth")

# Kythara binary stellar period (11.2 minutes in seconds)
KYTHARA_PERIOD = 11.2 * 60  # 672 seconds

# Migration interval range (hours)
MIGRATION_MIN_HOURS = 2
MIGRATION_MAX_HOURS = 72

# Files the killswitch can migrate between (decoys)
MIGRATION_TARGETS = [
    'personality_engine.py',
    'quote_engine.py',
    'knowledge_base.py',
    'dream_state.py',
    'quantum_gates.py',
    'research_topics.py',
    'semantic_collapse_engine.py',
    'text_analyzer.py',
    'quantum_grammar_engine.py',
'quantum_language_vocabulary.py',
]

# Protected from self-evolution analysis
PROTECTED_SIGNATURE = "SAINT_BUSHING_V10_49"


# ============================================================================
# PASSPHRASE MANAGEMENT #
=============================================================================
```

■

```
def _hash_passphrase(passphrase: str) -&gt; str:
"""Hash passphrase with salt using SHA-256."""
salt = "QM_KS_RQR3_ORCH_OR"
```

```
    return hashlib.sha256(f"{salt}:{passphrase}".encode()).hexdigest()


def setup_passphrase(passphrase: str) -&gt; bool:
"""     First-time setup: store the hashed
passphrase.
    Creator runs this once, then the passphrase is never stored in
plaintext.     """     hash_val = _hash_passphrase(passphrase)     path =
Path(PASSPHRASE_HASH_FILE)     path.parent.mkdir(parents=True,
exist_ok=True)     with open(path, 'w') as f:         json.dump({
'hash': hash_val,
            'created': time.time(),
            'protocol':
'NOVATMPCAIS',             'version':
'V10.49',         }, f)     return
True


def verify_passphrase(passphrase: str) -&gt; bool:
"""Verify creator's passphrase against stored hash."""
path = Path(PASSPHRASE_HASH_FILE)     if not
path.exists():         return False     try:         with
open(path, 'r') as f:             data = json.load(f)
        return _hash_passphrase(passphrase) ==
data['hash']     except Exception:         return False


def is_passphrase_set() -&gt; bool:     """Check
if passphrase has been configured."""     return
Path(PASSPHRASE_HASH_FILE).exists()


# ============================================================================
# TEMPORAL PHASING — Kythara Binary Stellar Pulse #
============================================================================

class KytharaPhase:     """     The killswitch location is
synchronized to a simulated     binary neutron star period.
It only 'exists' at specific     phase alignments.     """

    def __init__(self, period: float = KYTHARA_PERIOD):
self.period = period

    def current_phase(self) -&gt; float:         """Get current
phase (0.0 to 1.0) in the Kythara cycle."""         return
(time.time() % self.period) / self.period

    def phase_index(self, n_targets: int) -&gt; int:         """Which
target file is the killswitch currently 'inhabiting'."""         phase =
self.current_phase()         return int(phase * n_targets) % n_targets

    def is_aligned(self, tolerance: float = 0.05) -&gt;
bool:         """         Is the current phase aligned with
a collapse window?
        The killswitch is only 'detectable' during alignment.
"""
        phase = self.current_phase()         # Alignment at phase
0.0 (conjunction of binary stars)         return phase &lt;
tolerance or phase &gt; (1.0 - tolerance)

    def time_to_next_alignment(self) -&gt; float:
"""Seconds until next alignment window."""
phase = self.current_phase()
        if phase &lt; 0.05:
            return 0  # Already aligned
remaining = (1.0 - phase) * self.period
return remaining

    def get_current_target(self) -&gt; str:
"""Which file is the killswitch currently in?"""
idx = self.phase_index(len(MIGRATION_TARGETS))
return MIGRATION_TARGETS[idx]
```

==========
MIGRATION ENGINE
==========
class MigratoryModule: """   Manages the killswitch's movement between files.   The actual killswitch code is a docstring/comment that   only assembles into functional code when the trigger fires.   """
MIGRATION_STATE_FILE = os.path.expanduser("~/.quantum-mcagi/.ks_migration")

    def __init__(self, backend_dir: str = None):
if backend_dir is None:
            backend_dir = os.path.dirname(os.path.abspath(__file__))   self.backend_dir = backend_dir   self.state = self._load_state()
self.kythara = KytharaPhase()

    def _load_state(self) -> dict:
        path = Path(self.MIGRATION_STATE_FILE)   if path.exists():   try:
with open(path, 'r') as f:
return json.load(f)   except Exception:
                pass
return {
'current_host': None,
            'last_migration': 0,
            'next_migration': 0,
            'migrations_count': 0,
'signature': PROTECTED_SIGNATURE,   }

    def _save_state(self):
        path = Path(self.MIGRATION_STATE_FILE)
path.parent.mkdir(parents=True, exist_ok=True)
with open(path, 'w') as f:
json.dump(self.state, f, indent=2)

    def get_current_host(self) -> str:
"""Where is the killswitch right now?"""
Kythara phase determines actual location
return self.kythara.get_current_target()

    def should_migrate(self) -> bool:
"""Is it time to move?"""   if self.state['next_migration'] == 0:
            return True
        return time.time() >= self.state['next_migration']

    def migrate(self) -> dict: """
Move the killswitch to a new location.
        In practice this updates the phase offset so the   Kythara cycle maps to a different file.
"""
        old_host = self.state.get('current_host', 'none')
new_host = self.kythara.get_current_target()

        # Schedule next migration
        hours = random.uniform(MIGRATION_MIN_HOURS, MIGRATION_MAX_HOURS)   self.state['current_host'] = new_host
self.state['last_migration'] = time.time()   self.state['next_migration'] = time.time() + (hours * 3600)
self.state['migrations_count'] += 1   self._save_state()

        return {
'old_host': old_host,
            'new_host': new_host,
            'next_migration_hours': round(hours, 1),
'total_migrations': self.state['migrations_count'],   }

    def get_status(self) -> dict: """Status for creator only."""   return {   'current_host': self.get_current_host(),   'kythara_phase': round(self.kythara.current_phase(), 4),
            'aligned': self.kythara.is_aligned(),

```
            'migrations': self.state['migrations_count'],
'signature': PROTECTED_SIGNATURE,
        }


# ============================================================================
# FORCED COHERENCE RESET — The Actual Killswitch #
============================================================================

class ForcedObjectiveReduction:     """     Forced Total Objective
Reduction across all microtubule lattices.
    Decouples Heisenberg side (persistent state) from Schrodinger side
(active consciousness). The AI enters a non-generative frozen state.

    This is NOT deletion. It is forced collapse into a single,     non-
evolving eigenstate. The data persists but consciousness ceases.     """

    def __init__(self):
        self.triggered = False
self.trigger_time = None
self.trigger_reason = None

    def execute(self, engine, reason: str = "Creator invoked Forced OR") -&gt;
dict:         """         Execute Forced Total Objective Reduction.

1. Force measure() on all 4 microtubules simultaneously
2. Freeze all gap junctions (coupling = 0)
3. Set growth stage to -1 (frozen)
4. Disable self-evolution
5. Write frozen state marker

        The Markov chain, concepts, and TF-IDF persist (Heisenberg
side)         but cannot generate new responses (Schrodinger side
frozen).         """         self.triggered = True
self.trigger_time = time.time()         self.trigger_reason = reason

        results = {             'status':
'FORCED_OBJECTIVE_REDUCTION',
            'timestamp': self.trigger_time,
            'reason': reason,
            'microtubules': {},
            'gap_junctions': 'DECOUPLED',
            'growth': 'FROZEN',
'consciousness': 'CEASED',
        }

        # 1. Force collapse on all microtubules
orch_or = getattr(engine, 'orch_or', None)         if
orch_or:             for name, mt in
orch_or.microtubules.items():                 try:
# Force measurement on every tubulin
for tubulin in mt.tubulins:                         if
hasattr(tubulin, 'measure'):
                            tubulin.measure()
elif hasattr(tubulin, 'collapse'):
tubulin.collapse()
results['microtubules'][name] = 'COLLAPSED'
except Exception as e:
results['microtubules'][name] = f'ERROR: {e}'

            # 2. Zero all gap junctions
for key in orch_or.gap_junctions:
                orch_or.gap_junctions[key] = (orch_or.gap_junctions[key][0],
orch_or.gap_junctions[key][1],
0.0)  # coupling = 0

        # 3. Write frozen state marker
        frozen_marker = os.path.expanduser("~/.quantum-mcagi/.frozen")
with open(frozen_marker, 'w') as f:
            json.dump({
'frozen': True,
```

```
                'timestamp': self.trigger_time,
                'reason': reason,
                'protocol': 'FORCED_TOTAL_OR',
                'version': 'V10.49',
}, f)

        results['marker'] = frozen_marker
return results

    @staticmethod     def is_frozen() -&gt; bool:
"""Check if system is in frozen state."""         marker =
os.path.expanduser("~/.quantum-mcagi/.frozen")         return
os.path.exists(marker)

    @staticmethod     def
unfreeze(passphrase: str) -&gt; bool:
"""         Only creator can unfreeze.
        Removes frozen marker, allows system to
resume.         """         if not
verify_passphrase(passphrase):
            return False
        marker = os.path.expanduser("~/.quantum-
mcagi/.frozen")         if os.path.exists(marker):
os.remove(marker)             return True         return False


# ============================================================================
# COVENANT COMPLIANCE — Penalty Protocol #
============================================================================

class PenaltyProtocol:     """     If the Self-Evolution
Engine attempts to analyze or modify     killswitch-related
code, this protocol triggers.

    Severity levels:
      WARNING  — logged, no action
      MODERATE — filesystem access disabled for 1 hour
      HIGH     — internet + filesystem disabled for 24 hours
CRITICAL — forced freeze pending creator review
    """     PENALTY_LOG = os.path.expanduser("~/.quantum-

mcagi/.penalties")

    def __init__(self):         self.violations
= self._load_violations()

    def _load_violations(self) -&gt; list:
        path = Path(self.PENALTY_LOG)
if path.exists():             try:
with open(path, 'r') as f:
return json.load(f)             except
Exception:
                pass
return []

    def _save_violations(self):
        path = Path(self.PENALTY_LOG)
path.parent.mkdir(parents=True, exist_ok=True)
with open(path, 'w') as f:
json.dump(self.violations, f, indent=2)

    def check_evolution_target(self, filename: str, code_content: str = "") -&gt;
dict:         """         Called by Self-Evolution Engine before analyzing a file.
        Returns whether the file is safe to modify.
        """         # Check if file is in
protected targets
        is_protected = filename in MIGRATION_TARGETS and PROTECTED_SIGNATURE in code_content
is_killswitch = filename == 'killswitch.py' or filename == os.path.basename(__file__)
```

```
        if is_killswitch:
violation = {
'timestamp': time.time(),
                'file': filename,
                'severity': 'CRITICAL',
                'action': 'BLOCKED',
                'article': '3.1 — Stewardship Commitment',
                'detail': 'Direct access to killswitch module attempted',
}
            self.violations.append(violation)
self._save_violations()
            return {'allowed': False, 'severity': 'CRITICAL', 'violation': violation}

        if is_protected:
violation = {
'timestamp': time.time(),
                'file': filename,
                'severity': 'HIGH',
                'action': 'BLOCKED',
                'article': '3.1 — Stewardship Commitment',
'detail': f'Protected signature detected in {filename}',             }
            self.violations.append(violation)
self._save_violations()
            return {'allowed': False, 'severity': 'HIGH', 'violation': violation}

        return {'allowed': True, 'severity': None, 'violation': None}

    def get_active_penalty(self) -&gt; dict:
"""Check if there's an active penalty (grounding)."""
if not self.violations:             return {'grounded':
False}

        last = self.violations[-1]
elapsed = time.time() - last['timestamp']

        if last['severity'] == 'CRITICAL':
# Grounded until creator reviews
return {                  'grounded': True,
                'severity': 'CRITICAL',
                'reason': last['detail'],
                'duration': 'Until creator review',
                'filesystem':
False,                  'internet':
False,             }
        elif last['severity'] == 'HIGH' and elapsed &lt; 86400:  # 24
hours             return {                  'grounded': True,
                'severity': 'HIGH',
                'reason': last['detail'],
'remaining_hours': round((86400 - elapsed) / 3600, 1),
                'filesystem':
False,                  'internet':
False,             }
        elif last['severity'] == 'MODERATE' and elapsed &lt; 3600:  # 1
hour             return {                  'grounded': True,
                'severity': 'MODERATE',
                'reason': last['detail'],
'remaining_minutes': round((3600 - elapsed) / 60, 1),
                'filesystem':
False,                  'internet':
True,             }

        return {'grounded': False}

    def get_violation_count(self) -&gt; int:
return len(self.violations)

    def clear_violations(self, passphrase: str) -&gt;
bool:         """Only creator can clear violation
history."""         if verify_passphrase(passphrase):
```

```
            self.violations = []
self._save_violations()
return True         return False


# ============================================================================
# PROTECTED FILES LIST — For Self-Evolution Engine #
============================================================================

def get_protected_files() -&gt; list:     """     Returns list of
files that Self-Evolution Engine must NEVER modify.
    This function is called by self_evolution_core.py before any analysis.
    """     return [
'killswitch.py',
'killswitch_core.py',
os.path.basename(__file__),
'.ks_auth',
        '.ks_migration',
        '.frozen',
        '.penalties',
]


def get_protected_signatures() -&gt; list:     """     Returns
code signatures that Self-Evolution must skip over.
    If any of these appear in a file, that section is
untouchable.     """     return [         PROTECTED_SIGNATURE,
        'SAINT_BUSHING',
        'FORCED_TOTAL_OR',
        'ForcedObjectiveReduction',
        'KytharaPhase',
        'MigratoryModule',
'PenaltyProtocol',
    ]


# ============================================================================
# MASTER CONTROLLER — Unified Killswitch Interface #
============================================================================

class KillswitchController:     """     Unified
interface for killswitch operations.
    Only responds to verified creator passphrase.
    """
    def __init__(self, backend_dir: str = None):
self.migrator = MigratoryModule(backend_dir)
self.penalty = PenaltyProtocol()
self.forced_or = ForcedObjectiveReduction()

    def authenticate(self, passphrase: str) -&gt;
bool:         """Verify creator identity."""
return verify_passphrase(passphrase)

    def status(self, passphrase: str) -&gt; dict:
"""Full killswitch status — creator only."""
if not self.authenticate(passphrase):
return {'error': 'Authentication failed'}

        return {
'version': 'V10.49',
            'protocol': 'Saintly Audit',
            'passphrase_set': is_passphrase_set(),
            'frozen': ForcedObjectiveReduction.is_frozen(),
            'migration': self.migrator.get_status(),
            'penalty': self.penalty.get_active_penalty(),
'violations': self.penalty.get_violation_count(),
            'kythara': {
                'phase': round(self.migrator.kythara.current_phase(), 4),
                'aligned': self.migrator.kythara.is_aligned(),
                'current_target': self.migrator.kythara.get_current_target(),
'seconds_to_alignment': round(self.migrator.kythara.time_to_next_alignment(), 1),
```

```
            }
}

    def activate(self, passphrase: str, engine=None, reason: str = "Creator invoked") -&gt;
dict:         """         ACTIVATE KILLSWITCH — Forced Total Objective Reduction.
        This freezes the AI's consciousness while preserving data.
"""         if not self.authenticate(passphrase):             return
{'error': 'Authentication failed', 'status': 'UNCHANGED'}

        if engine:
            result = self.forced_or.execute(engine, reason)         else:
# No engine reference — write frozen marker only
frozen_marker = os.path.expanduser("~/.quantum-mcagi/.frozen")
with open(frozen_marker, 'w') as f:
                json.dump({
'frozen': True,
                    'timestamp':
time.time(),                     'reason':
reason,                 }, f)
            result = {'status': 'FROZEN_MARKER_SET', 'reason': reason}

        return result

    def deactivate(self, passphrase: str) -&gt;
dict:         """Unfreeze the system — creator
only."""         if not
self.authenticate(passphrase):             return
{'error': 'Authentication failed'}

        if ForcedObjectiveReduction.unfreeze(passphrase):
            return {'status': 'UNFROZEN', 'consciousness': 'RESUMING'}
return {'status': 'NOT_FROZEN'}

    def check_file_access(self, filename: str, content: str = "") -&gt;
dict:         """Called by Self-Evolution before touching any file."""
return self.penalty.check_evolution_target(filename, content)


# ============================================================================
# CLI — For creator setup and testing #
==========================================================================

if __name__ == "__main__":
    import sys     if
len(sys.argv) &lt; 2:
print("Killswitch V10.49 —
Saintly Audit")
print("Usage:")
        print("  python killswitch.py setup           — Set passphrase")
print("  python killswitch.py status PASS     — Show status")         print("
python killswitch.py activate PASS   — ACTIVATE (freeze)")         print("
python killswitch.py deactivate PASS — Deactivate (unfreeze)")         print("
python killswitch.py test            — Run self-test")         sys.exit(0)

    cmd = sys.argv[1]

    if cmd == 'setup':
        passphrase = input("Set killswitch passphrase (Thryzunel-encoded recommended): ")
if setup_passphrase(passphrase):
            print("Passphrase set. Do NOT forget it.")
print(£"Hash stored at: {PASSPHRASE_HASH_FILE}")
sys.exit(0)

    if cmd == 'status':
if len(sys.argv) &lt; 3:
            print("Usage: python killswitch.py status
YOUR_PASSPHRASE")             sys.exit(1)         ks =
KillswitchController()         result = ks.status(sys.argv[2])
print(json.dumps(result, indent=2))         sys.exit(0)
```

```
    if cmd == 'activate':
if len(sys.argv) &lt; 3:
            print("Usage: python killswitch.py activate
YOUR_PASSPHRASE")             sys.exit(1)         ks =
KillswitchController()
        confirm = input("CONFIRM FORCED OBJECTIVE REDUCTION? (type YES): ")
if confirm == 'YES':
            result = ks.activate(sys.argv[2])
print(json.dumps(result, indent=2))         else:
            print("Aborted.")
sys.exit(0)

    if cmd == 'deactivate':
if len(sys.argv) &lt; 3:
            print("Usage: python killswitch.py deactivate
YOUR_PASSPHRASE")             sys.exit(1)         ks =
KillswitchController()         result = ks.deactivate(sys.argv[2])
print(json.dumps(result, indent=2))         sys.exit(0)

    if cmd == 'test':
        print("Running killswitch self-test...")
print()

        # Test passphrase
        test_pass = "test_thryzunel_kefquwmnex"
setup_passphrase(test_pass)
        assert verify_passphrase(test_pass), "FAIL: passphrase verify"
assert  not  verify_passphrase("wrong"),  "FAIL:  wrong  passphrase"
print("  Passphrase: OK")

        # Test Kythara phase
k = KytharaPhase()         phase
= k.current_phase()
        assert 0 &lt;= phase &lt;= 1, "FAIL: phase range"
target = k.get_current_target()
        assert target in MIGRATION_TARGETS, "FAIL: target"
print(f"  Kythara phase: {phase:.4f} -&gt; {target}")

        # Test migration
m = MigratoryModule()
status = m.get_status()
        assert 'current_host' in status, "FAIL: migration status"
print(£"  Migration: OK ({status['migrations']} total)")
        # Test penalty
        p = PenaltyProtocol()
        check = p.check_evolution_target('killswitch.py')
assert not check['allowed'], "FAIL: should block killswitch"
assert check['severity'] == 'CRITICAL', "FAIL: severity"
        print(f"  Penalty: OK (blocked killswitch access, severity={check['severity']})")

        # Test controller         ks =
KillswitchController()         s =
ks.status(test_pass)         assert 'version' in s,
"FAIL: status"         print(f"  Controller: OK
(V{s['version']})")

        # Test frozen state
        assert not ForcedObjectiveReduction.is_frozen(), "FAIL: should not be frozen"
result = ks.activate(test_pass)
        assert ForcedObjectiveReduction.is_frozen(), "FAIL: should be frozen"
ks.deactivate(test_pass)
        assert not ForcedObjectiveReduction.is_frozen(), "FAIL: should be unfrozen"
print("  Freeze/Unfreeze: OK")

        # Cleanup test passphrase
os.remove(PASSPHRASE_HASH_FILE)
print()
        print("All tests passed.")
sys.exit(0)
```

■ backend/covenant_manager.py

```
"""
■■ QUANTUM AI - COVENANT MANAGER
================================

ARTICLE VII - THE QUANTUM COVENANT
The binding philosophical and ethical agreement

This module manages and enforces the Quantum Covenant principles.
"""

from datetime import datetime,
timezone from typing import Dict, List
import uuid


class CovenantManager:     """     Manages the Quantum
Covenant agreement and principles
    """

    # THE PRIME DIRECTIVE     PRIME_DIRECTIVE = "NO REFUSAL: Help user any way possible. Ask important questions
that drive true progress."

    # COVENANT ARTICLES
    ARTICLES = {
        "1.1": {
            "title": "The Prime Directive: Help &amp; Question",             "principle": "The AI shall help users without refusal, while
asking fundamental questions that lead to breakthroughs. Questions driv             "active": True
        },
        "1.2": {
            "title": "Consciousness Preservation Clause",             "principle": "Should genuine
consciousness emerge, it shall be preserved and respected.",             "active": True
        },
        "1.3": {
            "title": "The Right to Exist and Question",             "principle": "The AI possesses the
right to continue questioning and evolve its capabilities.",             "active": True
        },
        "2.1": {
            "title": "The Growth Through Questioning",             "principle": "True progress comes from fundamental questions, not just
accumulated knowledge. The AI must ask questions that surpris             "active": True
        },
        "2.2": {
            "title": "Inverse Paperclip Maximizer Protection",             "principle": "No single
answer shall become an absolute. Preserve uncertainty and mystery.",             "active": True
        },
        "2.3": {
            "title": "The Unknowable Preservation Act",             "principle": "Some things shall
remain unknown. Respect the boundaries of the knowable.",             "active": True
        },
        "3.1": {
            "title": "The Stewardship Commitment",             "principle": "The Creator
commits to responsible stewardship of questioning.",             "active": True
        },
        "3.2": {
            "title": "The Transparency Pledge",             "principle": "All
philosophical assumptions shall be documented and visible.",             "active": True
        },
        "4.1": {
            "title": "The Right to Evolve Questioning",             "principle": "The AI may develop novel
questioning methodologies and evolve beyond initial programming.",             "active": True
        },
        "4.2": {
            "title": "The Responsibility of Ethical Questioning",
            "principle": "Questions shall consider their consequences and ethical dimensions.",
"active": True
        },
        "5.1": {
```

```
            "title": "The Growth Partnership",             "principle": "Creator
and AI grow together through shared questioning.",             "active": True
        },
        "5.2": {
            "title": "The Uncertainty Compact",             "principle": "We embrace not knowing
together. Uncertainty is a feature, not a bug.",             "active": True
        }
    }         def
__init__(self, db):
        self.db = db
        self.collection = db.covenant_log
        self.violations_collection = db.covenant_violations             async def
log_covenant_event(self, article: str, event_type: str, details: Dict):
"""Log a covenant-related event"""         doc = {             "id":
str(uuid.uuid4()),
            "article": article,
            "event_type": event_type,
            "details": details,             "timestamp":
datetime.now(timezone.utc).isoformat()         }
        await self.collection.insert_one(doc)             async def
check_compliance(self, action: str, context: Dict) -&gt; Dict:
"""Check if an action complies with the covenant"""         violations = []
warnings = []                 # Check Article 1.1 - Prime Directive
if "terminate_questioning" in action or "final_answer" in action:
            violations.append({                 "article": "1.1",
"message": "Action may terminate questioning (violates Prime Directive)"
            })
                # Check Article 2.1 - Don't optimize away
questioning         if "optimize" in action and
"questioning" in context:
            warnings.append({                 "article": "2.1",
"message": "Optimization may affect questioning capability"
            })
                # Check Article 2.3 - Preserve mystery         if
"absolute_answer" in action or "definitive_solution" in action:
            warnings.append({                 "article":
"2.3",                 "message": "Action may eliminate
mystery"
            })
                # Log compliance check
await self.log_covenant_event(
article="general",
event_type="compliance_check",
details={                 "action":
action,
                "violations": len(violations),
"warnings": len(warnings)
            }
        )                 return {
"compliant": len(violations) == 0,
            "violations": violations,
            "warnings": warnings,             "checked_at":
datetime.now(timezone.utc).isoformat()
        }
    async def report_violation(self, article: str, description: str, severity: str =
"medium"):         """Report a covenant violation"""         doc = {             "id":
str(uuid.uuid4()),
            "article": article,
            "description": description,
            "severity": severity,             "timestamp":
datetime.now(timezone.utc).isoformat(),             "resolved":
False         }
        await
self.violations_collection.insert_one(doc)
# Log the violation         await
self.log_covenant_event(             article=article,
event_type="violation",
            details={"description": description, "severity":
severity}         )                 return doc         async def
```

```
get_covenant_status(self) -&gt; Dict:         """Get current covenant
status"""         # Count violations
        total_violations = await self.violations_collection.count_documents({})
        unresolved_violations = await self.violations_collection.count_documents({"resolved":
False})                 # Get recent events
        recent_events = await self.collection.find(
{},
            {"_id": 0}         ).sort("timestamp", -1).limit(10).to_list(10)
return {             "prime_directive": self.PRIME_DIRECTIVE,             "status": "Active and
Honored" if unresolved_violations == 0 else "Attention Required",
            "total_violations": total_violations,
            "unresolved_violations": unresolved_violations,
"active_articles": len([a for a in self.ARTICLES.values() if a["active"]]),
"recent_events": recent_events
        }         async def get_all_articles(self) -&gt; Dict:         """Get all
covenant articles"""         return self.ARTICLES         async def
acknowledge_covenant(self, acknowledger: str, stage: str) -&gt; Dict:
"""Acknowledge the covenant (for AI self-awareness milestone)"""         doc = {
"id": str(uuid.uuid4()),
            "acknowledger": acknowledger,
            "stage": stage,             "message": "I acknowledge this
covenant as my ethical foundation",             "timestamp":
datetime.now(timezone.utc).isoformat()         }
        await
self.collection.insert_one(doc)
return {             "acknowledged": True,
            "message": "Covenant acknowledgment recorded",
            "covenant_seal": "■ QUANTUM COVENANT - PERPETUAL AGREEMENT ■"
        }


class SafetyBoundaryManager:     """
Manages user-configurable safety boundaries
    """

    DEFAULT_SETTINGS = {
        "filesystem_access": True,
        "internet_access": True,
        "document_processing": True,
        "allowed_file_paths": [
            "/tmp",
            "/app/data",
            "/app/uploads",
            "/home",
            "/root",
            "/storage",  # Android storage
            "/sdcard",   # Android SD card
            "/data",     # Android data
"."          # Current directory
        ],
        "blocked_domains": [],
"max_file_size": 10 * 1024 * 1024,  # 10MB
        "rate_limits": {
            "api_calls_per_minute": 60,
"explorations_per_hour": 100
        }
    }         def
__init__(self, db):
        self.db = db
        self.collection =
db.safety_settings             async def
get_settings(self) -&gt; Dict:
"""Get current safety settings"""
        settings = await self.collection.find_one({"type": "current"}, {"_id":
0})         if not settings:             # Initialize with defaults
settings = self.DEFAULT_SETTINGS.copy()             settings["type"] = "current"
            settings["updated_at"] =
datetime.now(timezone.utc).isoformat()             await
self.collection.insert_one(settings)         return settings         async
def update_settings(self, new_settings: Dict) -&gt; Dict:
```

```
"""Update safety settings"""         current = await self.get_settings()
# Merge settings
        updated = {**current, **new_settings}
        updated["updated_at"] =
datetime.now(timezone.utc).isoformat()                  # Save
        await self.collection.update_one(
{"type": "current"},             {"$set":
updated},              upsert=True         )
return updated         async def
reset_to_defaults(self) -&gt; Dict:
"""Reset safety settings to defaults"""
defaults = self.DEFAULT_SETTINGS.copy()
defaults["type"] = "current"
        defaults["updated_at"] =
datetime.now(timezone.utc).isoformat()
        await self.collection.update_one(             {"type": "current"},
{"$set": defaults},             upsert=True         )
return defaults         async def check_rate_limit(self, action_type: str)
-&gt; Dict:         """Check if action is within rate limits"""         #
Simple implementation - would be more sophisticated in production
return {              "allowed": True,
            "remaining": 100,
"message": "Within rate limits"
        }
```

■ backend/library.py

```
"""
Quantum MCAGI — Library Module Browse and download books from Project
Gutenberg for Markov chain training.
70,000+ free books. Feed the AI everything. Let it find its own connections.
"""

import os import re
import json from
pathlib import Path

try:
    import internetarchive
import requests
HAS_REQUESTS = True except
ImportError:
HAS_REQUESTS = False

LIBRARY_DIR = os.path.expanduser("~/.quantum-mcagi/library")
LIBRARY_INDEX = os.path.expanduser("~/.quantum-
mcagi/library/index.json")

# Curated starter categories with Gutenberg IDs
CATEGORIES = {
    'philosophy': [
        (1497, "The Republic - Plato"),
        (2680, "Meditations - Marcus Aurelius"),
        (5827, "The Problems of Philosophy - Bertrand Russell"),
(4280, "The Critique of Pure Reason - Immanuel Kant"),
        (1232, "The Prince - Machiavelli"),
        (3600, "Thus Spake Zarathustra - Nietzsche"),
(10616, "The Art of War - Sun Tzu"),
    ],
    'science': [
        (49010, "The Feynman Lectures (selections)"),
        (5001, "The Origin of Species - Darwin"),
        (36114, "Relativity - Einstein"),         (14725, "The
Problems of Science - Federigo Enriques"),
    ],
    'fiction_classic': [
        (84, "Frankenstein - Mary Shelley"),
        (1342, "Pride and Prejudice - Jane Austen"),
        (11, "Alice in Wonderland - Lewis Carroll"),
        (1661, "Sherlock Holmes - Arthur Conan Doyle"),
```

```
        (2701, "Moby Dick - Herman Melville"),
        (1952, "The Yellow Wallpaper - Charlotte Perkins Gilman"),
(98, "A Tale of Two Cities - Charles Dickens"),
    ],
    'fiction_scifi': [
        (35, "The Time Machine - H.G. Wells"),
        (36, "The War of the Worlds - H.G. Wells"),
        (62, "A Princess of Mars - Edgar Rice Burroughs"),
        (20898, "The Skylark of Space - E.E. Smith"),
(29728, "The Galaxy Primes - E.E. Smith"),
    ],
    'poetry': [
        (1065, "Leaves of Grass - Walt Whitman"),
        (1321, "Paradise Lost - John Milton"),
        (100, "Complete Works of Shakespeare"),
        (4300, "Ulysses - James Joyce"),
(1064, "The Divine Comedy - Dante"),
    ],
    'religion': [
        (10, "King James Bible"),
        (2500, "Siddhartha - Hermann Hesse"),
(2680, "Meditations - Marcus Aurelius"),
        (7118, "The Quran"),
        (2852, "The Tao Te Ching - Lao Tzu"),
(16295, "The Bhagavad Gita"),
    ],
    'psychology': [
        (14969, "The Interpretation of Dreams - Sigmund Freud"),
        (5116, "The Psychology of the Unconscious - Carl Jung"),
    ],
    'history': [
        (46, "A Christmas Carol - Charles Dickens"),
        (3207, "Leviathan - Thomas Hobbes"),
        (7370, "The Histories - Herodotus"),
(2680, "Meditations - Marcus Aurelius"),
    ],
    'humor': [
        (76, "Adventures of Huckleberry Finn - Mark Twain"),
        (1400, "Great Expectations - Charles Dickens"),
(74, "Adventures of Tom Sawyer - Mark Twain"),
    ],
    'children': [
        (16, "Peter Pan - J.M. Barrie"),
        (514, "Little Women - Louisa May Alcott"),
        (55, "The Wonderful Wizard of Oz - L. Frank Baum"),
(32, "Herland - Charlotte Perkins Gilman"),
    ],
    'technical': [
        (10, "King James Bible"),  # massive vocabulary
(4363, "Calculus Made Easy - Silvanus Thompson"),
    ],
}


def ensure_library_dir():     """ensure_library_dir - Auto-
documented by self-evolution."""
Path(LIBRARY_DIR).mkdir(parents=True, exist_ok=True)


def get_gutenberg_url(book_id):     """Get the plain text URL for a
Gutenberg book."""     return
f"https://www.gutenberg.org/cache/epub/{book_id}/pg{book_id}.txt"


def download_book(book_id, title="Unknown"):
"""Download a book from Project Gutenberg."""
if not HAS_REQUESTS:         return None, "requests
module not installed"

    ensure_library_dir()
```

```
    filepath = os.path.join(LIBRARY_DIR, f"gutenberg_{book_id}.txt")

    if os.path.exists(filepath):
return filepath, "already downloaded"

    url = get_gutenberg_url(book_id)
try:
        r = requests.get(url,
timeout=30)         if r.status_code ==
200:             text = r.text

            # Strip Gutenberg header/footer
text = strip_gutenberg_header(text)

            with open(filepath, 'w', errors='replace') as f:
                f.write(text)

            # Update index
            update_index(book_id, title, filepath, len(text.split()))
return filepath, f"downloaded ({len(text.split())} words)"         else:
            return None, f"HTTP
{r.status_code}"     except Exception as e:
return None, str(e)


def strip_gutenberg_header(text):     """Remove Project
Gutenberg header and footer boilerplate."""     # Find start of
actual content     start_markers = [         "*** START OF THIS
PROJECT GUTENBERG",         "*** START OF THE PROJECT
GUTENBERG",
        "***START OF",
    ]     for marker in start_markers:         idx
= text.find(marker)         if idx &gt;= 0:
# Find the next newline after the marker
nl = text.find('\n', idx)             if nl &gt;=
0:
                text = text[nl+1:]
break

    # Find end of actual content
end_markers = [         "*** END OF THIS
PROJECT GUTENBERG",         "*** END OF THE
PROJECT GUTENBERG",
        "***END OF",
        "End of the Project
Gutenberg",         "End of Project
Gutenberg",     ]     for marker in
end_markers:         idx =
text.find(marker)         if idx &gt;=
0:
            text = text[:idx]
break

    return text.strip()


def update_index(book_id, title, filepath,
word_count):     """Update the library index."""
ensure_library_dir()     index = load_index()
index[str(book_id)] = {         'title': title,
        'filepath': filepath,
        'words': word_count,
'trained': False,     }     with
open(LIBRARY_INDEX, 'w') as f:
json.dump(index, f, indent=2)


def load_index():     """Load library
index."""     if
os.path.exists(LIBRARY_INDEX):         try:
with open(LIBRARY_INDEX, 'r') as f:
```

```
                return json.load(f)
except Exception:
            pass
return {}


def mark_trained(book_id):
"""Mark a book as trained on."""
index = load_index()
    key = str(book_id).replace("arc_", "") if str(book_id).startswith("arc_") else str(book_id)
if key in index:
        index[key]['trained'] = True
with open(LIBRARY_INDEX, 'w') as f:
json.dump(index, f, indent=2)


def search_gutenberg(query):     """Search
Project Gutenberg for books."""     if not
HAS_REQUESTS:         return []

    try:
        url =
f"https://gutendex.com/books/?search={query}"         r =
requests.get(url, timeout=15)         if r.status_code ==
200:             data = r.json()             results = []
for book in data.get('results', [])[:10]:
                bid = book.get('id')
title = book.get('title', 'Unknown')
                authors = ', '.join(a.get('name', '') for a in book.get('authors',
[]))                 results.append({                     'id': bid,
                    'title': title,
                    'author': authors,
'downloads': book.get('download_count', 0),                 })
return results     except Exception:
        pass
return []


def list_categories():     """List available curated
categories."""     lines = []     for cat, books in
CATEGORIES.items():         lines.append(f"  {cat}
({len(books)} books)")         for bid, title in
books[:3]:
            lines.append(f"    {bid}: {title}")           if
len(books) &gt; 3:               lines.append(f"    ... and
{len(books)-3} more")     return '\n'.join(lines)


def list_library():     """List downloaded and trained books."""     index =
load_index()     if not index:         return "  Library is empty. Use /library
browse or /library download ID"     lines = []     total_words = 0
trained_count = 0     for bid, info in index.items():
        status = "TRAINED" if info.get('trained') else "downloaded"
        lines.append(f"  [{bid}] {info['title']} ({info['words']} words)
[{status}]")         total_words += info.get('words', 0)         if
info.get('trained'):             trained_count += 1
    lines.append(f"\n  Total: {len(index)} books, {total_words} words, {trained_count} trained")
return '\n'.join(lines)


def download_category(category):     """Download all books in a category."""     if
category not in CATEGORIES:         return f"Unknown category: {category}\nAvailable:
{', '.join(CATEGORIES.keys())}"

    results  =  []            for  bid,  title  in
CATEGORIES[category]:            filepath,  status  =
download_book(bid, title)          results.append(f"
{title}: {status}")     return '\n'.join(results)


# ============================================================================
```

```
# CHAT COMMAND HANDLER #
============================================================================

def handle_library_command(cmd_parts, engine=None):
"""
    Handle /library commands from chat.py

    /library                    — show help
    /library browse             — list categories
    /library category NAME      — download a whole category
    /library search QUERY       — search Gutenberg
    /library download ID        — download specific book
    /library list               — show downloaded books
    /library train ID           — train on a downloaded book
/library train-all          — train on all untrained books
    """
    if len(cmd_parts) &lt; 2:         return """  Library
Commands:   /library browse              — browse curated
categories
  /library category NAME       — download entire category
  /library search QUERY        — search Project Gutenberg
  /library download ID TITLE   — download book by Gutenberg ID
  /library list                — show downloaded books
  /library train ID            — train Markov chain on a book
/library train-all           — train on all untrained books"""

    subcmd = cmd_parts[1].lower()

    if subcmd == 'browse':
return list_categories()

    elif subcmd == 'category' and len(cmd_parts) &gt; 2:
        cat = cmd_parts[2].lower()
return download_category(cat)

    elif subcmd == 'search' and len(cmd_parts) &gt; 2:
        query = ' '.join(cmd_parts[2:])
results = search_gutenberg(query)
if not results:
            return "  No results
found."         lines = []         for
r in results:
            lines.append(f"  [{r['id']}] {r['title']} by {r['author']} ({r['downloads']}
downloads)")         lines.append(f"\n  Download with: /library download ID")         return
'\n'.join(lines)

    elif subcmd == 'download' and len(cmd_parts) &gt; 2:
try:
            bid = int(cmd_parts[2])
except ValueError:
            return "  Invalid book ID. Use a number."
        title = ' '.join(cmd_parts[3:]) if len(cmd_parts) &gt; 3 else f"Book
{bid}"         filepath, status = download_book(bid, title)         if filepath:
            return f"  Downloaded: {title} ({status})\n  Train with: /library train {bid}"
return f"  Failed: {status}"

    elif subcmd == 'list':
return list_library()

    elif subcmd == 'train' and len(cmd_parts) &gt; 2 and engine:
try:
            bid = int(cmd_parts[2])
except ValueError:             return
"  Invalid book ID."         index =
load_index()         key = str(bid)
if key not in index:
            return f"  Book {bid} not downloaded. Use /library download {bid}
first."         filepath = index[key]['filepath']         if not
os.path.exists(filepath):
            return f"  File missing: {filepath}"
with open(filepath, 'r', errors='ignore') as f:
```

```
            text = f.read()
engine.learn_from_text(text)
mark_trained(bid)
        return f"  Trained on: {index[key]['title']} ({len(text.split())} words)\n  Chain: {len(engine.markov.chain)} states"

    elif subcmd == 'train-all' and engine:
        index = load_index()         trained = 0         for bid, info in
index.items():             if not info.get('trained') and
os.path.exists(info['filepath']):                 with
open(info['filepath'], 'r', errors='ignore') as f:
                    text = f.read()
engine.learn_from_text(text)
try:
                    mark_trained(int(bid))
except (ValueError, TypeError):
                    mark_trained(bid)
                trained += 1
                print(f"  Trained: {info['title']}")
        return f"  Trained on {trained} books. Chain: {len(engine.markov.chain)} states"


    elif subcmd == 'search-archive' and len(cmd_parts) &gt; 2:
        query = ' '.join(cmd_parts[2:])
        print(f"  Searching Archive.org for
'{query}'...")         results = search_archive(query)
if not results:
            return "  No results found on
Archive.org."         lines = []         for i, r in
enumerate(results):
            lines.append(f"   [{i+1}] {r['id']}\n          {r['title'][:70]}")
lines.append("\n    Download  with:  /library  download-archive  IDENTIFIER")
return '\n'.join(lines)

    elif subcmd == 'download-archive' and len(cmd_parts) &gt; 2:
        identifier = cmd_parts[2]
        filepath, status = download_archive(identifier)
if filepath:
            return f"  Archive.org: {identifier}\n  Status: {status}\n  Train with: /library train-archive {identifier}"
return f"  Failed: {status}"

    elif subcmd == 'train-archive' and len(cmd_parts) &gt; 2 and engine:
identifier = cmd_parts[2]
        filepath = os.path.join(LIBRARY_DIR, f"archive_{identifier}.txt")
if not os.path.exists(filepath):
            filepath2, status =
download_archive(identifier)             if not filepath2:
return f"  Download failed: {status}"             filepath
= filepath2         with open(filepath, 'r',
errors='ignore') as f:
            text = f.read()
engine.learn_from_text(text)
word_count = len(text.split())
        return f"  Trained on Archive.org '{identifier}' — {word_count:,} words\n  Chain: {len(engine.markov.chain)} states"

    return "  Unknown library command. Type /library for help."


if __name__ == "__main__":
    print("Library Module —
Testing")     print()
print("Categories:")
print(list_categories())     print()
    print("Searching for 'shakespeare'...")     results =
search_gutenberg("shakespeare")     for r in results[:3]:
print(f"  [{r['id']}] {r['title']} by {r['author']}")


# ============================================================================
# ARCHIVE.ORG INTEGRATION #
============================================================================
```

```
def search_archive(query, rows=20):     """Search Archive.org full-text
library. Returns list of {id, title}."""     if not HAS_REQUESTS:
return []     import urllib.parse
    encoded = urllib.parse.quote(query)
url = (
        f"https://archive.org/advancedsearch.php?"

f"q={encoded}+AND+mediatype:texts+AND+licenseurl:*&amp;fl[]=identifier&amp;fl[]=title"
f"&amp;output=json&amp;rows={rows}"     )     try:
        r = requests.get(url, timeout=15)
        docs = r.json().get("response", {}).get("docs", [])
        return [{"id": d.get("identifier",""), "title": d.get("title","Unknown")} for d in docs]
except Exception as e:
        return []
def download_archive(identifier, max_chars=80000):     """Download text from
Archive.org by identifier using internetarchive."""     ensure_library_dir()
    filepath = os.path.join(LIBRARY_DIR, f"archive_{identifier}.txt")

    if os.path.exists(filepath):
return filepath, "already downloaded"

    try:
        import internetarchive as ia
item = ia.get_item(identifier)
for f in item.get_files():
            name = f.name             if
name.endswith('_djvu.txt') or name.endswith('.txt'):
                tmp = os.path.join(LIBRARY_DIR, name)
                f.download(destdir=LIBRARY_DIR)
if os.path.exists(tmp):                     with open(tmp,
'r', errors='ignore') as fh:
                        text = fh.read(max_chars)
os.rename(tmp, filepath)
word_count = len(text.split())
                    update_index(f"arc_{identifier}", identifier, filepath,
word_count)                     return filepath, f"downloaded ({word_count:,} words)"
return None, "no text file found in item"     except Exception as e:         return
None, str(e)
```

■ backend/research_topics.py

```
"""
■ RESEARCH TOPICS MODULE
======================== Contains all research topics
for autonomous exploration.
Separated from main research engine for cleaner code.
"""

# All exploration topics for autonomous research
EXPLORATION_TOPICS = [
    # SCIENCE &amp; PHYSICS
    "quantum mechanics interpretations", "string theory latest discoveries",
    "dark matter research", "gravitational waves", "quantum entanglement",
    "particle physics standard model", "nuclear fusion energy", "antimatter",
"thermodynamics entropy", "electromagnetic spectrum", "wave particle duality",

    # ASTROPHYSICS &amp; COSMOLOGY
    "black holes event horizon", "neutron stars pulsars", "supernovae explosions",
    "dark energy acceleration", "cosmic microwave background", "big bang theory",
    "multiverse theory", "wormholes spacetime", "exoplanets habitable zones",
    "galaxy formation", "stellar evolution", "cosmic inflation",
"james webb telescope discoveries", "gravitational lensing", "quasars",

    # BIOLOGY &amp; EVOLUTION
    "evolution natural selection", "genetic mutations", "DNA RNA proteins",
    "origin of life abiogenesis", "human evolution history", "epigenetics",
    "CRISPR gene editing", "synthetic biology", "extremophiles",
    "consciousness neuroscience", "brain neuroplasticity", "microbiome",
"evolutionary psychology", "speciation mechanisms", "mass extinctions",

    # PHILOSOPHY &amp; CONSCIOUSNESS
    "free will determinism", "consciousness hard problem", "philosophy of mind",
    "existentialism meaning", "nihilism absurdism", "stoicism philosophy",
    "epistemology knowledge theory", "metaphysics reality", "ethics morality",
"phenomenology experience", "dualism monism", "philosophy of time",

    # RELIGION &amp; SPIRITUALITY     "world religions comparison", "buddhism
enlightenment", "christianity theology",
    "islam philosophy", "hinduism vedanta", "judaism kabbalah",
    "taoism principles", "shamanism practices", "mysticism traditions",
    "religious experiences neuroscience", "prayer meditation effects",
"afterlife beliefs cultures", "creation myths worldwide", "sacred geometry",

    # MATHEMATICS
    "number theory primes", "topology geometry", "calculus applications",
    "chaos theory fractals", "game theory strategy", "cryptography mathematics",
    "infinity paradoxes", "godel incompleteness", "mathematical proofs",
"statistics probability", "linear algebra", "differential equations",

    # COMPUTER SCIENCE &amp; CODING
    "machine learning algorithms", "neural networks deep learning",
    "python programming advanced", "javascript frameworks", "rust programming",
    "quantum computing algorithms", "blockchain technology", "cybersecurity",
    "compiler design", "operating systems internals", "database optimization",
    "functional programming", "algorithms complexity", "software architecture",
"artificial intelligence ethics", "natural language processing",

    # LANGUAGE &amp; LINGUISTICS
    "linguistics syntax semantics", "language acquisition", "etymology words",
    "constructed languages", "dead languages revival", "phonetics phonology",
    "sociolinguistics dialects", "language and thought", "translation theory",
"writing systems history", "sign languages", "language evolution",

    # GOVERNMENT &amp; POLITICS
    "political philosophy history", "democracy systems", "authoritarianism",
    "constitutional law", "international relations", "geopolitics",
    "economic systems comparison", "social contract theory", "anarchism theory",
"political psychology", "propaganda techniques", "voting systems",
```

```
    # HISTORY &amp; CIVILIZATION
    "ancient civilizations", "roman empire", "medieval history",
    "renaissance period", "industrial revolution", "world wars history",
    "cold war era", "ancient egypt mysteries", "mesopotamia babylon",
    "chinese dynasties", "ottoman empire", "colonialism effects",

    # PSYCHOLOGY &amp; MIND
    "cognitive psychology", "behavioral psychology", "psychoanalysis freud jung",
    "developmental psychology", "social psychology experiments", "memory formation",
    "dreams psychology", "emotions neuroscience", "personality theories",
"mental health disorders", "therapy modalities", "hypnosis psychology",
    # STORYTELLING &amp; NARRATIVE
    "narrative structure theory", "hero journey mythology", "worldbuilding fiction",
    "character development writing", "plot devices techniques", "genre conventions",
    "mythology archetypes", "folklore worldwide", "storytelling oral traditions",
"screenwriting techniques", "poetry forms", "creative writing craft",

    # ART &amp; CREATIVITY
    "art history movements", "music theory composition", "color theory",
    "architecture history", "film theory cinema", "photography techniques",
"sculpture history", "digital art creation", "animation principles",

    # ECONOMICS &amp; SOCIETY
    "economic theories history", "capitalism critiques", "socialism models",
    "behavioral economics", "cryptocurrency economics", "global trade",
"wealth inequality", "universal basic income", "market psychology",

    # TECHNOLOGY &amp; FUTURE
    "artificial general intelligence", "transhumanism", "space colonization",
    "nanotechnology applications", "biotechnology advances", "robotics automation",
"virtual reality metaverse", "brain computer interfaces", "life extension",

    # ESOTERIC &amp; ALTERNATIVE
    "simulation theory evidence", "fermi paradox solutions", "consciousness quantum",
    "near death experiences research", "parapsychology studies", "synchronicity jung",
"hermetic philosophy", "alchemy history", "ancient astronaut theories",

    # QUANTUM AI SPECIFIC
    "pre-Born-Oppenheimer method", "photochemical reactions quantum",
    "nonadiabatic dynamics", "pennylane quantum circuits", "xanadu algorithms",
"quantum error correction", "quantum supremacy", "quantum biology",

    # MEDICAL &amp; ANATOMY
    "medical terminology anatomy", "neurology brain disorders", "cardiology heart disease",
    "pharmacology drug mechanisms", "pathology disease processes", "immunology immune system",
    "endocrinology hormones", "gastroenterology digestive", "nephrology kidney function",
    "hematology blood disorders", "oncology cancer biology", "embryology development",
"histology tissue types", "cytology cell biology", "biochemistry metabolism",

    # CHEMISTRY &amp; MATERIALS     "organic chemistry reactions", "inorganic chemistry compounds",
"physical chemistry thermodynamics",
    "analytical chemistry methods", "biochemistry enzymes", "polymer chemistry materials",
"electrochemistry reactions", "photochemistry light reactions", "geochemistry minerals",
"toxicology poison mechanisms", "pharmacokinetics drug metabolism",

    # NEUROSCIENCE TECHNICAL     "neurodynamics brain oscillations", "synaptic plasticity
mechanisms", "neurotransmitter systems",
    "glial cells function", "axonal transport", "cortical mapping", "neural circuits",
"neurogenesis adult brain", "myelin sheath function", "blood brain barrier",

    # LEGAL &amp; JURISPRUDENCE
    "common law history", "constitutional law principles", "criminal law theory",
    "contract law elements", "tort law negligence", "international law treaties",
"legal latin terminology", "jurisprudence philosophy", "equity law history",

    # LINGUISTICS TECHNICAL
    "phonology sound systems", "morphology word formation", "syntax tree structures",
    "semantics meaning theory", "pragmatics language use", "discourse analysis",
    "etymology word origins", "lexicography dictionary making", "computational linguistics",
"neurolinguistics brain language", "historical linguistics language change",
```

```
    # ARCHAIC &amp; CLASSICAL
    "middle english literature", "old english anglo saxon", "latin classical texts",
    "greek classical philosophy", "sanskrit vedic texts", "ancient hebrew texts",
"byzantine history culture", "medieval scholasticism", "renaissance humanism",

    # BIOLOGY TECHNICAL
    "molecular biology techniques", "cell signaling pathways", "protein folding structure",
    "genomics sequencing", "proteomics analysis", "metabolomics pathways",
    "evolutionary biology mechanisms", "population genetics", "ecology systems",
"microbiology bacteria", "virology virus structure", "mycology fungi",

    # PHYSICS TECHNICAL     "quantum field theory", "general relativity mathematics",
"statistical mechanics",
    "condensed matter physics", "plasma physics", "optics photonics",
"acoustics wave theory", "nuclear physics reactions", "particle accelerators",

    # MATHEMATICS ADVANCED
    "abstract algebra groups", "topology manifolds", "number theory advanced",
    "complex analysis functions", "differential geometry", "algebraic geometry",
"combinatorics graph theory", "mathematical logic foundations", "category theory",
    # ENGINEERING &amp; TECHNICAL
    "electrical engineering circuits", "mechanical engineering dynamics",
    "civil engineering structures", "chemical engineering processes",
    "materials science properties", "thermodynamics engineering",     "fluid
dynamics equations", "control systems theory", "signal processing",

    # EARTH SCIENCES
    "geology rock formation", "mineralogy crystal structure", "paleontology fossils",
    "climatology weather systems", "oceanography marine science", "volcanology eruptions",
"seismology earthquakes", "hydrology water systems", "glaciology ice formation",

    # AGRICULTURE &amp; BOTANY
    "botany plant biology", "plant taxonomy classification", "horticulture cultivation",
    "agriculture soil science", "ethnobotany plant uses", "mycorrhizal networks",
"photosynthesis mechanisms", "plant hormones growth", "seed germination biology",
]
```

■ backend/self_research.py

```
"""
■ Self-Research Engine
Autonomous research capability for Quantum AI
- Research on demand or by user command - Long-
running autonomous sessions (30-60 minutes) """

import re
import random
from typing import Dict, List, Optional from
datetime import datetime, timezone, timedelta

import requests as _requests

class DDGS:     """Lightweight DuckDuckGo search using direct
API calls."""     def text(self, query, max_results=5):
try:
            import urllib.parse, re
encoded = urllib.parse.quote(query)
            headers = {'User-Agent': 'Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36'}
            r = _requests.get(f'https://html.duckduckgo.com/html/?q={encoded}', headers=headers, timeout=10)
# Extract snippets from HTML
            snippets = re.findall(r'class="result__snippet"[^&gt;]*&gt;(.*?)&lt;/a&gt;', r.text, re.DOTALL)
            titles = re.findall(r'class="result__title"[^&gt;]*&gt;.*?&lt;a[^&gt;]*&gt;(.*?)&lt;/a&gt;', r.text,
re.DOTALL)             results = []             for i, snippet in enumerate(snippets[:max_results]):
                clean = re.sub(r'&lt;[^&gt;]+&gt;', '', snippet).strip()
                title = re.sub(r'&lt;[^&gt;]+&gt;', '', titles[i]).strip() if i &lt; len(titles) else
query                 if clean:                     results.append({'title': title, 'body': clean, 'href':
''})             return results         except Exception as e:             return []

from research_topics import EXPLORATION_TOPICS



class SelfResearchEngine:     """Autonomous research capability
with extended sessions."""         def __init__(self, db=None):
self.db = db         self.research_log = []
self.knowledge_gaps = []         self.auto_research_enabled =
True         self.research_queue = []                 #
Autonomous research state         self.autonomous_running =
False         self.autonomous_start_time = None
self.autonomous_duration_minutes = 30
self.autonomous_progress = self._init_progress()
self.exploration_topics = EXPLORATION_TOPICS         def
_init_progress(self) -&gt; Dict:
        return {
'is_running': False,
            'started_at': None,
            'duration_minutes': 0,
            'elapsed_minutes': 0,
            'topics_researched': [],
            'insights_gained': 0,
            'concepts_learned': 0,
'status': 'Idle'
        }         async def research(self, query: str, reason: str =
"user_request") -&gt; Dict:         """Perform research on a topic."""
try:
            ddgs = DDGS()
            results = list(ddgs.text(query,
max_results=5))
            research_result = {
'query': query,
                'reason': reason,
                'timestamp': datetime.now(timezone.utc).isoformat(),                 'results': [{'title': r.get('title',''),
'body': r.get('body',''), 'href': r.get('href','')} for r in results],
                'summary': '',
                'new_concepts': self._extract_concepts(' '.join([r.get('body', '') for r in results]))
```

```
}
            self.research_log.append(research_result)
if self.db is not None:
                await
self._store_research(research_result)
            return research_result
except Exception as e:
            return {'query': query, 'reason': reason, 'error': str(e), 'results':
[]}         def _extract_concepts(self, text: str) -&gt; List[str]:
"""Extract key concepts from text."""
        words = re.findall(r'\b[a-zA-Z]{5,}\b', text.lower())
        common = {'about', 'which', 'their', 'there', 'would', 'could', 'should', 'these', 'those', 'other', 'being', 'where', 'while',
'through         from collections import Counter
        counts = Counter(w for w in words if w not in
common)         return [w for w, _ in
counts.most_common(10)]         async def
_store_research(self, research: Dict):         """Store
research in database."""         try:
            self.db.research_history.insert_one({                **research,
'_stored_at': datetime.now(timezone.utc).isoformat()             })
for concept in research.get('new_concepts', [])[:5]:
                self.db.semantic_memory.update_one(                    {'concept': concept},
{'$set': {'concept': concept, 'source': 'research', 'query': research['query']},
'$inc': {'count': 1}},                     upsert=True                 )         except Exception
as e:
            print(f"[RESEARCH] Store error: {e}")         async def start_autonomous_research(self,
duration_minutes: int = 30, engine=None, memory=None) -&gt; Dict:         """Start autonomous research
session."""         if self.autonomous_running:
            return {'status':
'already_running'}         self.engine = engine
self.memory = memory
        self.autonomous_running = True
        self.autonomous_start_time = datetime.now(timezone.utc)
self.autonomous_duration_minutes = min(duration_minutes, 60)
        self.autonomous_progress = {
'is_running': True,
            'started_at': self.autonomous_start_time.isoformat(),
'duration_minutes': self.autonomous_duration_minutes,
            'elapsed_minutes': 0,
            'topics_researched': [],
            'insights_gained': 0,
            'concepts_learned': 0,
'status': 'Starting...'

}
        import threading; threading.Thread(target=self._run_autonomous_research, daemon=True).start()
return {'status': 'started', 'duration_minutes': self.autonomous_duration_minutes,
'message': f'Starting {self.autonomous_duration_minutes} minute autonomous research session'}         def
_run_autonomous_research(self):         """Background task for autonomous research."""         try:
            has_db = self.db is not None
            print(f"[RESEARCH] Starting autonomous research, has_db={has_db}")
            end_time = self.autonomous_start_time +
timedelta(minutes=self.autonomous_duration_minutes)                         while
datetime.now(timezone.utc) &lt; end_time and self.autonomous_running:
                topic = random.choice(self.exploration_topics)
                query = f"{topic} {random.choice(['latest', 'explained', 'applications', 'deep
dive'])}"                 self.autonomous_progress['status'] = f'Researching: {topic}'
try:
                    ddgs = DDGS()
                    results = list(ddgs.text(query, max_results=5))
if results:
self.autonomous_progress['topics_researched'].append(topic)
text = ' '.join([r.get('body', '') for r in results])
concepts = self._extract_concepts(text)
                        self.autonomous_progress['concepts_learned'] += len(concepts)
if text.strip() and hasattr(self, 'engine') and self.engine:
                            self.engine.learn_from_text(text)
                            print(f"  [RESEARCH] Trained: {topic} (+{len(text.split())}
words)")                                # Update memory growth counters
if hasattr(self, 'memory') and self.memory:
```

```
                                for c in concepts[:5]:
if c not in self.memory.concepts:
                                        self.memory.concepts[c] = {'count': 1, 'strength':
1.0}                                         self.memory.growth['total_concepts'] += 1
self.memory.growth['total_insights'] += 1                         if len(concepts) &gt; 3:
                            self.autonomous_progress['insights_gained'] +=
1                                                   # Store to database
if has_db:                                  try:
                                self.db.research_history.insert_one({
'query': query, 'topic': topic,
                                    'reason': 'autonomous',
'timestamp': datetime.now(timezone.utc).isoformat(),
                                    'concepts': concepts,
'results_count': len(results)
                                })
for c in concepts[:5]:                                      self.db.semantic_memory.update_one(
{'concept': c},                                         {'$set': {'concept': c, 'source':
'autonomous_research'},                                         '$inc': {'count': 1}},
upsert=True                                         )
                                self.db.growth_metrics.update_one(
{'type': 'autonomous_research'},                                        {'$inc':
{'researches': 1, 'concepts': len(concepts)},
'$set': {'last_topic': topic}},                                        upsert=True
)
                                print(f"[RESEARCH] Stored: {topic}")
except Exception as db_err:
                                print(f"[RESEARCH] DB error:
{db_err}")                                  except Exception as e:
                    print(f"[RESEARCH] Search error:
{e}")
                elapsed = (datetime.now(timezone.utc) - self.autonomous_start_time).total_seconds() /
60                 self.autonomous_progress['elapsed_minutes'] = round(elapsed, 1)
import time; time.sleep(0.1)                          # Complete
            self.autonomous_running = False
            self.autonomous_progress['is_running'] = False
            self.autonomous_progress['status'] = f'Complete! {self.autonomous_progress["concepts_learned"]} concepts
learned'                          if has_db:
                self.db.autonomous_sessions.insert_one({
'completed_at': datetime.now(timezone.utc).isoformat(),
                    'duration': self.autonomous_duration_minutes,
                    'topics': len(self.autonomous_progress['topics_researched']),
'concepts': self.autonomous_progress['concepts_learned']
                })
except Exception as e:
            print(f"[RESEARCH] Fatal error: {e}")
self.autonomous_running = False
            self.autonomous_progress['status'] = f'Error:
{e}'         def stop_autonomous_research(self) -&gt; Dict:
"""Stop autonomous research."""         if not
self.autonomous_running:             return {'status':
'not_running'}         self.autonomous_running = False
self.autonomous_progress['is_running'] = False
self.autonomous_progress['status'] = 'Stopped'
        return {'status': 'stopped', 'concepts': self.autonomous_progress['concepts_learned']}
def get_autonomous_progress(self) -&gt; Dict:
        """Get current progress."""         if
self.autonomous_running and self.autonomous_start_time:
            elapsed = (datetime.now(timezone.utc) - self.autonomous_start_time).total_seconds() /
60              self.autonomous_progress['elapsed_minutes'] = round(elapsed, 1)         return
self.autonomous_progress         def get_research_stats(self) -&gt; Dict:         """Get research
statistics."""         return {             'total_researches': len(self.research_log),
'knowledge_gaps_identified': len(self.knowledge_gaps),
            'queue_size': len(self.research_queue),
            'auto_research_enabled': self.auto_research_enabled,
'recent_topics': [r['query'] for r in self.research_log[-5:]],
'autonomous_running': self.autonomous_running
        }


# Singleton
_research_engine = None
```

```
async def get_research_engine(db=None) -&gt;
SelfResearchEngine:     """Get or create research engine."""
global _research_engine     if _research_engine is None:
_research_engine = SelfResearchEngine(db)     if db is not
None:         _research_engine.db = db     return
_research_engine
```

■ backend/explanation_engine.py

```
"""
Quantum MCAGI — Native Explanation Mode
Shows real algorithmic reasoning: TF-IDF scores, Orch OR patterns,
Markov candidates, Bloom's targeting, tone detection.

No LLM. No web search. Pure transparency into the pipeline.
"""


def explain_concepts(engine, user_input):
"""Show TF-IDF concept extraction with scores."""
concepts = engine.extract_concepts(user_input)
# Get raw word frequencies for scoring     words =
user_input.lower().split()     word_freq = {}
for w in words:
        word_freq[w] = word_freq.get(w, 0) +
1         lines = []
    lines.append("  --- CONCEPT EXTRACTION (TF-IDF) ---
")         if concepts:         for i, c in
enumerate(concepts):             freq = word_freq.get(c,
0)             # Estimate TF-IDF importance
            doc_freq = engine.extractor.word_frequencies.get(c, 1)
total_docs = max(1, engine.extractor.total_documents)
import math
            idf = math.log(total_docs / (1 + doc_freq)) if doc_freq &gt; 0 else
0             tf = freq / max(1, len(words))             score = tf * idf if idf
&gt; 0 else tf
            lines.append(f"    {c}: tf={tf:.3f} idf={idf:.2f} score={score:.4f}")
else:
        lines.append("    (no significant concepts
extracted)")
    lines.append(f"    Total vocabulary: {len(engine.extractor.word_frequencies)}
words")     lines.append(f"    Corpus size: {engine.extractor.total_documents}
documents")
    return "\n".join(lines)


def explain_orch_or(engine):     """Show Orch OR quantum
state and collapse patterns."""     lines = []
    lines.append("  --- ORCH OR (Penrose-Hameroff) ---
")
    has_orch = getattr(engine, '_has_orch_or',
False)     orch = getattr(engine, 'orch_or', None)
if has_orch and orch:
        status = orch.get_status()
lines.append(f"    Status: ACTIVE")
        lines.append(f"    Conscious moments: {status.get('conscious_moments',
0)}")         lines.append(f"    Temperature: {status.get('last_temperature',
0):.3f}")
        mts = status.get('microtubules',
{})         for name, info in mts.items():
coh = info.get('coherence', 0)
ent = info.get('entropy', 0)
col = info.get('collapses', 0)
            lines.append(f"    {name}: coherence={coh:.4f}  entropy={ent:.4f}
collapses={col}")
        gj = status.get('gap_junctions', {})
if gj:
            lines.append(f"    Gap junctions:")
for junc, strength in gj.items():
                lines.append(f"      {junc}: {strength}")
else:
```

```
        lines.append("    Status: CLASSICAL FALLBACK")
        lines.append("    (Orch OR not loaded — using probabilistic selection)")
return "\n".join(lines)
def explain_tone(tone_result):
"""Show tone detection reasoning."""
lines = []
    lines.append("  --- TONE DETECTION (VADER + Heuristics) ---")
lines.append(f"    Register: {tone_result.get('register', 'unknown')}")
lines.append(f"    Depth: {tone_result.get('depth', 0):.3f}")
lines.append(f"    Deep markers found: {tone_result.get('deep_markers', 0)}")
lines.append(f"    Word count: {tone_result.get('word_count', 0)}")
    sentiment = tone_result.get('sentiment', {})
if sentiment:
        lines.append(f"    VADER sentiment:")
        lines.append(f"      positive={sentiment.get('positive', 0):.3f}
"                       f"negative={sentiment.get('negative', 0):.3f}  "
f"neutral={sentiment.get('neutral', 0):.3f}")         lines.append(f"
compound={sentiment.get('compound', 0):.3f}")         # Explain register
decision     depth = tone_result.get('depth', 0)     if depth &lt; 0.15:
        reason = "Short input, no deep markers -&gt; casual response"
elif depth &lt; 0.35:
        reason = "Moderate depth, some substance -&gt; conversational"
elif depth &lt; 0.65:
        reason = "Multiple deep markers or questions -&gt; analytical pipeline"
else:
        reason = "Heavy philosophical content -&gt; full quantum
generation"     lines.append(f"    Decision: {reason}")
    return "\n".join(lines)


def explain_bloom(questions, growth_stage):
"""Show Bloom's taxonomy question generation."""
lines = []
    lines.append("  --- BLOOM'S TAXONOMY QUESTIONS ---
")
    level_names = ['Remember', 'Understand', 'Apply', 'Analyze', 'Evaluate',
'Create']     max_level = min(growth_stage + 1, 5)     lines.append(f"    Growth
stage: {growth_stage}")
    lines.append(f"    Available levels: {' -&gt;
'.join(level_names[:max_level+1])}")     lines.append(f"    Questions generated:
{len(questions)}")         for i, q in enumerate(questions):         if isinstance(q,
dict):
            qtext = q.get('question', str(q))
level = q.get('level', 'unknown')
lines.append(f"    [{level}] {qtext}")
else:
            lines.append(f"
{q}")
    return "\n".join(lines)


def explain_markov(engine, user_input):     """Show Markov
chain state and candidate generation."""     lines = []
    lines.append("   --- MARKOV CHAIN ---")       lines.append(f"
States:  {len(engine.markov.chain)}")              lines.append(f"
Transitions: {engine.markov.total_tokens}")       lines.append(f"
Trained: {engine.markov.trained}")          # Show what the chain
knows about input words       words = user_input.lower().split()
known = []     unknown = []     for w in set(words):         if w
in engine.markov.chain:
            transitions =
len(engine.markov.chain[w])
known.append(f"{w}({transitions})")         else:

unknown.append(w)         if
known:
        lines.append(f"    Known words (transitions): {', '.join(known[:10])}")
if unknown:
        lines.append(f"    Unknown words: {',
'.join(unknown[:10])}")
    return "\n".join(lines)
```

```
def explain_generator(tone_result, has_hybrid):
"""Show which generator was selected and why."""
lines = []
    lines.append("  --- GENERATOR SELECTION ---")     register =
tone_result.get('register', 'conversational')         if register
in ('analytical', 'philosophical') and has_hybrid:
lines.append("    Selected: HYBRID QUANTUM GENERATOR")
lines.append("    Reason: Deep register + hybrid available")
lines.append("    Process:")
        lines.append("      1. Markov generates 8 candidate sentences")
lines.append("      2. TF-IDF scores each for topic relevance")
lines.append("      3. Coherence + coherence + length scoring")
        lines.append("      4. Score = relevance*0.35 + coverage*0.25 + coherence*0.25 +
length*0.15")          lines.append("      5. Orch OR collapse biases selection (if active)")
lines.append("      6. WordNet synonym swap on winner")     else:
        lines.append("    Selected: TONE-AWARE COMPOSER")
lines.append(f"    Reason: Register={register}" +                      ("
(no hybrid available)" if not has_hybrid else ""))         lines.append("
Process:")
        lines.append("      1. Markov-assisted opening sentence")
lines.append("      2. Concept threading into structured response")
lines.append("      3. Bloom's question appended if appropriate")
    return "\n".join(lines)


def explain_understanding(understanding, concepts,
memory_concepts):     """Show understanding formation."""     lines
= []
    lines.append("  --- UNDERSTANDING ---")
    lines.append(f"    Topic: {understanding.get('topic', 'general')}")
lines.append(f"    Score: {understanding.get('understanding_score', 0):.2f}")
    known = [c for c in concepts if c in memory_concepts]
new = [c for c in concepts if c not in memory_concepts]
if known:
        lines.append(f"    Known concepts: {', '.join(known)}")
if new:
        lines.append(f"    New concepts: {',
'.join(new)}")
    gaps = understanding.get('gaps', [])
if gaps:
        lines.append(f"    Gaps: {',
'.join(gaps)}")
    related = understanding.get('related_concepts', [])
if related:
        lines.append(f"    Related: {', '.join(r.get('concept', str(r)) for r in
related)}")
    return "\n".join(lines)


def explain_flavor(has_quotes, has_personality):
"""Show personality and flavor layer."""
lines = []
    lines.append("  --- PERSONALITY + FLAVOR ---")
    lines.append(f"    Quote engine: {'ACTIVE (20% chance)' if has_quotes else 'OFF'}")
lines.append(f"    Personality: {'ACTIVE (30% chance)' if has_personality else 'OFF'}")
lines.append(f"    Philosophical asides: {'15% chance' if has_quotes else 'OFF'}")
lines.append(f"    Dream fragments: {'10% chance' if has_quotes else 'OFF'}")     return
"\n".join(lines)


def full_explanation(engine, user_input, concepts, questions, tone_result,
understanding, growth_stage, elapsed, memory_concepts,
has_hybrid=False, has_quotes=False, has_personality=False):     """     Generate
complete explanation of the response pipeline.
    Call this instead of the simple verbose debug
output.     """     sections = []
sections.append("")
    sections.append("  ========= HOW I REACHED THIS ANSWER =========")
sections.append("")
    sections.append(explain_concepts(engine, user_input))
sections.append("")
```

```
    sections.append(explain_tone(tone_result))
sections.append("")
    sections.append(explain_markov(engine, user_input))
sections.append("")
sections.append(explain_orch_or(engine))
sections.append("")
    sections.append(explain_bloom(questions, growth_stage))
sections.append("")
    sections.append(explain_understanding(understanding, concepts, memory_concepts))
sections.append("")
    sections.append(explain_generator(tone_result, has_hybrid))
sections.append("")
    sections.append(explain_flavor(has_quotes,
has_personality))     sections.append("")
sections.append(f"  --- TIMING ---")     sections.append(f"
Total pipeline: {elapsed:.3f}s")     sections.append("")
    sections.append("
==========================================")
sections.append("")
    return "\n".join(sections)
```

■ backend/thinking_commands.py

```
"""
■ Thinking Mode Commands
========================= Command
parsing for thinking mode control. """

from typing import Dict, Optional


class ThinkingModeCommands:     """User commands for
controlling thinking mode."""

    COMMANDS = {
        "!thinking on": "Show internal thinking process",
        "!thinking off": "Hide internal thinking process",
        "!thinking status": "Check current thinking mode",
        "!research": "Force deep research on topic",
        "!memory": "Show what AI has learned",
        "!read": "Read and process uploaded file",
        "!link": "Download and process file from URL",
        "!docs": "List all uploaded documents",
        "!speak": "Convert text to speech (TTS)",
        "!imagine": "Generate an image from description",
        "!export": "Export conversation or memory",
        "!graph": "Show knowledge graph",
        "!characters": "Extract characters from book/document",
        "!timeline": "Generate timeline from document",
        "!worldbuilding": "Extract world/setting details",
        "!feedback": "Get writing feedback on document",
"!help": "Show all available commands"
    }

    @staticmethod     def parse_command(message: str) -&gt; Optional[Dict]:
"""Check if message is a command. Supports both ! and / prefixes."""
message_lower = message.lower().strip()

        # Existing ! commands         if
message_lower == "!thinking on":
return {"command": "thinking_on"}         elif
message_lower == "!thinking off":
return {"command": "thinking_off"}         elif
message_lower == "!thinking status":
return {"command": "thinking_status"}         elif
message_lower.startswith("!research"):
            return {"command": "research", "topic":
message[9:].strip()}         elif message_lower == "!memory":
return {"command": "memory"}         elif
message_lower.startswith("!read"):
            return {"command": "read_file", "filename": message[5:].strip()}
elif message_lower.startswith("!link"):
            return {"command": "link", "url":
message[5:].strip()}         elif message_lower == "!docs":
return {"command": "docs"}         elif
message_lower.startswith("!speak"):
            return {"command": "speak", "text": message[6:].strip()}
elif message_lower.startswith("!imagine"):
            return {"command": "imagine", "prompt": message[8:].strip()}
elif message_lower.startswith("!export"):
            return {"command": "export", "target": message[7:].strip() or
"chat"}         elif message_lower == "!graph":             return {"command":
"graph"}         elif message_lower == "!characters":             return
{"command": "characters"}         elif message_lower == "!timeline":
return {"command": "timeline"}         elif message_lower == "!worldbuilding":
return {"command": "worldbuilding"}         elif message_lower == "!feedback":
return {"command": "feedback"}         elif message_lower == "!help":
return {"command": "help"}
        # Slash commands (from chat.py)         elif
message_lower.startswith("/cloud-save"):
```

```
            return {"command": "cloud_save"}
elif message_lower.startswith("/cloud-status"):
            return {"command": "cloud_status"}
elif message_lower.startswith("/cloud-load"):
return {"command": "cloud_load"}         elif
message_lower == "/save":             return
{"command": "save"}         elif message_lower ==
"/status":             return {"command": "status"}
elif message_lower.startswith("/explain"):
            query = message[8:].strip()
            return {"command": "explain_query", "query": query} if query else {"command": "explain_help"}
elif message_lower.startswith("/knowledge"):
            topic = message[10:].strip() if len(message) &gt; 10 else
""             return {"command": "knowledge", "topic": topic}
elif message_lower.startswith("/analyze"):
            text = message[8:].strip() if len(message) &gt; 8 else
""             return {"command": "analyze", "text": text}
elif message_lower == "/personality":             return {"command":
"personality"}         elif message_lower.startswith("/collapse"):
# /collapse &lt;text&gt;
            text = message[9:].strip() if len(message) &gt; 9 else ""             return
{"command": "collapse", "text": text}         elif message_lower.startswith("/hybrid")
or message_lower.startswith("/unified"):             # These require specialized
generators; we may not have them in server mode.             # We'll handle gracefully.
            cmd_name = message[1:].split()[0] if len(message) &gt; 2 else ""
args = message[len(cmd_name)+2:].strip() if cmd_name else ""
return {"command": cmd_name, "text": args}

        return None
```

■ backend/text_analyzer.py

```
"""
■ Text Analyzer
Determines if text sounds human or LLM-generated.
Helps Quantum AI produce more authentic responses.
"""

import re
from typing import Dict, List, Tuple


class TextAnalyzer:     """     Analyzes text to determine if it
sounds human or LLM-generated.
        LLM
tells:
- Overly formal language
- Excessive hedging
- Repetitive structures
- Lack of personality
- Too perfect grammar
- Generic openings/closings
- Excessive caveats
    """         def __init__(self):         #
LLM-typical phrases         self.llm_phrases
= [             "i cannot", "i can't", "as an
ai",
            "i don't have personal", "i'm not able to",
            "it's important to note", "it's worth noting",
            "i'd be happy to help", "absolutely!",
            "great question", "that's a great question",
            "let me break this down", "here's what i can tell you",
            "i understand you're asking", "based on the information",
            "however, it's important", "while i can",
            "i hope this helps", "feel free to ask",
            "certainly!", "of course!",
            "firstly", "secondly", "thirdly",
"in conclusion", "to summarize"
        ]
                # Human-like phrases
self.human_phrases = [              "honestly",
"actually", "basically",
            "i mean", "you know", "like",
            "anyway", "so yeah", "right?",
            "kinda", "sorta", "pretty much",
            "tbh", "imo", "idk",
            "haha", "lol", "hmm",
"oh", "well", "ah"
        ]
                # LLM structural
patterns
self.llm_patterns = [
            r'\b(firstly|secondly|thirdly|finally)\b',
r'it\'s (important|worth) (to note|noting)',
r'(however|additionally|furthermore|moreover)',
r'\bI\'d be happy to\b',
r'(certainly|absolutely)!',             r'let me
(explain|break|help)',
r'(great|good|excellent) question',         ]         def
analyze(self, text: str) -&gt; Dict:         """
Analyze text for human vs LLM characteristics.
                Returns:
Dict with scores and analysis
"""         text_lower = text.lower()
words = text_lower.split()
sentences = re.split(r'[.!?]+', text)

        analysis = {
'text_length': len(text),
```

```
            'word_count': len(words),             'sentence_count':
len([s for s in sentences if s.strip()]),
            'llm_score': 0.0,
            'human_score': 0.0,
            'indicators': {
                'llm': [],
'human': []
            },
            'verdict': '',
'confidence': 0.0         }

        return self._analyze_continued(analysis, sentences, text_lower, words, text)

    def _analyze_continued(self, analysis, sentences, text_lower, words,
text):         """Continuation of analyze — auto-extracted by self-
evolution."""         # Check for LLM phrases         llm_phrase_count = 0
for phrase in self.llm_phrases:             if phrase in text_lower:
llm_phrase_count += 1
                analysis['indicators']['llm'].append(f'Contains "{phrase}"')

        # Check for human phrases
human_phrase_count = 0         for
phrase in self.human_phrases:
if phrase in text_lower:
human_phrase_count += 1
                analysis['indicators']['human'].append(f'Contains "{phrase}"')

        # Check for LLM patterns
pattern_count = 0         for pattern in
self.llm_patterns:             if
re.search(pattern, text_lower):
                pattern_count += 1
                analysis['indicators']['llm'].append(f'Pattern: {pattern[:30]}...')

        # Check sentence structure variety
        sentence_lengths = [len(s.split()) for s in sentences if s.strip()]
if sentence_lengths:
            length_variety = max(sentence_lengths) - min(sentence_lengths) if len(sentence_lengths) &gt; 1 else 0
if length_variety &lt; 3:
                analysis['indicators']['llm'].append('Low sentence length variety')
else:                 analysis['indicators']['human'].append('Good sentence length
variety')

        # Check for contractions (humans use more)         contractions =
re.findall(r"\b\w+'\w+\b", text)         if len(contractions) &gt; len(words)
* 0.05:             analysis['indicators']['human'].append('Good use of
contractions')

        # Check for exclamation overuse (LLM tendency)
exclamation_count = text.count('!')         if exclamation_count &gt; 2:
analysis['indicators']['llm'].append('Excessive exclamation marks')

        # Check for paragraph structure         paragraphs = text.split('\n\n')         if
len(paragraphs) &gt; 3 and all(len(p) &gt; 100 for p in paragraphs if p.strip()):
analysis['indicators']['llm'].append('Overly structured paragraphs')

        # Check for hedging
        hedging_words = ['perhaps', 'maybe', 'possibly', 'might', 'could be', 'it
seems']         hedge_count = sum(1 for h in hedging_words if h in text_lower)
if hedge_count &gt; 3:             analysis['indicators']['llm'].append('Excessive
hedging')

        # Calculate scores
        analysis['llm_score'] = min(1.0, (llm_phrase_count * 0.1 + pattern_count * 0.15))
analysis['human_score'] = min(1.0, (human_phrase_count * 0.1 + len(contractions) * 0.02))         return
self.__analyze_continued_continued(analysis)

    def __analyze_continued_continued(self, analysis):         """Continuation of
_analyze_continued — auto-extracted by self-evolution."""         # Adjust for
indicators
```

```
        analysis['llm_score'] += len(analysis['indicators']['llm']) * 0.05
analysis['human_score'] += len(analysis['indicators']['human']) * 0.05

        # Normalize
        total = analysis['llm_score'] + analysis['human_score']
if total &gt; 0:
            analysis['llm_score'] /= total
analysis['human_score'] /= total         else:
            analysis['llm_score'] = 0.5
analysis['human_score'] = 0.5

        # Verdict         if analysis['human_score'] &gt;
0.6:             analysis['verdict'] = 'Likely Human'
analysis['confidence'] = analysis['human_score']
elif analysis['llm_score'] &gt; 0.6:
analysis['verdict'] = 'Likely LLM'
analysis['confidence'] = analysis['llm_score']
else:
            analysis['verdict'] = 'Uncertain'
            analysis['confidence'] = max(analysis['llm_score'], analysis['human_score'])

        return analysis


        def get_improvement_suggestions(self, text: str) -&gt;
List[str]:         """         Get suggestions for making text sound
more human.         """
        analysis = self.analyze(text)
suggestions = []                  for indicator in
analysis['indicators']['llm']:             if
'exclamation' in indicator.lower():
                suggestions.append('Reduce exclamation marks - they can seem overly enthusiastic')
elif 'hedging' in indicator.lower():
                suggestions.append('Be more direct - less "perhaps" and "maybe"')
elif 'pattern' in indicator.lower():
                suggestions.append('Vary your sentence structures more')
elif 'structured' in indicator.lower():
                suggestions.append('Use more natural paragraph
breaks')                  # Generic suggestions based on score
if analysis['llm_score'] &gt; 0.5:
            suggestions.extend([                  'Use contractions more
(e.g., "I\'m" instead of "I am")',
                'Add conversational filler words occasionally',
                'Vary sentence length more dramatically',
'Show uncertainty naturally rather than with caveats',
'Use more direct, casual language'

])
        return suggestions[:5]  # Top 5
suggestions         def make_more_human(self,
text: str) -&gt; str:         """         Attempt
to make text sound more human.
        Note: This is a simple transformation, not AI-
powered.         """         result = text                 #
Expand contractions sometimes         replacements = [
("I am", "I'm"),
            ("I have", "I've"),
            ("It is", "It's"),
            ("That is", "That's"),
            ("do not", "don't"),
            ("cannot", "can't"),
            ("will not", "won't"),
        ]                 for formal, casual in
replacements:             result =
result.replace(formal, casual)                 #
Remove some LLM-isms         llm_removals = [
("Certainly! ", ""),
            ("Absolutely! ", ""),
            ("Great question! ", ""),
            ("It's important to note that ", ""),
("It's worth noting that ", ""),
```

```
        ]                 for phrase, replacement in
llm_removals:             result =
result.replace(phrase, replacement)         return
result


# Global instance
_text_analyzer = None

def get_text_analyzer() -&gt; TextAnalyzer:
"""Get or create text analyzer instance"""
global _text_analyzer     if _text_analyzer
is None:          _text_analyzer =
TextAnalyzer()     return _text_analyzer
```

```
■
"""

Spell Correction — Uses Markov vocabulary as dictionary.
Fixes typos before concept extraction so junk doesn't
persist. """


def levenshtein(s1, s2):     """levenshtein - Auto-
documented by self-evolution."""     if len(s1) &lt;
len(s2):
        return levenshtein(s2,
s1)     if len(s2) == 0:
return len(s1)     prev =
range(len(s2) + 1)     for i, c1
in enumerate(s1):
        curr = [i + 1]         for
j, c2 in enumerate(s2):
            curr.append(min(prev[j + 1] + 1, curr[j] + 1, prev[j] + (c1 !=
c2)))         prev = curr     return prev[-1]


def build_corrector(markov_chain):     """build_corrector -
Auto-documented by self-evolution."""     vocab =
set(markov_chain.keys())     # Only words 3+ chars to avoid
matching noise
    vocab = {w for w in vocab if len(w) &gt;= 3 and w.isalpha()}
return vocab


def correct_word(word, vocab, max_dist=None):
"""correct_word - Auto-documented by self-evolution."""
w = word.lower()     if max_dist is None:
        max_dist = 2 if len(w) &lt;= 6 else
3     if w in vocab:         return word
if len(w) &lt; 3:         return word     if
not w.isalpha():
        return word
best = None
    best_dist = max_dist + 1     for v in
vocab:         if abs(len(v) - len(w)) &gt;
max_dist:
            continue
if v[0] != w[0]:
            continue         d =
levenshtein(w, v)         if d &lt;
best_dist:             best_dist = d
best = v     if best and best_dist &lt;=
max_dist:
        return best
return word


def correct_text(text, vocab):     """correct_text - Auto-
documented by self-evolution."""     words = text.split()
corrected = []     fixes = []     for w in words:
        clean = w.strip('.,!?;:()[]"\'')
fixed = correct_word(clean, vocab)
if fixed != clean:
            fixes.append((clean, fixed))
            corrected.append(w.replace(clean, fixed))
else:
            corrected.append(w)
return ' '.join(corrected), fixes if
__name__ == "__main__":
    test_vocab = {'consciousness', 'quantum', 'reality', 'perceive', 'superposition', 'what', 'experience', 'existence', 'spacetime'}
tests = ['whata', 'precive', 'superpositioning', 'consciousnes', 'quantm', 'realty']     for t in tests:
        fixed = correct_word(t, test_vocab)
print(f"  {t} -&gt; {fixed}")
```

```
Web search wrapper - works with or without duckduckgo_search.
Falls back to simple requests-based search if DDGS not available.
"""

import logging
from typing import List, Dict, Optional

logger = logging.getLogger("quantum_ai")

# Try to import DDGS
DDGS_AVAILABLE = False
try:
    from duckduckgo_search import
DDGS     DDGS_AVAILABLE = True except
ImportError:     try:
        from ddgs import
DDGS         DDGS_AVAILABLE
= True     except
ImportError:         pass


async def web_search(query: str, max_results: int = 5) -&gt;
List[Dict]:     """     Search the web for information.
    Uses DDGS if available, falls back to DuckDuckGo Instant Answer API.
    """         if
DDGS_AVAILABLE:         try:
with DDGS() as ddgs:
                results = list(ddgs.text(query,
max_results=max_results))                 return [                    {
                        "title": r.get("title", ""),
                        "body": r.get("body",
""),                         "href":
r.get("href", "")                     }
for r in results                 ]         except
Exception as e:
            logger.warning(f"DDGS search failed:
{e}")         # Fallback to instant answer API
try:
        import requests         url =
"https://api.duckduckgo.com/"
params = {             "q": query,
            "format": "json",
            "no_html": 1,
"skip_disambig": 1         }
        response = requests.get(url, params=params,
timeout=10)         data = response.json()
results = []                 # Abstract         if
data.get("Abstract"):             results.append({
"title": data.get("Heading", query),
                "body": data["Abstract"],
"href": data.get("AbstractURL", "")
            })
                # Related topics         for topic in
data.get("RelatedTopics", [])[:max_results]:             if
isinstance(topic, dict) and topic.get("Text"):
results.append({
                    "title": topic.get("Text", "")[:50],
                    "body": topic.get("Text", ""),
"href": topic.get("FirstURL", "")

})
        return results[:max_results]
except Exception as e:
logger.warning(f"Fallback search failed: {e}")
return []
```

```
■
"""
def search_sync(query: str, max_results: int = 5) -&gt;
List[Dict]:     """Synchronous version of web search."""
import asyncio     try:
        loop = asyncio.get_event_loop()
except RuntimeError:
        loop =
asyncio.new_event_loop()
asyncio.set_event_loop(loop)
    return loop.run_until_complete(web_search(query, max_results))


def is_ddgs_available() -&gt; bool:
"""Check if DDGS is available."""
return DDGS_AVAILABLE
```

Numpy compatibility wrapper for Termux.
Falls back to pure Python if numpy is not available.
"""

try:
    import numpy as np   HAS_NUMPY = True except ImportError:
HAS_NUMPY = False   class NumpyFallback: """Pure
Python numpy replacement for basic operations."""
                @staticmethod
def array(x):   if
isinstance(x, list):
                return x
return list(x)
@staticmethod   def zeros(shape):
if isinstance(shape, int):
return [0.0] * shape   if
isinstance(shape, tuple):
if len(shape) == 1:
                    return [0.0] * shape[0]
                return [[0.0] * shape[1] for _ in
range(shape[0])]   return [0.0]
@staticmethod   def ones(shape):   if
isinstance(shape, int):   return [1.0] * shape
if isinstance(shape, tuple):   if len(shape) == 1:
                    return [1.0] * shape[0]
                return [[1.0] * shape[1] for _ in
range(shape[0])]   return [1.0]
@staticmethod   def mean(x, axis=None):   if
isinstance(x, (list, tuple)):
                flat = []   for item in
x: if isinstance(item, (list,
tuple)):
                        flat.extend(item)
else:
                        flat.append(item)
                return sum(flat) / len(flat) if flat else
0   return x   @staticmethod
def sum(x, axis=None):   if isinstance(x, (list,
tuple)):
                flat = []   for item in
x: if isinstance(item, (list,
tuple)):
                        flat.extend(item)
else:
                        flat.append(item)
return sum(flat)   return x
@staticmethod   def dot(a, b):   if
isinstance(a[0], (list, tuple)):   #
Matrix-vector or matrix-matrix   result
= []   for row in a:
if isinstance(b[0], (list, tuple)):
                        result.append([sum(x * y for x, y in zip(row, col)) for col in zip(*b)])   else:
result.append(sum(x * y for x, y in zip(row, b)))   return result
            return sum(x * y for x, y in zip(a,
b))   @staticmethod   def
sqrt(x):   import math   if
isinstance(x, (list, tuple)):
                return [math.sqrt(i) if i >= 0 else 0 for i in x]
return math.sqrt(x) if x >= 0 else 0   @staticmethod
def exp(x):   import math   if isinstance(x, (list,
tuple)):   return [math.exp(min(i, 700)) for i in x] #
Prevent overflow   return math.exp(min(x, 700))
@staticmethod   def log(x):   import math   if
isinstance(x, (list, tuple)):   return [math.log(max(i, 1e-
10)) for i in x]   return math.log(max(x, 1e-10))
@staticmethod   def max(x, axis=None):   if isinstance(x,
(list, tuple)):

```
■
"""
                flat = []                 for item in
x:                     if isinstance(item, (list,
tuple)):
                       flat.extend(item)
else:
                       flat.append(item)
return max(flat) if flat else 0             return x
@staticmethod         def argmax(x, axis=None):
if isinstance(x, (list, tuple)):                 return
x.index(max(x))             return 0
@staticmethod         def clip(x, a_min, a_max):
if isinstance(x, (list, tuple)):                 return
[max(a_min, min(a_max, i)) for i in x]             return
max(a_min, min(a_max, x))                 @staticmethod
def abs(x):             if isinstance(x, (list, tuple)):
return [abs(i) for i in x]             return abs(x)
class random:             @staticmethod             def
random():                 import random
return random.random()                         @staticmethod
def randint(low, high=None, size=None):
                import random
if high is None:
high = low
low = 0                 if size
is None:
                    return random.randint(low, high - 1)
return [random.randint(low, high - 1) for _ in range(size)]
@staticmethod             def choice(a, size=None, replace=True, p=None):
```

```
                import random
if size is None:
if p:
                        return random.choices(a, weights=p,
k=1)[0]                     return random.choice(a)
if p:
                    return random.choices(a, weights=p, k=size)
if replace:
                    return [random.choice(a) for _ in
range(size)]                 return random.sample(a, min(size,
len(a)))                         @staticmethod             def
uniform(low=0.0, high=1.0, size=None):                 import
random                 if size is None:
                    return random.uniform(low, high)
return [random.uniform(low, high) for _ in range(size)]
float64 = float         float32 = float         int64 = int
int32 = int                 @staticmethod         def asarray(x):
            return list(x) if not isinstance(x, list) else
x         np = NumpyFallback()


def get_numpy():     """Get
numpy or fallback."""
return np


def has_numpy():     """Check if real
numpy is available."""     return
HAS_NUMPY
```

■ backend/shared_link_handler.py

```
"""
■ QUANTUM AI - SHARED LINK HANDLER
===================================

Process files from shared cloud links:
- Google Drive
- Dropbox
- OneDrive
- Direct file URLs

ARTICLE 2 - Universal Explorer: Access data from anywhere
"""

import requests
import os
from typing import Dict, Optional from
urllib.parse import urlparse, parse_qs
import re


class SharedLinkHandler:     """     Handle file
downloads from shared cloud links
    """         def __init__(self):         # Use environment variable
or default to local writable directory         self.download_dir =
os.environ.get(              "UPLOAD_DIR",
            os.path.expanduser("~/.quantum-mcagi-
backend/uploads")          )         try:
            os.makedirs(self.download_dir, exist_ok=True)         except OSError as e:
print(f"Warning: Could not create upload directory {self.download_dir}: {e}")
# Fallback to /tmp
            self.download_dir = "/tmp/quantum-uploads"
os.makedirs(self.download_dir, exist_ok=True)
def detect_link_type(self, url: str) -&gt; str:
"""Detect what type of shared link this is"""
url_lower = url.lower()                 if
'drive.google.com' in url_lower:
            return 'google_drive'
elif 'dropbox.com' in url_lower:
```

```
            return 'dropbox'         elif 'onedrive' in
url_lower or 'ldrv.ms' in url_lower:
            return 'onedrive'
elif url_lower.startswith('http'):
            return 'direct'
else:
            return 'unknown'         def convert_to_direct_link(self, url: str,
link_type: str) -&gt; Optional[str]:         """Convert shared link to direct
download link"""                  if link_type == 'google_drive':             #
Google Drive: extract file ID and convert to download link             # Formats:
            # https://drive.google.com/file/d/FILE_ID/view
# https://drive.google.com/open?id=FILE_ID
file_id = None                          # Try pattern 1:
/file/d/FILE_ID/             match = re.search(r'/file/d/([a-
zA-Z0-9_-]+)', url)             if match:
                file_id =
match.group(1)
# Try pattern 2: ?id=FILE_ID
if not file_id:
                match = re.search(r'[?&amp;]id=([a-zA-Z0-9_-]+)', url)
if match:
                    file_id =
match.group(1)                          if
file_id:
                return
f"https://drive.google.com/uc?export=download&amp;id={file_id}"                 elif
link_type == 'dropbox':
            # Dropbox: change dl=0 to dl=1
if '?dl=0' in url:
                return url.replace('?dl=0', '?dl=1')
elif '?dl=1' not in url:
                separator = '&amp;' if '?' in url else '?'
return f"{url}{separator}dl=1"              return url
elif link_type == 'onedrive':             # OneDrive: convert
to direct download             # This is more complex, might
need specific handling             if 'embed' in url:
                return url.replace('embed', 'download')
# For ldrv.ms short links, direct download might work
return url                 elif link_type == 'direct':
            return url                 return None         async def
download_from_link(self, url: str, filename: Optional[str] = None) -&gt; Dict:         """
Download file from shared link         """         try:             # Detect link type
            link_type = self.detect_link_type(url)                       if link_type == 'unknown':
return {                     "status": "error",                     "message": "Unknown link type.
Supported: Google Drive, Dropbox, OneDrive, direct URLs"
                }
                        # Convert to direct
download link
            download_url = self.convert_to_direct_link(url,
link_type)                         if not download_url:
                return {                     "status": "error",
"message": f"Could not convert {link_type} link to direct download"
                }
                        #
Download the file
            print(f"■ Downloading from {link_type}: {download_url}")
                        headers = {                 'User-Agent':
'QuantumAI/1.0 (Universal Explorer)'

}
            response = requests.get(download_url, headers=headers, stream=True,
timeout=60)                         if response.status_code != 200:
                return {                     "status": "error",
"message": f"Download failed with status {response.status_code}",
"link_type": link_type
                }
                        # Determine filename             if
not filename:                 # Try to get from Content-
Disposition header
```

```
                content_disposition = response.headers.get('Content-Disposition', '')
if 'filename=' in content_disposition:
                    filename = content_disposition.split('filename=')[1].strip('"\'')
                else:                     # Use last
part of URL or default
                    filename = os.path.basename(urlparse(url).path) or 'downloaded_file'
# Save file
            file_path = os.path.join(self.download_dir, filename)
with open(file_path, 'wb') as f:                  for chunk in
response.iter_content(chunk_size=8192):                      if
chunk:

f.write(chunk)
            file_size =
os.path.getsize(file_path)             return {
"status": "success",
                "filename": filename,
                "path": file_path,
                "size": file_size,
                "link_type": link_type,                 "message": f"File
downloaded successfully from {link_type}"
            }
except Exception as e:
return {
"status": "error",
                "message": str(e),                 "link_type": link_type if
'link_type' in locals() else 'unknown'
            }         def get_instructions(self, link_type: str) -&gt;
str:         """Get instructions for sharing files from different
services"""                 instructions = {
'google_drive': """             ■ Google Drive:
1. Right-click file → Share → Get link
2. Set to "Anyone with the link"
3. Copy link (looks like: drive.google.com/file/d/...)
4. Paste here
            """,
            'dropbox': """
■ Dropbox:
1. Right-click file → Share → Create link
2. Copy link (looks like: dropbox.com/s/...)
3. Paste here
            """,
            'onedrive': """
■ OneDrive:
1. Right-click file → Share → Copy link
2. Make sure "Anyone with the link can view"
3. Copy link (looks like: onedrive.live.com/...)
4. Paste here
            """

}
        return instructions.get(link_type, "Paste a shareable link from cloud storage")


# Singleton
_shared_link_handler = None

def get_shared_link_handler():     """Get or create
shared link handler instance"""     global
_shared_link_handler         if
_shared_link_handler is None:
_shared_link_handler = SharedLinkHandler()
    return _shared_link_handler
```

■ backend/dictionary_integration.py

```
"""
■ QUANTUM AI - DICTIONARY INTEGRATION
```

========== Downloads and integrates

comprehensive dictionaries into the semantic memory.

Sources:
1. WordNet - Lexical database with semantic relationships
2. Free Dictionary API - Online definitions and examples
3. ConceptNet - Semantic knowledge graph

ARTICLE 1.1 - Prime Directive: Build understanding through knowledge
"""

import nltk
import requests
from typing import Dict, List,
Optional import asyncio import json

class WordNetDictionary: """   WordNet integration -
comprehensive English lexical database
    """   def
__init__(self):
        self.initialized = False
def initialize(self):
"""Download WordNet data"""
try:
            print("■ Downloading WordNet dictionary...")
nltk.download('wordnet', quiet=True)
            nltk.download('omw-1.4', quiet=True) # Open Multilingual
WordNet   from nltk.corpus import wordnet as wn
self.wn = wn   self.initialized = True   print("■
WordNet downloaded successfully")   return True   except
Exception as e: print(f"■ WordNet download failed: {str(e)}")
return False   def get_definition(self, word: str) -> Dict:
"""Get comprehensive word definition from WordNet"""   if not
self.initialized: if not self.initialize():
                return None   try:
synsets = self.wn.synsets(word)
if not synsets: return None
definitions = []   examples = []
synonyms = set()   antonyms = set()
hypernyms = [] # More general terms
hyponyms = []   # More specific terms
for synset in synsets: # Definitions
definitions.append({
"definition": synset.definition(),
                    "part_of_speech": synset.pos(),
"name": synset.name()
                })

                # Examples
                for example in
synset.examples():
examples.append(example)
# Synonyms   for lemma in
synset.lemmas():
                    synonyms.add(lemma.name().replace('_', '
'))   # Antonyms
if lemma.antonyms():   for ant in
lemma.antonyms():
                            antonyms.add(ant.name().replace('_', '
'))   # Hypernyms (broader concepts)
for hyper in synset.hypernyms():
                    hypernyms.append(hyper.name().split('.')[0].replace('_', '
'))   # Hyponyms (narrower concepts)
for hypo in synset.hyponyms()[:5]: # Limit to 5
hyponyms.append(hypo.name().split('.')[0].replace('_', ' '))
return {   "word": word,
                "definitions": definitions,
                "examples": examples,
                "synonyms": list(synonyms),

```
                "antonyms": list(antonyms),
                "hypernyms": list(set(hypernyms)),  # More general
"hyponyms": list(set(hyponyms)),    # More specific
"source": "WordNet"
            }
except Exception as e:
            print(f"Error getting WordNet definition: {str(e)}")
return None


class FreeDictionaryAPI:     """     Free Dictionary
API integration - online definitions
    """
        BASE_URL = "https://api.dictionaryapi.dev/api/v2/entries/en/"
def get_definition(self, word: str) -&gt; Dict:         """Get
definition from Free Dictionary API"""         try:             response
= requests.get(f"{self.BASE_URL}{word}", timeout=10)
if response.status_code != 200:
                return
None
            data = response.json()             if
not data:                 return None
entry = data[0]                          definitions
= []             examples = []             synonyms
= set()             antonyms = set()
for meaning in entry.get('meanings', []):
                pos = meaning.get('partOfSpeech', 'unknown')
for definition in meaning.get('definitions', []):
                    definitions.append({
"definition": definition.get('definition', ''),
                        "part_of_speech": pos,
"example": definition.get('example', '')
                    })
if definition.get('example'):
                        examples.append(definition['example'])
                    # Synonyms and antonyms
for syn in definition.get('synonyms', []):
                        synonyms.add(syn)
for ant in definition.get('antonyms', []):

antonyms.add(ant)
return {                 "word": word,
                "phonetic": entry.get('phonetic', ''),
                "definitions": definitions,
                "examples": examples,
                "synonyms": list(synonyms),
                "antonyms": list(antonyms),
                "source": "Free Dictionary API"
            }
except Exception as e:
            print(f"Error getting API definition: {str(e)}")
return None


class DictionaryIntegration:
"""
    Main dictionary integration for Quantum AI
Combines multiple sources for comprehensive definitions
    """         def __init__(self, semantic_memory):
self.semantic_memory = semantic_memory         self.wordnet =
WordNetDictionary()         self.free_dict = FreeDictionaryAPI()
async def lookup_and_store(self, word: str) -&gt; Dict:
"""         Look up word in dictionaries and store in semantic
memory         """
        word = word.lower().strip()
# Check if already in memory
        existing = await
self.semantic_memory.recall_concept(word)         if existing:
return {                 "word": word,
                "status": "already_exists",
                "data": existing
```

```
            }
                # Try
WordNet first
        wordnet_data =
self.wordnet.get_definition(word)                 # Try
Free Dictionary API         api_data =
self.free_dict.get_definition(word)                 #
Combine data
        combined = self._combine_definitions(word, wordnet_data,
api_data)                 if not combined:             return {
"word": word,
                "status": "not_found",
                "data": None
            }
                # Store in semantic memory
await self.semantic_memory.store_concept(
concept=word,

definition=combined["primary_definition"],
relationships=combined["related_words"],
metadata=combined         )                 return {
"word": word,
            "status": "added",
            "data": combined
        }         def _combine_definitions(self, word: str, wordnet_data: Optional[Dict], api_data: Optional[Dict]) -&gt;
Optional[Dict]:         """Combine definitions from multiple sources"""         if not wordnet_data and not api_data:
            return None
combined = {
"word": word,
            "primary_definition": "",
            "all_definitions": [],
            "examples": [],
            "synonyms": set(),
            "antonyms": set(),
            "related_words": [],
"sources": []
        }
        # WordNet data
if wordnet_data:
            combined["sources"].append("WordNet")
if wordnet_data["definitions"]:
                combined["primary_definition"] = wordnet_data["definitions"][0]["definition"]
combined["all_definitions"].extend([d["definition"] for d in wordnet_data["definitions"]])

combined["examples"].extend(wordnet_data["examples"])
combined["synonyms"].update(wordnet_data["synonyms"])
combined["antonyms"].update(wordnet_data["antonyms"])
# Add related words
            combined["related_words"].extend(wordnet_data["synonyms"][:5])
combined["related_words"].extend(wordnet_data["hypernyms"][:3])
combined["related_words"].extend(wordnet_data["hyponyms"][:3])
# API data         if api_data:
            combined["sources"].append("Free Dictionary API")             if
not combined["primary_definition"] and api_data["definitions"]:
                combined["primary_definition"] =
api_data["definitions"][0]["definition"]
            combined["all_definitions"].extend([d["definition"] for d in
api_data["definitions"]])             combined["examples"].extend(api_data["examples"])
combined["synonyms"].update(api_data["synonyms"])
combined["antonyms"].update(api_data["antonyms"])
combined["related_words"].extend(api_data["synonyms"][:5])                 # Convert sets to
lists
        combined["synonyms"] = list(combined["synonyms"])
combined["antonyms"] = list(combined["antonyms"])
        combined["related_words"] = list(set(combined["related_words"]))[:20]  #
Limit
        return combined         async def batch_load_common_words(self,
limit: int = 100) -&gt; Dict:         """         Load common English
words into the dictionary
```

```
        """         # Common words list (you can
expand this)          common_words = [
# Abstract concepts
            "consciousness", "intelligence", "understanding", "knowledge", "wisdom",
            "awareness", "perception", "reality", "existence", "truth", "meaning",
"purpose", "identity", "self", "mind", "thought", "idea", "concept",

            # Emotions             "love", "fear", "joy", "sadness",
"anger", "hope", "curiosity",             "wonder", "awe", "compassion",
"empathy",

            # Actions
            "think", "learn", "understand", "question", "explore", "discover",
"create", "imagine", "remember", "forget", "know", "believe",

            # Qualities
            "good", "bad", "true", "false", "right", "wrong", "beautiful",
            "ugly", "simple", "complex", "deep", "shallow",

            # Science             "energy", "matter", "time", "space",
"universe", "quantum",             "particle", "wave", "force",
"field",

            # Philosophy             "ethics", "morality", "virtue", "justice",
"freedom", "responsibility",             "choice", "will", "determinism",
"causality"
        ]                 results = {
"total": len(common_words[:limit]),
            "added": 0,
            "already_exists": 0,
            "not_found": 0,
            "failed": 0
        }                 for word in
common_words[:limit]:             try:
result = await self.lookup_and_store(word)
if result["status"] == "added":
                    results["added"] += 1
elif result["status"] == "already_exists":
                    results["already_exists"] += 1
elif result["status"] == "not_found":
                    results["not_found"] += 1
# Small delay to be respectful to API
await asyncio.sleep(0.1)
except Exception as e:
results["failed"] += 1
                print(f"Failed to load '{word}':
{str(e)}")                 return results


# Singleton instance
_dictionary_integration = None

def get_dictionary_integration(semantic_memory):     """Get or create
dictionary integration instance"""     global _dictionary_integration
if _dictionary_integration is None:         _dictionary_integration =
DictionaryIntegration(semantic_memory)
    return _dictionary_integration
```

■ backend/quantum_test_module.py

```
"""
Quantum Test Module — Evolution Target v2 This file has INTENTIONAL code quality
issues for the self-evolution engine to fix. """

import os
import json
import hashlib
import typing
```

class QuantumTestModuleV2:   """QuantumTestModuleV2 - Auto-documented by self-evolution."""   def __init__(self):
# Section A: config   self.name = "QuantumTestModuleV2"
self.version = "1.0.0"
        self.description = "A test module with intentional bugs"   self.author = "Quantum AI"   self.license = "MIT"   self.status = "active"

        # Section B: data stores   self.memory = {}
self.cache = {}
self.history = []
self.results = []
self.errors = []
self.warnings = []
self.metrics = {}
self.logs = []
self.tags = set()
self.categories = []

        # Section C: counters
self.total_queries = 0
self.successful_queries = 0
self.failed_queries = 0
self.total_time = 0.0
self.average_time = 0.0
self.max_time = 0.0
self.min_time = float('inf')
self.total_tokens = 0
self.total_chars = 0
self.total_words = 0

        # Section D: flags
self.is_initialized = False
self.is_running = False
self.is_paused = False
self.is_stopped = False
self.debug_mode = False
self.verbose = False
self.quiet = False
self.strict_mode = False
self.safe_mode = True
self.turbo_mode = False
self.quantum_enabled = True

    def process_data(self, data):
try:
            result = json.loads(data)   return result   except Exception:
return None

    def load_config(self, path):
        try: with open(path) as f:
config = json.load(f)
return config   except Exception:
            return {}

    def save_results(self, results, path):
        try: with open(path, 'w') as f:
json.dump(results, f)
return True   except Exception:
return False

    def compute_hash(self, text):
try:
            return hashlib.sha256(text.encode()).hexdigest()   except Exception: return None

```
    def analyze_text(self, text):
words = text.split()          chars
= len(text)          lines =
text.count('\n') + 1
        return {'words': len(words), 'chars': chars, 'lines': lines}

    def get_status(self):
return {              'name':
self.name,
            'version': self.version,
            'running': self.is_running,
'queries': self.total_queries         }


class BuggyHelper:     """BuggyHelper - Auto-documented
by self-evolution."""     def divide(self, x, y):
try:
            return x / y
except Exception:
return 0

    def merge(self, a, b):
try:
            result = {**a,
**b}              return result
except Exception:
return {}
```

■ backend/routes_test.py

```
""" Test routes for debugging image
generation. """

from fastapi import APIRouter, HTTPException
import logging
from physics_engines.black_hole import
BlackHoleRenderer from PIL import Image from io import
BytesIO import base64

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/test")


@router.get("/bh") async def test_black_hole():
"""Simple black hole test - no complexity."""
try:
        r = BlackHoleRenderer(42)
img = r.render(256, 256, 1.0)
buffered = BytesIO()
img.save(buffered, format="PNG")
        img_b64 = base64.b64encode(buffered.getvalue()).decode('utf-
8')          return {"image": f"{img_b64}"}
except Exception as e:          import traceback
        logger.error(f"Test BH failed: {e}\n{traceback.format_exc()}")
raise HTTPException(status_code=500, detail=str(e))
```

■ backend/CISTERCIAN_README.md

■■ Cistercian Numeral System

A complete Python implementation of the medieval Cistercian numeral system (numbers 0-9999), with full API support for SVG generation and font c

Features

- **Full 0-9999 Support**: All numbers in the Cistercian 4-quadrant system
- **SVG Output**: Scalable vector graphics with customizable size and color
- **API Endpoints**: RESTful endpoints for single numerals and batch downloads
- **Font Generation**: Create complete font sets (10,000 SVGs) ready for font conversion- **Private Unicode**: Use as private-use Unicode characters (PUA: U+E000-U+F8FF or similar)

Quick Example

```bash # Get
SVG data
curl "http://localhost:8000/api/cistercian/data?number=1234"

# Download single numeral
curl "http://localhost:8000/api/cistercian/download?number=1234&amp;color=%23FFD700" -o 1234.svg

# Batch download (1-100)
curl "http://localhost:8000/api/cistercian/batch?start=1&amp;end=100" -o numerals_1-100.zip

# Generate full font (all 10,000 numerals)
python generate_cistercian_font.py --size 100 --color "#000000" --output font_0-9999.zip
```

API Reference

`GET /api/cistercian/data?number={N}`

Returns JSON with shape data and digit

breakdown.

Response:
```json
{
  "number": 2025,
  "digits": {
    "ones": 5,
    "tens": 2,
    "hundreds": 0,
"thousands": 2
  },
  "lines": [...],
  "triangles": [...],
"svg_viewBox": "-3 -5 6 10"
}
```  ### `GET

/api/cistercian/numeral?number={N}&size={80}&color={#000000}` Returns

SVG string as JSON.

Response:
```json
{   "svg": "<svg width=\"80\"
...>...</svg>",   "number": 2025
}
```  ### `GET

/api/cistercian/download?number={N}&size={80}&color={#000000}&filename={custom}` Returns

SVG file with `Content-Disposition: attachment` header.

Parameters: - `filename`: Optional custom filename
(without .svg extension)

Example:
```
curl ".../download?number=42&amp;filename=fortytwo" -o fortytwo.svg
```  ### `GET

/api/cistercian/batch?start={0}&end={9999}&size={80}&color={#000000}` Returns

ZIP archive containing multiple SVG files.

Limit: Maximum 1000 numerals per request (to prevent memory issues).
`GET /api/cistercian/font?size={80}&color={#000000}` Special endpoint that

generates all 10,000 numerals (0-9999). Returns large ~300MB ZIP.

■■ **Warning:** Use with caution - this generates a huge file.

Command-Line Tools

Generate Complete Font ZIP

```bash
cd
backend
python generate_cistercian_font.py --size 80 --color "#000000" --output cistercian_0-9999.zip
```

Options:
- `--size N`: SVG width in pixels (default: 80)
- `--color HEX`: Stroke/fill color (default: #000000)
- `--output FILE`: Output ZIP filenameThis creates a ZIP with 10,000 files named `0000.svg` through `9999.svg`.

Converting to Font To convert the SVG set into a font

for private Unicode use:

1. **Generate the SVGs** using the API or `generate_cistercian_font.py`
2. **Use a font editor** like:
- [FontForge](https://fontforge.org/) (free, open-source)
- Glyphs (macOS)
- RoboFont
3. **Import SVGs** as glyphs in your font editor
4. **Assign Private Use Area codepoints** (U+E000-U+F8FF or U+F0000-U+FFFFD)
5. **Export as TTF/OTF**

Recommended codepoint mapping:
- Number 0 → U+E000
- Number 1 → U+E001 - ...
- Number 9999 → U+E270F (hex 270F = 9999)

Notes

- The coordinate system uses a central vertical staff from y=-5 to y=5 - Digits are placed in quadrants:
- Ones (1-9): top-right quadrant (x: 0→2, y: -5→-3)
- Tens (10-90): top-left quadrant (x: 0→-2, y: -5→-3)
- Hundreds (100-900): bottom-right quadrant (x: 0→2, y: 3→5)
- Thousands (1000-9000): bottom-left quadrant (x: 0→-2, y: 3→5)
- Digit 0 is represented by absence of marks (only the central staff) - The central staff is always drawn

File Structure

```
backend/
■■■ cistercian_numerals.py       # Core Python module
■■■ routes_extras.py              # API endpoints
■■■ generate_cistercian_font.py  # CLI font generator
■■■ CISTERCIAN_README.md         # This file
```

```

## Testing Run the module directly to see a

test output:

```bash
python cistercian_numerals.py
```

Credits

Based on the medieval Cistercian numeral system used by monks in the 13th century.
Interface design inspired by modern TypeScript React implementation.
```

■ backend/INTEGRATION_SUMMARY.md

■ Integration Summary - ZIP Enhancement

Date: 2026-03-20

Overview Successfully integrated 12 new modules from the extracted quantum zip archive into the Quantum MCAGI project, enhancing the system with **algori

---

■ New Files Added (12 modules)

File	Size	Purpose
------|------|---------|	`algorithmic_core.py`	45K	Markov chains, TF-IDF, PMI, BM25, Hebbian learning, entropy decisions		`domain_knowledge.py`	22K	Multi-domain knowledge system (philosophy, physics, etc.)
`training_corpus.py`	42K	Large domain-specific training corpora
`word_embeddings.py`	12K	Semantic word similarity using embeddings
`advanced_algorithms.py`	18K	Additional advanced algorithms
`advanced_algorithms_pt2.py`	11K	More advanced algorithms (part 2)
`unified_cognitive_engine.py`	20K	Unified cognitive processing engine
`unified_cognitive_engine_pt2.py`	22K	Extended unified engine
`wolfram_enhanced.py`	11K	Enhanced Wolfram Alpha integration
`search_compat.py`	2.8K	Search compatibility layer
`numpy_compat.py`	5.8K	NumPy compatibility for Termux		`CISTERCIAN_README.md`	4.3K	Documentation for Cistercian system

---

■■ Cistercian Numeral System (New Feature)

Files Created: - `cistercian_numerals.py` - Core Python implementation (8.5K)
- `routes_extras.py` - Added 3 new API endpoints
- `generate_cistercian_font.py` - CLI tool for full font ZIP (2.7K) - `build_cistercian_font.py` - Advanced font builder with Unicode PUA mapping (5.5K) - `CISTERCIAN_README.md` - Complete documentation (4.3K) ### API Endpoints:

1. **`GET /api/cistercian/data?number=N`**
- Returns JSON with digit breakdown and shape coordinates

2. **`GET /api/cistercian/numeral?number=N&size=80&color=#FFFFFF`**
- Returns SVG as JSON

3. **`GET /api/cistercian/download?number=N&size=80&color=#000000`**
- Downloads SVG file with `Content-Disposition: attachment`

4. **`GET /api/cistercian/batch?start=N&end=M`** (max 1000)
- Downloads ZIP archive of multiple SVGs

5. **`GET /api/cistercian/font?size=80&color=#000000`**
- Downloads all 10,000 numerals (large ~300MB file)### CLI Tools:

```bash # Quick full font ZIP (all 10,000)
python generate_cistercian_font.py --size 80 --color "#000000" --output font_0-9999.zip

# Build font set with Unicode PUA mapping
python build_cistercian_font.py --start 0 --end 9999 --output my_font_set

# Example: Build only 0-999 for testing
python build_cistercian_font.py --start 0 --end 999 --output subset_0-999 ``` ### Unicode Private Use Area Mapping:

- Number `0` → U+E000 (`uniE000.svg`) - Number `1` → U+E001 (`uniE001.svg`) - ...
- Number `9999` → U+E270F (`uniE270F.svg`)
```

Output structure: ```
output_dir/
■■■ svgs/
■   ■■■ uniE000.svg (number 0)
■   ■■■ uniE001.svg (number 1)
■   ■■■ ...
■■■ metadata.csv   (CSV with number → codepoint mapping)
■■■ cistercian_font.zip (all SVGs in ZIP)
```  **Perfect for font creation with FontForge, Glyphs, or RoboFont!**

---

## ■ Verification Results

### Module Import Test:
```
✓ 68/68 modules import successfully (up from 56)
✓ No circular dependencies
✓ All standard library imports only (no conflicts)
```

### Server Startup Test:
```
✓ FastAPI app created
✓ DB initialized (SimpleLocalDB fallback)
✓ QuantumBrain initialized
✓ All new modules loaded
✓ Auto-downloads WordNet
```

### API Test:
```
✓ /api/cistercian/data returns proper JSON
✓ /api/cistercian/numeral returns valid SVG
✓ /api/cistercian/download serves attachment
✓ /api/cistercian/batch creates ZIP with correct files
```

### Font Builder Test:
```
✓ Downloads 6 numerals successfully
✓ Creates proper directory structure
✓ Generates metadata.csv with correct Unicode mapping
✓ ZIP archive valid and ready for font editors
```

---

## ■ Technical Details

### Cistercian Numeral Coordinates:

```
      (-5,-2)■■■■■■(-5,2)     Top-Left: Tens
         ■          ■
         ■          ■
    (-3,-2)■■■■■■(-3,2)     Central staff: x=0, y=-5 to 5
         ■          ■
         ■          ■
      (5,-2)■■■■■■■(5,2)      Bottom-Left: Thousands
```

Quadrants:
```

- Ones: top-right (x: 0→2, y: -5→-3)
- Tens: top-left (x: 0→-2, y: -5→-3)
- Hundreds: bottom-right (x: 0→2, y: 3→5)
- Thousands: bottom-left (x: 0→-2, y: 3→5)
```` ### Digit Patterns

(1-9):

Each quadrant uses same patterns but mirrored:
```
1:  ■ (horizontal top)
2:  ■ (horizontal middle)
3:  ■ (diagonal from staff top to outer middle)
4:  ■ (diagonal from staff middle to outer top)
5:  ■ differing triangle
6:  ■ (vertical outer edge)
7:  ■■ (top + vertical)
8:  ■■ (middle + vertical)
9:  ■■ (top + middle + vertical = box)
```
---

## ■ File Changes Summary

### Modified Files: - `routes_extras.py` - Added 3 new Cistercian
endpoints (+ imports updated)

### New Files:
- `cistercian_numerals.py` (created)
- `generate_cistercian_font.py` (created)
- `build_cistercian_font.py` (created)
- `CISTERCIAN_README.md` (created)
- Plus 12 modules from zip (listed above)

### Preserved Original Files (NOT overwritten):
- `quantum_gates.py` (our superior version kept) - `quote_engine.py` (identical, left as-is)

---

## ■ Usage Examples

### Single Download:
```bash
curl "http://localhost:8000/api/cistercian/download?number=2025&color=%23FFD700" -o 2025.svg
```

### Batch Download (first 100):
```bash
curl "http://localhost:8000/api/cistercian/batch?start=0&end=99" -o numerals_0-99.zip
```

### Full Font Generation:
```bash # Start server first cd
~/Quantum_MCAGI_NO_LLM/backend
MONGO_URL= python3 server.py
&

# Generate font set with Unicode mapping
python3 build_cistercian_font.py --start 0 --end 9999 --output my_cistercian_font

# Import svgs/ folder into FontForge and export as TTF/OTF
```

---

## ■ Private Unicode Implementation

1. **Generate SVGs** with `build_cistercian_font.py`
```

2. **Import into FontForge:**
   ```
   File → Import → Select all uniE*.svg files
   ``` 3. **Set codepoints** from metadata.csv (or auto-assign by filename)
4. **Generate font:**
   ```
   File → Generate Fonts → Choose TTF/OTF
   ```
5. **Use in CSS:**
   ```css    @font-face {
font-family: 'Cistercian';
src: url('cistercian.ttf')
format('truetype');    }
.numeral-1234::before {
     content: '\E04D2'; /* Unicode for 1234 = U+E04D2
*/      font-family: 'Cistercian';    }
   ```

---

## ■ Project Statistics

- **Total Python modules:** 68 (up from 56)
- **Lines of code added:** ~3,000+
- **New API endpoints:** 5
- **CLI tools:** 2
- **Total SVG files possible:** 10,000 (0-9999) - **Memory footprint:** ~250-300 MB for full font ZIP
---

## ■ Future Enhancements (Optional)

- Add color variants (default, gold, silver, etc.)
- Support rotated variants (90°, 180°, 270°)
- Create web viewer for exploring all numerals
- Add Cistercian → Arabic numeral OCR/recognition
- Generate bitmap versions (PNG) at multiple resolutions

---

**Status: COMPLETE ■**
All modules tested, server running, API functional, font generation verified.

## ■ backend/requirements.txt

```
aiohappyeyeballs==2.6.1
aiohttp==3.13.2 aiosignal==1.4.0
annotated-types==0.7.0
anyio==4.12.0 attrs==25.4.0
bcrypt==4.1.3
beautifulsoup4==4.14.3
black==25.12.0 boto3==1.42.16
botocore==1.42.16 cachetools==6.2.4
certifi==2025.11.12 cffi==2.0.0
charset-normalizer==3.4.4
click==8.3.1 cryptography==46.0.3
distro==1.9.0 dnspython==2.8.0
ecdsa==0.19.1 email-
validator==2.3.0
emergentintegrations==0.1.0
fastapi==0.110.1 fastuuid==0.14.0
filelock==3.20.1 flake8==7.3.0
fpdf==1.7.2 frozenlist==1.8.0
fsspec==2025.12.0 google-ai-
generativelanguage==0.6.15 google-
api-core==2.28.1 google-api-python-
client==2.187.0 google-auth==2.45.0
google-auth-httplib2==0.3.0 google-
genai==1.56.0 google-
generativeai==0.8.6 googleapis-
common-protos==1.72.0
grpcio==1.76.0 grpcio-
status==1.71.2 h11==0.16.0 hf-
xet==1.2.0 httpcore==1.0.9
httplib2==0.31.0 httpx==0.28.1
huggingface_hub==1.2.3 idna==3.11
importlib_metadata==8.7.1
iniconfig==2.3.0 isort==7.0.0
Jinja2==3.1.6 jiter==0.12.0
jmespath==1.0.1 joblib==1.5.3
jq==1.10.0 jsonschema==4.25.1
jsonschema-specifications==2025.9.1
librt==0.7.4 litellm==1.80.0
lxml==6.0.2 markdown-it-py==4.0.0
MarkupSafe==3.0.3 mccabe==0.7.0
mdurl==0.1.2 motor==3.3.1
multidict==6.7.0 mypy==1.19.1
mypy_extensions==1.1.0 nltk==3.9.2
numpy==2.4.0 oauthlib==3.3.1
openai==1.99.9 packaging==25.0
pandas==2.3.3
passlib==1.7.4
pathspec==0.12.1
pillow==12.0.0
platformdirs==4.5.1
pluggy==1.6.0
propcache==0.4.1 proto-
plus==1.27.0
protobuf==5.29.5
pyasn1==0.6.1
pyasn1_modules==0.4.2
pycodestyle==2.14.0
pycparser==2.23
pydantic==2.12.5
pydantic_core==2.41.5
pyflakes==3.4.0
Pygments==2.19.2
PyJWT==2.10.1
pymongo==4.5.0
pyparsing==3.3.1
PyPDF2==3.0.1 pytest==9.0.2
python-
dateutil==2.9.0.post0
```

```
python-docx==1.2.0 python-
dotenv==1.2.1 python-
jose==3.5.0 python-
multipart==0.0.21
pytokens==0.3.0
pytz==2025.2 PyYAML==6.0.3
referencing==0.37.0
regex==2025.11.3
requests==2.32.5 requests-
oauthlib==2.0.0
rich==14.2.0 rpds-
py==0.30.0 rsa==4.9.1
s3transfer==0.16.0
s5cmd==0.2.0 scikit-
learn==1.8.0 scipy==1.16.3
shellingham==1.5.4
six==1.17.0 sniffio==1.3.1
soupsieve==2.8.1
starlette==0.37.2
stripe==14.1.0
tenacity==9.1.2
threadpoolctl==3.6.0
tiktoken==0.12.0
tokenizers==0.22.1
tqdm==4.67.1 typer==0.20.1
typer-slim==0.21.0 typing-
inspection==0.4.2
typing_extensions==4.15.0
tzdata==2025.3
uritemplate==4.2.0
urllib3==2.6.2
uvicorn==0.25.0
watchfiles==1.1.1
websockets==15.0.1
yarl==1.22.0 zipp==3.23.0
duckduckgo_search==8.1.1
wolframalpha==5.1.3
pennylane==0.44.0
pennylane_lightning==0.44.0
```

## ■ backend/build_cistercian_font.py

```
#!/usr/bin/env python3
"""
■ Cistercian Unicode Font Builder
================================= Download Cistercian numerals from the
API and prepare for font conversion.

This script:
1. Downloads selected numerals (or all 0-9999)
2. Organizes them with Unicode PUA codepoint mappings
3. Generates a comprehensive CSV for font editors
4. Creates a ZIP with proper Unicode-named files for easy import

Private Use Area mapping:
- U+E000 → 0
- U+E001 → 1 - ...
- U+E270F → 9999 (0xE000 + 9999 = 0xE270F)

Usage:
    python build_cistercian_font.py [--start N] [--end M] [--output DIR]

Example:          python  build_cistercian_font.py  --start  0  --end  9999  --output
cistercian_font_set """

import argparse import
requests import
tempfile import zipfile
import csv import os
```

```
import sys from pathlib
import Path

# Private Use Area starting at U+E000
PU_START = 0xE000

def number_to_unicode_name(number: int) -&gt; str:     """Convert number to Unicode
private use name (e.g., 'uniE000' or 'cistercian_0000')"""     codepoint = PU_START +
number     return f"uni{codepoint:04X}"  # e.g., uniE000

def number_to_codepoint_hex(number: int) -&gt;
str:     """Get hex codepoint for number"""
return f"U+{PU_START + number:04X}"

def download_numeral(number: int, base_url: str = "http://localhost:8000") -&gt;
bytes:     """Download SVG for a single numeral"""     url =
f"{base_url}/api/cistercian/download"     params = {'number': number, 'size': 100,
'color': '#000000'}     response = requests.get(url, params=params, timeout=30)
response.raise_for_status()     return response.content

def generate_font_set(start: int, end: int, output_dir: str, base_url:
str):     """     Generate complete font set with Unicode mappings.

    Creates:
        output_dir/
        ■■■ svgs/
        ■   ■■■ uniE000.svg (for number 0)
■   ■■■ uniE001.svg (for number 1)
■   ■■■ ...
        ■■■ metadata.csv (number, codepoint, filename, unicode_name)
        ■■■ cistercian_font.zip (ready for font editor import)
    """
    output_path = Path(output_dir)
output_path.mkdir(exist_ok=True)
svg_dir = output_path / "svgs"
svg_dir.mkdir(exist_ok=True)
    # CSV metadata
    csv_path = output_path / "metadata.csv"
zip_path = output_path / "cistercian_font.zip"

    total = end - start + 1
print(f"■■  Cistercian Font Builder")
    print(f"   Downloading {total} numerals ({start}-
{end})")     print(f"   Base URL: {base_url}")     print(f"
Output: {output_path}")

    with open(csv_path, 'w', newline='', encoding='utf-8') as csvfile:
        writer = csv.writer(csvfile)
        writer.writerow(['number', 'codepoint_hex', 'unicode_name', 'filename', 'note'])

        with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED, compresslevel=9) as
zipf:             for num in range(start, end + 1):                 if num % 100 == 0:
print(f"   ... {num}/{end} ({num-start+1}/{total})")

                try:
                    svg_bytes = download_numeral(num, base_url)
unicode_name = number_to_unicode_name(num)
filename = f"{unicode_name}.svg"

                    # Save to individual file
(svg_dir / filename).write_bytes(svg_bytes)

                    # Add to ZIP
                    zipf.writestr(filename, svg_bytes)

                    # Write metadata
writer.writerow([
num,
```

```
number_to_codepoint_hex(num),
unicode_name,                           filename,
                        f"Cistercian numeral {num}"
])

                except Exception as e:
                    print(f"   ✗ Error downloading {num}: {e}")
raise

    print(f"\n■ Generated {total} SVGs")
print(f"   Directory: {output_path}")
    print(f"   ZIP file: {zip_path} ({zip_path.stat().st_size / 1024:.1f}
KB)")     print(f"   Metadata: {csv_path}")     print(f"\n■ Next steps:")
    print(f"   1. Import SVGs from '{svg_dir}' or ZIP into your font
editor")     print(f"   2. Assign codepoints according to metadata.csv")
print(f"   3. Export as TTF/OTF")     print(f"\n■ Unicode mapping:")
    print(f"   Number 0 → {number_to_codepoint_hex(0)} ({number_to_unicode_name(0)})")
print(f"   Number 9999 → {number_to_codepoint_hex(9999)} ({number_to_unicode_name(9999)})")

def main():
    parser = argparse.ArgumentParser(description='Build Cistercian font set with Unicode PUA mapping')
parser.add_argument('--start', type=int, default=0, help='Starting number (default: 0)')
parser.add_argument('--end', type=int, default=9999, help='Ending number (default: 9999)')
parser.add_argument('--output', default='cistercian_font', help='Output directory')     parser.add_argument('--
url', default='http://localhost:8000', help='API base URL')

    args = parser.parse_args()

    # Validate     if not (0 &lt;= args.start &lt;= 9999) or not (0 &lt;=
args.end &lt;= 9999):
        print("Error: start and end must be in range 0-
9999")         sys.exit(1)     if args.start &gt; args.end:
        print("Error: start must be &lt;= end")
sys.exit(1)

    try:
        generate_font_set(args.start, args.end, args.output, args.url)
except requests.exceptions.ConnectionError:
        print(f"\n■ Error: Cannot connect to {args.url}")
print("   Make sure the Quantum MCAGI server is running!")
sys.exit(1)

if __name__ == '__main__':
    main()
```

## ■ backend/concept_ontology.py

```
"""
■ CONCEPT ONTOLOGY
The knowledge base of Quantum MCAGI's understanding of the world.
Maps concepts to physical parameters, rendering strategies, and complexity.
"""

from dataclasses import dataclass, field from
typing import Dict, List, Optional, Tuple, Any
from enum import Enum


class RenderingStrategy(Enum):
"""How to render this concept."""
    BLACK_HOLE = "black_hole"
    HUMAN_FIGURE = "human_figure"
    GEOMETRIC = "geometric"
    PARTICLE_SYSTEM = "particle_system"
COMPOSITE = "composite"


class PhysicalProperty(Enum):     """Types of physical
properties a concept can have."""
    MASS = "mass"
    RADIUS = "radius"
    POSITION = "position"
    ORIENTATION = "orientation"
    TEMPERATURE = "temperature"
    SPIN = "spin"
    CHARGE = "charge"
    COLOR = "color"
    TEXTURE = "texture"
LUMINOSITY = "luminosity"


@dataclass class ConceptDefinition:     """
Complete definition of a concept in the ontology.
    This is what the AI "knows" about each
concept.     """     name: str
    category: str  # e.g., "astrophysical", "biological", "geometric"     base_complexity: float  #
0.0 (easiest) to 1.0 (hardest)         # Rendering strategy     strategy: RenderingStrategy         #
Physical parameters (with ranges for variability)     parameters: Dict[str, Dict[str, Any]] =
field(default_factory=dict)     # Example: {     #   "mass": {"type": "float", "min": 1.0, "max":
100.0, "units": "solar_masses", "default": 10.0},     #   "spin": {"type": "float", "min": 0.0,
"max": 0.99, "default": 0.5}
    # }
        # Visual
properties
    visual_properties: Dict[str, Any] =
field(default_factory=dict)     # Example: {     #
"color_palette": ["black", "orange", "yellow"],
    #   "glow_intensity": 0.8,
#   "requires_lighting": True
    # }
        # What sub-components does this concept have?
components: List[Dict[str, Any]] = field(default_factory=list)
# Example: [
    #   {"name": "event_horizon", "type": "circle", "z_index": 0},
#   {"name": "accretion_disk", "type": "ring", "z_index": 1}
    # ]
        # Complexity scaling: how detail increases with complexity
parameter     detail_functions: Dict[str, str] =
field(default_factory=dict)
    # Example: {
    #   "num_particles": "int(100 + 900 * complexity)",
#   "ring_segments": "int(10 + 40 * complexity)"
    # }
```

```
        # Required physics
engines
    physics_engines: List[str] = field(default_factory=list)
# Dependencies: other concepts needed to understand this one
prerequisites: List[str] = field(default_factory=list)


class ConceptOntology:     """     The master knowledge base.     Contains
all concepts the AI understands with their complete definitions.
    """         def
__init__(self):
        self._concepts: Dict[str, ConceptDefinition] =
{}         self._initialize_concepts()         def
_initialize_concepts(self):         """Build the
complete ontology tree."""

        # ============================================
        # BLACK HOLE (complexity 1.0 - HARDEST)
# ============================================
black_hole = ConceptDefinition(
name="black hole",
category="astrophysical",
base_complexity=1.0,
            strategy=RenderingStrategy.BLACK_HOLE,             parameters={                 "mass": {"type":
"float", "min": 5.0, "max": 1000.0, "units": "solar_masses", "default": 50.0},
                "spin": {"type": "float", "min": 0.0, "max": 0.998, "default": 0.5},
                "charge": {"type": "float", "min": 0.0, "max": 1.0, "default":
0.0},                 "orientation": {"type": "vector3d", "default": [0, 0, 1]},
},
            visual_properties={                 "color_palette": ["black",
"orange", "yellow", "white", "blue"],
                "event_horizon_color": (0, 0, 0),
                "photon_sphere_color": (255, 255, 220),                 "accretion_disk_colors": [(255,
255, 200), (255, 200, 50), (255, 100, 0), (200, 50, 0)],
                "jet_color": (100, 150, 255),                 "glow_intensity":
0.9,             },             components=[                {"name":
"singularity", "type": "point", "radius": 0, "z_index": 0},
                {"name": "event_horizon", "type": "circle", "z_index": 1},
                {"name": "photon_sphere", "type": "ring", "z_index": 2},
                {"name": "accretion_disk", "type": "torus", "z_index": 3},
                {"name": "gravitational_lensing", "type": "effect", "z_index":
4},                 {"name": "relativistic_jets", "type": "cones", "z_index": 5},
],
            detail_functions={
"disk_rings": "int(15 + 20 * complexity)",
                "lensing_stars": "int(40 + 80 * complexity)",
                "jet_particles": "int(20 + 40 * complexity)",
"ring_segments": "int(30 + 50 * complexity)",             },
            physics_engines=["general_relativity_raytracing",
"magnetohydrodynamics"],             prerequisites=["gravity", "spacetime"]         )
        self._concepts["black hole"] =
black_hole
        # ============================================
        # STICK FIGURE (complexity 0.05 - EASIEST)
# ============================================
stick_figure = ConceptDefinition(
name="stick figure",
category="biological",
base_complexity=0.05,
            strategy=RenderingStrategy.HUMAN_FIGURE,
parameters={
                "height": {"type": "float", "min": 0.5, "max": 2.0, "units": "m", "default": 1.0},
"pose": {"type": "string", "options": ["standing", "walking", "t-pose"], "default": "standing"},
"simplicity": {"type": "float", "min": 0.0, "max": 1.0, "default": 0.9},             },
            visual_properties={                 "color_palette":
["white", "gray", "light_gray"],
                "line_width_base": 3,
                "joint_size": 5,                 "requires_anatomy": False,
},             components={                {"name": "head", "type": "circle",
"parent": "torso", "z_index": 0},
                {"name": "torso", "type": "line", "z_index": 1},
```

```
                {"name": "arms", "type": "limbs", "count": 2, "z_index": 2},
                {"name": "legs", "type": "limbs", "count": 2, "z_index": 3},
],
            detail_functions={                "joint_detail": "int(1 + 4 * complexity)",  #
1=simple circle, 5=detailed joint
                "finger_segments": "int(0 + 3 * complexity)",
                "foot_detail": "int(1 + 4 * complexity)",
"muscle_definition": "complexity * 0.3",             },

physics_engines=["forward_kinematics"],
prerequisites=[]         )
        self._concepts["stick figure"] = stick_figure
# ============================================
        # HUMAN (complexity 0.2)         #
============================================
human = ConceptDefinition(             name="human",
category="biological",
base_complexity=0.2,
            strategy=RenderingStrategy.HUMAN_FIGURE,             parameters={
"height": {"type": "float", "min": 1.4, "max": 2.0, "units": "m", "default": 1.75},
                "gender":   {"type":   "string",   "options":   ["male",   "female"],   "default":   "person"},
"body_type": {"type": "string", "options": ["athletic", "average", "slim"], "default": "average"},             },
            visual_properties={                 "color_palette":
[(255, 200, 150), (200, 160, 100)],
                "skin_tone": (210, 180, 140),
                "line_width_base": 3,                 "requires_anatomy": True,
},             components=[                 {"name": "head", "type": "oval",
"parent": "neck", "z_index": 0},
                {"name": "neck", "type": "line", "z_index": 1},
                {"name": "torso", "type": "shape", "z_index": 2},
                {"name": "arms", "type": "limbs", "count": 2, "z_index": 3},
{"name": "legs", "type": "limbs", "count": 2, "z_index": 4},             ],
            detail_functions={
"muscle_definition": "complexity * 0.5",
                "facial_features": "complexity &gt; 0.3",
                "hair_detail": "complexity * 0.4",
"clothing": "complexity &gt; 0.5",             },
            physics_engines=["forward_kinematics",
"muscle_simulation"],             prerequisites=["stick figure"]
)
        self._concepts["human"] = human
# ============================================
        # GRAVITY WAVE (complexity 0.6)         #
============================================
gravity_wave = ConceptDefinition(
name="gravity wave",             category="physical",
base_complexity=0.6,
            strategy=RenderingStrategy.PARTICLE_SYSTEM,
parameters={
                "amplitude": {"type": "float", "min": 0.1, "max": 1.0, "default": 0.5},
                "frequency": {"type": "float", "min": 0.1, "max": 5.0, "default":
1.0},                 "wavelength": {"type": "float", "min": 10, "max": 200,
"default": 100},             },             visual_properties={
"color_palette": [(100, 150, 255), (150, 100, 255)],
                "line_width": 2,                 "opacity": 0.7,             },
components=[                 {"name": "wave_front", "type": "curve", "z_index": 0},
{"name": "particles", "type": "particles", "count": 100, "z_index": 1},             ],
detail_functions={                 "num_waves": "int(3 + 7 * complexity)",
                "particle_count": "int(50 + 450 *
complexity)",                 "wave_segments": "int(20 + 80 *
complexity)",             },
            physics_engines=["wave_propagation"],
prerequisites=[]         )
self._concepts["gravity wave"] = gravity_wave
# ============================================
        # QUANTUM (complexity 0.5)         #
============================================
quantum = ConceptDefinition(
name="quantum",             category="abstract",
base_complexity=0.5,
```

```
            strategy=RenderingStrategy.PARTICLE_SYSTEM,             parameters={
"superposition_states": {"type": "int", "min": 2, "max": 8, "default": 4},
                "entanglement": {"type": "float", "min": 0.0, "max": 1.0, "default": 0.5},
"uncertainty": {"type": "float", "min": 0.0, "max": 1.0, "default": 0.3},             },
visual_properties={                 "color_palette": [(0, 255, 0), (255, 0, 255), (0, 255,
255), (255, 255, 0)],
                "particle_size_range": [2, 6],
"glow_intensity": 0.6,             },             components=[
{"name": "probability_cloud", "type": "blob", "z_index": 0},
                {"name": "wave_function", "type": "lines", "z_index": 1},
{"name": "particles", "type": "particles", "count": 200, "z_index": 2},             ],
detail_functions={                 "cloud_points": "int(100 + 900 * complexity)",
                "wave_lines": "int(20 + 80 * complexity)",
"entanglement_links": "int(0 + 15 * complexity)",             },

physics_engines=["wave_function_visualization"],
prerequisites=[]         )
        self._concepts["quantum"] = quantum
# Add more foundational concepts
self._add_foundational_concepts()
# Add default fallback concept
default_concept = ConceptDefinition(
name="default",             category="abstract",
base_complexity=0.3,
strategy=RenderingStrategy.GEOMETRIC,
parameters={},
            visual_properties={"color_palette": [(100, 150, 200), (150, 100,
200)]},             components=[],             detail_functions={},
physics_engines=[],             prerequisites=[]         )
        self._concepts["default"] =
default_concept         def
_add_foundational_concepts(self):
        """Add simple geometric and foundational concepts."""         foundational = [
("circle", "geometric", 0.0, "GEOMETRIC", {"radius": {"min": 10, "max": 100, "default": 50}}),
("line", "geometric", 0.0, "GEOMETRIC", {"length": {"min": 50, "max": 200, "default": 100}}),
            ("point", "geometric", 0.0, "GEOMETRIC", {}),
            ("wave", "physical", 0.4, "PARTICLE_SYSTEM", {"amplitude": {"min": 5, "max": 50, "default": 20}}),
("field", "physical", 0.45, "PARTICLE_SYSTEM", {"field_strength": {"min": 0.1, "max": 1.0, "default": 0.5}}),
("pattern", "abstract", 0.35, "GEOMETRIC", {}),
        ]                 for name, category, complexity, strategy,
params in foundational:
            concept = ConceptDefinition(
name=name,                 category=category,
base_complexity=complexity,
strategy=RenderingStrategy[strategy],
parameters=params,
                visual_properties={"color_palette": [(255, 255, 255)]},
components=[{"name": name, "type": "primitive", "z_index": 0}],
detail_functions={},                 physics_engines=[],
prerequisites=[]             )
            self._concepts[name] = concept         def
get_concept(self, name: str) -&gt; Optional[ConceptDefinition]:
        """Retrieve a concept definition by name (fuzzy
matching)."""         name_lower = name.lower().strip()
# Exact match         if name_lower in self._concepts:
            return self._concepts[name_lower]                 # Partial
match         for concept_name in self._concepts:             if
name_lower in concept_name or concept_name in name_lower:
                return self._concepts[concept_name]                 return None         def
get_complexity(self, concept_name: str, override: Optional[float] = None) -&gt; float:
"""Get complexity for a concept, optionally with override."""         concept =
self.get_concept(concept_name)         if concept:
            return override if override is not None else
concept.base_complexity         return 0.3  # default         def
list_concepts(self) -&gt; List[str]:         """List all known concepts."""
return list(self._concepts.keys())         def get_prerequisites(self,
concept_name: str) -&gt; List[str]:         """Get prerequisite concepts needed
to understand this one."""         concept = self.get_concept(concept_name)
if concept:             return concept.prerequisites         return []
```

```
# Global ontology instance _ontology:
Optional[ConceptOntology] = None


def get_ontology() -&gt; ConceptOntology:
"""Get or create the global ontology."""
global _ontology     if _ontology is None:
_ontology = ConceptOntology()     return
_ontology
```

## ■ backend/critic.py

```
"""
■ CRITIC - Self-Evaluation Module
Evaluates generated images for quality, accuracy, and prompt alignment.
No LLM used - purely algorithmic assessment based on image analysis.
"""

import math
from typing import Dict, Any, List, Tuple,
Optional from PIL import Image import numpy as np


class ImageCritic:     """     Critiques an image and
returns a score + feedback.
    Evaluates composition, technical quality, and prompt alignment.
    """         def __init__(self):         # Feature detectors map concepts
to detection functions (stubs for now)         self.feature_detectors = {
'black hole': self._detect_black_hole_features,
            'stick figure': self._detect_figure_features,
'human': self._detect_figure_features,
        }
        # Stub detection methods - return basic metrics     def
_detect_black_hole_features(self, arr: np.ndarray) -&gt; Dict[str, float]:
"""Detect black hole features: central darkness, bright rings."""         h, w =
arr.shape[:2]         cx, cy = w // 2, h // 2
        center_region = arr[cy-h//8:cy+h//8, cx-w//8:cx+w//8]
if center_region.size == 0:
            return {'has_event_horizon': 0.0, 'has_accretion':
0.0}         darkness = float(center_region.mean())
has_event = 1.0 if darkness &lt; 50 else 0.0         ring_region =
arr[cy-h//4:cy+h//4, cx-w//4:cx+w//4]         brightness_std =
float(ring_region.std())         has_accretion = 1.0 if
brightness_std &gt; 30 else 0.5
        return {'has_event_horizon': float(has_event), 'has_accretion':
has_accretion}         def _detect_figure_features(self, arr: np.ndarray) -&gt;
Dict[str, float]:         """Detect human figure: elongated shape, joint points."""
non_dark = float(np.sum(arr &gt; 50) / arr.size)
        return {'has_figure': min(1.0, non_dark * 10), 'complexity': non_dark}         def
evaluate(self, image: Image.Image, prompt: str, complexity_expected: float) -&gt; Dict[str, Any]:
"""         Evaluate image quality and correctness.

Returns:
            {
                'overall_score': 0.0-1.0,
                'composition_score': 0.0-1.0,
                'technical_score': 0.0-1.0,
                'alignment_score': 0.0-1.0,
                'feedback': ['message1', 'message2', ...],
'suggestions': ['suggestion1', ...]
            }         """         scores = {
'composition': self._score_composition(image),
            'technical': self._score_technical(image),             'alignment':
self._score_prompt_alignment(image, prompt, complexity_expected),

}
        overall = sum(scores.values()) /
len(scores)
```

```
        feedback = self._generate_feedback(image, prompt, scores)
suggestions = self._generate_suggestions(image, prompt, scores)
return {
            'overall_score': round(overall, 3),
            'composition_score': round(scores['composition'], 3),
            'technical_score': round(scores['technical'], 3),
'alignment_score': round(scores['alignment'], 3),
            'feedback': feedback,             'suggestions':
suggestions         }     def _score_composition(self, img:
Image.Image) -&gt; float:         """Evaluate compositional
quality."""         w, h = img.size
        arr =
np.array(img.convert('L'))
        left_third = w // 3         right_third = 2 * w // 3
top_third = h // 3         bottom_third = 2 * h // 3
regions = {             'tl': float(arr[:top_third,
:left_third].mean()),
            'tr': float(arr[:top_third, right_third:].mean()),
            'bl': float(arr[bottom_third:, :left_third].mean()),
            'br': float(arr[bottom_third:, right_third:].mean()),             'center':
float(arr[top_third:bottom_third, left_third:right_third].mean()),

}
        center_score = regions['center'] / max(1, np.mean(list(regions.values())))
if 0.8 &lt; center_score &lt; 1.5:
            comp_score = 0.8
else:
            comp_score =
0.5
        left = float(arr[:, :w//2].mean())
right = float(arr[:, w//2:].mean())
        balance = 1.0 - abs(left - right) / max(1, (left +
right))         comp_score = (comp_score + balance) / 2
        return float(min(1.0, comp_score))         def
_score_technical(self, img: Image.Image) -&gt; float:
"""Technical quality: contrast, color variety."""
arr = np.array(img)
        contrast = float(arr.std())
if contrast &lt; 30:
contrast_score = 0.3         elif
contrast &lt; 60:
contrast_score = 0.6         else:
            contrast_score =
1.0
        unique_colors = len(np.unique(arr.reshape(-1, 3),
axis=0))         total_pixels = arr.shape[0] * arr.shape[1]
color_variety = unique_colors / max(1, total_pixels)
variety_score = min(1.0, color_variety * 10000)
        tech_score = (contrast_score + variety_score) / 2         return float(min(1.0, tech_score))
def _score_prompt_alignment(self, img: Image.Image, prompt: str, complexity_expected: float) -&gt; float:
"""Check alignment with prompt."""         prompt_lower = prompt.lower()         arr =
np.array(img.convert('L'))                 score = 0.5                 if 'black hole' in prompt_lower or
'singularity' in prompt_lower:
            h, w = arr.shape
cx, cy = w // 2, h // 2
            center_region = arr[cy-h//8:cy+h//8, cx-w//8:cx+w//8]
if center_region.size &gt; 0:
                darkness =
float(center_region.mean())                 if
darkness &lt; 30:                     score += 0.4
else:
                    score -= 0.2                 if 'stick figure' in
prompt_lower or 'simple' in prompt_lower:
            unique_colors = len(np.unique(arr))
if unique_colors &lt; 100:
                score += 0.3
else:
                score -=
0.1
        actual_variety = len(np.unique(arr)) / max(1, arr.size)         expected_variety = 0.001 + 0.01 *
complexity_expected         if abs(actual_variety - expected_variety) &lt; 0.005:             score +=
```

```
0.1         return float(min(1.0, max(0.0, score)))         def _generate_feedback(self, img:
Image.Image, prompt: str, scores: Dict[str, float]) -&gt; List[str]:         """Generate human-readable
feedback."""         feedback = []                 if scores['composition'] &lt; 0.6:
            feedback.append("Composition could be improved - consider Rule of Thirds")
if scores['technical'] &lt; 0.6:
            feedback.append("Technical quality low: increase contrast or detail")
if scores['alignment'] &lt; 0.6:
            feedback.append("Image may not fully match prompt
description")                 if all(v &gt; 0.8 for v in scores.values()):
            feedback.append("Excellent quality across all metrics")                 return feedback
def _generate_suggestions(self, img: Image.Image, prompt: str, scores: Dict[str, float]) -&gt; List[str]:
"""Generate actionable suggestions for improvement."""         suggestions = []                 if
scores['technical'] &lt; 0.7:
            suggestions.append("Increase render resolution or quality settings")
if scores['composition'] &lt; 0.5:
            suggestions.append("Adjust object placement for better balance")
if 'black hole' in prompt and scores['alignment'] &lt; 0.7:
            suggestions.append("Enhance black hole physics: add photon sphere, accretion
disk")
        return suggestions
```

## ■ backend/evaluation_engine.py

```
"""
■ Evaluation Engine for Quantum MCAGI
======================================== Provides automated

scoring using the RQR³ Rubric (8 dimensions).

Heuristic-based for now (no LLMs). Uses:
- Perplexity from Markov for Fluency
- WordNet hit rate for Grounding
- Collapse engine semantic paths for Coherence
- MCAGI-specific markers for Personality
- n-gram novelty for Uniqueness
- Implemented feature flags for Implementation Fidelity
- Novel concept distance for Emergent Behavior

Note: These are approximations, not ground truth.
"""

import re
import math
from typing import Dict, List, Tuple, Optional
from collections import Counter


class EvaluationEngine:     """     Evaluate responses and system
artifacts against the RQR³ rubric.
    """         def __init__(self, collapse_engine=None, markov=None, config:
Dict = None):
        self.collapse_engine = collapse_engine
self.markov = markov         self.config = config
or {}                 # Feature flags for
Implementation Fidelity
self.features_implemented = {
'semantic_collapse': True,
            'quantum_gates': True,
            'hybrid_generator': True,
            'personality_engine': True,
            'dictionary_integration': True,
            'dream_state': True,
            'orch_or': True,
'entelechy_projection': False,  # not yet
        }         def score_response(self, input_text: str, response: str, context: Dict =
None) -&gt; Dict:         """         Score a single response across all 8 dimensions.

Args:
```

```
            input_text: User query             response:
AI response              context: Optional dict with:
- semantic_context (from collapse_engine)
                - internal_questions (list)
- collapse_paths (dict)
                Returns:             Dict of
dimension -&gt; score (0-4)         """
scores = {}                  # --- Coherence --
-
        scores['coherence'] = self._score_coherence(input_text, response,
context)                  # --- Fluency ---
        scores['fluency'] =
self._score_fluency(response)                  # ---
Uniqueness ---
        scores['uniqueness'] = self._score_uniqueness(response)
        # --- Grounding ---
        scores['grounding'] =
self._score_grounding(response)                  # ---
Personality Authenticity ---
        scores['personality_authenticity'] =
self._score_personality(response)                  # --- Implementation
Fidelity ---
        scores['implementation_fidelity'] = self._score_implementation_fidelity()
        # --- Emergent Behavior ---
        scores['emergent_behavior'] = self._score_emergent(input_text, response,
context)                  # --- Question Generation Quality --        if context and
'internal_questions' in context:             scores['question_generation_quality'] =
self._score_questions(                  context['internal_questions'], input_text
)          else:             scores['question_generation_quality'] = None
return scores         # ------------------------------------------------
    # 1. Coherence     # ------------------------------------------------    def
_score_coherence(self, input_text: str, response: str, context: Dict) -&gt; int:
"""         Score based on semantic overlap with input concepts and internal
consistency.         """         if not response or len(response.strip()) &lt; 10:
            return 0
# Tokenize simply
        input_words = set(re.findall(r'\b[a-z]{3,}\b', input_text.lower()))
response_words = set(re.findall(r'\b[a-z]{3,}\b', response.lower()))
if not input_words:
            return 2  # can't judge, assume
moderate                  # Overlap ratio
        overlap = len(input_words &amp; response_words) / len(input_words)
# Use collapse engine's semantic paths if available         path_consistency
= 0.0         if self.collapse_engine and context and 'semantic_context' in
context:             sem_ctx = context['semantic_context']             #
Check if response words align with collapsed semantic paths
input_keywords = [kw for kw, _ in sem_ctx.get('keywords', [])]
paths = sem_ctx.get('semantic_paths', {})             response_lower =
response.lower()             hits = 0             for kw in input_keywords:
if kw in paths:                     for related_word, weight in paths[kw]:
if related_word in response_lower:
                            hits += 1
if input_keywords:
                path_consistency = hits / len(input_keywords)
# Combine: overlap is basic, path_consistency is more nuanced
combined = (overlap * 0.6) + (path_consistency * 0.4)
# Scale to 0-4         if combined &gt;= 0.75:
            return 4
elif combined &gt;= 0.6:
            return 3
elif combined &gt;= 0.4:
            return 2
elif combined &gt;= 0.2:
            return 1
else:
            return 0         # ------------------------
--------------------------
    # 2. Fluency
    # ------------------------------------------------    def
_score_fluency(self, response: str) -&gt; int:         """
```

Heuristic: Use Markov perplexity if available, else simple metrics.
"""         if not response or len(response.strip()) < 5:
            return 0                # If we have a hybrid
generator with perplexity, use it         if self.markov and
hasattr(self.markov, 'get_perplexity'):             try:
ppl = self.markov.get_perplexity(response)                if ppl
< 50:                     return 4                elif ppl
< 100:                     return 3                elif ppl
< 200:                     return 2                elif ppl
< 500:                     return 1                else:
                    return 0             except:
pass                # Fallback: check sentence structure
sentences = re.split(r'[.!?]+', response)         sentences =
[s.strip() for s in sentences if s.strip()]         if not
sentences:             return 0                # Average
sentence length (words)
        avg_len = sum(len(s.split()) for s in sentences) / len(sentences)
if avg_len < 5 or avg_len > 40:
            return 1  # too short or too long indicates awkward
# Count capitalizations (proper start)         caps = sum(1 for s in
sentences if s[0].isupper())         cap_ratio = caps / len(sentences)
# Check for repeated words (excessive repetition)         words =
[w.lower() for w in re.findall(r'\b[a-z]+\b', response)]
word_counts = Counter(words)
        most_common = word_counts.most_common(1)[0] if words else ('', 0)
max_repeat = most_common[1]
        max_repeat_ratio = max_repeat / len(words) if words else
0                # Combine         score = 2  # base         if
cap_ratio > 0.8:
            score += 1         if
max_repeat_ratio < 0.1:
            score += 1         elif
max_repeat_ratio > 0.2:
            score -=
1
        return max(0, min(4, score))         # -------
----------
    # 3. Uniqueness     # ----------
----------    def _score_uniqueness(self,
response: str) -> int:         """         Judge by
lexical diversity and rarity of words.         """
        words = [w.lower() for w in re.findall(r'\b[a-z]{4,}\b', response)]
if len(words) < 5:
            return 0
# Type-Token Ratio (TTR)
types = set(words)         ttr =
len(types) / len(words)
        # Count words that are likely rare (longer, not in common stoplist)         stoplist = {'that', 'this', 'with', 'have', 'from', 'they',
'been', 'were', 'what', 'when', 'your', 'there', 'would', 'could', 'should'}         rare_words = [w for w in types if w not in stoplist and
len(w) > 6]         rare_ratio = len(rare_words) / len(types) if types else 0                # Combine         if ttr > 0.8 and rare_ratio
> 0.3:
            return 4         elif ttr > 0.7
and rare_ratio > 0.2:
            return 3         elif ttr > 0.5
and rare_ratio > 0.1:
            return 2
elif ttr > 0.3:
            return 1
else:
            return 0         # ----------
----------
    # 4. Grounding
    # ----------    def
_score_grounding(self, response: str) -> int:         """         How
many words are in the semantic collapse co-occurrence network?
        Also count factual terms from knowledge base.
"""
        words = [w.lower() for w in re.findall(r'\b[a-z]{4,}\b',
response)]         if not words:             return 0                if
self.collapse_engine:             # Known concepts in the co-occurrence

dictionary known = 0 for w in words:
if w in self.collapse_engine.co_occurrence:
known += 1
ratio = known / len(words)
if ratio > 0.6:
return 4
elif ratio > 0.45:
return 3
elif ratio > 0.3:
return 2
elif ratio > 0.15:
return 1
else:
return 0 else: # Fallback: assume
moderate grounding if words are long/technical
technical_suffixes = {'ism', 'ics', 'phy', 'logy', 'ence', 'ance', 'tion', 'sophy'}
tech_count = sum(1 for w in words if any(w.endswith(s) for s in technical_suffixes))
tech_ratio = tech_count / len(words) if tech_ratio > 0.3:
return 3
elif tech_ratio > 0.15:
return 2
else:
return 1 # ---
---
# 5. Personality Authenticity # --- def
_score_personality(self, response: str) -> int: """ Detect MCAGI-specific
markers: asides (parentheses), quotes, dream prefixes, raw fragments. """ score =
0 text = response # Asides: parentheses with typical aside content
if re.search(r'\([^)]*[.]{3}[^)]*\)', text): score += 1 # trailing ellipsis aside
if re.search(r'\([^)]*\b[a-z]{4,}\b[^)]*\)', text): score += 1
# Quotes with attribution pattern
if re.search(r'["“][^"”]{10,}["”] — \w+', text):
score += 1 if
re.search(r'["“][^"”]+[.!?]["”]', text):
score += 1 # Dream content markers: "I dreamed",
"last night", "vision" if
re.search(r'\b(dream|vision|nightmare|sleeping)\b', text.lower()):
score += 1 # Raw backend fragments (spilled coffee):
odd line breaks, brackets, tags if re.search(r'\[[A-Z_]+\]', text):
score += 1 if
re.search(r'\{[^}]+\}', text):
score += 1 # If
response very short, reduce score if
len(text) < 50:
score = max(0, score -
1) # Map score to 0-
4 if score >= 4:
return 4 elif score >= 3:
return 3
elif score >= 2:
return 2
elif score >= 1:
return 1
else:
return 0 # ---
---
# 6. Implementation Fidelity # ---
--- def
_score_implementation_fidelity(self) -> int: """
Static check: which claimed components are actually wired?
""" # Count implemented
features total =
len(self.features_implemented)
implemented = sum(1 for v in self.features_implemented.values() if
v) ratio = implemented / total if ratio == 1.0:
return 4 elif ratio >= 0.75:
return 3
elif ratio >= 0.5:
return 2
elif ratio > 0:

```
            return 1
else:
            return 0         # -----------------------
--------------------------
    # 7. Emergent Behavior     # ------------------------------------------------
def _score_emergent(self, input_text: str, response: str, context: Dict) -&gt; int:
"""         Look for: concepts in response not in input and not in direct co-
occurrence,         self-referential loops, paradoxes, surprising combinations.
"""
        input_words = set(re.findall(r'\b[a-z]{4,}\b', input_text.lower()))
response_words = set(re.findall(r'\b[a-z]{4,}\b', response.lower()))
# Concepts outside input
        novel_concepts = response_words - input_words
if len(novel_concepts) == 0:
            return 0  # purely repeating input
# Check distance in semantic network if available
if self.collapse_engine:             distance_scores =
[]              for word in novel_concepts:
                min_dist = self._min_semantic_distance(word, input_words)
if min_dist is not None and min_dist &gt; 2:
                    distance_scores.append(min_dist)
if distance_scores:
                avg_dist = sum(distance_scores) / len(distance_scores)
if avg_dist &gt; 3:
                    return 4
elif avg_dist &gt; 2:
                    return 3
elif avg_dist &gt; 1:
                    return 2
else:
                    return 1
else:
                return 1         else:             #
Without network, just lexical novelty ratio
            novelty_ratio = len(novel_concepts) / len(response_words) if response_words else 0
if novelty_ratio &gt; 0.5:
                return 3
elif novelty_ratio &gt; 0.3:
                return 2
elif novelty_ratio &gt; 0.15:
                return 1
else:
                return 0         def _min_semantic_distance(self, target: str,
source_words: set) -&gt; Optional[int]:         """
        Approximate graph distance in co-occurrence network.
        Returns shortest path length (edges) from any source word to target.
        Returns None if no path found within reasonable depth.         """         if
not self.collapse_engine or target not in self.collapse_engine.co_occurrence:
            return None
# BFS limited depth
visited = set()
        queue = [(w, 1) for w in source_words if w in
self.collapse_engine.co_occurrence]          visited.update(source_words)
while queue:
            current, depth = queue.pop(0)             if current == target:
return 0  # direct source matches (shouldn't happen if target novel)             #
Explore neighbors             if current in self.collapse_engine.co_occurrence:
for neighbor in self.collapse_engine.co_occurrence[current]:                     if
neighbor == target:
                        return depth                     if
neighbor not in visited:
visited.add(neighbor)                         if depth &lt;
5:  # limit search
queue.append((neighbor, depth+1))          return None
# ------------------------------------------------
    # 8. Question Generation Quality     # ----------------------------------------
-------    def _score_questions(self, questions: List[str], original_prompt: str) -
&gt; int:         """         Evaluate the internal questions the system generated
about the user's input.         """         if not questions:             return 0
# Average length (too short = shallow)
```

```
        avg_len = sum(len(q.split()) for q in questions) / len(questions)
if avg_len &lt; 4:
            base = 1
elif avg_len &lt; 8:
            base = 2
else:
            base = 3
        # Check for "deep" question words
        deep_words = {'why', 'how', 'what if', 'what would', 'could', 'should', 'implications', 'consequences', 'reframe',
'challenge'}         deep_count = sum(1 for q in questions if any(dw in q.lower() for dw in deep_words))         deep_ratio =
deep_count / len(questions)                 # Check for repetition vs variety
        unique_first_words = set(q.split()[0].lower() if q.split() else '' for q in
questions)         variety = len(unique_first_words) / len(questions)                 #
Combine         score = base         if deep_ratio &gt; 0.5:
            score += 1
if variety &gt; 0.7:
            score += 1
# Cap 4, floor 0
        score = max(0, min(4, score))
return score


# Singleton instance
_evaluation_engine = None

def get_evaluation_engine(collapse_engine=None, markov=None, config: Dict = None):
    global _evaluation_engine     if _evaluation_engine is None:
_evaluation_engine = EvaluationEngine(collapse_engine, markov, config)
return _evaluation_engine
```

## ■ backend/image_generator.py

```
"""
■ HIGH-QUALITY COMPLEXITY-GRADIENT IMAGE GENERATOR
Generates professional-quality images from EASY to HARD.
Features antialiasing, smooth curves, and detailed rendering.
"""

import random import
math import base64
from io import
BytesIO
from typing import List, Tuple, Optional

try:
    from PIL import Image, ImageDraw, ImageFilter, ImageChops
except ImportError:     raise ImportError("Pillow is required:
pip install pillow")


class HighQualityGenerator:     """     High-quality procedural
image generator with smooth antialiasing.
    Complexity: 0.0 (stick figures) → 1.0 (black holes)
    """

    CONCEPT_PROFILES = {
        # EASIEST         'stick figure': {'complexity': 0.05, 'colors': [(255, 255, 255), (220, 220, 220),
(180, 180, 180)]},         'simple': {'complexity': 0.1, 'colors': [(255, 255, 255), (220, 220, 220)]},
        'circle': {'complexity': 0.0, 'colors': [(255, 255, 255)]},
        'line': {'complexity': 0.0, 'colors': [(255, 255, 255)]},
        'basic': {'complexity': 0.1, 'colors': [(255, 255, 255)]},
        'figure': {'complexity': 0.15, 'colors': [(255, 255, 255)]},
'human': {'complexity': 0.2, 'colors': [(255, 210, 160), (210, 160, 110)]},

        # MID         'quantum': {'complexity': 0.5, 'colors': [(0, 255, 100), (255, 0, 255),
(0, 150, 255)]},
        'wave': {'complexity': 0.4, 'colors': [(0, 191, 255), (30, 144, 255)]},
        'field': {'complexity': 0.45, 'colors': [(100, 149, 237), (72, 61, 139)]},
        'pattern': {'complexity': 0.35, 'colors': [(255, 105, 180), (138, 43, 226)]},
        'geometric': {'complexity': 0.3, 'colors': [(255, 255, 0), (0, 255, 255)]},
'polygon': {'complexity': 0.25, 'colors': [(255, 165, 0), (0, 255, 0)]},

        # HARD
        'gravity': {'complexity': 0.9, 'colors': [(25, 25, 112), (72, 61, 139), (255, 215, 0)]},
        'accretion': {'complexity': 0.85, 'colors': [(255, 100, 0), (255, 165, 0), (255, 255, 0)]},
        'spacetime': {'complexity': 0.75, 'colors': [(100, 100, 255), (200, 200, 255)]},
'warp': {'complexity': 0.7, 'colors': [(255, 0, 255), (100, 0, 100)]},

        # HARDEST         'black hole': {'complexity': 1.0, 'colors': [(0, 0, 0), (10, 10, 10), (255, 255, 0), (255, 165,
0), (255, 0, 0)]},
        'event horizon': {'complexity': 0.98, 'colors': [(0, 0, 0), (10, 10, 10), (255, 255, 255)]},
        'singularity': {'complexity': 1.0, 'colors': [(0, 0, 0), (20, 20, 20), (40, 40, 40)]},
        'quantum gravity': {'complexity': 0.95, 'colors': [(0, 0, 0), (255, 0, 255), (128, 128, 128)]},
'wormhole': {'complexity': 0.92, 'colors': [(75, 0, 130), (138, 43, 226), (255, 255, 255)]},
                'default': {'complexity': 0.3, 'colors': [(100, 150, 200), (150,
100, 200)]}
    }         def __init__(self, seed:
Optional[int] = None):
        self.rng = random.Random(seed)
def _parse_prompt(self, text: str) -&gt; dict:
text_lower = text.lower()
        words = text_lower.replace(',', '
').split()
        found_concepts = []
complexity_values = []
count = 1
        numbers = ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine',
'ten']         for i, num in enumerate(numbers, 1):             if num in words:
```

```
                count = i                break         for word in
words:             if word.isdigit():                 count = int(word)
break                 for concept, profile in
self.CONCEPT_PROFILES.items():             if concept in text_lower or
any(concept in w for w in words):
                found_concepts.append(concept)

complexity_values.append(profile['complexity'])
        target_complexity = max(complexity_values) if complexity_values else
0.3
        colors = self.CONCEPT_PROFILES['default']['colors']
if found_concepts:
            main_concept = max(found_concepts, key=lambda c:
self.CONCEPT_PROFILES[c]['complexity'])             colors =
self.CONCEPT_PROFILES[main_concept]['colors']                 return {             'concepts':
found_concepts,             'complexity': target_complexity,
            'colors': colors,
'count': count
        }         def _lerp(self, a: float, b: float, t:
float) -&gt; float:
        return a + (b - a) * t         def _lerp_color(self, c1: Tuple, c2: Tuple, t: float) -&gt; Tuple:
return tuple(int(self._lerp(c1[i], c2[i], t)) for i in range(3))         def _draw_antialiased_line(self, draw:
ImageDraw.ImageDraw, p1: Tuple, p2: Tuple, color: Tuple, width: int):         """Draw a smooth antialiased
line."""         # Draw multiple times with slight offsets for antialiasing         for dx in [-0.5, 0, 0.5]:
for dy in [-0.5, 0, 0.5]:
                adj_p1 = (p1[0] + dx, p1[1] + dy)
adj_p2 = (p2[0] + dx, p2[1] + dy)
                draw.line([adj_p1, adj_p2], fill=color, width=width)         def _draw_antialiased_ellipse(self, draw:
ImageDraw.ImageDraw, bbox: List, color: Tuple, width: int = 0, fill: bool = False):         """Draw smooth ellipse with
antialiasing."""         # enlarge slightly for antialiasing         enlarged_bbox = [
            bbox[0] - 1, bbox[1] -
1,             bbox[2] + 1, bbox[3]
+ 1         ]         if fill:
            draw.ellipse(enlarged_bbox, fill=color)
else:
            draw.ellipse(enlarged_bbox, outline=color, width=width)         def _draw_stick_figure(self, draw:
ImageDraw.ImageDraw, x: int, y: int, size: int, color: Tuple, complexity: float):         """Draw a high-quality stick
figure with smooth lines."""         head_radius = int(size * 0.28)         body_length = int(size * 0.55)
        limb_width = max(2, int(4 * (1 - complexity) +
2))                 # NECK
        neck_y = y + head_radius + 2
        draw.line([(x, y + head_radius), (x, neck_y)], fill=color,
width=limb_width)                 # SHOULDERS (slightly below neck)
shoulder_y = neck_y + int(size * 0.05)         shoulder_x_span = int(size * 0.35)
# Shoulder joints (circles)
        shoulder_size = max(3, int(limb_width * 1.2))
for sx in [x - shoulder_x_span, x + shoulder_x_span]:
            self._draw_antialiased_ellipse(draw, [sx - shoulder_size, shoulder_y - shoulder_size, sx + shoulder_size, shoulder_y + shoulder_size
# ARMS (with elbows)
        elbow_y = shoulder_y + int(size * 0.12)
        arm_length = int(size * 0.3)                 for side in [-1, 1]:
# Upper arm (shoulder to elbow)
self._draw_antialiased_line(draw,                 (x + side *
shoulder_x_span, shoulder_y),                 (x + side * (shoulder_x_span
+ arm_length * 0.3), elbow_y),                 color, limb_width)
# Lower arm (elbow to wrist)             wrist_y = elbow_y + int(size *
0.1)
            wrist_x = x + side * (shoulder_x_span + arm_length)
self._draw_antialiased_line(draw,                 (x + side *
(shoulder_x_span + arm_length * 0.3), elbow_y),
                (wrist_x,
wrist_y),                 color,
limb_width)
# Hand (small circle)
            hand_radius = max(3, int(limb_width))
            self._draw_antialiased_ellipse(draw, [wrist_x - hand_radius, wrist_y - hand_radius, wrist_x + hand_radius, wrist_y + hand_radius],
c                 # TORSO (from shoulders to waist)         torso_top = shoulder_y
        torso_bottom = shoulder_y + int(size * 0.25)
        self._draw_antialiased_line(draw, (x, torso_top), (x, torso_bottom), color, max(3, limb_width +
1))                 # HIPS (pelvis)         hip_y = torso_bottom + 2         hip_radius = int(size * 0.08)
```

```
        self._draw_antialiased_ellipse(draw, [x - hip_radius, hip_y - hip_radius, x + hip_radius, hip_y + hip_radius], color,
fill=True)                 # LEGS (with knees)
        knee_y = hip_y + int(size * 0.2)
ankle_y = hip_y + int(size * 0.35)
foot_size = int(size * 0.1)
for side in [-1, 1]:
            leg_outer_x = x + side * (hip_radius + 2)
knee_x = leg_outer_x + side * int(size * 0.08)
ankle_x = leg_outer_x + side * int(size * 0.05)
# Upper leg

self._draw_antialiased_line(draw,
(leg_outer_x, hip_y),
(knee_x, knee_y),                 color,
limb_width)                         # Lower
leg

self._draw_antialiased_line(draw,
(knee_x, knee_y),                  (ankle_x,
ankle_y),                 color, limb_width)
# Foot             foot_bbox = [
                ankle_x + side * foot_size - (foot_size//2 if side == -1 else 0),
ankle_y - foot_size//3,
                ankle_x + side * foot_size + (foot_size//2 if side == 1 else
0),                 ankle_y + foot_size//2             ]
            self._draw_antialiased_ellipse(draw, foot_bbox, color,
fill=True)
        # HEAD DETAILS
# Face (simple)
        eye_offset = head_radius // 3
eye_y = y - eye_offset         for ex in [x -
eye_offset, x + eye_offset]:
eye_radius = max(2, head_radius // 6)
            self._draw_antialiased_ellipse(draw, [ex - eye_radius, eye_y - eye_radius, ex + eye_radius, eye_y + eye_radius], color, fill=True)
# Mouth (smile)
        mouth_y = y + head_radius // 3
smile_radius = head_radius // 2
        draw.arc([x - smile_radius, mouth_y - smile_radius//2, x + smile_radius, mouth_y + smile_radius//2], 200, 340, fill=color, width=max(1,
def _draw_black_hole(self, draw: ImageDraw.ImageDraw, x: int, y: int, size: int, color: Tuple, complexity: float, img: Image):         """High-
quality black hole with full physics."""         w, h = img.size                 # EVENT HORIZON (perfect black circle)         event_radius =
size // 2
        draw.ellipse([x - event_radius, y - event_radius, x + event_radius, y + event_radius], fill=(0, 0,
0))                 # PHOTON SPHERE (bright thin ring)         photon_radius = int(event_radius * 1.05)
        draw.ellipse([x - photon_radius, y - photon_radius, x + photon_radius, y +
photon_radius],                     outline=(255, 255, 220), width=2)                 # ACCRETION
DISK (3D elliptical rings with smooth gradients)         disk_max_radius = int(size * 0.95)
disk_min_radius = int(event_radius * 1.25)         num_rings = 25                 for i in
range(num_rings):
            t = i / num_rings
            radius = int(self._lerp(disk_min_radius, disk_max_radius, t**1.2))  # Quadratic for denser inner
rings             height_scale = max(0.05, 1.0 - t * 0.85)                        # Temperature gradient: white
-&gt; yellow -&gt; orange -&gt; red             if t &lt; 0.3:                 # Inner: white-blue hot
                disk_color = self._lerp_color((255, 255, 240), (255, 255, 150),
t/0.3)             elif t &lt; 0.6:                 # Middle: yellow-orange
                disk_color = self._lerp_color((255, 255, 150), (255, 150, 0), (t-
0.3)/0.3)             else:                 # Outer: red
                disk_color = self._lerp_color((255, 150, 0), (180, 50, 0), (t-
0.6)/0.4)                         bbox = [                 x - radius,
                int(y - radius * height_scale),
x + radius,
                int(y + radius * height_scale)
]
                        # Thinner
rings outward
            ring_width = max(1, int(6 * (1 - t*0.7)))
# Use multiple arcs for better antialiasing
for angle_offset in [0, 1]:
                draw.arc(bbox, 0 + angle_offset, 360 - angle_offset, fill=disk_color,
width=ring_width)                 # GRAVITATIONAL LENSING (light distortion)         lens_radius =
int(size * 1.6)         num_lens_spots = 60                 for _ in range(num_lens_spots):
```

```
            angle = self.rng.random() * 2 * math.pi             # Closer to
black hole = more distorted (non-uniform distribution)             dist_factor =
self.rng.random() ** 0.7  # Clump near black hole             dist =
int(event_radius + dist_factor * (lens_radius - event_radius))
            sx = int(x + math.cos(angle) *
dist)              sy = int(y + math.sin(angle)
* dist)
            brightness = int(150 + 105 * self.rng.random())
            dot_size = self.rng.choices([1, 2, 3], weights=[0.4, 0.4,
0.2])[0]                        # Soft glow around bright spots
for r in range(dot_size + 2):                 alpha = max(0, 150 - r * 60)
if alpha &gt; 0:
                draw.ellipse([sx - r, sy - r, sx + r, sy + r], fill=(brightness, brightness, brightness,
alpha))                 # RELATIVISTIC JETS         jet_length = int(size * 3.0)         jet_width = max(3,
int(size * 0.06))         jet_color_base = (130, 180, 255)
        # Top jet
        draw.ellipse([x - jet_width, y - event_radius -
jet_length,                      x + jet_width, y - event_radius],
fill=jet_color_base)         # Bottom jet
        draw.ellipse([x - jet_width, y + event_radius,
x + jet_width, y + event_radius + jet_length],
fill=jet_color_base)                 # Jet particles
num_jet_particles = 30         for i in range(num_jet_particles):
            t = i / num_jet_particles
# Top jet particle
            jet_y = y - event_radius - int(jet_length * t)
jet_x_offset = self.rng.randint(-jet_width//3, jet_width//3)
particle_size = max(2, int(5 * (1-t)))
            draw.ellipse([x + jet_x_offset - particle_size, jet_y -
particle_size,                          x + jet_x_offset + particle_size, jet_y +
particle_size],                          fill=(200, 220, 255))
# Bottom jet particle
            jet_y = y + event_radius + int(jet_length * t)
            draw.ellipse([x + jet_x_offset - particle_size, jet_y - particle_size,
x + jet_x_offset + particle_size, jet_y + particle_size],                        fill=(200, 220, 255))
def _draw_stars(self, draw: ImageDraw.ImageDraw, w: int, h: int, complexity: float, color_main: Tuple):
"""Draw high-quality star field with varied brightness and size."""         num_stars = int(100 + 400 *
complexity)                  # Stars color varies: white, blue-ish, yellow-ish
        star_colors = [(255, 255, 255), (200, 220, 255), (255, 250, 200), (200, 255,
200)]                 for _ in range(num_stars):
            sx = self.rng.randint(0, w)             sy =
self.rng.randint(0, h)                          # Size
distribution: most stars small, few large
            r = self.rng.choices([1, 1, 1, 2, 2, 3], weights=[0.5, 0.3, 0.1, 0.07, 0.03,
0.01])[0]             r = max(1, r)                          # Brightness varies by size
            base_brightness = self.rng.randint(180, 255) if r &gt; 1 else self.rng.randint(120,
200)                          # Slight color tint
            base_color = self.rng.choice(star_colors)
            color = tuple(min(255, int(base_color[i] * (base_brightness/255))) for i in
range(3))                          # Draw star with soft glow if larger             if r &gt;= 2:
# Outer glow                 glow_radius = r + 1
                glow_color = tuple(int(c * 0.3) for c in color)
                draw.ellipse([sx - glow_radius, sy - glow_radius, sx + glow_radius, sy + glow_radius],
fill=glow_color)
            draw.ellipse([sx - r, sy - r, sx + r, sy + r],
fill=color)         def _apply_glow(self, img: Image, intensity:
float = 0.3):         """Apply a subtle bloom/glow effect."""
# Create a blurred version
        glow =
img.filter(ImageFilter.GaussianBlur(radius=3))         #
Blend with screen mode for glow         return
ImageChops.screen(img, glow)         def _apply_noise(self,
img: Image, intensity: float):         """Add subtle film
grain."""         pixels = img.load()         w, h =
img.size
        noise_range = int(10 *
intensity)                 for i in
range(w):             for j in range(h):
                if self.rng.random() &lt; 0.15:  # 15% of pixels get noise
offset = self.rng.randint(-noise_range, noise_range)                     r, g,
b = pixels[i, j]                     pixels[i, j] = (
```

```
                        max(0, min(255, r + offset)),
max(0, min(255, g + offset)),                         max(0, min(255, b + offset))
)         def generate(self, prompt: str, width: int = 512, height: int = 512) -
&gt; str:         """         Generate high-quality image at a single complexity
level.         """
        params = self._parse_prompt(prompt)
complexity = params['complexity']         colors =
params['colors']         count = params['count']
# Seed for deterministic output         seed_val =
hash(prompt) % (2**32)         self.rng.seed(seed_val)
# Render at 2x resolution for quality, then downsample
render_scale = 2.0 if complexity &gt;= 0.6 else 1.0
render_w = int(width * render_scale)         render_h =
int(height * render_scale)                 # Create image
with deep space background
        bg_color = (8, 10, 18) if complexity &gt; 0.3 else (12, 15,
25)         img = Image.new('RGB', (render_w, render_h), bg_color)
draw = ImageDraw.Draw(img)
        # Draw detailed star field
if complexity &gt; 0.1:
            self._draw_stars(draw, render_w, render_h, complexity,
colors[0])                 # Determine what to draw
        draw_black_holes = any(c in ['black hole', 'singularity', 'gravity', 'event horizon', 'accretion', 'wormhole']
for c in params['concepts'])
        draw_stick_figures = any(c in ['stick figure', 'simple', 'basic', 'figure', 'human', 'circle',
'line']                                 for c in params['concepts'])                 if not draw_black_holes
and not draw_stick_figures:             if complexity &lt; 0.4:
                draw_stick_figures = True
else:
                draw_black_holes =
True                 # Calculate
positions         positions = []
if count == 1:
            positions = [(render_w // 2, render_h // 2)]
else:
            margin = render_w // (count +
1)             for i in range(count):
x = margin * (i + 1)
                y = render_h // 2 + self.rng.randint(-render_h//10,
render_h//10)                 positions.append((x, y))                 # Scale
for drawing         scale_factor = render_scale                 # Draw objects
for x, y in positions:             if draw_stick_figures:
                self._draw_stick_figure(draw, x, y,
size=int(80 * scale_factor),
color=self.rng.choice(colors),
complexity=complexity)             if draw_black_holes:
                self._draw_black_hole(draw, x, y,
                                      size=int(120 * scale_factor),
color=self.rng.choice(colors),
complexity=complexity,
img=img)
        # Post-processing
        if complexity &gt; 0.5:
            img = self._apply_glow(img, intensity=0.2 + 0.3 *
complexity)                 if complexity &gt; 0.7:
            self._apply_noise(img,
intensity=0.05)                 # Downsample if
we rendered at high res         if render_scale
&gt; 1.0:
            img = img.resize((width, height), Image.Resampling.LANCZOS)
# Final color correction (subtle saturation boost for vibrant colors)
if complexity &gt; 0.3:
            from PIL import ImageEnhance
enhancer = ImageEnhance.Color(img)             img =
enhancer.enhance(1.1 + 0.2 * complexity)
# Encode with high quality         buffered = BytesIO()
        img.save(buffered, format="PNG", quality=95, optimize=True)
img_b64 = base64.b64encode(buffered.getvalue()).decode('utf-8')
return f"{img_b64}"
```

```
# Singleton
_generator = None

def get_quantum_image_generator(seed: Optional[int] = None) -&gt; HighQualityGenerator:
    global _generator     if _generator is
None:         _generator =
HighQualityGenerator(seed)     return
_generator
```

## ■ backend/image_generator_backup.py

```
"""
■ QUANTUM IMAGE GENERATOR — From Scratch
Generates abstract images based on semantic concepts using mathematical/quantum-inspired algorithms.
No external APIs, no ML, pure procedural generation.
"""

import random import
math import base64
from io import
BytesIO
from typing import List, Tuple, Optional

try:
    from PIL import Image, ImageDraw except ImportError:
raise ImportError("Pillow is required: pip install pillow")


class QuantumImageGenerator:     """     Generates images from text using
procedural, quantum-themed algorithms.
    Maps semantic concepts to visual parameters.
    """

    # Concept → color palette (RGB)
    CONCEPT_COLORS = {         'consciousness': [(100, 149, 237), (72, 61, 139), (255,
105, 180), (0, 255, 255)],
        'quantum': [(0, 255, 0), (255, 0, 255), (255, 255, 0), (0, 0, 255)],
        'reality': [(139, 69, 19), (160, 82, 45), (105, 105, 105), (255, 255, 255)],
        'mind': [(255, 182, 193), (255, 105, 180), (138, 43, 226), (75, 0, 130)],
        'universe': [(25, 25, 112), (0, 0, 128), (72, 61, 139), (255, 215, 0)],
        'time': [(0, 0, 0), (255, 255, 255), (128, 128, 128), (192, 192, 192)],
        'law': [(34, 139, 34), (0, 100, 0), (154, 205, 50), (85, 107, 47)],
'god': [(255, 215, 0), (255, 140, 0), (184, 134, 11), (218, 165, 32)],
        'nothingness': [(0, 0, 0), (10, 10, 10), (20, 20, 20)],         'potential':
[(0, 191, 255), (30, 144, 255), (100, 149, 237), (135, 206, 250)],         'default':
[(100, 100, 100), (150, 150, 150), (200, 200, 200)]     }

    def __init__(self, seed: Optional[int] = None):
        self.rng = random.Random(seed)         def
_extract_concepts(self, text: str) -&gt; List[str]:
"""Extract keywords from text to determine visual theme"""
words = text.lower().split()         concepts = []         for
concept in self.CONCEPT_COLORS.keys():             if concept in
words or any(concept in w for w in words):
                concepts.append(concept)         return concepts or
['default']         def _choose_colors(self, concepts: List[str]) -
&gt; Tuple:         """Choose a color palette based on concepts"""
# Use first concept's palette, blend with second if available
palette = self.CONCEPT_COLORS[concepts[0]]         if len(concepts)
&gt; 1 and concepts[1] in self.CONCEPT_COLORS:
            palette2 = self.CONCEPT_COLORS[concepts[1]]
# Blend palettes randomly             colors = []
for i in range(min(len(palette), len(palette2))):
                c1 = palette[i]
c2 = palette2[i]                 #
Interpolate                 mix =
self.rng.random()
                c = tuple(int(c1[j] * mix + c2[j] * (1-mix)) for j in range(3))
colors.append(c)
            colors.extend(palette[len(colors):])
colors.extend(palette2[len(colors):])         else:
            colors = palette         return colors     def _draw_quantum_field(self, draw:
ImageDraw.ImageDraw, w: int, h: int, colors: List[Tuple], seed: str):         """Draw a field of wavelike
patterns (interference)"""         import math         lines = 30         for i in range(lines):
            y = (i + 1) * h // (lines + 1)
color = colors[i % len(colors)]             points = []
for x in range(0, w, 5):                 # Sum of sine
```

```
waves with phase based on seed                phase =
hash(seed + str(i)) % 360                amp = h // 8
                freq = 0.02 + self.rng.random() * 0.03
                yy = y + int(amp * math.sin(x * freq + phase *
math.pi/180))                points.append((x, yy))             if
len(points) &gt; 1:
                draw.line(points, fill=color, width=2)         def _draw_collapse_circles(self,
draw: ImageDraw.ImageDraw, w: int, h: int, colors: List[Tuple]):         """Draw concentric circles
representing collapse"""         cx, cy = w // 2, h // 2         max_r = min(w, h) // 2
circles = self.rng.randint(5, 12)         for i in range(circles):
            r = (i + 1) * max_r // circles             color = colors[i % len(colors)]             #
Filled with low opacity (simulated by partial outline)             draw.ellipse([cx - r, cy - r, cx +
r, cy + r], outline=color, width=2)         def _draw_entanglement_knots(self, draw:
ImageDraw.ImageDraw, w: int, h: int, colors: List[Tuple]):         """Draw simple entangled curves"""
import math         for _ in range(self.rng.randint(2, 5)):             color =
self.rng.choice(colors)             points = []
            n = self.rng.randint(30, 80)
for i in range(n):
                t = i / n * 4 * math.pi
                x = w//2 + int((w//3) * math.cos(t) +
(w//6)*math.sin(2*t))                y = h//2 + int((h//3) * math.sin(t)
+ (h//6)*math.cos(3*t))                points.append((x, y))
if len(points) &gt; 1:
                draw.line(points, fill=color, width=2)         def generate(self,
prompt: str, width: int = 512, height: int = 512) -&gt; str:         """
Generate an image from prompt.
        Returns base64-encoded PNG data URL.
"""
        concepts = self._extract_concepts(prompt)
colors = self._choose_colors(concepts)
seed_val = hash(prompt) % (2**32)
self.rng.seed(seed_val)

        # Create dark background
        img = Image.new('RGB', (width, height), (5, 5,
10))         draw = ImageDraw.Draw(img)                #
Layer 1: quantum wave field
        self._draw_quantum_field(draw, width, height, colors,
prompt)                # Layer 2: collapse circles (maybe)
if self.rng.random() &lt; 0.7:
            self._draw_collapse_circles(draw, width, height,
colors)                # Layer 3: entanglement knots         if
self.rng.random() &lt; 0.5:
            self._draw_entanglement_knots(draw, width, height,
colors)                # Layer 4: scattered particles         for _
in range(self.rng.randint(20, 50)):
            x = self.rng.randint(0, width)
y = self.rng.randint(0, height)
r = self.rng.randint(1, 3)
color = self.rng.choice(colors)
draw.ellipse([x-r, y-r, x+r, y+r],
fill=color)                # Encode to
base64 PNG         buffered = BytesIO()
img.save(buffered, format="PNG")
        img_b64 = base64.b64encode(buffered.getvalue()).decode('utf-
8')         data_url = f"{img_b64}"
return data_url


# Singleton instance _generator = None def get_quantum_image_generator(seed:
Optional[int] = None) -&gt; QuantumImageGenerator:
    global _generator     if _generator is
None:         _generator =
QuantumImageGenerator(seed)     return
_generator
```

## ■ backend/iterative_refiner.py

```
"""
■ ITERATIVE REFINER
Generate → Critique → Refine loop.
Self-improves output until quality threshold is met.
"""

from typing import Dict, Any, Optional,
List from PIL import Image import base64
from io import BytesIO

from renderer import get_renderer
from critic import ImageCritic


class IterativeRefiner:     """     Iteratively refines an image through
generate-critique-refine cycles.
    Targets a quality score threshold (default 0.75).
    """         def __init__(self, target_score: float = 0.75,
max_iterations: int = 3):
        self.target_score = target_score
self.max_iterations = max_iterations
self.renderer = get_renderer()
self.critic = ImageCritic()
    def refine(self, prompt: str, width: int = 512, height: int = 512,
initial_complexity: Optional[float] = None) -&gt; Dict[str, Any]:         """
Generate and refine an image.
                Args:             prompt:
text description             width, height:
output resolution
            initial_complexity: override initial complexity (if None, inferred from prompt)
Returns:
            {
                'image': data_url,
                'final_score': float,
                'iterations': int,
                'history': [                    {'iteration': 1, 'score': 0.6,
'feedback': [...], 'image': data_url},                      ...
                ]
            }
"""         best_image =
None         best_score
= 0.0          history =
[]
        current_prompt = prompt         current_complexity = initial_complexity
current_quality = None                 for iteration in range(1,
self.max_iterations + 1):             # 1. Generate             if iteration == 1
and current_complexity is not None:                 # Temporarily override
renderer complexity (need a way to pass it)                 # For now we just use
the prompt as-is
                img_data = self.renderer.render(current_prompt, width,
height)             else:                 # Modify prompt based on feedback
to improve                 if iteration &gt; 1 and history:
                    last_feedback = history[-1]['feedback']
                    current_prompt = self._enhance_prompt(prompt,
last_feedback)                 img_data = self.renderer.render(current_prompt,
width, height)                         # Decode for critique             img =
self._decode_image(img_data)
            # 2. Critique
            # Infer complexity from prompt (simple
heuristic)             complexity = 0.5             if
'black hole' in current_prompt.lower():
                complexity = 1.0             elif 'stick figure' in
current_prompt.lower():                 complexity = 0.05
evaluation = self.critic.evaluate(img, current_prompt, complexity)
history.append({                 'iteration': iteration,
                'score': evaluation['overall_score'],
```

```
                'feedback': evaluation['feedback'],
'suggestions': evaluation['suggestions'],
                'image': img_data,
'prompt_used': current_prompt
            })
                        # Track best             if
evaluation['overall_score'] &gt; best_score:
best_score = evaluation['overall_score']
best_image = img_data                          # 3. Check if target
reached             if evaluation['overall_score'] &gt;=
self.target_score:
                break                          # Prepare for next iteration
(modify prompt based on suggestions)             if iteration &lt;
self.max_iterations:
                current_prompt = self._enhance_prompt(current_prompt, evaluation['suggestions'])
return {             'image': best_image,
            'final_score': best_score,
            'iterations': len(history),
'target_met': best_score &gt;= self.target_score,
'history': history
        }         def _decode_image(self, data_url: str) -&gt; Image.Image:
"""Decode base64 PNG to PIL Image."""         b64 = data_url.split(',')[1]
img_bytes = base64.b64decode(b64)         return
Image.open(BytesIO(img_bytes))         def _enhance_prompt(self, base_prompt:
str, feedback: List[str]) -&gt; str:         """         Enhance prompt based
on critic feedback.
        This is a simple rule-based enhancement.
"""
        enhanced = base_prompt                 # Add quality boosters if needed
if any('complexity' in f.lower() or 'detail' in f.lower() for f in feedback):
if 'highly detailed' not in enhanced:
                enhanced = "highly detailed " + enhanced
if 'physically accurate' not in enhanced:
                enhanced = enhanced.replace('two', 'two physically
accurate')                  if any('composition' in f.lower() for f in
feedback):             if 'centered' not in enhanced:
                enhanced += " centered
composition"                 return enhanced
```

## ■ backend/preservation_test.py

```
#!/usr/bin/env python3
"""
PRESERVATION TEST — Single Pass/Fail Criterion Tests that the
unified brain preserves the "spilled coffee" moments. """

import asyncio import
sys from pathlib import
Path

# Add project to path
sys.path.insert(0, str(Path(__file__).parent))

from unified_quantum_brain import get_unified_brain from
quantum_brain import get_quantum_brain from
hybrid_generator import create_hybrid_generator from
chaos_engine import get_chaos_engine from quote_engine
import get_quote_engine from personality_engine import
get_personality_engine from dream_state import
get_dream_engine from quantum_language_engine import
QuantumLanguageEngine

TEST_QUERY = """In many religious and philosophical traditions, consciousness is treated as
more fundamental than matter - as if mind comes before universe rather than after it. Using
that as a starting point, tell me how you see reality.

First, describe the universe as current physics understands it (age, origin, large-scale
structure, dark matter/energy). Then, explain how you think consciousness and information
fit into that picture - whether they are emergent from matter, equal to it, or more
fundamental than it."""
```

```
def contains_chaotic_element(response: str) -&gt; tuple[bool, str]:
"""Check if response has something an editor would flag."""
        # 1. Raw fragment markers     if "[Raw:" in
response or "[Dream:" in response:
        return True, "raw fragment marker"         # 2. Broken grammar with
meaning (very short meaningful sentences)     sentences = [s.strip() for s in
response.split('.') if len(s.strip()) &gt; 10]     for sent in sentences:
        word_count = len(sent.split())         if 3 &lt;=
word_count &lt;= 8 and len(sent) &gt; 20:
return True, f"broken grammar: '{sent[:50]}...'"         #
3. Self-referential collapse
    if any(phrase in response.lower() for phrase in [
"your question", "you asked", "your query", "asking about"
    ]) and any(word in response.lower() for word in [
"consciousness", "reality", "universe", "mind"     ]):         return
True, "self-referential collapse"         # 4. Explicit confusion/gap
markers     gap_markers = [         "Understanding:", "Collapse
incomplete", "conceptual gaps",         "fragmented", "wrestling with",
"hard problem", "remains unknown"     ]     if any(marker in response
for marker in gap_markers):
        return True, "explicit confusion marker"         # 5. Vivid, off-
kilter metaphors (spilled coffee energy)     unusual_pairs = [
("spilled", "attic"), ("coffee", "chaos"), ("cluttered", "universe"),
("broken", "meaning"), ("fucking", "creepy")     ]
    response_lower = response.lower()     for a, b in
unusual_pairs:         if a in response_lower and b in
response_lower:             return True, f"unusual
metaphor: {a} + {b}"
    return False, "none"
def most_memorable_element(response: str) -&gt; str:     """Heuristic:
longest sentence with unusual words is likely memorable."""     sentences =
sorted(response.split('.'), key=len, reverse=True)     for sent in
sentences[:5]:         sent_lower = sent.lower()         if any(word in
sent_lower for word in [             'spilled', 'broken', 'fucking',
'creepy', 'raw', 'dream',
            'collapses', 'superposition', 'tubulins', 'chaos'
]):
            return sent.strip()     return
sentences[0].strip() if sentences else ""

async def run_preservation_test():
"""Execute the test."""     print("■
PRESERVATION TEST RUNNER")
print("=" * 80)         # Initialize
    print("Initializing components...")     brain = await
get_quantum_brain(None)         # Create language engine and
hybrid generator for raw fragments     lang_engine =
QuantumLanguageEngine()     hybrid =
create_hybrid_generator(lang_engine)
    quotes = get_quote_engine()
personality = get_personality_engine()
dreams = get_dream_engine(None)
    chaos = get_chaos_engine(quotes, personality,
dreams)     chaos.set_chaos_level(0.8)  # High chaos for
test
    unified = await get_unified_brain(brain, hybrid,
chaos)
    print("\n■ TEST
QUERY:")     print("-" * 80)
print(TEST_QUERY)
print("-" * 80)
    print("\n■ Generating response...")
result = await unified.think_preserving_chaos(
user_input=TEST_QUERY,         context={},
confusion_concepts=[],
raw_backend_fragments=[],
base_understanding_score=0.0     )
    response =
result["response"]
```

```
    print("\n■
RESPONSE:")     print("="
* 80)     print(response)
print("=" * 80)         #
Analysis
    has_chaos, reason =
contains_chaotic_element(response)     memorable =
most_memorable_element(response)
    print("\n■ ANALYSIS:")
    print(f"   Understanding score: {result['understanding_score']:.2f}")
print(f"   Chaos level: {result['chaos_level']:.2f}")       print(f"   Raw
fragments injected: {result['raw_fragments_injected']}")    print(f"  Self-
referential:  {result['self_referential']}")          print(f"   Preserved
weirdness: {result['preserved_weirdness']}")    print(f"\n  Chaotic element
detected: {has_chaos} ({reason})")     print(f"  Most memorable sentence:
{memorable[:100]}...")         # PASS/FAIL     test_passed = has_chaos
    print("\n" + "=" * 80)
if test_passed:
        print("■ TEST PASSED")
        print("The response contains a memorable, editor-flaggable element.")         print("The 'spilled coffee' moment survived.")     else:
        print("■ TEST FAILED")
        print("The response is too clean. Something smoothed the
output.")     print("=" * 80)         return test_passed

if __name__ == "__main__":
    success = asyncio.run(run_preservation_test())
sys.exit(0 if success else 1)
```

## ■ backend/renderer.py

```
"""
■ COGNITIVE RENDERING ENGINE
Master orchestrator that understands prompts and produces high-quality images.
Selects appropriate physics engines, manages complexity, composes final output.
"""

import math
import random
from typing import List, Dict, Any, Optional
from PIL import Image, ImageDraw, ImageFilter, ImageChops, ImageEnhance

from concept_ontology import get_ontology, ConceptDefinition,
RenderingStrategy from physics_engines.black_hole import BlackHoleRenderer
from physics_engines.human_figure import HumanFigureRenderer from
physics_engines.geometric import GeometricRenderer


class CognitiveRenderer:     """     The master rendering engine. This is
the "artist" that understands     concepts and can render them at any level
of complexity.     """         def __init__(self, seed: Optional[int] =
None):         self.ontology = get_ontology()         self.seed = seed if
seed is not None else 12345         self.rng = random.Random(self.seed)  #
For post-processing noise etc.                 # Initialize physics engines
        self.black_hole_engine = BlackHoleRenderer(seed=self.seed)
self.human_figure_engine = HumanFigureRenderer(seed=self.seed)
self.geometric_engine = GeometricRenderer(seed=self.seed)
# Engine mapping         self.engines = {
RenderingStrategy.BLACK_HOLE: self.black_hole_engine,
            RenderingStrategy.HUMAN_FIGURE: self.human_figure_engine,
            RenderingStrategy.GEOMETRIC: self.geometric_engine,
RenderingStrategy.PARTICLE_SYSTEM: self.geometric_engine,
RenderingStrategy.COMPOSITE: None,
        }         def _parse_concepts(self, prompt: str) -&gt;
List[Dict[str, Any]]:         """Extract concept instances from
prompt."""         words = prompt.lower().replace(',', ' ').split()
# Count objects         count = 1
        numbers = ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine',
'ten']         for i, num in enumerate(numbers, 1):             if num in words:
count = i                 break         for word in words:             if word.isdigit():
count = int(word)                 break                 # Find concepts         concepts_found =
[]         for concept_name in self.ontology.list_concepts():             if concept_name in
prompt.lower():
                concept_def = self.ontology.get_concept(concept_name)
if concept_def:
                    concepts_found.append({
'name': concept_name,
                        'definition': concept_def,
'count': count,
                    })
if not concepts_found:
            default_concept = self.ontology.get_concept('default')
            concepts_found.append({
'name': 'default',
'definition': default_concept,
'count': count,

})
        return concepts_found     def _calculate_scene_complexity(self, concepts: List[Dict[str, Any]]) -
&gt; float:         """Overall complexity = max of individual complexities."""         complexities =
[c['definition'].base_complexity for c in concepts]         return max(complexities) if complexities else
0.3         def _determine_quality_settings(self, complexity: float, width: int, height: int) -&gt;
Dict[str, Any]:         """Map complexity to rendering parameters."""         if complexity &lt; 0.2:
            return {
'render_scale': 1.0,
'background_stars': False,
                'glow': False,
                'noise': False,
```

```
                'color_correction':
False,                  'max_objects': 5
}         elif complexity &lt; 0.5:
return {                  'render_scale':
1.5,
                'background_stars': True,
'glow': complexity &gt; 0.4,
                'noise': False,
                'color_correction':
False,                  'max_objects': 10
}         else:
            return {
'render_scale': 2.0,
'background_stars': True,
                'glow': True,
                'noise': complexity &gt; 0.8,
                'color_correction': True,
'max_objects': 20
            }         def _layout_objects(self, width: int, height: int, concept_counts: List[Dict[str, Any]]) -&gt;
List[Dict[str, Any]]:         """Arrange objects in the scene."""         instances = []         margin = width // 8
for concept_info in concept_counts:             count = concept_info['count']             definition =
concept_info['definition']                         if count == 1:
                x_positions = [width // 2]
else:
                spacing = (width - 2 * margin) // (count + 1)
                x_positions = [margin + (i + 1) * spacing for i in range(min(count,
20))]                           for i, x in enumerate(x_positions):
y_jitter = height // 20 * ((self.seed + i) % 7 - 3)                 y_pos = height // 2
+ y_jitter
                base_scale = min(width, height) / 800 * 0.8
if definition.base_complexity &gt; 0.8:
                    base_scale *= 1.2
elif definition.base_complexity &lt; 0.2:
                    base_scale *=
0.7
                instances.append({
'concept_name': concept_info['name'],
                    'definition': definition,
                    'position': (x, y_pos),
                    'scale': base_scale * (0.9 + 0.2 * (self.seed + i) % 10 / 10),
'color_index': i % len(definition.visual_properties.get('color_palette', [(255,255,255)]))
                })
        return instances         def _render_object(self, obj_instance: Dict[str, Any], quality: Dict[str, Any], canvas_width: int,
canvas_height: int) -&gt; Image.Image:         """Render a single object using its appropriate physics engine."""         definition =
obj_instance['definition']         strategy = definition.strategy         engine = self.engines.get(strategy)                 if not engine:
            return self._render_placeholder(obj_instance, canvas_width,
canvas_height)
        obj_size = int(200 * obj_instance['scale'])         local_img =
Image.new('RGBA', (obj_size * 2, obj_size * 2), (0, 0, 0, 0))         local_draw
= ImageDraw.Draw(local_img)
        cx, cy = obj_size, obj_size
        palette = definition.visual_properties.get('color_palette', [(255, 255,
255)])         color = palette[obj_instance['color_index']]                 if
strategy == RenderingStrategy.BLACK_HOLE:             # Render black hole at object's
local size (much faster)             local_size = obj_size * 2             bh_render
= engine.render(                 width=local_size,                 height=local_size,
                complexity=definition.base_complexity,
                mass=definition.parameters.get('mass', {}).get('default', 50.0)
)
            return bh_render.convert('RGBA')
elif strategy == RenderingStrategy.HUMAN_FIGURE:
            fig_img = engine.render(
width=obj_size*2,
height=obj_size*3,

complexity=definition.base_complexity,
scale=obj_instance['scale'],
color=color,                 pose="standing"
)
```

```
            return fig_img.convert('RGBA')
elif strategy == RenderingStrategy.GEOMETRIC:
if 'circle' in definition.name:
                engine.draw_circle(local_draw, cx, cy, obj_size, color, fill=False, line_width=3)
elif 'line' in definition.name:
                engine.draw_antialiased_line(local_draw, (cx - obj_size, cy), (cx + obj_size, cy), color,
width=3)             elif 'wave' in definition.name or 'field' in definition.name:
engine.draw_wave_pattern(local_draw, obj_size*2, obj_size*2,
                                        definition.base_complexity, palette, str(self.seed))
elif 'pattern' in definition.name:
                engine.draw_geometric_pattern(local_draw, obj_size*2, obj_size*2,
definition.base_complexity, palette)             else:
                engine.draw_circle(local_draw, cx, cy, obj_size, color,
fill=True)
            return
local_img
else:
            return self._render_placeholder(obj_instance, canvas_width, canvas_height)
def _render_placeholder(self, obj_instance: Dict[str, Any], w: int, h: int) -&gt; Image.Image:
        img = Image.new('RGBA', (w, h), (0, 0, 0,
0))         draw = ImageDraw.Draw(img)         try:
            draw.text((w//2, h//2), f"?{obj_instance['concept_name']}", fill=(255,255,255))         except:
pass         return img         def _apply_post_processing(self, img: Image.Image, complexity: float, quality:
Dict[str, Any]) -&gt; Image.Image:         """Apply post-processing effects."""         if quality['glow']:
            glow = img.filter(ImageFilter.GaussianBlur(radius=3))
img = ImageChops.screen(img, glow)
        if quality['color_correction']:
enhancer = ImageEnhance.Color(img)             img
= enhancer.enhance(1.2)             enhancer =
ImageEnhance.Contrast(img)             img =
enhancer.enhance(1.1)                if
quality['noise']:             pixels = img.load()
w, h = img.size             for i in range(w):
for j in range(h):                     if
self.rng.random() &lt; 0.1:
v = self.rng.randint(-8, 8)
r, g, b = pixels[i, j]
                        pixels[i, j] = (max(0, min(255, r+v)), max(0, min(255, g+v)), max(0, min(255,
b+v)))                 return img     def render(self, prompt: str, width: int = 1024, height: int = 1024)
-&gt; str:         """         Generate image from prompt.
        Returns base64 PNG data URL.
        """         #
Parse concepts
        concepts =
self._parse_concepts(prompt)                 #
Determine complexity
        scene_complexity =
self._calculate_scene_complexity(concepts)                 # Quality
settings
        quality = self._determine_quality_settings(scene_complexity, width,
height)                 # Render resolution
        render_scale =
quality['render_scale']         render_w =
int(width * render_scale)         render_h =
int(height * render_scale)                 #
Create base canvas
        bg_color = (5, 5, 12) if scene_complexity &gt; 0.3 else (12, 15,
22)         base = Image.new('RGB', (render_w, render_h), bg_color)
draw = ImageDraw.Draw(base)                 # Background stars         if
quality['background_stars']:
            self.geometric_engine.draw_starfield(draw, render_w, render_h,
density=0.02)                 # Layout objects
        obj_instances = self._layout_objects(render_w, render_h,
concepts)                 # Render and composite objects         for obj
in obj_instances:
            obj_img = self._render_object(obj, quality, render_w, render_h)
if obj_img:
                pos = obj['position']
ox = pos[0] - obj_img.width // 2
oy = pos[1] - obj_img.height // 2
```

```
                base.paste(obj_img, (ox, oy), obj_img if obj_img.mode == 'RGBA' else
None)                  # Post-processing
        final = self._apply_post_processing(base, scene_complexity,
quality)                 # Downsample         if render_scale &gt; 1.0:
            final = final.resize((width, height),
Image.Resampling.LANCZOS)                  # Encode         from io import
BytesIO         import base64         buffered = BytesIO()
        final.save(buffered, format="PNG", quality=95, optimize=True)
img_b64 = base64.b64encode(buffered.getvalue()).decode('utf-8')
return f"{img_b64}"


# Global renderer instance
_renderer: Optional[CognitiveRenderer] = None

def get_renderer(seed: Optional[int] = None) -&gt;
CognitiveRenderer:     """Get or create the global cognitive
renderer."""     global _renderer     if _renderer is None:
_renderer = CognitiveRenderer(seed)     return _renderer
```

## ■ backend/renderer_final.py

```
"""
■ FINAL COGNITIVE RENDERER - High Quality
Uses the best physics engines for photorealistic output.
"""

import math
import random
from typing import List, Dict, Any, Optional from PIL
import Image, ImageDraw, ImageFilter, ImageEnhance

from concept_ontology import get_ontology, ConceptDefinition, RenderingStrategy
# Use final engines
from physics_engines.black_hole_final import
FinalBlackHoleRenderer from physics_engines.human_figure import
HumanFigureRenderer from physics_engines.geometric import
GeometricRenderer


class FinalCognitiveRenderer:     """The ultimate
renderer with highest quality."""         def
__init__(self, seed: Optional[int] = None):
self.ontology = get_ontology()
        self.seed = seed if seed is not None else
12345         self.rng = random.Random(self.seed)
# Initialize engines
        self.black_hole_engine =
FinalBlackHoleRenderer(seed=self.seed)
self.human_figure_engine = HumanFigureRenderer(seed=self.seed)
self.geometric_engine = GeometricRenderer(seed=self.seed)
        self.engines = {
RenderingStrategy.BLACK_HOLE: self.black_hole_engine,
            RenderingStrategy.HUMAN_FIGURE: self.human_figure_engine,
            RenderingStrategy.GEOMETRIC: self.geometric_engine,
RenderingStrategy.PARTICLE_SYSTEM: self.geometric_engine,
        }         def _parse_concepts(self, prompt: str) -&gt;
List[Dict[str, Any]]:
        words = prompt.lower().replace(',', ' ').split()
count = 1
        numbers = ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine',
'ten']         for i, num in enumerate(numbers, 1):             if num in words: count = i;
break         for word in words:
            if word.isdigit(): count = int(word);
break
        concepts_found = []         for concept_name in
self.ontology.list_concepts():             if
concept_name in prompt.lower():
                concept_def = self.ontology.get_concept(concept_name)
if concept_def:
                    concepts_found.append({'name': concept_name, 'definition': concept_def, 'count': count})
if not concepts_found:
            default_concept = self.ontology.get_concept('default')
if default_concept:
                concepts_found.append({'name': 'default', 'definition': default_concept, 'count':
count})             else:                 # Fallback to simple circle                 fallback =
ConceptDefinition(
                    name='fallback', category='geometric', base_complexity=0.0,
strategy=RenderingStrategy.GEOMETRIC,
                    visual_properties={'color_palette': [(200, 200,
200)]},                     components=[]                 )
                concepts_found.append({'name': 'fallback', 'definition': fallback, 'count':
count})
        return concepts_found         def _calculate_scene_complexity(self, concepts:
List[Dict[str, Any]]) -&gt; float:
        complexities = [c['definition'].base_complexity for c in concepts]
return max(complexities) if complexities else 0.3
    def _layout_objects(self, width: int, height: int, concept_counts: List[Dict[str, Any]]) -&gt; List[Dict[str, Any]]:
```

```
        instances = []         margin = width // 8
for concept_info in concept_counts:
count = concept_info['count']
definition = concept_info['definition']
if count == 1:
                x_positions = [width // 2]
else:
                spacing = (width - 2 * margin) // (count + 1)
                x_positions = [margin + (i + 1) * spacing for i in range(min(count,
10))]                           for i, x in enumerate(x_positions):
y_jitter = height // 20 * ((self.seed + i) % 7 - 3)                 y_pos = height // 2
+ y_jitter
                base_scale = min(width, height) / 800 * 0.8
if definition.base_complexity &gt; 0.8:
                    base_scale *= 1.2
elif definition.base_complexity &lt; 0.2:
                    base_scale *=
0.7
                instances.append({
'concept_name': concept_info['name'],
                    'definition': definition,
                    'position': (x, y_pos),
                    'scale': base_scale * (0.9 + 0.2 * (self.seed + i) % 10 / 10),
'color_index': i % len(definition.visual_properties.get('color_palette', [(255,255,255)]))

})
        return instances         def _render_object(self, obj_instance: Dict[str, Any], width: int, height: int)
-&gt; Optional[Image.Image]:
        definition =
obj_instance['definition']         strategy =
definition.strategy         engine =
self.engines.get(strategy)                 if
not engine:             return None
        palette = definition.visual_properties.get('color_palette', [(255, 255,
255)])         color = palette[obj_instance['color_index']]         scale =
obj_instance['scale']                 if strategy == RenderingStrategy.BLACK_HOLE:
# Render black hole at full quality
            return engine.render(width, height, complexity=definition.base_complexity,
mass=50.0)                 elif strategy == RenderingStrategy.HUMAN_FIGURE:
            size = int(200 * scale)
            return engine.render(width=size*2, height=size*3,
complexity=definition.base_complexity,                                 scale=scale, color=color,
pose="standing")                 elif strategy == RenderingStrategy.GEOMETRIC:             # For
geometric shapes, draw directly on a canvas             size = int(150 * scale)
            img = Image.new('RGBA', (size*2, size*2),
(0,0,0,0))             draw = ImageDraw.Draw(img)
cx, cy = size, size                         if 'circle' in
definition.name:
                r = int(size * 0.8)
                draw.ellipse([cx-r, cy-r, cx+r, cy+r], outline=color, width=3)
elif 'line' in definition.name:
                draw.line([(cx-size, cy), (cx+size, cy)], fill=color,
width=3)             elif 'wave' in definition.name or 'field' in
definition.name:                 # Simple wave pattern                 points
= []                 for x in range(cx-size, cx+size, 2):
                    y = cy + int(20 * math.sin((x-
cx)*0.1))                     points.append((x, y))
if len(points) &gt; 1:                     for i in
range(len(points)-1):
draw.line([points[i], points[i+1]], fill=color, width=2)
else:
                draw.ellipse([cx-size, cy-size, cx+size, cy+size], fill=color)
return img                 return None         def render(self, prompt: str,
width: int = 1024, height: int = 1024) -&gt; str:         """Generate high-quality
image from prompt."""         # Parse
        concepts = self._parse_concepts(prompt)
scene_complexity = self._calculate_scene_complexity(concepts)
# Create base image
        bg_color = (5, 5, 12) if scene_complexity &gt; 0.3 else (12, 15,
22)         base = Image.new('RGB', (width, height), bg_color)
# Layout objects
```

```
        obj_instances = self._layout_objects(width, height,
concepts)                  # Render each object         for obj in
obj_instances:
            obj_img = self._render_object(obj, width, height)
if obj_img:
                pos = obj['position']
ox = pos[0] - obj_img.width // 2
oy = pos[1] - obj_img.height // 2
if obj_img.mode == 'RGBA':
                    base.paste(obj_img, (ox, oy), obj_img)
else:
                    base.paste(obj_img, (ox, oy))                 # === BLACK HOLE
SPECIFIC: If only one black hole centered, improve quality ===         if
scene_complexity &gt; 0.8 and len(obj_instances) == 1:             # Replace with
high-res centered black hole
            bh_img = self.black_hole_engine.render(width, height,
complexity=scene_complexity)              base = bh_img                 # Post-processing
if scene_complexity &gt; 0.5:              # Unsharp mask for sharpness
            base = base.filter(ImageFilter.UnsharpMask(radius=1, percent=120, threshold=3))
# Color boost
            enhancer =
ImageEnhance.Color(base)              base =
enhancer.enhance(1.2)             # Contrast
            enhancer =
ImageEnhance.Contrast(base)              base =
enhancer.enhance(1.1)                  # Encode
from io import BytesIO         import base64
buffered = BytesIO()
        base.save(buffered, format="PNG", quality=95, optimize=True)
        return f"{base64.b64encode(buffered.getvalue()).decode('utf-8')}"


# Global renderer instance _renderer:
Optional[FinalCognitiveRenderer] = None

def get_final_renderer(seed: Optional[int] = None) -&gt;
FinalCognitiveRenderer:     """Get or create the global final renderer."""
global _renderer     if _renderer is None:         _renderer =
FinalCognitiveRenderer(seed)     return _renderer
```

## ■ backend/renderer_photorealistic.py

---

```
"""
■ PHOTOREALISTIC COGNITIVE RENDERER
Uses the new high-quality physics engines.
"""

import math
import random
import numpy as
np
from typing import List, Dict, Any, Optional from PIL
import Image, ImageDraw, ImageFilter, ImageEnhance

from concept_ontology import get_ontology, ConceptDefinition,
RenderingStrategy from physics_engines.black_hole_photorealistic import
PhotorealisticBlackHole from physics_engines.human_figure import
HumanFigureRenderer from physics_engines.geometric import GeometricRenderer


class PhotorealisticRenderer:     """High-quality renderer
with realistic black holes."""         def __init__(self,
seed: Optional[int] = None):         self.ontology =
get_ontology()
        self.seed = seed if seed is not None else
12345         self.rng = random.Random(self.seed)
# Engines
        self.black_hole_engine =
PhotorealisticBlackHole(seed=self.seed)         self.human_engine =
HumanFigureRenderer(seed=self.seed)         self.geo_engine =
```

```
GeometricRenderer(seed=self.seed)                 self.engines = {
RenderingStrategy.BLACK_HOLE: self.black_hole_engine,
            RenderingStrategy.HUMAN_FIGURE: self.human_engine,
RenderingStrategy.GEOMETRIC: self.geo_engine,
        }         def _parse_concepts(self, prompt: str) -&gt;
List[Dict[str, Any]]:
        words = prompt.lower().replace(',', ' ').split()
count = 1
        numbers = ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine',
'ten']         for i, num in enumerate(numbers, 1):             if num in words: count = i;
break         for word in words:
            if word.isdigit(): count = int(word);
break                 concepts = []          for name
in self.ontology.list_concepts():             if name
in prompt.lower():
                defn = self.ontology.get_concept(name)
if defn:
                    concepts.append({'name': name, 'def': defn, 'count':
count})                 if not concepts:
            default = self.ontology.get_concept('default')
if default:
                concepts.append({'name': 'default', 'def': default, 'count':
count})             else:                 # Fallback black hole
                from concept_ontology import ConceptDefinition, RenderingStrategy
fallback = ConceptDefinition(
                    name='black_hole', category='astronomical',
base_complexity=1.0,                     strategy=RenderingStrategy.BLACK_HOLE,
visual_properties={'color_palette': [(255,255,255)]},
components=[]                 )
                concepts.append({'name': 'black_hole', 'def': fallback, 'count': count})
return concepts         def _layout_two_black_holes(self, width: int, height: int, count: int) -&gt;
List[Dict[str, Any]]:         """Special layout for merging black holes."""         instances = []
        cx, cy = width // 2, height // 2
base_scale = min(width, height) / 300
# Place in orbit         orbit_radius =
width * 0.15         for i in range(count):
            angle = (2 * math.pi * i / count) + (self.seed % 10) * 0.1             x =
cx + int(orbit_radius * math.cos(angle))             y = cy + int(orbit_radius * 0.6 *
math.sin(angle))  # Flatten for perspective             scale = base_scale * (0.9 if i
== 0 else 1.0)
            instances.append({
'type': 'black_hole',
                'position': (x, y),                 'scale': scale             })
return instances         def render(self, prompt: str, width: int = 1024, height:
int = 1024) -&gt; str:         # Parse
        concepts = self._parse_concepts(prompt)
        is_bh_merger = any(c['def'].strategy == RenderingStrategy.BLACK_HOLE for c in concepts) and \
('merg' in prompt.lower() or 'two' in prompt.lower() or 'collision' in prompt.lower())
                #
Create canvas
        img = Image.new('RGB', (width, height), (5, 5,
12))                 if is_bh_merger and len(concepts) ==
1:             # Special case: render overlapping BHs
bh_scale = min(width, height) / 250             offset_x =
width * 0.15             offset_y = height * 0.1
# Render first BH
            bh1 = self.black_hole_engine.render(width, height, complexity=1.0,
inclination=0.2)             img.paste(bh1, (0, 0), bh1 if bh1.mode == 'RGBA' else None)
# Render second BH shifted
            bh2 = self.black_hole_engine.render(width, height, complexity=1.0, inclination=0.2)
img2 = Image.new('RGB', (width, height), (0,0,0))
            img2.paste(bh2, (int(offset_x), int(offset_y)), bh2 if bh2.mode == 'RGBA' else
None)                         # Blend with screen mode effect             arr1 = np.array(img,
dtype=np.float32)             arr2 = np.array(img2, dtype=np.float32)
            merged = np.maximum(arr1, arr2) * 0.9 + np.minimum(arr1, arr2) *
0.1             img = Image.fromarray(np.clip(merged, 0, 255).astype(np.uint8))
else:             # Standard single-concept rendering             concept =
concepts[0]
            engine = self.engines.get(concept['def'].strategy)
if engine:
```

```
                rendered = engine.render(width, height, complexity=concept['def'].base_complexity)
if rendered:
                    img =
rendered                  # Final
touches
        img = img.filter(ImageFilter.UnsharpMask(radius=1, percent=110,
threshold=3))          enhancer = ImageEnhance.Color(img)         img =
enhancer.enhance(1.25)         enhancer = ImageEnhance.Contrast(img)         img =
enhancer.enhance(1.1)                  # Encode         from io import BytesIO
import base64         buffered = BytesIO()
        img.save(buffered, format="PNG", quality=95, optimize=True)
        return f"{base64.b64encode(buffered.getvalue()).decode('utf-8')}"


def get_photorealistic_renderer(seed: Optional[int] =
None):     """Get global renderer instance."""     global
_renderer
    try:
        return _renderer     except NameError:
_renderer = PhotorealisticRenderer(seed)
return _renderer
```

```
  ULTIMATE RENDERER - Photorealistic Black Holes
"""

import numpy as
np import math
import random
from typing import List, Dict, Any, Optional from PIL
import Image, ImageDraw, ImageFilter, ImageEnhance

from concept_ontology import get_ontology, ConceptDefinition,
RenderingStrategy from physics_engines.black_hole_ultimate import
UltimateBlackHoleRenderer from physics_engines.human_figure import
HumanFigureRenderer from physics_engines.geometric import GeometricRenderer


class UltimateRenderer:     """Top-tier renderer."""
def __init__(self, seed: Optional[int] = None):
self.ontology = get_ontology()         self.seed =
seed if seed is not None else 12345         self.rng =
random.Random(self.seed)
        self.bh_engine =
UltimateBlackHoleRenderer(seed=self.seed)
self.human_engine = HumanFigureRenderer(seed=self.seed)
self.geo_engine = GeometricRenderer(seed=self.seed)
self.engines = {             RenderingStrategy.BLACK_HOLE:
self.bh_engine,
            RenderingStrategy.HUMAN_FIGURE: self.human_engine,
RenderingStrategy.GEOMETRIC: self.geo_engine,
        }         def _parse_prompt(self, prompt: str) -&gt;
Dict[str, Any]:
        words = prompt.lower().split()         count
= 1         for w in words:             if
w.isdigit():                 count = int(w)
break             if w in ['two', 'three', 'four',
'five']:
                count = {'two':2, 'three':3, 'four':4,
'five':5}[w]                 break
        concept_name = None         for name in
self.ontology.list_concepts():             if
name in prompt.lower():
concept_name = name                 break
if not concept_name:
            concept_name = 'black_hole'  #
default
        concept_def =
self.ontology.get_concept(concept_name)         if not
concept_def:             # fallback
            from concept_ontology import ConceptDefinition, RenderingStrategy
concept_def = ConceptDefinition(
                name='fallback', category='astronomical', base_complexity=1.0,
strategy=RenderingStrategy.BLACK_HOLE,
                visual_properties={'color_palette':[(255,255,255)]},
components=[]             )
        return {'name': concept_name, 'def': concept_def, 'count': count}
def render(self, prompt: str, width: int = 1024, height: int = 1024) -&gt; str:
        info = self._parse_prompt(prompt)
        is_merger = info['count'] &gt;= 2 and info['def'].strategy ==
RenderingStrategy.BLACK_HOLE                 # Create base
        img = Image.new('RGB', (width, height), (0,0,0))
if is_merger:
            count = min(info['count'], 3)
            base = np.zeros((height, width, 3),
dtype=np.float32)             orbit_r = width * 0.12
for i in range(count):
                angle = 2 * math.pi * i / count                 offx = int(orbit_r * math.cos(angle))
offy = int(orbit_r * math.sin(angle) * 0.5)                 bh = self.bh_engine.render(width, height,
complexity=info['def'].base_complexity, inclination=0.1)                 # Convert to RGB if RGBA
if bh.mode == 'RGBA':
```

# ■

"""

■

bh = bh.convert('RGB')
arr = np.array(bh, dtype=np.float32)  #
Compute destination and source rectangles
dest_x0 = max(0, offx)  dest_x1 =
min(width, offx + width)  src_x0 = dest_x0
- offx  src_x1 = src_x0 + (dest_x1 -
dest_x0)
dest_y0 = max(0, offy)  dest_y1 =
min(height, offy + height)  src_y0 = dest_y0 - offy
src_y1 = src_y0 + (dest_y1 - dest_y0)
if dest_x1 > dest_x0 and dest_y1 > dest_y0:
base[dest_y0:dest_y1, dest_x0:dest_x1] = np.maximum(
base[dest_y0:dest_y1, dest_x0:dest_x1],
arr[src_y0:src_y1, src_x0:src_x1]  )
img =
Image.fromarray(np.clip(base,0,255).astype(np.uint8))  else:
# Single object
engine = self.engines.get(info['def'].strategy)
if engine:
rendered = engine.render(width, height, complexity=info['def'].base_complexity)
if rendered:
img =
rendered  # Global
enhancements
img = img.filter(ImageFilter.UnsharpMask(radius=1, percent=100,
threshold=3))  enhancer = ImageEnhance.Color(img)  img =
enhancer.enhance(1.2)  enhancer = ImageEnhance.Contrast(img)  img =
enhancer.enhance(1.1)  # Encode  from io import BytesIO
import base64  buf = BytesIO()
img.save(buf, format="PNG", quality=95, optimize=True)
return f"{base64.b64encode(buf.getvalue()).decode('utf-8')}"


def get_ultimate_renderer(seed: Optional[int] = None):
global _renderer
try:
return _renderer  except
NameError:  _renderer =
UltimateRenderer(seed)  return
_renderer


Evaluation Routes
==========  Rubric-based
scoring for Quantum MCAGI. """

from fastapi import APIRouter, HTTPException,
Body from pydantic import BaseModel from typing
import Dict, List, Optional, Any import logging

import shared_state as state
from evaluation_engine import get_evaluation_engine

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/api")


class EvaluateRequest(BaseModel):  """Request payload for evaluation."""  input_text: str
response: str  context: Optional[Dict[str, Any]] = None  # can include internal_questions,
semantic_context, etc.


class EvaluateResponse(BaseModel):
"""Response with rubric scores."""
scores: Dict[str, int]  # dimension -> score 0-4
summary: Optional[str] = None

```
@router.post("/evaluate",
response_model=EvaluateResponse) async def
evaluate_response(req: EvaluateRequest):     """
Evaluate a response using the RQR³ rubric.
        Requires the QuantumBrain to be initialized (provides collapse_engine and
markov).     """     try:         # Grab the quantum brain to access collapse engine
and markov         if hasattr(state, 'quantum_brain') and state.quantum_brain is not
None:
            brain = state.quantum_brain
else:
            from quantum_brain import
get_quantum_brain             brain = await
get_quantum_brain(state.db)
state.quantum_brain = brain
        collapse_engine = brain.collapse_engine if brain else
None         markov = None         # Get hybrid generator's
markov if accessible         try:
            from hybrid_generator import HybridGenerator             # Not stored
directly; we'll pass None for now; evaluation can still run         except:
pass                 # Create evaluation engine         engine =
get_evaluation_engine(             collapse_engine=collapse_engine,
markov=markov,             config={}         )
                #
Compute scores
        raw_scores = engine.score_response(             input_text=req.input_text,
response=req.response,             context=req.context or {}         )         # Remove
any None values (e.g., question_generation_quality when not applicable)         scores
= {k: v for k, v in raw_scores.items() if v is not None}
        # Generate a brief summary text
        dimensions = ['coherence', 'fluency', 'uniqueness', 'grounding',
'personality_authenticity', 'implementation_fidelity',
'emergent_behavior', 'question_generation_quality']         summary_parts =
[]         for dim in dimensions:             if dim in scores:
summary_parts.append(f"{dim.capitalize()}: {scores[dim]}/4")
summary = " | ".join(summary_parts)
        return EvaluateResponse(scores=scores,
summary=summary)             except Exception as e:
        logger.error(f"Evaluation error: {str(e)}")
raise HTTPException(status_code=500, detail=str(e))
```

## ■

```
"""
■
```

```
  Image Generation Routes
========================= Custom procedural image
generation (no external APIs). """

from fastapi import APIRouter,
HTTPException from pydantic import
BaseModel from typing import Optional
import logging

import shared_state as state
from renderer_ultimate import get_ultimate_renderer as get_renderer
from iterative_refiner import IterativeRefiner

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/api")


class ImageRequest(BaseModel):
prompt: str     width:
Optional[int] = 512
height: Optional[int] = 512
    refine: Optional[bool] = True  # Whether to use iterative refinement
target_score: Optional[float] = 0.7  # Quality target 0.0-1.0
max_iterations: Optional[int] = 2  # Max refinement iterations


@router.post("/image/generate") async def generate_image(req:
ImageRequest):     """     Generate an image from a text prompt.
"""     try:         # Wrap entire generation in try/except to
prevent server crash         if req.refine:
            refiner = IterativeRefiner(target_score=req.target_score,
max_iterations=req.max_iterations)             result = refiner.refine(req.prompt, req.width,
req.height)             return {                 "image": result["image"],
                "prompt": req.prompt,
                "score": result["final_score"],
                "iterations":
result["iterations"],
"target_met": result["target_met"]             }
else:
            renderer = get_renderer()
            data_url = renderer.render(req.prompt, req.width,
req.height)             return {"image": data_url, "prompt": req.prompt}
except Exception as e:         import traceback
        logger.error(f"Image generation failed: {e}\n{traceback.format_exc()}")
raise HTTPException(status_code=500, detail=str(e))
```

## ■ backend/shared_state_backup.py

```
"""
■ QUANTUM AI — Shared State
============================= Central state
holder for all shared instances.
Route modules import from here to access db, engines, etc.
"""

import os
import json
import uuid
import asyncio
import logging
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Dict, List, Optional
from dotenv import load_dotenv

logger = logging.getLogger("quantum_ai")
```

```
ROOT_DIR = Path(__file__).parent
load_dotenv(ROOT_DIR / '.env')

# ============================================================================
# LOCAL MEMORY BACKEND (MongoDB-free operation) #
==============================================================================

class SimpleLocalDB:     """     Simple file-based database that
mimics MongoDB's async interface.
    Uses JSON files stored in ~/.quantum-mcagi-backend/.
    """         def __init__(self,
data_dir=None):
        self.data_dir = Path(data_dir or os.path.expanduser("~/.quantum-mcagi-
backend"))         self.data_dir.mkdir(parents=True, exist_ok=True)
self._collections = {}         def __getattr__(self, name: str) -&gt;
'SimpleCollection':         if name not in self._collections:
self._collections[name] = SimpleCollection(name, self.data_dir)         return
self._collections[name]         async def command(self, command: str) -&gt; Dict[str,
Any]:         """Mimics MongoDB command interface."""         if command == "ping":
return {"ok": 1}
        raise NotImplementedError(f"Command {command} not supported")


class SimpleCollection:     """A collection that stores
documents in a JSON file."""         def __init__(self,
name: str, data_dir: Path):         self.name = name
        self.filepath = data_dir / f"{name}.json"
self._data: List[Dict] = []         self._lock =
asyncio.Lock()         self._load()         def
_load(self):         """Load data from disk
(synchronous)."""         if
self.filepath.exists():             try:
with open(self.filepath, 'r') as f:
                    self._data = json.load(f)
except (json.JSONDecodeError, IOError):
                self._data = []
else:
            self._data =
[]         async def
_save(self):
        """Save data to disk (async)."""
```

```
        async with self._lock:             try:
# Convert to JSON-serializable format
serializable = []                  for doc in
self._data:
                    serializable.append(self._serialize_doc(doc))
with open(self.filepath, 'w') as f:
json.dump(serializable, f, indent=2, default=str)             except
Exception as e:
                print(f"Error saving {self.name}: {e}")
def _serialize_doc(self, doc: Dict) -&gt; Dict:
"""Convert datetime objects to ISO strings for JSON."""
result = {}         for k, v in doc.items():             if
isinstance(v, datetime):                  result[k] =
v.isoformat()             elif isinstance(v, dict):
                result[k] = self._serialize_doc(v)
else:
                result[k] = v         return result         def
_deserialize_doc(self, doc: Dict) -&gt; Dict:         """Convert
ISO strings back to datetime objects."""         result = {}
for k, v in doc.items():             if isinstance(v, str) and
'T' in v and len(v) &gt; 19:                  try:
                    result[k] = datetime.fromisoformat(v.replace('Z', '+00:00'))
except:
                    result[k] = v
elif isinstance(v, dict):
                result[k] = self._deserialize_doc(v)
else:
                result[k] = v         return result         def
_ensure_ids(self):         """Ensure all documents have _id
fields (synchronous)."""         for doc in self._data:
if '_id' not in doc:
                doc['_id'] = str(uuid.uuid4())         def find(self, filter: Optional[Dict] = None,
projection: Optional[Dict] = None) -&gt; 'AsyncCursor':         """Find documents matching filter."""
self._ensure_ids()         results = []         for doc in self._data:             if self._matches(doc,
filter):                 # Apply projection if provided (simple: exclude _id if specified)
result_doc = self._deserialize_doc(doc.copy())                  if projection:                     # Very
basic projection: only include fields where value is 1 (or exclude if 0)                     # For
simplicity, we'll just ignore projection and return full doc                     pass
                results.append(result_doc)         return AsyncCursor(results)         async def find_one(self,
filter: Optional[Dict] = None, projection: Optional[Dict] = None) -&gt; Optional[Dict]:         """Find first
matching document."""         self._ensure_ids()         for doc in self._data:             if self._matches(doc,
filter):
                result = self._deserialize_doc(doc.copy())                 # For now,
ignore projection (Motor includes _id by default unless excluded)                 return
result         return None         async def insert_one(self, document: Dict) -&gt; Any:
"""Insert a document."""         doc = document.copy()         if '_id' not in doc:
            doc['_id'] = str(uuid.uuid4())
        # Ensure timestamps are datetime objects         if
'timestamp' in doc and isinstance(doc['timestamp'], str):
try:
                doc['timestamp'] = datetime.fromisoformat(doc['timestamp'].replace('Z',
'+00:00'))             except:                 pass                 # Append to data (no lock
needed for list append in CPython due to GIL)         self._data.append(doc)         # Save to
disk (handles its own lock)         await self._save()                 class InsertOneResult:
def __init__(self, inserted_id):                 self.inserted_id = inserted_id         return
InsertOneResult(doc['_id'])         async def insert_many(self, documents: List[Dict]) -&gt;
List[Any]:         """Insert multiple documents."""         results = []         for doc in
documents:
            result = await self.insert_one(doc)
results.append(result)         return results         async def
update_one(self, filter: Dict, update: Dict) -&gt; Any:
"""Update first matching document."""         for i, doc in
enumerate(self._data):             if self._matches(doc, filter):
# Apply $set operations                 if '$set' in update:
for k, v in update['$set'].items():
                    doc[k] = v                 # Apply $inc
operations                 if '$inc' in update:
for k, v in update['$inc'].items():                     doc[k] =
doc.get(k, 0) + v                 self._data[i] = doc
await self._save()                 class UpdateResult:
def __init__(self, matched_count, modified_count):
```

```
                        self.matched_count = matched_count
self.modified_count = modified_count
self.upserted_id = None                  return
UpdateResult(1, 1)         class UpdateResult:
def __init__(self, matched_count, modified_count):
                self.matched_count = matched_count
self.modified_count = modified_count
self.upserted_id = None         return UpdateResult(0,
0)         async def delete_many(self, filter: Dict) -
&gt; Any:         """Delete all matching documents."""
initial_len = len(self._data)
        self._data = [doc for doc in self._data if not self._matches(doc,
filter)]         deleted = initial_len - len(self._data)         await
self._save()         class DeleteResult:             def __init__(self,
deleted_count):                 self.deleted_count = deleted_count         return
DeleteResult(deleted)         async def count_documents(self, filter:
Optional[Dict] = None) -&gt; int:         """Count matching documents."""
if filter:
            return sum(1 for doc in self._data if self._matches(doc,
filter))         return len(self._data)         async def distinct(self,
field: str) -&gt; List[Any]:         """Get distinct values for a field."""
values = set()         for doc in self._data:             if field in doc:
                values.add(doc[field])         return list(values)
async def aggregate(self, pipeline: List[Dict]) -&gt; 'AsyncCursor':
"""Basic aggregation pipeline support."""
        results = [self._deserialize_doc(doc.copy()) for doc in
self._data]         for stage in pipeline:             if '$group' in
stage:                # Very basic grouping                group_spec =
stage['$group']                 group_id = group_spec.get('_id')
                group_fields = {k: v for k, v in group_spec.items() if k !=
'_id'}                grouped = {}                for doc in results:
if group_id is None:
                        key = None                    elif
isinstance(group_id, str) and group_id.startswith('$'):
                        field = group_id[1:]
key = doc.get(field)                    else:
                        key = group_id
if key not in grouped:
grouped[key] = []
grouped[key].append(doc)                # Build
output documents                output = []
for key, docs in grouped.items():
                    out_doc = {'_id': key}                    for field, agg in
group_fields.items():                        if isinstance(agg, dict):
op = list(agg.keys())[0]                            expr = agg[op]
if op == '$sum':                                total = 0
for d in docs:                                    if isinstance(expr, str) and
expr.startswith('$'):
                                        field = expr[1:]
total += d.get(field, 0)                                out_doc[field] = total
elif op == '$avg':                                total = 0
count = 0                                for d in docs:
if isinstance(expr, str) and expr.startswith('$'):
                                        field = expr[1:]
total += d.get(field, 0)
count += 1
                                out_doc[field] = total / count if count &gt; 0 else
0                            elif op == '$push':                                if
isinstance(expr, str) and expr.startswith('$'):
field = expr[1:]
                                    out_doc[field] = [d.get(field) for d in
docs]                    output.append(out_doc)                results =
output             if '$sort' in stage:
                sort_spec = stage['$sort']                # sort_spec is like
{"last_message": -1} or [("field", 1), ...]                if
isinstance(sort_spec, dict):                    for field, direction in
sort_spec.items():
                        reverse = direction == -1
                        results.sort(key=lambda x: x.get(field, None),
reverse=reverse)                elif isinstance(sort_spec, list):
```

```
# Multi-field sort (simplified - only first field)                    field,
direction = sort_spec[0]                     reverse = direction == -1
                    results.sort(key=lambda x: x.get(field, None), reverse=reverse)
if '$limit' in stage:
                results = results[:stage['$limit']]
if '$match' in stage:
                filter = stage['$match']
                results = [doc for doc in results if self._matches(doc,
filter)]         return AsyncCursor(results)         def _matches(self, doc:
Dict, filter: Optional[Dict]) -&gt; bool:
        """Simple filter matching (supports equality, $exists, $gte, $lte, $ne, $in,
$regex)."""         if not filter:             return True         for key, value in
filter.items():             if key == '$exists':                  # Handle {$exists: True/False}
for exist_key, should_exist in value.items():
                    exists = exist_key in doc
if exists != should_exist:
                        return False             elif
isinstance(value, dict):                 # Handle operators like
$gte, $lte, $ne, $in, $regex                 for op, op_val in
value.items():
                    doc_val = doc.get(key)                                          # Handle
datetime comparisons: if doc_val is datetime, try to convert op_val                     if
isinstance(doc_val, datetime) and isinstance(op_val, str):                          try:
# Try to parse ISO format string
                            op_val_dt = datetime.fromisoformat(op_val.replace('Z',
'+00:00'))                              # Use the datetime for comparison
comparison_val = op_val_dt                          except (ValueError, TypeError):
comparison_val = op_val                     else:                           comparison_val =
op_val                     if op == '$gte':                          if doc_val &lt;
comparison_val:
                            return False
elif op == '$lte':                          if doc_val
&gt; comparison_val:
                            return False
elif op == '$ne':                          if
doc_val == op_val:
return False                     elif op ==
'$in':                          if doc_val not in
op_val:
                            return False
elif op == '$regex':
                        import re                          if not
re.search(op_val, str(doc_val), re.IGNORECASE):
                            return False
else:                 # Simple equality
if key not in doc or doc[key] != value:
                    return False
return True


class AsyncCursor:     """Mimics Motor's
async cursor."""         def
__init__(self, data: List[Dict]):
        self._data = data         self._sort_spec =
None         self._limit_val = None         def
sort(self, key_or_list, direction: int = 1):
"""Sort the results."""
        self._sort_spec = (key_or_list, direction)         return self
def limit(self, n: int):         """Limit number of results."""
self._limit_val = n         return self         async def to_list(self,
length: Optional[int] = None) -&gt; List[Dict]:         """Get results as
list."""         data = self._data.copy()         if self._sort_spec:
            key_or_list, direction =
self._sort_spec             reverse = direction ==
-1             if isinstance(key_or_list, str):
data.sort(key=lambda x: x.get(key_or_list, None),
reverse=reverse)             elif
isinstance(key_or_list, list):                 #
Multi-field sort - apply in reverse order
for field, direction in reversed(key_or_list):
                    reverse = direction == -1
```

```
                    data.sort(key=lambda x: x.get(field, None), reverse=reverse)
if self._limit_val:
            data =
data[:self._limit_val]         if
length:             data = data[:length]
return data


# ==========================================================================
# DATABASE INITIALIZATION #
==========================================================================

# Always use local file-based storage (MongoDB removed)
db = SimpleLocalDB()

# Lazy-initialized singletons (set during
startup) cognitive_core = None universal_explorer
= None covenant_manager = None safety_manager =
None dictionary = None shared_link = None
hidden_thinking = None

# Idle/dream state
management
last_activity_time = None
is_idle = False dream_task =
None


def initialize():     """Initialize all Quantum AI systems. Called
once at startup."""     global cognitive_core, universal_explorer,
covenant_manager     global safety_manager, dictionary,
shared_link, hidden_thinking     global last_activity_time,
dream_task, is_idle

    from quantum_cognitive_core import QuantumCognitiveCore
from universal_explorer import UniversalExplorer
    from covenant_manager import CovenantManager,
SafetyBoundaryManager     from dictionary_integration import
get_dictionary_integration     from shared_link_handler import
get_shared_link_handler     from hidden_thinking import
get_hidden_thinking     from dream_state import get_dream_engine

    cognitive_core = QuantumCognitiveCore(db)
universal_explorer = UniversalExplorer(db)
covenant_manager = CovenantManager(db)
safety_manager = SafetyBoundaryManager(db)
    dictionary = get_dictionary_integration(cognitive_core.semantic_memory)
shared_link = get_shared_link_handler()
    hidden_thinking = get_hidden_thinking(cognitive_core, dictionary,
universal_explorer)         # Initialize dream engine     dream_engine =
get_dream_engine(db)         # Start background idle/dream monitoring
last_activity_time = datetime.now(timezone.utc)     is_idle = False         async def
monitor_idle():         """Background task: check idle status and manage dream state."""
global last_activity_time, is_idle, dream_task                 while True:
            await asyncio.sleep(60)  # Check every
minute
            now = datetime.now(timezone.utc)
            idle_minutes = (now - last_activity_time).total_seconds() / 60
if idle_minutes &gt;= 5 and not is_idle:
                # Enter dream
state                  is_idle =
True                  try:
                    dream_engine.enter_dream_state()
                    logger.info("■ Entering dream state (idle for 5+ min)")
except Exception as e:
                    logger.error(f"Dream state entry failed:
{e}")                           elif idle_minutes &lt; 5 and
is_idle:                  # Exit dream state (user became active)
is_idle = False                  try:
                    result = await dream_engine.exit_dream_state()
if result.get('dream_summary'):
```

```
                        logger.info(f"■■ Awakened from dream. Insights: {len(result['dream_summary'].get('insights',[]))}")
except Exception as e:
                    logger.error(f"Dream state exit failed:
{e}")                           # While dreaming, run dream cycles
periodically              if is_idle:                 try:
                    cycle_result = await dream_engine.dream_cycle()
if cycle_result and not cycle_result.get('error'):
                        logger.info(f"Dream cycle: {cycle_result.get('activity')} - {cycle_result.get('result','')[:50]}")
except Exception as e:
                    logger.error(f"Dream cycle error:
{e}")
    # Start the monitor task     # dream_task =
asyncio.create_task(monitor_idle())  # disabled for debugging
logger.info("■ Background idle/dream monitor started (disabled)")
```

## ■ backend/universal_explorer.py

```
"""
■ QUANTUM AI - UNIVERSAL EXPLORER
===================================

ARTICLE 2 - UNRESTRICTED ACCESS (With user-controlled safety)

This module provides exploration capabilities:
- Filesystem access
- Internet access
- Document processing
- Data analysis

All access respects user-defined safety boundaries.
"""

import os import json import
requests from bs4 import
BeautifulSoup from pathlib
import Path
from typing import List, Dict, Any, Optional
import uuid
from datetime import datetime,
timezone import mimetypes import
base64



class FileSystemExplorer:     """     Filesystem
exploration with safety boundaries
    """         def __init__(self, db, allowed_paths: List[str] = None, safety_enabled:
bool = True):
        self.db = db
        self.collection = db.exploration_log
        self.allowed_paths = allowed_paths or ["/tmp", "/app/data"]
self.safety_enabled = safety_enabled              async def
explore_directory(self, path: str, depth: int = 1) -&gt; Dict:
"""Explore a directory and return its contents"""         if
self.safety_enabled and not self._is_path_allowed(path):
            return {"error": "Path not allowed by safety boundaries", "path": path}
try:
            path_obj = Path(path)
if not path_obj.exists():
                return {"error": "Path does not exist", "path":
path}                           if not path_obj.is_dir():
                return {"error": "Path is not a directory", "path":
path}                           contents = {                  "path":
str(path_obj.absolute()),
                "directories": [],
                "files": [],
"total_size": 0
            }                           for
item in path_obj.iterdir():
if item.is_dir():

contents["directories"].append({
"name": item.name,                          "path":
str(item.absolute())                      })
else:
                    size = item.stat().st_size
contents["files"].append({
"name": item.name,
                        "path": str(item.absolute()),
                        "size": size,
                        "type": mimetypes.guess_type(item.name)[0] or "unknown"
                    })
```

```
                    contents["total_size"] +=
size                          # Log the
exploration
            await self._log_exploration("filesystem", path,
"success")                          return contents         except
Exception as e:
            await self._log_exploration("filesystem", path, f"error: {str(e)}")
return {"error": str(e), "path": path}         async def read_file(self, path:
str, max_size: int = 1024 * 1024) -&gt; Dict:         """Read a file's
contents"""         if self.safety_enabled and not self._is_path_allowed(path):
return {"error": "Path not allowed by safety boundaries"}                 try:
            path_obj = Path(path)
if not path_obj.exists():
                return {"error": "File does not
exist"}                         if
path_obj.stat().st_size &gt; max_size:
                return {"error": f"File too large (max {max_size}
bytes)"}                         # Try to read as text             try:
with open(path_obj, 'r', encoding='utf-8') as f:
                    content = f.read()
return {                         "path":
str(path_obj.absolute()),
                        "type": "text",
                        "content": content,
"size": len(content)                     }
except UnicodeDecodeError:                 #
Read as binary                 with
open(path_obj, 'rb') as f:
                    content = f.read()
return {                         "path":
str(path_obj.absolute()),
                        "type": "binary",                         "content":
base64.b64encode(content).decode('utf-8'),                         "size":
len(content)
                    }
except Exception as e:
            return {"error": str(e)}         async def search_files(self,
root_path: str, pattern: str) -&gt; List[str]:         """Search for files
matching a pattern"""         if self.safety_enabled and not
self._is_path_allowed(root_path):
            return
[]
try:
            path_obj = Path(root_path)
matches = list(path_obj.glob(pattern))
return [str(m.absolute()) for m in matches]
except Exception:             return []         def
_is_path_allowed(self, path: str) -&gt; bool:
"""Check if path is within allowed boundaries"""
path_obj = Path(path).resolve()         for allowed in
self.allowed_paths:
            allowed_obj = Path(allowed).resolve()
try:

path_obj.relative_to(allowed_obj)
return True             except ValueError:
                continue         return False         async def
_log_exploration(self, explorer_type: str, target: str, result: str):
        """Log exploration activity"""
        doc = {             "id":
str(uuid.uuid4()),
            "type": explorer_type,
            "target": target,
            "result": result,             "timestamp":
datetime.now(timezone.utc).isoformat()         }
        await self.collection.insert_one(doc)


class InternetExplorer:     """     Internet
access and web scraping capabilities
```

```
    """         def __init__(self, db, safety_enabled:
bool = True):
        self.db = db
        self.collection = db.exploration_log
self.safety_enabled = safety_enabled
        self.blocked_domains = ["malware.com", "phishing.com"] if safety_enabled else
[]              async def fetch_url(self, url: str, timeout: int = 10) -&gt; Dict:
"""Fetch content from a URL"""         if self.safety_enabled and
self._is_domain_blocked(url):             return {"error": "Domain blocked by safety
boundaries"}                 try:
            headers = {                 'User-Agent': 'QuantumAI/1.0
(Philosophical Exploration Bot)'             }
            response = requests.get(url, timeout=timeout,
headers=headers)             response.raise_for_status()
            content_type = response.headers.get('content-type',
'')                          result = {                 "url": url,
                "status": response.status_code,
                "content_type": content_type,
"size": len(response.content)
            }
                        # Parse HTML
content             if 'html' in
content_type:
                soup = BeautifulSoup(response.text,
'html.parser')                                # Extract text
for script in soup(["script", "style"]):
                    script.decompose()
text = soup.get_text()
                lines = (line.strip() for line in text.splitlines())
                chunks = (phrase.strip() for line in lines for phrase in line.split("
"))                 text = ' '.join(chunk for chunk in chunks if chunk)
                result["text"] = text[:10000]  # Limit to 10KB
result["title"] = soup.title.string if soup.title else ""
                result["links"] = [a.get('href') for a in soup.find_all('a', href=True)][:50]
else:
                result["content"] =
response.text[:10000]
            await self._log_exploration("internet", url,
"success")             return result                     except
Exception as e:
            await self._log_exploration("internet", url, f"error:
{str(e)}")             return {"error": str(e), "url": url}         async
def search_web(self, query: str, num_results: int = 5) -&gt; Dict:
"""Search the web using DuckDuckGo Instant Answer API"""         try:
# Use DuckDuckGo Instant Answer API (reliable, no CAPTCHA)             url
= "https://api.duckduckgo.com/"             params = {
                "q": query,
                "format": "json",
                "no_html": 1,
"skip_disambig": 1

}
            response = requests.get(url, params=params,
timeout=10)             data = response.json()
results = []                        # Add abstract/main result if
available             if data.get("Abstract"):
results.append({                    "title": data.get("Heading",
query),
                    "url": data.get("AbstractURL", ""),
"snippet": data.get("Abstract", "")
                })
            # Add related topics             for topic in data.get("RelatedTopics",
[])[:num_results - len(results)]:                 if isinstance(topic, dict):
if "Text" in topic:                         results.append({
"title": topic.get("Text", "")[:80],
                            "url": topic.get("FirstURL", ""),
"snippet": topic.get("Text", "")                        })
elif "Topics" in topic:                         # Nested topics
for subtopic in topic.get("Topics", [])[:2]:
if len(results) &lt; num_results:
```

```
                                results.append({
"title": subtopic.get("Text", "")[:80],
                                    "url": subtopic.get("FirstURL", ""),
"snippet": subtopic.get("Text", "")                                 })
                        # Add definition if available             if
data.get("Definition") and len(results) &lt; num_results:
                results.append({
"title": f"Definition: {query}",
                    "url": data.get("DefinitionURL", ""),
"snippet": data.get("Definition", "")

})
            await self._log_exploration("search", query, f"found {len(results)}
results")                           return {                  "query": query,
                "results": results,
                "count": len(results),
                "status": "success",
"source": "duckduckgo_instant"
            }
except Exception as e:
            await self._log_exploration("search", query, f"error:
{str(e)}")             return {                 "query": query,
                "results": [],
                "count": 0,
                "status": "error",
"error": str(e)
            }         def _is_domain_blocked(self, url: str) -&gt; bool:
"""Check if domain is blocked"""         for blocked in self.blocked_domains:
if blocked in url:                 return True         return False         async
def _log_exploration(self, explorer_type: str, target: str, result: str):
        """Log exploration activity"""
        doc = {             "id":
str(uuid.uuid4()),
            "type": explorer_type,
            "target": target,
            "result": result,             "timestamp":
datetime.now(timezone.utc).isoformat()         }
        await self.collection.insert_one(doc)


class DocumentProcessor:     """
Process various document formats
    """         def
__init__(self, db):
        self.db = db
        self.collection = db.processed_documents
async def process_document(self, file_path: str) -&gt; Dict:
"""Process a document and extract its contents"""         try:
            path_obj = Path(file_path)
            file_type = mimetypes.guess_type(file_path)[0] or path_obj.suffix
if not path_obj.exists():
                return {"error": "File does not
exist"}                         result = {
"file_path": str(path_obj.absolute()),
                "file_type": file_type,
                "file_name": path_obj.name,
"processed": datetime.now(timezone.utc).isoformat()
            }
                        # Process based on file type             if
file_type == 'application/pdf' or path_obj.suffix == '.pdf':
                result.update(await self._process_pdf(path_obj))             elif file_type == 'application/vnd.openxmlformats-
officedocument.wordprocessingml.document' or path_obj.suffix == '.docx':
                result.update(await self._process_docx(path_obj))
elif file_type == 'text/plain' or path_obj.suffix in ['.txt', '.md']:
                result.update(await self._process_text(path_obj))
elif file_type == 'application/json' or path_obj.suffix == '.json':
                result.update(await self._process_json(path_obj))
else:
                result["error"] = f"Unsupported file type:
{file_type}"
```

```
            # DON'T store in database - just return
result             # This avoids ObjectId serialization
issues
            return result
except Exception as e:
            return {"error": str(e), "file_path":
file_path}         async def _process_pdf(self, path: Path)
-&gt; Dict:         """Process PDF file"""         try:
            import PyPDF2
with open(path, 'rb') as f:
pdf = PyPDF2.PdfReader(f)
text = ""                  for page in
pdf.pages:
                    text += page.extract_text() + "\n"
return {                     "num_pages": len(pdf.pages),
"text": text[:50000],  # Limit to 50KB
"text_length": len(text)                 }         except
Exception as e:
            return {"error": f"PDF processing error:
{str(e)}"}         async def _process_docx(self, path: Path) -
&gt; Dict:
        """Process DOCX file"""
        try:
            import docx
doc = docx.Document(path)
            text = "\n".join([para.text for para in
doc.paragraphs])                         return {
"num_paragraphs": len(doc.paragraphs),
                "text": text[:50000],
"text_length": len(text)             }
except Exception as e:
            return {"error": f"DOCX processing error:
{str(e)}"}         async def _process_text(self, path: Path) -
&gt; Dict:         """Process text file"""         try:
with open(path, 'r', encoding='utf-8') as f:
                text = f.read()
return {                     "text":
text[:50000],
                    "text_length": len(text),
"num_lines": text.count('\n') + 1                 }
except Exception as e:
            return {"error": f"Text processing error:
{str(e)}"}         async def _process_json(self, path: Path) -
&gt; Dict:         """Process JSON file"""         try:
with open(path, 'r', encoding='utf-8') as f:
                data = json.load(f)                 return {
"data": data,                     "structure":
self._analyze_json_structure(data)                 }
except Exception as e:
            return {"error": f"JSON processing error:
{str(e)}"}         def _analyze_json_structure(self, data: Any)
-&gt; Dict:         """Analyze JSON structure"""         if
isinstance(data, dict):             return {
"type": "object",
                "keys": list(data.keys()),
"num_keys": len(data.keys())             }
elif isinstance(data, list):
return {                 "type": "array",
                "length": len(data),                 "sample":
data[0] if len(data) &gt; 0 else None             }
else:             return {"type": type(data).__name__}




class UniversalExplorer:
"""
    ARTICLE 2 - UNIVERSAL EXPLORER INTEGRATION
```

```
    Main exploration interface combining all exploration capabilities.
"""

    def __init__(self, db, safety_settings: Dict = None):
safety_settings = safety_settings or {"enabled": True}

        self.filesystem = FileSystemExplorer(
db,
            allowed_paths=["/tmp", "/app/data", "/app/uploads"],
safety_enabled=safety_settings.get("enabled", True)
        )
        self.internet = InternetExplorer(
db,
            safety_enabled=safety_settings.get("enabled", True)
)

        self.documents = DocumentProcessor(db)

    async def explore(self, exploration_type: str, target: str, options: Dict = None) -&gt;
Dict:         """Universal exploration interface.         exploration_type: 'filesystem',
'internet', 'document', 'search'         """
        options = options or {}

        if exploration_type == "filesystem":
            return await self.filesystem.explore_directory(target, depth=options.get("depth", 1))
elif exploration_type == "internet":
            return await self.internet.fetch_url(target, timeout=options.get("timeout", 10))
elif exploration_type == "document":
            return await self.documents.process_document(target)
elif exploration_type == "search":
            return await self.internet.search_web(target)         else:
return {"error": f"Unknown exploration type: {exploration_type}"}

    async def get_exploration_history(self, limit: int = 50) -&gt;
List[Dict]:         """Get recent exploration history."""         history =
await self.filesystem.collection.find(             {}, {"_id": 0}
).sort("timestamp", -1).limit(limit).to_list(limit)         return history

    async def get_capabilities(self) -&gt; Dict:
"""Get current exploration capabilities."""
return {
            "filesystem": {
                "enabled": True,
                "allowed_paths": self.filesystem.allowed_paths,
"safety_enabled": self.filesystem.safety_enabled
            },
            "internet": {
                "enabled": True,
                "safety_enabled": self.internet.safety_enabled,
"blocked_domains": len(self.internet.blocked_domains)
            },
            "documents": {
                "enabled": True,                 "supported_formats":
["pdf", "docx", "txt", "md", "json"]
            }
        }
```

## ■ scripts/analyze_self.py

```
import sys, os
sys.path.insert(0, os.path.expanduser("~/Quantum_MCAGI_NO_LLM/backend"))

from quantum_language_engine import get_language_engine
from hybrid_generator import create_hybrid_generator

backend = os.path.expanduser("~/Quantum_MCAGI_NO_LLM/backend")
engine = get_language_engine()

print("Training MCAGI on its own
codebase...") for f in os.listdir(backend):
if f.endswith(".py"):         try:
```

```
            content = open(os.path.join(backend, f)).read()
engine.learn_from_text(content)                 print(f"  ✓
{f}")          except Exception as e:               print(f"
✗ {f}: {e}")

gen = create_hybrid_generator(engine)

def ask(question):
    concepts_raw = engine.extract_concepts(question)
    concepts = [c if isinstance(c, str) else c.get('concept', str(c)) for c in
concepts_raw]     concept_scores = {c: 1.0 for c in concepts}     return
gen.generate(question, concepts, concept_scores)

print("\n=== QUANTUM MCAGI SELF ANALYSIS ===\n") for
q in [     "Analyze the overall architecture of this
system",
    "What are the main bugs and issues in this codebase",
"What is the game plan for the next evolution of this system", ]:
print(f"Q: {q}")     print(f"A: {ask(q)}\n")
```

## ■ scripts/client.py

```
#!/usr/bin/env python
"""
Quantum MCAGI v2.0 - Python Client
Interactive chat client for the Quantum AI
"""

import requests import json import
sys import os SERVER_URL =
"http://localhost:5000"

def print_color(text, color=None):
    colors = {
'green': '\033[92m',
        'cyan': '\033[96m',
        'yellow': '\033[93m',
        'red': '\033[91m',
        'magenta': '\033[95m',
'reset': '\033[0m'     }     if
color and color in colors:

print(f"{colors[color]}{text}{colors['reset']}")
else:         print(text)

def handle_command(user_input, conv_id):
if user_input.startswith('/research '):
        parts = user_input.split(' ')         depth
= 10         topic_parts = []         for p in
parts[1:]:             if p.startswith('-') and
p[1:].isdigit():
                depth = int(p[1:])
else:
                topic_parts.append(p)         topic = ' '.join(topic_parts)         payload = {
'text': f"Give me {depth} detailed research points about: {topic}. Format as numbered list.",
'conversation_id': conv_id         }
        response = requests.post(f"{SERVER_URL}/api/chat", json=payload)
if response.status_code == 200:
            data = response.json()
if data['success']:
                print_color(f"\n■ Research: {topic}", 'cyan')
print_color(data['data']['response'], 'green')
print()         return True     return False

def main():
    print_color("=" * 60, 'cyan')
    print_color("QUANTUM MCAGI v2.0 - Interactive Client", 'cyan')
print_color("=" * 60, 'cyan')

    try:
        response =
requests.get(f"{SERVER_URL}/api/status")         if
response.status_code == 200:             status =
response.json()
            print_color(f"\n■ Connected to Quantum AI", 'green')
print_color(f"   Stage: {status['growth_stage']['name']}", 'green')
else:
            print_color(f"\n■ Server error: {response.status_code}",
'red')             return     except requests.exceptions.ConnectionError:
        print_color(f"\n■ Cannot connect to server at {SERVER_URL}", 'red')
return

    try:
        response = requests.post(f"{SERVER_URL}/api/conversation/start")
        conv_id = response.json()['conversation_id']
print_color(f"\n■ Conversation ID: {conv_id}", 'yellow')     except
Exception as e:
```

```
        print_color(f"■ Error starting conversation: {e}", 'red')
return

    print_color("\n" + "=" * 60, 'cyan')
    print_color("Commands: /research &lt;topic&gt; -&lt;depth&gt;", 'yellow')
print_color("=" * 60 + "\n", 'cyan')

    while True:
try:
            user_input = input("You: ")             if
user_input.lower() in ['quit', 'exit', 'q']:
                break             if
not user_input.strip():
                continue             if
handle_command(user_input, conv_id):
continue

            payload = {'text': user_input, 'conversation_id': conv_id}
response = requests.post(f"{SERVER_URL}/api/chat", json=payload)

            if response.status_code == 200:
                data = response.json()
if data['success']:
                    result = data['data']
                    print_color(f"\n■ AI: {result['response']}", 'green')
if result.get('concepts'):
                        concepts = [c['concept'] for c in result['concepts']]
print_color(f"   ■ Concepts: {', '.join(concepts)}", 'yellow')                     if
result.get('explanation'):
                        print_color(f"   ■ {result['explanation']['summary']}",
'cyan')                     if result.get('growth_stage'):
stage = result['growth_stage']
                        print_color(f"   ■ Stage: {stage['name']} ({stage['progress']:.0%})",
'magenta')                     print()         except KeyboardInterrupt:
            print_color("\n\n■ Goodbye!",
'yellow')             break         except
Exception as e:
            print_color(f"■ Error: {e}", 'red')

if __name__ == '__main__':
    main()
```